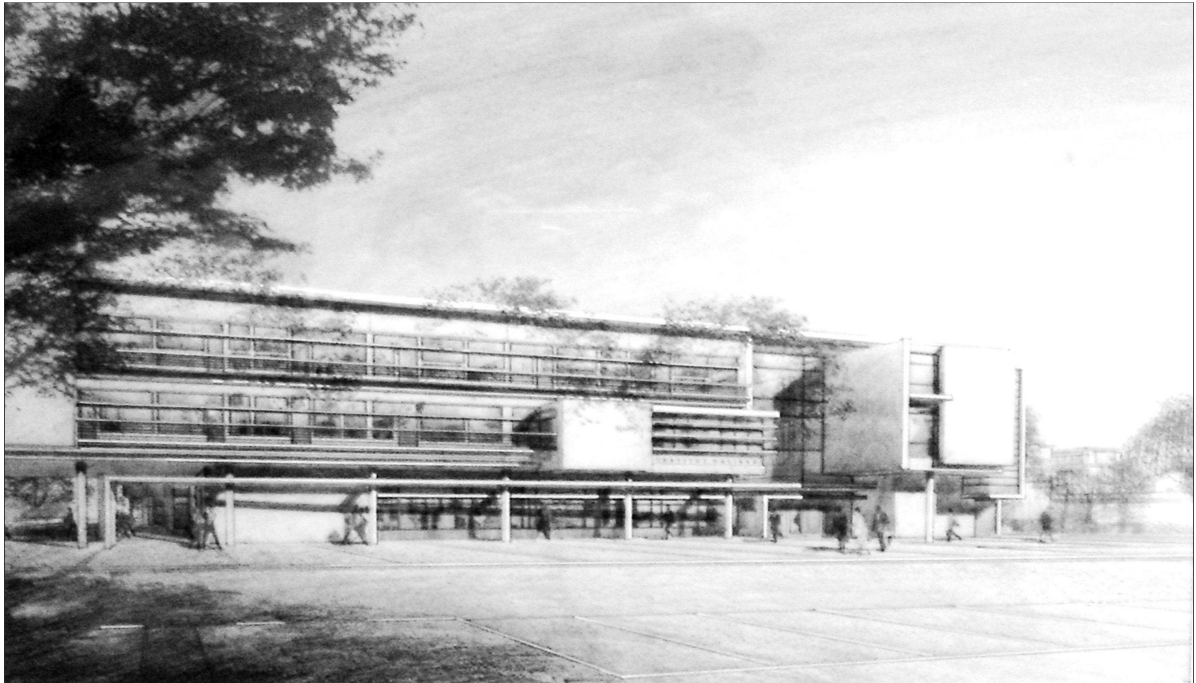




ASPECTS MATERIELS & LOGICIELS DU MICROCONTRÔLEUR PIC 16F877

SOMMAIRE GENERAL

I. GENERALITES ET NOTIONS DE BASE	005
II. ARCHITECTURE DU PIC 16F877	057
III. CPU, MEMOIRES, PILE ET CONFIGURATION	065
IV. LE JEU D'INSTRUCTIONS	101
V. PETITS EXERCICES D'INITIATION CORRIGES	165
VI. EXEMPLES EN ASSEMBLEUR ET EN C	189
VII. METHODOLOGIE DE TRAVAIL EN TP	213
VIII. PERIPHERIQUES ET INTERFACES DU PIC 16F877	217

THIEBAUT Bernard

A V E R T I S S E M E N T (à lire avant toute chose !!!)

Le présent document polycopié ne doit pas être considéré comme une présentation intégrale du cours de Monsieur THIEBAUT.

Il ne dispense en aucune façon de la présence aux cours. Il est destiné à fournir aux étudiants intéressés un support essayant de traiter les points essentiels et fondamentaux.

Comme dans toute discipline " fortement " technique, il est vivement conseillé aux étudiants de compléter sa lecture « dans un second temps » par l'utilisation d'ouvrages plus importants et plus détaillés, et notamment des ouvrages constructeurs en anglais : c'est une excellente façon de s'immerger dans cette langue si présente et si importante dans le domaine des sciences et des techniques.

Consultez aussi la liste d'ouvrages de la page suivante et allez jeter un œil à la bibliothèque universitaire ...

N'hésitez pas non plus à consulter les sites Web également sur la page suivante : vous y trouverez beaucoup d'informations sur les PIC.

Il faut se rappeler que dans le domaine de l'électronique numérique :

- *la documentation d'une porte logique ou d'une bascule tient à peine sur une page,*
- *celle d'un compteur, d'un décodeur ... ou de circuits de ce type prendra peut-être 2 ou 3 pages avec une table de vérité et quelques chronogrammes de fonctionnement,*
- *mais dès que l'on passe à un microcontrôleur, entre les aspects purement matériels et ceux purement logiciels, plus les particularités de l'outil de développement à mettre en œuvre pour réaliser une application, il n'est pas rare que l'ensemble de cette documentation comporte plusieurs centaines de pages, souvent réparties sur plusieurs ouvrages, aides en ligne ou sur CDROM. De plus cette documentation, en anglais dans la plupart des cas (rassurez vous, ce n'est pas du Shakespeare) n'est pas à la base prévue pour un débutant mais pour un lecteur ayant déjà une certaine expérience. De quoi dérouter plus d'un étudiant, malgré toute la curiosité ou les efforts dont il peut faire preuve ! Et pour des circuits encore plus performants comme les microprocesseurs les plus récents ou les DSP (Digital Signal Processor) cela peut même atteindre plusieurs milliers de pages.*

Il ne faut donc pas considérer, qu'à terme, ce polycopié puisse se substituer à la documentation constructeur MICROCHIP sur le PIC 16F877. Il faut plutôt le considérer comme un support permettant à un étudiant débutant d'acquérir une bonne compréhension des principes de base mis en œuvre, un effort tout particulier ayant été fait pour regrouper sur un seul document la "substantifique moelle" en essayant d'aller à l'essentiel tout en favorisant le développement des capacités d'autonomie de l'étudiant intéressé par ce domaine.

Chaque fois que cela sera possible, on indiquera les pages du datasheet du PIC 16F877 en rapport avec la partie introduite et/ou traitée dans ce polycopié, histoire justement de faire un lien avec la documentation constructeur en anglais.

<i>Ce polycopié doit impérativement être complété, dans un deuxième temps, par la lecture de celui concernant l'outil de développement avec lequel les étudiants vont travailler en séances de Travaux Pratiques et/ou en séances de Projet.</i>

QUELQUES OUVRAGES SUR LE SUJET

DUNOD :

Microcontrôleurs PIC 10,12,16
Description et mise en œuvre
Christian TAVERNIER

DUNOD :

Microcontrôleurs PIC 18
Description et mise en œuvre
Christian TAVERNIER

DUNOD :

Programmation en C des PIC
Christian TAVERNIER

ETSF :

S'initier à la programmation des PIC
Alain REBOUX

ETSF :

Progresser avec les Microcontrôleurs PIC
G. SAMBLANCAT

VOLPE :

Pratique des Microcontrôleurs PIC
Francesco & Sabinaz VOLPE

CDROM

MPLAB IDE

(Data Sheet, Documentations, Product Line Info, Application Notes, Tutorials, Development Tools Guide, Demo and Evaluation Kit Software and Documentation ...)

DATASHEET du PIC 16F877 : présent sur les PC des labos.

QUELQUES SITES WEB

Constructeur + outil de développement MPLAB: www.microchip.com

Compilateur C : version de démo
www.htsoft.com/downloads/

Quelques passionnés des PIC :

<http://www.abcelectronique.com/bigonoff/>

<http://angeliz.free.fr/>

<http://academic.csuohio.edu/simond/courses/eec417/ESPWithThePIC16F877.pdf>

I. GENERALITES ET NOTIONS DE BASE

I.1. MICROPROCESSEUR ET MICROCONTRÔLEUR

C'est un fait : les microprocesseurs ont envahi notre vie de tous les jours...

Différents constructeurs (**MOTOROLA, INTEL, PHILIPS, ZILOG, AMTEL...**) se sont partagés le marché et ont développé, au fur et à mesure des évolutions technologiques et des applications les mettant en œuvre, des familles de microprocesseurs comme par exemple (et pour ne citer que les plus emblématiques) :

- **Intel** avec la lignée : 8088 (8bits), 8086 (16bits), 80186/286/386/486 (16/32bits), Pentium I,II,III...(64bits) ...

- **Motorola** avec la lignée : 6809 (8bits), 68000/010/020/030/040 (16/32 bits), Power PC 6xx/Gx (64bits) ...

Comme on le voit sur ces deux exemples, les versions 64 bits ont maintenant remplacé les 16/32 bits déjà anciens des années 80/90 et les 8 bits des années 70.

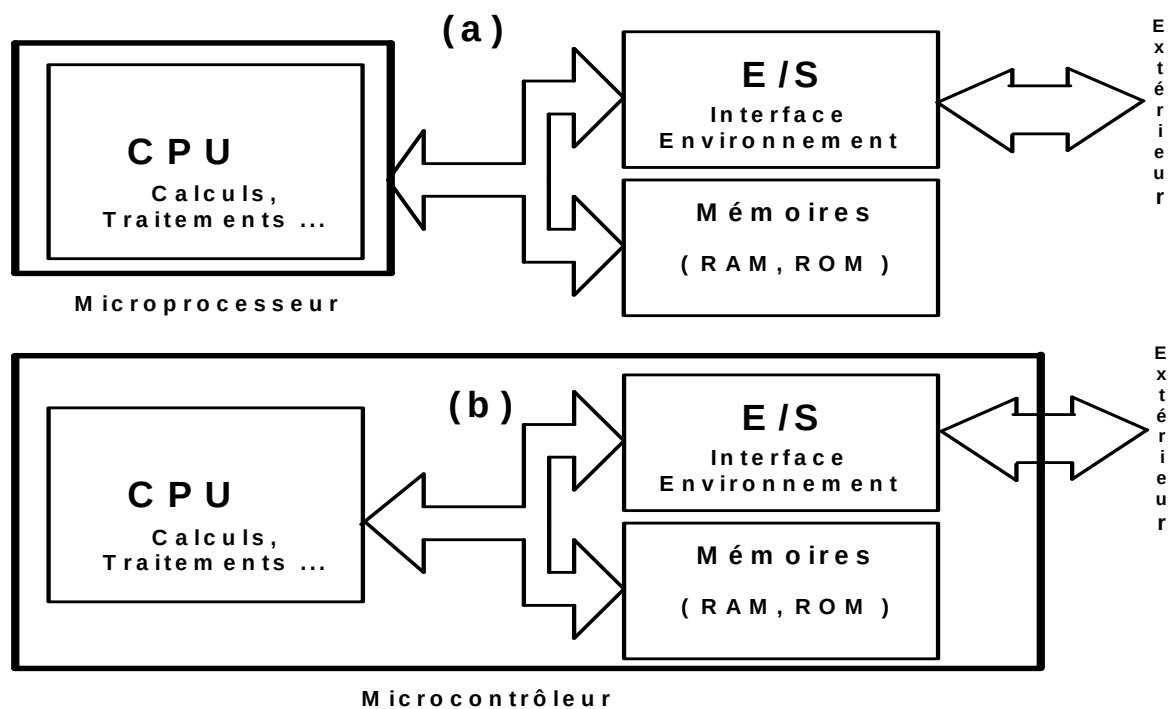
Ainsi les tailles des registres internes et des différents bus n'ont jamais cessé d'augmenter pour atteindre des performances toujours plus importantes offrant des capacités de calcul sans cesse croissantes.

Parallèlement, les horloges de fonctionnement, initialement de l'ordre du MHz ont atteint des fréquences avoisinant la dizaine de GHz, et certainement plus dans les années à venir.

Cependant, certaines applications (encore fréquentes) n'ont pas à gérer des environnements complexes et n'ont donc pas besoin de toute la puissance de calcul ou des tailles de mémoire gigantesques propres aux microprocesseurs les plus performants et aux systèmes d'exploitation si « gourmands » en mémoire.

C'est dans cet esprit qu'ont été développés les microcontrôleurs. Des solutions les utilisant ont pu voir le jour pour des coûts et des durées de développement défiant toute concurrence. On retrouvera d'ailleurs souvent des versions à base d'architectures plus simples sur 8 bits ou 16 bits, mais rarement au-delà (en tout cas pour le moment), car les applications concernées ne le nécessitent pas.

Pour diminuer la taille des systèmes réalisés, la plupart des fonctions périphériques notées en abrégé E/S qui en principe sont délocalisées **hors** du microprocesseur sur les systèmes architecturés autour d'un microprocesseur (cf. figure I.1.a.), se retrouvent **intégrées dans le boîtier** même du microcontrôleur (cf. figure I.1.b.) donnant ainsi un ensemble **compact, complet, souple** et relativement **facile à utiliser**.



CPU : Central Process Unit :là où sont faits les divers traitements
C'est le « cœur » des micro/processeurs/contrôleurs

Figure I.1.

Pour les organes d'entrées/sorties, on trouvera notamment et principalement (sans que cette liste ne soit exhaustive) :

- des entrées/sorties parallèles pour la gestion de capteurs ou d'actionneurs logiques,
- des entrées/sorties série synchrones ou asynchrones pour les communications avec l'extérieur,
- des timers (ou compteurs) pour la gestion du temps (ou des événements extérieurs),
- des convertisseurs analogique numérique et/ou numérique/analogique pour l'interfaçage avec le monde physique (capteurs ou actionneurs analogiques),
- des contrôleurs d'interruptions (gestion des interruptions, différenciation, autorisation et interdiction par masquage, niveaux de priorité...),
- d'autres modules plus ou moins complexes et évoluant au gré des normes technologiques.

Ces différentes fonctions se rajoutent au cœur du microcontrôleur constitué d'une unité centrale (CPU) des plus classiques et aux différents types de mémoire (RAM, ROM, EPROM ou EEPROM) qui se trouvent également dans le même boîtier.

On obtient ainsi un composant alliant **simplicité de mise en œuvre et d'utilisation** (là où le développement à partir d'un microprocesseur nécessiterait d'utiliser un nombre de composants bien plus important) et ceci pour un **coût relativement avantageux**.

En fait le vrai prix à payer est une **diminution des performances intrinsèques** par rapport à ce que l'on peut trouver avec les microprocesseurs les plus récents.

Mais pour des applications telles que :

- *les petits automatismes industriels,*
- *l'électronique embarquée : en automobile (pour l'ABS, l'injection électronique, les tableaux de bord, les systèmes anti-vols, les GPS...), dans les terminaux de paiement, la petite robotique ...,*
- *la domotique,*
- *les téléphones portables (et autres applications portables type MP3),*
- *les appareils photos numériques,*
- *l'électroménager (téléviseurs, télécommandes, machines à laver, microondes, cafetières...),*
- *les jouets (véhicules télécommandés, système légo Mindstorms...),*
- *les applications spécifiques et/ou délocalisées (distributeurs divers...),*
- *la sécurité, la signalisation et la protection,*
- *et bien d'autres applications ...*

L'utilisation des microcontrôleurs s'est peu à peu imposée et a trouvé sa place dans le secteur du développement et de la production de cartes électroniques.

Le microcontrôleur s'avère également être un excellent **outil pédagogique** qui permet de comprendre, sur une architecture relativement simple, les grands principes mis en œuvre sur des cas plus complexes, en mettant notamment en évidence **le lien** entre les aspects purement **matériels (hardware)** et ceux plutôt **logiciels (software)**, les deux piliers de base de la grande majorité des systèmes programmables.

En fait, pour tenter de répondre à la question :

« pourquoi et comment ça marche ? »

Le microcontrôleur que nous avons choisi appartient à la famille **PIC** de chez **MICROCHIP**.

C'est une famille de microcontrôleurs assez répandue dans l'industrie mais aussi chez les étudiants ou les particuliers passionnés qui s'intéressent comme un loisir à l'informatique industrielle.

Globalement, ce polycopié est divisé en **cinq grandes parties**.

① **chapitre I** : les généralités, les notions de base nécessaires et les grands principes de fonctionnement des systèmes architecturés autour des micro/processeurs/contrôleurs.

② **chapitres II, III et IV** : encore des généralités (plus pointues) mais aussi les spécificités du microcontrôleur **PIC 16F877** qui sera utilisé en TP (éléments sur son architecture interne, sur son fonctionnement et sur son jeu d'instructions).

③ **chapitres V et VI** : des exercices corrigés et des exemples de programmes de base en assembleur et en C.

④ **chapitre VII** : la méthodologie de travail en TP.

⑤ **chapitre VIII** : les ressources du **PIC 16F877** en tant qu'interface avec son environnement.

Ces deux derniers chapitres seront bien évidemment plus détaillés dans le cadre des TP où ces interfaces seront mises en œuvre.

**Mais, si vous comprenez bien toutes les notions de base présentées ici,
vous n'aurez aucun mal à les transposer
sur tous les PIC commercialisés par MICROCHIP.
Mais aussi, plus généralement, sur tous les types de
micro/processeurs/contrôleurs.**

I.2. RAPPEL ET INTRODUCTION DE QUELQUES NOTIONS DE BASE

Les notions de base qu'il faut connaître pour se plonger dans les grands principes de fonctionnement, sont en fait assez simples et relèvent :

- de la **logique combinatoire** : algèbre de Boole, fonctions (ou portes, **gates**) logiques de base (**AND, OR, NOT, NAND, NOR, XOR**),

Entrées		INVERSION		ET (AND)	OU Inclusif (IOR)	OU eXclusif (XOR)
A	B	$\bar{A}$	$\bar{B}$	$A \cdot B$	$A + B$	$A \oplus B$
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	1	0

- mais surtout de la **logique séquentielle**, dont l'élément de base, la **bascule (flip-flop)** permet de réaliser la **mémorisation** d'un niveau logique élémentaire, le quantum d'information le plus petit, appelé aussi le **bit** :

- soit le niveau **0** (ou niveau **bas, low, L**) qui correspond souvent à 0V, (en logique dite **positive**. En logique négative c'est l'inverse)
- soit le niveau **1** (ou niveau **haut, high, H**) qui correspond à 5V (voire 3,3V), (en logique dite **positive**. En logique négative c'est l'inverse)

Autant dire que ce cours démarre bien de la base ... à vous de vous investir !!!

I.2.1. REGISTRE ET CODAGE HEXADECIMAL

Cette notion de bit, nous amène tout naturellement à celle de **REGISTRE (REGISTER** en anglais), car en pratique, ces bascules élémentaires sont organisées en registres.

Registres qui seront **intensément** utilisés par la suite.

(registres **à l'intérieur** de la CPU du micro/contrôleur/processeur, emplacements mémoire qui peuvent être considérés formellement comme des registres **externes** à la CPU du micro/contrôleur/processeur, registres dans les modules périphériques programmables...).

Qu'est-ce qu'un registre ?

C'est un ensemble de **n** bascules (la plupart du temps $n = 2^k$) regroupées dans une entité qui portera fréquemment un nom (le nom du registre) ou un numéro (pour une case mémoire) et qui sera représentée le plus souvent par un schéma comme celui de la figure I.2.

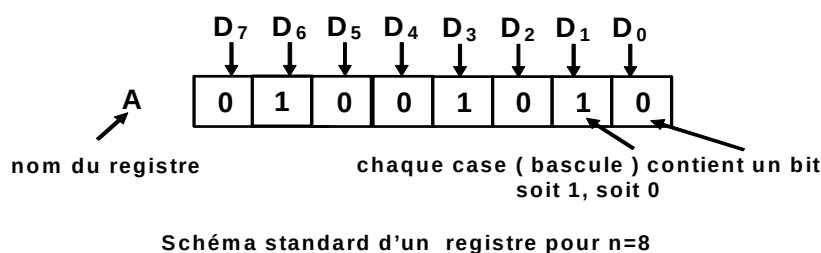


Figure I.2.

L'entier **n** représente la **taille** du registre et vaudra :

- **n = 4** : on parle alors de **quartet** (ou **nibble** en anglais), cas assez rare,
- **n = 8** : on parle d'octet (ou **byte**), cas très fréquent comme sur le schéma de la figure I.2.,
- **n = 16** : soit un double octet appelé **mot** (ou **word**), assez fréquent également,
- **n = 32** : soit un quadruple octet appelé **double mot** (ou **double** plus succinctement),
- voire **une puissance de 2** plus grande sur des architectures plus récentes ou futures.

Comme l'a montré la figure I.2. précédente, chaque bit d'un registre est repéré par la lettre **D_i**, avec $0 \leq i \leq n-1$. ☛ ☛ : attention, les indices commencent à 0 et pas à 1 !!!!

D₀ (le plus à droite) est appelé le **LESS SIGNIFICANT BIT (LSB** ou **BIT DE POIDS FAIBLE)**

D_{n-1} (le plus à gauche) est appelé le **MOST SIGNIFICANT BIT (MSB** ou **BIT DE POIDS FORT)** :

ainsi, le MSB sera **D₇** (pour un octet) et **D₁₅** (pour un mot), **D₃₁** (pour un double mot)...

Cette séquence **D_{n-1} D_{n-2} ... D₁ D₀** , formée d'une suite n bits, chacun valant soit 0 soit 1, représente en fait le **CODAGE BINAIRE** (en **BASE 2**) de l'information contenue dans le registre. C'est de cette façon qu'elle est stockée à l'intérieur du registre.

Remarque :

*En fait, le codage binaire est le **langage naturel** des micro/processeurs/contrôleurs, le seul qu'ils comprennent au final et au niveau le plus « profond » de leur architecture. On parle aussi de **langage binaire** (ou dans certains cas de **langage machine**, notion qui sera détaillée dans la suite de ce polycopié).*

Par exemple, lorsque vous programmez en langage évolué, comme le C/C++, c'est le couple compilateur/linker qui traduira votre fichier texte source en langage machine compréhensible par le micro/processeur/contrôleur.

Si on veut représenter le contenu du registre en **DECIMAL** (c'est-à-dire en **BASE 10**), il suffit de calculer :

$$D_{n-1} \cdot 2^{n-1} + D_{n-2} \cdot 2^{n-2} + \dots + D_2 \cdot 2^2 + D_1 \cdot 2^1 + D_0 \cdot 2^0$$

On voit donc que l'indice **i** de chaque **D_i** représente la puissance de 2 correspondante. D'où le qualificatif de **poids** associé à chacun des bits **D_i** .

En pratique le codage décimal, très pratique dans la vie de tous les jours car nous possédons 10 doigts, n'est pas adapté dans ce cas et **sera donc très peu** utilisé.

Mais pour éviter les longues séquences de 1 et de 0 du codage binaire, pas très lisibles et sources d'erreurs fréquentes, on préférera utiliser le **CODAGE HEXADÉCIMAL** (c'est-à-dire la **BASE 16** et donc avec 16 « chiffres » différents : de **0** à **9** pour les dix premiers complétés par les 6 chiffres suivants notés **A, B, C, D, E** et **F**), mieux adapté ici que le système décimal de base 10, car plus proche de l'architecture hardware des composants en électronique numérique programmable.

Règle simple à connaître par cœur :

- pour passer du binaire à l'hexadécimal, on regroupe les bits **par paquet de 4**, à partir du **LSB** (et donc de la droite) et on convertit chaque paquet obtenu en utilisant la correspondance entre les deux dernières colonnes (**binaire** et **hexadécimal**) fournies dans le tableau de la figure I.3. suivante.

La première colonne (codage **décimal**) a été rajoutée pour la lisibilité et la compréhension de ce tableau mais ne sert à rien ici pour appliquer cette règle...

- pour passer de l'hexadécimal au binaire, c'est l'opération inverse...

Codage décimal	Codage binaire	Codage hexadécimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Figure I.3.

Exemple : (cf. figure I.4.)

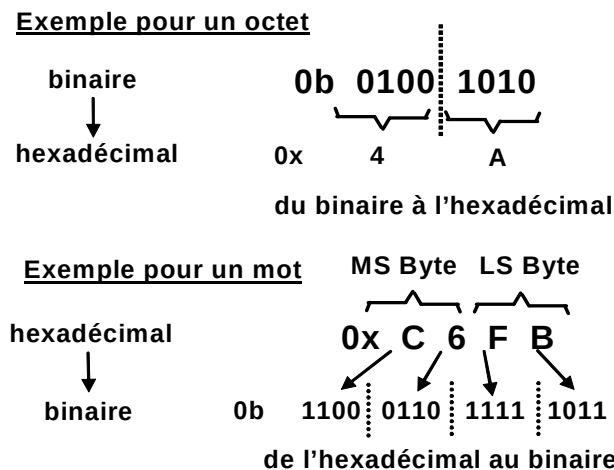


Figure I.4.

Je vois trop souvent des étudiants qui ne savent pas appliquer cette règle, pourtant simple, et sont du coup obligés de passer par (l'inutile) étape intermédiaire du décimal et de faire de longues suites de divisions successives par 2 pour aboutir finalement au binaire : perte de temps, manque d'efficacité et risque d'erreurs fréquentes ...

Passer **donc un peu de temps** à faire quelques conversions (comme représentées sur l'exemple de la figure I.4.) pour vous familiariser avec cette méthode.

**Il est important de bien maîtriser ce codage, proche du hardware et de l'électronique et de passer facilement du binaire à l'hexadécimal et réciproquement.
Vous vous en rendrez rapidement compte en TP !!!**

Remarques et convention :

Sur la figure I.4. précédente :

- le symbole préfixe **0b** devant les nombres 0100 1010 ou 1100 0110 1111 1011 indique qu'il s'agit d'un codage binaire. Parfois, on peut aussi rencontrer le préfixe **%** en lieu et place de **0b**.
- le symbole préfixe **0x** devant les nombres 4A ou C6FB indique qu'il s'agit d'un codage hexadécimal pour éviter la confusion avec le décimal (si les digits sont des chiffres et pas des lettres). Parfois, une autre convention consiste à rajouter le suffixe **h** ou **H** (comme **h(H)**exadécimal) après le nombre pour indiquer qu'il est exprimé en hexadécimal (exemple : 4Ah ou C6FBh). Et parfois aussi, c'est le préfixe **\$** (surtout chez Motorola) qui remplace le **0x** du C (exemple : \$4A ou \$C6FB).
- l'**absence de symbole** signifie que la valeur doit s'interpréter en décimal.

Ainsi, par exemple :

10 (sans symbole) = (10)₁₀ (décimal) = **0x0A** = **0Ah** = **\$0A** = **0b1010** = **%1010**.

Mais ≠ **0x10** (= (16)₁₀) et ≠ **0b10** (= (2)₁₀).

Attention de ne pas confondre toutes ces conventions !!!!

Pour repérer les bits d'un octet, on peut utiliser l'**indice i** du **bit D_i**.

Par exemple **D₀** est le **bit 0**, **D₁** est le **bit 1**,, **D₇** est le **bit 7**.

Remarque :

*En pratique, on peut aussi utiliser l'**hexadécimal pour nommer** les différents bits d'un octet:*

***D₀** s'appelle le bit **0x01** car correspond au masque **0b 0000 0001** : seul **D₀** à 1, les autres à 0.*

***D₁** s'appelle le bit **0x02** car correspond au masque **0b 0000 0010** : seul **D₁** à 1, les autres à 0.*

***D₂** s'appelle le bit **0x04** car correspond au masque **0b 0000 0100** : seul **D₂** à 1, les autres à 0.*

***D₃** s'appelle le bit **0x08** car correspond au masque **0b 0000 1000** : seul **D₃** à 1, les autres à 0.*

***D₄** s'appelle le bit **0x10** car correspond au masque **0b 0001 0000** : seul **D₄** à 1, les autres à 0.*

***D₅** s'appelle le bit **0x20** car correspond au masque **0b 0010 0000** : seul **D₅** à 1, les autres à 0.*

***D₆** s'appelle le bit **0x40** car correspond au masque **0b 0100 0000** : seul **D₆** à 1, les autres à 0.*

***D₇** s'appelle le bit **0x80** car correspond au masque **0b 1000 0000** : seul **D₇** à 1, les autres à 0.*

Ce repérage d'un bit avec des masques est à mon avis plus parlant et plus évocateur car il fait le lien avec l'hexadécimal et permet, grâce au masque, de repérer le bit à 1 positionné dans l'octet.

*Il sera utilisé préférentiellement pour définir **des masques**.*

La plupart des registres possèdent deux propriétés essentielles :

- on peut les **lire** (on parle alors de cycle de **lecture**) pour accéder à leur contenu (sans le modifier),
- on peut y **écrire** (on parle alors de cycle d'**écriture**) pour modifier leur contenu. Notons que dans ce cas la suite de 0 et 1 précédemment présente est remplacée (écrasée et donc perdue) par la nouvelle séquence qui vient d'être écrite.

Il existe deux grands procédés pour ces phases de lecture et d'écriture :

① le mode parallèle :

Dans ce mode, tous les bits D_i sont lus (ou écrits) **tous en même temps**, en un seul coup d'horloge, comme indiqué sur la figure I.5. :

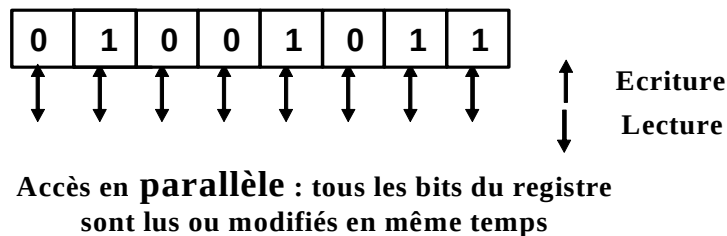


Figure I.5.

Ce mode d'accès, nécessitant au moins 8 fils (pour un octet) plus un fil pour la masse est le plus rapide mais sera en général réservé lorsque les composants sont positionnés sur la même carte électronique où le nombre de fils (en fait des pistes de cuivre sur le circuit imprimé) n'est pas un problème en soi.

② le mode série :

Les bits D_i sont lus (ou écrits) **les uns après les autres**, ce qui est donc nécessairement plus lent, car nécessite 8 coups d'horloge, toujours pour un octet.(cf. figure I.6.).

On parle alors de fonctionnement type **registre « à décalage »** (shift register).

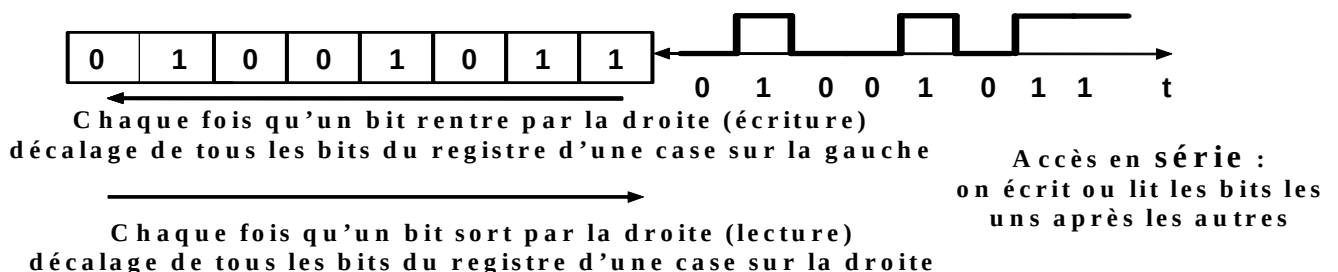


Figure I.6.

Par contre, l'accès série peut se faire avec un minimum de deux fils : un fil pour l'entrée/sortie des bits plus un fil pour la masse.

En conclusion, l'accès en parallèle, **plus rapide**, est donc celui qui sera utilisé le plus fréquemment, surtout au niveau de l'architecture électronique et des échanges à **l'intérieur** des systèmes.

Les échanges entre systèmes se feront par contre souvent en mode série pour diminuer le nombre de câbles ou de fils. Ce sera le cas par exemple pour les réseaux informatiques. Mais aussi dans les interfaces comme l'**USB (Universal Serial Bus)** qui servent pour les communications avec **l'extérieur** et qui sont bien en mode série.

Notons enfin que certains registres ont aussi la propriété supplémentaire de pouvoir être utilisés en mode **compteur**, c'est-dire qu'il existe la possibilité d'augmenter de 1 leur contenu (**incréméntation** ou **count up**) ou de diminuer de 1 leur contenu (**décréméntation** ou **count down**).

Les registres ayant été définis, il est intéressant maintenant de faire le lien entre ces registres de base et **la mémoire vive** que l'on trouve dans tous les systèmes architecturés autour d'un micro/processeur/contrôleur.

I.2.2. MEMOIRE ET DIFFERENTS TYPES DE MEMOIRE VIVE

Dans tous les systèmes à base de microprocesseur ou de microcontrôleur, il existe de la mémoire. (cf. schéma de la figure I.1.)

Celle que l'on va présenter ici fait partie de la famille des **mémoires vives** réalisées sous la forme d'un circuit intégré (en « mono chip » ou implantées sur des barrettes à connecter), par opposition avec la mémoire dite **de masse** correspondant aux disques, CD, DVD et autres supports auxiliaires...

Rappelons que dans les microcontrôleurs, cette mémoire se trouvera à l'intérieur même du composant.

De façon très générale, et ceci sera détaillé par la suite, la mémoire sert à stocker :

- les programmes : on parle aussi du **CODE** et donc de mémoire de **code** ou de mémoire de **programme**,
- et les données (ou **DATAS**) : on parle alors de mémoire de **données** de ce programme.

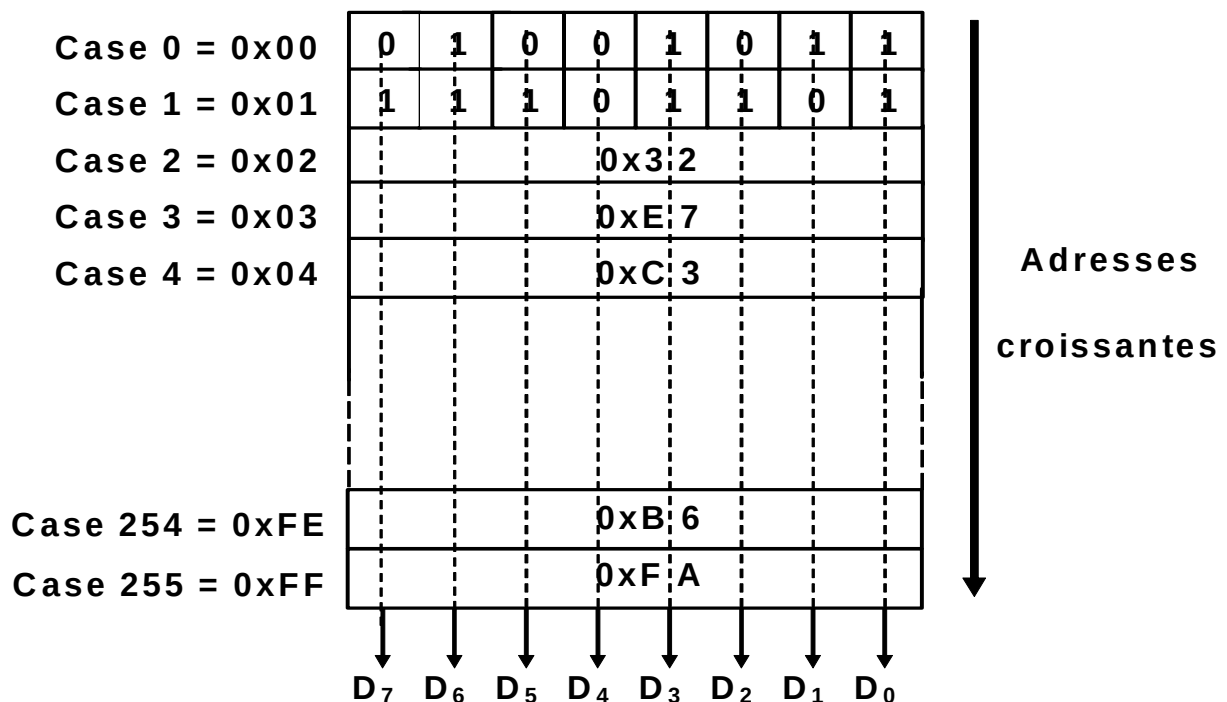
Il faut se représenter mentalement une mémoire **vive** comme une gigantesque " armoire " munie de tiroirs contigus (il peut y en avoir jusqu' à plusieurs millions), chaque tiroir étant vu comme un registre c'est-à-dire lui-même divisé en **n** compartiments élémentaires, chaque compartiment pouvant, comme on l'a vu plus haut stocker un **1** ou un **0**.

Car pour représenter cette image de la mémoire « armoire », on rencontrera souvent dans la littérature technique, le schéma assez **intuitif et parlant** fourni figure I.7.

Avec cette image en tête :

- l'armoire complète représente le circuit intégré mémoire vive,
- chaque tiroir est un **EMPLACEMENT MEMOIRE** (on dit aussi **CASE MEMOIRE**),
- chaque compartiment contient un bit (**1** ou **0**).

Pour pouvoir être différenciée de ses voisines, chaque case mémoire est caractérisée par un numéro (c'est en quelque sorte le nom « numérique » de la case) qui lui est propre et que l'on appelle son **ADRESSE** (un peu comme chaque individu possède sa propre adresse postale ou son adresse mail). Cette adresse est repérée sur la gauche de la figure I.7. à côté de chaque case. On met en général en haut de la représentation les adresses basses (les plus faibles) et en bas les adresses hautes (les plus élevées) comme l'indique la flèche « adresses croissantes » sur la droite du schéma de la figure I.7.



**Représentation “classique” d’une zone
de mémoire vive.
Ici d’une taille (très modeste) de 256 octets.**

Figure I.7.

Trois remarques **importantes** sur la correspondance adresse/case mémoire.

- ① dans un même système, deux cases physiquement **différentes** (et donc contenant des informations différentes) ne pourront pas avoir la même adresse (ce qui entraînerait un conflit de fonctionnement, comme si deux personnes différentes avaient la même adresse mail : problème qu’on détaillera plus loin et qu’on verra comment éviter)
- ② par contre, une même case mémoire peut avoir plusieurs adresses, faisant toutes référence à cette même case mémoire physique. Une parmi celles-ci pourra être utilisée de façon courante, les autres sont alors appelées **ADRESSES FANTÔMES** (ou **GHOST**), un peu comme quand vous possédez plusieurs adresses mail. On dit alors qu’on a réalisé un décodage incomplet. Cette propriété sera d’ailleurs utilisée pour accéder à certains registres du PIC.
- ③ enfin, certaines adresses peuvent ne correspondre à aucune case mémoire physique réelle, aucune case mémoire n’étant « implantée » à cette (ou ces) adresses. (comme pour une adresse mail choisie « au hasard » ne correspondant à aucune personne physique : à ne pas utiliser). Ce sera le cas par exemple, lorsque la quantité de mémoire présente et installée dans un système est plus petite que la capacité totale maximale adressable par le micro/processeur/contrôleur.

Cette représentation de la mémoire de la figure I.7. appelle également quelques précisions supplémentaires.

Comme on l'a dit, chaque case mémoire a bien été repérée par son numéro (son **adresse**), inscrit sur la gauche de la figure. On voit par exemple que la case **d'adresse 0** contient la valeur binaire constituée ici d'une séquence de 8 bits dont la valeur vaut **Ob 0 1 0 0 1 0 1 1**, valeur choisie ici totalement au hasard. Toujours sur cette figure, on a supposé que chaque case mémoire avait pour taille un octet (cas assez fréquent).

Le contenu des cases 2, 3, 4 ... a été mis en hexadécimal sur la figure précédente ainsi que les adresses des différentes cases mémoire ... car en pratique, les adresses des différentes cases mémoire, ainsi que leur contenu sont le plus souvent exprimés en hexadécimal.

On commence à sentir déjà l'importance et l'utilisation fréquente du codage hexadécimal.

Par contre :

**Il ne faut pas confondre (source d'erreurs fréquentes):
le nombre « inscrit à côté de la case » (son adresse, l'adresse de la donnée)
avec
le nombre « à l'intérieur de la case » (son contenu : la valeur de la donnée).**

Ce sont 2 notions différentes, mais liées. Parfois même imbriquées lorsque la valeur de la donnée est également une adresse : on parle alors de **pointeur**.

En fait, comme le montre également le schéma de la figure I.7., les bits correspondant au contenu des cases mémoire sont souvent repérés par la séquence **D₇ D₆ D₅ D₄ D₃ D₂ D₁ D₀** (accès en parallèle).

Ces bits **D_i** représentent ce qu'on appelle le **bus de données (DATA BUS)** :

c'est le chemin physique, constitué ici de lignes D_i , sur lesquelles les données (contenu des cases mémoire) seront échangées entre la mémoire et la CPU du micro/processeur/contrôleur.

Comme les registres, les mémoires possèdent la propriété de fournir le contenu de chacune de leurs cases (via les lignes **D_i** décrites précédemment) si on leur indique l'adresse de cette case.

Comment ?

Pour cela, les mémoires possèdent un certain nombre de **broches d'entrées** (nommées **A_i**) sur lesquelles la CPU devra positionner le numéro de la case mémoire à laquelle elle veut accéder. (chaque entrée **A_i** pouvant valoir soit **0**, soit **1** ...).

Ces lignes **A_i** constituent ce qu'on appelle **bus de d'adresses (ADRESS BUS)** :

c'est le chemin physique, constitué des lignes A_i, sur lesquelles les adresses (des différentes cases mémoire) seront envoyées par la CPU du micro/processeur/contrôleur vers la mémoire

Prenons un exemple simple :

Supposons qu'il existe 4 broches d'**entrées** nommées $A_3 A_2 A_1 A_0$ sur la mémoire. (A_0 représentant toujours le **LSB** du bus d'adresses).

Chacune de ces entrées pouvant valoir :

- soit **0** (le plus souvent 0V),
- soit **1** (le plus souvent 5V),

il y aura donc $2^4 = 16$ combinaisons différentes pour les A_i , et donc **16 cases mémoire différentes** dans ce composant.

Si on veut lire par exemple dans la case d'adresse $(12)_{10} = 0x0C$ il suffira pour la CPU d'imposer la combinaison $A_3 A_2 A_1 A_0 = 0b\ 1\ 1\ 0\ 0$ correspondant à la valeur binaire de cette adresse.

A l'intérieur de la mémoire, il existe en fait un décodeur qui permet de sélectionner **uniquement** la case dont le numéro correspond à la valeur présente en binaire sur les entrées A_i .

En réponse, la case d'adresse $(12)_{10}$, **et elle seule**, fournira son contenu sur le bus de données.

On voit donc que dans le cas général où on a des entrées $A_r, A_{r-1} \dots A_1 A_0$ c'est à dire **r+1** entrées d'adresses de case mémoire, la **CAPACITE TOTALE** (ou **taille**) et donc le nombre total de cases de cette mémoire sera de 2^{r+1} cases mémoire (une puissance de 2).

Sur le vocabulaire courant, concernant les tailles mémoire :

Puissance de 2 : capacité exacte	$\approx$ Puissance de 10	Arrondi à
$2^{10} = 1\ 024$	$\approx 10^3$	1 K (ilo)
$2^{20} = 1\ 048\ 576$	$\approx 10^6$	1 M (éga)
$2^{30} = 1\ 073\ 741\ 824$	$\approx 10^9$	1 G (iga)
$2^{40} \approx 1,099511627 \cdot 10^{12}$	$\approx 10^{12}$	1 T (éra)

En pratique, il existe deux grandes familles de mémoire vive :

① : les mémoires de type **RAM** : comme **R**andom **A**ccess **M**emory.

- elles sont en **lecture et en écriture. (READ and WRITE)**.

Notons que lorsque la CPU écrit une donnée en RAM, la donnée précédemment présente est remplacée (écrasée et donc perdue) par la nouvelle valeur qui vient d'y être écrite.

- elles perdent leur contenu lorsque leur alimentation est coupée : on dit qu'elles sont **volatiles**. A chaque mise sous tension, elles contiennent n'importe quoi !!!

- elles servent en général à stocker des données (mais aussi des programmes en cours de développement).

- elles peuvent être très rapides, surtout pour les RAM statiques.

② : les mémoires de type **ROM** : comme **R**ead **O**nly **M**emory.

- elles ne sont **qu'en lecture (Read Only)**.

- elles conservent leur contenu même lorsque leur alimentation est coupée : on dit qu'elles sont **non volatiles**. A chaque mise sous tension, elles contiennent toujours la même chose : les valeurs qui y ont été introduites quand on les a programmées !!!

- elles servent à stocker les programmes systèmes qui s'exécuteront à chaque mise sous tension et parfois même certaines données importantes (qu'on appelle aussi parfois **permanentes ou semi permanentes**) à conserver après chaque coupure d'alimentation : mot de passe, paramètres de configuration ou de réglage ...

- elles sont, en général, plus lentes que les RAM.

Remarque 1 sur le vocabulaire :

L'acronyme RAM présenté de cette façon par opposition à celui de ROM peut prêter parfois à confusion : il aurait été mieux de les appeler R.W.M. (**R**ead and **W**rite **M**emory), car une ROM est aussi à " Random Access ".

En effet, la notion de "Random Access" (ou Accès Aléatoire) rappelle que chaque case de ce type de mémoire est accessible directement par son adresse. Par opposition aux mémoires dites séquentielles où l'accès est effectué par approches successives, comme sur une bande magnétique par exemple.

Remarque 2 sur les ROM :

Les ROM peuvent aussi se classer en plusieurs sous-familles, en fonction de la façon dont on peut y « écrire » les programmes (plus exactement, on dit la façon dont on peut les programmer) :

- les **ROM masquées : programmées en usine** au moment de la fabrication de la mémoire, elles ne se reprogramment plus (le code est directement dans les masques de fabrication). Ce procédé est valable pour de grandes quantités (fabrication en grande série) mais il est utilisé de moins en moins car non évolutif.
- les **ROM OTP (One Time Programmable)** : programmables **une seule fois** par l'utilisateur (système de fusibles), elles sont assez chères et peu utilisées en pratique,
- les **EPROM (Erasable Programmable Read Only Memory)** : elles sont programmables électriquement autant de fois que l'on veut sur un programmeur adapté et effaçables aux Ultraviolets (grâce à une petite fenêtre transparente sur leur boîtier),
- les **EEPROM (Electrically Erasable Programmable Read Only Memory)** même chose que les précédentes mais l'effacement est aussi fait électriquement. On parle aussi de mémoire de type **Flash**.

En pratique ce sont les RAM, les EPROM et surtout les EEPROM qui sont les plus utilisées.

Remarque 3 sur les signaux de commande d'une mémoire vive :

La dernière chose à signaler dans cette présentation générale des mémoires, est l'existence d'une autre **entrée** nécessaire notamment pour les RAM. qui peuvent être utilisées soit en écriture, soit en lecture. Cette autre entrée sera souvent (mais pas toujours) appelée $R/\overline{W}$ (comme READ / $\overline{\text{WRITE}}$), notation signifiant que si :

- $R/\overline{W} = 1$: phase de **lecture (READ)** d'une case (sélectionnée par les A_i) de la RAM,
- $R/\overline{W} = 0$: phase de **écriture (WRITE)** dans une case (sélectionnée par les A_i) de la RAM.

Note : en général, la présence d'une « barre » sur un signal logique indique que ce signal est **actif au niveau bas**.

Par contre, l'absence de « barre » sur un signal logique signifie que ce signal est **actif au niveau haut**.

Dans le cas des ROM, qui ne fonctionnent pas en écriture, une entrée de type $\overline{\text{OE}}$ (comme **OUTPUT ENABLE**) est présente à la place de la commande $R/\overline{W}$ des RAM.

Cette entrée obéit à la logique suivante :

- $\overline{\text{OE}} = 1$, le contenu de la case mémoire (sélectionnée par les A_i) **n'est pas présent** sur les D_i ,
- $\overline{\text{OE}} = 0$, le contenu de la case mémoire (sélectionnée par les A_i) **est présent** sur les D_i .

Ce signal peut donc être interprété un peu comme une entrée de type $\overline{\text{RD}}$ (comme $\overline{\text{READ}}$).

Les signaux $R/\overline{W}$ et $\overline{OE}$ appartiennent à un troisième bus : le bus de contrôle (**CONTROL BUS**):

c'est le chemin physique où sont véhiculés tous les signaux nécessaires au fonctionnement et à la synchronisation complète des différents composants du système (signaux de commande et de contrôle divers, horloges, signaux de RESET ...)

Au final une mémoire vive se présentera, « vue de l'extérieur », comme indiqué sur la figure I.8. :

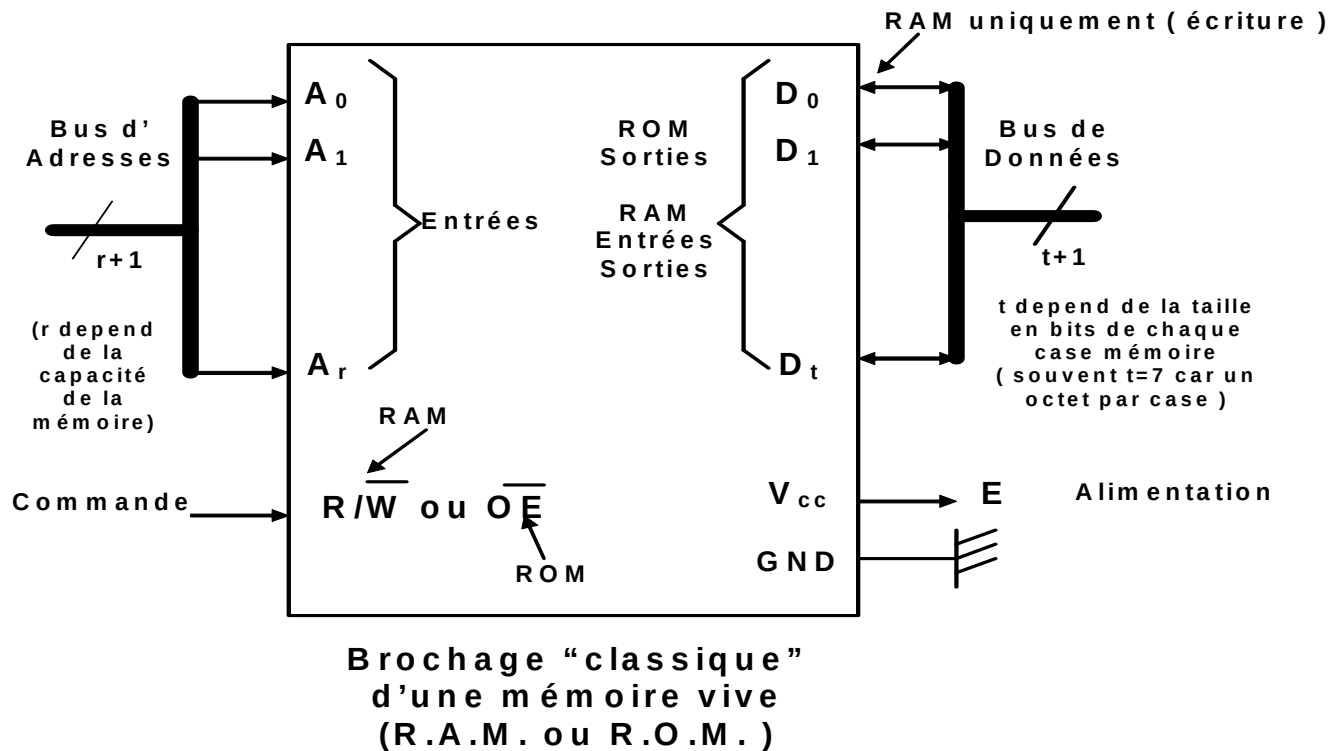


Figure I.8.

La documentation (cf. figure I.9.) de la page suivante montre un exemple de circuit mémoire ROM de type EPROM. Jetez y un coup d'œil pour retrouver toutes les notions de base que l'on vient de présenter et faites le petit exercice qui suit ...

Exercice :

- retrouver les différentes broches vues dans les généralités (voir le **PIN CONFIGURATION**).
- quelle est la taille d'une case mémoire ? Justifier.
- quelle est la capacité mémoire totale de ce composant ? Justifier. Faire le lien avec le 32 de la référence 2732.
- retrouver le rôle de l'entrée $\overline{OE}$ (voir le tableau **MODE SELECTION**).

Noter l'existence à l'intérieur du composant (voir le **BLOCK DIAGRAM**) du décodeur. Rappeler quel est son rôle.

2732

32K (4K x 8) UV ERASABLE PROM

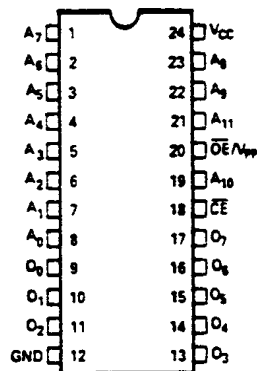
- Single +5V $\pm$ 5% Power Supply
- Output Enable for MCS-85™ and MCS-86™ Compatibility
- Fast Access Time: 450ns Max.
- Low Power Dissipation:
150mA Max. Active Current
30mA Max. Standby Current
- Pin Compatible to Intel® 2716 EPROM and 2332/2364 ROMs
- Completely Static
- Simple Programming Requirements
— Single Location Programming
— Programs with One 50ms Pulse
- Three-State Output for Direct Bus Interface

The Intel® 2732 is a 32,768-bit ultraviolet erasable and electrically programmable read-only memory (EPROM). The 2732 operates from a single 5-volt power supply, has a standby mode, and features an output enable control. For production, the pin compatible 2332 and 2364 ROMs are available. The total programming time for all bits is three and a half minutes. All these features make designing with the 2732 in microcomputer systems faster, easier, and more economical.

An important 2732 feature is the separate output control, Output Enable ($\overline{OE}$), from the Chip Enable control ($\overline{CE}$). The $\overline{OE}$ control eliminates bus contention in multiple bus microprocessor systems. Intel's Application Note AP-30 describes the microprocessor system implementation of the $\overline{OE}$ and $\overline{CE}$ controls on Intel's 2716 and 2732 EPROMs. AP-30 is available from Intel's Literature Department.

The 2732 has a standby mode which reduces the power dissipation without increasing access time. The maximum active current is 150mA, while the maximum standby current is only 30mA, an 80% savings. The standby mode is achieved by applying a TTL-high signal to the $\overline{CE}$ input.

PIN CONFIGURATION



PIN NAMES

A ₀ -A ₁₁	ADDRESSES
$\overline{CE}$	CHIP ENABLE
$\overline{OE}$	OUTPUT ENABLE
O ₀ -O ₇	OUTPUTS

MODE SELECTION

MODE \ PINS	$\overline{CE}$ (18)	$\overline{OE}/V_{pp}$ (20)	V _{CC} (24)	OUTPUTS (9-11,13-17)
Read	V _{IL}	V _{IL}	+5	D _{OUT}
Standby	V _{IH}	Don't Care	+5	High Z
Program	V _{IL}	V _{pp}	+5	D _{IN}
Program Verify	V _{IL}	V _{IL}	+5	D _{OUT}
Program Inhibit	V _{IH}	V _{pp}	+5	High Z

BLOCK DIAGRAM

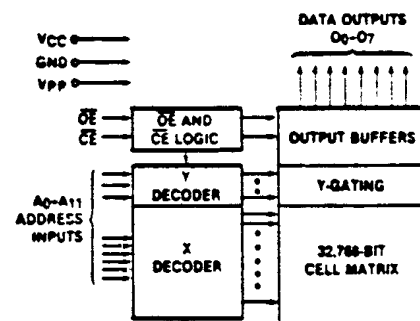


Figure I.9.

A titre indicatif, les courbes de la figure I.10., **très simplifiées**, montrent les signaux utiles sur l'exemple d'un accès mémoire en écriture (**WRITE CYCLE**) dans une RAM de 64 Ko, (avec 16 lignes d'adresses A15 à A0) représentés sous la forme de 3 chronogrammes (numérotés de ① à ③), avec à chaque fois le temps en abscisse.

En pratique, d'autres signaux sont souvent utiles sur les chronogrammes réels (horloge pour les transferts synchrones, signaux d'acquittement pour les échanges asynchrones...) mais ils n'ont pas été représentés ici, dans un souci de simplification.

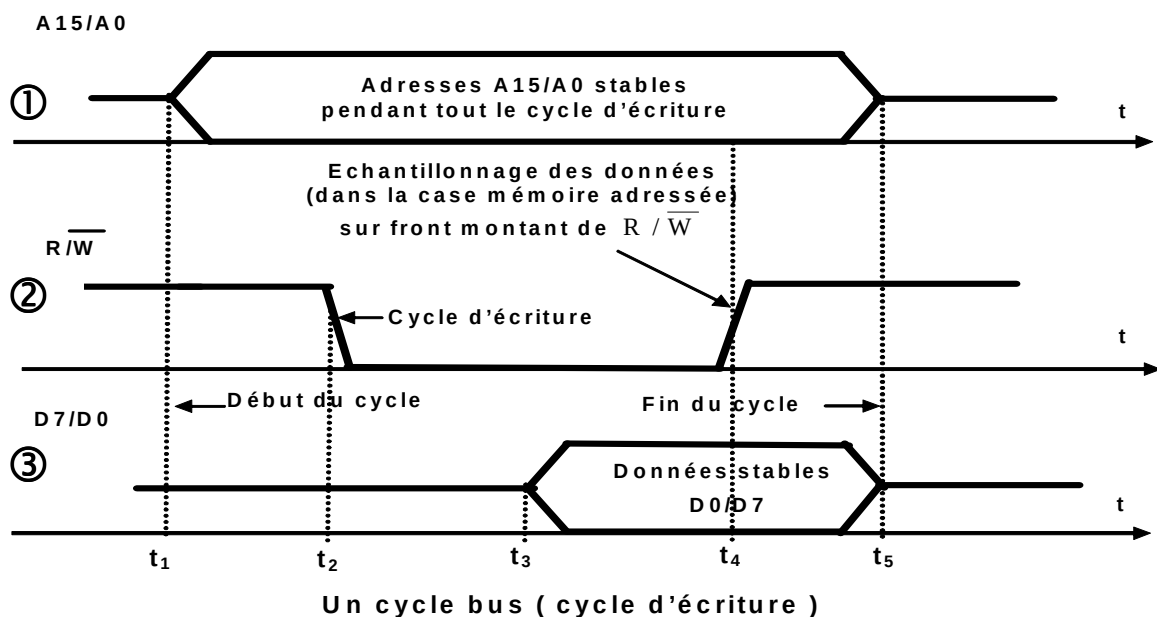
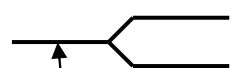


Figure I.10.

Décrivons les instants caractéristiques apparaissant sur ces chronogrammes.

Instant t_1 (début du cycle) : à partir de cet instant, le bus d'adresses véhicule une adresse **valide et stable** pendant toute la durée du cycle. Pour ne pas alourdir le schéma (il aurait fallu rigoureusement un chronogramme pour chaque ligne A_i), l'ensemble du bus d'adresses a été représenté sur le chronogramme ①.

La convention



(très souvent utilisée sur les documentations) veut

dire que le niveau présent sur chaque ligne A_i peut-être **soit le niveau bas, soit le niveau haut**, et donc que cette valeur importe peu (n'est pas pertinente) pour comprendre le fonctionnement du cycle. Mais, ce qui est important, c'est que la valeur présente sur chaque ligne A_i (soit niveau bas, soit niveau haut) doit être stable, car c'est cette combinaison qui permettra de sélectionner la case mémoire concernée par ce cycle d'écriture. Avant cette stabilisation du cycle (et donc ici avant t_1), le niveau sur les A_i n'est pas pertinent ou alors en haute impédance (représenté sur le chronogramme par un niveau intermédiaire entre le niveau bas et le niveau haut).

Instant t_2 : le micro/processeur/contrôleur indique à la mémoire que c'est un cycle d'écriture en faisant passer sa sortie $R/\overline{W}$ à 0 (cf. chronogramme ②). A partir de cet instant, la mémoire (qui voit passer son entrée $R/\overline{W}$ à 0) sait qu'elle va recevoir sur le bus de données D7/D0 une combinaison de 0 et de 1 qu'elle devra la stocker à l'adresse présente sur le bus d'adresses A15/A0.

Instant t_3 : le micro/processeur/contrôleur sort une donnée valide sur le bus de données D7/D0 (cf. chronogramme ③). Dans ce cas là (cycle d'écriture), les données vont dans le sens micro/processeur/contrôleur vers mémoire. Pour un cycle de lecture, ce serait l'inverse.

Instant t_4 : sur le front montant du signal $R/\overline{W}$ (cf. chronogramme ②), la donnée présente et toujours valide sur le bus de données D7/D0 est « lachée » c'est-à-dire effectivement écrite et mémorisée dans la case mémoire toujours sélectionnée par les A_i (qui sont toujours stables à cet instant t_4).

Instant t_5 (fin du cycle) : les différents bus « se relâchent » et sont prêts pour l'accès mémoire suivant, qui peut être soit un nouveau cycle d'écriture, soit un cycle de lecture...

L'intervalle de temps entre t_1 et t_5 représentant **la durée totale de ce cycle d'écriture** (qui dépend de la fréquence d'horloge du micro/processeur/contrôleur) n'aura duré au total que quelques ns.

On voit donc que l'activité est permanente sur les différents bus. Il n'y aurait qu'à mettre une sonde d'oscilloscope sur ces différents signaux pour s'en convaincre.

Il existe d'ailleurs des appareils, appelés **analyseurs logiques** (ou **analyseurs de bus**) qui permettent de mesurer cette activité permanente sur les différents bus, et qui sont en fait mieux adaptés qu'un simple oscilloscope pour ce type de mesure.

En pratique, on peut être amené à vouloir visualiser certaines zones de mémoire pour en vérifier le contenu, ou pour suivre l'évolution de tel ou tel registre ou de telle ou telle variable.
On pourra par exemple obtenir les types d'écran suivants qu'il faut savoir lire et interpréter :

- contenu « brut » en hexadécimal/ASCII (sans référence symbolique) :

avec affichage du contenu de 16 cases mémoire consécutives par ligne.

① ← ② → ③

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
000	--	FF	18	1C	2A	10	01	98	00	00	00	00	00	00	C5	40	-...*... ..@
010	00	00	00	FF	00	01	40	00	00	00	00	28	20	00	00	00	@. ... (...
020	00	00	A0	00	2B	00	00	00	08	28	12	0C	02	20	02	20	+... .(....
030	00	01	00	05	00	00	02	00	80	04	20	10	20	01	00	00	
040	10	00	00	40	20	00	18	06	00	00	00	0A	00	04	00	08	...@
050	2B	00	08	00	20	08	00	08	00	08	80	11	0E	20	08	00	+... ..
060	02	10	38	81	09	12	00	10	10	20	00	00	42	00	01	81	..8.... ..B...
070	RR	E0	00	00	08	80	10	34	12	41	00	44	00	08	40	40	R.....4 .A.D..@@
080	--	FF	7F	1C	2A	3F	BF	FF	FF	07	00	00	00	00	00	--	*?... ..-
090	--	00	FF	00	00	--	--	--	02	00	--	--	07	00	00	00	
0A0	00	80	04	00	11	08	10	00	C0	58	80	00	20	02	00	00	X... ..
0B0	00	01	10	00	00	00	4A	40	40	08	C0	60	08	22	41	C2	J@ @...`"A.
0C0	40	80	64	20	06	02	00	42	00	00	01	00	00	40	01	20	@.d ...B@.
0D0	00	80	15	00	10	C0	20	00	04	18	00	01	08	14	00	00	
0E0	00	00	00	06	00	04	04	30	10	80	00	40	00	20	00	00	0 ...@. ...
0F0	RR	E0	00	00	08	80	10	34	12	41	00	44	00	08	40	40	R.....4 .A.D..@@
100	--	FF	7F	1C	2A	--	01	--	--	--	00	00	E3	B8	3D	1C	*...- ...=.
110	00	10	80	00	10	80	00	08	00	00	25	80	00	00	00	00	\$.....

Hex Symbolic

- contenu sous différents formats avec référence symbolique:

avec affichage du contenu d'une case mémoire par ligne et le nom du registre (à mettre en correspondance avec la documentation du PIC) ou de la variable (déclarée et définie dans votre programme source)

① ← ② → ③

Address	Hex	Decimal	Binary	Char	Symbol Name
000	--	-	-----	-	INDF
001	0xFF	255	11111111	'.'	TMR0
002	0x18	24	00011000	'.'	PCL
003	0x1C	28	00011100	'.'	STATUS
004	0x2A	42	00101010	'*'	FSR
005	0x10	16	00010000	'.'	PORTA
006	0x01	1	00000001	'.'	PORTB
007	0x98	152	10011000	'.'	PORTC
008	0x00	0	00000000	'.'	PORTD
009	0x00	0	00000000	'.'	PORTE
00A	0x00	0	00000000	'.'	PCLATH
00B	0x00	0	00000000	'.'	INTCON
00C	0x00	0	00000000	'.'	PIR1
00D	0x00	0	00000000	'.'	PIR2
00E	0xC5	197	11000101	'.'	TMR1
00F	0x40	64	01000000	'@'	TMR1H
010	0x00	0	00000000	'.'	T1CON
011	0x00	0	00000000	'.'	TMR2

Hex Symbolic

LES DEUX PAGES DE RESUME SUR LES MEMOIRES VIVES ET LES DIFFERENTS BUS :

Ces deux pages, résument l'essentiel des notions que l'on vient de présenter.

Le tableau ci-dessous rappelle les principales propriétés des mémoires vives :

Propriétés	RAM	ROM
Surtout pour :	Les données	Le programme (instructions)
Mode d'accès	Aléatoire	Aléatoire
Type d'accès	Lecture et écriture	Lecture seulement
Coupure d'alimentation	Volatile	Non volatile
Vitesse (cf. temps d'accès)	Rapides (surtout RAM statiques)	Plus lentes que les RAM
Capacité pour $A_r, A_{r-1} \dots A_1 A_0$	2^{r+1} cases mémoire	2^{r+1} cases mémoire

Les différents bus qui relient CPU et mémoire seront donc :

- le bus d'adresses :

regroupant toutes les lignes d'adresses permettant de pointer vers (et donc d'accéder à) toutes les cases mémoire du système.

Par exemple, pour un système à 16 bits d'adresses, il sera classiquement noté $A_{15} A_{14} \dots A_2 A_1 A_0$ (ou $[A_{15} \dots A_0]$), A_0 désignant son bit de poids faible (**LSB**) et A_{15} désignant son bit de poids fort (**MSB**).

Dans ce cas, il y aura $2^{16} = 65\,536 \approx 64\text{ K}$ cases mémoires accessibles, de l'adresse **0x0000** pour la case **0** à l'adresse **0xFFFF** pour la case **65535**.

Par exemple si à un instant donné, on a la combinaison suivante en binaire sur le bus d'adresses :

A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

(ce qui donne en hexadécimal **0x5555**, valeur purement arbitraire), cela correspond à l'adresse (au numéro) de la case mémoire où le micro/processeur/contrôleur veut accéder (pour une lecture ou une écriture dans cette case).

Notons que ce bus d'adresses est **monodirectionnel** :

il est toujours orienté de la CPU vers la mémoire, car c'est le micro/processeur/contrôleur, sous contrôle du programme, qui sélectionne une case mémoire.

- le bus de données :

regroupant toutes les lignes de données (correspondant au contenu de la case mémoire pointée pour un **accès en lecture** ou à la valeur que l'on veut écrire dans cette case pour un **accès en écriture**).

Par exemple , il sera classiquement noté $D_7 D_6 \dots D_1 D_0$ (ou $[D_7 \dots D_0]$) pour un bus 8 bits.

Si par exemple le contenu de la case mémoire pointée est **0xAA** cela signifie (pour une opération de lecture), que le bus de données prendra la valeur suivante en binaire, valeur fournie par la case mémoire adressée et qui sera transportée par le bus de données de la mémoire vers la CPU :

D7	D6	D5	D4	D3	D2	D1	D0
1	0	1	0	1	0	1	0

Pour une opération d'écriture dans une case mémoire, le bus de données reflète en binaire la valeur à écrire dans la case mémoire sélectionnée par le bus d'adresses.

Ce bus est donc **bidirectionnel** :

- il vient de la mémoire (sorties mémoire vers entrées CPU) pour une opération de lecture,
- il va vers la mémoire (sorties CPU vers entrées mémoire) pour une opération d'écriture.

- le bus de contrôle :

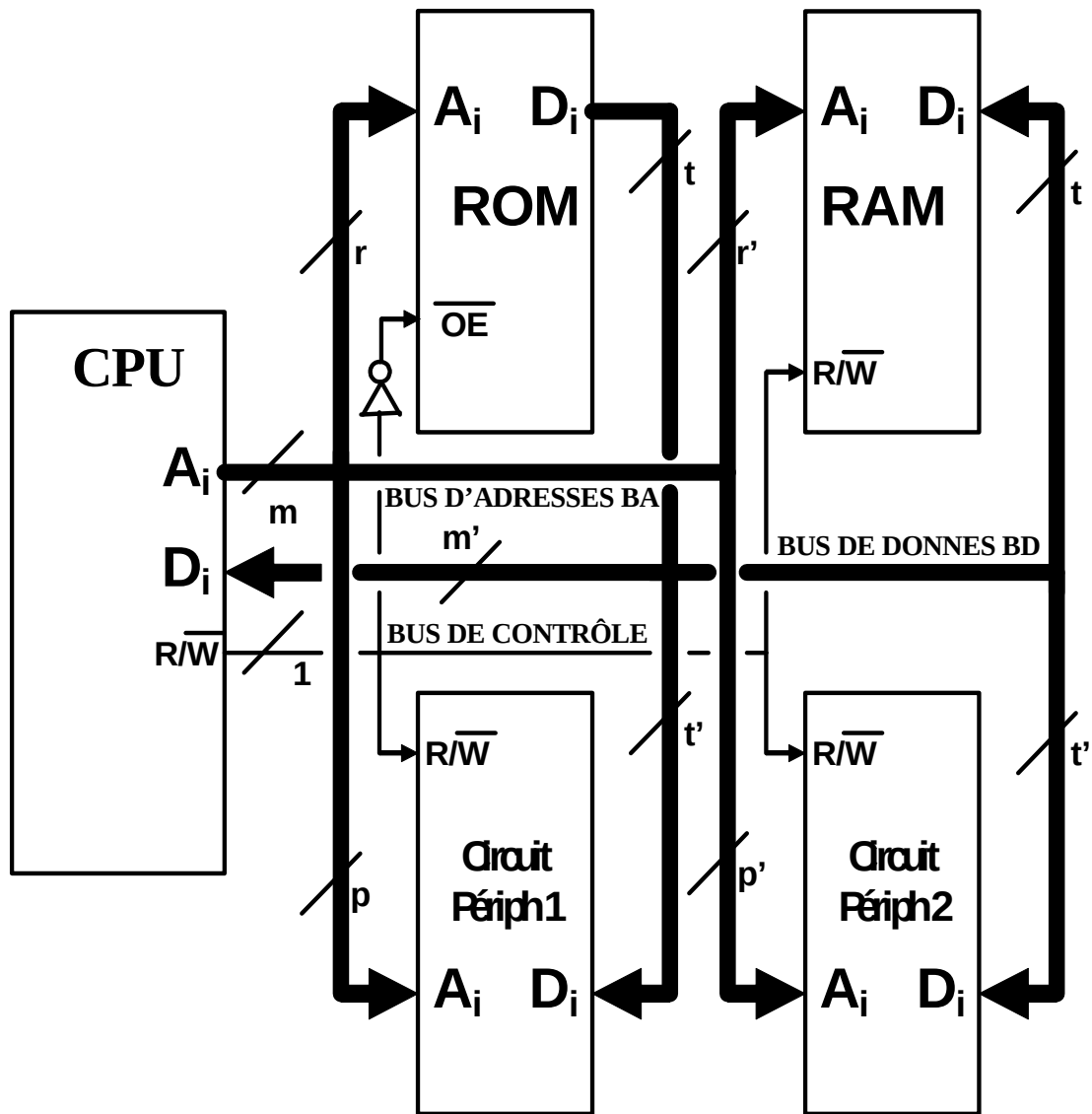
véhiculant toutes les différentes informations permettant au système de fonctionner :

- signal de lecture ou d'écriture,
- horloges,
- reset,
- divers signaux de synchronisation,
- signaux d'interruption,
- signaux d'acquittement,
- signaux de contrôle divers ...

C'est en pratique le bus le plus complexe, car contrairement aux deux autres bus précédents, chaque ligne de ce bus possède une signification spécifique et particulière.

I.2.3. SYNOPTIQUE SIMPLIFIE

Ces notions nous permettent d'aboutir au synoptique (**très** simplifié) de la figure I.11. ci-dessous, présentant une architecture type d'un système à base de micro/processeur/contrôleur.



Architecture type dite de Von Neuman

Figure I.11.

Les blocs **Circuits Périph 1 et 2** représentent les interfaces (Entrées/Sorties) avec le monde extérieur (l'environnement).

Cette architecture précise le schéma que l'on avait donné en introduction sur la figure I.1.a.

Rappelons, comme on l'a vu aussi au tout début de ce cours, que cette architecture est entièrement intégrée dans le boîtier même du microcontrôleur (cf. figure I.1.b.).

Les lignes en **traits épais** représentent les **différents bus** et la lettre (ou le chiffre) « **slashée** » à proximité (comme **/r** , **/t** pu même **/1**) représente la **largeur** de ce bus : le nombre de lignes constitutives du bus.

Ce type d'architecture est appelée architecture de **Von Neuman** (un **seul** bus d'adresse et un **seul** de données).

Petit exercice :

Expliquer le rôle de l'inverseur utilisé pour piloter l'entrée $\overline{\text{OE}}$ de la ROM.

Eléments de réponse :

Examiner bien la logique du signal **R / $\overline{\text{W}}$** fournie par la CPU...

I.2.4. DECODAGE D'ADRESSES ET MAPPING MEMOIRE

Dans cette présentation (et architecture précédente) simplifiée, nous avons fait l'impasse sur le procédé qui permet de différencier les différents composants adressables (mémoires, registres internes des circuits périphériques d'E/S) existant autour de la CPU.

En effet, comme on l'a vu, chaque mémoire, possédant un bus d'adresses $[A_r \dots A_0]$ contient une case d'adresse **0**, une case d'adresse **1** ... une case d'adresse 2^{r+1} .

Lorsque la CPU voudra lire le contenu de la case d'adresse 0x0010 (par exemple), va t'elle accéder à la RAM (qui possède une case d'adresse 0x0010) ou à la ROM (qui possède aussi une case d'adresse 0x0010) ?

Avec notre système **très** simplifié précédent, il faut répondre :

"les deux, mon capitaine !"

Cette situation n'est pas acceptable, car pour une adresse, il ne doit y avoir qu'une seule case physique associée. Sinon, on a ce qu'on appelle classiquement un **conflit de bus**.

C'est pourquoi, il faut pouvoir **différencier de façon certaine** les deux (ou plus) circuits de mémoire (et aussi les circuits périphériques bien sûr).

Pour éviter ces conflits lorsque la CPU veut écrire (ou lire) dans une case donnée (pour éviter qu'elle n'écrive ou ne lise en même temps dans plusieurs cases physiquement différentes), il existe un système appelé **décodage d'adresses** qui permet d'associer une plage d'adresses différente à chaque circuit mémoire ou à chaque circuit périphérique.

Pour cela, chaque composant adressable et implanté dans une architecture type celle de la figure I.11. précédente, possède une entrée notée :

- souvent $\overline{CS}$ comme $\overline{CHIP\ SELECT}$ (**sélection de composant**),
- mais parfois aussi $\overline{CE}$ comme $\overline{CHIP\ ENABLE}$ (**validation de composant**)

actives toutes les deux au niveau bas.

(voir la documentation de l'EPROM **2732** présentée précédemment sur la figure I.9. pour s'en convaincre).

Cela signifie que tant que cette entrée de sélection $\overline{CS} = 1$, tout ce passe comme si le composant n'existait pas : il demeure « insensible » à la combinaison présente sur ses entrées d'adresses A_i , comme s'il était enlevé de l'architecture.

Par contre, dès que cette entrée $\overline{CS}$ passe à 0, le composant redevient **actif** et peut répondre à un ordre d'écriture ou de lecture venant de la CPU.

Remarque :

Parfois, sur certains composants (plus rare) cette entrée est appelée CS ou CE. Dans ce cas, absence de « barre » sur le nom, cela signifie qu'elle sera active au niveau haut.

La logique de décodage permet de s'assurer que parmi les N composants adressables présents dans l'architecture, un seul ait son $\overline{CS}$ à 0, les N-1 restants ayant leurs $\overline{CS}$ respectifs à 1.

Cette technique permet au final de réaliser la **cartographie mémoire (memory mapping)** du système.

On va en voir le principe sur un exemple très simple.

Exemple de cahier des charges :

Supposons qu'on doive implanter sur une CPU ayant 16 broches d'adresses (donc un bus d'adresses de A_0 à A_{15} , pouvant adresser au maximum $2^{16} \approx 64$ K cases mémoire différentes), une RAM de 32Ko (32 768 octets) et une ROM de la même capacité, pour pouvoir bénéficier du maximum de mémoire adressable par le système.

La technique utilisée pour commander les entrées $\overline{CS}$ de la RAM et de la ROM fonctionne en trois temps (ou 3 étapes) :

Etape 1 :

Cette étape, **théorique**, se fait « sur le papier ». Elle consiste à définir la **CARTOGRAPHIE MEMOIRE** (ou **MEMORY MAPPING** ou **CARTE MEMOIRE**) du système.

Le mapping consiste à partager la capacité totale adressable par le microprocesseur en **différentes zones (qui ne se « chevauchent » pas)** et ensuite d'affecter les composants à chacune de ces zones.

C'est le concepteur de la carte et de l'architecture d'un nouveau système qui choisit son mapping, en fonction du nombre et de la taille des composants adressables à mettre en œuvre ou d'autres contraintes que nous ne détaillerons pas ici (cf. par exemple la remarque 2 page suivante).

L'utilisateur de la carte se contentera de respecter ce mapping (ce qui sera fait grâce au linker qui permet de rendre un fichier source compilé compatible avec ce mapping et donc avec l'architecture matérielle de la carte utilisée).

Dans notre exemple (cf. figure I.12.), on peut procéder de la façon suivante :

- affecter la **RAM** aux **adresses absolues** comprises entre **0 (0x0000)** et **32 767 (0x7FFF)** (adresses **basses** du mapping),
(ainsi la **case 0** de la RAM correspondra à l'adresse absolue **0x0000**, sa case **1** à l'adresse absolue **0x0001**, et sa dernière case (la **32767**) à l'adresse absolue **0x7FFF**).
- affecter la **ROM** aux **adresses absolues** comprises entre **32 768 (0x8000)** et **65 535 (0xFFFF)** (adresses **hautes** du mapping).
(ainsi la **case 0** de la ROM correspondra à l'adresse absolue **0x8000** (soit la 32769^{ème} du mapping), sa case **1** à l'adresse absolue **0x8001** (soit la 32770^{ème} du mapping) et sa dernière case (la **32767**) à l'adresse absolue **0xFFFF** (soit la 65536^{ème} du mapping).
($\equiv$ ainsi 2 personnes différentes ont chacune leur propre adresse mail : **OK**)

Ce découpage est résumé sur le schéma de la figure I.12.

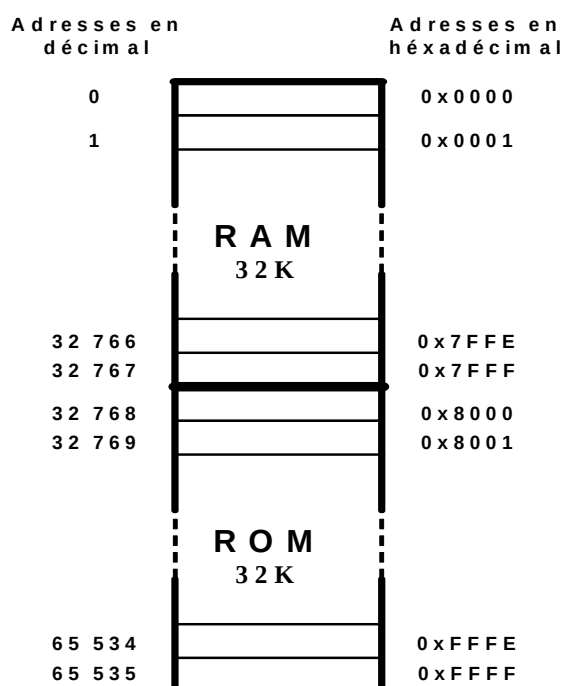


Figure I.12.

Remarque 1:

A titre indicatif, lorsqu'il n'y a pas de décodage (comme dans notre système simplifié de la figure I.11. précédente), le mapping aurait été celui de la figure I.13. ci-dessous :

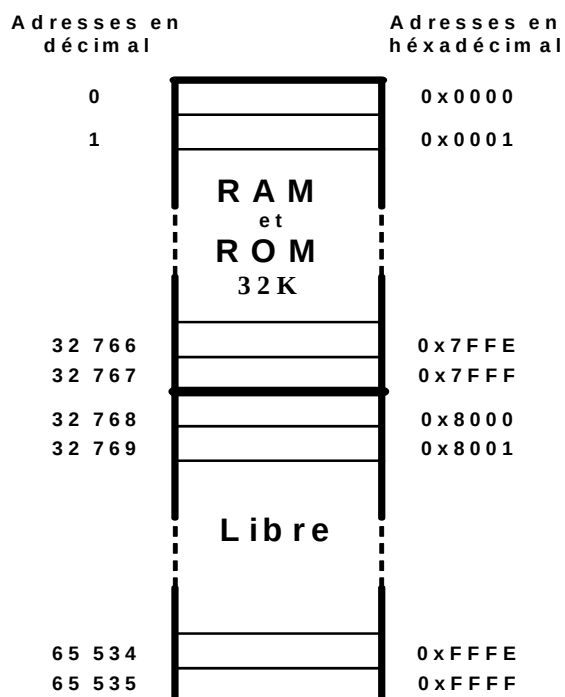


Figure I.13.

Et on voit que dans ce cas là, on ne peut pas différencier la RAM de la ROM : il y aura bien conflit sur le bus de données (impossible de différencier les cases mémoire de la RAM et celles de la ROM !!!). Ce mapping ne marche pas car il y a 2 cases d'adresses absolues 0x0000, 2 cases d'adresses absolues 0x0001 ...

(= ici 2 personnes différentes ont la même adresse mail : **KO**)

Remarque 2 :

Dans certains cas, en fonction du micro/processeur/contrôleur, on sera obligé de mettre la ROM à une adresse particulière. Car, après un RESET ou à la mise sous tension, le micro/processeur/contrôleur recherche toujours la première instruction **à la même adresse** imposée par le constructeur du micro/processeur/contrôleur: il faudra donc placer de la ROM à cette adresse, car si c'est de la RAM, il y aura n'importe quoi à cette adresse et le micro/processeur/contrôleur ne trouvera pas la première instruction à exécuter : le système plantera à tous les coups...

Tous les systèmes auront donc leur propre cartographie mémoire, qu'il faudra respecter pour accéder aux différents composants

(mémoires, registres internes et circuits périphériques)

On aura l'occasion d'examiner plus loin ce mapping pour le PIC 16F877.

Etape 2 :

Il s'agit, dans cette étape de trouver les **équations logiques** permettant de fabriquer les signaux $\overline{\text{CS}}$ de la RAM et de la ROM en fonction des valeurs des A_i .

Pour cela, en pratique, il vaut mieux exprimer les adresses en notation hexadécimale, ou mieux en binaire.

Nous pouvons réécrire le mapping de la figure I.12. d'une autre façon, en développant l'**ADDRESS BUS** en binaire et en écrivant pour chaque zone, la première (**début**) et la dernière adresse (**fin**), comme dans le tableau ci-dessous :

Circuit	Hexadécimal	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀
RAM	début: 0x0000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	à	0	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
	fin : 0x7FFF	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
ROM	début: 0x8000	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	à	1	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
	fin : 0xFFFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Sur ce tableau, les **X** représentent indifféremment la valeur 0 ou la valeur 1.

En examinant ce tableau, il apparaît clairement que le seul bit significatif pour différencier les 2 circuits de mémoire RAM et ROM est le bit A_{15} , les autres bits du bus d'adresses (A_{14} à A_0) pouvant prendre toutes les valeurs pour la RAM et pour la ROM.

Ainsi :

- pour la RAM : $A_{15} = 0$, quelque soient les autres adresses,
- pour la ROM : $A_{15} = 1$, quelque soient les autres adresses.

C'est ce signal que nous utiliserons pour le décodage de la façon suivante :

- $\overline{CS}_{RAM} = A_{15}$ et donc A_{15} connecté au $\overline{CS}$ de la RAM (actif au niveau bas),
- $\overline{CS}_{ROM} = \overline{A_{15}}$ et donc $\overline{A_{15}}$ connecté au $\overline{CS}$ de la ROM (actif au niveau bas).

Etape 3 :

On procède enfin à la **réalisation électronique** du **DECODAGE D'ADRESSES** :

c'est l'ensemble des composants qui concourent à la différenciation totale des circuits contenant des informations échangeables avec le micro/processeur/contrôleur
(mémoires, registres internes et circuits périphériques)

Ici, le cas est très simple puisqu'un **simple inverseur** suffit pour réaliser ce décodage d'adresses comme on peut le voir sur la figure I.14. ci-dessous :

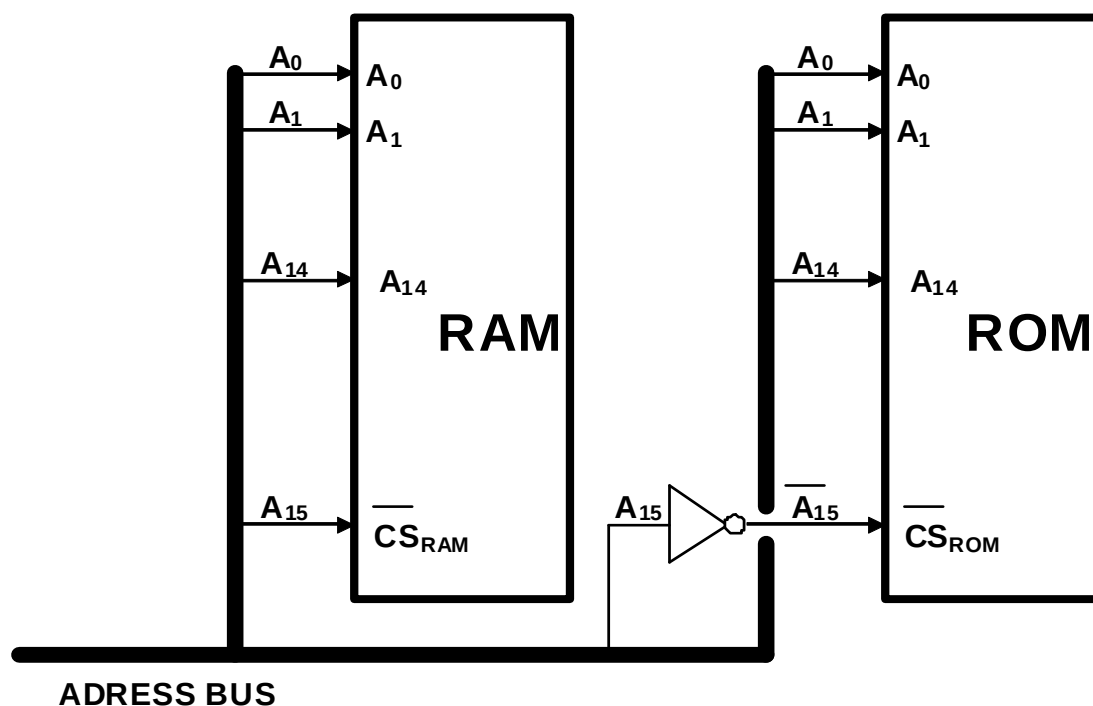


Figure I.14.

Au final, toujours pour le même exemple :

- la ligne A_{15} permet de choisir soit la RAM (quand elle vaut 0) soit la ROM (quand elle vaut 1),
- les lignes A_0 à A_{14} permettent de choisir une case dans la mémoire sélectionnée par A_{15} .

Et de façon très générale :

- un certain nombre de bits du bus d'adresses (à partir des MSB) permettront de différencier les composants en participant à la génération de leurs différents $\overline{CS}$,
- les autres bits du bus d'adresses (les poids faibles) permettront alors de sélectionner une case dans le composant sélectionné par les bits de poids forts.

Je propose dans les pages suivantes un exercice partiellement corrigé permettant de comprendre sur un cas concret plus complet que dans l'exemple que l'on vient de voir, comment est réalisée une logique de décodage d'adresses à partir d'un « BCD-TO-DECIMAL DECODER »...

Il est vivement conseillé (et même obligatoire) d'y jeter un œil !!!

**Cela vous permettra de voir et de vérifier si vous avez bien assimilé
les notions de base présentées jusqu'ici ...**

I.2.5. EXERCICE : ANALYSE D'UN CAS CONCRET

ETUDE DU DECODAGE D'ADRESSES SUR UN SYSTEME

Le but de cet exercice est d'analyser un système à base d'un microcontrôleur (cf figure I.15.), pour déterminer la **cartographie** (ou mapping) des différentes mémoires implantées sur ce système (plages d'adresses affectées à chaque composant), phase nécessaire avant d'y implanter des programmes.

Le microcontrôleur étudié ici présente, entre autres :

- un **bus d'adresses** sur 16 bits noté : **[A15 .. A0]**,
- un **bus de données** sur 8 bits noté : **[D7 .. D0]**,
- une **sortie** notée $\overline{AS}$, comme **Adress Strobe**, active **au niveau bas**, indiquant aux circuits extérieurs que la combinaison présente sur le bus d'adresse est stable, c'est-à-dire qu'il n'y a pas de changement d'état des **Ai** tant que $\overline{AS} = 0$.
- une sortie **R/W** : signal de **lecture (= 1)** ou **d'écriture (= 0)**.

1. Questions d'introduction.

1.1. De façon générale, quelle est l'utilité du **mapping mémoire** ?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

1.2. Indiquez le lien entre **décodage d'adresses** et **mapping mémoire**.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Le schéma (incomplet) que l'on se propose d'étudier est donné sur la figure I.15. ci-dessous.
(tous les signaux nécessaires au bon fonctionnement du système n'étant pas représentés, par souci de simplification).

Certaines parties de ce schéma seront d'ailleurs à compléter dans les questions suivantes.

La documentation du décodeur **74HC42** utilisé ici est fournie sur la figure I.16. de la page suivante.

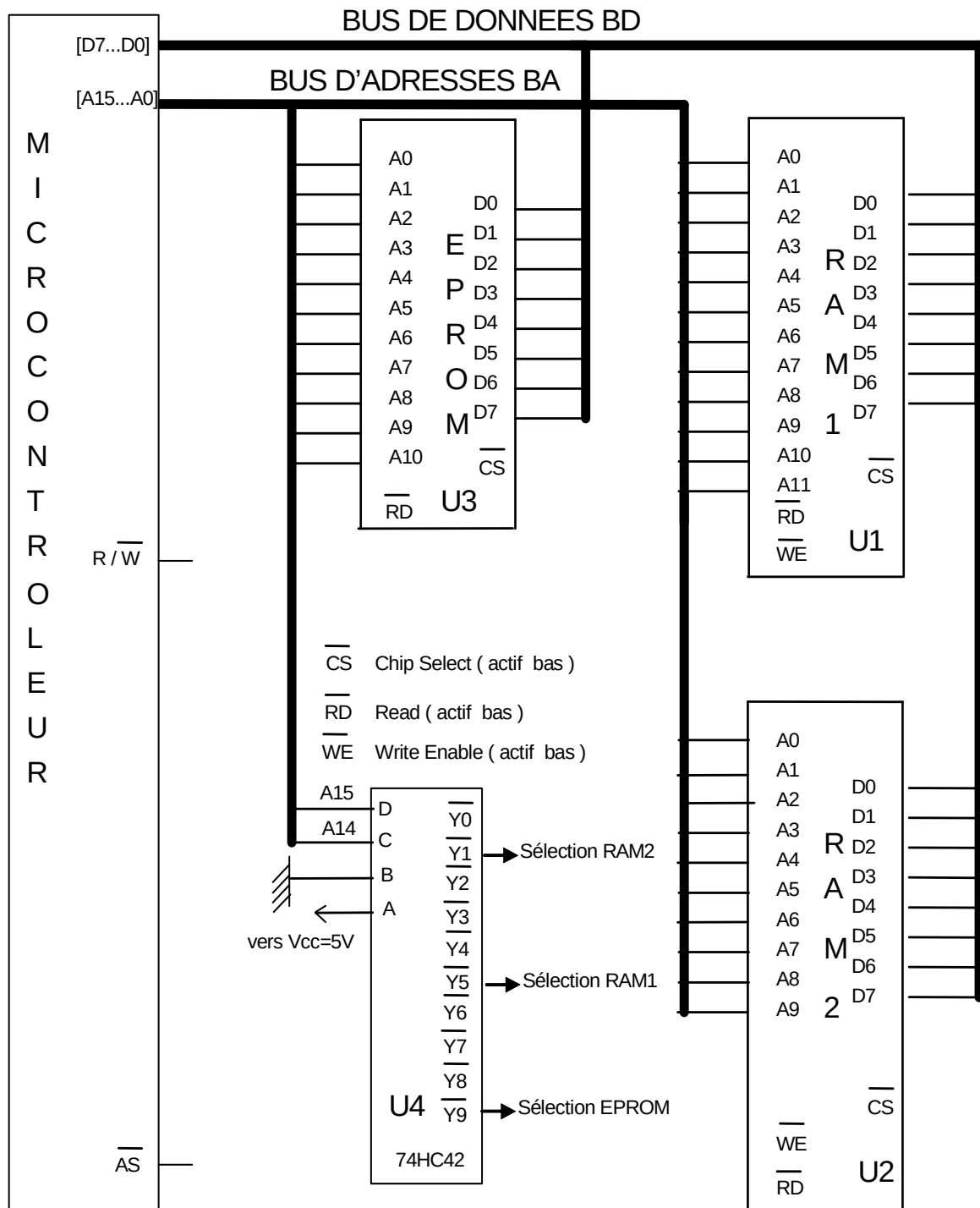


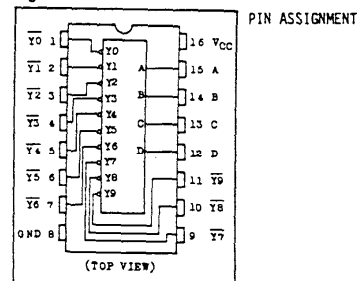
Figure I.15.

DOCUMENTATION ET « TRUTH TABLE » DU 74HC42

TOSHIBA

TC74HC42P BCD-TO-DECIMAL DECODER

The TC74HC42 is a high speed CMOS BCD-TO-DECIMAL DECODER fabricated with silicon gate C²MOS technology. It achieves the high speed operation similar to equivalent LSTTL while maintaining CMOS low power dissipation. A BCD code applied to the four inputs (A – D) provides a low level at the selected one of ten decimal decoded outputs. An illegal BCD code such as eleven thru fifteen gives a high level at all outputs. This device also can be used as 3-to-8 LINE DECODER, when D input is assigned as a disable input. This device is useful for code conversion, address decoding, memory selection, demultiplexing, or readout decoding. All inputs are equipped with protection circuits against static discharge or transient excess voltage.



TRUTH TABLE

CODE No.	BCD INPUTS				DECIMAL OUTPUTS									
	D	C	B	A	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇	Y ₈	Y ₉
0	L	L	L	L	L	H	H	H	H	H	H	H	H	H
1	L	L	L	H	H	L	H	H	H	H	H	H	H	H
2	L	L	H	L	H	H	L	H	H	H	H	H	H	H
3	L	L	H	H	H	H	H	L	H	H	H	H	H	H
4	L	H	L	L	H	H	H	H	L	H	H	H	H	H
5	L	H	L	H	H	H	H	H	H	L	H	H	H	H
6	L	H	H	L	H	H	H	H	H	H	L	H	H	H
7	L	H	H	H	H	H	H	H	H	H	H	L	H	H
8	H	L	L	L	H	H	H	H	H	H	H	H	L	H
9	H	L	L	H	H	H	H	H	H	H	H	H	H	L
10	H	L	H	L	H	H	H	H	H	H	H	H	H	H
11	H	L	H	H	H	H	H	H	H	H	H	H	H	H
12	H	H	L	L	H	H	H	H	H	H	H	H	H	H
13	H	H	L	H	H	H	H	H	H	H	H	H	H	H
14	H	H	H	L	H	H	H	H	H	H	H	H	H	H
15	H	H	H	H	H	H	H	H	H	H	H	H	H	H

AC ELECTRICAL CHARACTERISTICS (C_L = 50 pF, Input t_r = t_f = 6 ns)

PARAMETER	SYMBOL	V _{CC}	T _a = 25 °C			T _a = -40 ~ 85 °C		UNIT
			MIN.	TYP.	MAX.	MIN.	MAX.	
Propagation Delay Time	t _{PLH}	2.0	—	80	165	—	200	ns
	t _{PHL}	4.5	—	21	33	—	40	
		6.0	—	18	28	—	34	
Input Capacitance	C _{IN}		—	5	10	—	10	pF
Power Dissipation Capacitance	C _{PD}		—	67	—	—	—	pF

Figure I.16.

2. Capacité mémoire totale adressable.

Indiquer la **capacité mémoire totale** (exprimée en Kilo octets) **adressable** par ce microcontrôleur, en supposant que chaque case mémoire a une taille de 8 bits.

Éléments de correction :

Le bus d'adresses a une taille de 16 bits (de A_0 à A_{15}) donc il y aura combinaisons différentes d'adresses. Soit exactement adresses différentes, cette valeur étant arrondie couramment à Koctets.

(car chaque adresse correspond à 1 octet : le data bus est sur bits).

3. Repérage des composants.

Identifier les composants **Ui** (hormis le microprocesseur) et remplir le tableau ci-dessous qui résume leurs principales propriétés et caractéristiques.

Éléments de correction :

Composant	Type	Description essentielle
U3	EPROM	Conserve son contenu même lorsqu'on coupe sa tension d'alimentation (non volatilité) Accès aléatoire par adresse. Lecture par $\overline{RD}$ au niveau bas La sélection se fait par l'entrée $\overline{CS}$ (au niveau bas) Elle se programme électriquement et s'efface aux ultraviolets (contrairement aux EEPROM qui s'effacent électriquement) Elle sert le plus souvent à stocker le code.
U1 et U2	A compléter	A compléter
U4	A compléter.	A compléter

4. Capacité mémoire réellement implantée.

Donner les **capacités** (exprimée en Kilo octets) des 3 mémoires utilisées (**EPROM**, **RAM1** et **RAM2**).

Eléments de correction :

EPROM U3 : 11 lignes du bus d'adresses (de A_0 à A_{10}) donc octets dans la mémoire, soit exactement octets, valeur arrondie couramment à Kioctets.

RAM U1 : A compléter

RAM U2 : A compléter

5. Rôle du signal $\overline{AS}$.

5.1. Expliquer alors pour quelle raison faut-il utiliser la sortie $\overline{AS}$ du microcontrôleur pour valider les signaux $\overline{CS}$ des mémoires ?

Aide : sur un bus d'adresses, les signaux A_i changent constamment d'état au rythme du déroulement du programme en cours d'exécution. Or, le **74HC42**, ne possède pas d'entrée de validation.

Eléments de correction :

Si on ne valide pas les signaux de sortie $\overline{Y}_i$ du décodeur 74HC42 par le signal $\overline{AS}$, il risque d'y avoir des décodages « intempestifs » (lorsque le bus d'adresses change d'état), ce qui peut entraîner des sélections aléatoires des mémoires à des instants non souhaités.

Il faut donc utiliser la sortie $\overline{AS}$ du microcontrôleur (active au niveau bas) pour valider le décodage d'adresses. La sortie $\overline{Y}_i$ (du 74HC42) ne sera considérée comme valable que si $\overline{AS} = 0$.

Si $\overline{AS} = 1$ bus d'adresses non stable, en changement d'état), il ne faut alors décoder aucune mémoire (tous les $\overline{CS}_i$ à 1).

D'où la table de vérité suivante, donnant $\overline{CS}_i$: signal de sélection d'une mémoire en fonction de $\overline{Y}_i$: sortie du décodeur correspondante et de $\overline{AS}$ (à compléter).

$\overline{Y}_i$	0	1
$\overline{AS}$		
0		...
1	...	...

Et donc (à compléter) l'équation logique suivante donnant les $\overline{CS}_i$ en fonction des $\overline{Y}_i$ et de $\overline{AS}$:

$\overline{CS}_i = \dots\dots\dots$

5.2. Proposer une solution à base de **portes logiques** de votre choix pour effectuer cette validation et réaliser les équations logiques précédentes :

Eléments de correction (et qui vous permet de vérifier votre réponse à la question précédente !!!) :

Il faudra donc 3 portes OU (1 pour chaque mémoire) pour valider le décodage d'adresses, comme sur le schéma de principe de la figure I.17. ci-dessous :

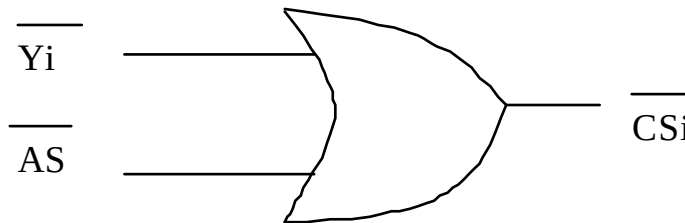


Figure I.17.

6. Plage d'adresses de l'EPROM.

6.1. Indiquer la plage d'adresses de l' **EPROM**. Donner toutes les explications utiles pour calculer cette plage (notamment en utilisant la **table de vérité** du **74HC42**).

Eléments de correction :

Pour calculer la plage d'adresses de l'EPROM U3, il faut chercher pour quelles combinaisons d'adresses on a son entrée de sélection $\overline{CS}_{EPROM} = 0$.

D'après le schéma, on voit que :

$\overline{CS}_{EPROM} = 0$ si $\overline{AS} = 0$: normal pour avoir le bus d'adresses stable

et

$\overline{Y9} = 0$

D'après la table de vérité du 74HC42, on doit avoir :

	D	C	B	A	
	1	0	0	1	($\equiv (9)_{10}$)
Soit :	A15	A14	OK	OK	B relié à la masse et A au 5V.

Ce qui donne finalement toutes les combinaisons binaires d'adresses suivantes comprises entre Min et Max définis ci-dessous :

	A15	A14	A13	A12	A11	A10.....	A1	A0
Min	1	0	X	X	X	0	0	0
Max	1	0	X	X	X	1	1	1

avec **X = indifférent = 0 ou 1.**

Donc en prenant X X X = 0 0 0 on obtient la plage comprise entre :

(10000...00)₂ = 0x8000 et (100001...11)₂ = 0x87FF

soit 2 Koctets correspondant bien à la taille de l'EPROM vue précédemment.

8. On complète le schéma ...

8.1. Compléter (par des liaisons en utilisant les résultats précédents) le schéma de la figure I.15. en y intégrant :

- les signaux de lecture et d'écriture des 3 mémoires (les $\overline{RD}$ et les $\overline{WE}$),
- la gestion des entrées de validation $\overline{CS}$, actives au niveau bas, des 3 mémoires.

A compléter sur le schéma.....

8.2. . Entourer de façon **visible** sur votre schéma la partie décodage d'adresses ainsi réalisée.

A compléter sur le schéma.....

9. Synthèse : le mapping de cette carte ...

Normalement, si vous avez répondu aux questions, vous devez obtenir le mapping mémoire de la figure I.18. pour cette carte :

Taille		Adresses en hexa
1K	RAM2 U2 <u>Normal</u>	0x0000
		0x03FF
1K	RAM2 U2 Ghost 1	0x0400
		0x07FF
1K	RAM2 U2 Ghost 2	0x0800
		0x0BFF
12K	RAM2 U2 Ghost 3 à 14	
		0x3CFF
1K	RAM2 U2 Ghost 15	0x3FFF
		0x4000
4K	RAM1 U1 <u>Normal</u>	0x4FFF
		0x5000
4K	RAM1 U1 Ghost 1	0x5FFF
		0x6000
4K	RAM1 U1 Ghost 2	0x6FFF
		0x7000
4K	RAM1 U1 Ghost 3	0x7FFF
		0x8000
2K	EPROM U3 <u>Normal</u>	0x87FF
		0x8800
2K	EPROM U3 Ghost 1	0x8FFF
		0x9000
2K	EPROM U3 Ghost 2	0x97FF
		0x9BFF
8K	EPROM U3 Ghost 3 à 6	
		0xB800
2K	EPROM U3 Ghost 7	0xBFFF
		0xC000
16K	Libre	0xFFFF

Figure I.18.

I.2.7. VERS UNE ARCHITECTURE TYPE HARVARD

Comme on l'a vu, une ROM contient le code, c'est-à-dire l'ensemble des instructions élémentaires (voir chapitre suivant et notamment la figure I.22.) constituant le programme. Une RAM quant à elle, contient les données, c'est-à-dire les valeurs des variables sur lesquelles le programme va agir. Les contenus de la ROM et de la RAM sont donc de nature complètement différentes (plus précisément complémentaires).

Dans le cas de l'architecture de Von Neumann présentée sur les figures I.11. ou I.15. précédentes, il n'y a :

- qu'un seul bus d'adresses **BA** : commun à la mémoire contenant le programme (en général la ROM) et à la mémoire contenant les données (en général la RAM),
- qu'un seul bus de données **BD** : commun à la mémoire contenant le programme (en général la ROM) et à la mémoire contenant les données (en général la RAM).

Cette architecture présente des inconvénients car elle nécessite à chaque cycle d'accès mémoire en lecture de bien différencier le contenu des cases mémoire contenant le code (en fait le code machine de chaque instruction) et le contenu des cases mémoire contenant les opérandes utilisés par ces instructions (en fait les valeurs des variables), comme indiqué sur la partie nommée « Aiguillage Instructions/Données » de la figure I.19.

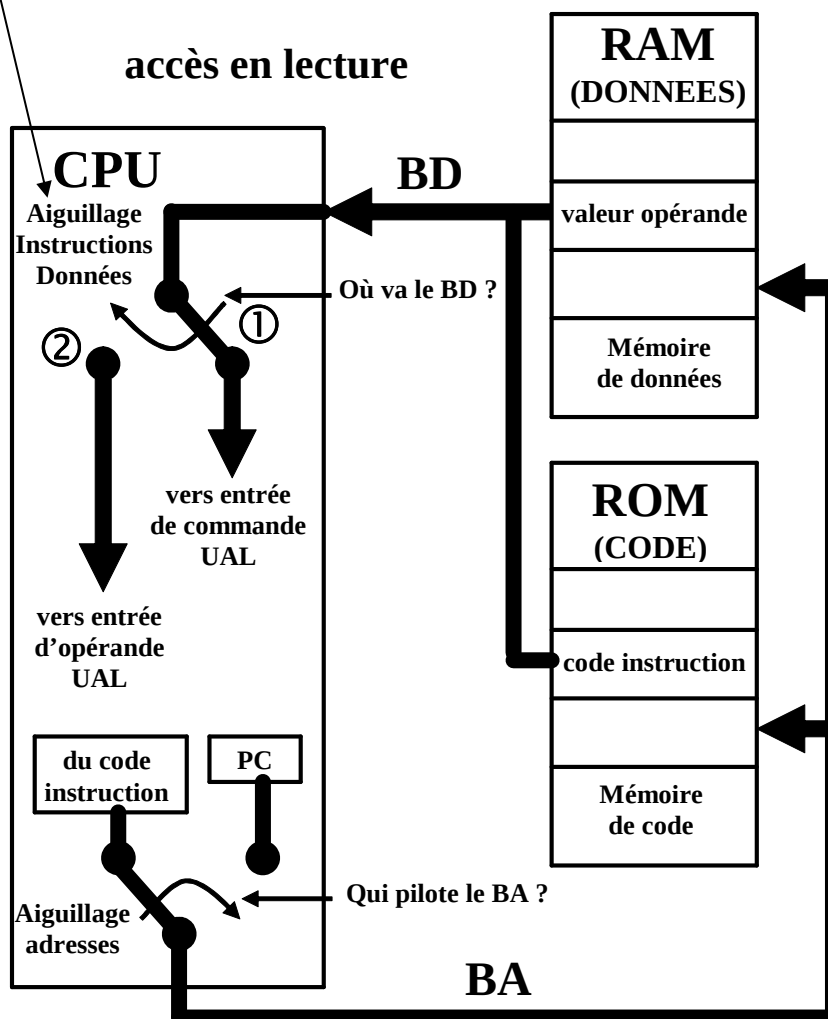


Figure I.19.

En effet, les contenus de la ROM et de la RAM ne sont pas gérés au niveau de la CPU de la même façon. Pour comprendre cela, il faut « rentrer » à l'intérieur de la CPU. Au cœur de cette dernière, il existe une **Unité Arithmétique et Logique (U.A.L.)** chargée d'effectuer tous les traitements élémentaires (addition/soustraction, incrémentation/décrémentation, opérations logiques...) sur les opérandes qui peuvent être des registres internes de la CPU ou des valeurs de variables contenues en mémoire RAM.

Cette **UAL** peut être représentée formellement par le schéma de la figure I.20.

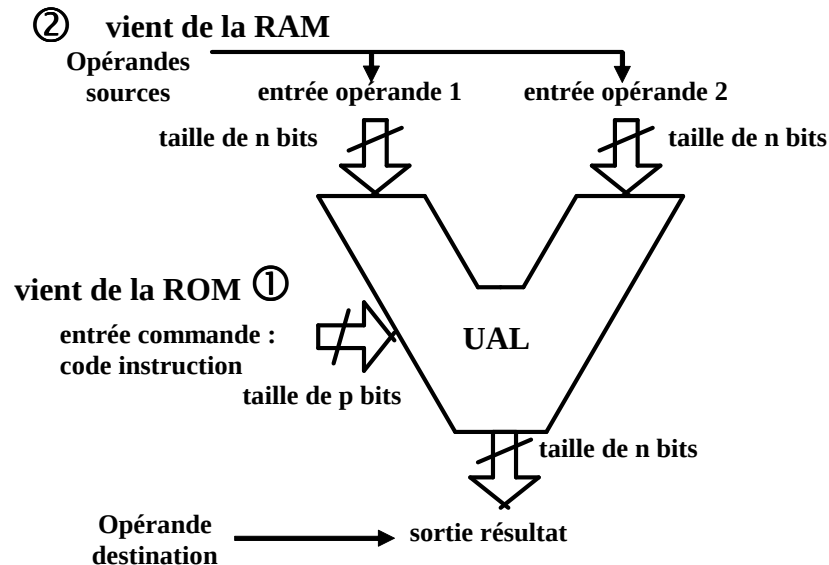


Figure I.20.

Comme on peut le voir sur le schéma de la figure I.19., le code machine des instructions constitue **l'entrée de commande** de l'UAL ① (pour lui indiquer quelle opération effectuer) et la valeur de la variable correspond à une des entrées opérande source de l'UAL ②.

Ce qui nécessite d'effectuer un démultiplexage temporel par aiguillage du bus de données **BD** à l'intérieur de la CPU (position ① vers l'entrée de commande de l'UAL et position ② vers une entrée opérande de l'UAL).

On pourrait montrer qu'un aiguillage est également nécessaire à l'intérieur de la CPU sur le bus d'adresses **BA** pour accéder à la case mémoire ROM contenant le code de l'instruction suivante en mémoire (voir le compteur de programme **PC** dans le chapitre suivant) et pour accéder à la case mémoire RAM contenant la valeur de la variable concernée par l'instruction en cours (qui est codée dans le code machine de cette instruction : voir également le séquençement d'une instruction dans le chapitre suivant).

Au final, ce type d'architecture n'est pas le mieux adapté pour obtenir un fonctionnement optimal.

Une solution, connue sous le nom d'architecture de type **HARVARD**, consiste à **séparer cette fois-ci complètement** la mémoire de code (qui contient toujours les instructions du programme) et la mémoire de données (qui contient toujours les opérandes de ces instructions, c'est-à-dire les valeurs des variables du programme) en leur associant à chacune leur propre bus d'adresses et leur propre bus de données.

Ainsi, comme le montre le schéma de la figure I.21., il y aura au total **4 bus indépendants** :

- un bus **BAC** : **B**us d'**A**dresses pour le **C**ode,
- un bus **BAD** : **B**us d'**A**dresses pour les **D**onnées,
- un bus **BDC** : **B**us de **D**onnées pour le **C**ode,
- un bus **BDD** : **B**us de **D**onnées pour les **D**onnées.

(sans oublier que dans le cas d'un microcontrôleur cette architecture est entièrement intégrée dans le boîtier même du microcontrôleur).

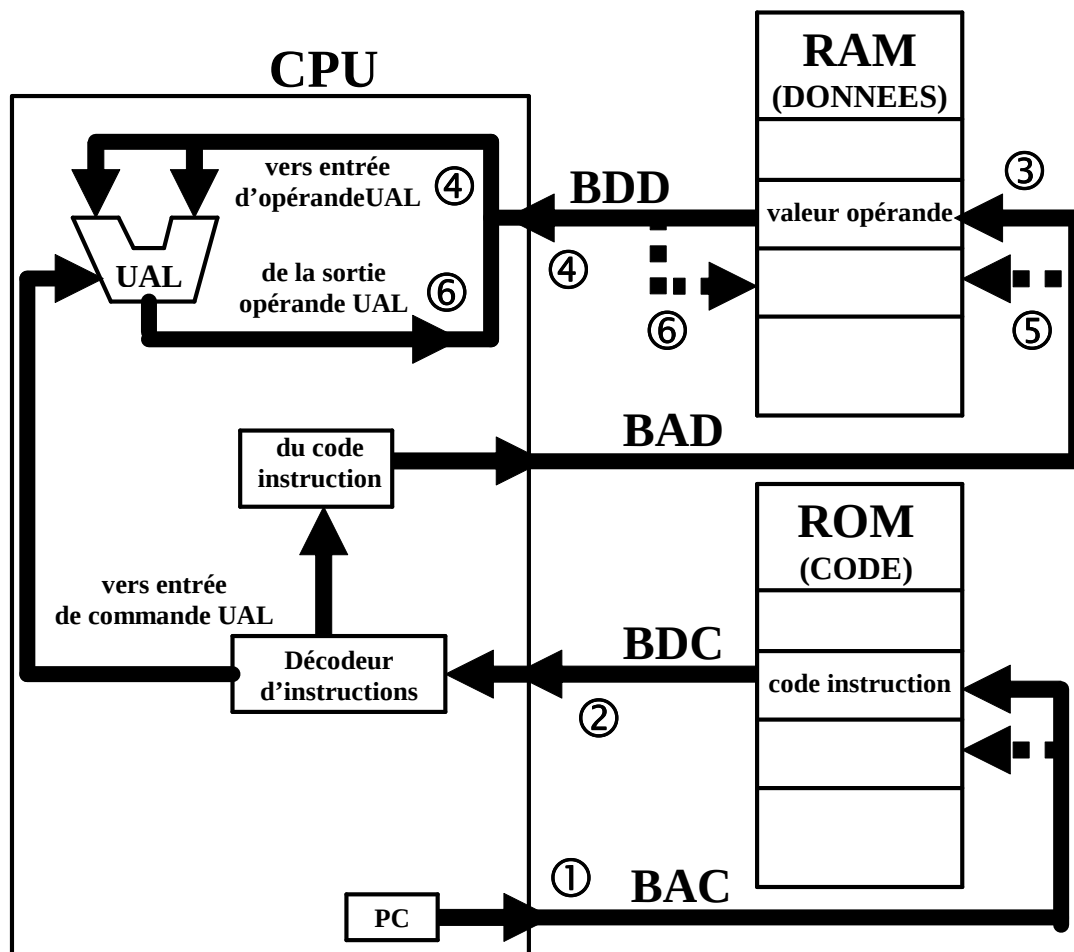


Figure I.21.

Les aiguillages internes à la CPU ont maintenant disparu et le fonctionnement s'en trouve accéléré.

Les mémoires de code et de données ont été complètement découplées.

Ce type d'architecture permet également l'implantation d'une structure de type « pipe-line » à 2 niveaux.

Qu'est-ce que c'est ? Nous allons en parler dans le chapitre suivant.

I.2.8. SEQUENCEMENT D'UNE INSTRUCTION ET NOTION DE PIPE

Quelque soit le type de micro/processeur/contrôleur, il existe un registre spécial qu'on appellera ici le **PROGRAM COUNTER (PC)** mais qui est parfois aussi connu sous le nom d'INSTRUCTION POINTER ou de COMPTEUR ORDINAL.

Son rôle est fondamental pour assurer le déroulement d'un programme.

**Le PC contient une adresse (c'est un pointeur) et cette adresse est celle de la case mémoire contenant le code de la prochaine instruction à exécuter.
Il pointe donc vers la mémoire de code.**

En fait, un programme n'est rien d'autre qu'une suite d'instructions rangées en mémoire de code dans des cases mémoires contiguës (qui se suivent) comme indiqué sur la figure I.22.

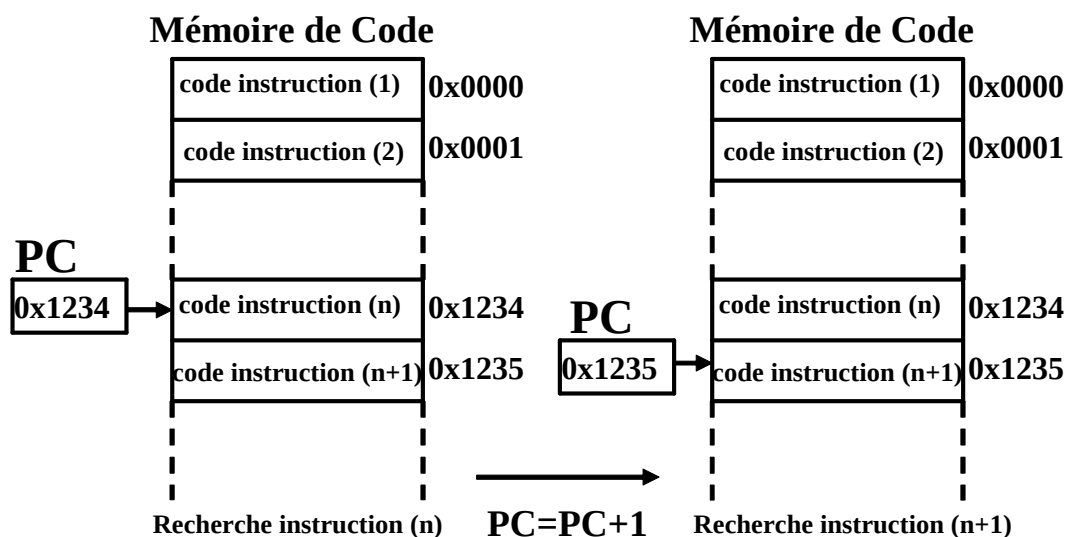


Figure I.22.

Le rôle du PC est donc d'indiquer à tout moment où se trouve la prochaine instruction à exécuter.

Il joue ainsi le rôle de séquenceur (ou numérateur) d'instructions.

Le principe de séquençage des instructions est assez général et fonctionne, dans un cas assez général, comme un moteur à 4 temps (ou étapes).

La chronologie ①, ②, ③ ... qui apparaît dans les explications suivantes est indiquée également sur le schéma de la figure I.21. ce qui permet, une fois de plus, de faire le lien entre le hardware (l'architecture du système) et le software (l'exécution du programme).

- **étape 1** : recherche de l'instruction en mémoire de code (**INSTRUCTION FETCH**).

La CPU sort sur le **BAC** la valeur du **PC** (①). La case de la mémoire de code qui est sélectionnée, et elle seule (cf partie précédente sur le décodage d'adresses) va fournir en retour sur le **BDC** (②) le code machine de l'instruction (n) qui sera lu par la CPU.

Dans la majorité des cas (déroulement séquentiel du programme où la prochaine instruction (n+1) à exécuter se trouve dans la case mémoire suivante) et suite à cette phase, le PC est incrémenté automatiquement de 1 (**PC=PC+1**) pour pointer ensuite sur l'instruction (n+1) comme indiqué sur la figure I.22. (on verra plus loin que dans certains cas, appelés rupture synchrone de séquence, la nouvelle valeur du PC peut être différente de PC+1).

- **étape 2** : décodage de l'instruction dans la CPU (**INSTRUCTION DECODE**).

Un bloc, interne à la CPU, et parfois appelé **DECODEUR D'INSTRUCTIONS** récupère dans le code machine toutes les informations nécessaires à cette instruction comme :

- le type d'opération à effectuer : est-ce une addition, une soustraction, une autre opération logique... ? : il s'agit de préparer l'entrée de commande de l'U.A.L.
- la valeur des opérandes (dans la CPU ou en mémoire de données) sur lesquels agira cette opération (préparer les entrées opérandes sources de l'U.A.L.). Lorsque l'opérande est en mémoire de données, celui-ci sera repéré dans le code instruction par son adresse, qui sera fournie sur le **BAD** (③). La case de la mémoire de donnée sélectionnée (et elle seule) fournira en retour sur le **BDD** (④) la valeur de cet opérande (cycle de lecture d'une variable dans la mémoire de données).
- où se situe (dans la CPU ou en mémoire de données) le registre ou la case mémoire dans lequel sera rangé le résultat de cette opération (préparer le rangement de la sortie opérande destination de l'U.A.L.).

- **étape 3** : exécution de l'instruction dans l'U.A.L. (**INSTRUCTION EXECUTE 1**).

Cette phase se déroule entièrement dans l'U.A.L. et ne nécessite aucun accès hors de la CPU.

- **étape 4** : rangement de l'opérande destination (**INSTRUCTION EXECUTE 2**).

Cette 4^{ème} étape n'existe pas pour toutes les instructions. Mais quand c'est le cas, c'est-à-dire lorsque l'opérande destination doit être écrit en mémoire de donnée (mise à jour d'une variable), l'adresse de destination doit être fournie sur le **BAD** (⑤) et la valeur de cette variable doit être placée sur le **BDD** (⑥) (cycle d'écriture d'une variable dans la mémoire de données).

Les chronogrammes correspondant à ces différentes étapes sont fournis sur la figure I.23.

L'horloge système $F_{OSC} = \frac{1}{T_{OSC}}$ (qui est souvent celle fournie par un quartz à 4 ou 20 MHz) est en

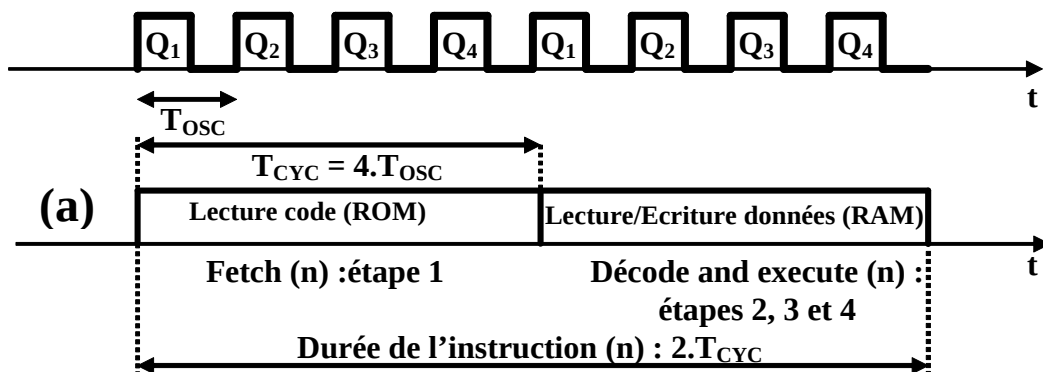
fait divisée en interne par 4, ce qui correspond à une période $T_{CYC} = 4.T_{OSC}$.

Cette division est nécessaire pour réaliser les opérations internes correspondant à chacune des 4 étapes vues précédemment.

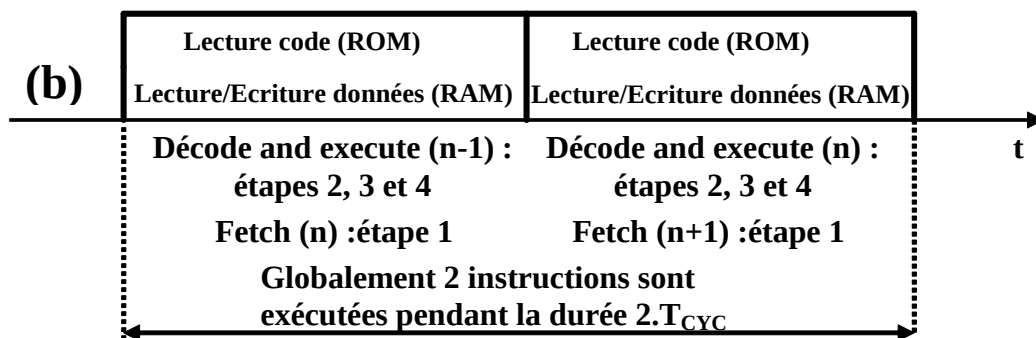
Mais attention !!! Ce n'est pas une période du quartz par étape : l'étape 1 dure 4 périodes de quartz, et les étapes 2,3 et 4 durent ensemble aussi 4 périodes du quartz.

Remarque : il faut 4 « coups d'horloge » (Q1 à Q4) pour un accès mémoire

Quartz de fréquence F_{OSC} et de période $T_{OSC} = 1/F_{OSC}$



ARCHITECTURE VON NEUMANN
Fonctionnement en séquentiel



ARCHITECTURE HARVARD
Fonctionnement en parallèle : « pipe » fois 2

Figure I.23.

Sur une architecture de Type Von Neuman, l'existence d'un seul bus d'adresses et d'un seul bus de données constituait une espèce de « goulot d'étranglement » car ces 4 étapes devaient s'exécuter de façon séquentielle, les unes après les autres (par exemple, pas de possibilité d'accéder en même temps à la mémoire pour y lire un code instruction et pour y lire un opérande/écrire un résultat). Globalement, cela augmentait la durée complète d'exécution d'une instruction, comme on peut le voir sur la figure I.23.(a.). Le cycle instruction vaut alors $2.T_{CYC}$, ce qui correspondrait par exemple à $2 \mu s$ pour un quartz à 4 MHz.

Mais dans le cas d'une architecture Harvard, certaines de ces phases peuvent être effectuées en parallèle (en même temps) comme indiqué sur la figure I.23.(b.) où la phase de recherche de l'instruction (n) aura lieu en même temps que le décodage et l'exécution de l'instruction précédente (n-1).

T_{CYC} représentera bien dans ce cas la durée totale d'une instruction, soit par exemple 1 μs pour un quartz à 4 MHz (soit une vitesse de 1Mips =1 Million d'instructions par seconde). Et un gain d'un facteur 2 par rapport au cas précédent.

On dit que dans ce cas on a un pipe fois 2.

Ce qui peut se représenter par

la figure I.24., directement issue de la documentation constructeur.

EXAMPLE 3-1: INSTRUCTION PIPELINE FLOW

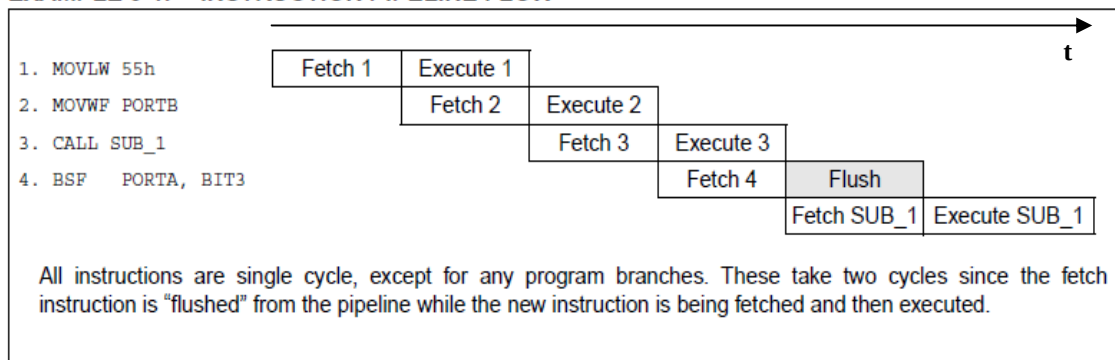


Figure I.24.

Les lignes 1 à 4 représentent 4 instructions assembleur. Pour l'instant, peu importe ce qu'elles font (elles seront détaillées plus tard dans ce cours). Ce qu'il est important de voir et de comprendre, ce sont les informations écrites sur la droite de chacune d'entre elles. Par exemple, pendant que le PIC recherche l'instruction 2 en mémoire de code (Fetch 2), il exécute l'instruction 1 (Execute 1). Ce qui est conforme avec ce que l'on a dit précédemment. Et idem pour les suivantes.

Remarque :

Le cas de l'instruction 3 représente une rupture de séquence (détaillée plus loin). Dans ce cas, l'instruction suivante qui doit être exécutée n'est plus située dans la case mémoire suivante, mais à une autre adresse en mémoire (ici c'est l'appel d'un sous-programme par l'instruction CALL). Dans ce cas là, il faut faire une « vidange » du pipe (Flush), pour ne pas exécuter l'instruction 4 qui avait été recherchée en mémoire par Fetch 4. On perd alors ponctuellement 1 cycle instruction. Les instructions de rupture de séquence s'exécutent en deux cycles au lieu d'un.

Au final, on retiendra que pour les PIC, chaque instruction dure : $T_{CYC} = \frac{4}{F_{OSC}}$.

(sauf pour les ruptures de séquence qui durent : $2.T_{CYC} = \frac{8}{F_{OSC}}$).

Ce qui donne une vitesse de traitement de 1 Mips pour un quartz système à 4 MHz et de 5 Mips pour un quartz à 20 MHz.

Le séquençement des instructions fonctionne comme « un moteur à deux temps imbriqués »

I.2.9. STRUCTURE CISC OU RISC

Un dernier élément intervient aussi dans la façon dont la CPU (ou plus exactement son ALU) est structurée en interne.

Il existe deux grands procédés qui possèdent chacun leurs avantages et leurs inconvénients :

- la technologie **CISC** (**C**omplex **I**nstruction **S**et **C**omputer) : micro/processeur/contrôleur à jeu d'instructions complexe,
- la technologie **RISC** (**R**educed **I**nstruction **S**et **C**omputer) : micro/processeur/contrôleur à jeu d'instructions réduit.

Qu'est-ce que cela signifie ?

Tout d'abord, il faut répondre aux deux questions suivantes :

Qu'est-ce qu'une instruction élémentaire ? Qu'entend-on par jeu d'instructions ?

Une instruction élémentaire est la plus petite tâche que peut effectuer une CPU, comme un simple transfert entre un de ses registres internes vers la mémoire, une simple addition ou une opération logique entre deux registres ...

Attention : ne pas confondre la notion de fonction (comme elle existe en C par exemple) avec celle d'instructions élémentaires dont on parle ici: une fonction comme **printf** est peut-être constituée de plusieurs centaines (voire plus) d'instructions élémentaires. Même le simple `i++` du C, peut correspondre à quelques instructions élémentaires de la CPU.

Une image : l'instruction élémentaire est « l'atome » de base d'un programme, le grain de sable qui permettra de faire des briques (les fonctions de base) puis des murs (les fonctions plus complexes utilisant (imbriquant) ces fonctions de base) et enfin des maisons (les programmes) ou des villes...

Ainsi, chaque CPU sera caractérisée par son **propre jeu d'instructions** :

qu'on appelle aussi **INSTRUCTION SET**.

Le jeu d'instructions est la liste exhaustive de toutes les instructions élémentaires que sait effectuer cette CPU.

Dans le cas d'une technologie **CISC**, la CPU possède un jeu d'instructions assez important, pouvant aller par exemple jusqu'à plusieurs centaines (par exemple environ 150 pour le microcontrôleur **HC11** de chez Motorola). Parmi celles-ci, certaines sont simples (et s'exécute en un seul cycle). Mais d'autres, plus complexes comme la multiplication ou la division par exemple vont nécessiter plusieurs cycles pour s'exécuter.

Dans le cas d'une technologie **RISC**, la CPU possède un ensemble d'instructions réduit, avec par exemple 35 instructions de base pour le **PIC 16F877**. Et chacune de ces instructions (hormis les ruptures de séquence) s'exécute en un seul cycle comme on l'a dit dans le chapitre précédent.

Comment comparer ces deux technologies ? Par leurs avantages/inconvénients, résumés sur le tableau de la figure I.25. ci-dessous :

Points abordés	CISC	RISC
Jeu d'instructions et programmation en assembleur.	Le nombre important d'instructions donne un choix important mais souvent elles ne sont pas toutes utilisées par le programmeur. La puissance de certaines instructions complexes entraîne des programmes plus courts.	Moins d'instructions à disposition obligeant à plus d'ingéniosité dans la programmation. Les tâches plus complexes sont réalisées en enchaînant des instructions simples. Les programmes occupent plus de place en mémoire.
Taille d'une instruction en mémoire.	Elle dépend du type instruction.	En général de taille fixe et souvent une seule case mémoire par instruction
Durée d'exécution des instructions.	Variable en fonction des instructions ce qui peut poser, dans certains cas critique, des problèmes de temps réel.	Toujours un cycle par instruction ce qui peut être plus performant sur du temps réel.
Nombre de mode d'adressage. (techniques pour accéder aux variables)	En général assez important. Ce qui simplifie les accès aux structures de données complexes.	Plus réduit.
Programmation en langage évolué. (C, C++, Java)	Le compilateur est plus simple car il peut utiliser toute la puissance de certaines instructions.	Compilateur plus complexe : c'est « l'intelligence » du compilateur qui doit compenser le manque d'instructions complexes de la CPU. On dit alors que toute la puissance du traitement se trouve dans le compilateur.

Figure I.25.

En résumé :

**Les microcontrôleurs PIC possèdent en interne une architecture de type HARVARD.
Ils fonctionnent avec une structure de type pipe-line fois 2.
Ils sont basés sur une technologie RISC.**

I.2.10. CODAGE ASCII

En plus des codages binaire et hexadécimal qui servent à travailler avec les nombres, il existe un autre codage qui permet de travailler avec les caractères : le code **ASCII** (**A**merican **S**tandard **C**ode for **I**nformation **I**nterchange).

Ce code permet de traiter les caractères imprimables comme les lettres (majuscules et minuscules), les chiffres, les caractères de ponctuation et certains caractères de contrôle. Certains terminaux dits « alphanumériques » ou en mode texte (afficheur, écran, console, imprimante) sont parfaitement compatibles avec ce format de codage.

Ce code ASCII, sur 8 bits n'est réellement normalisé que sur les 7 bits de poids faible, le **MSB** valant 0, ce qui donne au total une normalisation pour les 128 premiers caractères.

Pour les 128 restants (avec le **MSB** valant 1), cela dépend des spécificités de chaque terminal.

La page suivante (cf. figure I.27.) fournit le code ASCII des 128 caractères normalisés.

Chaque élément de ce tableau se lit de la façon suivante (cf. figure I.26.) :

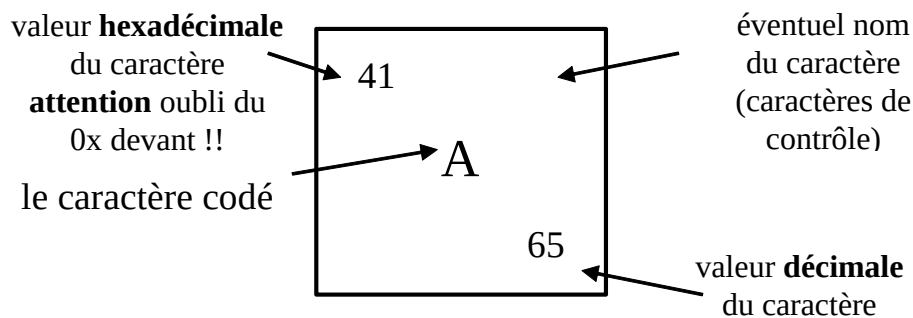


Figure I.26.

On reconnaît sur les colonnes **3 à 8** du tableau de la figure I.27. les principaux caractères imprimables.

Attention pour le codage des chiffres :

Ne pas confondre $0x32 = '2'$ (code ASCII du 2 en hexadécimal) avec $0x02 = 2$ (la valeur 2).
Pour afficher 2 sur un terminal en mode texte, il faut lui envoyer $0x32$ et non $0x02$!!!
Par contre, pour faire une addition avec le chiffre 2, il faut utiliser $0x02$ et non $0x32$!!!
Tout dépend donc du contexte d'utilisation !!!

Les colonnes **1 et 2** de la figure I.27. (32 premières valeurs) correspondent à des caractères de contrôle non directement affichables ou imprimables mais qui serviront dans certains protocoles de communication ou permettront par exemple de gérer un curseur sur un terminal comme pour :

- sauter à la ligne suivante (**LF** = $0x0A$: **L**ine **F**eed),
- revenir en début de ligne (**CR** = $0x0D$: **C**arriage **R**eturn),
- ou faire « beeper » un écran (**BEL** = $0x07$) ...

TABLE DES CODES ASCII POUR LES 128 PREMIERS CARACTERES (MSB = 0)

Colonnes 1 et 2

Colonnes 3 à 8

00 NUL	10 DLE	20	30	40	50	60	70
^@ 0	^P 16	32	0	@ 64	P 80	` 96	p 112
01 SOH	11 DC1	21	31	41	51	61	71
^A 1	^Q 17	33	1	A 65	Q 81	a 97	q 113
02 STX	12 DC2	22	32	42	52	62	72
^B 2	^R 18	34	2	B 66	R 82	b 98	r 114
03 ETX	13 DC3	23	33	43	53	63	73
^C 3	^S 19	35	3	C 67	S 83	c 99	s 115
04 EOT	14 DC4	24	34	44	54	64	74
^D 4	^T 20	36	4	D 68	T 84	d 100	t 116
05 ENQ	15 NAK	25	35	45	55	65	75
^E 5	^U 21	37	5	E 69	U 85	e 101	u 117
06 ACK	16 SYN	26	36	46	56	66	76
^F 6	^V 22	38	6	F 70	V 86	f 102	v 118
07 BEL	17 ETB	27	37	47	57	67	77
^G 7	^W 23	39	7	G 71	W 87	g 103	w 119
08 BS	18 CAN	28	38	48	58	68	78
^H 8	^X 24	40	8	H 72	X 88	h 104	x 120
09 HT	19 EM	29	39	49	59	69	79
^I 9	^Y 25	41	9	I 73	Y 89	i 105	y 121
0A LF	1A SUB	2A	3A	4A	5A	6A	7A
^J 10	^Z 26	42	:	J 74	Z 90	j 106	z 122
0B VT	1B ESC	2B	3B	4B	5B	6B	7B
^K 11	^[27	43	;	K 75	[91	k 107	{ 123
0C FF	1C FS	2C	3C	4C	5C	6C	7C
^L 12	^\ 28	44	<	L 76	\ 92	i 108	! 124
0D CR	1D GS	2D	3D	4D	5D	6D	7D
^M 13	^] 29	45	=	M 77	] 93	m 109	} 125
0E SO	1E RS	2E	3E	4E	5E	6E	7E
^N 14	^^ 30	46	>	N 78	^ 94	n 110	~ 126
0F SI	1F US	2F	3F	4F	5F	6F	7F
^O 15	^_ 31	47	?	O 79	_ 95	o 111	~ 127

Figure I.27.

II. ARCHITECTURE INTERNE ET « EXTERNE » DU PIC 16F877

II.1. LA FAMILLE PIC CHEZ MICROCHIP

La référence d'un PIC s'écrit généralement sous la forme d'une liste de quatre symboles du type : **PIC n R p - q**

Par exemple, pour le PIC étudié ici (**PIC 16 F 877A-04**) : n = 16, R = F, p = 877A et q = 04.

Que représentent ces symboles ?

- **n : type de famille de PIC:**

Chez MICROCHIP, il existe 3 grandes familles de PIC, classées par type d'architecture plus ou moins simplifiée et aussi par la taille mémoire de chaque instruction (12 bits, 14 bits ou 16 bits) :

- les « bas de gamme » (famille **Base-Line**) : n = 10 ou 12,
- les « milieux de gamme » (famille **Mid-Range**) : n = 14 ou 16,
- les « hauts de gamme » (famille **High-End**) : n = 18.

- **R : type de mémoire ROM implantée dans le PIC :**

- mémoire ROM **OTP** (**O**ne **T**ime **P**rogrammable) : R = C,
- mémoire EEPROM **FLASH** : R = F,

- **p : en lien avec les caractéristiques du PIC :**

- taille de la mémoire ROM de programme,
- nombre de broches d'I/O (**I**nput/**O**utput ou entrées/Sorties),
- type d'interfaces implantées...

- **q: fréquence maximum de l'horloge système en MHz.**

Il faut noter que les PIC sont des composants dits « **statiques** » car ils peuvent fonctionner avec des fréquences plus faibles que leur fréquence maximum (sans dysfonctionnement).

Une version **-20** (MHz) peut donc fonctionner à 4 MHz, mais pas l'inverse naturellement.

Ce que nous allons présenter dans la suite de ce document concerne le **PIC 16F877A**, mais beaucoup de ces éléments, s'ils ont bien été compris et assimilés pourront être transposés à un autre type de PIC, même s'il n'appartient pas à la même famille.

II.2. LE BROCHAGE EXTERNE DU PIC 16F877

Vu de l'extérieur, et pour un boîtier de type **DIL** (**D**ual **I**n **L**ine), le constructeur fournit sur la **page 3** du datasheet, le brochage de la figure II.1. ci-dessous :

40-Pin PDIP

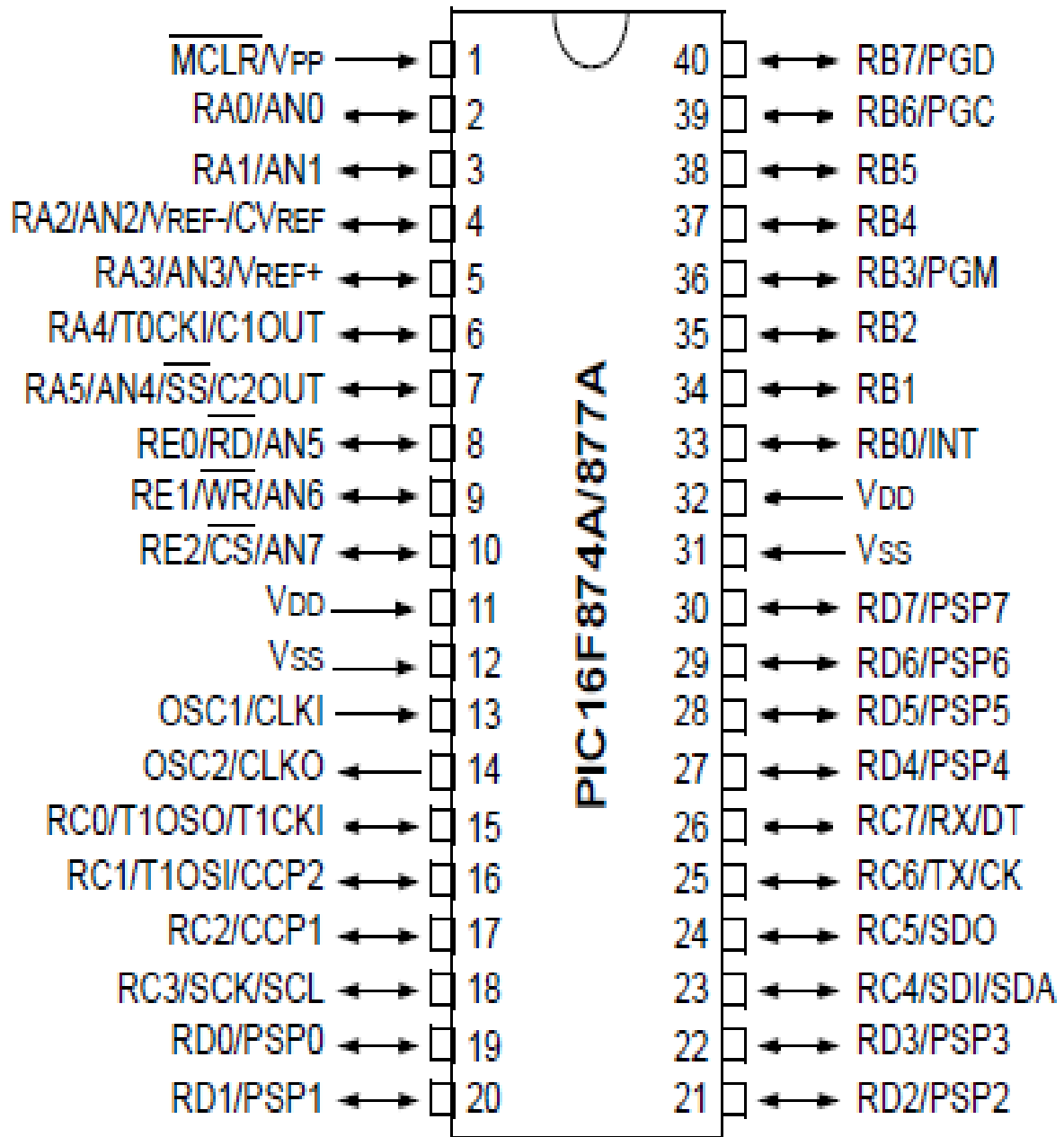


Figure II.1.

Comme on le verra plus loin, la plupart des broches du PIC sont « **multifonctionnelles** » (celles dont le nom de la broche comporte un caractère /) c'est-à-dire qu'elles peuvent servir pour différents périphériques internes, suivant l'application réalisée et la configuration que l'on aura choisie par soft.

Mais, on peut déjà, sur ce brochage, repérer facilement les ports d'entrée/sortie logiques.

Comment sont-ils repérés ?

Par exemple pour le **PORTB** (de taille 8 bits), chacun des bits du registre de données de ce port (registre portant le nom de **PORTB** comme indiqué sur la figure II.2. ci-dessous), est relié à une broche appelée **RB_i**, pour $i=0$ (LSB) à 7 (MSB).

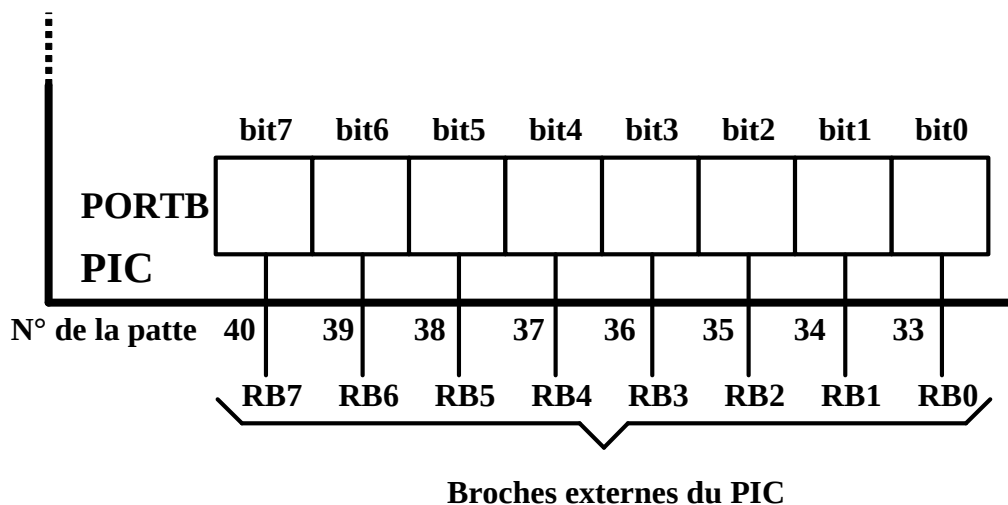


Figure II.2.

Finalement un port se comporte donc comme une case mémoire (son registre de port, comme par exemple **PORTB** sur la figure II.2.) possédant un accès sur l'extérieur (comme par exemple les broches **RB7** à **RB0** sur la figure II.2.).

Ce qui permettra d'échanger des informations avec l'extérieur, c'est-à-dire avec ce qui aura été connecté sur ces broches du PIC !!!

Ici, on a affaire à un port parallèle.

Mais nous reparlerons de tout cela dans la suite de ce polycopié.

II.3. ARCHITECTURE INTERNE DU PIC 16F877

C'est le moment de rentrer à l'intérieur du PIC de façon générale et « grossière » dans cette partie puis un peu plus en détail plus loin dans les chapitres **III** (CPU, mémoires) et **VIII** (interfaces).

Il est intéressant de commencer par donner l'ensemble des blocs fonctionnels présents dans le PIC 16F877, comme indiqué sur la figure II.3. (cf. **page 7** du datasheet constructeur).

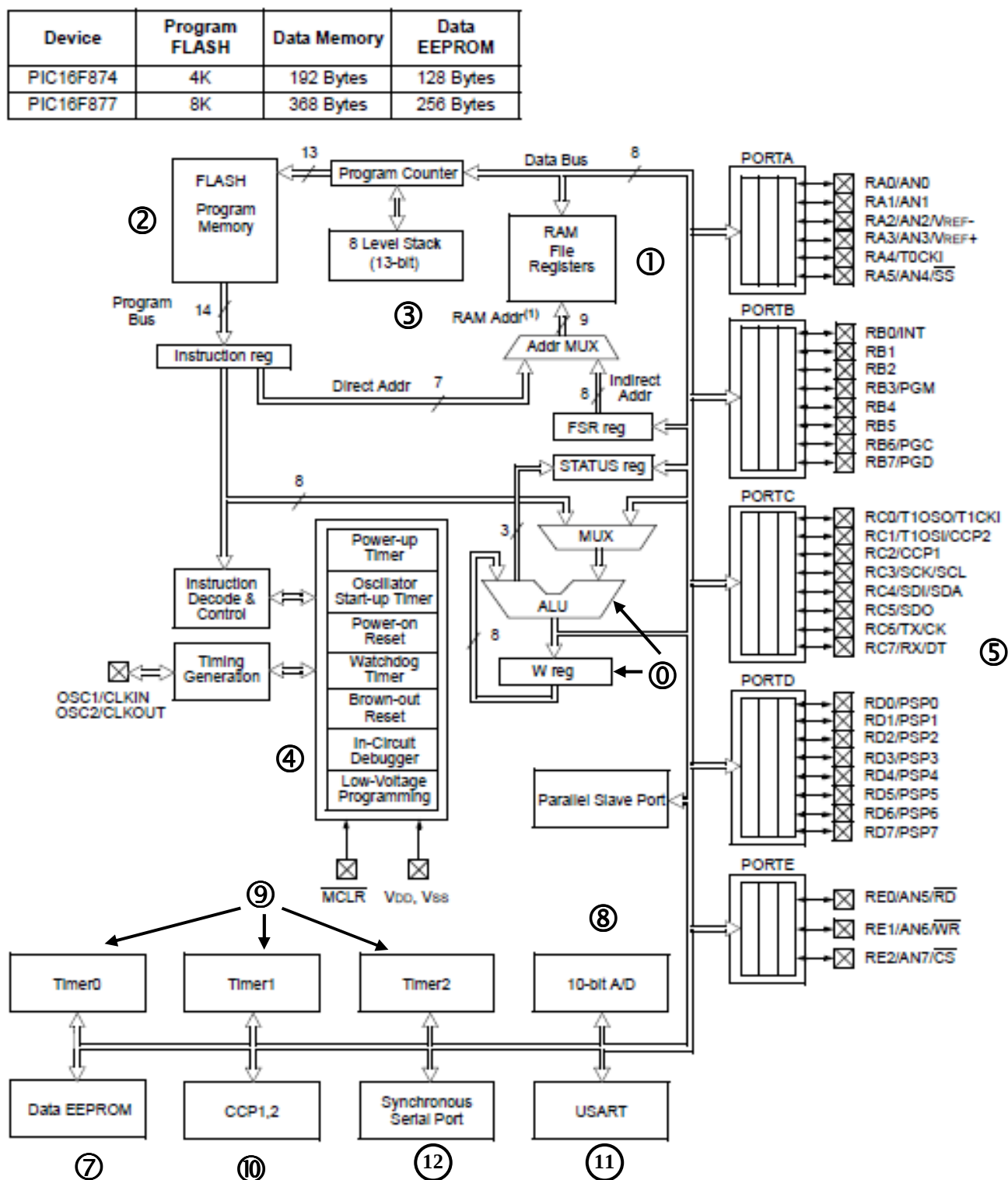


Figure II.3.

Par souci de simplification et de pédagogie, j'ai repris, avec le même repérage (①, ②, ③...) tous ces éléments internes sur le schéma de la figure II.4. ci-dessous :

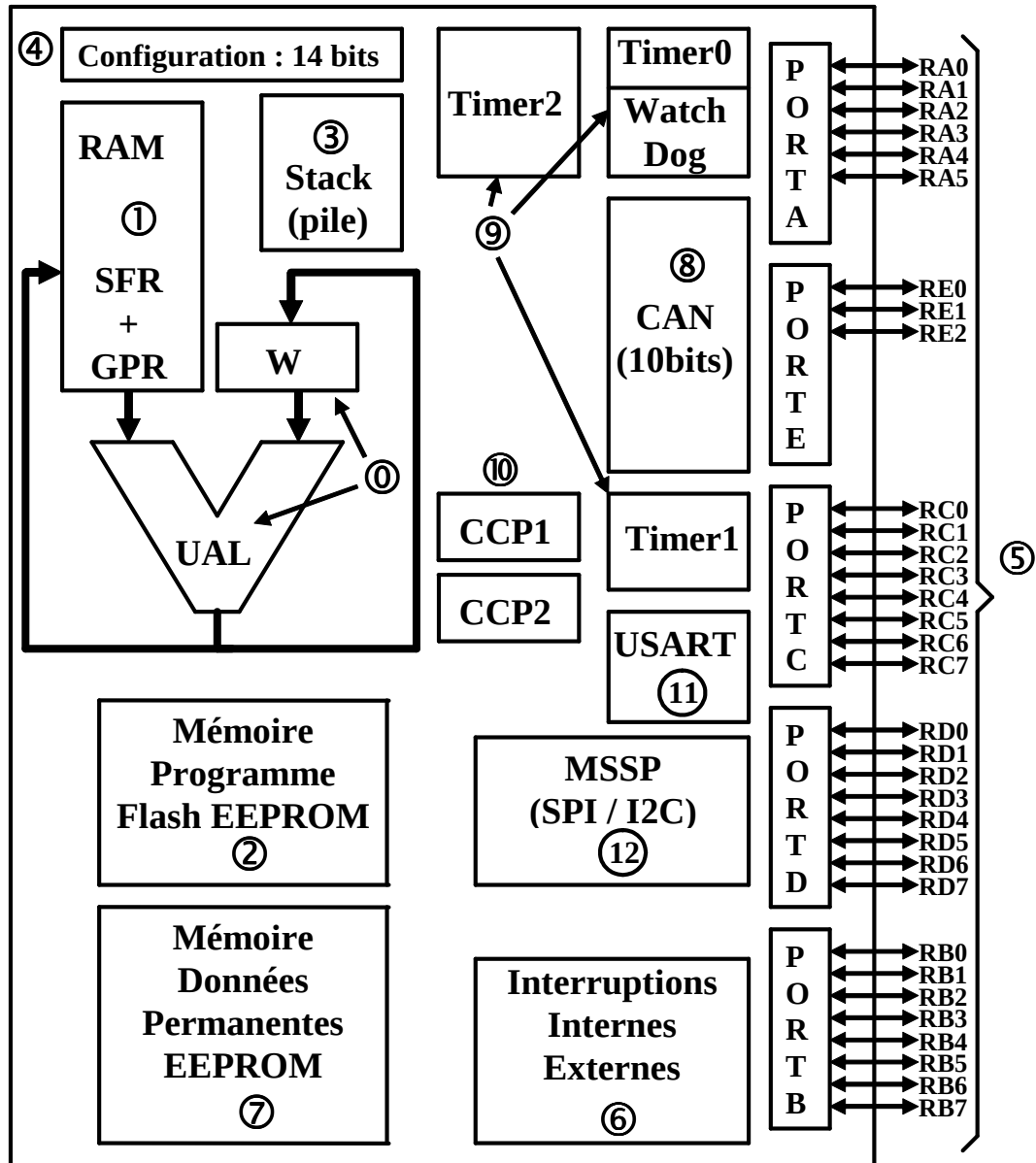


Figure II.4.

Les éléments **essentiels** et les **différents blocs fonctionnels** internes sont repérés sur ces deux figures par les points ⑩ à ⑫.

Nous allons maintenant en donner les principales caractéristiques.

- ① : l'**ALU** et l'accumulateur **W** qui est un registre de travail intervenant beaucoup d'instructions,
- ① : une **RAM interne** de **512** cases de taille 1 octet chacune (chaque case est appelée aussi **register file** par MICROCHIP) séparées en deux parties bien différentes :
 - d'une part **96** octets sont alloués aux registres internes du PIC (appelés **SFR** comme **Special Function Registers**) servant au fonctionnement et à la programmation des interfaces,
 - et d'autre part **416** octets sont réservés à des registres de données (appelés **GPR** comme **General Purpose Registers**) qui serviront de stockage pour les variables des programmes utilisateur,
- ② une **ROM interne** de **8K** cases de taille 14 bits chacune. C'est une **FLASH EEPROM**. Cette mémoire contiendra le programme applicatif qui sera téléchargé et « flashé » sur le PIC lorsqu'il sera entièrement débogué. Mais un programme applicatif peut aussi y écrire, pendant son fonctionnement, pour y stocker des données permanentes ou semi permanentes,
- ③ une **pile (stack)** à ne pas confondre avec une pile électrique (si, si, j'ai vu des étudiants qui faisaient cette erreur !!!) possédant 8 positions mémoire de taille 13 bits chacune permettant donc jusqu'à 8 niveaux d'imbrication de fonctions/interruption,
- ④ **registre de configuration**, pour définir 14 bits qui seront copiés dans l'EEPROM (exactement à l'adresse **0x2007**) lorsque la ROM de code sera programmée. Ce registre sert notamment à fixer des règles de sécurité (comme la protection du code et des données contre le piratage) et de fonctionnement pour le programme implanté,
- ⑤ **33 lignes d'entrée/sorties bidirectionnelles** réparties sur 5 ports parallèle : **PORTA**, **PORTB**, **PORTC**, **PORTD** et **PORTE**,
- ⑥ un module de **gestion des interruptions** extérieures et intérieures (au total **14 sources d'interruption** sont implantables),

- ⑦ une **ROM interne** de **256** octets. C'est aussi une **FLASH EEPROM**. Cette mémoire contiendra les données permanentes (ou semi permanentes) du programme qui ne doivent pas être perdues à chaque coupure d'alimentation (mots de passe, valeurs de configuration...). Elle peut être également flashée au téléchargement avec des valeurs initiales de données permanentes,
- ⑧ un **CAN** (**C**onvertisseur **A**nalogique/**N**umérique) à **huit canaux** avec **10 bits** de précision,
- ⑨ trois **timers/compteurs indépendants** **TIMER0**, **TIMER1** et **TIMER2**, permettant de compter le temps et/ou des événements extérieurs avec gestion intégrée du **WATCH DOG**,
- ⑩ deux modules **CCP1** et **CCP2** pour dater des événements et/ou générer des signaux de type **PWM** par exemple,
- ⑪ un bloc **USART** (**U**niversal **A**synchronous **S**ynchronous **R**eceiver **T**ransmitter) pour les liaisons séries,
- ⑫ un bloc **MSSP** (**M**aster **S**lave **S**erial **P**ort) pour gérer les liaisons série de type **SPI** ou **I²C**.

Nous détaillerons dans la suite de ce polycopié ces différentes parties.

III. CPU, MEMOIRES, PILE ET CONFIGURATION

Cette partie concerne les blocs ①, ②, ③ et ④ des figures II.3. et II.4. précédentes.

III.1. ACCUMULATEUR W ET UNITE ARITHMETIQUE ET LOGIQUE

Le registre **W** (comme **Working Register**) s'appelle un accumulateur. C'est un registre de travail de **taille 8 bits** qui intervient dans beaucoup d'instructions (mais pas dans toutes).

Comme on l'a déjà dit, l'UAL exécute :

- toutes les instructions arithmétiques comme :

- les **additions** : **ADD**,
- les **soustractions** : **SUB**,
- les **incrémentations** : **INC**,
- les **décrémentations** : **DEC**,

- toutes les instructions logiques comme :

- la fonction **et** : **AND**,
- la fonction **ou inclusif** : **IOR**,
- la fonction **ou exclusif** : **XOR**,
- la fonction **inversion** : **COM**,
- le **décalage/rotation** à droite et à gauche : **RR** et **RL** (**Rotate Right** ou **Left**),
- la **permutation** : **SWAP**,
- la **mise à zéro** : **CLR** et **BC** (**Bit Clear**),
- la **mise à un** : **BS** (**Bit Set**).

Toutes ces suites de quelques lettres sont appelées des **mnémonics**.

Ce sont en fait des moyens **mnémotechniques** pour savoir et reconnaître facilement ce que fait chaque instruction.

En plus de ce mnémonic, qui indique **ce que l'ALU doit faire**, chaque instruction doit connaître également **sur quoi doit porter cette opération** (par exemple, indiquer sur quelles cases mémoire ou sur quels registres doit porter l'addition, la soustraction, l'incrément... et où ranger le résultat de cette opération).

Ces informations, venant compléter chaque mnémonic, s'appellent les opérandes **de l'instruction** : opérande **source** (sur quoi porte le mnémonic) et opérande **destination** (pour savoir où ranger le résultat de l'opération réalisée).

En pratique, pour ce type d'instruction (orientée registre ou variable), il peut y avoir :

- soit un seul opérande concerné : dans ce cas là cet opérande peut être :
 - soit le registre **W** (pour la seule instruction **CLRW** : mise à 0x00 de **W**),
 - soit un des registres internes du PIC : un **SFR** (Special Function Register),
 - soit une variable située dans un **GPR** (General Purpose Register)
- soit deux opérandes concernés : dans ces cas là, l'un de ces 2 opérandes sera forcément le registre **W**, l'autre pouvant être :
 - soit un des registres internes du PIC : un **SFR** (Special Function Register),
 - soit une variable située dans un **GPR** (General Purpose Register)
 - soit une constante, appelée alors un **Littéral**.

Ecrire un programme en utilisant ces mnémonics associés à leurs opérandes correspond à ce qu'on appelle la programmation en assembleur, dont on a déjà donné un exemple précédemment sur la figure I.24.

En pratique, ce type d'instruction sera codée sur **14 bits**, qui correspond à la taille de chaque case de la ROM interne de programme (cf. figure III.1.).

La suite de 1 et de 0 codant chaque instruction est appelée le **langage machine** (seul compréhensible par le microcontrôleur) et il existe une bijection entre le langage machine et le langage assembleur.

Comme en pratique, on ne peut pas directement programmer en langage machine (trop compliqué, trop long à écrire et trop d'erreurs possible), on utilisera un logiciel qui permet de passer du langage assembleur au langage machine.

Ce logiciel s'appelle un logiciel d'assemblage ou plus succinctement un **assembleur**.

Remarque 1 :

*Pour faire l'opération inverse, qui consiste à passer du langage machine vers le langage assembleur, on utilise un **désassembleur**.*

Remarque 2 :

Lorsqu'on écrira des programmes en langage C, il faudra utiliser un compilateur C qui transformera votre source écrit en C en langage machine, c'est-à-dire en un fichier exécutable par le microcontrôleur. Finalement, un assembleur n'est rien d'autre qu'un cas particulier de compilateur entre le langage assembleur et le langage machine.

Comme le montre le schéma de la figure III.1., sur ces 14 bits, certains bits correspondront au codage du mnémonic (champ bits **OPCODE**), d'autres correspondront au codage du registre **SFR** ou **GPR** concerné par le mnémonic (champ bits **FILE**) et enfin un bit indiquera l'opérande destination (champ bit **d**).

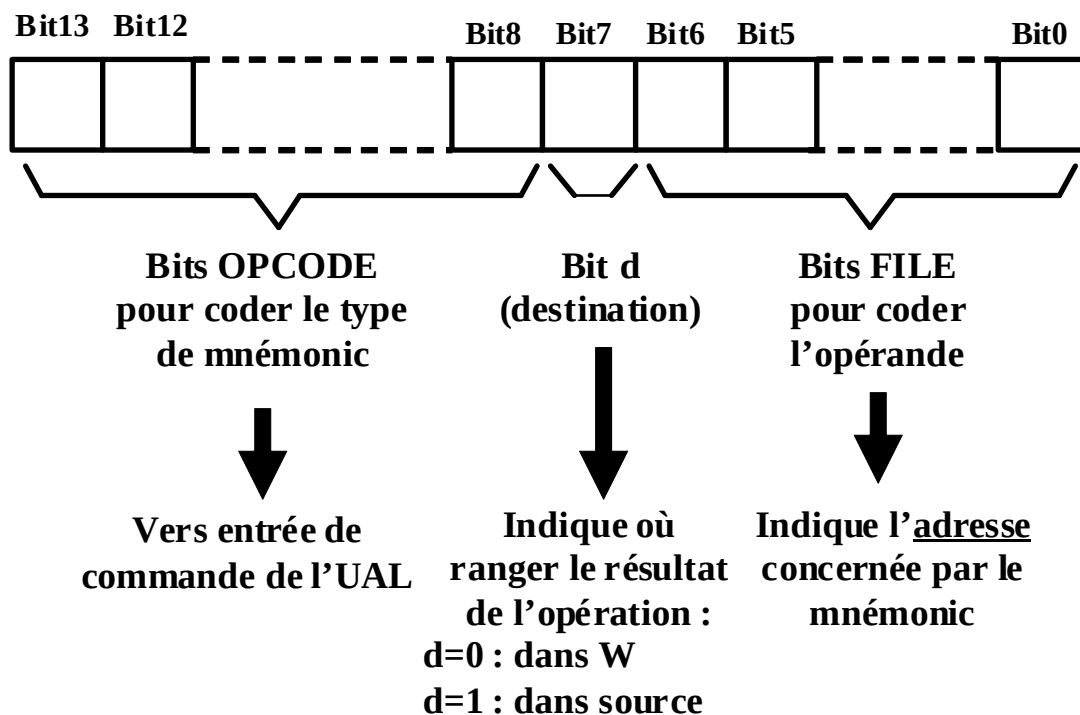


Figure III.1.

Remarquons dès à présent que le fait que l'adresse de l'opérande source soit codée sur 7 bits, ce qui permet de distinguer 128 positions mémoire différentes (de 0x00 à 0x7F) aura des conséquences sur le découpage de la RAM internes en 4 zones, comme on va le voir dans la partie suivante.

Nous reviendrons plus en détail sur ces différents points dans le **chapitre IV.** concernant le jeu d'instructions du PIC 16F877A.

III.2. LA MEMOIRE INTERNE RAM : SFR ET GPR

(cf. pages 16 et 17 du datasheet)

Elle est constituée de **512** cases de taille **8 bits** chacune, qui contiennent :

- les **SFR** : registres internes pour le fonctionnement du PIC et ses périphériques, (ce sont un peu les registres « système »),
- les **GPR** : cases mémoire disponibles pour les variables du programme, (ce sont un peu les registres « utilisateur »).

Pour différencier 512 cases différentes, il faut 9 bits d'adresses ($2^9=512$). Or, dans le code de chaque instruction, l'adresse de l'opérande concerné n'est codée que sur 7 bits. Il manque donc 2 bits.

Ces 2 bits, appelés **RP1** et **RP0** seront rajoutés par le PIC pour former une adresse de 9 bits, comme indiqué sur le schéma de la figure III.2.

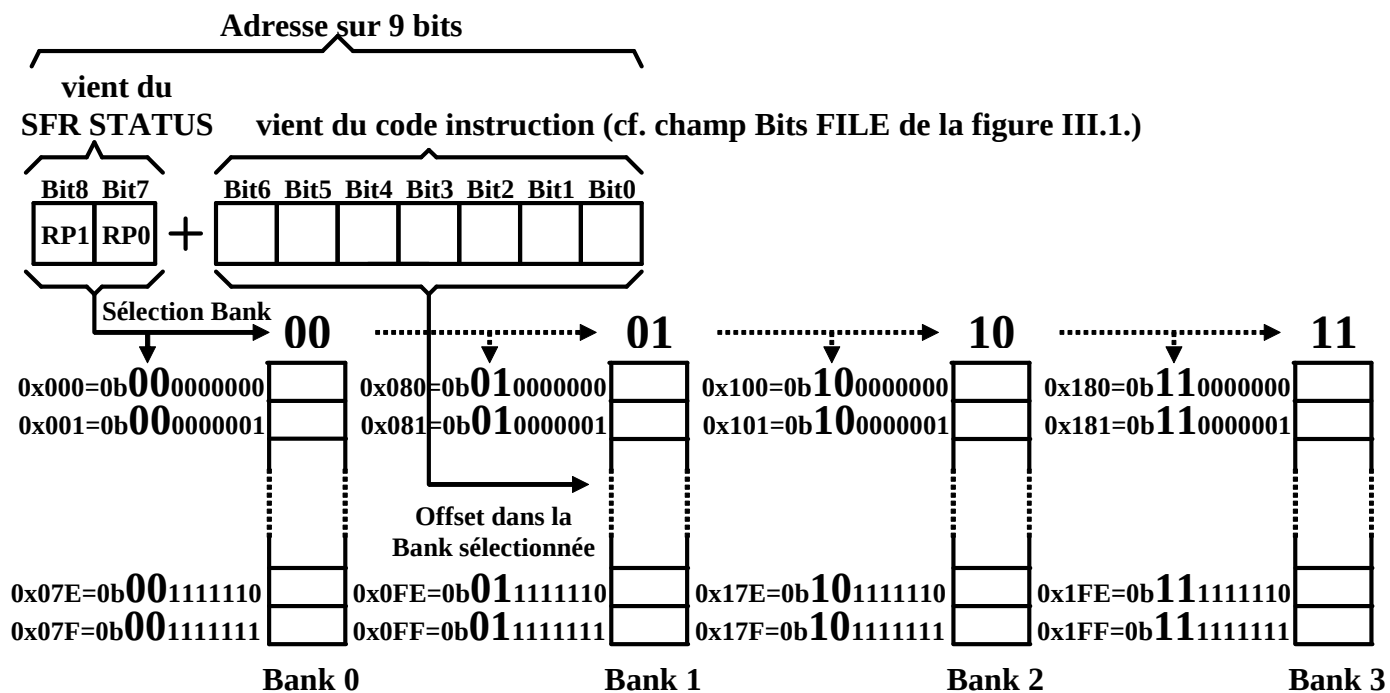


Figure III.2.

Sur cette figure, les adresses ont été écrites en hexadécimal, mais aussi en binaire, pour bien voir le rôle des deux bits **RP1** et **RP0**.

On voit donc qu'au final, les 512 octets de la RAM internes seront divisés en 4 banks (terminologie de MICROCHIP) de 128 octets chacune :

Bank 0 : de **0x000** à **0x07F** (**RP1RP0=00**) **Bank 1** : de **0x080** à **0x0FF** (**RP1RP0=01**)
Bank 2 : de **0x100** à **0x17F** (**RP1RP0=10**) **Bank 3** : de **0x180** à **0x1FF** (**RP1RP0=11**)

- les deux bits **RP1-RP0** permettent de **choisir** (sélectionner) une bank,
- les 7 bits issus du code instruction (champ Bits FILE) permettent de choisir une case parmi les 128 cases contenues dans la bank sélectionnée par **RP1-RP0**. Ils représentent donc l'offset (c'est-à-dire le décalage par rapport à l'adresse de base (du début) de la bank sélectionnée).

Pour accéder à une case mémoire de la RAM, il faudra donc auparavant penser à positionner correctement les bits **RP1-RP0** en fonction de la bank où se trouve cette case. On verra dans un instant où se trouvent ces 2 bits **RP1-RP0**.

Ce mode d'adressage qui permet d'accéder directement à chacune des 512 positions de la RAM s'appelle **l'adressage direct**.

Notons dès à présent qu'il existe un autre mode d'adressage, **l'adressage indirect**, permettant d'utiliser un pointeur (et qui facilite justement l'implantation de la notion de pointeur que l'on trouve (et utilise abondamment) dans des langages évolués comme le C par exemple).

On peut maintenant détailler l'organisation de la RAM interne du PIC en donnant son mapping résumant la signification de chacune des cases.

Ce mapping, extrait de la **page 17** du datasheet constructeur du PIC 16F877A est donné ci-dessous sur la figure III.3.

On y retrouve bien les 4 banks précédemment introduites.

File Address	File Address	File Address	File Address
Indirect addr. ^(*) 00h	Indirect addr. ^(*) 80h	Indirect addr. ^(*) 100h	Indirect addr. ^(*) 180h
TMR0 01h	OPTION_REG 81h	TMR0 101h	OPTION_REG 181h
PCL 02h	PCL 82h	PCL 102h	PCL 182h
STATUS 03h	STATUS 83h	STATUS 103h	STATUS 183h
FSR 04h	FSR 84h	FSR 104h	FSR 184h
PORTA 05h	TRISA 85h	105h	185h
PORTB 06h	TRISB 86h	PORTB 106h	TRISB 186h
PORTC 07h	TRISC 87h	107h	187h
PORTD ^(*) 08h	TRISD ^(*) 88h	108h	188h
PORTE ^(*) 09h	TRISE ^(*) 89h	109h	189h
PCLATH 0Ah	PCLATH 8Ah	PCLATH 10Ah	PCLATH 18Ah
INTCON 0Bh	INTCON 8Bh	INTCON 10Bh	INTCON 18Bh
PIR1 0Ch	PIE1 8Ch	EEDATA 10Ch	EECON1 18Ch
PIR2 0Dh	PIE2 8Dh	EEADR 10Dh	EECON2 18Dh
TMR1L 0Eh	PCON 8Eh	EEDATH 10Eh	Reserved ^(*) 18Eh
TMR1H 0Fh	8Fh	EEADRH 10Fh	Reserved ^(*) 18Fh
T1CON 10h	90h	110h	190h
TMR2 11h	SSPCON2 91h	111h	191h
T2CON 12h	PR2 92h	112h	192h
SSPBUF 13h	SSPADD 93h	113h	193h
SSPCON 14h	SSPSTAT 94h	114h	194h
CCPR1L 15h	95h	115h	195h
CCPR1H 16h	96h	116h	196h
CCP1CON 17h	97h	General Purpose Register 16 Bytes 117h	General Purpose Register 16 Bytes 197h
RCSTA 18h	TXSTA 98h	118h	198h
TXREG 19h	SPBRG 99h	119h	199h
RCREG 1Ah	9Ah	11Ah	19Ah
CCPR2L 1Bh	9Bh	11Bh	19Bh
CCPR2H 1Ch	9Ch	11Ch	19Ch
CCP2CON 1Dh	9Dh	11Dh	19Dh
ADRESH 1Eh	ADRESL 9Eh	11Eh	19Eh
ADCON0 1Fh	ADCON1 9Fh	11Fh	19Fh
20h	A0h	120h	1A0h
General Purpose Register 96 Bytes	General Purpose Register 80 Bytes	General Purpose Register 80 Bytes	General Purpose Register 80 Bytes
Zone commune 70h	accesses 70h-7Fh	accesses 70h-7Fh	accesses 70h - 7Fh
7Fh	FFh	17Fh	1FFh
Bank 0	Bank 1	Bank 2	Bank 3

 Unimplemented data memory locations, read as '0'.
^{*} Not a physical register.

Figure III.3.

Huit remarques sur ce mapping :

① Les cases qui portent un nom à l'intérieur sont des **SFR** (registres de fonctionnement et registres pour les périphériques). Ce nom inscrit à l'intérieur est **le nom du registre**. Nous détaillerons leur rôle, leur signification et leur contenu (pour pouvoir les utiliser) dans la suite de ce polycopié.

Exemple : **TMR0, PCL, STATUS, FSR, PORTA, PORTB ...**

② Parmi ceux-ci, **certains sont présents dans plusieurs banks**, avec le même offset.

Exemple : **PCL** à l'offset **0x002** de chaque bank et donc aux adresses absolues :

- **0x002** : dans la bank **0**,
- **0x082** : dans la bank **1**,
- **0x102** : dans la bank **2**,
- **0x182** : dans la bank **3**.

Ceci est réalisé sur le principe des adresses fantômes (ghost) obtenues par un décodage incomplet comme on l'a vu dans le **chapitre I**. Un même emplacement physique possède plusieurs adresses.

Cela veut dire, qu'on pourra accéder à ces registres avec n'importe laquelle de ces adresses, et donc qu'on n'aura pas à gérer les bits **RP1** et **RP0** (seul l'offset de ces registres est significatif).

Cela raccourcira le code et accélèrera les accès à ces SFR.

③ les zones grisées et hachurées dans le mapping ne correspondent à aucun SFR.

④ le registre à l'offset 0x000 de toutes les banks (**Indirect addr**) sert pour l'adressage indirect qu'on verra plus loin.

⑤ au final, et compte tenu des 4 remarques précédentes, il y a **54 SFR** différents.

⑥ Les cases qui n'ont qui **n'ont pas de nom de registres à l'intérieur** sont des **GPR**, c'est-à-dire des cases disponibles pour stocker les variables de votre programme.

⑦ Les 16 derniers octets de chaque bank sont une **zone commune** : cela signifie que les plages adresses **0x0F0-0x0FF (bank 1)**, **0x170-0x17F (bank 2)**, **0x1F0-0x1FF (bank 3)**, sont des ghosts de la plage d'adresses **0x070-0x07F** de la **bank 0**.

Par exemple, écrire (ou lire) à l'adresse **0x175** revient à écrire (ou lire) à l'adresse **0x075**.

⑧ Au final, et compte tenu des 2 remarques précédentes, il y a **368 GPR** différents, et donc la possibilité de stocker 368 octets en RAM.

Nous allons maintenant détailler quelques uns des registres **SFR**, et dans un premier temps ceux qui sont intimement liés au fonctionnement de la CPU.

Les autres, liés aux différents périphériques, seront présentés plus loin.

Astuce « pratico-pratique » pour gagner du temps :

Vous trouverez d'ailleurs aux **pages 19, 20 et 21** du datasheet, sous forme de tableaux, la signification de chacun de ces registres et le nom de chacun de leurs bits individuels.

La dernière colonne de ces tableaux est intéressante car elle renvoie à la page du datasheet où des explications complémentaires sont données sur le rôle de chacun de ces SFR et de leurs bits individuels.

III.2.1. LE REGISTRE STATUS

(cf. **page 22** du datasheet)

On le nomme aussi le **registre d'état**. Il est situé à l'offset **0x03** de chacune des 4 banks.

De façon très générale, on trouvera un registre d'état (bien sûr pas avec le même format qu'ici) dans tous les micro/processeurs/contrôleurs. Ce registre sert à donner des informations sur l'**état** de la CPU après l'exécution de chaque instruction, informations utiles qui peuvent être testées pour assurer le bon déroulement du programme.

Son format, pour le PIC 16F877, est fourni sur la figure III.4.

Sur cette figure, comme dans beaucoup d'autres plus loin, les notations **R/W-x**, **R-x** ou **W-x**, situées au dessus de chaque bit veulent dire :

R/W : bit accessible en **lecture** ou en **écriture** (Read/Write) : on peut le lire **ou** le modifier,

R : bit accessible uniquement en lecture (Read) : on ne peut **que** le lire,

W : bit accessible uniquement en écriture (Write) : on ne peut **que** le modifier,

-x : valeur à la mise sous tension du PIC (0, 1 ou X comme indéterminé).

STATUS REGISTER (situé aux adresses 0x003, 0x083, 0x103, 0x183)
(page 18 du datasheet)

STATUS	R/W-0	R/W-0	R/W-0	R-1	R-1	R/W-X	R/W-X	R/W-X
	IRP	RP1	RP0	$\overline{TO}$	$\overline{PD}$	Z	DC	C
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure III.4.

Les **bits 4 et 3** (appelés $\overline{TO}$ (comme **T**ime **O**ut bit) et $\overline{PD}$ (comme **P**ower **D**own bit), tous les 2 actifs à l'état bas, sont liés à des fonctionnements particuliers du PIC.

Ils permettent de connaître la cause ayant provoqué le dernier arrêt du PIC (démarrage normal, mise en sommeil ou problème de Watch Dog). Ils seront présentés plus tard et notamment dans le **chapitre IV**. (cf. instructions **CLRWDT** et **SLEEP**).

III.2.1.1. BITS RP1-RP0 : ADRESSAGE DIRECT

Commençons par les **bits 6 et 5** de ce registre. Ce sont les bits **RP1** et **RP0** qui servent dans le mode d'adressage direct. Ils permettent, comme on l'a vu sur la figure III.2. précédente, de sélectionner une des 4 banks (celle où se trouve le **SFR** ou la **variable** concernée par l'instruction).

III.2.1.2. LES INDICATEURS D'ETAT

Les trois bits de poids faibles sont appelés des **FLAGS** (drapeaux). Qu'est ce que c'est ?

Ces bits indiquent des situations particulières et seront utilisés lors des **ruptures conditionnelles synchrones de séquence** (en les testant).

• FLAG Z :

Prenons le cas du **bit 2** du registre **STATUS**, appelé bit **Z** comme **Zero**.

Certaines instructions peuvent fournir un résultat nul. Par exemple, si on incrémente le registre **W** alors qu'il contient **0xFF**, si on soustrait deux opérandes de **valeur égale**, si on charge une valeur **nulle** dans un **GPR**...). Dans ces cas là, le bit **Z** sera positionné à **1**.

Par contre, lorsque ces même instructions donnent un résultat non nul (on incrémente **W** qui contient **autre chose que 0xFF**, on soustrait deux opérandes **différents**, la valeur chargée dans un **GPR** est **non nulle**...), le bit **Z** sera positionné à **0**.

• FLAG C :

Prenons maintenant le cas du **bit 0** du registre **STATUS**, appelé bit **C** comme **Carry** (ou bit de **retenue**).

Imaginons que l'on fasse une addition entre un premier opérande de taille un octet et un deuxième opérande de la même taille.

Si dans certains cas le résultat tient bien sur un octet (si la somme est $\leq 0xFF$ et donc si les opérandes ne sont pas trop grands par exemple), dans le cas général il peut y avoir une retenue qui doit être considérée comme le 9^{ème} bit du résultat (le **MSB**) et qui doit être prise en compte pour que cette addition soit juste. Le bit **C** est justement ce 9^{ème} bit.

Le bit **C** sert aussi dans le cas d'une soustraction (il s'appelle **alors BORROW** $\equiv$ **retenue négative**).

Pour comprendre comment fonctionne une soustraction, il faut faire un rappel sur le **codage des entiers négatifs**.

Rappel : codage des entiers.

Pour les entiers, le contenu binaire (ou hexadécimal) d'une case mémoire peut être **interprété** de 2 façons différentes, comme indiqué sur le tableau de la figure III.5. suivante.

Sur ce tableau, on a supposé un codage sur un octet, mais le principe serait le même si l'entier était codé sur 2 ou 4 octets situés dans des cases de RAM contiguës.

		①	②
Hexadécimal	Binaire (naturel)	Non signé (valeur décimale)	Signé (valeur décimale)
0x00	0b 0000 0000	0	0
0x01	0b 0000 0001	1	+1
0x02	0b 0000 0010	2	+2
....			
0x7E	0b 0111 1110	126	+126
0x7F	0b 0111 1111	127	+127
0x80	0b 1000 0000	128	-128
0x81	0b 1000 0001	129	-127
0x82	0b 1000 0010	130	-126
....			
0xFE	0b 1111 1110	254	-2
0xFF	0b 1111 1111	255	-1

Figure III.5.

① Entiers non signés (cf. colonne repérée ① sur la figure III.5.)

Pour les entiers non signés et donc tous positifs, c'est le codage binaire **simple** ou **naturel**.

Chaque bit représente son poids en base 2.

On voit qu'on peut les représenter :

- sur 1 octet de : $0 \text{ à } 2^8 - 1$ soit de 0 à 255,
- sur 2 octets de : $0 \text{ à } 2^{16} - 1$ soit de 0 à 65 535,
- sur 4 octets de : $0 \text{ à } 2^{32} - 1$ soit de 0 à 4 294 967 295.

C'est le codage utilisé par le C sur des déclarations du type :

unsigned		char	// taille 8 bits
unsigned	(short)	int	// taille 16 bits
unsigned	short long		// taille 24 bits
unsigned	long		// taille 32 bits

② Entiers signés : (cf. colonne repérée ② sur la figure III.5.)

Pour les entiers signés, on utilise le codage dit **en complément à deux**.

On voit qu'on peut les représenter :

- sur 1 octet de : $-2^7 \text{ à } 2^7 - 1$ soit de -128 à +127,
- sur 2 octets de : $-2^{15} \text{ à } 2^{15} - 1$ soit de -32 768 à 32 767
- sur 4 octets de : $-2^{31} \text{ à } 2^{31} - 1$ (à calculer ...)

C'est le codage utilisé par le C sur une déclaration du type :

signed		char	// taille 8 bits
signed	(short)	int	// taille 16 bits
signed	short long		// taille 24 bits
signed	long		// taille 32 bits

Pour ce type de codage, le bit **MSB** prend un sens particulier :

- il vaut **0** pour tous les entiers **positifs ou nul**,
- il vaut **1** pour tous les entiers **négatifs**.

On l'appelle pour cette raison **le bit de signe**.

Si on n'a pas le tableau de la figure III.5. sous les yeux, comment trouver le codage d'un entier en représentation signée ?

- s'il est positif, c'est facile, c'est le binaire naturel,
- s'il est négatif, la méthode, générale, est donnée sur l'exemple suivant.

Exemple : codage de **(-5)** sur un octet en binaire signé.

Valeur absolue de -5 :	0b 0000 0101	
Inversion de tous les bits :	0b 1111 1010	(complément à 1)
Ajout de 1 :	+ 0b 0000 0001	
Soit au final :	= 0b 1111 1011	= 0xFB

Et donc **(-5)** sera codé **0xFB** en binaire signé.

Pour trouver la valeur d'un entier négatif codé en binaire signé, on fait la même chose.

Exemple : entier négatif codé par **0x83** en signé :

Valeur en binaire signé :	0b 1000 0011	= 0x83
Inversion de tous les bits ;	0b 0111 1100	(complément à 1)
Ajout de 1 :	+ 0b 0000 0001	
Soit au final :	= 0b 0111 1101	= 0x7D = (125)₁₀

Soit 125 en décimal donnant la valeur absolue de l'entier négatif.

Et donc **\$83** (en binaire signé) représente l'entier **-125** (en décimal).

Il est intéressant de se rappeler que le nombre binaire constitué que de 1 soit :

0xFF sur **1 octet**,

ou

0xFF FF sur **2 octets**,

ou

0xFF FF FF FF sur **4 octets**

ou ...

représente le codage de l'entier - 1 en binaire signé (sur 1, 2, 4 ... octets).

Ces rappels ayant été faits, comment fonctionne une **soustraction** dans l'ALU ?

La réponse est : comme une **addition** !!!

Pour faire **A - B**, l'ALU fait **A + (-B)**,

où **(-B)** représente le codage en complément à deux de l'entier **-B**.

Voyons cela sur trois exemples simples résumés sur la figure III.6. suivante, qui nous permettront de voir également comment sont gérés les flags **C** et **Z** dans ces trois cas là.

A	B	A-B	Codage de -B	A+(-B)	Flag Z	Flag C
7	5	2	0b 1111 1011	0b 0000 0111 + 0b 1111 1011 = 1 0b 0000 0010 (+2 OK)	0	1
7	7	0	0b 1111 1001	0b 0000 0111 + 0b 1111 1001 = 1 0b 0000 0000 (0 OK)	1	1
7	9	-2	0b 1111 0111	0b 0000 0111 + 0b 1111 0111 = 0 0b 1111 1110 (-2 OK)	0	0

Figure III.6.

Au final, et après une soustraction de type $A - B$ (pour A et B en non signés) :

- si **Z** = **1** alors : **A = B**,
- si **C** = **1** alors : **A ≥ B**,
- si **C** = **0** alors : **A < B**.

Ce qui permettra de comparer deux variables A et B et d'implanter par exemple les tests conditionnels que l'on trouve dans les langages de haut niveau comme :

```

if ( A == B) {.....} else {.....};    // par test à 1 du flag Z
if ( A >= B) {.....} else {.....};    // par test à 1 du flag C
if ( A < B)   {.....} else {.....};    // par test à 0 du flag C

```

....

ou tout autre type de structure conditionnelle, facilitant la programmation en langage évolué.

• **FLAG DC :**

Le **bit 1** de **STATUS**, appelé **DC** (comme **D**igit **C**arry) joue un peu le même rôle que le bit **C**, mais dans le cas d'une retenue entre le 4^{ème} (bit 3) et le 5^{ème} bit (bit 4) lors d'une addition entre deux octets.

Cela est utile lorsqu'on effectue les calculs en **DCB** (**D**écimal **C**odé **B**inaire).

On rappelle qu'en **DCB**, le quartet de poids fort d'un octet code les dizaines et celui des poids faibles code les unités d'un nombre décimal quelconque.

(de **0b0000** pour coder le chiffre **1** à **0b1001** pour coder le chiffre **9**).

Exemple :

Premier opérande :	en binaire : 0b 0000 1001	codant en DCB :	09
+			
Deuxième opérande :	en binaire : 0b 0000 1000	codant en DCB :	08
=			
Résultat	en binaire : 0b 0001 0001	soit en DCB :	11

Le résultat est faux car on doit obtenir en DCB 17 soit 0b0001 0111.

Il y a eu une retenue entre le bit 3 et le bit 4.

Et dans ce cas le bit **DC** sera positionné à **1** ce qui permettra de détecter cette erreur et de la corriger.

On retiendra que :

- si certaines instructions modifient les flags
en les positionnant à 1 ou à 0 suivant le résultat obtenu
(faire ainsi une espèce de « nano-photographie » sur 3 bits de l'état CPU),
- d'autres, immédiatement après, devront les tester
(exploiter la « nano-photographie » transmise par l'instruction précédente)
pour prendre en compte ces positionnements.

C'est le principe de la **rupture synchrone conditionnelle de séquence**.

Mais attention, seules certaines instructions modifient les flags, alors que d'autres n'ont aucune incidence sur eux (voir pour cela la liste complète des instructions sur la figure IV.9.)

III.2.1.3. BIT IRP : ADRESSAGE INDIRECT

(cf. **page 31** du datasheet)

Pour comprendre la signification du dernier bit du registre **STATUS**, le **bit 7** ou bit **IRP** (comme **I**ndirect **R**egister **P**ointer), il faut dire quelques mots sur la notion **d'adressage indirect**.

On a vu l'adressage direct qui consistait à accéder à une valeur de variable (en lecture ou en écriture) directement en utilisant son adresse. Ce mode est intéressant pour les variables globales, car le logiciel d'assemblage (ou le compilateur pour le C) connaît l'adresse de celles-ci au moment de la compilation.

C'est l'adressage direct qui sera utilisé par exemple par une affectation du type $i = 5$; (où i par exemple est un **int** déclaré en global).

Mais dans certains cas, il est intéressant de pouvoir modifier une variable à partir de son adresse, en utilisant un pointeur sur cette variable, comme dans :

$*p=5$; (où p est un pointeur, déclaré par exemple comme un **int***)

C'est là qu'intervient la notion d'adressage indirect, qui permet de gérer des pointeurs au niveau de la CPU.

Deux registres **SFR** interviennent dans le mode d'adressage indirect, présents tous les deux dans les 4 banks :

- le registre **FSR** (comme **F**ile **S**elect **R**egister, à ne pas confondre avec **SFR** !!!) à l'offset **0x04** de chaque bank : il contient les 8 bits de poids faible du pointeur,
- le registre **Indirect.addr** (appelé aussi **INDF** comme **IND**irect **F**ile) à l'offset **0x00** de chaque bank : il sert pour indiquer l'utilisation de l'adressage indirect.

Comme avec 8 bits, on ne peut accéder qu'à $2^8=256$ cases mémoires, et en utilisant le même principe que les 2 bits **RP1-RP0** de l'adressage direct, on rajoute un 9^{ème} bit au pointeur : c'est le bit **IRP**. Ce qui permettra d'accéder aux 512 positions de la RAM.

Le principe est fourni sur le schéma de la figure III.7.

Pour l'adressage indirect, la RAM est vue comme 2 zones de 256 octets chacune appelée **page**.

- la **page 0**, correspond à l'association des **banks 0 et 1** :
adresses de **0x000** à **0x0FF** avec **IRP=0**,
- la **page 1**, correspond à l'association des **banks 2 et 3**.
adresses de **0x100** à **0x1FF** avec **IRP=1**.

Au final, ce principe est assez similaire à celui de la figure III.2. précédente.

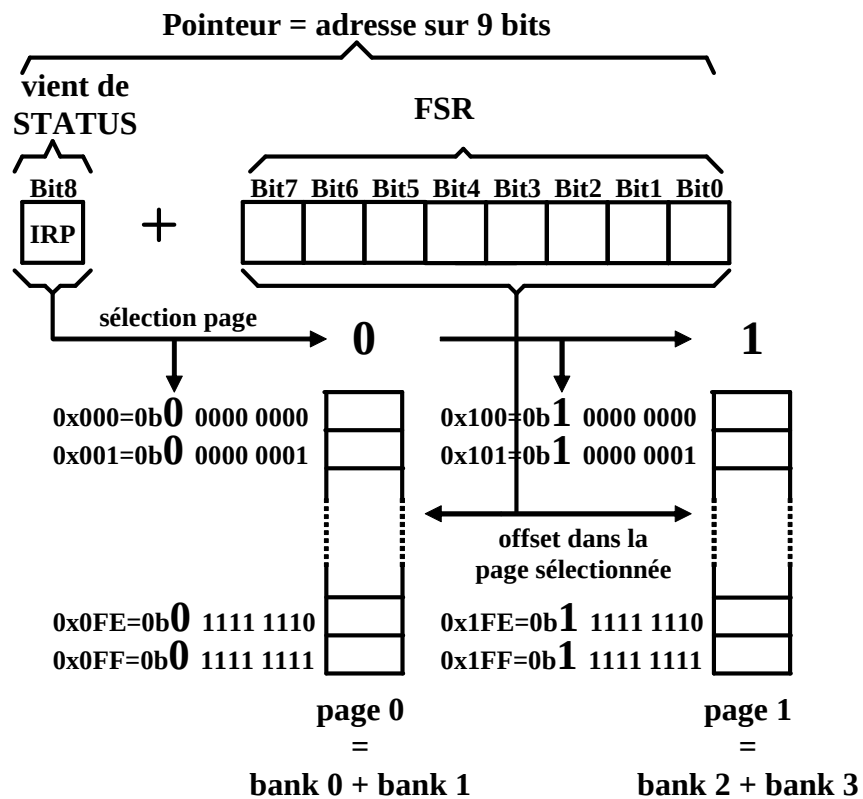


Figure III.7.

Comment marche alors l'adressage indirect ?

En quatre temps :

Il faut :

- ① : positionner le bit **IRP** de **STATUS** en fonction de la page dans laquelle se trouve la première variable pointée,
- ② : initialiser le registre **FSR** avec l'adresse de cette variable (offset dans la page sélectionnée),
- ③ : faire un accès (en lecture ou en écriture) sur le registre **INDF**,
- ④ : mettre à jour la seconde variable que l'on doit modifier.

Cela est résumé sur le schéma de la figure III.8. où on se propose de charger dans la variable **i** le **contenu** de la case mémoire d'adresse **0x020** (et donc à l'offset **0x20** de la **page 0** de la RAM) en utilisant l'adressage indirect.

Les 4 étapes précédentes ont également été indiquées sur le schéma de la figure III.8.

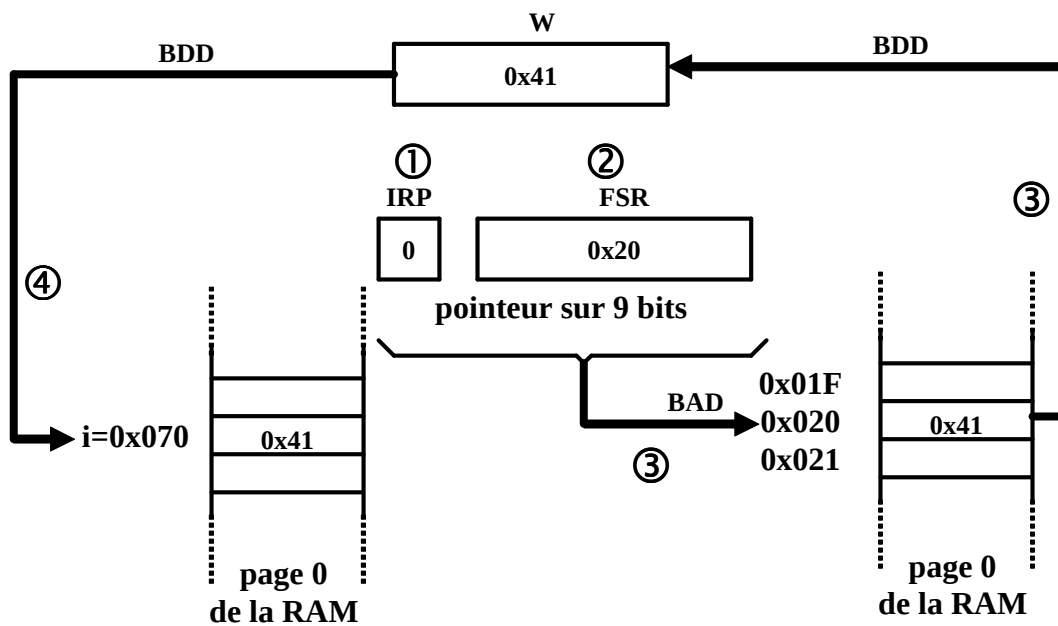


Figure III.8.

Voilà ce que donnerait cet exemple en assembleur.

(les instructions assembleur utilisées ici seront détaillées plus loin, dans le **chapitre IV**.)

étape ① : mise à 0 du bit **IRP** du registre **STATUS** par une instruction **BCF** (Bit Clear File) sur **STATUS**

```
BCF      STATUS,IRP
```

étape ② : initialisation du registre **FSR** avec **0x20**

```
MOVLW    0x20      ; charge l'offset de l'adresse dans W
MOVWF    FSR        ; et le met dans FSR
```

étape ③ : accès indirect, via **INDF**, pour charger le registre **W** avec le contenu de la case **0x020**

```
MOVE     INDF,W      ; lecture indirecte de la case d'adresse 0x020 dans W
```

étape ④ : et enfin, **mise à jour** de la variable **i** (par adressage direct).

Ici, le contenu de **W** est placé dans la case **0x070** (adresse de **i**)

```
MOVWF    i           ; copie de W dans la case de i
                        ; sans gérer RP0/RP1 : zone commune
```

En langage C, cette séquence sera avantageusement remplacée par les lignes suivantes utilisant un **pointeur** (ici de valeur 0x020).

```
int * p ;           // déclaration d'un pointeur de type int *
int i ;            // déclaration d'une variable i de type int
.
.
.
p = 0x020;         // initialisation du pointeur p
i = *p;            // initialise i avec le contenu de la case pointée par p
                   // c'est-à-dire avec le contenu la case d'adresse 0x020
```

Nous verrons d'autres exemples plus loin dans ce polycopié et notamment dans les exercices corrigés **9** et **10** du chapitre **V**. que je vous invite à faire et à bien comprendre.

L'adressage indirect apparaît donc comme un moyen très efficace pour gérer des variables dont l'adresse n'est pas connue au moment de la compilation (adressage dynamique de variables), ou pour accéder à des structures de données plus complexes, comme les tableaux par exemple (le registre FSR contiendra alors l'adresse de base de ce tableau, c'est-à-dire l'adresse du premier élément du tableau).

Finalement, chaque fois que l'assembleur rencontrera INDF dans une instruction, il sait qu'il s'agit de la case mémoire dont l'adresse (sur 9 bits) se trouve dans le registre FSR pour ses 8 bits de poids faible complétée par le bit IRP du registre STATUS pour son 9^{ème} bit.

III.2.2. LE REGISTRE PC

(cf. **page 30** du datasheet)

On rappelle que dans tout micro/processeur/contrôleur, le registre **PC** est un registre essentiel au déroulement d'un programme, puisque c'est lui qui indique, à tout moment, l'adresse de la prochaine instruction à exécuter (instruction qui se trouve en ROM) (cf. figure I.22.).

**On dit qu'il pointe sur la prochaine instruction à exécuter, car il contient son adresse :
c'est un registre pointeur vers la mémoire de code.**

Ici deux registres **SFR** de taille **8 bits** chacun seront mis à contribution pour jouer le rôle du **PC**. Ces deux registres, on peut le voir sur la figure III.3. sont présents dans les 4 banks :

- **PCL** : à l'offset **0x02** de chaque bank,
- **PCLATH** : à l'offset **0x0A** de chaque bank.

La façon de fabriquer **PC** avec **PCL** et **PCLATH** dépend de la façon dont la mémoire de code interne est organisée dans le PIC, puisque le registre **PC** sert à pointer vers cette mémoire.

Il faut donc commencer à étudier la mémoire ROM interne. C'est l'objet de la partie suivante.

Puis nous reviendrons sur le **PC** ensuite.

III.3. LA MEMOIRE INTERNE ROM

(cf. **page 15** du datasheet)

Il s'agit d'une **FLASH EEPROM** d'une capacité de **8K cases** (ou **mots**) de taille **14 bits** chacune, correspondant, comme on l'a déjà vu à la taille d'une instruction assembleur codée en langage machine.

Cette mémoire de code est paginée : elle est divisée en 4 pages de **2K cases** comme indiqué sur la figure III.9. suivante :

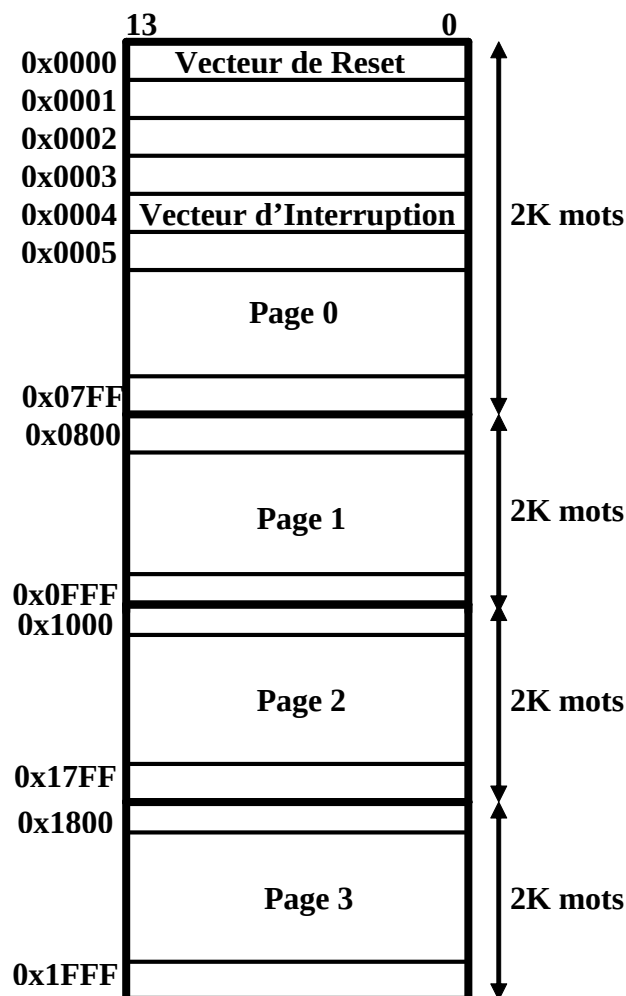


Figure III.9.

On remarque sur cette figure que les cases d'adresses **0x0000** et **0x0004** sont un peu particulières car elles contiennent respectivement le **vecteur de reset** et le **vecteur d'interruption**. Nous aurons l'occasion de reparler de cela dans un chapitre ultérieur consacré justement aux interruptions et à la façon de les gérer.

Pour accéder à ces **8K** mots, il faut **13 lignes d'adresses** ($2^{13} = 8192 = 8K$). Il manque donc **5 bits** au registre **PCL** qui jouera le rôle des 8 bits de poids faibles du **PC**.

Pour comprendre la gestion du **PC**, il faut fournir quelques éléments (précisés plus loin) sur la structure d'un programme source **écrit en assembleur**, représentés sur la figure III.10. ci-dessous.

Champ des adresses (labels, étiquettes)	Champ des instructions (mnémonics + opérandes de ces mnémonics)	Champ des commentaires (commençant par ;)
debut	Instruction 1	; commentaire 1
	Instruction 2	; commentaire 2
	Instruction 3	; commentaire 3
		

Figure III.10.

Il est important de respecter ces alignements en colonnes pour augmenter la lisibilité et la compréhension des programmes.

Le **champ des adresses** correspond aux adresses des cases mémoire de la ROM où se trouve le code machine des instructions du programme (**champ mnémonics+opérandes de ces mnémonics**). Par exemple, dans l'exemple de la figure III.10 *debut* est appelé un label ou une étiquette.

Le **champ commentaire** est bien sûr « shunté » par la compilation.

Ainsi, il faut voir le programme source ci-dessus comme une représentation symbolique (en langage assembleur) de la ROM programme, comme indiqué sur la figure III.11. ci-dessous, en supposant sur cet exemple que l'adresse *debut* vaut 0x800 (ce qui signifie que ce petit programme sera implanté à partir de l'adresse 0x800).

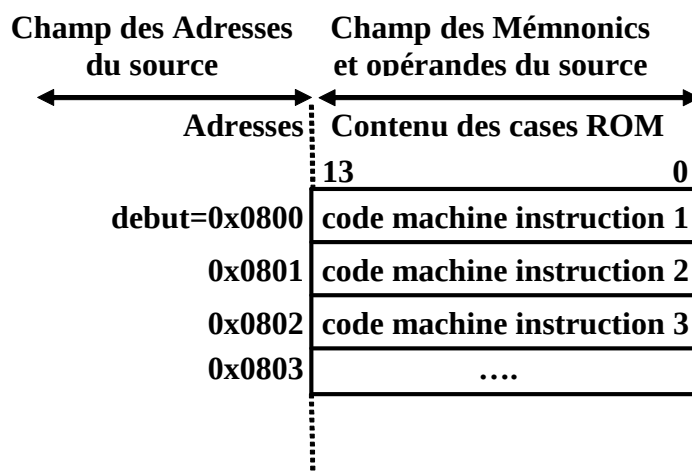


Figure III.11.

L'**instruction 1** se trouve à l'adresse **0x800**, l'**instruction 2** à l'adresse suivante **0x801**, l'**instruction 3** à l'adresse suivante **0x802** etc. ...

Ainsi, le programme, au rythme d'évolution du **PC** va exécuter dans l'ordre l'instruction 1, puis la 2, puis la 3 etc. ...

C'est ce qu'on appelle un **déroulement séquentiel du programme** : l'instruction suivante qui sera exécutée se trouve dans la case mémoire suivante. C'est le cas la plupart du temps.

Dans ce cas là le **PC** est fabriqué **automatiquement** par concaténation des **5 bits** de poids faible de **PCLATH** avec les **8 bits** de **PCL**, comme indiqué sur la figure III.12.

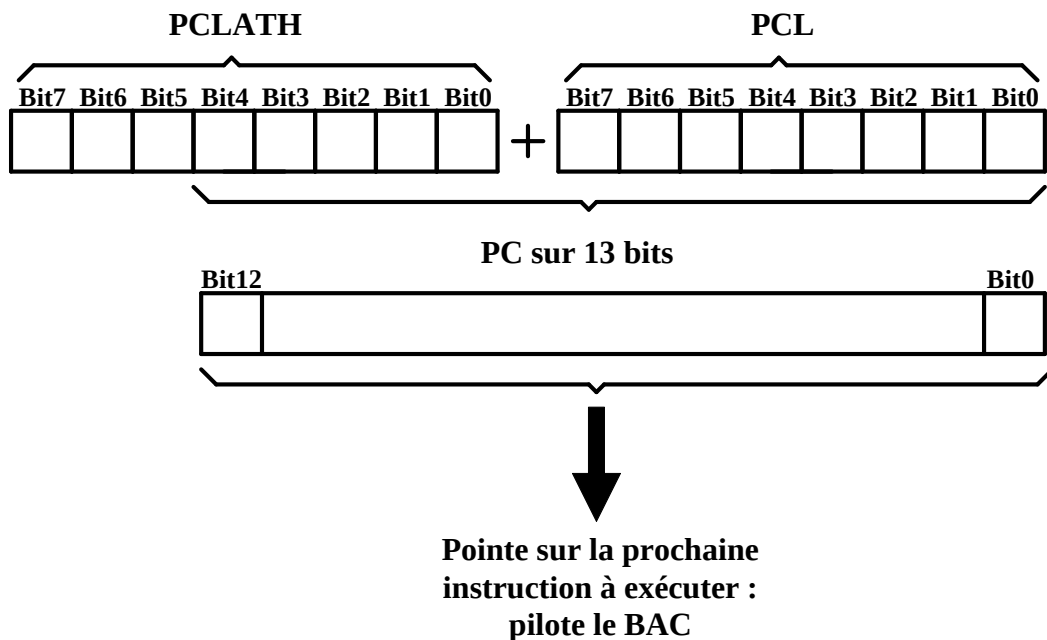


Figure III.12.

Ce mécanisme sera aussi utilisé si **PCL** est modifié par programme.

Mais on conçoit bien que si on se limitait à cette présentation simplifiée, un programme ne s'exécuterait qu'une seule fois de « A à Z » (de sa première instruction jusqu'à sa dernière, dans un ordre figé qui est celui du rangement en mémoire, en effectuant toujours la même chose et ensuite ... « planterait ») ce qui manquerait d'intérêt.

Or, on sait, en regardant les langages évolués, que certaines parties d'un programme peuvent s'exécuter plusieurs fois (cas des **boucles for** par exemple) ou que, suivant une certaine condition, certains bouts de programmes peuvent s'exécuter si cette condition est vraie ou d'autres bouts, différents, dans le cas contraire (cas des structures conditionnelles du type **if (condition) { ... } else { ... }** par exemple).

En fait tout ce qui relève des instructions de contrôle algorithmiques de haut niveau.

L'appel des fonctions relève aussi en partie de ce type de fonctionnement.

Si on peut implanter ces structures dans des langages évolués de haut niveau, c'est donc que le bas niveau de fonctionnement, où on se situe ici, le permet.

C'est ce qu'on appelle une **rupture synchrone de séquence**.

On va présenter cette notion grâce à deux instructions assembleur :

① instruction **GOTO** de format :

GOTO label

Cette instruction permet de continuer l'exécution à une autre position repérée par son adresse **label**. (et pas forcément à l'adresse suivante en mémoire).

Comme dans l'exemple fourni sur la figure III.13.

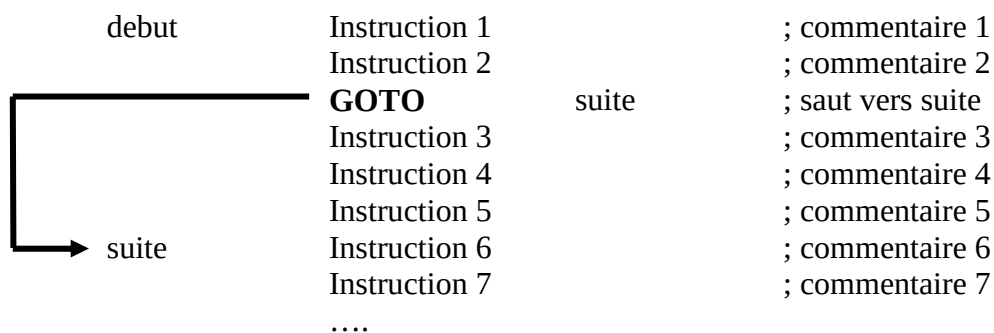


Figure III.13.

Comme on le voit, après l'instruction 2, grâce à l'instruction **GOTO**, on passe toujours (saut inconditionnel) à l'instruction 6 (et non à l'instruction 3), puis à l'instruction 7 etc. ...

On dit que l'on « **saute** » à l'adresse **suite**.

Remarque :

*On verra plus loin qu'il existe aussi des instructions de **saut** appelé alors **conditionnel** qui ne se réalisent que si une **certaine condition est vérifiée** (par exemple sur la valeur d'un des flags).*

② instruction **CALL** de format :

CALL nom_sous_programme

Cette instruction permet d'appeler un **sous-programme**, qu'on nomme aussi un **module**. En langage C, on appelle cela une **fonction**. Ces différents synonymes seront indifféremment utilisés ici.

Cette importante notion mérite qu'on s'y arrête et qu'on donne quelques explications supplémentaires.

Si on réfléchit un peu plus à la programmation et au traitement de l'information, on peut imaginer que certaines tâches puissent être exécutées plusieurs fois et à des moments différents par un même programme, chaque fois qu'il en a besoin.

Par exemple (et liste non exhaustive) :

- celle de recevoir (et donc de récupérer) un caractère arrivant sur une liaison (et qui sera répétée pour chaque nouveau caractère reçu),
- celle pour allumer un pixel en (X,Y) sur un écran graphique (et qui sera répétée plusieurs fois (en changeant bien sûr les coordonnées du pixel) pour tracer une droite, un cercle ou toute autre forme géométrique),
- celle qui consiste à faire une conversion analogique/numérique et qui sera répétée à chaque période d'échantillonnage ...

Voyons encore cela sur un dernier exemple encore plus précis.

Supposons qu'on ait à réaliser une application qui envoie des caractères sur un terminal type écran cathodique en mode texte (console alphanumérique par exemple).

Il sera intéressant alors de développer une tâche qu'on peut appeler **PRINT_CHAR**.

En langage C ce serait une fonction de prototype **void PRINT_CHAR (char c)** qui afficherait n'importe quel caractère ASCII sur cet écran.

Remarquons aussi que cette tâche pourrait être avantageusement réutilisée pour en construire ensuite une autre de type **PRINT_STRING**. En langage C, cela correspondrait au prototype **void PRINT_STRING (char* message)** qui afficherait cette fois-ci une chaîne de caractères (en appelant autant de fois **PRINT_CHAR** qu'il le faut jusqu'à la reconnaissance du caractère nul (0x00) indiquant la fin du message à afficher).

Supposons maintenant que, pour des raisons d'économie ou d'encombrement, vous deviez remplacer votre terminal par un écran à cristaux liquides (type LCD par exemple), plus petit et moins encombrant.

Si vous avez procédé ainsi, vous pourrez garder en principe la même structure du programme principal, ne pas modifier la fonction **PRINT_STRING** : il vous suffira juste de réécrire la fonction **PRINT_CHAR** adaptées aux nouvelles particularités du LCD par rapport au terminal alphanumérique du départ.

C'est un peu en quelque sorte la notion de **driver de périphérique (device driver)** qu'on reconnaît ici.

On le voit les exemples sont nombreux et rappellent tous la notion de **fonction** si utile en langage C : on définit puis on écrit une fonction particulière et on l'appelle depuis le **programme principal** (le **main** du C) ou depuis une autre fonction (appel imbriqué).

En langage assembleur, au lieu de fonction, on parlera plutôt de **module**.

Les intérêts et avantages de procéder ainsi sont nombreux :

- **gain de mémoire** : le code de la fonction n'existe qu'en un seul exemplaire en mémoire, alors qu'il peut être exécuté autant de fois que nécessaire,
- on **découpe** un programme **complet** et **complexe** en **bouts plus simples**, ce qui **facilite** sa **conception**, sa **compréhension** et améliore sa **lisibilité** et son **évolutivité**,
- chaque fonction à réaliser peut être testée **individuellement** (et parfois **indépendamment**) du reste du programme principal : comme elle est en général plus simple que le main, cela **facilitera** les **tests** et la **mise au point**,
- l'assemblage de fonctions, toutes testées individuellement (et à **150%**), transforme la conception du programme principal en « **un jeu de Léo** », (rappelez-vous du sable pour faire des briques, des briques pour faire des murs, des murs pour faire des maisons ... Mais pour que vos murs soient solides, il faut que vos briques le soient également, pour que votre maison soit solide, il faut que vos murs le soient !!!)
- certaines fonctions (très **générales** et **génériques**) pourront avantageusement être **réutilisables** dans un autre contexte. Il n'y a qu'à voir la richesse des fonctions disponibles dans les **bibliothèques** du C pour s'en convaincre. De plus, une fonction issue d'une bibliothèque a été en principe validée complètement (c'est une brique solide !!!) : donc si votre programme qui l'utilise ne marche pas, l'erreur vient sûrement de vous ou d'une mauvaise interprétation de la documentation de cette fonction...

Bref beaucoup d'avantages pour l'écriture et la conception de programmes complexes.

Mais, une des difficultés dans la conception d'un nouveau logiciel applicatif est d'identifier ces fonctions (ou modules) de base qu'il faut écrire, puis ensuite tester et enfin rassembler entre elles pour réaliser l'application finale.

Mais, chaque fois que cela sera possible, nous essaierons de raisonner en utilisant des modules : ce sera un peu notre « **règle d'or** ».

En fait, tous les aspects dont on vient de parler ici, sous-entendent la notion de **sous-programme**.

Qu'est-ce c'est ?

C'est en fait un bout de programme, avec un début et une fin, conçu pour une tâche (un travail) précise et qui sera appelé par le programme principal chaque fois que cette tâche devra être effectuée.

A la fin de ce sous-programme, " la main " (c'est-à-dire le contrôle) doit être rendu au programme principal pour continuer normalement la suite du traitement.

Comme en langage C, quand on appelle une fonction, elle s'exécute sur la CPU et quand on arrive à son accolade fermante **}** ou quand elle effectue un **return** (valeur_retournée), on **revient** dans le main à l'endroit **situé juste après l'appel de la fonction** pour continuer la suite normale du programme principal.

On précisera tout cela dans la partie suivante consacrée à la pile.

Mais revenons à notre instruction **CALL**.

Cette instruction permet d'appeler un module, comme indiqué sur la figure III.14. suivante.

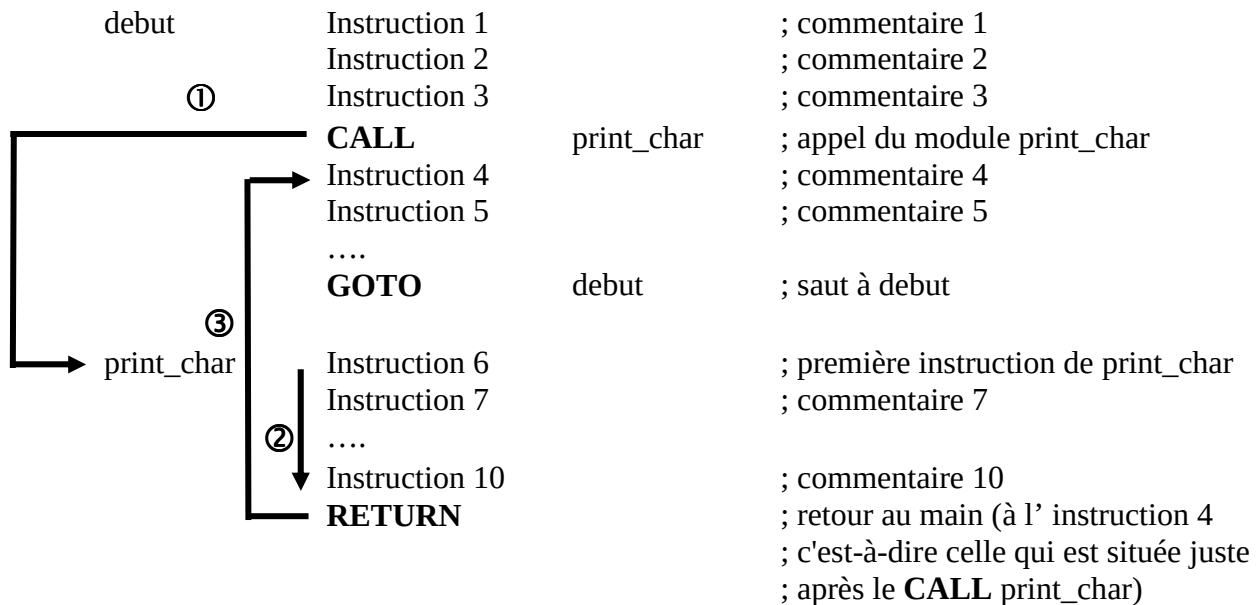


Figure III.14.

Après l'instruction :

CALL print_char

le contrôle est donné (voir flèche ① de la figure III.14.) à la première instruction du module print_char (Instruction 6). C'est-ce que j'appelle le « **ticket aller** ».

Puis on passe à l'instruction 7 (par déroulement séquentiel classique du module) etc. ... jusqu'à l'instruction 10 (voir flèche ② de la figure III.14. : phase d'exécution du module).

Et à la fin du module, après l'instruction :

RETURN

le contrôle est donné (voir flèche ③ de la figure III.14.) à l'instruction 4, c'est-à-dire à celle qui est située en mémoire juste après le **CALL** print_char. On a effectué le retour au main. C'est-ce que j'appelle le « **ticket retour** ».

Ensuite, le programme principal continue à se dérouler normalement et séquentiellement (instruction 4, instruction 5...)

Que ce soit pour l'instruction **GOTO** ou pour l'instruction **CALL**, pour effectuer la rupture synchrone de séquence, le **PC** doit être modifié pour pouvoir effectuer le saut demandé.

Dans le code machine des 2 instructions **GOTO** et **CALL**, l'adresse du saut est codée sur **11 bits** (c'est en fait l'opérande de ces deux instructions).

Or, rappelez-vous, il faut 13 bits pour repérer et atteindre n'importe quelle adresse de la mémoire de code. Dans ce cas, le **PC** permettant de fabriquer l'adresse du saut ou celle du début du sous-programme (sur 13 bits) est fabriqué automatiquement comme indiqué sur le schéma de la figure III.15. (en concaténant les **bits 4 et 3** de **PCLATH** avec les **11 bits de l'adresse du saut** à effectuer pour le **GOTO** ou les **11 bits de l'adresse de la première instruction du sous-programme** appelé par le **CALL**).

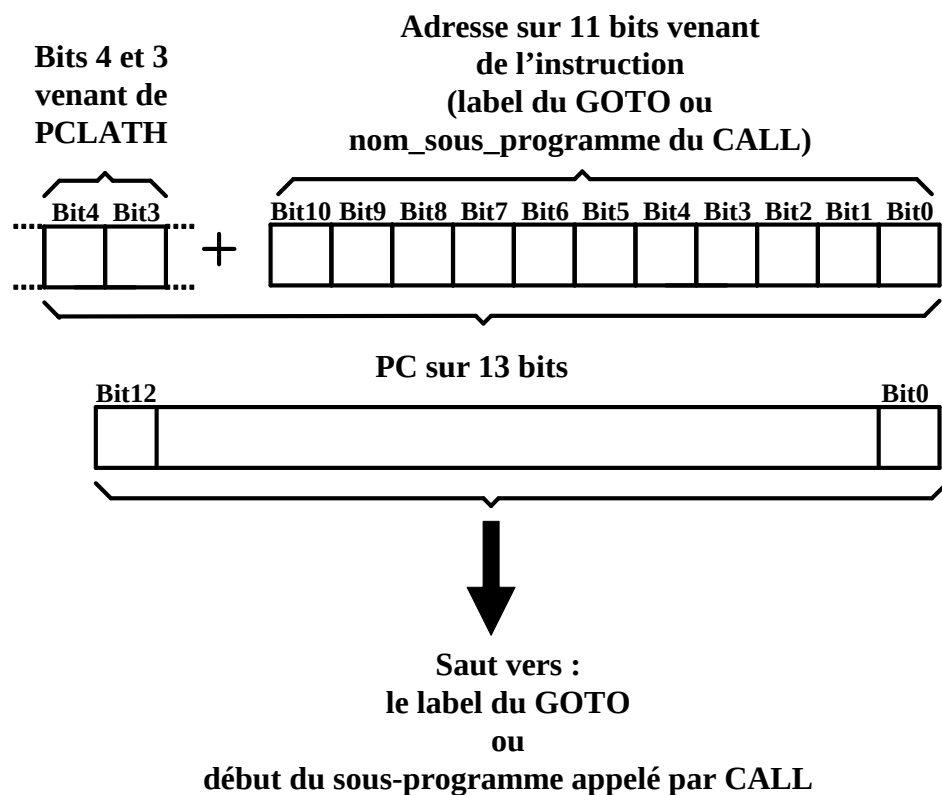


Figure III.15.

III.4. LA PILE

(cf. **page 15** du datasheet)

Il faut bien comprendre la différence entre une instruction de type **GOTO** et une instruction de type **CALL**.

Si ces deux instructions sont bien toutes les deux des ruptures synchrones de séquence, elles sont **fondamentalement** différentes.

Pour faire une rupture synchrone de séquence, on a vu qu'il fallait modifier le **PC**.

Dans le cas d'un **GOTO**, cela ne pose pas de problème particulier, puisqu'on n'a pas besoin de revenir vers l'endroit d'où on est parti : le **GOTO** est un « **ticket d'aller simple** ».

Par contre, on a vu que le **CALL** correspond à un « **ticket d'aller-retour** », et c'est le retour qui pose problème ici.

En effet, lorsque le **PC** est modifié pour sauter à la première instruction du module (cf. flèche ① de la figure III.14.), son ancienne valeur est perdue (n'oubliez pas qu'avant d'être modifié, le **PC** pointait sur l'instruction située en mémoire dans la case qui suit celle contenant le code machine du **CALL**).

Or c'était justement cette valeur qui permettait le retour. On l'appelle d'ailleurs **l'adresse de retour** : c'est l'adresse située en mémoire juste après celle qui contient le code machine du **CALL**.

Un peu comme si vous « brûliez » votre ticket de retour après l'avoir acheté. Idiot non ?

C'est là qu'intervient la **pile** (ou **stack**).

L'idée est de conserver **systématiquement** et **automatiquement** la valeur du **PC** (l'adresse de retour, c'est-à-dire votre « **ticket retour** ») dans un registre juste avant d'effectuer le **CALL** vers la première instruction du module (le « **ticket aller** »).

Ce registre de sauvegarde (vous mettez votre ticket de retour dans votre poche au lieu de le brûler, ce qui me semble plus malin mais aussi plus économique !!!) est situé dans une zone mémoire particulière et prévue pour cela qu'on appelle une **pile**.

S'agit-il à votre avis d'une mémoire de type RAM ou ROM ?

Et c'est l'instruction **RETURN**, située à la fin du module qui récupérera **automatiquement** dans la pile l'adresse de retour permettant le retour au bon endroit (cf. flèche ③ de la figure III.14.).

Une pile est une structure un peu particulière qui s'appelle une **LIFO** (**L**ast **I**n **F**irst **O**ut).

C'est la dernière valeur qui a été mise dans la pile qui sera la première à ressortir. Un peu comme une pile « d'assiettes » (c'est d'ailleurs de cette image que vient le nom de pile).

Dans le cas des PIC, cette zone de mémoire RAM pour la pile (d'une capacité de 8 cases de taille 13 bits chacune) est totalement indépendante de la RAM de données.

Dans d'autres micro/contrôleurs/processeurs, la pile est située dans la même RAM que celle utilisée pour les données globales du programme.

Cette dernière solution est plus intéressante car du coup on peut avoir plus de place pour la pile (pour un PIC, pas plus de 8 « assiettes »).

Par contre, il ne faut pas mélanger la mémoire dite " normale " (celle pour les données du programme) avec la zone de mémoire dédiée pour la pile.

Ce qui sera fait grâce à la gestion d'un registre particulier (et accessible au programmeur) appelé **pointeur de pile** (ou **Stack Pointer**), qui indique à tout instant la première case disponible sur la pile (on dit aussi le sommet de la pile c'est-à-dire la case mémoire où on peut ranger la prochaine « assiette »).

Dans le cas du PIC, ce registre pointeur de pile existe (en interne) mais n'est pas accessible par programmation.

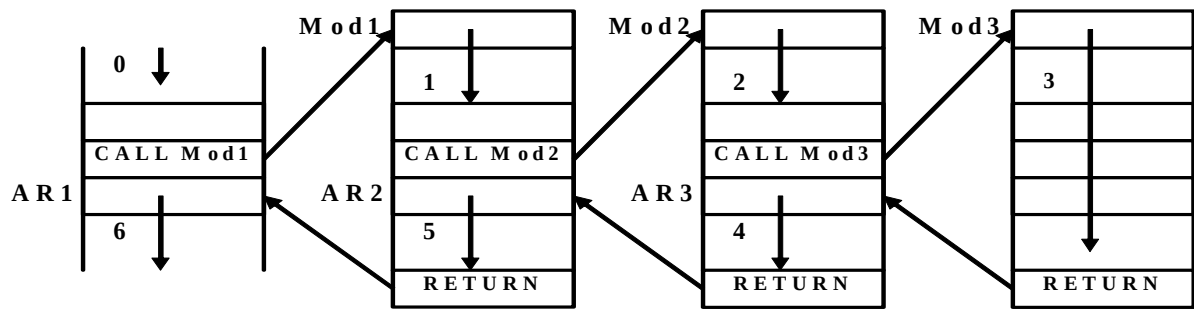
De même, pour les PIC, aucune instruction n'est disponible pour le programmeur pour placer des valeurs dans la pile (type l'instruction **PUSH** que l'on trouve dans certains micro/processeur/contrôleurs pour **empiler**) ou pour récupérer des valeurs de la pile (type l'instruction **PULL** que l'on trouve dans certains micro/processeur/contrôleurs pour **dépiler**).

Comme on l'a vu, les seuls cas d'empilement et de dépilement sont assurés **automatiquement** par le PIC au moment des appels (**CALL**) et des retours de module (**RETURN**). On verra plus loin que c'est aussi le cas pour les **interruptions** (qui sont considérées comme des ruptures de séquence **asynchrones**).

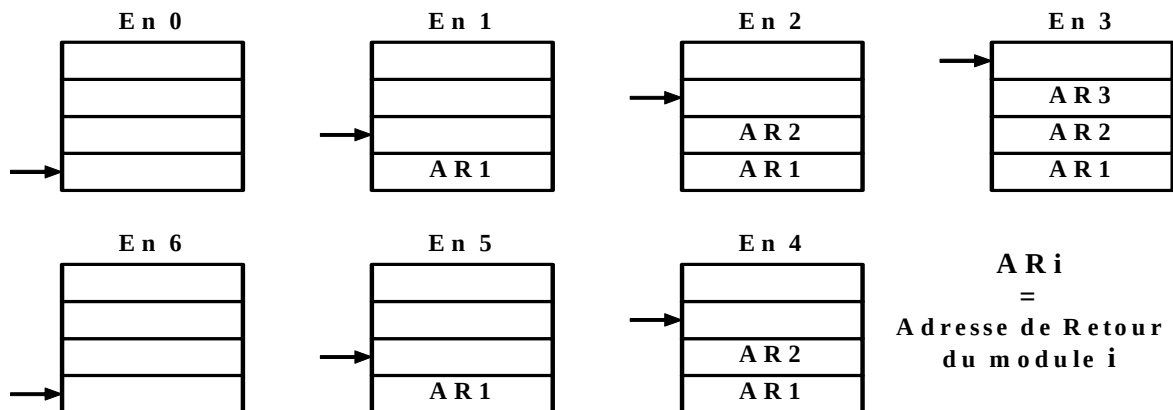
Pourquoi une structure de type **LIFO** (et pas de type **FIFO**, **F**irst **I**n **F**irst **O**ut, comme pour une file d'attente ou une queue) ?

Tout simplement pour assurer les imbrications de modules (appel d'un autre module par le code d'un premier module).

Ce principe est résumé sur la figure III.16.



Evolution de la pile au cours des différents appels



Principe d'imbrication des modules

Sur ce schéma, on a pris une pile avec 4 positions, mais pour le PIC 16F877, il y en a 8. On voit ici que si on a plus de 4 imbrications de fonctions, la pile déborde.

Figure III.16.

La pile est en fait gérée par la CPU du PIC comme un « **buffer circulaire** » :

- la neuvième imbrication (c'est-à-dire le neuvième empilement d'adresse de retour) écrasera la première,
- la dixième écrasera la deuxième...

On dit qu'il y a eu alors un **débordement de la pile**.

💣💣 : mais attention : il n'y a aucun indicateur de débordement de la pile.
**Si plus de 8 niveaux d'imbrication sont mis en œuvre,
 le PIC plantera sans autre forme de procès !!!**

La limite du nombre d'imbrications de fonctions/interruptions sera donc de **8** pour ce type de PIC.

Attention :

**ce n'est pas le nombre de fois qu'on appelle une fonction qui est limité à huit,
 mais c'est bien le nombre d'imbrications.**

Finalement, grâce à tout ce mécanisme, un sous-programme (ou une fonction du langage C) n'est rien de plus qu'une « **parenthèse active** » (effectuant un certain travail) au sein d'un programme. Ce qui était bien le but recherché.

Après l'exécution du sous-programme, le programme ayant appelé ce sous-programme peut continuer la suite de son traitement un peu comme s'il ne s'était rien passé, à ceci près que le sous-programme s'est exécuté et a fait le travail qu'on lui demandait de faire.

  : Erreur fréquente des étudiants :

Confusion :
entre un saut inconditionnel (ici par GOTO) ou conditionnel (voir plus loin)
(pas de possibilité automatique de retour (≡ ticket d'« aller simple »))
et
un appel de module (ici par CALL)
(avec retour systématique à l'instruction qui suit l'appel (≡ ticket « aller-retour »))

III.5. LA CONFIGURATION

(cf. pages 143 et 144 du datasheet)

C'est un registre un peu spécial, appelé registre de **CONFIGURATION** qui sera programmé dans l'EEPROM de programme à l'adresse **0x2007** (et donc en dehors de tout programme). On parle aussi de registre de « fusibles ».

Quand ce registre n'est pas programmé, il ne contient que des 1 (en l'occurrence 14, car sa taille est de **14 bits**, ce qui donne **0x3FFF**). Programmer un (ou plusieurs) bit(s) quelconque(s), c'est faire passer ce(s) bit(s) à 0, et ceci sera fait lors de l'implantation du programme en mémoire avec ce(s) bit(s) à 0.

Par contre, la valeur ainsi écrite restera dans le registre de **CONFIGURATION** même lorsque l'alimentation du PIC sera coupée (ce qui est normal car ce registre est situé en EEPROM !!!).

Donc, pour refaire passer ce(s) bit(s) à 1, il faudra le(s) re-modifier au moment de la prochaine implantation du programme (avec effacement complet préalable des EEPROMs internes).

Globalement ce registre sert à fixer des règles de protection et de sécurité pour le programme implanté :

- se protéger contre les piratages extérieurs (protection du code et des données permanentes/semi permanentes),
- éviter les effacements intempestifs et non souhaités des EEPROM,
- gérer les défauts d'alimentation,
- assurer un fonctionnement correct et propre du programme pendant les transitoires de démarrage,
- se protéger contre des éventuels parasites extérieurs,
- choisir l'horloge du système,
- mais aussi faciliter le débogage (par un **ICD**, comme **In Circuit Debugueur**),
- ou la programmation des EEPROM pour l'implantation des programmes ou des données permanentes/semi permanentes.

Le format de ce registre est fourni sur la figure III.17.

Registre de CONFIGURATION : adresse 0x2007

Bit13	Bit12	Bit11	Bit10	Bit9	Bit8	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
CP1	CP0	DEBUG	WRT	CPD	LVP	BODEN	CP1	CP0	PWRT	WDTE	FOCS1	FOCS0	

Figure III.17.

La signification de chacun de ces bits est donnée sur le tableau de la figure III.18. avec les différents noms des masques de configuration correspondants à utiliser.

Bits	Valeur	Commentaires
Protection mémoire EEPROM de programme contre lecture extérieure (piratage) ou écriture (suivant bit WRT)		
CP1:CP0	11	Aucune protection (_CP_OFF)
(Bits 13:12)	10	Plage 0x1F00/1FFF protégée (_CP_UPPER_256)
et	01	Plage 0x1000/1FFF protégée (_CP_HALF)
(Bits 5:4)	00	Mémoire programme complète protégée 0x0000/1FFF (_CP_ALL)
Débuggage possible « in circuit » via les lignes RB7 et RB6 du PORTB		
DEBUG	1	Pas de débbugage : RB7 et RB6 sont des I/O utilisables (_DEBUG_OFF)
(Bit 11)	0	RB7 et RB6 affectés au dialogue avec le débbugueur (_DEBUG_ON)
Bit 10	X	Non implémenté (lu à 1)
Autorisation d'écriture dans l' EEPROM de programme par le code		
WRT	1	Ecriture dans les zones non protégées par CP1:CP0 (_WRT_ENABLE_ON)
(Bit 9)	0	Ecriture impossible dans la mémoire de programme (_WRT_ENABLE_OFF)
Protection mémoire EEPROM de données contre lecture extérieure (piratage)		
CPD	1	Aucune protection (_CPD_OFF)
(Bit 8)	0	Mémoire de données complète protégée (_CPD_ON)
Utilisation possible de la ligne RB3/PGM comme broche de programmation		
LVP	1	RB3 permet la programmation du PIC sous 5V (_LVP_ON)
(Bit 7)	0	RB3 est une I/O utilisable (_LVP_OFF)
Provoque un reset du PIC en cas de chute anormale de la tension d'alimentation		
BODEN	1	En service (_BODEN_ON)
(Bit 6)	0	Désactivé (_BODEN_OFF)
Délai de démarrage à la mise sous tension (attention : automatiquement validé si BODEN = 1)		
PWRTE	1	Pas de délai (sauf si BODEN=1) (_PWRTE_OFF)
(Bit 3)	0	Délai en service (_PWRTE_ON)
Validation du Watch Dog (chien de garde) : sécurité parasitage ou plantage programme		
WDTE	1	Watch Dog en service (_WDT_ON)
(Bit 2)	0	Watch Dog désactivé (_WDT_OFF)
Choix du type d'oscillateur pour l'horloge système		
FOSC1:FOSC0	11	Oscillateur de type RC (_RC_OSC)
	10	Oscillateur haute fréquence (4 à 20 MHz) (_HS_OSC)
	01	Oscillateur basse fréquence (0,2 à 4 MHz) (_XT_OSC)
(Bits 1:0)	00	Oscillateur faible consommation (32 à 200 kHz) (_LP_OSC)

Figure III.18.

Comment utiliser ces différents masques d'options de configuration ?

Dans un programme écrit en assembleur :

Les masques d'option de configuration sont définies le fichier d'inclusion nommé **p16f877.inc**.
Il faudra donc rajouter la ligne suivante dans votre programme source en assembleur :

```
#include <p16f877.inc>
```

Les valeurs des différents masques d'option sont résumées sur le tableau de la figure III.19.

Masque option	Valeur en hexa	Valeur en binaire															
_CP_OFF	0x3FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
_CP_UPPER_256	0x2FEF	1	0	1	1	1	1	1	1	1	0	1	1	1	1	1	1
_CP_HALF	0x1FDF	0	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1
_CP_ALL	0x0FCF	0	0	1	1	1	1	1	1	0	0	1	1	1	1	1	1
_DEBUG_OFF	0x3FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
_DEBUG_ON	0x37FF	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1
_WRT_ENABLE_ON	0x3FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
_WRT_ENABLE_OFF	0x3DFF	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1
_CPD_OFF	0x3FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
_CPD_ON	0x3EFF	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1
_LVP_ON	0x3FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
_LVP_OFF	0x3F7F	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1
_BODEN_ON	0x3FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
_BODEN_OFF	0x3FBF	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1
_PWRTE_OFF	0x3FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
_PWRTE_ON	0x3FF7	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1
_WDT_ON	0x3FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
_WDT_OFF	0x3FFB	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1
_RC_OSC	0x3FFF	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
_HS_OSC	0x3FFE	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0
_XT_OSC	0x3FFD	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1
_LP_OSC	0x3FFC	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0

Figure III.19.

Ainsi, grâce à une combinaison avec des **et logique (&)**, on pourra choisir les différentes options en fonction de la configuration souhaitée.

Les bits à 0 des différents masques positionneront les bits à 0 correspondant dans le registre **CONFIGURATION**.

Il faudra donc, en plus, écrire une ligne comme ci-dessous (donnée à titre d'exemple) :

```
__CONFIG _CP_OFF & _WRT_ENABLE_OFF & _CPD_OFF & _LVP_ON &  
_BODEN_OFF & _PWRTE_ON & _WDT_OFF & _HS_OSC  
(attention, il y a deux « _ » devant CONFIG ...)
```

Dans un programme écrit en C :

Le principe reste le même grâce au fichier **pic.h**.

Il faudra donc rajouter la ligne suivante dans votre programme source en C :

```
#include    <pic.h>
```

Ce fichier renvoie ensuite sur le bon fichier .h correspondant au PIC que vous utilisez dans votre projet (exemple **pic1687x.h** pour un **PIC 16F877**).

Il faudra ensuite utiliser la macro **__CONFIG(x)** (attention avec deux « _ » devant CONFIG) qui se trouve dans **htc.h** en écrivant les 2 lignes suivantes :

```
#include    <htc.h>
```

```
__CONFIG(RC & WDTDIS & BORDIS & LVPEN); // par exemple
```

Attention :

Les masques de configuration n'ont pas le même nom en C et en assembleur : il faut donc regarder dans le fichier **pic1687x.h** qui donne les noms des masques.

Remarque :

En TP, nous utiliserons un PIC 16F877A, qui a un registre de configuration légèrement différent. Mais le principe restera le même.

Il faudra par contre utiliser le fichier **p16f877a.inc** et écrire :

```
#include    <p16f877a.inc>
```

IV. LE JEU D'INSTRUCTIONS

(cf. pages 159 à 165 du datasheet)

IV.1. INTRODUCTION

On l'a vu, les PIC ont une architecture **RISC** et donc avec peu d'instructions.

Il y a au total **35 types d'instructions différentes**, qui sont toutes codées sur 14 bits en langage machine, ce qui correspond à la taille de chaque case de la mémoire de programme.

Ces 14 bits coderont donc en binaire toute l'information nécessaire à l'exécution de chaque instruction, à savoir :

- le type d'instruction : champ **OPCODE** avec au **maximum 6 bits** ($2^6=64$ combinaisons différentes, ce qui est suffisant pour distinguer ces 35 instructions).

En effet, pour que le décodeur d'instructions puisse bien les différencier, il faut que chaque instruction possède son propre code, différent de celui des autres,

- et **les 8 bits minimum** restants coderont les **opérandes** (source et destination) concernés par chaque instruction.

Ce qui donnera, en 14 bits, toutes les informations nécessaires pour que l'instruction puisse s'exécuter complètement et correctement.

En pratique, MICROCHIP classe les instructions en **4** grandes familles :

- les instructions **BYTE-ORIENTED** (orientées **octet**) : cf. figure IV.1.

Elles travaillent sur des opérandes de taille un octet (et donc avec **W**, les **SFR** et les **GPR**) en adressage **direct** ou **indirect**.

C'est **l'adresse** du registre concerné (sur les 7 bits du champ **f**) qui est codée dans l'instruction.

De plus, un bit (le bit **d** comme **d**estination) est réservé pour indiquer le registre qui contiendra le résultat de l'instruction.

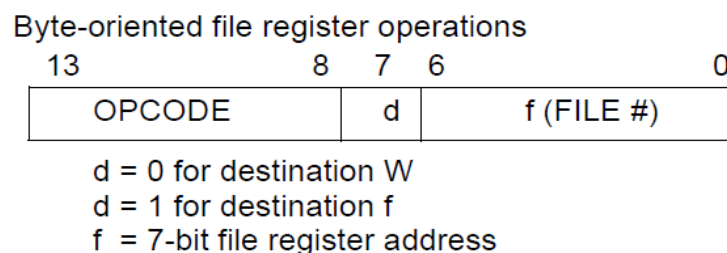


Figure IV.1.

- les instructions **BIT-ORIENTED** (orientées **bit**) : cf. figure IV.2.

Elles permettent de manipuler directement un bit d'un registre particulier (dans un **SFR** ou dans un **GPR**) en adressage **direct** ou **indirect**.

Ces instructions sont particulièrement intéressantes dans le cas d'un microcontrôleur car on est souvent amené à **positionner** ou à **tester** des bits individuels dans ses registres internes (surtout dans les registres **SFR** liés aux périphériques).

L'adresse du registre est toujours codée sur 7 bits (champ **f**) et le bit concerné par l'instruction (de 0 à 7) est codé sur 3 bits (de 0b 000 à 0b 111) dans le champ **b**.

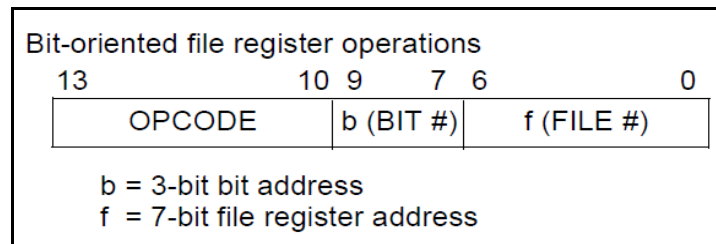


Figure IV.2.

- les instructions **LITTERAL-ORIENTED** (orientées **constante**) : cf. figure IV.3.

Elles permettent de travailler avec des constantes (que MICROCHIP appelle des valeurs **littérales**).

C'est le mode d'adressage nommé parfois aussi l'adressage **immédiat** : la valeur du littéral (de la constante) est **immédiatement** codée (sur les 8 bits du champ **k**) dans l'instruction.

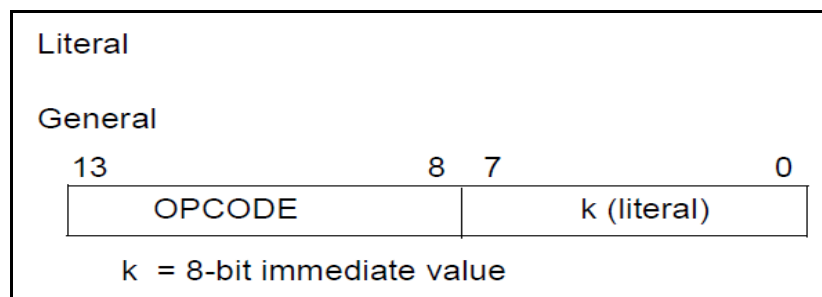


Figure IV.3.

- les instructions de **CONTRÔLE** : cf. figure IV.4.

Pour les ruptures de séquence et les modes de fonctionnement particuliers. Pour les ruptures de séquence, l'adresse du saut ou du sous-programme est codée sur les 11 bits du champ **k**.

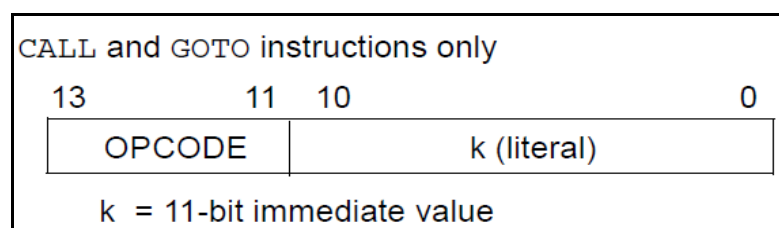


Figure IV.4.

IV.2. LISTE COMPLETE DES INSTRUCTIONS

Pour présenter l'ensemble des instructions disponibles, **MICROCHIP** fournit le tableau de la figure IV.5. (et donné à la **page 160** du datasheet).

Il est conforme au classement en 4 familles présenté ci-dessus.

La colonne ① donne la **syntaxe de l'instruction** : les champs mnémoniques et opérandes tels qu'ils apparaîtront dans une instruction assembleur.

f représente l'adresse d'un registre **SFR** ou **GPR** (comme **file**),

d représente la **d**estination du résultat :
soit W (si d=0),
soit f (si d=1 ou si d est omis dans le champ opérande),

b est le **b**it concerné,

k est la valeur du littéral **k**.

La colonne ② donne une **brève description** de l'instruction.

La colonne ③ donne la **durée de l'instruction** : pour la grande majorité, c'est un cycle (soit 1µs pour une horloge à 4 MHz ou 200 ns pour une horloge à 20 MHz), sauf pour les instructions de rupture synchrone de séquence qui durent alors 2 cycles quand cette rupture de séquence est effectivement réalisée (revoir le principe du pipe fois 2 de la figure I.24. si nécessaire).

La colonne ④ donne le **code machine** de chaque instruction.

La colonne ⑤ précise les **flags** du registre d'état **STATUS** (**C**, **DC**, **Z**, $\overline{\text{TO}}$, $\overline{\text{PD}}$) concernés et modifiés par l'instruction.

On voit que seules certaines instructions modifient les flags.

Par contre d'autres n'auront aucun effet sur ces flags.

La colonne ⑥ renvoie à des notes du bas de la **page 136** du datasheet (et non représentées ici).

Remarque et conseil :

Dans un premier temps, il n'est pas nécessaire de les connaître toutes par cœur.

Mais il est important à mon avis de bien les comprendre et de savoir « en gros » celles qui existent (voir d'ailleurs plus loin le classement que je propose et qui me semble mieux adapté pour les regroupements par type de famille). Quand vous en aurez besoin, ce sera plus facile de les retrouver si vous les avez déjà vues au moins une fois. Vous les apprendrez au fur et à mesure de leur utilisation. Une bonne méthode pour les mémoriser est d'apprendre leur signification en anglais.

①	②	③	④	⑤	⑥			
Mnemonic, Operands	Description	Cycles	14-Bit Opcode			Status Affected	Notes	
			MSb	LSb				
BYTE-ORIENTED FILE REGISTER OPERATIONS								
ADDWF	f, d	Add W and f	1	00	0111	dfff ffff	C,DC,Z	1,2
ANDWF	f, d	AND W with f	1	00	0101	dfff ffff	Z	1,2
CLRF	f	Clear f	1	00	0001	1fff ffff	Z	2
CLRW	-	Clear W	1	00	0001	0xxx xxxx	Z	
COMF	f, d	Complement f	1	00	1001	dfff ffff	Z	1,2
DECF	f, d	Decrement f	1	00	0011	dfff ffff	Z	1,2
DECFSZ	f, d	Decrement f, Skip if 0	1(2)	00	1011	dfff ffff		1,2,3
INCF	f, d	Increment f	1	00	1010	dfff ffff	Z	1,2
INCFSZ	f, d	Increment f, Skip if 0	1(2)	00	1111	dfff ffff		1,2,3
IORWF	f, d	Inclusive OR W with f	1	00	0100	dfff ffff	Z	1,2
MOVF	f, d	Move f	1	00	1000	dfff ffff	Z	1,2
MOVWF	f	Move W to f	1	00	0000	1fff ffff		
NOP	-	No Operation	1	00	0000	0xx0 0000		
RLF	f, d	Rotate Left f through Carry	1	00	1101	dfff ffff	C	1,2
RRF	f, d	Rotate Right f through Carry	1	00	1100	dfff ffff	C	1,2
SUBWF	f, d	Subtract W from f	1	00	0010	dfff ffff	C,DC,Z	1,2
SWAPF	f, d	Swap nibbles in f	1	00	1110	dfff ffff		1,2
XORWF	f, d	Exclusive OR W with f	1	00	0110	dfff ffff	Z	1,2
BIT-ORIENTED FILE REGISTER OPERATIONS								
BCF	f, b	Bit Clear f	1	01	00bb	bfff ffff		1,2
BSF	f, b	Bit Set f	1	01	01bb	bfff ffff		1,2
BTFSC	f, b	Bit Test f, Skip if Clear	1 (2)	01	10bb	bfff ffff		3
BTFSS	f, b	Bit Test f, Skip if Set	1 (2)	01	11bb	bfff ffff		3
LITERAL AND CONTROL OPERATIONS								
ADDLW	k	Add literal and W	1	11	111x	kkkk kkkk	C,DC,Z	
ANDLW	k	AND literal with W	1	11	1001	kkkk kkkk	Z	
CALL	k	Call subroutine	2	10	0kkk	kkkk kkkk		
CLRWDI	-	Clear Watchdog Timer	1	00	0000	0110 0100	$\overline{TO}, \overline{PD}$	
GOTO	k	Go to address	2	10	1kkk	kkkk kkkk		
IORLW	k	Inclusive OR literal with W	1	11	1000	kkkk kkkk	Z	
MOVLW	k	Move literal to W	1	11	00xx	kkkk kkkk		
RETFIE	-	Return from interrupt	2	00	0000	0000 1001		
RETLW	k	Return with literal in W	2	11	01xx	kkkk kkkk		
RETURN	-	Return from Subroutine	2	00	0000	0000 1000		
SLEEP	-	Go into standby mode	1	00	0000	0110 0011	$\overline{TO}, \overline{PD}$	
SUBLW	k	Subtract W from literal	1	11	110x	kkkk kkkk	C,DC,Z	
XORLW	k	Exclusive OR literal with W	1	11	1010	kkkk kkkk	Z	

Figure IV.5.

Il peut être intéressant de regarder sur un cas particulier le principe du codage de chaque instruction.

Cela permettra par la même occasion de donner quelques éléments sur l'écriture d'un fichier source en assembleur et sur sa transformation en fichier exécutable (code machine) par le logiciel d'assemblage.

Prenons l'instruction (prise totalement au hasard) :

INCF **PORTA**

Elle consiste à incrémenter le contenu du registre **SFR** appelé **PORTA** et situé à l'offset **0x05** de la **bank 0** et à placer le résultat dans ce même registre (rappelez-vous que si **d** omis dans le champ opérande, alors le résultat est placé dans **f**, donc ici dans le registre **PORTA**),

En regardant le codage en binaire (langage machine) de cette instruction sur la figure IV.6. ci-dessous (extraite bien sûr du tableau précédent),

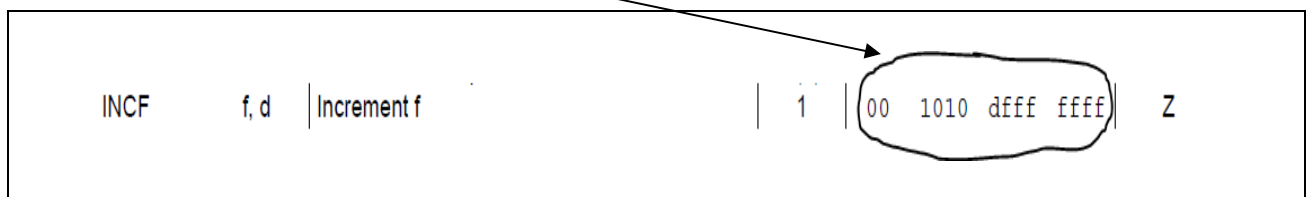


Figure IV.6.

on obtient :

- bit **d** = **1** : résultat dans **PORTA** (et pas dans **W** !!!)
- bits **fff ffff** = **0x05** = **000 0101** (revoir le tableau de la figure III.3. pour le mapping des SFR)

Ce qui donne au final le code suivant de cette instruction (sur 14 bits) :

00 1010 1000 0101 soit : **0x 0 A 8 5**

Ce que l'on vient de faire « à la main » ici, c'est en fait ce que fera le logiciel d'assemblage, qui va transformer votre programme source (constitué de toutes les instructions de votre programme écrites en langage assembleur) en langage machine.

On peut le vérifier en examinant la ligne assembleur de la figure IV.7. ci-dessous (supposée appartenir à votre source) :

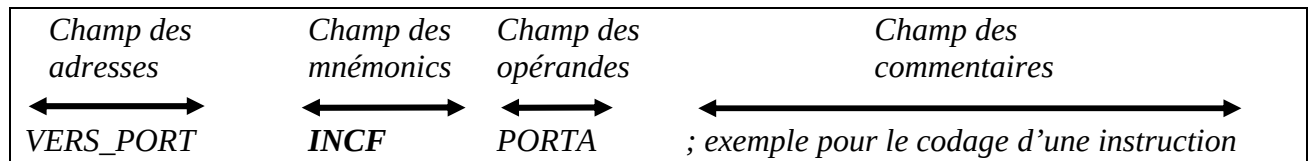


Figure IV.7.

qui sera transformée par l'assembleur comme indiqué sur la figure IV.8. :

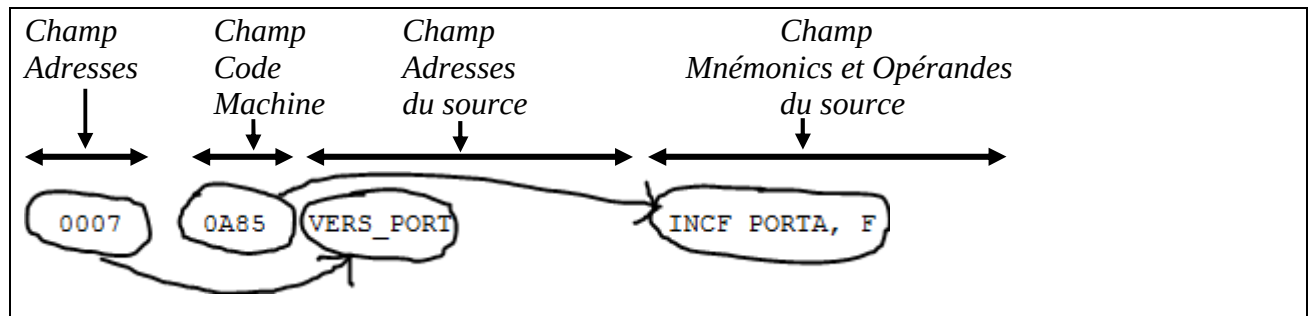


Figure IV.8.

Les deux dernières colonnes (Champ Adresses du source+Champ Mnémonics et Opérandes du source) sont en fait les même que celles du source (donc, pas d'information supplémentaire).

Par contre les deux premières colonnes (Champ Adresses et Champ Code Machine) ont été rajoutées par l'assembleur, au moment de la compilation.

On voit que dans le champ des adresses, apparaît la valeur 0007 : cela veut dire que cette instruction se trouvera dans la case d'adresse 0x0007 de la mémoire de programme.

Cette adresse correspond d'ailleurs ici à l'étiquette (label) VERS_PORT.

Si, on écrivait, par exemple, à un autre endroit du source une instruction comme :

GOTO VERS_PORT

le label VERS_PORT serait automatiquement remplacé par 0x0007 par l'assembleur. Ce qui veut dire qu'on n'a pas à se préoccuper, en écrivant son programme source, des adresses d'implantation des instructions, puisque c'est l'assembleur qui leur affectera une valeur.

Cette case 0x0007 contiendra la valeur 0x0A85, située juste à sa droite (donc dans le Champ Code Machine).

Comme on vient de le voir à la page précédente cette valeur correspond bien au code de l'instruction :

INCF PORTA,F

(qui se trouve sur la même ligne dans le Champ Mnémonics et Opérandes du source).

Notons également que grâce à la ligne suivante, placée au début de votre programme source :

#include <p16f877a.inc> ; définition de tous les SFR du PIC 16F877

le programmeur n'aura pas à connaître les adresses de tous les registres SFR du PIC, car ce fichier d'inclusion fera le lien entre le nom du registre SFR utilisé (ici par exemple PORTA) et son adresse (ici 0x05), comme le montre un extrait de ce fichier donné ci-dessous :

```
....  
INDF      EQU  H'0000'  
TMR0     EQU  H'0001'  
PCL      EQU  H'0002'  
STATUS   EQU  H'0003'  
FSR      EQU  H'0004'  
PORTA    EQU  H'0005'  
PORTB    EQU  H'0006'  
PORTC    EQU  H'0007'  
....
```

La **directive d'assemblage EQU** (voir la **partie IV.4.** sur les directives d'assemblage) assure la correspondance entre un nom symbolique (ici le nom de tous les registres SFR du PIC) et son adresse (exprimée ici en hexadécimal par **H'xxxx'**).

Chaque fois que PORTA sera rencontré dans le source, l'assembleur le remplacera par 0x05.

On donnera d'autres directives d'assemblage un peu plus loin dans ce chapitre.

**Il est donc important, dans votre programme source,
(que vous programmez en assembleur ou en C)
de conserver les mêmes noms pour les SFR que ceux fournis
dans la documentation hardware du PIC,
et de ne pas « inventer » vos propres noms.**

On s'y retrouve mieux et tout est prévu pour que ça marche !!!

**Par contre, vous pouvez choisir les noms que vous voulez pour vos variables
(stockées en GPR).**

**En essayant quand même de prendre un nom de variable en rapport avec le rôle jouée par
cette variable dans votre programme.**

**Quand un étudiant choisit de nommer « toto » une variable qui sert de compteur,
je me dis qu'il n'a pas pris en compte cette remarque.**

**De même, les noms de labels (ou étiquettes) et les noms des sous-programmes
doivent aussi être « parlants » pour aider :**

- à la compréhension,
 - à la lisibilité,
 - à la mise au point ...
- de votre programme ...**

IV.3. DETAILS ET EXEMPLES D'INSTRUCTIONS

Le classement des instructions en 4 familles proposé par MICROCHIP n'est pas à mon avis le mieux adapté et le plus pertinent pour utiliser ces instructions.

Je préfère les classer en fonction du type de traitement qu'elles effectuent.

Ce nouveau classement me semble plus opérationnel quand on recherche une instruction.

On distinguera ainsi :

- ① les instructions de **transfert de données** :
pour les mouvements entre registres,
- ② les instructions de **traitement arithmétique** :
pour les additions, les soustractions, les in(dé)crémentations, les « mises à 0 »
- ③ les instructions de **traitement logique** :
AND, IOR, XOR, COM, rotations, permutations, positionnement de bit,
- ④ les instructions de **rupture synchrones de séquence** :
inconditionnelles et conditionnelles,
- ⑤ les **autres** instructions:
pour les modes de fonctionnement particuliers...

Le tableau de la figure IV.9. regroupe les 35 instructions précédentes, en utilisant ces 5 nouvelles familles.

Et c'est dans ce sens là que l'on présentera ensuite chacune des instructions, en essayant de donner à chaque fois un petit exemple et, quand cela sera possible une équivalence en langage C.

Il me semble en effet intéressant de tout de suite relier l'assembleur avec le C. C'est un moyen :

- soit tout simplement de **mieux comprendre** le langage C,
- soit de mieux saisir ce qui se passe **au niveau matériel** quand on programme en langage de **haut niveau**,
- soit, et là c'est le « top » : les **deux en même temps** ...

**La dernière colonne de ce nouveau tableau indique d'ailleurs
la page où vous trouverez des informations supplémentaires pour chaque instruction.**

Dans ce tableau, la notation **{d = f ou W}**, spécifiant la destination du résultat, signifie :
 - si **d = 1** ou **f** ou **d absent** (par défaut) alors **résultat dans f**,
 - si **d = 0** ou **d = W** alors **résultat dans W**.

Mnémonic		Opérande	Description	Flags	Cycle	Page
① INSTRUCTIONS DE TRANSFERT						
1	MOVLW	k	k → W		1	110
2	MOVWF	f	W → f		1	111
3	MOVF	f,d	f → {d = f ou W}	Z	1	112
② INSTRUCTIONS DE TRAITEMENT ARITHMETIQUE						
4	ADDLW	k	k + W → W	C DC Z	1	113
5	ADDWF	f,d	f + W → {d = f ou W}	C DC Z	1	114
6	SUBLW	k	k - W → W	C DC Z	1	115
7	SUBWF	f,d	f - W → {d = f ou W}	C DC Z	1	116
8	INCF	f,d	f + 1 → {d = f ou W}	Z	1	117
9	DECF	f,d	f - 1 → {d = f ou W}	Z	1	118
10	CLRW		W = 0	Z = 1	1	119
11	CLRF	f	f = 0	Z = 1	1	120
③ INSTRUCTIONS DE TRAITEMENT LOGIQUE						
12	ANDLW	k	k AND W → W	Z	1	121
13	ANDWF	f,d	f AND W → {d = f ou W}	Z	1	122
14	IORLW	k	k IOR W → W	Z	1	123
15	IORWF	f,d	f IOR W → {d = f ou W}	Z	1	124
16	XORLW	k	k XOR W → W	Z	1	125
17	XORWF	f,d	f XOR W → {d = f ou W}	Z	1	126
18	COMF	f,d	$\bar{f} \rightarrow \{d ? f : W\}$	Z	1	128
19	RLF	f,d	rotation gauche de f par C → {d = f ou W}	C	1	129
20	RRF	f,d	rotation droite de f par C → {d = f ou W}	C	1	130
21	SWAPF	f,d	permute les 2 quartets de f → {d = f ou W}		1	131
22	BCF	f,b	mise à zéro du bit b de f		1	132
23	BSF	f,b	mise à un du bit b de f		1	133
④ INSTRUCTIONS DE RUPTURE DE SEQUENCE						
INCONDITIONNELLES						
24	GOTO	k	saute toujours à l'adresse k		2	135
25	CALL	k	appel du sous-programme k		2	136
26	RETURN		retour de sous-programme		2	137
27	RETLW	k	retour de sous-programme avec W = k		2	138
28	RETFIE		retour d'interruption		2	139
CONDITIONNELLES						
29	DECFSZ	f,d	f - 1 → {d = f ou W} et si 0 saute <u>une</u> instruction		1(2)	140
30	INCFSZ	f,d	f + 1 → {d = f ou W} et si 0 saute <u>une</u> instruction		1(2)	143
31	BTFSS	f,b	teste bit b de f et si 1 saute <u>une</u> instruction		1(2)	144
32	BTFSC	f,b	teste bit b de f et si 0 saute <u>une</u> instruction		1(2)	146
⑤ AUTRES INSTRUCTIONS						
33	NOP		No OPération :		1	147
34	CLRWDI		reset du timer du watch dog	$\overline{TO} \quad \overline{PD}$	1	148
35	SLEEP		passage dans le mode « standby »	$\overline{TO} \quad \overline{PD}$	1	150

Figure IV.9.

IV.3.1. INSTRUCTIONS DE TRANSFERT

Ce sont les trois instructions de la famille des **MOV**, qu'il ne faut pas confondre entre elles car chacune correspond à un transfert particulier.

En pratique ce seront celles qui seront le plus utilisées, car tout programme a besoin de déplacer les données pour pouvoir les traiter dans son **UAL**.

1. MOVLW k (comme MOVe Literal (k) to W)

Elle permet **de mettre** le **Littéral (L)** k, c'est-à-dire une **constante**, dans le registre **W**.

Utilisé pour initialiser le registre **W**.

Exemple en assembleur :

On peut le voir sur la figure IV.10.

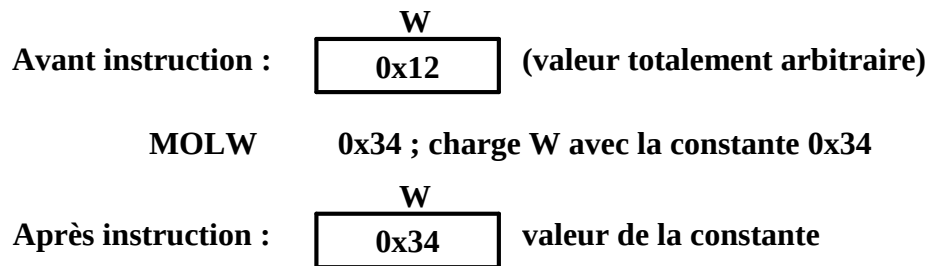


Figure IV.10.

Pas d'équivalent direct en C, car pas d'accès direct à **W** avec le langage C.

2. MOVWF f (comme MOVe W to File f)

Elle permet **de transférer** (copier) le contenu du registre **W** dans le registre d'adresse **f** de la RAM (**f** est un **SFR** ou un **GPR**), comme on peut en voir le principe de la figure IV.11. Ce qui correspond à un **cycle d'écriture** en RAM.

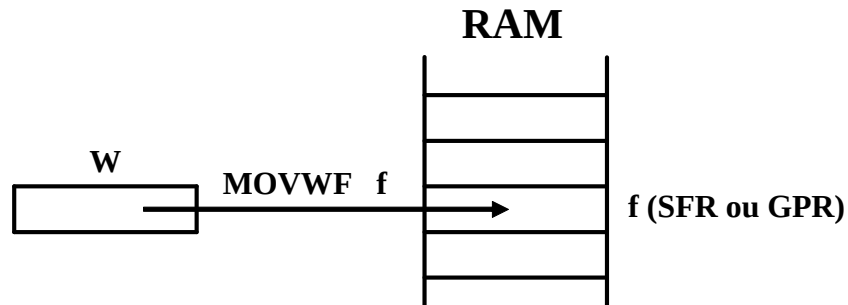


Figure IV.11.

Exemple en assembleur :

On peut le voir sur la figure IV.12. où le PORTB est supposé configuré en sortie :

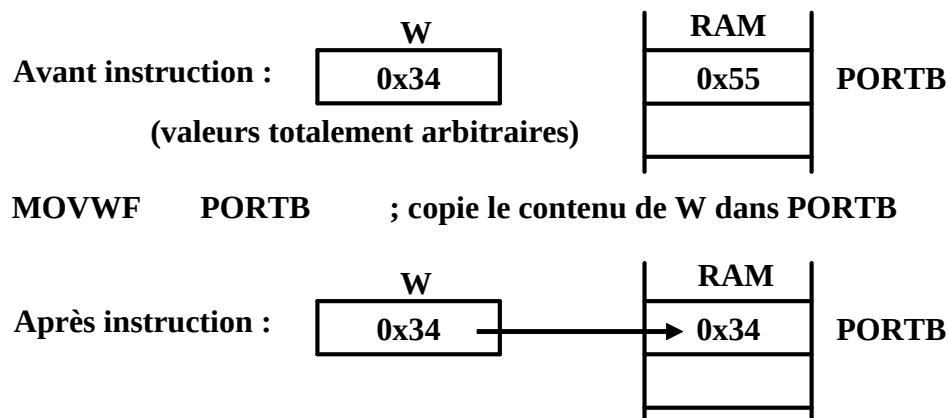


Figure IV.12.

On ne peut directement initialiser un **SFR** ou un **GPR** par une constante. Il faut passer par **W**.

Exemple en langage C :

Les 2 instructions assembleur suivantes permettant d'écrire dans le PORTB (configuré préalablement en sortie) :

```
MOVLW    0x34    ; constante 0x34 dans W
MOVWF    PORTB   ; copie W dans PORTB
```

seront équivalentes à la ligne en C suivante :

```
PORTB = 0x34 ;           // sort 0x34 sur le PORTB
```

3. MOVF f,d (comme MOVe File f to d)

Elle permet **de transférer** (copier) le contenu du registre d'adresse **f** de la RAM (**f** est un **SFR** ou un **GPR**), dans la destination **d**. Si **d = 0** (ou **W**), alors la destination est le registre **W**, comme on peut le voir sur le schéma de principe de la figure IV.13. C'est un **cycle de lecture** de la RAM.

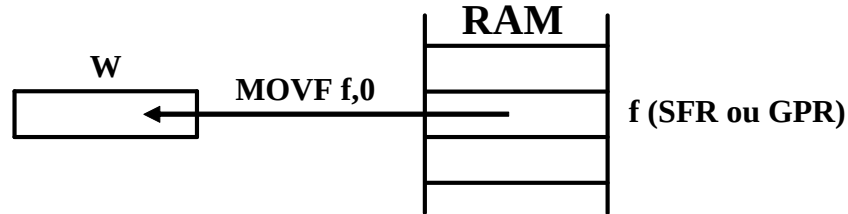


Figure IV.13.

Exemple en assembleur :

On peut le voir sur la figure IV.14. où le PORTB est supposé configuré en entrée :

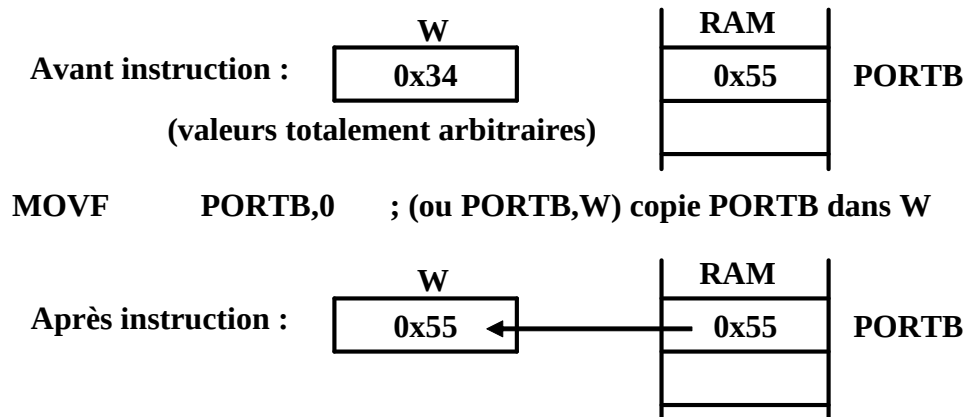


Figure IV.14.

Exemple en langage C :

Les 2 instructions assembleur suivantes, où **input** est une variable stockée dans un GPR, permettent de lire le PORTB (configuré préalablement en entrée) dans la variable **input** :

```
input          :          1          ; réservation de 1 octet en RAM pour la
                                   ; variable (en GPR) input
...
               MOVF      PORTB,W      ; copie PORTB dans W
               MOVWF     input         ; copie W dans la variable input
```

seront équivalentes, en langage C, à :

```
...           unsigned char input ;      // variable input de type char (taille 1 octet)
               input = PORTB ;           // lecture du PORTB dans input
```

Remarque :

Une instruction comme : **MOVF PORTB ; (ou PORTB,1 ou PORTB,f)**

semble inutile puisqu'elle copie PORTB (ou tout autre registre) sur lui-même, ce qui à priori ne sert à rien. En réalité, cette instruction va positionner le flag **Z**, ce qui permettra par exemple de tester **juste après** si le contenu de n'importe quel registre est nul !!!

IV.3.2. INSTRUCTIONS DE TRAITEMENT ARITHMETIQUE

Ce sont les instructions de la famille des **ADD, SUB, INC, DEC** et **CLR**.

En pratique, elles serviront pour les traitements arithmétiques de base.

Mais elles permettront aussi, en les associant de façon efficace, de réaliser des opérations plus complexes comme les multiplications ou les divisions par exemple.

4. ADDLW k (comme ADD Literal (k) and W)

Elle permet **d'additionner** le littéral (L) **k**, c'est-à-dire une **constante**, au contenu du registre **W**, le résultat de cette addition étant mis dans **W**.

Exemple en assembleur :

On peut le voir sur la figure IV.15.

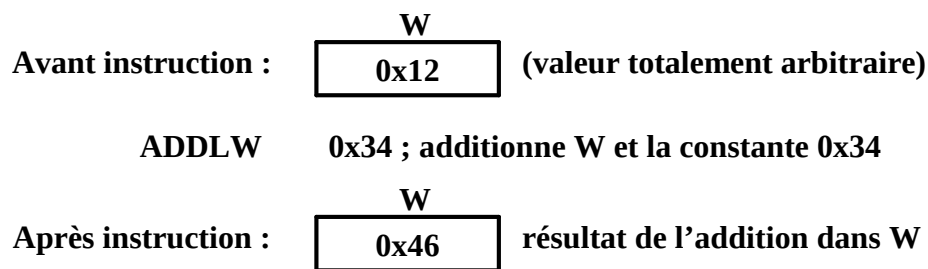


Figure IV.15.

Les flags **C**, **DC** et **Z** seront positionnés suivant le résultat obtenu dans **W**.

Rappel :

- si résultat nul alors $Z = 1$,
- si résultat ≥ 256 (ne tient pas sur un octet) alors $C = 1$,
- si résultat < 256 (tient sur un octet) alors $C = 0$,
- si retenue entre bit3 et bit4 $DC = 1$ sinon $DC = 0$.

Pas d'équivalent direct en C, car pas d'accès direct à W avec le langage C.

5. ADDWF f,d (comme ADD W and File f to d)

Elle permet d'**additionner** les contenus de **W** et de **f** en plaçant le résultat :

- soit dans f si d = 1 (ou d = f ou si d est omis),
- soit dans W si d = 0 (ou si d = W).

Exemple en assembleur :

On peut le voir sur la figure IV.16. où i est une variable (en **GPR**) de taille 1 octet :

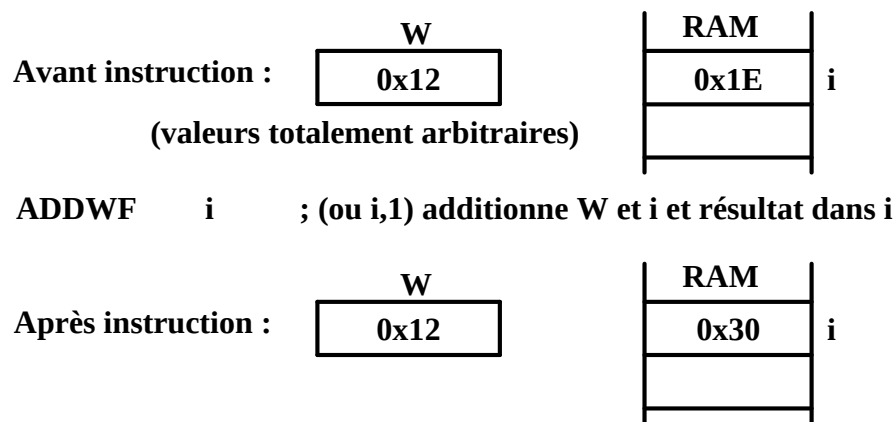


Figure IV.16.

Les flags **C**, **DC** et **Z** seront positionnés suivant le résultat de cette addition et comme indiqué dans le rappel de la page précédente.

Exemple en langage C :

L'exemple précédent de la figure IV.16. s'écrit tout simplement en C :

```
i = i + 18;           // additionne 18 (0x12) à i
ou
i += 18;              // idem
```

6. SUBLW k (comme SUBstract W from Literal k)

Elle permet **de soustraire** le contenu de **W** du littéral (**L**) **k**, c'est-à-dire d'une **constante**, le résultat de cette soustraction étant mis dans **W**. On utilise pour cela le complément à 2.

☛☛ : on fait **k - W** et pas **W - k** !!!

Du coup, cette instruction aurait du plutôt s'appeler **SUBWL** !!!

Exemple en assembleur :

On peut le voir sur la figure IV.17.

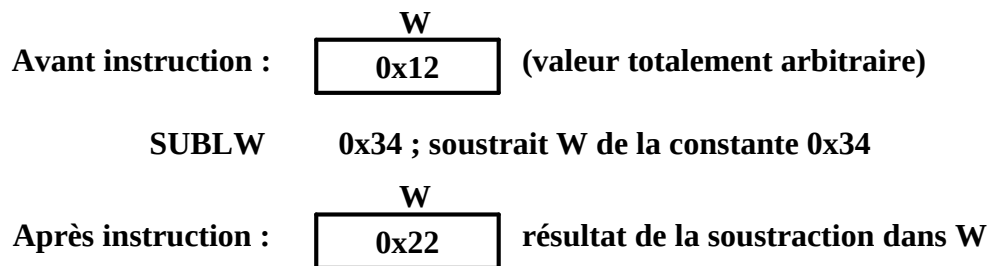


Figure IV.17.

Les flags **C**, **DC** et **Z** seront positionnés suivant le résultat obtenu dans **W**.

Rappel :

- si résultat nul alors Z = 1,
- si résultat ≥ 0 alors C = 1,
- si résultat < 0 alors C = 0.
- si retenue entre bit3 et bit4 DC = 0 sinon DC = 1.

Pas d'équivalent direct en C, car pas d'accès direct à W avec le langage C.

7. SUBWF f,d (comme SUBstract W from File f to d)

Elle permet **de soustraire** le contenu de **W** de celui de **f** en plaçant le résultat :

- soit dans f si d = 1 (ou d = f ou si d est omis),
- soit dans W si d = 0 (ou si d = W).

☛☛ : là aussi, on fait **f - W** et pas **W - f** !!!

Exemple en assembleur :

On peut le voir sur la figure IV.18. où i est une variable (en **GPR**) de taille 1 octet :

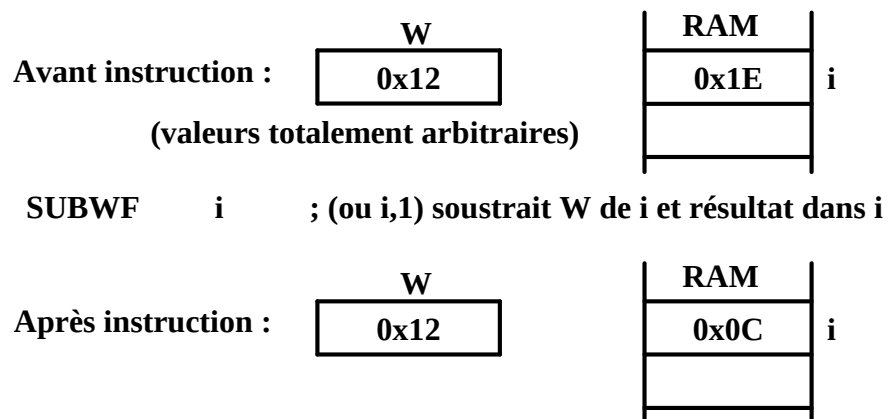


Figure IV.18.

Les flags **C**, **DC** et **Z** seront positionnés suivant le résultat de cette soustraction et comme indiqué dans le rappel de la page précédente.

Exemple en langage C :

L'exemple précédent de la figure IV.18. s'écrit tout simplement en C :

```
                         i = i - 18;                    // soustrait 18 (0x12) de i  
ou                        i -= 18                    // idem
```

8. INCF f,d (comme INCrémente File f to d)

Elle permet **d'incrémenter** (ajouter 1) le contenu du registre **f** en plaçant le résultat :

- soit dans f si d = 1 (ou d = f ou si d est omis),
- soit dans W si d = 0 (ou si d = W).

Exemple en assembleur :

On peut le voir sur la figure IV.19. où j est une variable (en **GPR**) de taille 1 octet :

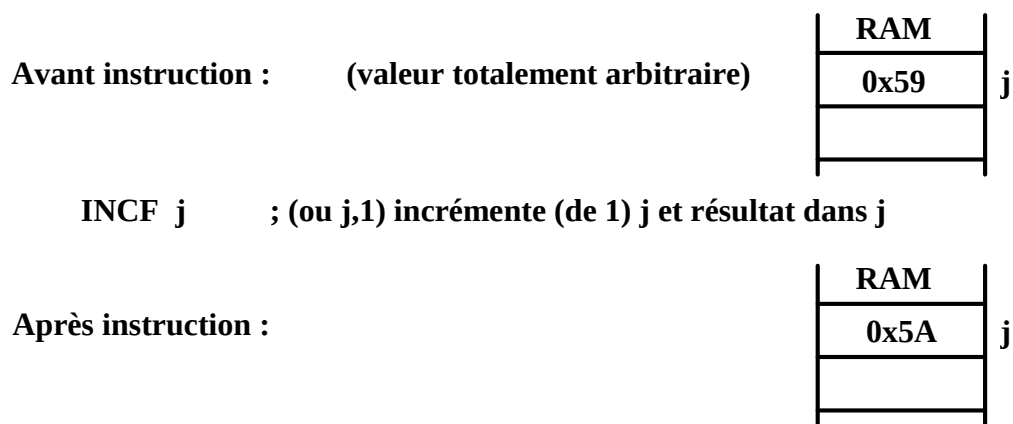


Figure IV.19.

Le flag **Z** est le seul à être positionné à 1 lorsque l'on passe de 0xFF à 0x00. Sinon il est mis à 0.

(l'information fournie par le flag **C** serait la même et donc redondante car c'est aussi le seul cas où C passerait à 1).

Remarque :

Pour ce même cas, si on avait écrit : **INCF j,W ; (ou j,0)**

alors le résultat, soit ici 0x5A aurait été mis dans **W** et la valeur de j n'aurait pas été changée, soit ici 0x59.

Exemple en langage C :

L'exemple précédent de la figure IV.19. s'écrirait tout simplement en C :

j++ ; // j = j + 1

9. DECF f,d (comme DECrémente File f to d)

Elle permet **de décrémenter** (enlever 1) au contenu du registre **f** en plaçant le résultat :

- soit dans f si d = 1 (ou d = f ou si d est omis),
- soit dans W si d = 0 (ou si d = W).

Exemple en assembleur :

On peut le voir sur la figure IV.20. où j est une variable (en **GPR**) de taille 1 octet :

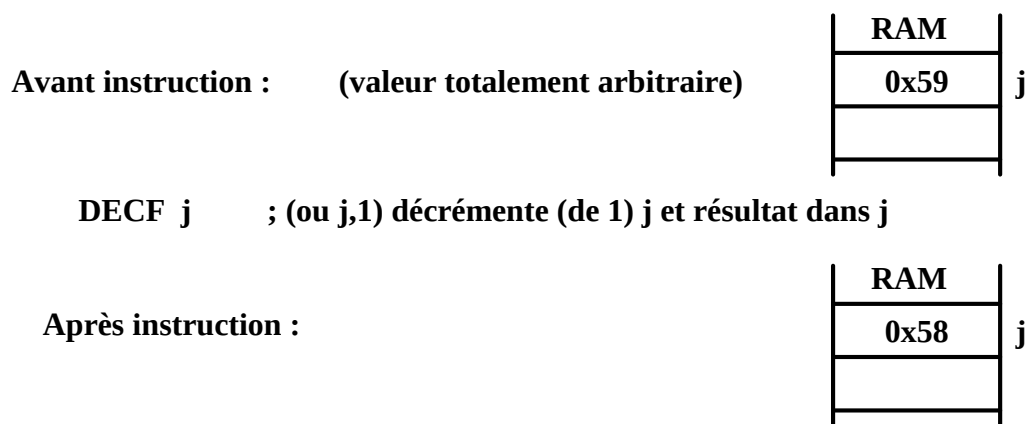


Figure IV.20.

Le flag **Z** est le seul à être positionné à 1 lorsque l'on passe de 0x01 à 0x00. Sinon il est mis à 0.

(l'information fournie par le flag **C** serait la même et donc redondante car c'est aussi le seul cas où C passerait à 1).

Remarque :

Pour ce même cas, si on avait écrit : **DECF j,W ; (ou j,0)**

alors le résultat, soit ici 0x58 aurait été mis dans **W** et la valeur de j n'aurait pas été changée, soit ici 0x59.

Exemple en langage C :

L'exemple précédent de la figure IV.20. s'écrit tout simplement en C :

j-- ; // j = j - 1

10. CLRW (comme **CLeaR W**)

Cette instruction permet « **d’effacer** » le registre **W**, c’est-à-dire le mettre à **0x00**.

De plus le flag **Z** est systématiquement mis à **1**.

Exemple en assembleur :

On peut le voir sur la figure IV.21.

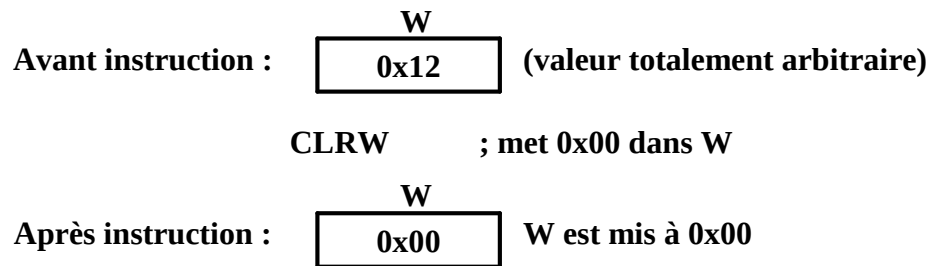


Figure IV.21.

Pas d’équivalent direct en C, car pas d’accès direct à W avec le langage C.

11. CLRF f (comme CLearN File f)

Cette instruction permet « **d’effacer** » le registre (SFR ou GPR) **f**, c’est-à-dire le mettre à **0x00**.

De plus le flag **Z** est systématiquement mis à 1.

Exemple en assembleur :

On peut le voir sur la figure IV.22. où k est une variable (en **GPR**) de taille 1 octet :

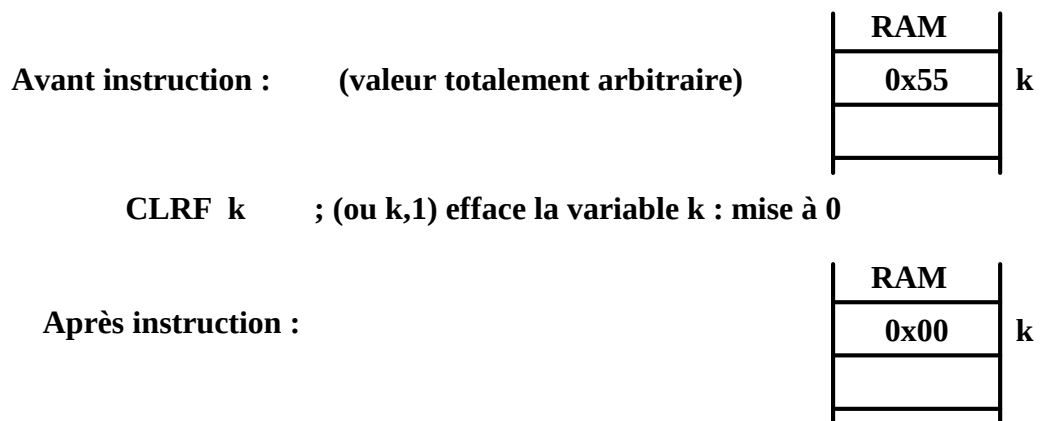


Figure IV.22.

Exemple en langage C :

L'exemple précédent de la figure IV.22. s'écrirait tout simplement en C :

```
k = 0;           // mise à 0 de k
```


IV.3.3. INSTRUCTIONS DE TRAITEMENT LOGIQUE

Ce sont les instructions de la famille des **AND**, **IOR**, **XOR**, **COM**, **RxF**, **SWAP** et **BxF**.

(reportez-vous au tableau donné au **chapitre I.2.** si vous avez oublié comment fonctionnent ces opérateurs logiques de base !!!).

En pratique, elles serviront pour les traitements logiques de base :

- techniques de masquage,
- techniques de décalage et/ou de rotation,
- techniques de « swapping »,
- action sur des bits individuels d'un registre **SFR** (cas fréquent) ou d'une variable **GPR**.

12. ANDLW k (comme AND Literal (k) with W)

Elle permet **de faire un ET logique « bit à bit »** entre le littéral (**L**) **k**, c'est-à-dire une **constante**, et le contenu du registre **W**, le résultat de cette opération étant mis dans **W**.

Le littéral **k** est souvent dénommé un **masque**.

Exemple en assembleur :

On peut le voir sur la figure IV.23. où le contenu du registre **W** et la valeur du masque (ici 0xF0) ont été représentés en binaire pour une meilleure compréhension de la fonction **AND** « bit à bit ».

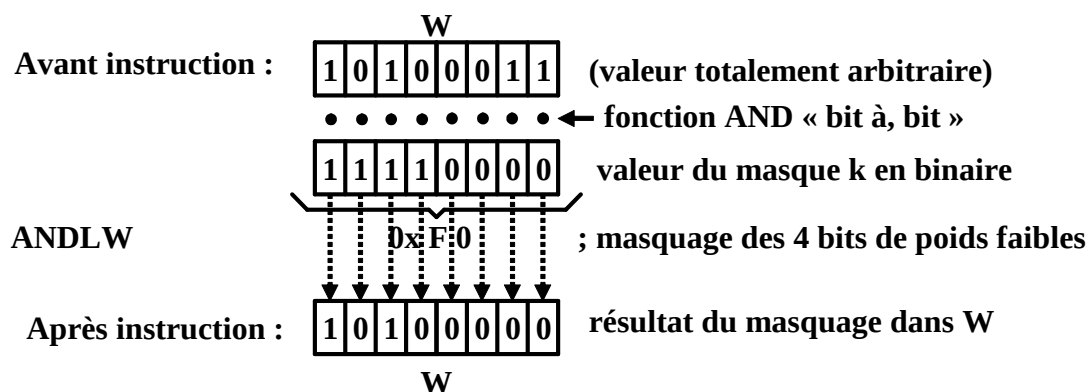


Figure IV.23.

On voit que, à cause de la propriété du ET logique : $1 \cdot x = x$ et $0 \cdot x = 0$:

- les bits de **W** qui sont au même poids que les bits à 1 du masque seront inchangés,
- les bits de **W** qui sont au même poids que les bits à 0 du masque seront mis à 0.

Un ET « bit à bit » permet donc de forcer à 0 certains bits sans toucher les autres, en fonction de la valeur choisie pour le masque.

Seul le flag **Z** sera positionné suivant le résultat obtenu dans **W**.

Pas d'équivalent direct en C, car pas d'accès direct à **W** avec le langage C.

13. ANDWF f,d (comme AND W with File f to d)

Elle permet **de faire un ET logique « bit à bit »** entre les contenus de **W** et de **f** en plaçant le résultat :

- soit dans **f** si **d = 1** (ou **d = f** ou si **d** est omis),
- soit dans **W** si **d = 0** (ou si **d = W**).

Exemple en assembleur :

On peut le voir sur la figure IV.24. où les contenus des registres **W** et **f** ont été représentés en binaire pour une meilleure compréhension de la fonction **AND** « bit à bit ». Dans cet exemple, le masque est dans **W** ce qui permet (contrairement au cas précédent) de faire des masques variables et le résultat du masquage est mis dans **f**. Ici, en masquant tous les bits, sauf le Bit 0, on peut détecter la parité d'une variable située dans le registre **f**.

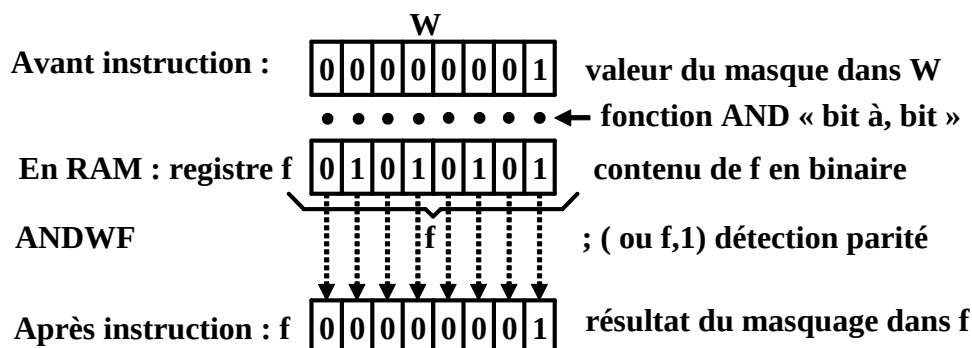


Figure IV.24.

On voit que, à cause de la propriété du ET logique : $1 \cdot x = x$ et $0 \cdot x = 0$:

- les bits de **f** qui sont au même poids que les bits à 1 du masque seront inchangés,
- les bits de **f** qui sont au même poids que les bits à 0 du masque seront mis à 0.

Un ET « bit à bit » permet donc de forcer à 0 certains bits sans toucher les autres, en fonction de la valeur choisie pour le masque.

Seul le flag **Z** sera positionné suivant le résultat obtenu dans **d**.

Exemple en langage C :

Les 2 instructions assembleur suivantes, où **car** est une variable de taille 1 octet stockée dans un **GPR** en RAM, permettent de passer du code ASCII au codage DCB sur 1 octet.

```
car          DB          0x35          ; car initialisée avec code ASCII de 5
...
              MOVLW      0x0F          ; masque quartet fort dans W
              ANDWF       car          ; ASCII vers décimal
```

seront équivalentes, en langage C, à :

```
...          unsigned char car = '5';    // variable car contenant code ASCII de 5
...          car = car & 0x0F;           // masquage de car par opérateur « bit » &
```

14. IORLW k (comme Inclusive OR Literal (k) with W)

Elle permet **de faire un OU inclusif logique « bit à bit »** entre le littéral (L) **k**, c'est-à-dire une **constante**, et le contenu du registre **W**, le résultat de cette opération étant mis dans **W**.

Le littéral **k** est souvent dénommé un **masque**.

Exemple en assembleur :

On peut le voir sur la figure IV.25. où le contenu du registre **W** et la valeur du masque (ici 0x3C) ont été représentés en binaire pour une meilleure compréhension de la fonction **IOR « bit à bit »**.

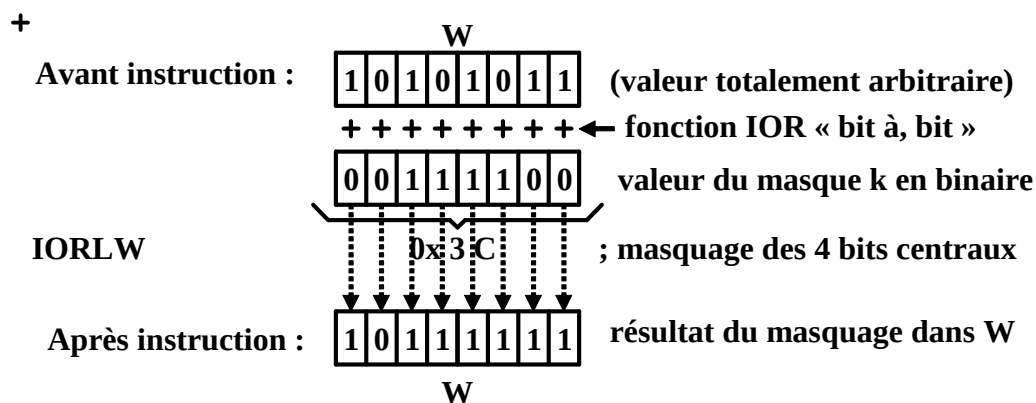


Figure IV.25.

On voit que, à cause de la propriété du IOR logique : $1 + x = 1$ et $0 + x = x$:

- les bits de **W** qui sont au même poids que les bits à 0 du masque seront inchangés,
- les bits de **W** qui sont au même poids que les bits à 1 du masque seront mis à 1.

Un IOR « bit à bit » permet donc de forcer à 1 certains bits sans toucher les autres, en fonction de la valeur choisie pour le masque.

Seul le flag **Z** sera positionné suivant le résultat obtenu dans **W**.

Pas d'équivalent direct en C, car pas d'accès direct à **W** avec le langage C.

15. IORWF f,d (comme Inclusif OR W with File f to d)

Elle permet **de faire un OU inclusif logique « bit à bit »** entre les contenus de **W** et de **f** en plaçant le résultat :

- soit dans **f** si **d = 1** (ou **d = f** ou si **d** est omis),
- soit dans **W** si **d = 0** (ou si **d = W**).

Exemple en assembleur :

On peut le voir sur la figure IV.26. où les contenus des registres **W** et **f** ont été représentés en binaire pour une meilleure compréhension de la fonction **IOR** « bit à bit ». Dans cet exemple, le masque est dans **W** ce qui permet (contrairement au cas précédent) de faire des masques variables et le résultat du masquage est mis dans **f**. Ici, on force les 2 bits de poids faibles de **f** à 1.

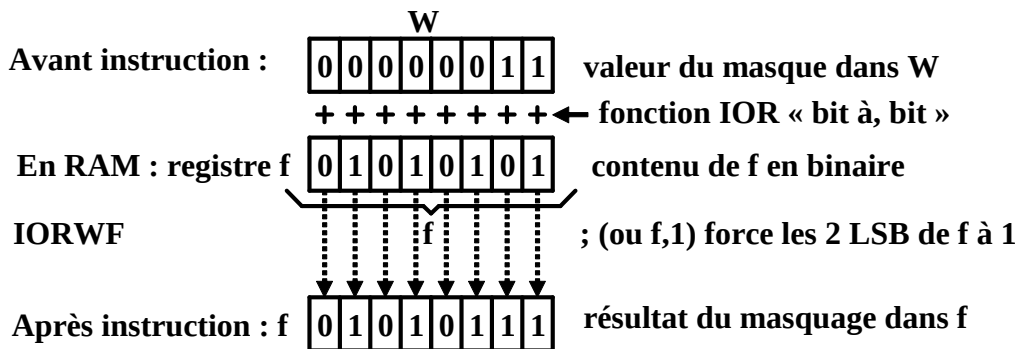


Figure IV.26.

On voit que, à cause de la propriété du IOR logique : $1 + x = 1$ et $0 + x = x$:

- les bits de **f** qui sont au même poids que les bits à 0 du masque seront inchangés,
- les bits de **f** qui sont au même poids que les bits à 1 du masque seront mis à 1.

Un IOR « bit à bit » permet donc de forcer à 1 certains bits sans toucher les autres, en fonction de la valeur choisie pour le masque.

Seul le flag **Z** sera positionné suivant le résultat obtenu dans **d**.

Exemple en langage C :

L'exemple de la figure IV.26. se traduit en langage C par :

```
... unsigned char var; // variable var de type char (taille 1 octet)
... var = var | 0x03; // mise à 1 des 2 LSB de var par opérateur « bit » |
```

16. XORLW k (comme eXclusive OR Literal (k) with W)

Elle permet **de faire un OU exclusif logique « bit à bit »** entre le littéral (**L**) **k**, c'est-à-dire une **constante**, et le contenu du registre **W**, le résultat de cette opération étant mis dans **W**.

Le littéral **k** est souvent dénommé un **masque**.

Exemple en assembleur :

On peut le voir sur la figure IV.27. où le contenu du registre **W** et la valeur du masque (ici 0x0F) ont été représentés en binaire pour une meilleure compréhension de la fonction **XOR** « bit à bit ».

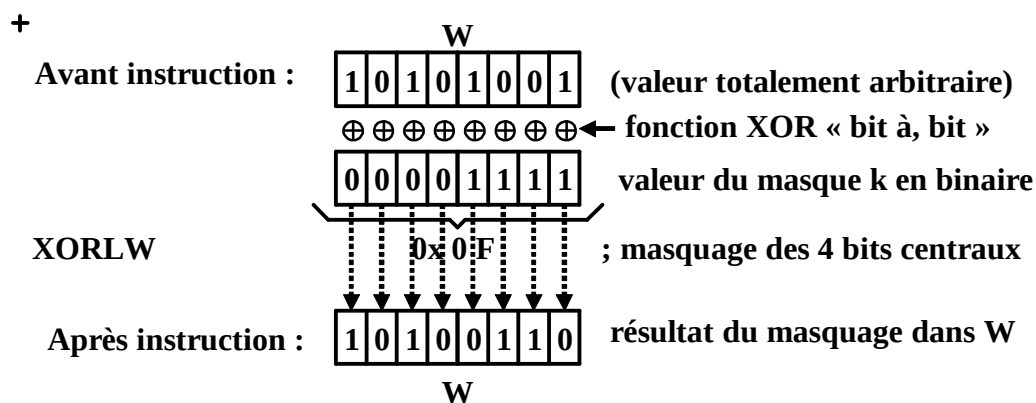


Figure IV.27.

On voit que, à cause de la propriété du XOR logique : $1 \oplus x = \bar{x}$ et $0 \oplus x = x$:

- les bits de **W** qui sont au même poids que les bits à 0 du masque seront inchangés,
- les bits de **W** qui sont au même poids que les bits à 1 du masque seront inversés.

Un XOR « bit à bit » permet d'inverser certains bits sans toucher les autres, en fonction de la valeur choisie pour le masque.

Si on utilise le masque 0xFF (tous les bits à 1), alors tous les bits de **W** seront inversés.

Seul le flag **Z** sera positionné suivant le résultat obtenu dans **W**.

Pas d'équivalent direct en C, car pas d'accès direct à **W** avec le langage C.

17. XORWF f,d (comme eXclusif OR W with File f to d)

Elle permet **de faire un OU exclusif logique « bit à bit »** entre les contenus de **W** et de **f** en plaçant le résultat :

- soit dans **f** si **d = 1** (ou **d = f** ou si **d** est omis),
- soit dans **W** si **d = 0** (ou si **d = W**).

Exemple en assembleur :

On peut le voir sur la figure IV.28. où les contenus des registres **W** et **f** ont été représentés en binaire pour une meilleure compréhension de la fonction **XOR** « bit à bit ». Dans cet exemple, le masque est dans **W** ce qui permet (contrairement au cas précédent) de faire des masques variables et le résultat du masquage est mis dans **f**. Ici, on inverse uniquement le bit de poids fort de **f**.

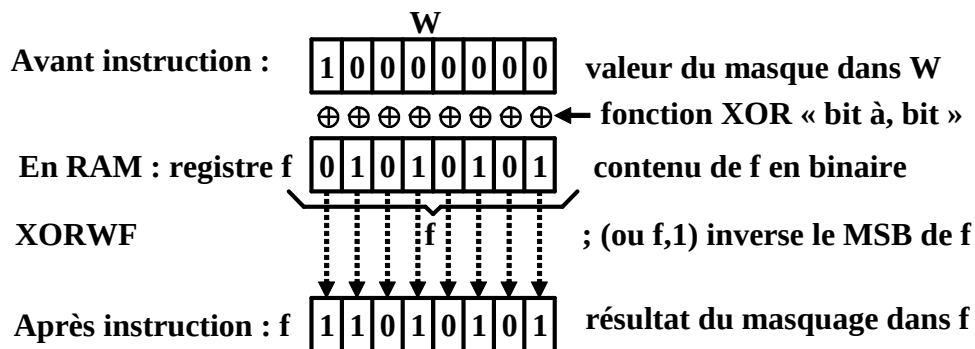


Figure IV.28.

On voit que, à cause de la propriété du XOR logique : $1 \oplus x = \bar{x}$ et $0 \oplus x = x$:

- les bits de **f** qui sont au même poids que les bits à 0 du masque seront inchangés,
- les bits de **f** qui sont au même poids que les bits à 1 du masque seront inversés.

Un XOR « bit à bit » permet d'inverser certains bits sans toucher les autres, en fonction de la valeur choisie pour le masque.

Seul le flag **Z** sera positionné suivant le résultat obtenu dans **d**.

Exemple en langage C :

Connaître la « distance de Hamming » entre deux mots mot1 et mot2 stockés en **GPR** (c'est-à-dire le nombre de bits différents entre ces deux mots).

Le principe en est donné sur la figure IV.29.

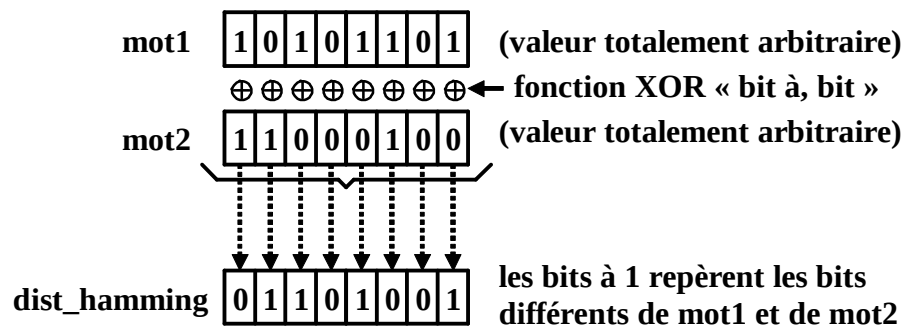


Figure IV.29.

Les 3 instructions assembleur suivantes :

```
mot1          DB      B'10101101' ; mot1 initialisé à 0xAD
mot2          DB      B'11000100' ; mot2 initialisé à 0xC4
dist_hamming  DB      0           ; dist_hamming initialisé à 0x00
...

      MOVE      mot1,W           ; copie mot1 dans W
      XORWF     mot2,W           ; 2 bits différents donnent le bit à 1 dans W
      MOWF      dist_hamming; les bits à 1 indiquent les bits différents
                                ; de mot1 et de mot2
```

seront équivalentes, en langage C, à :

```
unsigned char mot1 = 0xAD,           // déclaration et initialisation des
    mot2 = 0xC4,                     //variables de taille 1 octet
    dist_hamming = 0 ;

...

dist_hamming = mot1 ^ mot2 ;         // opérateur « bit » ^ du langage C
```

18. COMF f,d (comme COMplément File f to d)

Elle permet **d'inverser tous les bits** (complément à 1) du contenu de **f** en plaçant le résultat :

- soit dans f si d = 1 (ou d = f ou si d est omis),
- soit dans W si d = 0 (ou si d = W).

Exemple en assembleur :

On peut le voir sur la figure IV.30. où le file **f**, correspondant à une variable appelée **on**, a été représenté en binaire pour une meilleure compréhension de la fonction complément à 1.

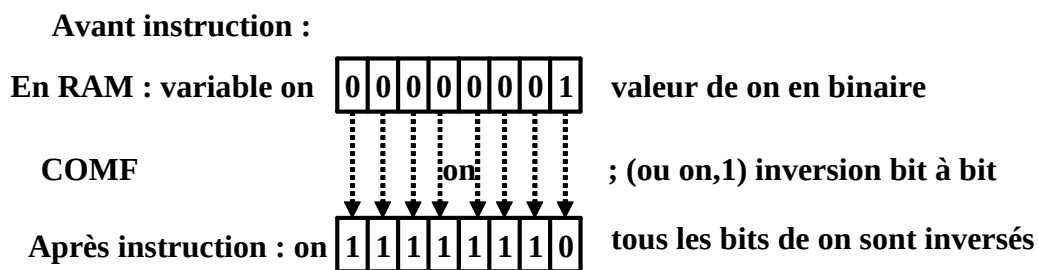


Figure IV.30.

Seul le flag **Z** sera positionné suivant le résultat obtenu dans **d**.

Exemple en langage C :

L'exemple de la figure IV.30. se traduit en langage C par :

```
...          unsigned char on;    // variable on de type char (taille 1 octet)

on = 1 ;     // initialisation de la variable on
on = ~ on ;  // inversion de tous les bits de on par
              // l'opérateur « bit » ~ du langage C
```


19. RLF f,d (comme Rotate Left File f to d)

Elle permet **d'effectuer une rotation de un bit sur la gauche** du contenu de **f**, en passant par le flag **C**, et en plaçant le résultat :

- soit dans **f** si **d = 1** (ou **d = f** ou si **d** est omis),
- soit dans **W** si **d = 0** (ou si **d = W**).

Exemple en assembleur :

Il faut considérer alors l'ensemble « **f + C** » comme un registre à décalage « sur 9 bits ».

Le flag **C** remplace le **LSB** de **f**, tous les bits de **f** sont décalés d'une position sur la gauche et le bit sortant par la gauche de **f** se retrouve dans le flag **C**, comme indiqué sur la figure IV.31.

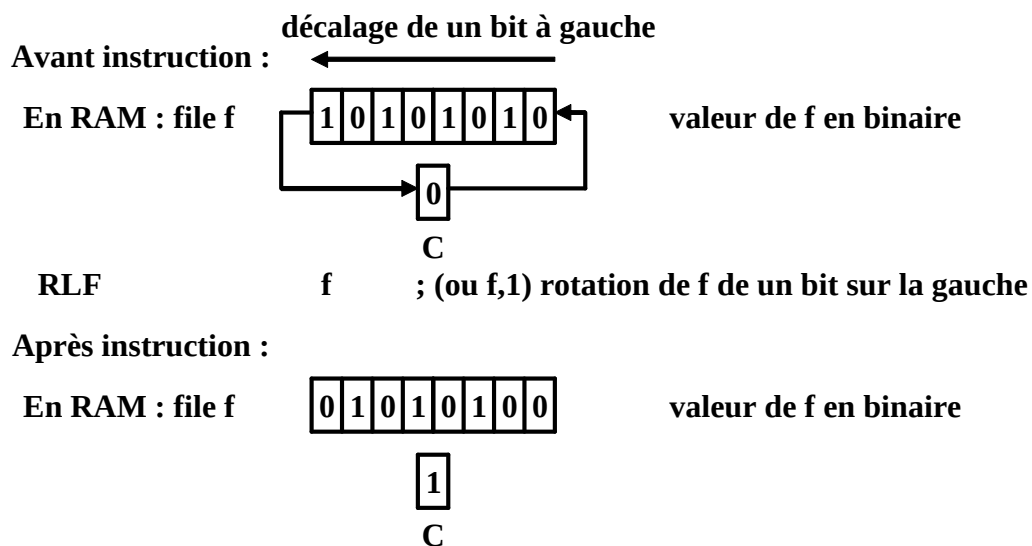


Figure IV.31.

En binaire non signé, si le flag **C** et le **MSB** de **f** sont initialement tous les deux 0 avant la rotation de un bit sur la gauche, alors le nouveau contenu de **f** représente le double de son contenu initial. (≅ multiplication par deux !!!)

Seul le flag **C** sera positionné suivant le **MSB** de **f** avant la rotation.

Exemple en langage C :

Cette instruction est à mettre en lien avec l'opérateur **décalage à gauche <<** du C. Mais dans le cas du C, ce décalage de **n** bits sur la gauche se fait comme si la variable décalée était un registre à décalage : les **n** bits de poids fort sont perdus, les **n** bits de poids faible sont remplacés par des 0 et les autres bits sont décalés de **n** positions sur la gauche, comme sur l'exemple ci-dessous :

```
unsigned char i = 0xFF ;    // les 8 bits de i à 1 : i = 0b 1111 1111
...
i = i << 4 ;                // décalage de i de 4 bits à gauche : i = 0xF0 = 0b 1111 0000
i = i << 2 ;                // décalage de i de 2 bits à gauche : i = 0xC0 = 0b 1100 0000
```

20. RRF f,d (comme Rotate Right File f to d)

Elle permet **d'effectuer une rotation de un bit sur la droite** du contenu de **f**, en passant par le flag **C**, et en plaçant le résultat :

- soit dans **f** si **d = 1** (ou **d = f** ou si **d** est omis),
- soit dans **W** si **d = 0** (ou si **d = W**).

Exemple en assembleur :

Il faut considérer alors l'ensemble « **f + C** » comme un registre à décalage « sur 9 bits ».

Le flag **C** remplace le **MSB** de **f**, tous les bits de **f** sont décalés d'une position sur la droite et le bit sortant par la droite de **f** se retrouve dans le flag **C**, comme indiqué sur la figure IV.32.

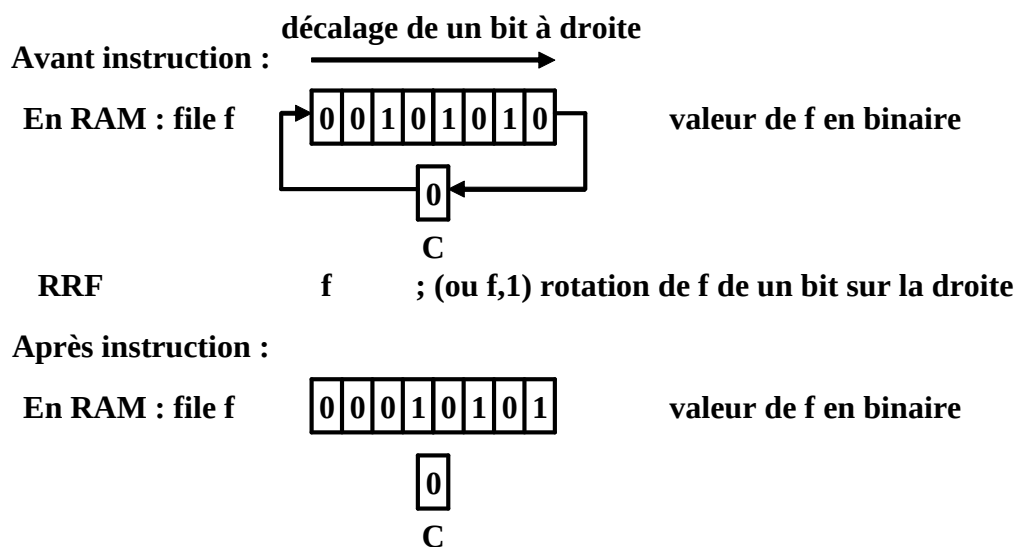


Figure IV.32.

En binaire non signé, si le flag **C** est initialement à 0 avant la rotation de un bit sur la droite, alors le nouveau contenu de **f** représente le quotient de la division entière de **f** par deux.

Exemple en langage C :

Cette instruction est à mettre en lien avec l'opérateur **décalage à droite >>** du C.

Dans le cas du C, pour les décalages à droite de **n** bits, il faut envisager deux cas :

- si la variable a été déclarée en **unsigned** : les **n** bits de poids fort sont remplacés par des 0, les **n** bits de poids faible sont perdus et les autres bits sont décalés de **n** positions sur la droite,

```
unsigned char i = 0xF0 ;        // les 8 bits de i à 1 : i = 0b 1111 0000
```

...

```
  i = i >> 2 ;                    // décalage de i de 2 bits à droite : i = 0x3C = 0b 0011 1100
```

- si la variable a été déclarée en **signed** : les **n** bits de poids fort sont égaux au bit de signe initial de la variable (le bit de signe s'est propagé sur **n** bits), les **n** bits de poids faible sont perdus et les autres bits sont décalés de **n** positions sur la droite,

```
signed char i = 0xF0 ;        // les 4 bits MSB de i à 1 : i = 0b 1111 0000
```

...

```
  i = i >> 2 ;                    // décalage de i de 2 bits à droite : i = 0xFC = 0b 1111 1100
```

21. SWAPF f,d (comme SWAP File f to d)

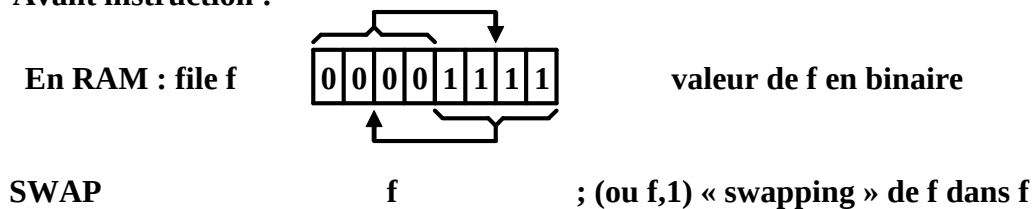
Elle permet **d'effectuer un échange** (une permutation) des **quatre bits de poids faibles** avec les **quatre bits de poids forts** du contenu de **f** en plaçant le résultat :

- soit dans f si d = 1 (ou d = f ou si d est omis),
- soit dans W si d = 0 (ou si d = W).

Exemple en assembleur :

Les quartets de poids faible et fort de **f** sont échangés comme indiqué sur la figure IV.33.

Avant instruction :



Après instruction :

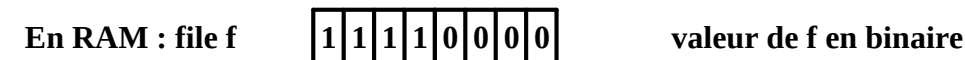


Figure IV.33.

Aucun flag n'est affecté par cette instruction.

Exemple en langage C :

Il n'existe pas d'instructions en C pour réaliser cette permutation entre les deux quartets d'une variable. On pourrait par exemple utiliser la fonction « équivalente en C » `swapf` décrite ci-dessous :

```
char swapf (unsigned char i)
{
    unsigned char j;
    j = i;
    i = i << 4;
    j = j >> 4;
    i = i + j;
    return(i);
}
unsigned (ou signed) char i;
...
    i = swapf ( i );
```

22. BCF f,b (comme Bit Clear in File (f) : bit b)

Cette instruction est orientée bit. Elle permet **de mettre à 0** le bit **b** (de 0 à 7) du file **f**.

Ce type d'instruction sera fréquemment utilisé car on est souvent amené à modifier un seul bit dans un des **SFR** du PIC.

Exemple en assembleur :

En assembleur, avant d'accéder à un registre interne (**SFR**) ou à une variable (**GPR**), il faut **toujours** s'assurer que les bits **RP1** et **RP0** du registre **STATUS** indiquent la bank où se trouve ce **SFR** ou ce **GPR** (cf. adressage direct vu au chapitre **III.2.1.1.**)

Par exemple, pour indiquer la bank 0, il faudra mettre ces bits **RP1** et **RP0** à 0, comme indiqué sur la figure IV.34.

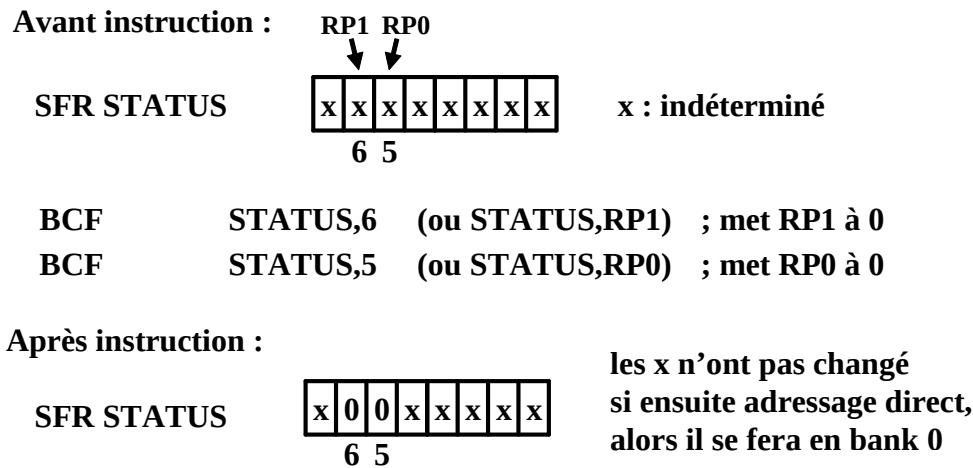


Figure IV.34.

Aucun flag n'est affecté par cette instruction.

Exemple en langage C :

L'instruction assembleur suivante permet de mettre à 0 la broche 7 du **PORTB**, c'est-à-dire la broche **RB7** (configurée préalablement en sortie) :

BCF PORTB,7 ; 0 V sur la ligne 7 du PORTB

sera équivalente à la ligne en C suivante :

RB7 = 0 ; // 0V sur la ligne 7 du PORTB

Remarque : le couple compilateur/linker du C assure, grâce à la ligne ci-dessous :

#include <pic.h>

la correspondance entre les noms des broches (ici **RB7**) et la **ligne 7 du PORTB**. Et ceci est vrai pour tous les registres **SFR** du PIC et pour tous leurs bits individuels, en gardant bien sûr les mêmes noms que ceux fournis dans le datasheet du PIC (**ne pas inventer ses propres noms !!!**). Ce fichier d'inclusion permet donc de faire le lien entre le hardware (mapping) et le software.

23. BSF **f,b** (comme Bit Set in File (f) : bit b)

Cette instruction est orientée bit. Elle permet **de mettre à 1** le bit **b** (de 0 à 7) du file **f**.

Ce type d'instruction sera fréquemment utilisé car on est souvent amené à modifier un seul bit dans un des **SFR** du PIC.

Exemple en assembleur :

En assembleur, avant d'accéder à un registre interne (**SFR**) ou à une variable (**GPR**), il faut **toujours** s'assurer que les bits **RP1** et **RP0** du registre **STATUS** indiquent la bank où se trouve ce **SFR** ou ce **GPR** (cf. adressage direct vu au chapitre **III.2.1.1**.)

Par exemple, pour indiquer la bank 3, il faudra mettre ces bits **RP1** et **RP0** à 1, comme indiqué sur la figure IV.35.

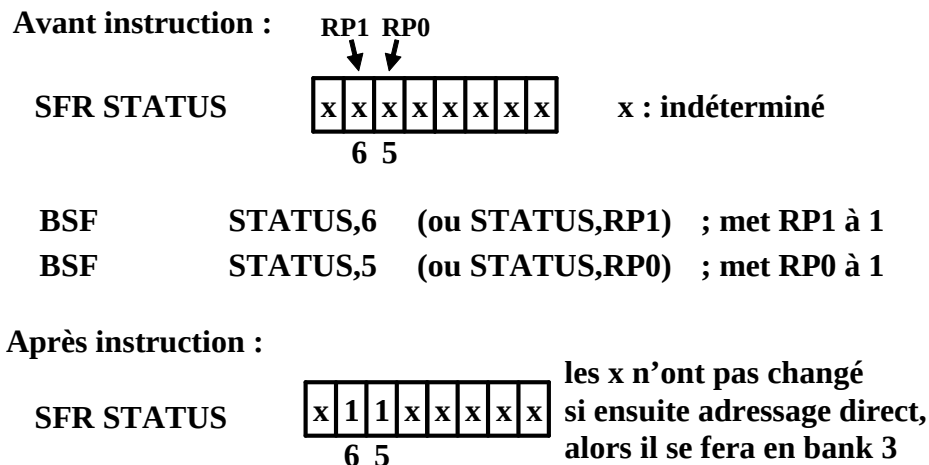


Figure IV.35.

Aucun flag n'est affecté par cette instruction.

Exemple en langage C :

L'instruction assembleur suivante permet de mettre à 1 la broche 6 du PORTB (configurée préalablement en sortie) :

BCF PORTB,6 ; 5V sur la ligne 6 du PORTB

sera équivalente à la ligne en C suivante :

```
RB6 = 0 ;           // 5V sur la ligne 6 du PORTB
```

Remarque : lire la remarque du bas de la page précédente pour l'explication de **RB6**.

IV.3.4. INSTRUCTIONS DE RUPTURE DE SEQUENCE

Ces instructions permettent de « briser » le déroulement séquentiel d'un programme, comme on l'a déjà dit dans le **chapitre III.3**.

Elles permettent d'implanter toutes les structures de contrôle qui existent dans les langages de haut niveau (if, while, for, switch, break ...).

IV.3.4.1. INSTRUCTIONS DE RUPTURE INCONDITIONNELLE

Ces ruptures de séquence auront **toujours** lieu quand ces instructions seront exécutées (pas de condition à respecter).

Elles dureront donc toujours **deux cycles** :

- le premier pour vider le pipe (cf. figure I.24.),
- le deuxième pour effectuer la rupture de séquence demandée.

IV.3.4.1.1. RUPTURES INCONDITIONNELLES SYNCHRONES

Elles sont dites **synchrones** avec le programme car elles sont donc exécutées sous l'entière responsabilité du programmeur en assembleur (ou du compilateur C pour les programmes écrits en C) quand c'est nécessaire pour respecter l'algorithme utilisé.

Elles seront donc exécutées à des moments (certes stratégiques pour le programme) mais bien connus.

24. GOTO k (comme GOTO label k)

Cette instruction a déjà bien été présentée dans le chapitre III.3. sur les généralités quand on a introduit le pointeur de programme **PC**.

On en rappelle le principe sur la figure IV.36.

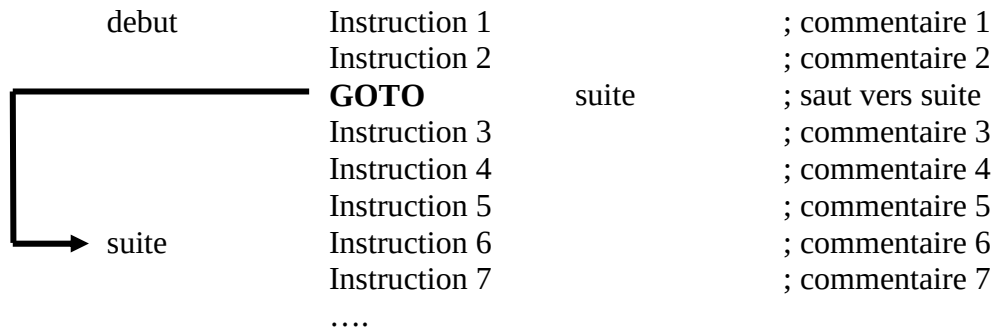


Figure IV.36.

Voici les points importants pour cette instruction **GOTO** :

- l'instruction **GOTO** permet **de sauter à une autre adresse** (précisée sur 11 bits par **k** dans le champ opérande du code machine de l'instruction) pour continuer l'exécution du programme à un autre endroit de la mémoire (avant ou après),
- pas de retour possible vers l'endroit de l'appel : « **ticket aller simple** »,
- grâce au système des labels, on peut donner un nom **symbolique** à **k**, comme sur la figure IV.36. : l'assembleur transformera le label **suite** en une adresse de 11 bits pour effectuer (comme demandé par **GOTO suite**) le saut vers l'adresse correspondant au label **suite**,
- en conséquence, utilisez des noms de labels « parlants » par rapport à la « logique » de votre programme : des labels « tata », « toto »... ne sont pas utilisés par les professionnels mais sont signe d'un « amateurisme » certain,
- attention quand on saute à une adresse située dans une autre page de la ROM de programme : penser à gérer ces changements de pages **avant** de faire le **GOTO** par les **bits 4 et 3** de **PCLATH** (cf. figure III.15.).

Remarque en langage C :

Bien que cette instruction existe aussi en langage C, il est déconseillé de l'utiliser au sein d'un programme écrit en C. En effet, elle présente le défaut de rendre les programmes plus difficiles à comprendre et à mettre au point. Elle pourra toujours être remplacée avantageusement par une boucle while ou for qui présentent l'avantage d'être plus « structurantes ».

Mais en assembleur, ces boucles de « haut niveau » n'existant pas, on sera souvent amené à utiliser le **GOTO**, surtout en lien avec les instructions de rupture conditionnelle de séquence, mais pas n'importe quand, pour conserver, quand même, un minimum de structuration dans les programmes écrits en assembleur (règle : « **trop de GOTO tue le GOTO !!!** »).

25. CALL k (comme CALL subroutine k)

Cette instruction a également été présentée dans le chapitre III.3. sur les généralités quand on a vu le principe d'un sous-programme (ou module en assembleur $\cong$ fonction du C). On en rappelle le principe sur la figure IV.37.

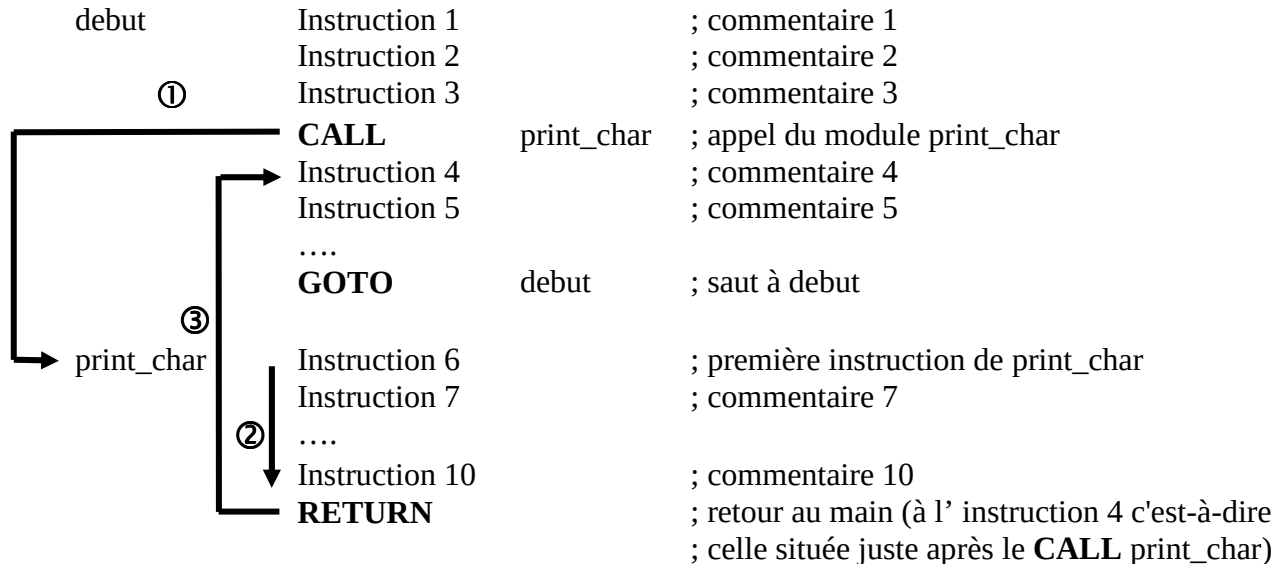


Figure IV.37.

Voici les points importants pour cette instruction **CALL** :

- l'instruction **CALL** permet **de sauter à une autre adresse** (précisée sur 11 bits par **k** dans le champ opérande du code machine de l'instruction = « **ticket aller** ») qui est celle de la première instruction du sous-programme :cf. ① et instruction 6 sur l'exemple de la figure IV.37.
- retour vers l'instruction qui suit l'appel (cf. ③ et instruction 4 sur l'exemple de la figure IV.37.) au moment du **RETURN** : « **ticket retour** » grâce à la pile,
- grâce au système des labels, on peut donner un nom **symbolique** à **k**, comme sur la figure IV.37. : l'assembleur transformera le label **print_char** en une adresse de 11 bits pour effectuer (comme demandé par **CALL print_char**) l'appel du module **print_char** situé à l'adresse correspondant au label **print_char**,
- en conséquence, utilisez des noms de modules « parlants » par rapport à la « logique » de votre programme : les modules « tata », « toto »... ne sont pas utilisés par les professionnels mais sont signe d'un « amateurisme » certain,
- attention quand on appelle un module situé dans une autre page de la ROM de programme : penser à gérer ces changements de pages **avant** de faire le **CALL** par les **bits 4 et 3 de PCLATH** (cf. figure III.15.).

Exemple en langage C :

L'exemple de la figure IV.37. pourrait s'écrire en C :

```
while(1)           // pour la boucle sans fin à cause du GOTO debut
{
    ...
    print_char(); // appel de la fonction print_char (ici sans paramètre)
    ...
}
```


26. RETURN (comme **RETURN from subroutine**)

C'est cette instruction qui, placée à la fin d'un module, en indique la fin, comme on le voit sur la figure IV.37.

On l'appelle aussi **le point de sortie** du module (par opposition à la première instruction du module qu'on appelle **le point d'entrée** du module).

En fait, cette instruction **RETURN** correspond à un transfert **automatique** du contenu du sommet de la pile dans le **PC** (c'est-à-dire de l'adresse de retour sauvegardée **automatiquement** au moment de l'appel du module).

Ce qui permet de continuer l'exécution du programme à l'instruction qui suit en mémoire l'appel du module (cf. ③ de la figure IV.37. précédente).

Si, en assembleur, il peut y avoir plusieurs points de sortie possible (et donc plusieurs **RETURN**) dans le même module, il est conseillé, pour la lisibilité, de n'en conserver qu'un seul en faisant autant de **GOTO** convergents vers cet unique point de sortie.

Exemple en langage C :

Il faut faire la correspondance entre cette instruction **RETURN** et l'accolade fermante **}** qui existe à la fin de toute fonction écrite en C.

27. RETLW k (comme RETURN from subroutine with Literal k in W)

En fait, c'est une variante de l'instruction **RETURN** précédente.

Elle consiste à recopier la valeur du littéral **k** (c'est-à-dire une constante) dans **W** puis à faire un **RETURN**. Ce qui permet de revenir au programme appelant avec une valeur déterminée dans **W**.

Attention, la valeur de **k** n'est pas calculée par le module : c'est une valeur connue au moment de la compilation.

Cette possibilité sera intéressante pour renvoyer un code d'erreur (par exemple 0 ou 1) au programme principal.

Exemple en assembleur :

Une variante d'utilisation de l'instruction **RETLW** permet d'accéder à des tableaux situés dans la mémoire de programme. Ici, on va accéder à n'importe quel élément d'une chaîne de caractères (se terminant par l'ASCIIZ 0). Soit le sous-programme suivant :

```
chaîne      ADDWF    PCL      ; pour pointer sur le caractère recherché
; définition de la chaîne de caractères sous forme d'instructions RETLW      code ASCII
; la chaîne de caractères est ici "Tel 1" et se termine par le caractère 0 (ASCIIZ).
      RETLW    'T'      ; code ASCII du T      (0x54)
      RETLW    'e'      ; code ASCII du e      (0x65)
      RETLW    'l'      ; code ASCII du l      (0x6C)
      RETLW    ' '      ; code ASCII de l'espace (0x20)
      RETLW    '1'      ; code ASCII du 1      (0x31)
      RETLW    0        ; ASCIIZ              (0x00)
```

La première instruction (**ADDWF PCL**) permet, en rajoutant le contenu de **W** au registre **PCL** (octet de poids faible de **PC**), de modifier **PCL** pour sauter à l'adresse contenant le bon **RETLW**. Il faut se rappeler que quand l'instruction **ADDWF PCL** est exécutée, le **PCL** pointe déjà sur la case mémoire suivante :

- ainsi, si par exemple **W** vaut 0, on exécutera ensuite l'instruction : **RETLW 'T'**
donc un retour au programme avec dans **W** le code ASCII de T : **premier** caractère de la chaîne,
- si par exemple **W** vaut 1, on exécutera ensuite l'instruction : **RETLW 'e'**
donc un retour au programme avec dans **W** le code ASCII de e : **deuxième** caractère de la chaîne,
- ...
- si par exemple **W** vaut 5, on exécutera ensuite l'instruction : **RETLW 0**
donc un retour au programme avec dans **W** l'ASCIIZ 0 : caractère de fin de chaîne.

On pourra ainsi « balayer » la chaîne de caractères grâce à la valeur que l'on mettra dans **W** avant d'appeler le module chaîne. **W** est l'offset (ou l'index) du caractère dans la chaîne (le tableau).

Pour accéder au quatrième caractère de cette chaîne, il suffira d'écrire :

```
      MOVLW    3          ; index du caractère
      CALL     chaîne     ; au retour de chaîne, W contiendra 0x20
```

☛☛ : l'instruction **ADDWF PCL** charge aussi **PCLATH** dans l'octet de poids fort de **PC**. Il faut donc s'assurer, **avant** l'appel du module chaîne, que le **PC** pointe bien sur ce module et permet, une fois modifié par l'addition de **PCL** avec **W**, de récupérer tous les éléments de la chaîne. Le principe présenté ci-dessus ne marchera donc tel quel que si la taille de la chaîne (ou du tableau) est ≤ 256 et que **PCLATH** n'ait pas à être modifié à l'intérieur du module. Si ces conditions ne sont pas respectées, quelques adaptations seront nécessaires.

28. RETFIE (comme **RETURN From Interrupt**)

En fait, il n'y en a qu'une : c'est celle qui indique la **fin d'un sous programme d'interruption**.

Par principe, une interruption correspond au traitement d'un événement extérieur et indépendant de l'instruction qui est exécutée par le programme au moment où cet événement se produit.

C'est bien une rupture de séquence, car le programme en cours doit « passer la main » à la routine de traitement de cette interruption (appelée aussi parfois plus succinctement **routine d'IT** comme **InT**erruption). Mais comme cette interruption peut arriver à n'importe quel moment, on dit qu'elle est **asynchrone** avec le programme principal (contrairement aux ruptures synchrones précédentes qui avaient lieu à des moments connus, certes stratégiques, mais bien précis du programme, c'est-à-dire quand elles étaient exécutées).

La fin d'une routine de traitement d'interruption (qui permet de revenir au programme principal) est donc elle aussi une rupture de séquence **asynchrone**.

Au final, une routine de traitement d'interruption est un peu comme un module (avec un point d'entrée et un point de sortie) mais, contrairement à un module qui est appelé par le programme, une routine de traitement d'interruption peut arriver n'importe quand.

Quand j'entends des étudiants dire :

« Quand **j'appelle** dans le programme ma routine d'interruption... »

c'est qu'ils n'ont pas bien compris ce principe : une routine d'IT **ne s'appelle pas par soft** (c'est-à-dire par un CALL) mais elle s'exécute automatiquement (si tout a été bien fait correctement bien sûr) quand l'évènement extérieur associé se produit.

Tous ces points seront détaillés dans le chapitre sur les interruptions.

Il suffit de savoir pour le moment que l'instruction **RETFIE** joue un peu le même rôle que l'instruction **RETURN** mais pour indiquer la fin d'une routine de traitement d'interruption (en fait elle ré autorise la prise en compte des interruptions (c'est-à-dire que les interruptions sont remises en service) **en plus** du retour vers le programme principal).

IV.3.4.2. INSTRUCTIONS DE RUPTURE CONDITIONNELLE

Ces instructions sont toutes **synchrones** (elles se font au moment où elles sont exécutées).

Par contre, contrairement aux ruptures de séquence inconditionnelles précédentes, la rupture de séquence ne se fera que si une condition est réalisée. Sinon, le programme continue en passant à l'instruction qui se trouve dans la case mémoire suivante (déroulement séquentiel).

Ce type d'instructions sera particulièrement adapté pour implanter des boucles ou des tests conditionnels.

29. DECFSZ f,d (comme DECrémente File (f to d) and Skip if Zero)

Cette instruction se déroule en deux temps :

- **décrément** du contenu du registre **f** et résultat dans **d**,
- **test du résultat** de la décrément :
 - **si nul** : **saute l'instruction suivante** en mémoire (une seule sautée) (rupture conditionnelle de séquence),
 - **si non nul** : passage à **l'instruction suivante** en mémoire (déroulement séquentiel).

La durée totale est de 1 cycle si déroulement séquentiel ou de 2 cycles si saut de l'instruction suivante.

Aucun flag n'est modifié par cette instruction.

Exemple en assembleur :

Voici, sur la figure IV.38., un petit exemple de structure de programme mettant en œuvre cette instruction.

Champ Adresses (labels)	Champ mnémonics	Champ opérandes
	MOVLW	10
	MOVWF	compteur
debut_dans_boucle	instruction 1 dans boucle	opérande
	instruction 2 dans boucle	opérande
	...	
	...	
	DECFSZ	compteur
	GOTO	debut_dans_boucle
debut_hors_boucle	instruction 1 hors boucle	opérande
	instruction 2 hors boucle	opérande
	...	

Figure IV.38.

L'organigramme de la figure IV.39. peut nous aider à comprendre le fonctionnement de ce petit programme.

Remarque :

En assembleur, l'approche graphique offerte par un organigramme a du sens (en langage évolué, à cause de la structuration naturelle offerte par les instructions de contrôle de haut niveau, on peut éviter les organigrammes). Par contre, et ceci est une remarque générale, lorsqu'un organigramme est trop important (par exemple si sa taille dépasse une page) il est déconseillé de l'utiliser, car il existe d'autres méthodes (comme la programmation modulaire) qui sont mieux adapté dans ce cas.

Un organigramme se lit du haut vers le bas en suivant le sens des flèches.

Il montre par un graphique la dynamique d'exécution de cette boucle.

- un **rectangle** représente un **traitement** (une instruction ou une suite d'instructions s'exécutant séquentiellement).
- un **losange** représente un **test logique** avec deux sorties possibles (un peu comme le test conditionnel du C : **if () then else**) :
 - une sortie **vraie** vers le bas (OUI),
 - une sortie **fausse** à droite ou à gauche (NON).

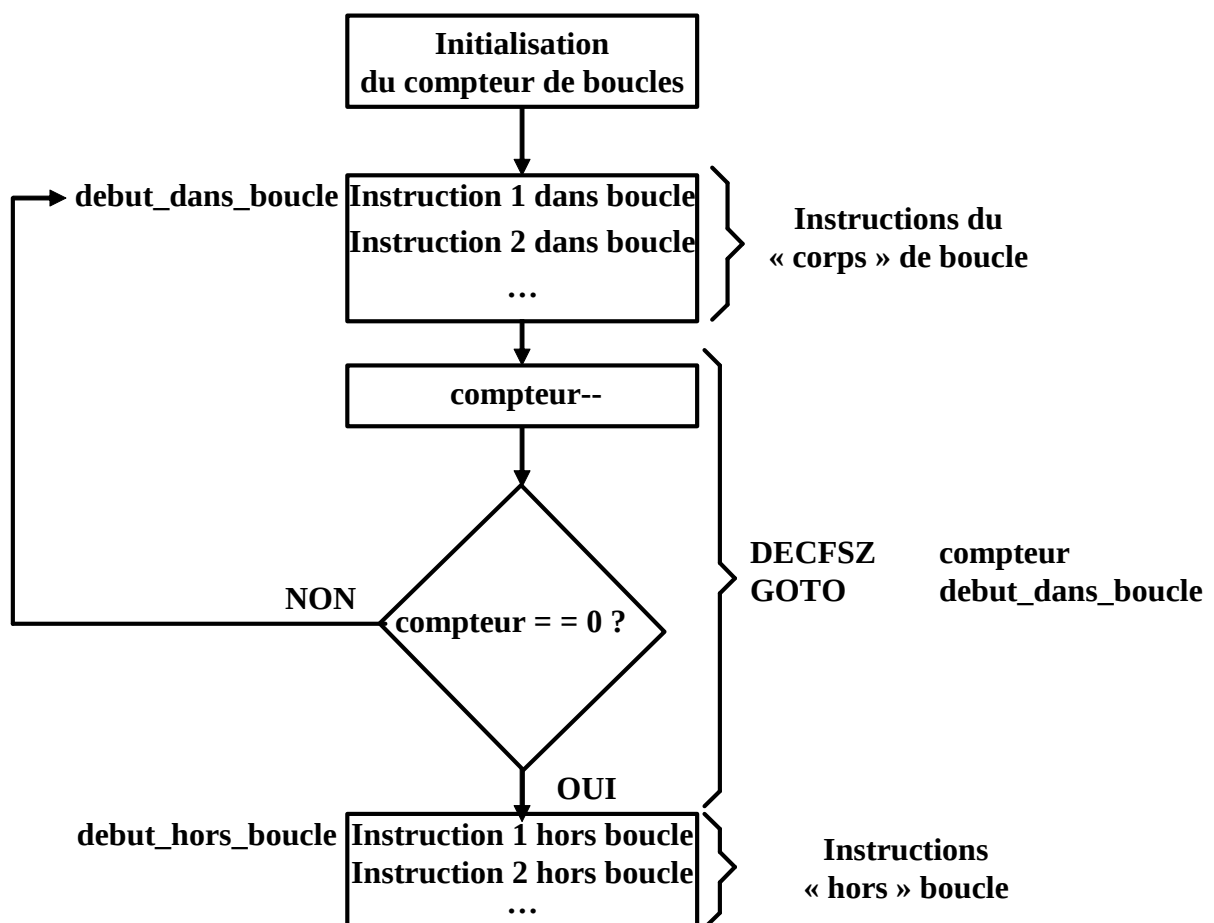


Figure IV.39.

Quelques remarques et pièges à éviter :

Cette boucle sera ici répétée 10 fois.

Mais attention, si on initialise compteur à 0, alors elle sera répétée 256 fois. Si on ne veut pas effectuer la boucle dans ce cas, il faudra faire un test de compteur pour éviter de rentrer dans la boucle dans ce cas (problème des effets de bord !!!)

Autre type d'erreur à éviter :

Si on écrit : **DECFSZ** compteur, **W** au lieu de : **DECFSZ** compteur

alors la variable compteur ne sera pas décrémentée et on ne sortira jamais de cette boucle !!!

Exemple en langage C :

Le petit programme de la figure IV.38. s'écrirait en langage C :

```
compteur = 10;           // initialisation du compteur de boucles
do
{
    ....                 // instructions de la boucle
    compteur--;
}
while (compteur !=0);
```

30. INCFSZ f,d (comme INCrémente File (f to d) and Skip if Zero)

Le principe est le même que pour l'instruction **DECFSZ** précédente.

Cette instruction **INCFSZ** se déroule également en deux temps :

- **incrément** du contenu du registre **f** et résultat dans **d**,
- **test du résultat** de l'incrément :
 - **si nul** : **sauter l'instruction suivante** en mémoire (une seule sautée)
(rupture conditionnelle de séquence),
 - **si non nul** : passage à **l'instruction suivante** en mémoire
(déroulement séquentiel).

La durée totale est de 1 cycle si déroulement séquentiel ou de 2 cycles si saut de l'instruction suivante.

Aucun flag n'est modifié par cette instruction.

Les exemples qu'on pourrait fournir en assembleur et en C sont du même type que pour l'instruction **DECFSZ**.

31. BTFSS f,b (comme Bit Test (b) in File (f) and Skip if Set)

Cette instruction est orientée bit. Elle permet **tester** le bit **b** (de 0 à 7) du file **f** et en fonction de la valeur de ce bit :

- **si b = 1 :** **saute l'instruction suivante** en mémoire (une seule sautée) (rupture conditionnelle de séquence),
- **si b = 0 :** passage à **l'instruction suivante** en mémoire (déroulement séquentiel).

La durée totale est de 1 cycle si déroulement séquentiel ou de 2 cycles si saut de l'instruction suivante.

Aucun flag n'est modifié par cette instruction.

Ce type d'instruction sera fréquemment utilisé car on est souvent amené à tester un bit particulier d'un **SFR** (la plupart du temps) ou d'un **GPR**.

Exemple en assembleur :

Le cas que l'on va voir ici est assez fréquent dans les applications à base de microcontrôleur : attendre qu'un bit b, normalement à 0, d'un registre SFR f, passe à 1.(ce pourrait aussi être l'inverse !!!)

En général, ce passage de 0 à 1 indique qu'un évènement extérieur a eu lieu, et qu'on peut alors effectuer le traitement correspondant à cet évènement.

Les exemples sont nombreux (liste pas du tout exhaustive) :

- un caractère vient d'arriver sur une liaison série : il faut le lire,
- un timer (compteur de temps) a atteint une valeur définie : il faut faire une action donnée comme incrémenter un compteur de secondes par exemple,
- une broche d'un port en entrée est passée de 0 à 1 par appui sur un interrupteur : il faut effectuer l'action prévue consécutive à cet appui,
- un convertisseur analogique/numérique a fini sa conversion : il faut lire la valeur numérique du résultat ...

Ces différents cas peuvent être traités efficacement (mais un peu plus difficilement) par la technique des interruptions, comme on le verra plus loin dans ce polycopié.

Mais il existe une autre façon de les gérer, plus facile à implanter et à mettre au point, et qui est utilisée lorsque les contraintes de temps réel ne sont pas critiques ou primordiales.

Cette technique s'appelle la « **scrutation** » (appelée en anglais le « **pooling** »).

Elle consiste à surveiller (scruter) l'état du bit b concerné et attendre qu'il passe à 1. On cherche en fait à synchroniser le programme avec cet évènement extérieur, matérialisé par le front montant du bit b lors de son passage de 0 à 1.

Les deux lignes assembleur de la figure IV.40. suivante permettent de réaliser cette synchronisation :

synchronisation	BTFSS	f,b	; bit b de f = 1 ?
	GOTO	synchronisation	; non, alors reboucler sur
			; le label synchronisation et continuer
			; à attendre
bit_b_a_1			; suite du programme
			; à partir de là le bit b de f est passé à 1.

Figure IV.40.

On voit sur la figure IV.41. qu'avant l'instant t_1 , le programme reboucle en permanence sur le label **synchronisation**. A l'instant t_1 où b passe à 1, l'instruction :

GOTO synchronisation

est sautée et le programme peut continuer au label **bit_b_a_1**.

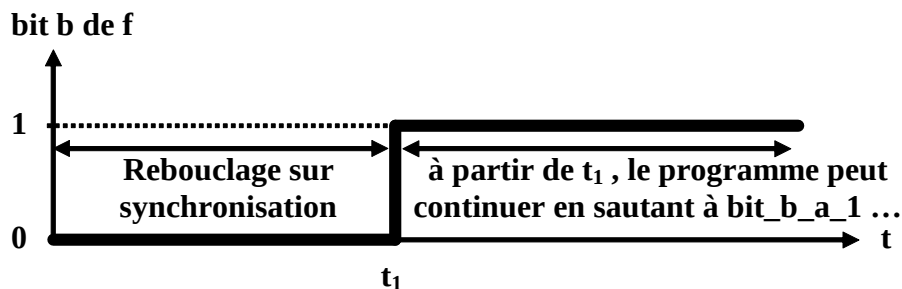
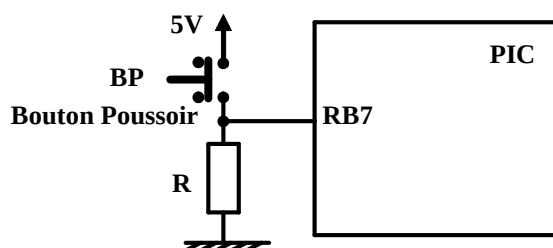


Figure IV.41.

Exemple en langage C :

On suppose que le bit **b** correspond à la broche 7 du PORTB supposée fonctionner en entrée. Il est câblé sur un bouton poussoir **BP** comme indiqué sur la figure IV.42.



Fonctionnement :

BP non enfoncé : RB7 = 0V
(comme sur cette figure)

BP enfoncé : RB7 = 5V

Figure IV.42.

Le petit programme de la figure IV.40. s'écrirait simplement en langage C :

```
while (RB7 == 0) ;           // attend que RB7 passe à 1, c'est-à-dire
                             // attente d'un appui sur BP
```

ou

```
while (!RB7) ;              // idem
```

32. BTFSC f,b (comme Bit Test (b) in File (f) and Skip if Clear)

Le principe est le même que pour l'instruction **BTFSS** précédente.

L'instruction **BTFSC** permet **tester** le bit **b** (de 0 à 7) du file **f** et en fonction de la valeur de ce bit :

- **si b = 0 :** **saute l'instruction suivante** en mémoire (une seule sautée)
(rupture conditionnelle de séquence),
- **si b = 1 :** passage à **l'instruction suivante** en mémoire
(déroulement séquentiel).

La durée totale est de 1 cycle si déroulement séquentiel ou de 2 cycles si saut de l'instruction suivante.

Aucun flag n'est modifié par cette instruction.

Par rapport à l'exemple présenté pour l'instruction **BTFSS**, l'instruction **BTFSC** permettrait de se synchroniser sur un front descendant de 1 à 0 du bit b de f.

IV.3.5. AUTRES INSTRUCTIONS

Les trois dernières instructions sont un peu à part : elles sont liées à des fonctionnements ou à des modes de fonctionnement un peu particuliers du PIC.

33. NOP (comme **No OPeration**)

Instruction qui dure un cycle mais qui ne fait rien !!!

On trouvera dans tout micro/processeur/contrôleur ce type d'instruction, toujours un peu choquante la première fois qu'on la découvre. Quel est donc son intérêt, puisqu'elle ne fait rien ?

Elle sert la plupart du temps à « perdre » du temps CPU, notamment dans les boucles de temporisation logicielles qu'on verra plus loin dans le **chapitre V**.

34. CLRWDT

(comme CLear Watch Dog Timer)

Tout d'abord, qu'est qu'un Watch Dog ? Littéralement, un chien de garde !!!

Cette fonction est en général mise en place dans les **dernières phases de développement** d'un projet, c'est-à-dire quand le programme fonctionne correctement, car c'est en fait une sécurité de fonctionnement qui est ainsi rajoutée : le **chien de garde**.

Imaginons un programme pilotant une application quelconque et fonctionnant normalement.

Un parasite extérieur (rayonnement électromagnétique, bruits électroniques...) peut faire « planter » votre programme. Dans ce cas là, le minimum serait de faire un reset. Mais pour des applications isolées ou difficilement accessibles, ce n'est pas toujours possible d'aller appuyer sur le bouton reset. Il faut donc pouvoir faire un reset automatique. C'est le rôle du chien de garde. Mais rassurez-vous celui-là ne mord pas !!!

L'idée est d'utiliser un système type « **monostable retrigérable** » **logiciel** dont la durée s'appelle le **TIME OUT**.

Il faut régulièrement re-déclencher (on dit aussi ré-armer) ce monostable par une procédure appelée **réarmement software** et réalisée justement par l'instruction **CLRWDT**.

Tant que l'intervalle de temps entre 2 ré-armements successifs est inférieur au **TIME OUT**, il ne se passe rien de particulier, si ce n'est que votre programme continue de fonctionner normalement sous surveillance du watch dog.

Par contre, si l'intervalle de temps entre 2 ré-armements successifs est supérieur au **TIME OUT**, cela déclenche **automatiquement** un reset de votre PIC.

Ce mécanisme est expliqué sur la figure IV.43.

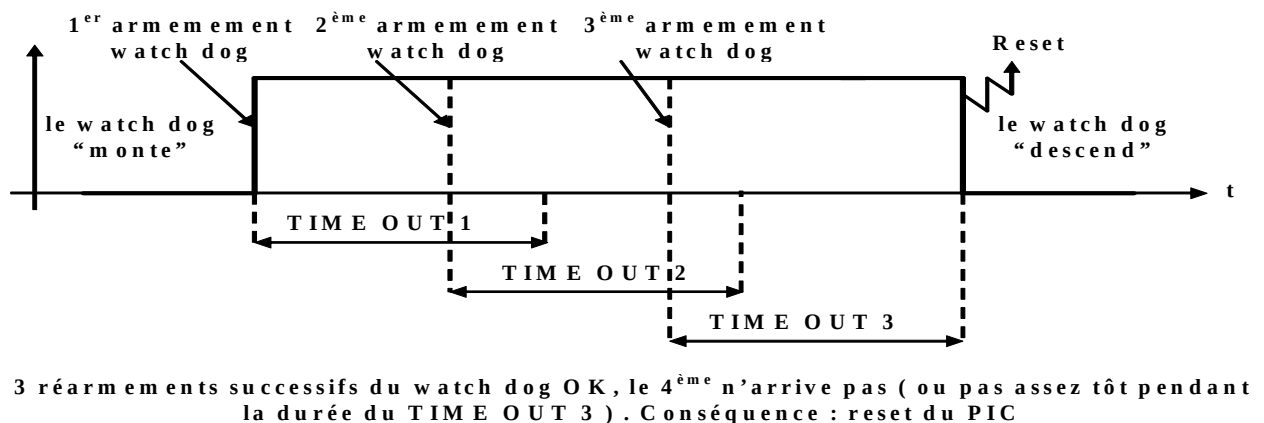


Figure IV.43.

Au final, une bonne image pour comprendre ce principe est celle du conducteur de train qui doit régulièrement appuyer sur un bouton pour signaler qu'il ne s'est pas endormi : s'il ne le fait pas, le train s'arrête automatiquement (ou un seau d'eau lui est versé sur la tête !!!).

Voilà comment il faut implanter cette fonction Watch Dog :

lorsque votre programme marche correctement, vous y rajoutez à plusieurs endroits bien choisis l'instruction **CLRWDT** (de telle sorte qu'en exécution normale de votre programme, l'intervalle de temps entre deux instructions **CLRWDT** consécutives soit inférieur au **TIME OUT**).

Le choix de la valeur du **TIME OUT** est fait dans le module **TIMER0** (voir **chapitre VIII.**).

Il faut aussi penser à valider la fonction watch dog dans le registre **CONFIGURATION** déjà présenté au **chapitre III.5.**

Lorsque toute cette procédure a été faite dans les règles de l'art, supposons qu'un parasite vienne faire « planter » votre programme.

Dans ce cas là, les réarmements successifs du watch dog ne seront plus faits et après la durée **TIME OUT**, un reset sera automatiquement réalisé, ce qui est un moindre mal car cela permettra à votre application au moins de redémarrer (et pas de rester « plantée » indéfiniment !!!).

De plus le bit $\overline{\text{TO}}$ du registre SFR **STATUS** sera mis à 0 pour signaler ce **TIME OUT**.

Notons enfin que les bits $\overline{\text{TO}}$ et $\overline{\text{PD}}$ du registre SFR **STATUS** sont modifiés par l'instruction **CLRWDT** :

ils sont tous les deux mis à 1 (voir instruction suivante **SLEEP** pour savoir pourquoi).

35. SLEEP **(comme SLEEP !!!)**

Littéralement : « Endors toi !!! ».

Cette instruction permet de passer dans un mode dit de « sommeil ».

Ce mode est intéressant car c'est un mode de basse consommation énergétique : si aucune tâche n'est à faire par le PIC, il « s'endort » suite à cette instruction **SLEEP**, ce qui permettra, on l'aura compris, d'augmenter l'autonomie des batteries dans des applications (très fréquentes) portables.

Il y a 3 façons de le « réveiller »:

- un reset,
- le Watch Dog (cf. page précédente et tableau de la figure IV.44. ci-dessous),
- une interruption.

Notons enfin que les bits $\overline{\text{TO}}$ et $\overline{\text{PD}}$ du registre SFR **STATUS** sont modifiés par l'instruction **SLEEP** :

- $\overline{\text{TO}}$ est mis à 1,
- $\overline{\text{PD}}$ est mis à 0.

Au final, ces deux bits du registre SFR **STATUS** donneront les informations résumées dans le tableau de la figure IV.44. :

$\overline{\text{TO}}$	$\overline{\text{PD}}$	Commentaires
1	1	fonctionnement « normal »
1	0	on est dans le mode « sommeil »
0	1	il y a eu un TIME OUT du Watch Dog en mode « réveillé »
0	0	il y a eu un TIME OUT du Watch Dog en mode « sommeil » Pour « réveiller » le PIC par exemple

Figure IV.44.

IV.4. DIRECTIVES D'ASSEMBLAGE

Dans cette partie, nous allons aborder les principaux éléments concernant la programmation en langage **assembleur**. On a vu, dans les chapitres précédents, que ce langage était très proche des aspects matériels. Notamment, beaucoup de notions présentées jusque là faisaient référence soit à des données constantes (adressage littéral), soit à des valeurs de variables globales (adressage direct), soit à des adresses (adressage indirect et notion de pointeur) exprimées souvent **en hexadécimal**.

Mais, dès que la taille des programmes et des données augmente, il devient difficile de s'y retrouver (avec toutes ces valeurs en hexadécimal qui donnent un caractère un peu « ésotérique » voire « mystérieux » aux programmes, un peu style « MATRIX » comme me l'a dit un jour en plaisantant un étudiant plein d'humour). Difficile dans ces conditions, de produire du code **facilement lisible, compréhensible et adapté à une mise au point aisée, sans parler de son manque d'évolutivité**.

Pour cela, il existe un certain nombre **d'aides à l'écriture de programmes**, connues sous le nom de **directives d'assemblage**.

Nous allons apprendre à les utiliser, histoire de prendre rapidement les bonnes habitudes de programmation.

Ce sont des **mots réservés** du langage, constitués d'une suite de **quelques lettres** qui permettent de rendre les fichiers sources « **symboliques** », c'est-à-dire qui autorisent l'utilisation de noms « symboliques » pour représenter les valeurs hexadécimales (noms pour les constantes, pour les variables globales et pour les adresses), ce qui naturellement augmentera la lisibilité du source.

Attention ☹️❗️:

Même si les directives d'assemblage et les mnémonics sont dans la même colonne d'un fichier source, il ne faut pas les confondre.

- les directives d'assemblage « ne sont pas vues » par le PIC mais sont gérées par le logiciel d'assemblage (en amont de l'exécution).

Par contre :

- les mnémonics sont « vues » par le PIC (sous leur forme code machine) car elles correspondent aux instructions élémentaires qui seront exécutées par celui-ci.

Ces directives dépendent du logiciel d'assemblage utilisé pour la compilation.

Ici, on travaillera avec l'assembleur **MPASM (MPLINK et MPLIB)** fourni avec l'environnement de développement **MPLAB IDE**, le tout disponible **gratuitement** sur le site de **MICROCHIP**.

N'hésitez pas à télécharger MPLAB pour vous « muscler » en PIC chez vous !!!

Nous allons donner dans les pages suivantes les principales directives qui permettent de commencer à travailler en assembleur. Pour la liste complète de toutes les directives, il faudra consulter la documentation de l'assembleur présente sur les PC des labos ...

L'objectif de cette partie est de construire « petit à petit » un fichier **canevas** qu'on appellera **vierge_asm.asm** et qui servira de base pour les travaux **en assembleur** que nous ferons en TP.
(nous ferons la même chose avec un canevas pour le C : **vierge_c.c** pour les développements en C)

Les sept premières directives (**IV.4.1.** à **IV.4.7.**) sont classées dans la famille des **directives de contrôle** qui donnent des informations générales sur la façon dont **MPASM** va fonctionner.

IV.4.1. DIRECTIVE LIST

Elle permet de fixer certains paramètres généraux d'assemblage comme le type de PIC utilisé (**p** comme **p**rocessor) et la base par défaut pour les nombres utilisés dans le source (**r** comme **r**adix).

Syntaxe et exemple :

```
list    p = 16f877a, r = dec ; PIC 16F877A utilisé en TP
                                ; et base decimale pour les nombres « bruts »
                                ; donc pour l'hexadécimal : préfixe 0x ou H' ' ou suffixe h
                                ; et pour le binaire : B' ' ,
```

IV.4.2. DIRECTIVE #INCLUDE

Elle permet d'insérer un fichier **d'inclusion complémentaire** dans votre source.

Syntaxe et exemple :

```
#include <p16f877a.inc> ; fichier contenant le mapping et les noms des SFR
                        ; et de certains de leurs bits individuels
                        ; ainsi que les masques de configuration
```

Quelques extraits de ce fichier sur la figure IV.45. pour bien comprendre :

; repérage des SFR (revoir figure III.3.)				
...				
STATUS	EQU	H'0003'	; adresse de STATUS : chaque fois que le mot STATUS sera	
			; utilisé dans votre source, MPASM le remplace par 0x03	
FSR	EQU	H'0004'	; adresse de FSR	
PORTA	EQU	H'0005'	; adresse de PORTA	
PORTB	EQU	H'0006'	; ...	
PORTC	EQU	H'0007'		
...				
; repérage des bits de STATUS (revoir figure III.4.) : utile pour les instructions « orientées bit »				
IRP	EQU	H'0007'	; position bit IRP	(bit7)
RP1	EQU	H'0006'	; position bit RP1	(bit6)
RP0	EQU	H'0005'	; position bit RP0	(bit5)
NOT_TO	EQU	H'0004'	; position bit TO	(bit4)
NOT_PD	EQU	H'0003'	; position bit PD	(bit3)
Z	EQU	H'0002'	; position flag Z	(bit2)
DC	EQU	H'0001'	; position flag DC	(bit1)
C	EQU	H'0000'	; position flag C	(bit0)
...				

Figure IV.45.

Cela permet, dans le source, de faire directement référence au nom des différents registres **SFR** (nom conforme à la documentation de MICROCHIP) **sans avoir à connaître** leurs adresses dans le PIC.

Bonne habitude de travail :

Pour garder le fichier original **p16f877a.inc** de **MPASM** intègre, on le copiera dans son répertoire de projet et on « l'attachera » à son projet. C'est ce que vous ferez en TP.

IV.4.3. DIRECTIVE __CONFIG

Elle permet de définir la **configuration** du PIC (revoir **chapitre III.5.**)

Syntaxe et exemple :

```
__CONFIG  _CP_OFF & _WDT_OFF & _BODEN_OFF & _PWRTE_ON &  
          _HS_OSC & _WRT_OFF & _LVP_ON & _CPD_OFF
```

IV.4.4. DIRECTIVE EQU

Cette directive (**EQU**ate) permet de déclarer et de définir des constantes (pas de réservation mémoire en **GPR**).

Syntaxe :

```
NOM_CSTE      EQU      VALEUR
```

Le nom de la constante déclarée est **NOM_CSTE** et sa valeur est **VALEUR**.

En pratique, on distinguera surtout :

- ① les **constantes « hardware »** : ce sont le plus souvent des adresses

Exemples :

Les adresses des **SFR** déclarées par des **EQU** dans le fichier **p16f877a.inc** (cf. figure IV.45.)

On pourrait, par exemple, y rajouter les lignes suivantes pour définir les broches du PORTB du PIC :

```
RB0      EQU      0      ; bit 0 du PORTB  
RB1      EQU      1      ; bit 1 du PORTB  
RB2      EQU      2      ; bit 2 du PORTB  
...
```

- ② les **constantes** « **software** » : qui sont liées à votre programme (masques, valeurs initiales de compteurs ou de variables...). A ne pas trop utiliser pour les adresses de vos variables, car du coup, cette adresse sera fixe, ce qui peut poser parfois des problèmes à l'assemblage.

La directive **EQU** est liée à l'adressage **L**itéral.

Exemple :

```
NOTE_MAX      EQU      20
```

Ainsi, chaque fois que le mot **NOTE_MAX** (le nom de la constante) sera rencontré dans le fichier source, **MPASM** le remplacera par la valeur décimale **20** (=0x14).

Ce n'est ni plus ni moins qu'une opération de pur « traitement de texte ».

Ainsi, une instruction comme :

```
MOVLW      NOTE_MAX
```

sera alors reconnue et vue par **MPASM** comme (et aura le même effet que) :

```
MOVLW      20
```

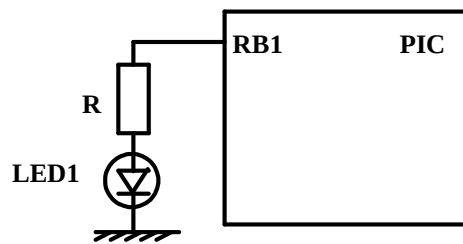
Mais l'utilisation de noms symboliques permettra d'améliorer la lisibilité de votre source.

IV.4.5. DIRECTIVE #DEFINE

C'est une version un peu plus « évoluée » de la directive **EQU** (substitution d'une chaîne de caractères par une autre) et cette directive est à rapprocher du **#define** du langage C.

Exemple :

Supposons qu'une LED soit branchée sur la broche **RB1** du **PORTB** (supposée fonctionner en sortie) comme indiqué sur le schéma de la figure IV.46.



Fonctionnement :

RB1 = 5V LED1 allumée

RB1 = 0V LED1 éteinte

Figure IV.46.

Pour allumer cette **LED1**, il faut faire passer la broche **RB1** à **5V**. Par une instruction comme :

```
BSF          PORTB, RB1
```

Mais si on n'écrit par exemple dans le source la ligne suivante :

```
#define          allumer_led1      BSF          PORTB, RB1
```

chaque fois que la chaîne « allumer_led1 » sera rencontrée dans le fichier source, **MPASM** le remplacera par la chaîne de substitution : « **BSF** PORTB, RB1 ».

Ainsi, si vous écrivez dans votre source une ligne comme :

```
allumer_led1          ; pour allumer la led1 : très lisible
```

elle sera alors remplacée par **MPASM** par :

```
BSF          PORTB, RB1      ; idem mais moins lisible
```

IV.4.6. DIRECTIVE ORG

Cette directive permet de fixer la position de l'implantation du code dans la mémoire de programme.

Syntaxe :

ORG ADRESSE

Lorsque le logiciel d'assemblage rencontrera une directive de ce type, toutes les instructions se trouvant sur les lignes suivantes seront implantées en mémoire au fur et à mesure et sans laisser de place (dans des cases mémoire contiguës, car la mémoire n'est pas un gruyère !!) à partir de l'adresse **ADRESSE**, jusqu'à la prochaine directive **ORG ...**

Exemple :

On peut en voir l'effet sur la figure IV.47. :

- fenêtre de gauche : le source assembleur,
- fenêtre de droite : l'exécutable fourni par **MPASM** et son implantation dans la mémoire de programme.

Le premier **ORG 0x200** implante les instructions qui le suivent à partir de l'adresse **0x0200** (cf. ①),

et le second **ORG 0x210** à partir de l'adresse **0x0210** (cf. ②).

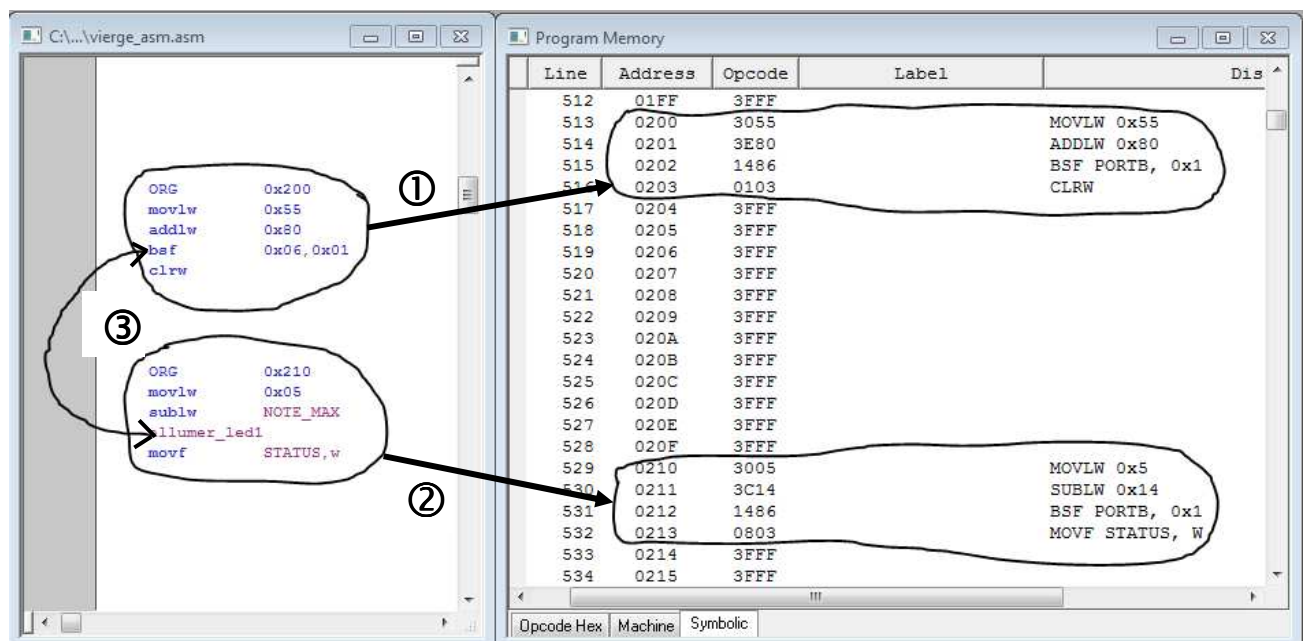


Figure IV.47.

Profitons de cet écran pour regarder les instructions après le **ORG 0x210** :

- le symbole **NOTE_MAX** a bien été remplacé par sa valeur (**0x14**) par **MPASM**,
- la ligne **allumer_led1** a bien été remplacée par **BSF PORTB,RB1**.
Notons que c'est la même instruction (cf. ③) que celle qui se trouve à l'adresse **0x202** (**BSF 0x06,0x1**) mais elle est bien plus lisible,
- le registre **STATUS** a bien été remplacé par son adresse (**0x03**) par **MPSAM**.

IV.4.7. DIRECTIVE END

Elle indique la fin du code à assembler.

Si on rajoute des instructions après cette directive **END**, elles ne seront pas compilées.

Si on oublie cette directive à la fin de son programme, on aura une erreur de compilation.

Les directives que nous allons présenter maintenant concernent les variables d'un programme.

Pour simplifier, on rencontrera trois types de variables globales :

- les variables **globales non initialisées** et situées en **RAM**,
- les variables **globales initialisées** et situées aussi en **RAM**,
- les variables **globales (semi)permanentes** et situées en **EEPROM de données**.

Au final, un fichier **source** écrit en assembleur est une suite de réservation mémoire pour les variables globales et de lignes d'instructions assembleur pour le code du programme.

Attention : erreur fréquente 🍷 🍷

Ne pas mélanger ces 2 zones :
on ne met pas de déclaration de variables globales dans les lignes de code
et pas de lignes de code dans la zone de déclaration/réservation des variables globales

IV.4.8. DIRECTIVE CBLOCK/ENDC

Les variables globales non initialisées sont situées dans les **GPR** de la RAM.

On peut consulter à nouveau le mapping interne de la RAM du PIC fourni sur la figure III.3. pour voir les places disponibles pour ces variables.

Ces variables pourront se trouver dans n'importe quelle case **GPR** des **banks 0 à 3**.

Je conseille de les placer dans la **zone commune** à toutes les banks, située dans chaque bank entre les offsets **0x71** et **0x7F** (la case d'adresse **0x70** serait à priori utilisée par **MPLAB** (plus précisément par le boîtier **ICD3** : cf.**chapitre VII.**) pour le débogage).

Ce n'est pas une obligation, mais cela évite d'avoir à gérer leur adressabilité via les bits **RP1** et **RP0** du **SFR STATUS**, puisque quelque soit la valeur de ces bits, elles seront toujours accessibles.

Pour cela, il faudra ouvrir une zone de stockage par le couple de directives **CBLOCK/ENDC** (voir exemple ci-dessous).

Les variables globales non initialisées nécessitent ensuite d'être déclarées dans cette zone de stockage en indiquant leur nom et leur taille. Au démarrage de votre programme, il y aura n'importe quoi dans ces variables situées en RAM (contenu aléatoire).

Syntaxe et exemple :

```
CBLOCK    0x71    ; ouverture de la zone de données en zone commune
              ; de la RAM à partir de l'adresse 0x071
```

; déclaration des variables par : **nom_variable : taille_variable en nombre d'octets**

```
compteur    :      1          ; variable compteur à l'adresse 0x71 de taille 1 octet
```

```
tableau      :      8          ; tableau de taille 8 octets
              ; premier élément du tableau à l'adresse 0x72
              ; deuxième élément du tableau à l'adresse 0x73
              ; ...
```

```
car          :      1          ; variable car de taille 1 octet à l'adresse 0x7A
```

```
i           :      2          ; i est un entier court sur 16 bits (2 octets)
              ; aux adresses 0x7B et 0x7C
```

```
...
```

```
ENDC              ; fin de la zone de données
```

IV.4.9. DIRECTIVES DB et DW

Elles permettent de donner une valeur initiale à des variables. Cela veut dire que ces variables initialisées auront déjà ces valeurs initiales **avant** que votre programme n'ait commencé à s'exécuter.

On trouvera par exemple les directives :

DB comme : **D**eclare data of one **B**yte : pour les variables de **taille un octet**,
DW comme : **D**eclare data of one **W**ord : pour les variables de **taille deux octets**.

Syntaxe et exemple :

tempo	DB	0x05	; variable tempo de taille 1 octet initialisée à 0x05
caractere	DB	‘p’	; variable caractere initialisée avec le code ASCII ‘p’
j	DW	0x1234	; j est un entier court sur 16 bits initialisé à 0x1234

Mais il faut tout de même savoir que le cas des données globales **initialisées** est plus **délicat**.

En effet, si on veut que ces variables soient initialisées avant que votre programme ne débute, il est nécessaire de prévoir un programme (en fait un sous-programme qui sera lié avec votre programme applicatif) qui viendra initialiser ces variables de façon « **quasiment transparente** » pour vous.

Ce sous-programme existe et est contenu dans le fichier **init_data.asm** (qu'il faudra « rattacher » à votre projet via le gestionnaire de projets) :

c'est la fonction **copy_init_data**.

C'est d'ailleurs ce qui est fait en langage C : le couple compilateur/linker rajoute à votre programme, et de façon transparente, le code nécessaire (**le runtime startup code**) pour initialiser les variables globales initialisées (comme par exemple `int i = 5` ; ou `char c = 'a'` ;) mais aussi pour mettre à 0 les variables globales non initialisées, et ceci avant l'exécution du `main`.

Nous choisirons de faire plus simple :

**les variables initialisées le seront au tout début,
par les premières instructions de votre programme.**

Example :

Au lieu de faire la déclaration suivante :

tempo	DB	0x05	; variable tempo de taille 1 octet initialisée à 0x05 ; avant toute exécution du programme
-------	----	------	---

On écrira, au tout début du programme, les deux instructions suivantes :

MOVLW	0x05	; charge W avec le littéral (constante) 0x05
MOVWF	tempo	; copie le contenu de W dans le GPR tempo
		; MPASM remplace tempo par son adresse

IV.4.10. DIRECTIVE DE

Les données permanentes et semi permanentes ne sont pas stockées en GPR, mais dans l'EEPROM de données.

Dans le PIC 16F877, les 256 octets constituant cette EEPROM de données se trouvent entre les adresses **0x2100** et **0x21FF**.

Pour initialiser des valeurs permanentes ou semi permanentes dans cette zone, il faut utiliser la directive **DE** (comme **D**eclare **E**EPROM data byte).

Syntaxe et exemple :

	ORG	0x2100	; adresse de base de l'EEPROM
	DE	"telecom"	; réservation pour un string (entre " ")
annee	DE	'1'	; réservation pour char ASCII entre ' '
code_pin	DE	0x01,0x05,0x07,0x09	; suite de 4 octets hexadécimaux

On peut voir sur la figure IV.48. le stockage en EEPROM réalisé par les déclarations précédentes :

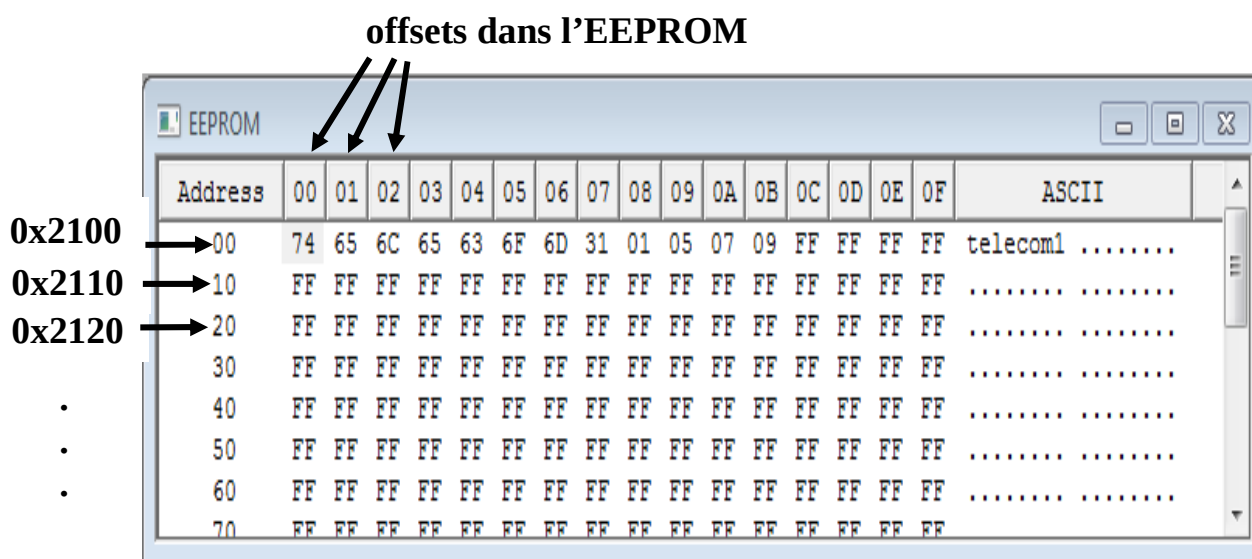


Figure IV.48.

Ces valeurs seront écrites dans l'EEPROM de données au moment de la programmation de l'EEPROM de programme. Bien sûr, et c'est une évidence, elles ne seront pas perdues quand on coupera l'alimentation du PIC.

On verra plus loin comment le programme peut les lire et même les modifier (par une procédure un peu plus compliquée que pour des données situées en RAM).

IV.4.11. AUTRES DIRECTIVES INTERESSANTES

La directive **DT** (comme **D**efine **T**able) permet de générer une « **table** » de **RETLW**.

On en a vu le principe quand on a présenté justement cette instruction **RETLW**.

Notamment sur l'exemple ci-dessous :

chaîne	ADDWF	PCL	; pour pointer sur le caractère recherché
	RETLW	'T'	; code ASCII du T (0x54)
	RETLW	'e'	; code ASCII du e (0x65)
	RETLW	'l'	; code ASCII du l (0x6C)
	RETLW	' '	; code ASCII de l'espace (0x20)
	RETLW	'1'	; code ASCII du 1 (0x31)
	RETLW	0	; ASCIIZ (0x00)

Grâce à la directive **DT**, on pourra écrire plutôt :

chaîne	ADDWF	PCL	
	DT	"Tel 1",0	; chaîne se terminant par l'ASCIIZ 0x00

Ce qui permet de rendre la chaîne de caractères plus lisible dans le source.

Voici deux directives qui permettent de s'assurer de l'adressabilité des **SFR** et des **GPR**.

- pour l'adressage direct :

Avant d'accéder à un SFR ou un GPR, on sait qu'il faut toujours que les bits **RP1** et **RP0** de **STATUS** permettent de **sélectionner la bank** où se trouve ce **SFR** ou ce **GPR** qu'on souhaite lire ou modifier.

La directive **BANKSEL** permet de positionner **automatiquement** les bits **RP1** et **RP0**

Syntaxe et exemple :

Supposons que la variable **k** se trouve dans la bank 3 (RP1=1 et RP0=1). Alors, si on écrit :

BANKSEL	k	; sélection de la bank où est k
MOVWF	k	; copie W dans k

MPASM remplacera la première ligne par les deux lignes suivantes :

BSF	STATUS,RP1	; mise à 1 de RP1
BSF	STATUS,RP0	; mise à 1 de RP0

Ce qui permettra ensuite de copier W dans k (et pas dans une case d'une autre bank ayant le même offset que k !!! ce qui arriverait si on oubliait de faire **BANKSEL k** avant de faire référence à la variable k). Notons qu'on a également le même problème avec les **SFR**.

Sauf lorsque le **SFR** ou le **GPR** est présent dans les 4 banks (par adresse ghost) car alors il n'y a pas besoin de s'occuper des bits **RP1** et **RP0**, puisque dans ce cas l'offset suffit.

- pour l'adressage indirect :

On a vu qu'il fallait dans ce cas positionner le bit **IRP** du **SFR STATUS** pour fabriquer une adresse (pointeur) sur 9 bits (cf. figure III.7. et III.8.) utilisée au moment de l'accès indirect.

La directive **BANKISEL** (**I** comme **I**ndirect) permet de positionner correctement le bit **IRP** en fonction de la page où se trouve la variable visée par l'adressage indirect.

Syntaxe et exemple :

MOVLW	VAR1	; met les 8 bits de poids faible de <u>l'adresse</u> de ; la variable VAR1 dans W (offset dans la page) ; à ne pas confondre avec MOVF VAR1,W ; qui met la <u>valeur</u> de VAR1 dans W par adressage ; direct
MOVWF	FSR	; puis prépare l'adressage indirect (pointeur) en ; écrivant cet offset dans FSR ; maintenant FSR contient l'adresse de VAR1
BANKISEL	VAR1	; si VAR1 en page 0 met IRP à 0 ; si VAR1 en page 1 met IRP à 1
MOVLW	0x55	; par exemple
MOVWF	INDF	; écrit 0x55 dans VAR1 (par adressage indirect) via INDF

Ces lignes assembleur seraient équivalentes en langage C à :

PTR_VAR1 = &VAR1;	// pointeur sur VAR1
*PTR_VAR1 = 0x55 ;	// VAR1 vaut maintenant 0x55

La dernière directive que je présenterai dans cette partie permet d'assurer l'adressabilité du code dans le cas de sauts « longs » (c'est-à-dire avec changement de page dans la mémoire de programme) pour les instructions **GOTO** ou **CALL** inter pages.

La directive **PAGESEL** permet de positionner correctement les **bits 4 et 3** de **PCLATH** en fonction de la page où se trouve l'adresse du saut (en fait les deux bits de poids forts de l'adresse d'arrivée, connus par **MPASM** au moment de la compilation, seront copiés respectivement sur les **bits 4 et 3** de **PCLATH**).

Syntaxe et exemple :

PAGESEL	LONG_SAUT	
GOTO	LONG_SAUT	; positionne les bits 4 et 3 pour fabriquer une ; adresse de 13 bits pour faire le saut

IV.4.12. LE FICHIER VIERGE_ASM.ASM

Au final, voici sur la figure IV.49., le fichier **vierge_asm.asm** à utiliser comme « base de départ » pour vos programmes assembleur.

Nous verrons en Travaux Pratiques comment l'insérer à tous vos nouveaux projets.

```

; canevas pour les projets en assembleur
; à utiliser comme » base de départ » pour vos nouveaux projets en assembleur
;*****
; exemple du cours complet page 152
    list          p=16f877a, r=dec          ; type de PIC et radix décimal
;*****
; fichiers d'inclusion : exemple du cours complet page 152
    #include      <p16f877a.inc>            ; fichier d'inclusion mapping et noms registres SFR
;*****
; configuration au téléchargement : exemple du cours complet page 153
__CONFIG_CP_OFF & _WDT_OFF & _BODEN_OFF & _PWRTE_ON & _HS_OSC
& _WRT_OFF & _LVP_OFF & _CPD_OFF
;*****
; constantes hardware
; broches du PORTB :exemple du cours complet page 153
;          RB0          EQU          0          ; broche RB0
;          RB1          EQU          1          ; broche RB1
;*****
; constantes software : exemple du cours complet page 154
;          NOTE_MAX     EQU    20
;*****
; les assignations : exemple du cours complet page 155
;          #define      allumer_led1  BSF          PORTB,RB1
;*****
; variables non initialisées en GPR zone commune : exemple du cours complet page 158
;          CBLOCK      0x71          ; début zone données
;compteur          :          1
;tableau           :          8
;car               :          1
;i                 :          2
;          ENDC          ; fin zone de données
;*****
; variables initialisées nécessitant la fonction copy_init_data pour les initialiser de la ROM vers la RAM
; extern copy_init_data exemple du cours complet page 159
;*****
; variables (semi permanentes) en EEPROM de données à partir de 0x2100 : exemple du cours page 160
;          ORG          0x2100
;          DE           "telecom"
;annee             DE           '1'
;code_pin          DE           0x01,0x05,0x07,0x09
;*****
; zone de code implantée en EEPROM de programme
;*****
;          ORG          0x0000          ; vecteur de reset
;          GOTO         main          ; vers le début du programme
;          ORG          0x0005          ; vecteur d'interruption
main
; mettre ici les instructions de votre programme
...
...
END          ; fin de l'assemblage ici

```

Figure IV.49.

V. PETITS EXERCICES D'INITIATION CORRIGES

De façon générale, lisez bien les commentaires qui sont fournis dans les sources des corrections fournies. Ils renseignent énormément sur la logique du traitement. Quand on écrit un programme, il faut d'ailleurs les écrire **en même temps**, et **pas une semaine après !!!** comme le font le plus souvent, et à tort, les étudiants ...

V.1. « SIGNATURE » DES BANKS DE LA RAM

1. Repérer sur la **figure III.3.** les adresses des premiers **GPR** disponibles pour l'utilisateur dans chaque bank de la RAM.

2. Ecrire un programme qui stocke :

- le code ASCII de **0** dans **le premier GPR disponible** de la bank **0**,
- le code ASCII de **1** dans **le premier GPR disponible** de la bank **1**,
- le code ASCII de **2** dans **le premier GPR disponible** de la bank **2**,
- le code ASCII de **3** dans **le premier GPR disponible** de la bank **3**.

Eléments de correction :

1. bank **0** : adresse de début **0x20** bank **1** : adresse de début **0xA0**
 bank **2** : adresse de début **0x110** bank **3** : adresse de début **0x190**

2. Le chargement du code ASCII se fait en adressage **littéral** via **W** suivi du stockage en **GPR** en adressage **direct**.

Ne pas oublier la **sélection** de la bank concernée en positionnant correctement les bits **RP1** et **RP0** du **SFR STATUS**. Le source de cet **exo1** est fourni sur la figure V.1.

exo1	ORG	0x0000	; vecteur de reset
	goto	exo1	; vers début exo1
	ORG	0x0005	
	movlw	'0'	; adressage littéral
	bcf	STATUS,RP1	; RP1 = 0
	bcf	STATUS,RP0	; RP0 = 0 donc bank 0
	movwf	0x20	; adressage direct
	movlw	'1'	; adressage littéral
	bsf	STATUS,RP0	; RP0 =1 comme RP1 = 0 : bank 1
	movwf	0xA0	; adressage direct
	movlw	'2'	; adressage littéral
	bsf	STATUS,RP1	; RP1 = 1
	bcf	STATUS,RP0	; RP0 = 0 donc bank 2
	movwf	0x110	; adressage direct
	movlw	'3'	; adressage littéral
	bsf	STATUS,RP0	; RP0 =1 comme RP1 = 1 : bank 3
	movwf	0x190	; adressage direct
	END		; fin de l'assemblage

Figure V.1.

V.2. « ZONE COMMUNE » DES QUATRE BANKS DE LA RAM :

1. Ecrire un programme qui stocke le **code ASCII de C** à l'adresse **0x7F**.
2. Puis, à la suite, mettre dans **W** le contenu (au choix) des **GPR** d'adresses **0xFF** ou **0x17F** ou **0x1FF**.
3. Conclusions.

Eléments de correction :

1 et 2. Idem exo 1 pour le stockage.

Le source de cet **exo2** est fourni sur la figure V.2.

exo2	movlw	'C'	
	bcf	STATUS,RP1	
	bcf	STATUS,RP0	; page 0
	movwf	0x7F	
	clrw		; efface W
	bsf	STATUS,RP0	; page 1
	movf	0xFF,w	; donne W = 0x43 = 'C'
	clrw		
	bsf	STATUS,RP1	
	bcf	STATUS,RP0	; page 2
	movf	0x17F,w	; donne W = 0x43 = 'C'
	clrw		
	bsf	STATUS,RP0	; page 3
	movf	0x1FF,w	; donne W = 0x43 = 'C'
	END		

Figure V.2.

3. La lecture des **GPR** d'adresses **0xFF** (ou **0x17F** ou **0x1FF**) donnera le même résultat, à savoir le code ASCII du C. On retrouvera donc 3 fois la même valeur dans **W**, à savoir le contenu de la case **0x7F**.

En fait, les zones (de taille 16 octets chacune) :

- [**0xF0 – 0xFF**] de la page 1,
- [**0x170 – 0x17F**] de la page 2,
- [**0x1F0 – 0x1FF**] de la page 3,

sont des ghosts de la zone commune [**0x70 – 0x7F**] de la page 0.

V.3. « PETITE ADDITION » ENTRE DEUX VARIABLES

1. Définir **deux variables non initialisées** **i** et **j** de **taille un octet chacune** dans la zone des données globales.
2. Dans les premières lignes de votre programme, initialiser **i** avec **250** et **j** avec **5**. Puis, additionner **i** et **j** avec résultat dans **W** et contrôler les bits **Z** et **C**.
3. Refaire la question 2. en initialisant cette fois-ci **j** avec **6**.
4. Et enfin, refaire la question 2. en initialisant cette fois-ci **j** avec **7**.

Eléments de correction :

Le source de cet **exo3** est fourni sur les figures V.3.a. et V.3.b.

1.

	CBLOCK	0x20	; début zone données
i	:	1	; variable i de taille de 1 octet
j	:	1	; variable j de taille de 1 octet
	ENDC		; fin zone de données

Figure V.3.a.

2.

exo3	movlw	250	
	banksel	i	
	movwf	i	; i = 250
	movlw	5	
	banksel	j	
	movwf	j	; j = 5
	addwf	i,w	; addition avec résultat dans W = 255 = 0xFF ; avec C = 0 : pas de retenue ; et Z = 0 : valeur nulle dans W
	END		

Figure V.3.b.

3.

C'est le même programme en changeant uniquement le littéral 5 (pour j) par **6** (250+6=256).

On obtient **W** = 0x00 avec :

C = 1 : retenue

Z = 1 : valeur nulle dans **W**.

4.

C'est le même programme en changeant uniquement le littéral 5 (pour j) par **7** (250+7=257).

On obtient **W** = 0x01 avec :

C = 1 : retenue

Z = 0 : valeur non nulle dans **W**.

V.4. « PETITE SOUSTRACTION » ENTRE DEUX VARIABLES

1. Définir **deux variables non initialisées** **i** et **j** de **taille un octet chacune** dans la zone des données globales.
2. Dans les premières lignes de votre programme, initialiser **i** avec **3** et **j** avec **2**. Puis, soustraire **j** de **i** avec résultat dans **W** et contrôler les bits **Z** et **C**.
3. Refaire la question 2. en initialisant cette fois-ci **j** avec **3**.
4. Et enfin, refaire la question 2. en initialisant cette fois-ci **j** avec **4**.

Eléments de correction :

Le source de cet **exo4** est fourni sur les figures V.4.a. et V.4.b.

1.

	CBLOCK	0x20	; début zone données
i	:	1	
j	:	1	
	ENDC		; fin zone de données

Figure V.4.a.

2.

exo4	movlw	3	
	banksel	i	
	movwf	i	; i = 3
	movlw	2	
	banksel	j	
	movwf	j	; j = 2
	subwf	i,w	; soustraction avec résultat dans W = 1 = 0x01 ; avec C = 1 : retenue (résultat ≥ 0) ; et Z = 0 : valeur non nulle dans W
	END		

Figure V.4.b.

3.

C'est le même programme en changeant uniquement le littéral 2 (pour j) par **3** ($3 - 3 = 0$).

On obtient **W** = 0x00 avec :

C = 1 : retenue (résultat ≥ 0)

Z = 1 : valeur nulle dans **W**.

4.

C'est le même programme en changeant uniquement le littéral 2 (pour j) par **4** ($3 - 4 = -1$)

On obtient **W** = 0xFF avec :

C = 0 : pas de retenue (résultat < 0)

Z = 0 : valeur non nulle dans **W**.

V.5. « COMPARAISON » ENTRE DEUX VARIABLES

1. Définir **deux variables non initialisées i et j de taille un octet chacune** et considérées comme des nombres **non signés** dans la zone des données globales. Puis, à la suite, définir trois variables de test également **non initialisées : plus_petit, egal et plus_grand de taille un octet chacune.**

2. Initialiser (par votre code) à 0 les trois variables de test.

3. Ecrire à la suite un programme qui :

- met à **0x01** (par adressage bit) la variable **plus_petit** si **i < j**,
- met à **0x01** (par adressage bit) la variable **egal** si **i = j**,
- met à **0x01** (par adressage bit) la variable **plus_grand** si **i > j**.

4. Proposer une méthode pour tester ce programme.

Eléments de correction :

Le source de cet **exo5** est fourni sur les figures V.5.a., V.5.b. et V.8.

1.

	CBLOCK	0x20	; début zone données
i	:	1	
j	:	1	
plus_petit	:	1	
egal	:	1	
plus_grand	:	1	
	ENDC		; fin zone de données

Figure V.5.a.

2.

exo5	banksel	plus_petit	; sélection bank des variables de test
	clrf	plus_petit	; mis à 0
	clrf	egal	; des trois
	clrf	plus_grand	; variables de test

Figure V.5.b.

3.

On va en fait tester les flags **C** et **Z** du **SFR STATUS**, qui nous indiquerons (cf. figure III.6.) :

- si **C = 1** alors : **i ≥ j**,
- si **Z = 1** alors : **i = j**,
- si **C = 0** alors : **i < j**.

Nous utiliserons donc des instructions de **rupture conditionnelle** de séquence.

Il faut se rappeler qu'une instruction de rupture conditionnelle de séquence réalise l'organigramme de la figure V.6.

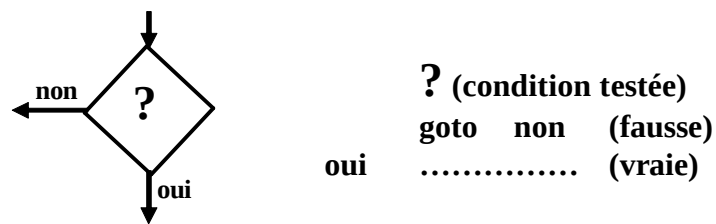


Figure V.6.

Sur cette figure, la condition testée est :

- résultat nul ($Z = 1$) pour ? = **DECFSZ** et ? = **INCFSSZ**,
- bit b (de f) = 1 pour ? = **BTFSS**,
- bit b (de f) = 0 pour ? = **BTFSC**.

Ce qui nous donne, pour le programme, l'organigramme de la figure V.7.

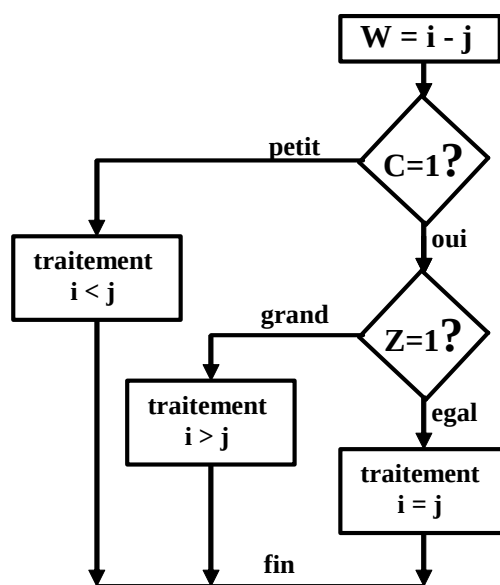


Figure V.7.

On pourra donc écrire le programme de la figure V.8. suivante conforme à cet organigramme :

	subwf	i,w	; W = i - j
	btfss	STATUS,C	; C = 1 ?
	goto	petit	; non, alors i < j
	btfss	STATUS,Z	; Z = 1 ?
	goto	grand	; non, alors i > j
egal	bsf	egal,0	; traitement i = j par adressage bit
	goto	fin	; et fini
petit	bsf	plus_petit,0	; traitement i < j par adressage bit
	goto	fin	; et fini
grand	bsf	plus_grand,0	; traitement i > j par adressage bit
	goto	fin	; et fini
fin	nop		; fini
	END		

Figure V.8.

Remarque : pour des variables signées, le traitement serait différent.

4. Il suffira d'initialiser **i** et **j** avec des valeurs permettant de tester le programme ;

V.6. « SORTIE DE BOUCLE » PAR DEPASSEMENT DE SEUIL

1. Définir **une variable non initialisée compteur** de **taille un octet** dans la zone des données globales.
2. Définir **une constante MAX_COMPTEUR** dans la zone des constantes software de valeur **10**.
3. Ecrire en assembleur l'équivalent du programme C suivant :

```
compteur = 0 ;  
do  
{  
  // mettre ici le traitement du corps de boucle..  
  compteur++ ;  
}  
while (compteur < max_compteur)
```

Eléments de correction :

Le source de cet **exo6** est fourni sur les figures V.9.a., V.9.b. et V.9.c.

1.

	CBLOCK	0x20	; début zone données
compteur	:	1	
	ENDC		; fin zone de données

Figure V.9.a.

2.

; constantes hardware			
MAX_COMPTEUR	EQU	10	

Figure V.9.b.

3.

Il s'agit d'une boucle avec comme **condition de sortie** :

compteur $\geq$ max_compteur,

c'est-à-dire quand $\text{compteur} - \text{max_compteur} \geq 0$ et donc quand **C = 1** après la soustraction.

```

exo6
    banksel    compteur
    clrf       compteur          ; compteur = 0

boucle                                ; do
    ; mettre ici le traitement du corps de boucle ...
    banksel    compteur
    incf       compteur          ; compteur++
    movlw      MAX_COMPTEUR
    subwf      compteur,w
    btfss      STATUS,C          ; sort si compteur ≥ MAX_COMPTEUR
    goto       boucle            ; while (compteur < MAX_COMPTEUR)

hors_boucle  nop

                                END

```

Figure V.9.c.

A titre indicatif, voici sur la figure V.10. ce que donne le compilateur C pour le programme donné à la question 3. de cet exercice, en terme d'assembleur et de code machine :

Line	Address	Opcode	Label	Disassembly
2029	07EC	01F6		CLRF compteur
2030	07ED	0183		CLRF STATUS
2031	07EE	120A		BCF PCLATH, 0x4
2032	07EF	118A		BCF PCLATH, 0x3
2033	07F0	2FF1		GOTO main
2034	07F1	1003	main	BCF STATUS, 0
2035	07F2	3000		MOVLW 0
2036	07F3	1803		BTFSC STATUS, 0
2037	07F4	3001		MOVLW 0x1
2038	07F5	00F6		MOVWF compteur
2039	07F6	3001		MOVLW 0x1
2040	07F7	00F7		MOVWF 0x77
2041	07F8	07F6		ADDWF compteur, F
2042	07F9	300A		MOVLW 0xa
2043	07FA	0276		SUBWF compteur, W
2044	07FB	1C03		BTFSS STATUS, 0
2045	07FC	2FF6		GOTO 0x7f6
2046	07FD	120A		BCF PCLATH, 0x4
2047	07FE	118A		BCF PCLATH, 0x3
2048	07FF	2800		GOTO 0
2049	0800	3FFF		

Figure V.10.

On y retrouve la base de ce que nous avons proposé en assembleur.

V.7. « TEST D'EGALITE » PAR OPERATEUR XOR

Proposer une solution **pour tester l'égalité** (et donc aussi **l'inégalité**) entre deux variables en utilisant un **ou exclusif**.

Eléments de correction :

L'idée est d'utiliser le fait que $x \oplus x = 0$ et ceci que $x = 0$ ou que $x = 1$.

Donc, si on fait un **XOR bit à bit** entre les deux variables :

- soit on obtient **0** (et **Z = 1**) si et seulement si les deux variables sont **égales**,
- soit on obtient un résultat non nul (et **Z = 0**) si les deux variables diffèrent d'au moins un bit et sont donc **différentes**.

Ce qui donne le source de cet **exo7** fourni sur les figures V.11.a. et V.11.b.

	CBLOCK	0x20	; début zone données
i	:	1	
j	:	1	
	ENDC		; fin zone de données

Figure V.11.a.

exo7	movlw	2	
	banksel	i	
	movwf	i	; i = 2
	movlw	2	
	banksel	j	
	movwf	j	; j = 2
	xorwf	i,w	; ou exclusif bit à bit entre i et j
	btfss	STATUS,Z	
	goto	different	; i ≠ j
	goto	egal	; i = j
different	...		; traitement si différent
.	...		
.			
.			
egal	...		; traitement si égal
.	...		
.			
.			
	END		

Figure V.11.b.

V.8. « CONCEPTION » D'UN MODULE

Ecrire en assembleur la fonction **valeur absolue** dont le prototype C est le suivant :

unsigned int abs (signed int i)

Cette fonction sera réalisée sous la **forme d'un module** qui sera appelé depuis le main par la séquence de la figure V.12. suivante :

exo8	movf	i,W	; W contient la valeur du paramètre i du module
	call	abs	; appel du module abs
	nop		; au retour du module, W contient la valeur absolue de i

Figure V.12.

Pour simplifier, on raisonnera avec des entiers de taille **un octet** (et donc de type **char**)

Eléments de correction :

Profitons de cet exercice pour donner des éléments de **méthodologie** pour écrire « proprement » un module et pour le valider. En effet la notion de module est « **au cœur** » de la programmation modulaire. Il est donc fondamental de bien comprendre et de savoir franchir les différentes étapes figurant dans cette correction.

La méthode est un peu toujours la même (au moins pour des cas simples comme dans cet exercice). Nous donnerons des compléments pour des cas un peu plus compliqués dans le chapitre suivant.

Etape 1 : découpage d'un problème complexe en bouts de taille plus petite : élaboration du cahier des charges du module

Il s'agit en fait de repérer des modules de base, correspondants à des tâches facilement identifiables. En assemblant ces modules de base, on réalisera ensuite une application plus complexe.

Cette étape fait donc partie de la phase d'analyse qui se trouve en amont de toute écriture de programme. C'est peut-être la partie la plus délicate, car un mauvais découpage peut conduire à un programme final plus difficile à réaliser.

En TP, vous n'aurez pas (ou peu) à réfléchir sur cette étape car je vous indiquerai le (ou les) module(s) à écrire : ce sera la contrainte (on dit aussi le cahier **des charges**) à **respecter**. C'est en fait le module « vu de l'extérieur », en quelque sorte les « **crochets** » qui permettront de relier ce module avec les autres modules.

Il est important et impératif que vous respectiez ce cahier des charges : si vous ne mettez pas ces « **crochets** » aux bons endroits, ou si vous en oubliez, votre module sera inutilisable.

De plus, cette méthode de travail est un bon exemple de « type de travail en équipe » : chaque élément de l'équipe fait ce qu'il a à faire, sous la direction d'un responsable de projet qui a fait préalablement le découpage du travail.

Pour que la somme de tous les développements de toutes les personnes de l'équipe soit exploitable, on conçoit bien que chaque personne ne peut pas faire ce qu'il veut « dans son coin ». Le cahier des charges est là justement pour « canaliser » le travail de chacun.

Comment est fourni, au final, le cahier des charges du module à concevoir ?

La meilleure image est en donnée à mon avis par le prototype d'une fonction en C, qu'on rappelle ci-dessous :

```
type valeur_retournée    nom_de_la_fonction (liste_et_type_des_paramètres) ;
```

A ce niveau, on ne voit pas comment est codée « l'intérieur » de la fonction. Mais on a tous les éléments pour savoir comment elle se comportera « vue de l'extérieur ».

La programmation nécessite de la rigueur (notamment comme on vient de le voir par le respect des contraintes imposées) mais aussi de l'imagination. Il ne faut pas croire que vos capacités imaginatives sont bridées parce qu'on vous impose un cahier des charges strict à respecter

Votre espace de liberté, nécessaire à toute programmation, se trouve à l'intérieur du module.

Etape 2 : préparation de « l'encapsulation » ou comment le module est vu de l'extérieur

On commence par « renseigner » le module en indiquant sa **documentation** (ce que fait le module et comment on l'appelle et on l'utilise, **le (ou les) paramètre(s) en entrée**, la **valeur retournée en sortie**).

L'idée est de fournir suffisamment d'éléments à un utilisateur potentiel de ce module (et qui ne l'aura pas forcément écrit) pour qu'il puisse s'en servir **proprement**.

Pour cet utilisateur (qui ne sera pas forcément vous) le module apparaît comme une « boîte noire » avec des entrées (le (ou les) paramètre(s)) et une sortie (la valeur retournée). Un peu comme quand vous utilisez des fonctions toutes faites du C.

On place ensuite juste au-dessous **le point d'entrée du module** (label portant le **nom du module** à mettre en rapport avec le nom d'une fonction en C) puis l'instruction **return** (**point de sortie du module pour le retour**).

Tout ce qui est entre ces deux lignes et qui est repéré par les « » sur la figure V.13. constitue le **corps du module** et devra être complété dans l'étape suivante.


```
;*****  
; cadre donnant une petite documentation du module  
module abs : valeur absolue de prtotype : unsigned char abs (signed char)  
; entrée : W contient le signed char  
; sortie : W renvoie la valeur absolue  
;*****  
abs ; point d'entrée = nom du module  
  
..... ; corps du module  
..... ; corps du module  
  
return ; point de sortie = retour au main
```

Etape 3 : écriture du corps du module ou comment le module agit à l'intérieur

Il faut bien comprendre une chose (et que beaucoup d'étudiants ont du mal à bien saisir) :

Mais aucune référence directe à i ne doit être faite dans le module, sous peine de « casser » son caractère général. Par exemple, supposons que vous écriviez :

et que à l'intérieur du module, vous écriviez une ligne comme :

votre module ne marchera que si votre paramètre s'appelle i, ce qui n'est pas acceptable, car il perd ainsi son caractère général et générique.

et il faut faire avec ça et qu'avec ça.

Revenons maintenant au corps du module **abs**.

Ici, l'idée est de tester le **MSB** de **W** (bit de signe) :

- si ce bit est à **0** : alors le paramètre est positif et on le renvoie tel quel : $\text{abs}(i) = i$ si $i \geq 0$
- si ce bit est à **1** : alors le paramètre est strictement négatif et on doit calculer son complément à deux
(par inversion de tous ses bits puis par addition de 1) avant de le renvoyer : $\text{abs}(i) = -i$ si $i < 0$.

Pour tester le **MSB**, on va utiliser un **et logique bit à bit** avec le masque **B'1000 0000' = 0x80**.

Par contre, nous allons devoir utiliser une **case mémoire « tampon »** pour stocker le registre **W** avant de faire le masque.

Cette case tampon est une **variable locale** du module **abs** (même définition qu'une variable locale à une fonction en C).

En principe, cette variable locale doit être placée dans la pile mais dans le cas des PIC, ce n'est pas possible (et ça n'a pas été prévu pour) car la pile ne sert que pour stocker les adresses de retour des modules.

Il faut donc réserver de la place pour cette variable locale dans un **GPR**.

C'est le rôle de la déclaration suivante, faite dans la zone **CBLOCK/END** des variables globales non initialisées.

```
abs_loc_1          :      1      ; première variable locale du module abs  
                    ; abs comme nom du module où elle est utilisée  
                    ; loc comme locale  
                    ; 1 car c'est la première variable locale du module.
```

Cette convention pour nommer les variables locales n'est pas une obligation mais elle permet de mieux s'y retrouver (le compilateur C utilise d'ailleurs une méthode similaire) et de gagner un temps non négligeable pendant la phase de développement. On peut aussi remplacer le chiffre final (numéro de la variable locale) par un nom « parlant » pour cette variable. Par exemple ici : **tampon** ce qui donnerait le nom final suivant : **abs_loc_tampon** à notre variable locale. Je sais, c'est un peu lourd, mais je vous assure que cette méthode sera payante au final.

Le corps de la fonction commentée est fourni sur la figure V.14.

abs			; point d'entrée = nom du module
	movwf	abs_loc_1	; sauve le paramètre dans la variable locale
	andlw	0x80	; masque sur MSB (bit de signe)
	btfsc	STATUS,Z	; Z = 0 ? (MSB = 1 ?)
	goto	abs_positif	; non, paramètre positif
abs_negatif	comf	abs_loc_1	; complément à 1 (inversion de tous les bits)
	incf	abs_loc_1	; complément à deux (+1)
abs_positif	movf	abs_loc_1,w	; retour valeur absolue dans W
	return		; point de sortie = retour au main

Figure V.14.

Noter aussi les noms donnés aux labels à l'intérieur du module :

nom_du_module_xxxx

avec :

- **nom_du_module** = nom du module (ici abs)
- **xxxx** = nom « parlant » en rapport avec « l'algorithmie » du traitement effectué (ici positif ou négatif).

Là aussi, cette façon de nommer les labels intra-module n'est pas une obligation, mais encore une fois, cela permet de mieux s'y retrouver et de gagner du temps pendant la phase de développement.

Etape 4 : test et validation du module

Cette étape, souvent négligée par les étudiants, est pourtant **la plus importante**.

Au niveau de la programmation pure, c'est certainement l'étape la plus facile, car elle consiste à écrire un main qui a la forme de celui de la figure V.12. et qui est constitué de 3 parties :

- préparation des paramètres à passer au module (on verra dans le chapitre suivant comment faire quand il y a plus qu'un seul paramètre),
- appel du module,
- et un « nop » final pour indiquer le retour (ce « nop » disparaîtra bien sûr quand vous exploiterez ultérieurement ce module...)

Vous le voyez, ce main n'a pas d'autre intérêt que de « tester » le module en lui fournissant l'environnement minimal pour qu'il puisse fonctionner

Il permet de vérifier, une fois qu'il aura été exécuté, que le module a bien effectué le travail qu'on lui a demandé.

Mais cette dernière étape est fondamentale. En effet, suivant la « philosophie » de la **programmation modulaire**, chaque module est une **brique**.

Et comment construire un mur solide avec des briques qui n'auraient pas été testées à **150%** ?

Si vous mettez à disposition des modules non fiables, si vous les placez en librairies, c'est une véritable « bombe à retardement » que vous abandonnez derrière vous...

Un utilisateur doit avoir une sécurité absolue sur les modules (fonctions) qu'il met en œuvre.

Si son programme ne marche pas, il cherchera les erreurs dans ce qu'il a lui-même écrit et n'ira pas chercher l'erreur à l'intérieur des fonctions de la bibliothèque.

Imaginez que vous écriviez un programme en utilisant une fonction toute faite fournie par le C. Si votre programme ne marche pas, il faut que ça vienne de ce que vous avez écrit et pas de cette fonction (à la limite cela peut venir d'une mauvaise utilisation de votre part de cette fonction, mais pas du code de celle-ci). Sinon comme on dit familièrement, cela risque de vite devenir **indémer...**

Pour le test d'un module, il faut toujours penser aux problèmes des « effets de bord » :

- faire les tests avec les valeurs limites (par exemple le dernier élément avant qu'une condition (sur une égalité ou une inégalité) soit fausse),
- penser à utiliser toutes les valeurs possibles pour le (ou les) paramètre(s) d'un module et même celles auxquelles on n'a pas pensé,
- faites preuve d'imagination ...

V.9. LES « DIFFERENTS » MOV

On donne sur la figure V.15. les déclarations de variables globales suivantes, utilisées dans le source de l'**exo9**.

		CBLOCK	0x71 ; début zone données
car	:	1	
tab	:	4	
i	:	2	
		ENDC	; fin zone de données

Figure V.15.

1. Relevez, dans un tableau, les noms de ces variables, leur taille en octets, leurs adresses de début et de fin et enfin la bank de RAM à laquelle elles appartiennent.

2. Lorsque l'assembleur **MPASM** rencontre dans un source :

- **car** : par quelle valeur le remplace t'il ?
- **tab** : par quelle valeur le remplace t'il ?
- **tab+1** : par quelle valeur le remplace t'il ?
- **i** : par quelle valeur le remplace t'il ?

3. Les lignes assembleur suivantes, fournies sur la figure V.16. sont exécutées dans le source de **exo9**.

exo9	
	movlw 'A'
	banksel car
	movwf car
	movlw car
	incf car
	movf car,w
	movlw '1'
	movwf tab+1
	movf tab,w
	movlw tab
	bankisel tab
	movwf FSR
	decf tab+1,w
	movwf INDF
	END

Figure V.16.

Remplir le tableau fourni sur la figure V.18.

4. Conclure en faisant une analogie avec le C :

notamment avec l'opérateur **&** de calcul d'adresse et l'opérateur ***** d'indirection.

Eléments de correction :

1. Réponses sur la figure V.17. ci-dessous :

Nom variable	Taille en octets	Adresse de début	Adresse de fin	Bank
car	1	0x71	0x71	0
tab	4	0x72	0x75	0
i	2	0x76	0x77	0

Figure V.17.

2.

- **car** : sera remplacé (et donc vu par **MPASM**) par son adresse : 0x71
- **tab** : sera remplacé (et donc vu par **MPASM**) par son adresse : 0x72
- **tab+1** : sera remplacé (et donc vu par **MPASM**) par son adresse+1 : 0x73
- **i** : sera remplacé (et donc vu par **MPASM**) par son adresse : 0x76

3.

Instruction	Adressage	Contenu de	Explications
movlw 'A'	littéral	W = 0x41	code ASCII de A
banksel car	directive	RP1 = 0 RP0 = 0	sélection de la bank 0
movwf car	direct	case 0x71 : 0x41	W est recopié dans la case 0x71 la variable car contient 0x41='A'
movlw car	littéral	W = 0x71	l'adresse de car (0x71) est mise dans W
incf car	direct	case 0x71 : 0x42	incrémente le contenu de la case 0x71 avec résultat dans la case 0x71
movf car,w	direct	W = 0x42	on met dans W le contenu de la case 0x71
movlw '1'	littéral	W = 0x31	code ASCII de 1
movwf tab+1	direct	case 0x73 : 0x31	W est recopié dans la case 0x73 (tab+1) tab[1] prend la valeur 0x31
movf tab,w	direct	W = ???	on met dans W le contenu de la case 0x72 tab[0] n'était pas initialisé
movlw tab	littéral	W = 0x72	l'adresse de tab (0x72) est mise dans W
banksel tab	directive	IRP = 0	sélection de la page 0 (tab est en page 0)
movwf FSR	direct	FSR = 0x72	initialisation du FSR avec adresse de tab cette instruction est bien de l'adressage direct pour préparer l'adressage indirect
decf tab+1,w	direct	W = 0x30	décrémente le contenu de la case 0x73 avec résultat dans W
movwf INDF	indirect	case 0x72 : 0x30	copie W dans la case dont l'adresse est dans FSR , c'est-à-dire dans la case d'adresse 0x72 (tab[0])

Figure V.18.

4. Analogie avec le C (et à bien comprendre pour éviter de s'y perdre) :

Au final, si **var** est le nom d'une variable utilisée dans un programme assembleur :

movlw **var** ; adressage **littéral** : met dans **W** **l'adresse de cette variable**
; « analogie » avec le C :
; à rapprocher de quelque chose comme **W = &car**

Dans certains ouvrages, on représente ce transfert par **var** → **W**
(où **var** représente l'adresse de la variable)

Avec cette instruction, **il y aura toujours** la même valeur dans **W** : à savoir l'adresse de la variable.

movf **var,w** ; adressage **direct** : met dans **W** **le contenu de l'adresse var**
; c'est-à-dire que **W** contient **la valeur** de cette variable
; « analogie » avec le C :
; à rapprocher de quelque chose comme **W = car**

Dans certains ouvrages, on représente ce transfert par (**var**) → **W**
(où (**var**) entre parenthèses représente le contenu de l'adresse **var**)

Avec cette instruction, **il n'y aura pas toujours** la même valeur dans **W** : cela dépendra du contenu de l'adresse **var**, c'est-à-dire de la valeur de la variable **var** au moment où cette instruction est exécutée.

Au final, pour MPASM

le nom d'une variable déclarée est remplacé son adresse
et
la valeur de cette variable correspond au contenu de cette adresse.

Et si on suppose que **FSR** contient l'adresse de la variable **var** : **FSR = &var**

movf **INDF,w** ; adressage **indirect** : met dans **W** **le contenu de la case mémoire**
; **dont l'adresse est dans FSR** (on dit aussi pointée par **FSR**)
; c'est-à-dire que **W** contient **la valeur** de cette variable
; « analogie » avec le C :
; à rapprocher de quelque chose comme **W = *FSR (= *&car)**

Dans certains ouvrages, on représente ce transfert par ((**INDF**)) → **W**

Avec cette instruction, **il n'y aura pas toujours** la même valeur dans **W** : cela dépendra du contenu de l'adresse **var**, c'est-à-dire de la valeur de la variable **var** au moment où cette instruction est exécutée.

V.10. « POINTEUR DU C » ET « ADRESSAGE INDIRECT »

On donne, sur la figure V.19. le petit programme C suivant :

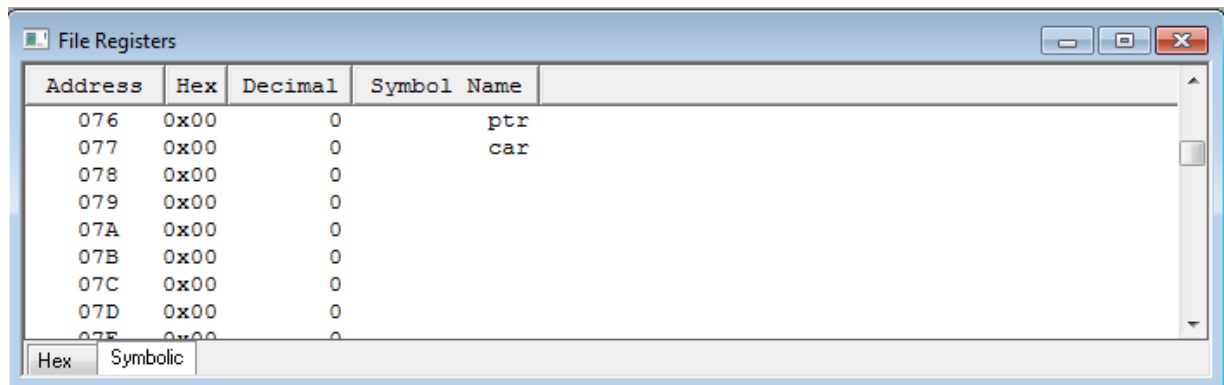
```
char *ptr;           // ptr est une variable de type pointeur sur un char
char car;            // car est une variable de type char

void main(void)
{
    car = 0xAA;

    ptr = &car;
    *ptr = 0x55;
}
```

Figure V.19.

Lorsqu'on visualise, après compilation de ce programme mais avant son exécution, l'état initial des **GPR**, on obtient l'écran de la figure V.20.

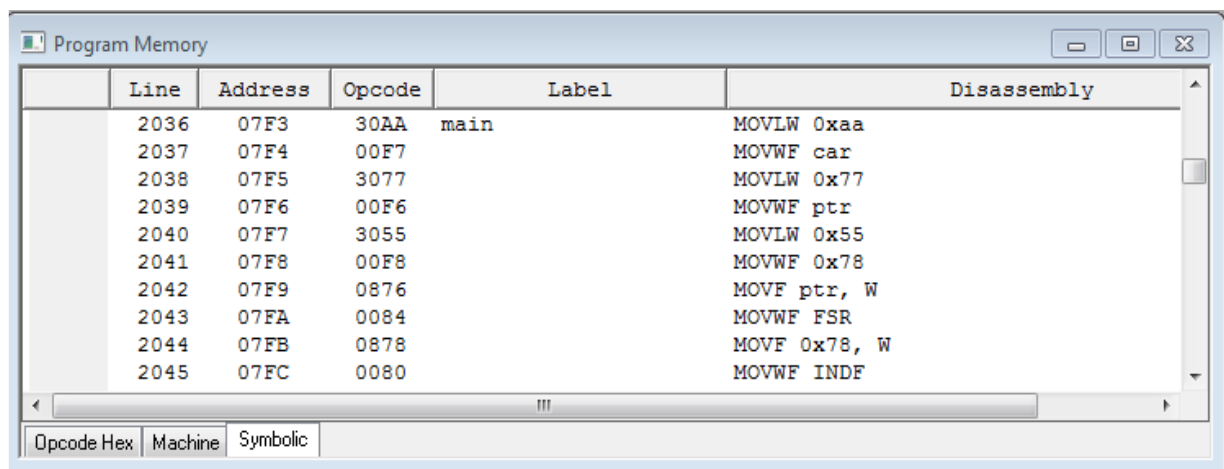


Address	Hex	Decimal	Symbol Name
076	0x00	0	ptr
077	0x00	0	car
078	0x00	0	
079	0x00	0	
07A	0x00	0	
07B	0x00	0	
07C	0x00	0	
07D	0x00	0	
07E	0x00	0	

Figure V.20.

1. Donner les adresses, affectées par le compilateur aux variables **ptr** et **car**. Quelles sont les valeurs **initiales** de ces variables ?

L'écran de la figure V.21. donne le code machine du programme compilé.



Line	Address	Opcode	Label	Disassembly
2036	07F3	30AA	main	MOVLW 0xaa
2037	07F4	00F7		MOVWF car
2038	07F5	3077		MOVLW 0x77
2039	07F6	00F6		MOVWF ptr
2040	07F7	3055		MOVLW 0x55
2041	07F8	00F8		MOVWF 0x78
2042	07F9	0876		MOVF ptr, W
2043	07FA	0084		MOVWF FSR
2044	07FB	0878		MOVF 0x78, W
2045	07FC	0080		MOVWF INDF

Figure V.21.

2. Donner l'adresse du **début du programme** et expliquer **son contenu**.

3. Repérer, sur l'écran de la figure V.21. les instructions assembleur correspondant aux affectations suivantes :

- car = 0xAA ; Mode d'adressage utilisé en assembleur ? Valeur de **car** après ?
- ptr = &car ; Mode d'adressage utilisé en assembleur ? Valeur de **ptr** après ?
- *ptr = 0x55 ; Mode d'adressage utilisé en assembleur ? Valeur de **car** après ?

Eléments de correction :

1.

Nom de la variable	Adresse de la variable	Valeur initiale de la variable
ptr	0x76	0x00
car	0x77	0x00

2. Le main commence à l'adresse **0x7F3** qui contient la valeur hexadécimale **0x30AA** correspondant au code machine de la première instruction du programme : **movlw 0xAA**.

3. Les différentes affectations du C sont repérées, en équivalence assembleur, sur l'écran de la figure V.22.

The screenshot shows a window titled "Program Memory" with a table of assembly instructions. Annotations on the right side link specific instructions to C code assignments:

- Annotation ①: **car = 0xAA** points to the instruction **MOVLW 0xaa** at line 2036.
- Annotation ②: **ptr = &car** points to the instruction **MOVLW 0x77** at line 2038.
- Annotation ③: ***ptr = 0x55** points to the instruction **MOVLW 0x55** at line 2040.

Line	Address	Opcode	Label	Disassembly
2036	07F3	30AA	main	MOVLW 0xaa
2037	07F4	00F7		MOVWF car
2038	07F5	3077		MOVLW 0x77
2039	07F6	00F6		MOVWF ptr
2040	07F7	3055		MOVLW 0x55
2041	07F8	00F8		MOVWF 0x78
2042	07F9	0876		MOVF ptr, W
2043	07FA	0084		MOVWF FSR
2044	07FB	0878		MOVF 0x78, W
2045	07FC	0080		MOVWF INDF

Figure V.22.

Précisions supplémentaires :

- ① l'affectation **car = 0xAA** correspond à de l'adressage **direct** par le : **movwf** **car** (précédé d'un adressage **littéral** pour mettre le littéral **0xAA** dans **W**).

Après cette affectation la variable **car** vaudra **0xAA**.

- ② l'affectation **ptr = &car** correspond à de l'adressage **direct** par le : **movwf** **ptr** (précédé d'un adressage **littéral** pour mettre le littéral **0x77** (adresse de **car**) dans **W**)

Après cette affectation la variable **ptr** vaudra **0x77**, c'est-à-dire l'adresse de **car**.

- ③ l'affectation ***ptr = 0x55** correspond à de l'adressage **indirect** par le : **movwf** **INDF** (toutes les instructions qui se trouvent avant (du **movlw** **0x55** au **movf** **0x78,W**) permettent de préparer cet adressage indirect via **FSR**)

La case d'adresse **0x78** ne sert ici que de tampon pour stocker le littéral **0x55** car le compilateur C « marche » de la droite vers la gauche. En effet sur une ligne comme ***ptr = 0x55**, il traite d'abord le membre de droite de l'affectation (le **0x55** qu'il stocke justement dans le tampon à l'adresse **0x78**) avant de traiter son membre de gauche.

Après cette affectation la variable **car** (pointée par **ptr**) vaudra **0x55**.

VI. EXEMPLES DE PROGRAMMES EN ASSEMBLEUR ET EN C

Tous les exemples présentés en langage C dans le chapitre précédent montrent qu'en programmant avec un langage de haut niveau, on n'a pas à connaître parfaitement l'architecture interne du micro/processeur/contrôleur car c'est le couple compilateur/linker qui assure l'interface avec le hardware.

Par contre, pour programmer en assembleur, il faut :

- bien connaître les **instructions** et les **directives d'assemblage**,
- s'assurer l'**accessibilité** des **SFR** et des **GPR** via les bits **RP1** et **RP0** de **STATUS** pour l'adressage **direct**,
- bien gérer le bit **IRP** de **STATUS** pour l'adressage **indirect**,
- bien **regarder et gérer** correctement les **flags** positionnés,
- faire attention aux **sauts de pages** dans la mémoire de programme,
- des **durées de développement** et de **mise au point** en général assez **longues**,
- **structurer** correctement son programme en le découpant en **modules**...

Tous ces éléments sont presque entièrement gérés par le compilateur C quand on programme en C.

Les avantages de la programmation en assembleur :

- est d'être **plus proche de l'électronique** (langage de « bas niveau »),
- avoir la **maîtrise complète** du système,
- générer des codes **plus courts** et donc moins gourmands en taille de mémoire,
- pouvoir mieux gérer et intégrer les contraintes **temps réel** notamment au niveau des **interruptions** avec un code **plus concis** et donc **plus rapide**...

Le principal inconvénient d'un programme écrit en assembleur est l'**absence de portabilité** entre différents types de micro/processeurs/contrôleurs, puisque chaque composant a son propre jeu d'instructions.

Les avantages de la programmation en langage C (ou autre type de langage évolué) :

- bénéficier des **instructions de contrôle de « haut niveau »** comme les boucles (while, for...), les tests conditionnels (if, switch...) inexistantes en assembleur,
- avoir une **structuration « naturelle »** des programmes grâce à ces instructions de contrôle,
- en général (si ce n'est toujours) des **durées de développement plus courtes**,
- avoir une **meilleure portabilité** (même si des « retouches » seront toujours nécessaires pour passer d'un microcontrôleur à un autre),
- langage évolué utilisé dans beaucoup de domaines (même s'il est de plus en plus en concurrence avec Java) et donc, quand on est étudiant, on ne perd jamais son temps quand on fait du C : toutes les occasions sont bonnes pour se « muscler » dans ce langage...

Le principal inconvénient d'un programme écrit en C est la **taille du programme** qui peut **rapidement augmenter** (surtout si le compilateur C n'est pas bien optimisé).

Comme sur les architectures à base de microcontrôleurs les capacités mémoire ne sont pas gigantesques, cette inflation de la taille des codes peut parfois poser quelques soucis.

L'autre inconvénient peut être dû au manque de « réactivité » du code machine produit par le compilateur C, ce qui peut être parfois gênant pour des aspects temps réel.

Nous allons étudier trois exemples de programmes dont les applications sont assez fréquentes et qui seront utilisés en TP.

VI.1. LES TEMPORISATIONS LOGICIELLES

On a souvent besoin d'introduire des **temporisations** (ou **délais** ou **attentes**) à certains moments **stratégiques** de l'exécution d'un programme.

Pour cela, le PIC dispose de 3 timers internes permettant de mesurer et de gérer le temps de façon très précise. Mais nous verrons plus tard, dans le chapitre **VIII.**, comment les utiliser.

Ici, on réalisera des temporisations grâce à de simples boucles. On parle alors de **temporisation logicielle** (par opposition à l'autre méthode utilisant les timers internes et appelée **temporisation matérielle**).

Cette méthode logicielle peut être utilisée lorsque l'application principale n'est pas critique au niveau temps réel (c'est-à-dire n'a pas beaucoup de choses à faire). En effet, une temporisation logicielle utilise la CPU du PIC pour « perdre » (consommer) du temps. Du coup, l'application ne peut pas faire autre chose pendant ce temps là.

L'idée de base consiste à initialiser une variable à une valeur donnée puis de la décrémenter en boucle jusqu'à ce qu'elle atteigne une valeur nulle. Comme ce type de traitement peut être utilisé à plusieurs endroits d'un programme, on le concevra directement sous la forme d'un sous-programme (module en assembleur ou fonction en C) qu'on appellera depuis le main chaque fois qu'on voudra réaliser la temporisation logicielle.

En pratique, on peut réaliser :

- des temporisations logicielles de **durée fixe**,
- des temporisations logicielles de **durée paramétrable**.

Les premières, comme leur nom l'indique, auront toujours la même durée. Ce qui nécessitera une adaptation de leur source si on souhaite obtenir des durées différentes.

Les secondes sont plus intéressantes car elles s'adapteront automatiquement (grâce à un paramètre) à la durée que l'on souhaite réaliser.

Nous allons présenter ici des exemples de durée fixe.

Et en TP, nous réaliserons des temporisations de durée paramétrable.

VI.1.1. CAS D'UNE SIMPLE BOUCLE

Examinons le module **tempo1** de la figure VI.1. ci-dessous :

```

;*****
;
; module tempo1 : temporisation void tempo1(void) : par simple boucle
;*****
tempo1                                ; point d'entrée = nom du module
    movlw    10                        ; valeur prise pour l'exemple
    movwf    tempo1_loc_1             ; variable locale 1 de tempo1
tempo1_encore  decfsz    tempo1_loc_1   ; tempo1_loc_1 --
    goto     tempo1_encore
    return                               ; point de sortie = retour au main

```

Figure VI.1.

Pour calculer la durée totale d'exécution de ce module, il faut le « dérouler mentalement » comme quand il s'exécute réellement sur le PIC, et compter le nombre de cycles, ce qui est résumé sur le tableau de la figure VI.2.

Instruction		Nombre cycles	Total
movlw	10	1	Exécutée une seule fois : 1
movwf	tempo1_loc_1	1	Exécutée une seule fois : 1
decfsz	tempo1_loc_1	1 si reste dans boucle 2 si saut hors boucle	Répétées 9 fois soit : $9 \times (2+1) = 27$ Quand saut hors boucle : Exécutée une seule fois : 2
goto	tempo1_encore	2 reste dans la boucle	
return		2	Exécutée une seule fois : 2
		Total	33

Figure VI.2.

Dans le cas général, si on appelle **x** la valeur littérale chargée dans **W** à la première instruction du module (ici x=10), alors le nombre de cycles N'_1 pris par le module **tempo1** est de :

$$N'_1 = 3.(x-1)+6 = 3x+3 \text{ cycles.}$$

En pratique, comme le module **tempo1** est appelé par une instruction **call** tempo1 depuis le programme principal (soit 2 cycles supplémentaires), il faut rajouter ces 2 cycles à N'_1 ce qui donne au final :

$$N_1 = 3x + 5 \text{ cycles pour la durée totale du module } \mathbf{tempo1}$$

(« aller+retour »)

On se rappelle (cf. chapitre **I.2.8.**) que la vitesse de traitement est de 1 Mips pour un quartz à 4 MHz, cela signifie qu'un cycle dure 1 μ s.

Et donc la durée T_1 du module **tempo1** sera de :

$$T_1 = (3x + 5) \mu s.$$

Remarque :

On peut utiliser l'instruction **nop** pour rajouter des cycles dans la boucle et modifier la formule précédente. C'est d'ailleurs un des intérêts de cette instruction un peu bizarre.

La valeur maximale de T_1 est obtenue pour la valeur maximale de x . De premier abord, on pourrait dire que **W** étant sur un octet, la valeur maximum est de $x = 0xFF = 255$.

Mais attention aux effets de bord : en fait, si on y regarde de plus près, la variable locale **tempo1_loc_1** est décrémentée **avant** d'être testée à 0 (par l'instruction **decfsz tempo1_loc_1**).

Donc si elle vaut **0x00** avant cette instruction, après la décrément, elle vaudra **0xFF** ($\neq 0x00$), ce qui veut dire qu'on restera dans la boucle.

Et donc, **bizarrement**, la valeur maximale T_{1max} de **tempo1** est obtenue pour $x = 256$ (dans la formule précédente mais obtenue par un **movlw 0x00** pour la première instruction du module **tempo1**) et vaut :

$$T_{1max} = 3.256 + 5 = 773 \mu s.$$

Assurez-vous d'avoir bien compris cette « singularité », car vous aurez à la gérer en TP dans le cas des temporisations paramétrables...

Si on veut des temporisations plus importantes, plusieurs techniques sont possibles et parmi celles-ci :

- on peut « **imbriquer** » des boucles les unes dans les autres,
- on peut appeler le module **tempo1** à l'intérieur d'un autre module **tempo2** (principe de la programmation modulaire).

VI.1.2. PAR IMBRICATION DE BOUCLES

Le principe est donné sur le source de la figure VI.3.

```
*****
;
; module tempo2 : temporisation void tempo2(void) : par boucles imbriquées
;*****
tempo2                                ; point d'entrée = nom du module
    movlw    10                        ; valeur prise pour l'exemple
    movwf    tempo2_loc_1             ; variable locale 1 de tempo2
    movlw    5                        ; valeur prise pour l'exemple
    movwf    tempo2_loc_2             ; variable locale 2 de tempo2
tempo2_encore  decfsz    tempo2_loc_1   ; tempo2_loc_1 --
               goto     tempo2_encore   ; boucle « profonde »
               movlw    10
               movwf    tempo2_loc_1    ; on recharge tempo2_loc_1
               decfsz    tempo2_loc_2    ; tempo2_loc_2 --
               goto     tempo2_encore    ; boucle « de surface »
               return                    ; point de sortie = retour au main
```

Figure VI.3.

Par un procédé analogue au cas précédent, on peut calculer le nombre de cycles de **tempo2**, qui dépendra :

- de x_1 : valeur littérale qui fixe la durée de la **boucle « profonde »** (ici $x_1=10$),
- de x_2 : valeur littérale qui fixe la durée de la **boucle « de surface »** (ici $x_2=5$).

Mais on voit que le calcul devient vite compliqué et un peu rébarbatif.

On verra d'ailleurs en TP comment **faire faire** ce calcul au simulateur de **MPLAB** grâce à la fenêtre « **Stopwatch** ».

Notons tout de même que rien n'empêcherait, sur le fond, de continuer ainsi les imbrications de boucles pour faire des temporisations de durées de plus en plus grandes.

VI.1.3. PAR PROGRAMMATION MODULAIRE

Il me semble plus intéressant, pour faire des temporisations plus longues (grosses briques) d'utiliser des temporisations plus courtes (petites briques). Cette méthode de procéder rentre mieux dans la démarche et « l'esprit » de la programmation modulaire.

Examinons le programme source de la figure VI.4.

```

;*****
;
; module tempo2 : temporisation void tempo2(void) : par appel de tempo1
;*****
tempo2                                ; point d'entrée = nom du module
                                ; valeur prise pour l'exemple
                                movlw      5
                                movwf      tempo2_loc_1 ; variable locale 1 de tempo2
tempo2_encore call      tempo1 ; appel de tempo1
                                decfsz      tempo2_loc_1 ; tempo2_loc_1 --
                                goto      tempo2_encore
                                return      ; point de sortie = retour au main

```

Figure VI.4.

Le source ainsi obtenu me semble plus lisible et plus facilement « debugable » que celui de la figure VI.3. avec deux boucles imbriquées (même si au final, là aussi les boucles sont imbriquées mais on ne le voit pas du fait d'une structuration du programme différente...).

Le nombre de cycles de **tempo2** peut se calculer d'une façon analogue en prenant en compte le fait qu'il s'agit de la même structure que **tempo1** (seule différence : rajouter les N_1 cycles de **tempo1** dans la boucle de **tempo2** et N_1 cycles lorsque l'on sort de cette boucle).

Ce qui donne (x_1 et x_2 ayant la même signification que précédemment) :

$$N'_2 = (x_2 - 1).(3+N_1) + N_1 + 6$$

Il faut de même rajouter les 2 cycles pour le **call** tempo2. Soit :

$$N_2 = N'_2 + 2 = (x_2 - 1).(3+N_1) + N_1 + 8 \quad \text{avec } N_1 = 3x_1 + 5$$

Ce qui donne au final :

$$N_2 = (x_2 - 1).(3x_1+8) + 3x_1 + 13 = (3x_1 + 8).x_2 + 5$$

Et, pour un quartz à 4 MHz :

$$T_2 = (x_2.(3x_1+8) + 5) \mu s$$

Exercice :

Calculer $T_{2 \max}$, durée maximale du module **tempo2**.

Réponse :

$T_{2 \max} = 198661 \mu s$ obtenu pour $x_1 = x_2 = 256$!!!

Ce procédé peut aussi être dupliqué. Mais, pour avoir une approche plus conviviale et plus facile à utiliser, il serait bien de disposer de temporisations paramétrables par l'utilisateur.

Par exemple :

- faire une temporisation de « courte durée » d'une durée **fixe** de **250 µs** : **tempo250**,
(en utilisant et adaptant les modules **tempo1** ou **tempo2** vu précédemment)
L'idée est de choisir au mieux x_1 ou x_1 et x_2 .
On pourra par exemple décomposer 245 (=250 – 5) en produit de facteurs premiers ...
- faire une temporisation de « moyenne durée -> » d'une durée **fixe** de **1 ms** : **tempo1ms**,
(en appelant par exemple 4 fois **tempo250**)
- faire une temporisation de « moyenne durée +> » d'une durée **fixe** de **4 ms** : **tempo4ms**,
(en appelant par exemple 4 fois **tempo1ms**)
- faire une temporisation de « longue durée » d'une durée **fixe** de **1 s** : **tempo1s**,
(en appelant par exemple 250 fois **tempo4ms**)

Ensuite, et **ensuite seulement**, on peut construire des temporisations **P**aramétrables de prototype :

- void **tempo_p_ms** (char ms) : **ms est le nombre de ms** de la durée souhaitée passé par **W**
qu'on testera comme indiqué ci-dessous :

```

                movf      ms,w ; si ms est une variable déclarée dans CBLOCK/ENDC
; ou            movlw     ms   ; si ms est défini par un EQU
; ou            movlw     5    ; avec une valeur de test fixe
; en fait la façon d'écrire l'instruction ci-dessus importe peu, car cette instruction ne sert ici qu'à
passer le paramètre pour tester le module tempo_p_ms.
```

```

call           tempo_p_ms
```

- void **tempo_p_s** (char s) : **s est le nombre de s** de la durée souhaitée passé par **W**
qu'on testera comme indiqué ci-dessous :

```

                movf      s,w ; si s est une variable déclarée dans CBLOCK/ENDC
; ou            movlw     s   ; si s est défini par un EQU
; ou            movlw     5    ; avec une valeur de test fixe
; en fait la façon d'écrire l'instruction ci-dessus importe peu, car cette instruction ne sert ici qu'à
passer le paramètre pour tester le module tempo_p_s.
```

```

call           tempo_p_s
```

Exercice :

Supposons qu'on utilise le module **tempo_p_s** pour faire une temporisation de 5 secondes.

Donner le nombre d'imbrications de modules pendant l'exécution de cette temporisation.

Faites le lien avec la pile.

Réponse :

① **tempo_p_s** appelle ② **tempo1s** qui appelle ③ **tempo4ms** qui appelle ④ **tempo1ms** qui appelle ⑤ **tempo250** qui appelle ⑥ **tempo1**.

Il y a donc **6 imbrications**. La pile n'a donc plus que **deux places** de libre.

Par contre, si on a, en plus, à gérer des interruptions, il faudra peut-être revoir cette séquence d'appels imbriqués, en prenant par exemple une tempo de bas niveau avec une durée plus importante (pour remplacer **tempo250**, ce qui est possible en faisant un autre choix pour x_1 et x_2).

On verra aussi en TP qu'avec ce procédé (intéressant pour la modularité) la précision se dégrade au fur et à mesure que les durées de temporisation augmentent (à cause des **call** et des **return** rajoutés à chaque nouvel appel ..)

VI.1.4. PAR LE LANGAGE C

Pour faire une temporisation en C, on a presque l'embarras du choix :

boucles **for**, **while**, **do ... while...**

Par contre, il est plus délicat de calculer « à la main » la durée réalisée car on ne sait pas, à priori, comment ces structures seront implantées en langage machine par le compilateur.

On est donc plus ou moins obligés (à part si on se plonge dans le code généré par le compilateur, ce qui peut être intéressant, mais qui est toujours assez long et fastidieux) de procéder **expérimentalement** en utilisant la fonction « **Stopwatch** » du simulateur.

C'est ce que j'ai fait dans les exemples suivants, en indiquant à chaque fois les valeurs de temporisations obtenues pour un quartz à 4 MHz.

- temporisations fixes :

Voici, sur la figure VI.5. trois exemples, parmi tant d'autres, d'une fonction de temporisation **tempo_1ms** d'une durée fixe la plus proche possible de **1 ms**

boucle for	boucle while	boucle do ... while
// tempo_1_ms : boucle for // dure 995 µs pour t1ms=76 // dure 1008 µs pour t1ms=77 void tempo_1ms(void) { unsigned int i; for(i=0;i< t1ms ;i++); }	// tempo_1_ms : boucle while // dure 991 µs pour t1ms=75 // dure 1006 µs pour t1ms=76 void tempo_1ms(void) { unsigned int i=0; while(i< t1ms) i++; }	// tempo_1_ms : boucle do while // dure 995 µs pour t1ms=76 // dure 1008 µs pour t1ms=77 // équivalent à la boucle for void tempo_1ms(void) { unsigned int i=0; do i++; while(i< t1ms); }

Figure VI.5.

Sur ces trois exemples, la constante **t1ms** a été définie par une directive **#define t1ms**

Remarque :

On pourrait facilement faire la même chose pour une fonction **tempo_1s** dont la durée serait le plus près possible de 1 seconde, en définissant et adaptant la valeur d'une constante **t1s**.

En pratique, il vaut mieux prendre une durée un peu plus faible car on peut toujours l'augmenter, sans en changer le code fonctionnel en « rajoutant des nop » (une diminution est plus problématique ...).

A ce sujet, voici comment on rajoute une (ou plusieurs) instructions assembleur **au sein** d'un programme écrit en C (on dit aussi qu'on rajoute de l'assembleur **in line**).

- pour **une seule** instruction :

```
asm("instruction");          // exemple asm("movlw    0x55");
```

- pour **un bloc de plusieurs instructions**, il faut utiliser les directives **#asm / #endasm** :

```
#asm
instruction 1          // exemple : movlw    0x35
instruction 2          // exemple : andlw    0x0F
instruction 3          // exemple : clrw
#endasm
```

Mais attention, il y a toujours le risque, quand on rajoute des instructions assembleur au sein d'un programme en C, qu'elles viennent perturber le fonctionnement normal du programme, comme dans les exemples ci-dessus où on modifie **W** par ces instructions rajoutées, alors que **W** pouvait à cet instant contenir une valeur importante pour le déroulement du programme en C.

Par contre, si on rajoute des **nop**, comme on va le faire, il n'y a aucun souci.

Prenons le cas de la tempo avec la boucle **for** (ou **do ... while**) et $t1ms = 76$. Elle dure en fait 995 μs .

Combien de **nop** faut-il rajouter et à quel endroit ?

Vous l'aviez sans doute deviné, il faut 5 **nop** (une instruction **nop** consomme un cycle soit 1 μs).

Si on la place dans la boucle **do ... while**, elle sera exécutée 76 fois, ce qui est trop.

Il faut donc placer ces « 5 **nop** » après la déclaration de la variable locale *i* et avant la boucle **do ... while**.

Remarque:

*Si on remplace le type **unsigned int** de la variable locale *i* de `tempo_1ms` par un type **unsigned char**, la tempo dure 694 μs !!! Pourquoi cette importante diminution ?*

Il faut se rappeler qu'un (unsigned) int est codé sur 16 bits et qu'un (unsigned) char l'est sur 8 bits.

*Pour gérer par exemple une instruction comme `i++`, il faudra moins d'instructions si *i* est codé sur 8 bits que si *i* est codé sur 16 bits, car dans ce dernier cas il faut s'occuper de l'octet de poids fort **et** l'octet de poids faible de *i* (rappelez vous le PIC 16F877 est un microcontrôleur basé sur une architecture 8 bits).*

D'où la diminution de la durée de la tempo, lorsqu'on change le type de la variable locale.

*Que se passe t'il, à votre avis, si on fait la déclaration **unsigned long** int *i* ?*

- temporisations paramétrables :

On peut aussi adapter les fonctions précédentes pour faire des temporisations de durées variables de prototype :

void **tempo_p_ms** (unsigned int ms) ; tempo de ms milliseconde(s)

void **tempo_p_s** (unsigned int s) ; tempo de s seconde(s)

Finalement, les deux principaux inconvénients des temporisations logicielles (fixes ou paramétrables) sont :

- un manque de précision,
- leur durée dépend du quartz utilisé : si on veut utiliser ces modules tel quel sur un PIC ayant un quartz de fréquence différente, les durées changeront ...

Du coup, comme nous l'avions dit au début, la meilleure méthode est d'utiliser les timers internes du PIC qui offrent une meilleure précision.

Ce qui sera vu plus tard, dans le chapitre consacré au timer.

VI.2. ACCES A UNE ZONE DE MEMOIRE EN ADRESSAGE INDIRECT

VI.2.1. REMPLISSAGE D'UNE ZONE DE MEMOIRE EN ASSEMBLEUR

Il est fréquent de travailler avec des zones mémoire constituées de cases contiguës (initialisation d'une zone de mémoire, déplacement d'une zone de mémoire vers une autre, comparaison de deux zones de mémoire...).

L'**adressage indirect** est bien adapté à cette problématique. En fait, l'idée de base est d'utiliser un pointeur sur la zone de travail et de gérer ensuite ce pointeur (par incrémentation le plus souvent) pour accéder aux différentes cases mémoire de cette zone.

Supposons par exemple qu'on veuille mettre à 0 tous les **GPR** de la **zone commune** de la RAM (c'est-à-dire entre les adresses **0x70** et **0x7F** incluses).

Sans l'utilisation de pointeurs, on pourrait écrire la séquence d'instructions suivante :

```
movlw    0x00 ; valeur d'initialisation commune pour toutes les cases mémoire
movwf    0x70 ; mise à 0 de la première case de la zone (par adressage direct)
movwf    0x71 ; mise à 0 de la deuxième case de la zone (par adressage direct)
:
:
movwf    0x7F ; mise à 0 de la dernière case de la zone (par adressage direct)
```

Le but serait atteint, mais cette façon de procéder n'est pas élégante, utilise beaucoup d'instructions, n'est pas souple, n'est pas générale et donc n'est pas ré exploitable.

L'utilisation d'un pointeur vers la zone permettra d'implanter l'algorithme fourni sur la figure IV.6.

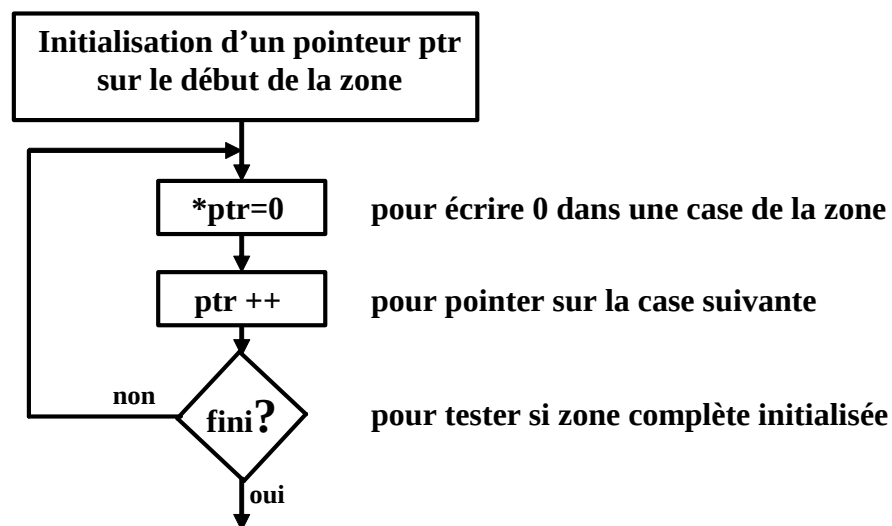


Figure VI.6.

En fait, on vient de décrire là l'équivalent d'une boucle **for** ou (**do... while**) du langage C.

L'intérêt est évident : on pourra, uniquement en changeant la valeur du pointeur, initialiser une autre zone mémoire, contrairement à la méthode précédente où il fallait changer toutes les adresses de chaque instruction du programme.

On peut même concevoir cette application directement sous la forme d'un module, dont le prototype en C (donc le cahier des charges à respecter) serait le suivant :

```
void remplir_zone (char* debut_zone, char* fin_zone, char initialisation) ;
```

et dont le fonctionnement est assez simple à comprendre : remplir toute la zone de mémoire comprise entre les adresses **debut_zone** et **fin_zone** incluses avec la valeur **initialisation**.

Notons également que cette fonction ne renvoie aucune valeur, puisqu'elle est de type void.

Remarque :

On aurait pu aussi choisir le prototype suivant :

```
void remplir_zone (char* debut_zone, char taille_zone, char initialisation) ;
```

Pour la méthodologie de conception d'un module, il est conseillé de relire la correction de **l'exercice V.8.** du chapitre précédent. Mais ici, contrairement à l'exemple traité dans cet exercice, nous avons trois paramètres pour le module **remplir_zone**.

En principe, lorsque la pile est accessible au programmeur, les paramètres d'une fonction sont passés par la pile. Or, on a vu que dans le cas des PIC, ce n'est pas possible (la pile ne servant que pour les adresses de retour). Il faudra donc passer les paramètres au module avec un autre procédé.

Le paramètre le plus à droite (dans la liste des paramètres) sera ici passé par **W**.

Les deux autres paramètres seront passés par registre **GPR**. Pour nommer ces registres, et leur réserver de la place, on utilisera la convention suivante :

nom_du_module_par_1 pour paramètre **1** du module **nom_du_module**
nom_du_module_par_2 pour paramètre **2** du module **nom_du_module**

On peut aussi remplacer le chiffre final (numéro du paramètre) par un nom « parlant » pour ce paramètre, ce qui permet peut-être de mieux s'y retrouver.

C'est que nous ferons avec les déclarations fournies sur la figure VI.7. suivante :

CBLOCK		0x20	; début zone données
remplir_zone_par_debut_zone	:	1	; pointeur sur le début de la zone concernée
remplir_zone_par_fin_zone	:	1	; pointeur sur la fin de la zone concernée
ENDC			; fin zone de données

Figure VI.7.

Et ainsi, « le main de test » du module **remplir_zone** s'écrira comme indiqué sur la figure VI.8. :

	ORG	0x0000	; vecteur de reset
	goto	main_de_test	
main_de_test			
	movlw	0x70	
	movwf	remplir_zone_par_debut_zone	; adresse première case
	movlw	0x7F	
	movwf	remplir_zone_par_fin_zone	; adresse dernière case
	movlw	0x00	; valeur d'initialisation
	call	remplir_zone	
	nop		

Figure VI.8.

Remarque :

*On aurait pu aussi passer le pointeur du début de zone directement par **FSR**.*

Il ne reste plus qu'à écrire le module concerné, en utilisant l'adressage indirect (bien lire les commentaires fournis sur le source de la figure VI.9. pour comprendre la logique de traitement).

;*****			
; module : void remplir_zone (char*, char*, char)			
; entrée : remplir_zone_par_debut_zone : pointeur sur le début de la zone à initialiser			
; entrée : remplir_zone_par_fin_zone : pointeur sur la fin de la zone à initialiser			
; entrée : W contient la valeur d'initialisation			
;*****			
remplir_zone			
	banksel	remplir_zone_par_debut_zone	; accès aux variables en bank 0
	bcf	STATUS,IRP	; zone en page 0
	movwf	remplir_zone_loc_tampon	; locale tampon
	movf	remplir_zone_par_debut_zone,w	; initialisation du pointeur
	movwf	FSR	; ptr = debut_zone
remplir_zone_encore			
	movf	remplir_zone_loc_tampon,w	
	movwf	INDF	; *ptr = 0x00
	incf	FSR	; ptr ++
	movf	FSR,w	
	subwf	remplir_zone_par_fin_zone,w	
	btfss	STATUS,Z	; fin_zone – ptr == 0 ?
	goto	remplir_zone_encore	; non, alors on continue
	return		; oui, c'est fini

Figure VI.9.

Finalement, le source proposé est assez proche de l'organigramme qu'on avait donné en introduction sur la figure VI.6.

Les erreurs fréquentes avec l'adressage indirect :

- oubli d'initialiser le pointeur via **FSR** : dans ce cas là, le registre **FSR** contient n'importe quoi et on n'accèdera pas à la case désirée,
- confusion entre la valeur du pointeur (une adresse) et la valeur de la variable pointée par ce pointeur (le contenu de la case mémoire dont l'adresse est la valeur du pointeur). Ce qui revient en langage C à confondre (si ptr est une variable pointeur) :

ptr et *ptr

- confusion entre l'adresse de la case mémoire où est stockée ce pointeur et l'adresse fournie par ce pointeur. Ce qui revient en langage C à confondre (si ptr est une variable pointeur) :

ptr et &ptr

L'exemple que l'on vient de traiter ici nous servira comme « tutorial » en TP pour apprendre à utiliser les outils de travail qui seront à notre disposition, notamment l'environnement fourni par **MPLAB**.

Il est important de bien l'avoir compris.

REMARQUE : sur la gestion des **variables locales** et des **paramètres** d'un module.

Cette méthode de gestion est basée sur la notion de **recouvrement de données (data overlaid)**.

Le principe est le suivant : deux variables différentes peuvent être situées à la même adresse à condition :

- qu'elles ne soient pas utilisées au même moment,
- qu'elles ne soient pas utilisées pour conserver une valeur temporaire qui sera utilisée plus loin dans le programme.

Le cas des variables locales (**non static**), utilisées dans un module, respecte bien ces deux conditions :

- elles ne sont utilisées que **lorsque ce module est exécuté**,
- elles **n'ont pas à conserver leur valeur** entre deux appels de ce même module.

Il en est de même pour les paramètres transmis à un module.

Or la taille mémoire sur un microcontrôleur n'étant pas gigantesque, toute économie est bonne à prendre. L'idée est de donc de faire du **recouvrement de données**.

Comme on a plus ou moins normalisé les noms de ces variables locales et de ces paramètres en les indexant avec un numéro, on fera **partager** la même case à toutes les variables locales (à condition bien sûr qu'elles aient la même taille) portant le même numéro dans tous les modules. Et idem avec les paramètres.

Exemple :

Supposons qu'on ait trois modules de noms :

- module1 (avec 3 variables locales),
- module2 (avec 2 variables locales),
- module3 (avec 1 variable locale)

on écrira alors :

```
#define module1_loc_1 loc_1 ; les trois variables locales de numéro 1 de
#define module2_loc_1 loc_1 ; module1, module2 et module3
#define module3_loc_1 loc_1 ; partagent la même adresse loc_1

#define module1_loc_2 loc_2 ; les deux variables locales de numéro 2 de
#define module2_loc_2 loc_2 ; module1 et module2 sont à l'adresse loc_2

#define module1_loc_3 loc_3 : la variable locale 3 de module1 est en loc_3
```

Ainsi, au lieu d'occuper 6 (3+2+1) cases, 3 suffiront.

Cette technique est intéressante, mais elle a, hormis les problèmes d'identité de taille dont on vient de parler, deux limitations importantes :

- elle ne s'applique pas si les modules sont imbriqués : en effet, lorsqu'un module en appelle un autre, il faut que ses propres variables locales conservent leurs valeurs au retour du module appelé,
- et en conséquence, elle ne s'applique pas aux variables locales des interruptions, puisque une interruption pouvant arriver n'importe quand et donc aussi pendant l'exécution d'un module, se retrouve alors de fait imbriquée à l'intérieur de ce module.

VI.2.2. REMPLISSAGE D'UNE ZONE DE MEMOIRE EN C

On peut aussi écrire le module précédent en langage C, ce qui est fait sur la figure VI.10.

```
// void remplir_zone(char* debut, char* fin, char init)

void remplir_zone(char* debut, char* fin, char init)
{
    char * ptr;
        ptr = debut;                ; initialisation du pointeur
        do
        {
            *ptr=init;              ; mise à 0 de la case pointée
            ptr++;                  ; pour pointer sur la case suivante
        }
        while (ptr < fin);          ; tant que la zone n'est pas entièrement initialisée
}
```

Figure VI.10.

Pour tester ce programme, il y a un petit piège : si on le teste pour effacer la zone commune de la RAM (entre les adresses **0X70** net **0x7F**) comme précédemment en assembleur, ce programme ne fonctionne pas. En fait, le compilateur C stocke la variable locale **ptr** de la fonction **remplir_zone** justement dans cette zone commune. Du coup, la fonction **remplir_zone**, qui est conçue pour effacer la RAM, efface aussi la valeur de ce pointeur :

tout se passe un peu comme si ce module « s'auto-détruisait » !!!

VI.3. POSITIONNEMENT ET TEST DE BITS INDIVIDUELS

J'ai choisi de traiter cet aspect car un microcontrôleur est souvent amené à :

- soit positionner certains bits de ses **SFR** : configuration de ses périphériques internes, initialisation diverses, choix de modes de fonctionnement ...
- soit tester certains bits de ses **SFR** : arrivée ou pas d'un évènement extérieur, durée écoulée ou pas ...

En assembleur, on l'a vu dans le **chapitre IV**, il existe 4 instructions pour réaliser cela :

- le **positionnement** à **1** du bit **b** (0 à 7) du **SFR f** : **bsf** **f,b**
- le **positionnement** à **0** du bit **b** (0 à 7) du **SFR f** : **bcf** **f,b**
- le **test** à **1** du bit **b** (0 à 7) du **SFR f** : **btfss** **f,b**
- le **test** à **0** du bit **b** (0 à 7) du **SFR f** : **btfsc** **f,b**

Les organigrammes de ces deux dernières instructions sont fournis sur les figures VI.11.a. et VI.11.b.

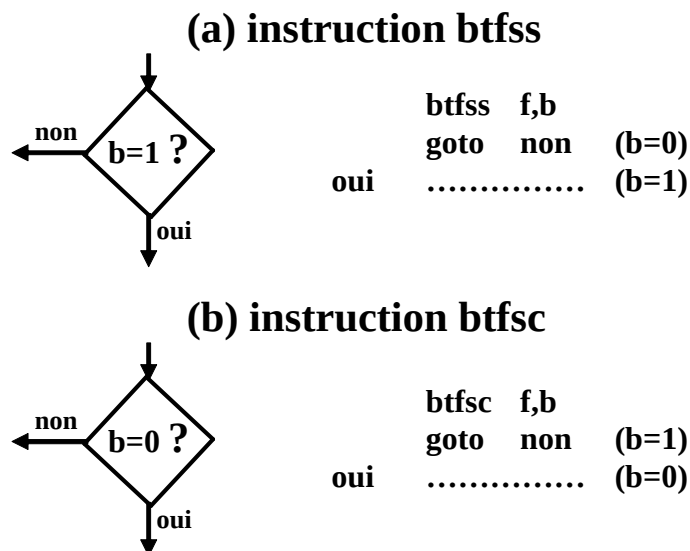


Figure VI.11.

Comme à terme, le PIC sera programmé en C, j'ai préféré donner plus d'informations sur la façon de procéder dans ce langage évolué.

VI.3.1. POSITIONNEMENT DE BITS INDIVIDUELS EN C

En C, il existe trois possibilités pour positionner un bit dans un **SFR** de nom est **FOO** :
(le terme FOO est souvent utilisé en informatique pour nommer quelque chose)

- la technique par des masques :

Les masques à utiliser, sont construits comme indiqué sur le tableau de la figure VI.12.

Bit concerné	Valeur du masque en binaire	Valeur du masque en hexadécimal
0	0b 0000 000 1	0x01
1	0b 0000 00 10	0x02
2	0b 0000 0 100	0x04
3	0b 0000 1000	0x08
4	0b 000 1 0000	0x10
5	0b 00 10 0000	0x20
6	0b 0 100 0000	0x40
7	0b 1000 0000	0x80

Figure VI.12.

Deux exemples pour illustrer la technique des masques :

- mise à **1** du bit **3** du registre FOO :
FOO = FOO | 0x08 ; // opérateur **ou logique** bit à bit | avec le masque
(ou FOO |= 0x08 ;) // idem
- mise à **0** du bit **3** du registre FOO :
FOO = FOO & (~0x08) ; // opérateur **et logique** bit à bit & avec le masque **inversé**
(ou FOO &= ~0x08 ;) // idem

On peut aussi rajouter la ligne suivante dans son source en C :

```
#define bit(x) (1 << (x)) // décalage de 1 = 0b 0000 0001 de x bits sur la gauche : opérateur <<
```

Ainsi, on n'a plus besoin de calculer soi-même les masques en hexadécimal.
Les deux exemples suivants ont les mêmes effets que les deux précédents :

- mise à **1** du bit **3** du registre FOO :
FOO = FOO | bit(3) ; // opérateur **ou logique** bit à bit | avec bit(3)
(ou FOO |= bit(3) ;) // idem
- mise à **0** du bit **3** du registre FOO :
FOO = FOO & (~bit(3)) ; // opérateur **et logique** bit à bit & avec bit(3) **inversé**
(ou FOO &= ~bit(3) ;) // idem

- la technique par des macros :

Il faut alors définir les deux macros suivantes dans son source :

```
#define bitset(var, bitno)    ((var) |= 1UL << (bitno))    // mise à 1 du bit bitno de var
#define bitclr(var, bitno)   ((var) &= ~(1UL << (bitno)))  // mise à 0 du bit bitno de var
```

Ainsi les deux exemples précédents s'écriront :

- mise à **1** du bit **3** du registre FOO :
 bitset(FOO,3) ;
- mise à **0** du bit **3** du registre FOO :
 bitclr(FOO,3) ;

- la technique par les noms symboliques des bits:

Les bits individuels de la plupart des registres **SFR** ont été déclarés (dans le fichier header pic.h) et sont reconnus par le couple compilateur/linker. Il faut bien sûr garder les mêmes noms de bits que ceux fournis et utilisés par **MICROCHIP** dans le datasheet du PIC (et ne pas inventer ses propres noms !!!)

Par exemple, le bit **0** du registre **SFR T1CON** (comme **TIMER1 CONTROL**) s'appelle **TMR1ON** (comme **TIMER1 ON**) : on le verra plus loin ce bit sert, lorsqu'on le fait passer à 1 à mettre en route le TIMER 1 du PIC. Et à arrêter ce même timer quand on le fait passer à 0.

Il suffira alors d'écrire la ligne suivante en C pour démarrer le timer 1 :

```
TMR1ON = 1 ;           // mise en route du timer 1 du PIC
```

ou la ligne suivante pour l'arrêter :

```
TMR1ON = 0 ;           // stoppe le timer 1 du PIC
```

Tous ces points seront bien sûr revus et utilisés dans le **chapitre VIII.** de ce polycopié qui détaillera tous les périphériques internes du PIC.

VI.3.2. TEST DE BITS INDIVIDUELS EN C

Dans une application à base de microcontrôleur interfacée avec l'extérieur, il est fondamental de savoir lorsqu'un événement extérieur (et nécessitant un traitement) s'est produit, justement pour lancer ce traitement.

Les exemples sont nombreux (liste non exhaustive) :

- une ligne d'un PORT (programmée en entrée) est passée du niveau bas au niveau haut ou du niveau haut au niveau bas,
- un caractère est arrivé sur une liaison série asynchrone,
- un caractère a été entièrement transmis sur une liaison série asynchrone,
- un intervalle de temps s'est écoulé,
- un convertisseur analogique/numérique a terminé sa conversion...

Chacun de ces événements se traduit au niveau du PIC par le passage de **0 à 1** (le plus souvent) ou de **1 à 0** d'un bit particulier situé dans un **SFR** du PIC. Et donc, chacun de ces événements est associé à ce bit qu'on appelle aussi un **flag** (ne pas confondre avec les flags Z, C et DC de la CPU du PIC).

Lorsqu'on ne travaille pas sous interruption (voir plus loin dans la partie consacrée aux interruptions), la technique à mettre en œuvre pour détecter l'apparition d'un événement s'appelle la **scrutation** (ou **pooling** en anglais). Elle consiste à surveiller l'état du flag associé à cet événement en attendant par exemple que ce flag passe de 0 à 1 (ou de 1 à 0).

Il existe plusieurs procédés pour effectuer ce pooling en C.

- la technique par des masques :

Quelques exemples pour illustrer la technique de pooling avec des masques :

```
// attente que le bit 7 du registre SFR FOO passe à 1
while ( !(FOO & 0x80) );           // tant que le bit 7 de FOO est à 0, on attend
...                               // à partir de là le bit 7 de FOO est à 1

// autre écriture du même exemple si on a écrit dans son source : #define bit(x) (1 << (x))
while ( !(FOO & bit(7)) );         // tant que le bit 7 de FOO est à 0, on attend
...                               // à partir de là le bit 7 de FOO est à 1

// attente que le bit 0 du registre SFR FOO passe à 0
while ( FOO & 0x01 );              // tant que le bit 0 de FOO est à 1, on attend
...                               // à partir de là le bit 0 de FOO est à 0

// autre écriture du même exemple si on a écrit dans son source : #define bit(x) (1 << (x))
while ( FOO & bit(0) );            // tant que le bit 0 de FOO est à 1, on attend
...                               // à partir de là le bit 0 de FOO est à 0
```

- la technique par les noms symboliques des bits:

Voyons également deux exemples pour illustrer la technique de pooling en utilisant le nom symbolique du bit testé.

Exemple 1 :

Il existe, pour le timer 0 du PIC, un bit particulier qui indique, lorsqu'il passe à 1 que le timer 0 est passé de **0xFF** à **0x00** : on dit que le timer 0 a subi un **overflow** (un dépassement de capacité). Ce bit s'appelle d'ailleurs le bit **TOIF** (**T**imer **O**verflo**w** **F**lag). C'est le bit 2 du registre SFR **INTCON**.

Si on souhaite détecter par pooling, le passage de 0 à 1 de ce bit TOIF, il faut réaliser, l'organigramme de la figure VI.13. ci-dessous :

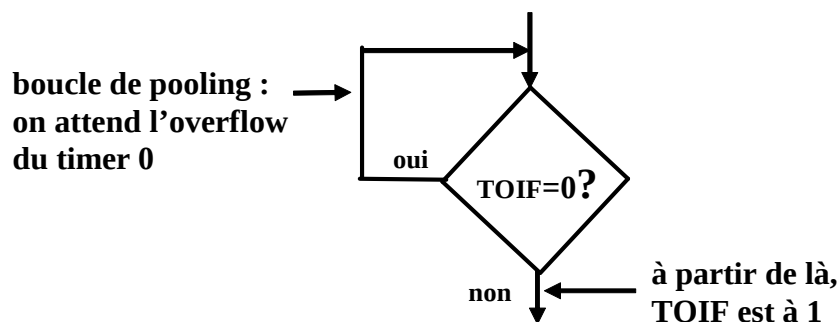


Figure VI.13.

La ligne suivante permet d'attendre que le flag **TOIF** passe à 1.

// attente que le flag **TOIF** (bit 2 du registre **SFR INTCON**) passe de 0 à 1

```
while( !TOIF );    // pooling sur TOIF, on attend tant que TOIF = 0
...                // à partir de là le flag TOIF est à 1
```

Exemple 2 :

// attente que la ligne RA4 (bit 4 du PORTA, supposée programmée en entrée) passe de 1 à 0

```
while(RA4);        // pooling sur RA4, on attend tant que RA4 = 1
...                // à partir de là, l'entrée RA4 est à 0
```

On pourrait multiplier les exemples en assembleur en écrivant des modules pour :

- faire des conversions entre différents types de codage (hexadécimal, DCB, ASCII ...),
- pour faciliter la gestion des chaînes de caractères (longueur, comparaison, concaténation...),
- pour multiplier et diviser deux opérandes ...

Certains seront réalisés en TP pour se familiariser avec l'assembleur.

Mais comme en pratique, on programme le plus souvent en C, tous ces modules existent déjà sous forme de fonctions dans les différentes bibliothèques du C.

Ce qui permettra de se concentrer davantage sur son application et sur la gestion des différents périphériques du PIC (cf. **chapitre VIII**.)

VII. METHODOLOGIE DE TRAVAIL EN TP

Il existe trois grandes méthodes pour développer des applications utilisant des PIC.

- méthode dite de « **simulation** » : **qui ne coûte rien**, pourvu qu'on dispose d'un PC (idéale pour les étudiants qui par essence ne sont pas très riches),
- méthode dite de « **in circuit débogage** » : idéale pour **la mise au point** des programmes,
- méthode dite de « **programmation** » : pour le stade final de tout projet qui tend à rendre votre application totalement **autonome**.

Nous aurons l'occasion de mettre en œuvre ces trois méthodes en TP.

Voyons en déjà ici les points essentiels.

VII.1. SIMULATION

Grâce au logiciel **MPLAB** (à télécharger **gratuitement** sur le site de **MICROCHIP**) vous disposez d'un environnement de développement assez puissant pour créer et faire évoluer vos applications.

Cet environnement fournit entre autre :

- des outils de gestion de projets,
- des outils d'édition des fichiers sources,
- des outils de compilation/linkage...

Vous pourrez programmer :

- **en assembleur** : la suite **MPASM/MPLINK/MPLIB** est fournie en standard avec **MPLAB**,
- **en langage C** : des versions gratuites et compatibles avec **MPLAB** (mais à durée limitée) existent aussi sur le net.

Le **simulateur** est également fourni avec **MPLAB**. Vous travaillez alors dans un environnement **virtuel** reproduisant le fonctionnement du PIC de votre choix.

Pour la mise au point des programmes, on dispose, sous contrôle du simulateur :

- du mode d'exécution **en pas à pas** (ou mode **single step** ou de **tracing**) soit au niveau du langage, soit au niveau du code machine,
- de la possibilité de visualiser l'évolution des **SFR** et des **GPR** sous forme symbolique grâce à des fenêtres « **watch** » permettant de suivre vos variables et donc de corriger les erreurs dans vos programmes (attention, il s'agit ici **des erreurs sur la logique de votre programme** et **pas des erreurs de syntaxe** qui ont été détectées (et corrigées par vous) au moment de la compilation),
- et de plein d'autres possibilités qu'on détaillera en TP.

Il est également possible de simuler des événements extérieurs ce qui permet de vérifier que votre programme répond bien aux modifications de son environnement et peut agir sur lui sous contrôle de votre programme.

Bref méthode idéale pour vous muscler (pour pas cher) en PIC !!!

VII.2. DEBUGAGE SUR L'APPLICATION

Cette deuxième méthode de travail est assez proche de ce que l'on fait dans l'industrie lorsque l'on conçoit de nouvelles applications à base de micro/processeur/contrôleur.

Elle nécessite en, plus de **MPLAB**, d'avoir deux matériels supplémentaires situés à l'extérieur de votre PC mais qui y seront connectés :

- un boîtier type **ICDx** (comme **In Circuit Débugueur**),
- une carte d'évaluation comportant le PIC 16F877 et quelques périphériques d'interface. En TP, nous utiliserons la carte **PICDEM 2 PLUS**.

La figure VII.1. montre les liaisons à réaliser.

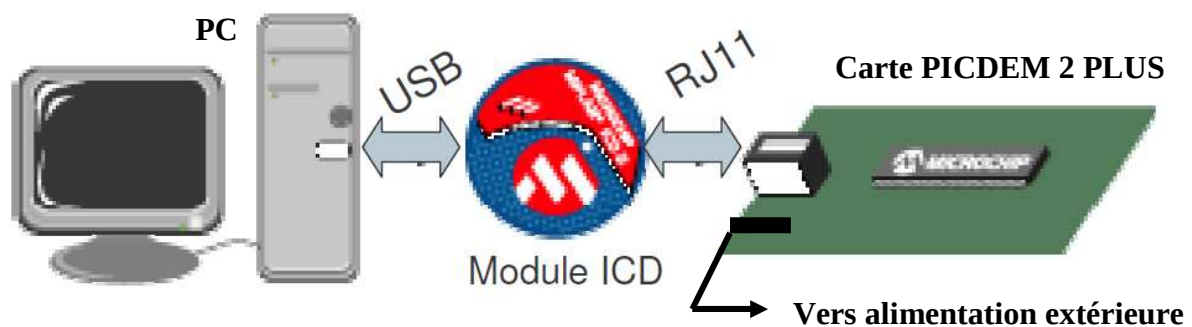


Figure VII.1.

De façon très générale :

- le **PC**, utilisé comme station de travail (ou **HOST**) possède tous les outils de développement nécessaires pour réaliser des programmes. Ici, comme dans la méthode précédente, on utilisera l'environnement logiciel fourni par **MPLAB**, mais au lieu de le configurer en mode simulation, on mettra en œuvre le mode « **débugage** » par **ICDx**,

- le boîtier **ICDx** est un outil puissant qui permet de s'interfacer, via une liaison **USB**, entre le **PC** et la carte applicative **PICDEM 2 PLUS**. Le programme sera alors directement mis au point sur la carte applicative mais fonctionnera sous contrôle total du boîtier **ICDx**. De fait c'est une version intermédiaire (de débogage) de votre programme qui sera implantée mais de façon transparente pour vous. Les possibilités en terme de mise au point de programmes seront les mêmes que dans la méthode de simulation (pas à pas, fenêtres watch...) mais ici, on n'est plus en simulation mais bien dans l'environnement réel et définitif de la carte applicative.

- la carte **PICDEM 2 PLUS** : on l'appelle alors la **carte cible (target board)** ou **carte mère**. Elle est reliée par liaison **RJ11** au boîtier **ICDx** et possède sa **propre alimentation** extérieure. Cette carte cible sert de support matériel où votre programme s'exécutera sous contrôle du boîtier **ICDx**.

Dans certains cas, on pourra en plus, brancher sur cette carte mère, une **carte fille (daughter board)** sur laquelle seront implantés d'autres types de périphériques. La liaison entre « la mère et la fille » se fera par des **nappes de conducteurs** adaptées aux connecteurs existants sur ces deux cartes.

Bref, on est passé **du monde virtuel au monde réel**.

Bref méthode idéale pour un débogage in situ !!!

VII.3. PROGRAMMATION

Cette méthode sera utilisée dans la **dernière** phase de développement de votre application.

L'EEPROM de code du PIC est programmée avec le code machine de votre programme qui en principe a atteint une version fiable et définitive suite à la méthode précédente.

De même, **l'EEPROM de données** sera programmée avec les données (semi)permanentes de votre programme.

A partir de cet instant, vous pouvez retirer la liaison **RJ11** entre la carte **PICDEM 2 PLUS** et le boîtier **ICDx** :

à chaque mise sous tension (ou à chaque reset) de la carte **PICDEM 2 PLUS**, c'est votre programme applicatif qui fonctionne.

Le **cordon ombilical** avec les outils de développement a été **coupé**.

Bref méthode idéale pour rendre votre application autonome !!!

VIII. LES PERIPHERIQUES ET INTERFACES DU PIC 16F877

Si nous rejetons un coup d'œil aux schémas des **figures II.3. et II.4.**, nous allons ici présenter dans ce dernier chapitre les blocs référencés ⑤ à ⑫

En fait c'est la partie la plus intéressante et la plus fondamentale, car ne l'oublions pas, un microcontrôleur est un composant prévu pour s'interfacer directement avec le monde extérieur. (récupérer des informations extérieures, les traiter et avoir une action sur cet extérieur).

Jusqu'à maintenant, à de rares exceptions près, nous étions restés à l'intérieur du microcontrôleur pour comprendre ses principes de fonctionnement et la façon de le programmer.

C'était une étape nécessaire mais loin d'être suffisante si on veut exploiter toutes les potentialités d'un microcontrôleur en ce qui concerne ses capacités d'interfaces.

Il est donc temps maintenant de voir comment ce composant peut **inter réagir** avec son environnement.

La plupart des exemples de programmes que nous donnerons seront en langage C.

VIII.1. LES ENTREES/SORTIES LOGIQUES

(cf. ⑤ du schéma de la **figure II.4.** et cf. **pages 41 à 52** du datasheet)

Ce sont les interfaces qui sont certainement **les plus souvent utilisées** et qui sont aussi peut-être **les plus simples à mettre en œuvre** (au moins sur les cas les plus basiques).

VIII.1.1. GENERALITES ET PRINCIPES

Comme on a pu le voir sur les **figures II.3. et II.4.** le **PIC 16F877** possède **un maximum de 33 lignes d'entrées/sorties logiques** (on dit aussi **entrées/sorties T_{out} (5 V) Ou R_{ien} (0 V)** ou **TOR** en abrégé) réparties sur 5 **ports** qu'on appelle :

PORTA, PORTB, PORTC, PORTD et PORTE

Remarquons tout d'abord que certaines lignes de ports **TOR** sont partagées avec d'autres fonctions par souci de diminution du nombre de broches du circuit intégré, et donc de son prix (revoir aussi le brochage du PIC de la **figure II.1.**) : auquel cas, si la **fonction correspondante est active**, la ligne de port (en tant qu'Entrée/Sortie logique) ne l'est plus.

Exemple:

Si on utilise le Convertisseur Analogique/Numérique via une des lignes du **PORTA**, cette ligne ne pourra plus, à cet instant là, fonctionner en entrée/sortie logique, mais fonctionnera en entrée analogique.

A chaque port est associé un **registre de PORT** de taille 8 bits portant le même nom que ce port.

Chaque registre de port fait partie des **SFR** du PIC et possède une adresse dans la cartographie de la RAM.

Je rappelle, sur la figure VIII.1. ci-dessous un extrait de la cartographie complète qu'on avait donnée sur la **figure III.3.** en isolant ici les registres correspondant à ces 5 ports.

PORTA	05h	TRISA	85h		105h	
PORTB	06h	TRISB	86h	PORTB	106h	TRISB
PORTC	07h	TRISC	87h		107h	
PORTD ⁽¹⁾	08h	TRISD ⁽¹⁾	88h		108h	
PORTE ⁽¹⁾	09h	TRISE ⁽¹⁾	89h		109h	
Bank0		Bank1		Bank2		Bank3

Figure VIII.1.

Par exemple pour le PORTB, le registre de port s'appellera **PORTB** représentable comme sur la figure VIII.2. ci-dessous :

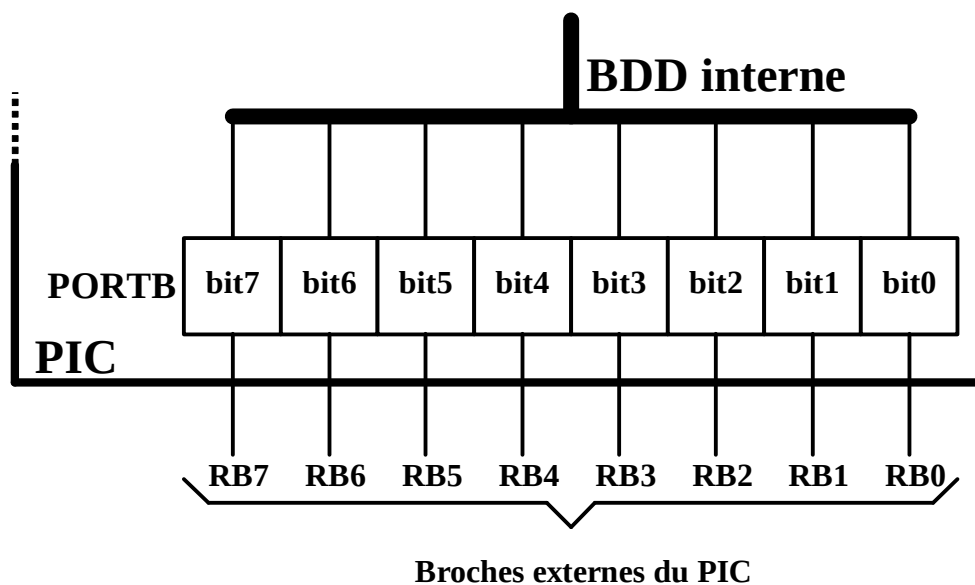


Figure VIII.2.

Ce registre de port se comporte un peu **comme une case mémoire** (reliée en interne au **Bus de Données** de la mémoire de **Données**) mais aussi reliée vers l'extérieur **bit à bit à huit broches** du PIC.

Par exemple (toujours sur la figure VIII.2.) pour le **PORTB** (de taille 8 bits), chacun des bits du registre de ce port est relié à une broche appelée **RBi**, pour $i=0$ (**LSB**) à 7 (**MSB**).

Chacune de ces lignes de port peut être **individuellement** programmée pour agir, à certains moments (mais bien sûr jamais les deux en même temps) :

- soit comme une **entrée : input**,
- soit comme une **sortie : output**.

On parle alors de **bidirectionnalité**.

Pour être encore plus précis, sur un port bidirectionnel, **à un instant donné, certaines lignes** peuvent être configurées **en sortie** et **d'autres en entrée** (paramétrage ligne à ligne et non globalement sur l'ensemble des 8 lignes du port).

Comment cela ?

Tout simplement parce qu'à chaque port **PORTX** est associé un **registre de direction** de port appelé **TRISX (X=A,B,...,E)**.

On peut d'ailleurs les voir sur l'extrait du mapping de la figure VIII.1. précédente.

Le **choix du sens des lignes de PORTX** (entrée ou sortie) se fait par l'intermédiaire de ce registre **TRISX**.

La correspondance et le choix sont faits **bit à bit** (cf. exemple de la figure VIII.3).

- chaque bit **TRISXi** de ce registre de direction **TRISX** est associé et agit sur la ligne **RXi** de même poids du port,
- **si ce bit TRISXi = 1**, la ligne correspondante **RXi** du port est **en entrée**,
- **si ce bit TRISXi = 0**, la ligne correspondante **RXi** du port est **en sortie**.

Moyen mnémotechnique :

TRISXi = 1 ⇒ ligne RXi en Input

TRISXi = 0 ⇒ ligne RXi en Output

Notons qu'au reset du PIC, toutes les lignes de tous les ports sont programmées en entrée pour des raisons de sécurité : éviter des écritures intempestives et non souhaitées vers des actionneurs extérieurs si ces lignes étaient en sortie !!!

Cette correspondance entre chaque bit des registres **PORTX** et **TRISX** est indiquée sur la figure VIII.3.

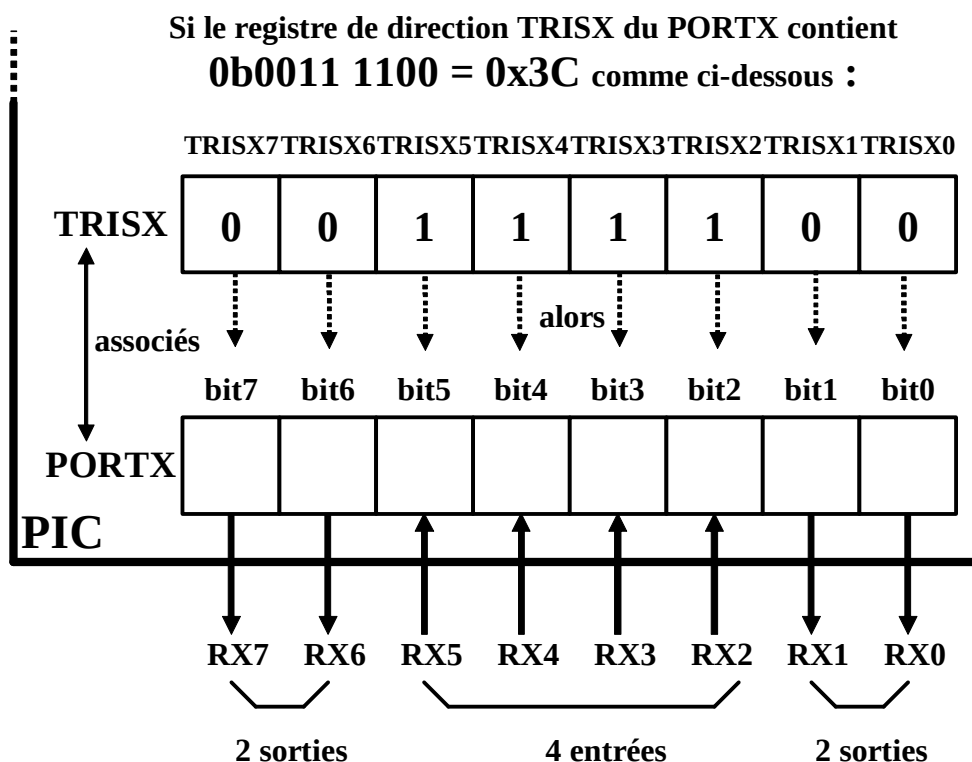


Figure VIII.3.

En fonction des affectations qu'il a choisies ou de celles imposées par le hardware, c'est le programmeur qui écrira la valeur adéquate dans le registre de direction **TRISX** pour piloter correctement chaque ligne du **PORTX**.

Ceci est fait en général dans une phase (écrite par exemple sous forme d'un module en assembleur ou d'une fonction en C) appelée **phase d'initialisation** du (ou des) port(s).

☛ ☛ ☛ **erreur fréquente** : on oublie d'initialiser les ports avant de les utiliser !!!

On voit bien que cette possibilité de bidirectionnalité est plus intéressante et offre plus de souplesse d'utilisation que si les ports étaient **monodirectionnels** (tout le temps figés en entrée ou tout le temps figés en sortie, comme cela existe sur d'autres types de microcontrôleurs).

Le fait également que chaque ligne d'un port soit programmable individuellement est plus souple que si c'était l'ensemble des 8 lignes du port qui était soit en entrée, soit en sortie.

La question pertinente que posent alors, et à juste titre, certains étudiants est la suivante :

« **Comment une même broche peut-elle fonctionner en entrée et en sortie ?** »

La réponse à cette question peut se donner en deux temps :

- tout d'abord, on le redit, une broche n'est jamais une entrée et une sortie au même instant,
- le **schéma de principe** de la figure VIII.4., qui sans rentrer dans le détail de tous les éléments et signaux de commande qui interviennent au niveau de l'interface avec une quelconque broche extérieure **RXi**, fournit néanmoins des éléments pour répondre à cette question :

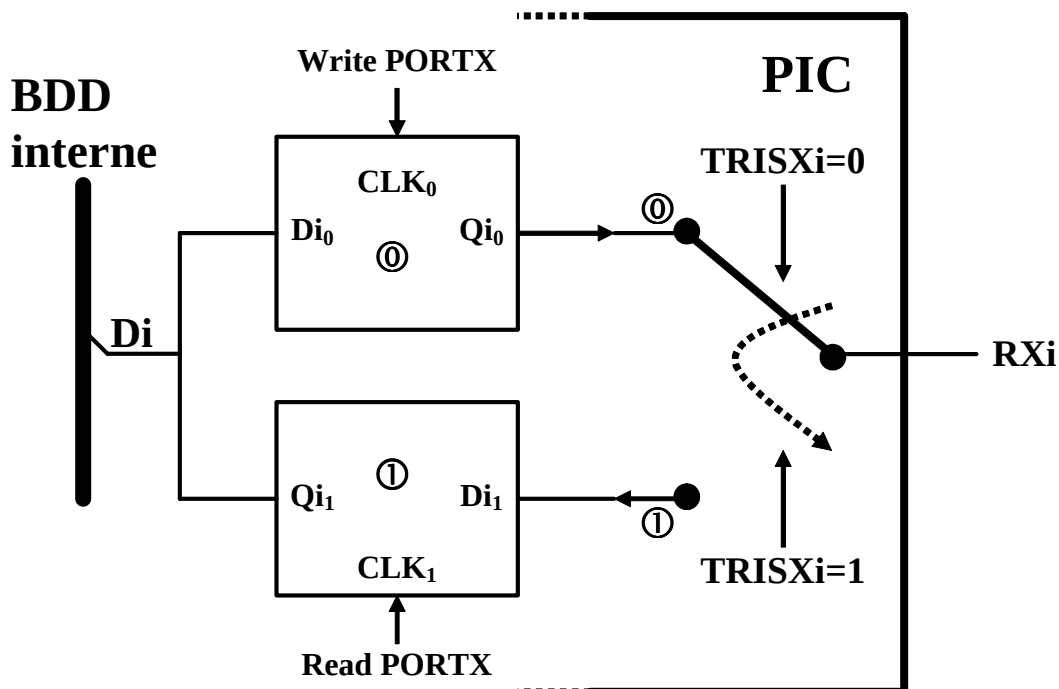


Figure VIII.4.

- lorsque le bit **TRISXi = 0**, le multiplexeur de sortie est en position ① : la broche **RXi** est une **sortie** reflétant la valeur qui aura été **écrite** sur la sortie Q_{i0} dans la bascule ① pendant la phase d'écriture du bit **Di** dans le **PORTX** (**write PORTX**),
- lorsque le bit **TRISXi = 1**, le multiplexeur de sortie est en position ② : la broche **RXi** est une **entrée** de la bascule ② dont on peut **lire** la sortie Q_{i1} pendant la phase de lecture du bit **Di** du **PORTX** (**read PORTX**). On dit que la broche de sortie **RXi** est alors en haute impédance dans ce cas car, quand le multiplexeur est en position ② la sortie Q_{i0} de la bascule ① est complètement déconnectée de la broche **RXi**.

Les schémas réels au niveau des broches diffèrent quelque peu de cette figure (on en reparlera plus loin) et ne sont pas les mêmes pour toutes les broches de tous les ports et si vous voulez plus de précision, il faut alors se reporter au datasheet du composant.

VIII.1.1.1. FONCTIONNEMENT EN SORTIE

L'écriture d'un octet dans le registre d'un port **PORTX** programmé en **sortie** se retrouve **bit à bit** sur les broches **RXi** correspondantes du PIC :

chaque bit à 1 donne 5V sur la broche correspondante et chaque bit à 0 donne 0V

Exemple, sur la figure VIII.5., où on a relié en plus une résistance R et une LED sur chaque ligne **RB_i** du **PORTB**.

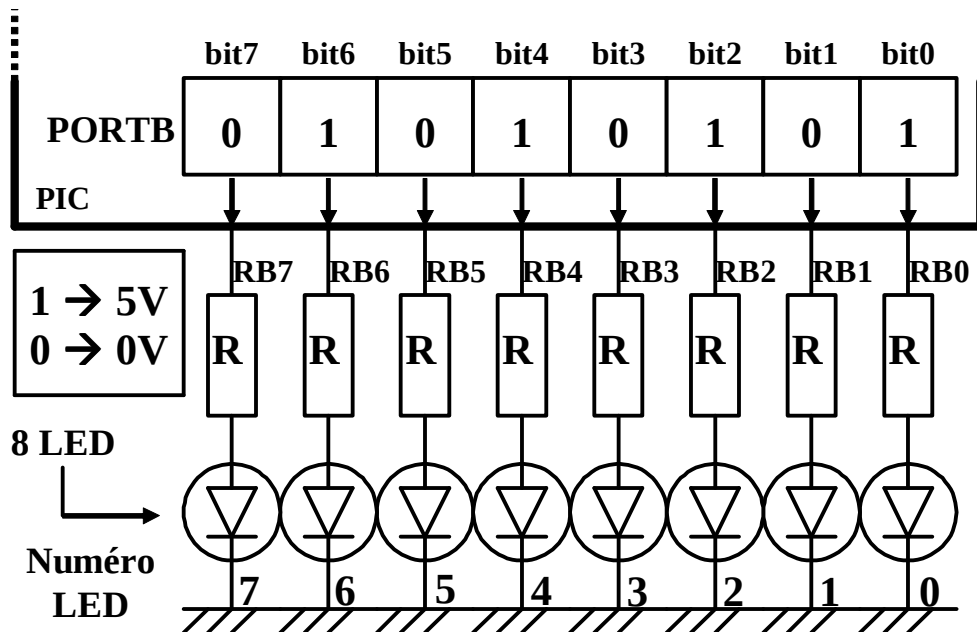


Figure VIII.5.

Quelles sont les leds allumées dans cette configuration ?

Le valeur de chaque résistance R se calcule comme suit, par une simple loi des mailles en sortie :

$V_{LED} + R.I = 5\text{ V}$ où V_{LED} représente la tension aux bornes d'une led allumée soit environ 1,5 V.

Le courant I dans une led allumée (dépendant du type de led) est pris ici pour le calcul à 10 mA, ce qui donne :

$$R = \frac{5 - V_{LED}}{I} \approx 350\ \Omega$$

En conséquence, une valeur couramment utilisée pour R est de 470 Ω .

La contrainte importante à respecter est de ne jamais dépasser un courant de sortie de 25 mA par broche, au risque d'endommager l'étage de sortie, comme le montre les deux lignes suivantes extraites du datasheet :

Maximum output current sunk by any I/O pin.....25 mA
Maximum output current sourced by any I/O pin25 mA

Deux remarques générales :

1. Les différentes adaptations :

Si on souhaite avoir un courant plus important, il faudra utiliser un étage intermédiaire à base de transistors petits signaux, de transistors de puissance ou de circuits intégrés amplificateur de courant. On parle d'adaptation en **courant**.

De même, un étage d'adaptation sera également nécessaire si on souhaite piloter des interfaces fonctionnant sous des tensions plus grandes que 5 V : on parle alors d'adaptation en **tension**.

Lorsque les deux types d'adaptation sont nécessaires, on parle d'adaptation en **puissance**.

2. Autre méthode pour piloter une led en sortie dite en « logique négative »

Cette méthode est donnée sur la figure VIII.6. avec la ligne **RB1** par exemple :

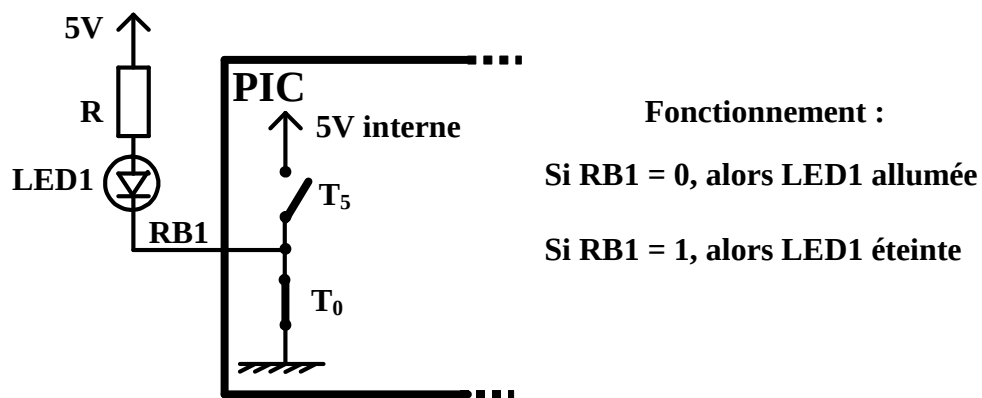


Figure VIII.6.

Lorsque **TRISB1=0**, les deux interrupteurs T_0 et T_5 sont en faits deux transistors commandés en opposition de phase et pouvant être soit dans l'état passant, soit dans l'état bloqué.

Ils forment ce qu'on appelle un étage « **totem pôle** ».

Mais on voit que la logique de commande pour piloter la led est ici l'inverse de celle présentée sur la figure VIII.5. précédente.

Quand **TRISB1=1**, T_0 et T_5 sont tous les deux ouverts, la broches **RB1** peut alors être utilisée comme une entrée (on dit que la sortie est en **haute impédance** ou « **High Z** »).

Le calcul de la résistance R est ici un peu différent, car dans la maille de sortie, il faut prendre en plus le niveau de tension à l'état bas de la broche de sortie qui est typiquement de l'ordre de 0,5 V mais au final cela ne changera pas la valeur de R .

Comment piloter avec le langage C un port parallèle en sortie ? C'est assez simple :

- **figure VIII.7. : cas où on veut agir en même temps sur les 8 bits d'un port** (cf. figure VIII.5.)

```
#include "pic.h"
#include "pic168xa.h"

#define bit(x) (1<<(x))    // fabrication des masques par décalage de 1 = 0b0000 0001
                           // de x bits sur la gauche
...
    TRISB = 0;              // les 8 bits du PORTB sont tous en sortie
                           // à mettre plutôt dans une fonction de prototype
                           // void init_port (void)
...

    PORTB = 0b0101 0101 ;   // pour obtenir l'état de la figure VIII.5.
// ou  PORTB = 0x55 ;       // idem mais moins lisible
// ou  PORTB = bit(6)+bit(4)+bit(2)+bit(0) // idem avec les masques bit(x)
...

    PORTB = PORTB | (bit(3)+bit(1)) // allume les leds 3 et 1
                                   // (sans modifier l'état des autres)
...

    PORTB = PORTB & ~(bit(3)+bit(1)) // éteint les leds 3 et 1
                                   // (sans modifier l'état des autres)
```

Figure VIII.7.

- **figure VIII.8. : cas où on veut agir sur une seule ligne du port** (cf. figure VIII.6.)

```
#include "pic.h"
#include "pic168xa.h"

#define bit(x) (1<<(x))    // fabrication des masques par décalage de 0b0000 0001
                           // de x bits sur la gauche
...
    TRISB = TRISB & ~bit(1); // on n'agit que sur le bit 1 de TRISB
                           // la broche RB1 est maintenant une sortie
                           // à mettre plutôt dans une fonction de prototype
                           // void init_port (void)
// ou  TRISB1 = 0           // idem en plus simple
...

    RB1 = 0;                // allume la led 1 : attention ici « logique négative »
                           // cf. figure VIII.6. (sans modifier l'état des autres)
...

    RB1 = 1;                // éteint la led 1 : attention ici « logique négative »
                           // cf. figure VIII.6. (sans modifier l'état des autres)
```

Figure VIII.8.

VIII.1.1.2. FONCTIONNEMENT EN ENTREE

La **lecture** de l'octet contenu dans un registre de port en **entrée** reflète **bit à bit** la valeur des tensions présentes sur les broches correspondantes du PIC :

5V sur la broche donne le bit correspondant à 1 et 0V sur la broche donne le bit à 0

Exemple, sur la figure VIII.9., où on a relié une résistance R (référéncée au 5V) et un **interrupteur** **Ki** sur chaque ligne **RCi** du **PORTC**.

On voit que, grâce aux résistances R de « **pull-up** » (**tirage vers le haut**, ici vers 5 V) :

- si un interrupteur **Ki** est **fermé (ON)**, la tension sur la broche **RCi** correspondante vaut **0V**,
- si un interrupteur **Ki** est **ouvert (OFF)**, la tension sur la broche **RCi** correspondante vaut **5V**.

Ainsi, en lisant le registre **PORTC**, on pourra connaître l'état individuel de chaque interrupteur.

- si le **bit i** de **PORTC** à **0** alors l'interrupteur **Ki** est sur **ON**,
- Si le **bit i** de **PORTC** à **1** alors l'interrupteur **Ki** est sur **OFF**.

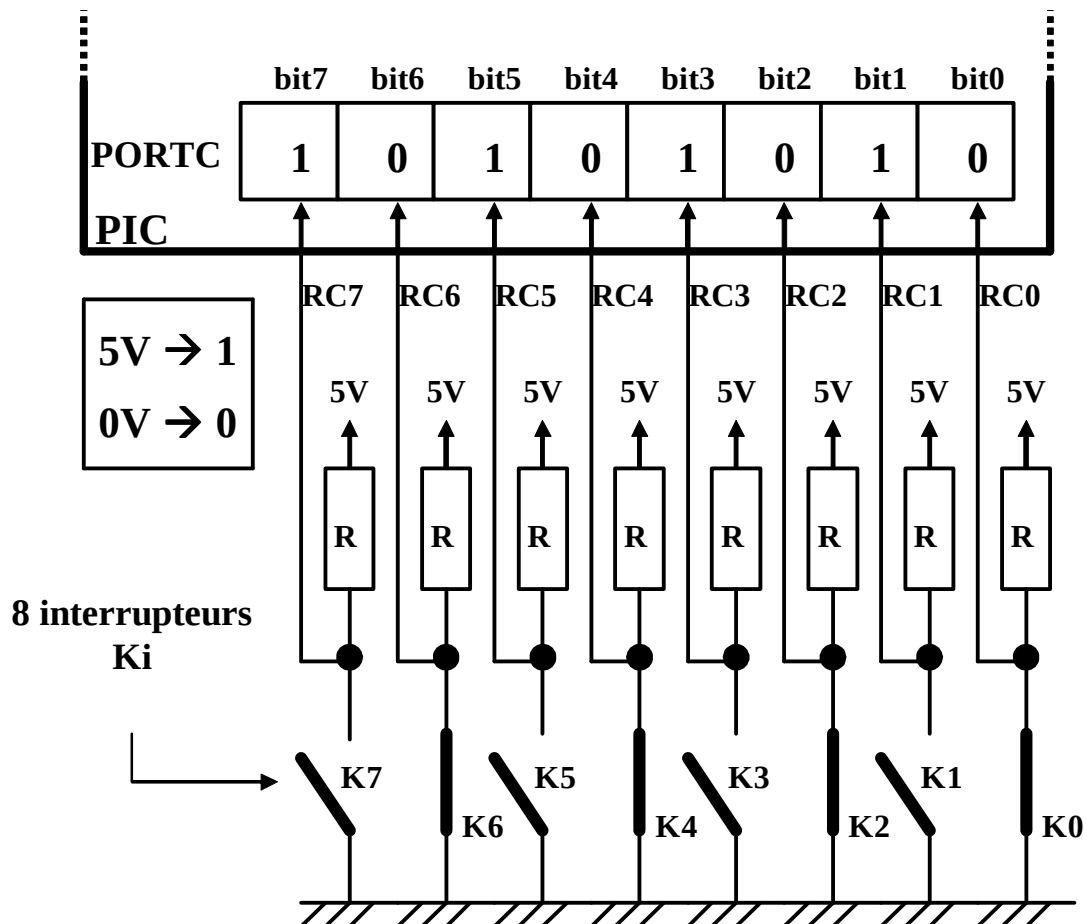


Figure VIII.9.

Les valeurs des résistances R sont de l'ordre de 10 k Ω .

On aurait pu aussi réaliser le schéma de la figure VIII.10.

Dans ce cas là, la résistance R est référencée par rapport à la masse et devient une résistance de « **pull-down** » (**tirage vers le bas**, ici vers 0 V).

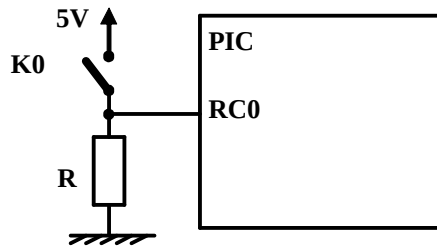


Figure VIII.10.

La logique est alors inversée par rapport à celle de la figure VIII.9. :

- si **K0 fermé (ON)** alors : **RC0 = 5V**,
- si **K0 ouvert (OFF)** alors : **RC0 = 0V**.

Dans ces deux exemples, on a utilisé des interrupteurs « **ON/OFF** » à deux états **permanents**.

Il existe aussi des **Boutons Poussoirs (BP)** qui ne maintiennent un de ces deux états (généralement l'état **ON**) que quand on appuie dessus (état appelé aussi **fugitif**).

Dans ces cas là, la problématique est de détecter cet appui par le logiciel.

Comment savoir qu'on a appuyé sur le Bouton Poussoir **BP** de la figure VIII.11 ?

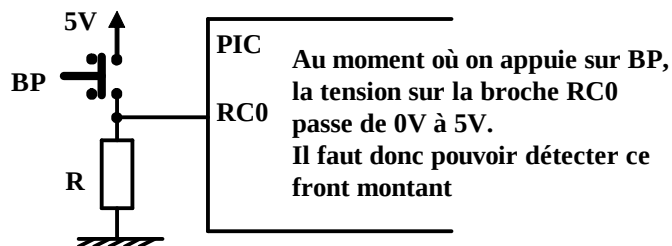


Figure VIII.11.

Les applications sont nombreuses :

- boutons de sélection (déplacement dans un menu, choix divers ...)
- boutons de réglage (volume +, volume -, fréquence +, fréquence - ...)
- boutons de validation (confirmer ?, mémorisation ? ...)
- claviers divers (matriciels, de téléphone, de terminal de paiement...) ...

Comment piloter avec le langage C un port parallèle en entrée ? C'est assez simple :

- **figure VIII.12. : cas où on lire en même temps les 8 bits d'un port** (cf. figure VIII.9.)

```
#include "pic.h"
#include "pic168xa.h"

...

char  etat_interrupteurs ;    // variable de taille un octet pour stocker l'état du PORTC

...

    TRISC = 0xFF;             // les 8 bits du PORTC sont tous en entrée
                                // à mettre plutôt dans une fonction de prototype
                                // void init_port (void)

...

    etat_interrupteurs = PORTC ;    // lecture du PORTC pour obtenir l'état
                                    // des interrupteurs de la figure VIII.9.
                                    // les bits à 0 de etat_interrupteurs indiquent les
                                    // interrupteurs fermés (ON)
                                    // les bits à 1 de etat_interrupteurs indiquent les
                                    // interrupteurs ouverts (OFF)
```

Figure VIII.12.

- **figure VIII.13. : cas où on veut agir sur une seule ligne du port** (cf. figure VIII.11.)

```
#include "pic.h"
#include "pic168xa.h"

#define bit(x) (1<<(x))      // fabrication des masques par décalage de 0b0000 0001
                                // de x bits sur la gauche

...

    TRISC = TRISC | bit(0);    // on n'agit que sur le bit 0 de TRISC
                                // la broche RC0 est maintenant une entrée
                                // à mettre plutôt dans une fonction de prototype
                                // void init_port (void)
// ou  TRISC0 = 1              // idem en plus simple
...

    while (!RC0);              // boucle while de scrutation de l'état du BP de la
                                // figure VIII.11. : on attend l'appui sur ce BP

...

                                // à partir de là, il y a eu un appui sur le BP
```

Figure VIII.13.

Tous ces points seront mis en pratique pendant les séances de TP.

☛ ☛ ☛ : de façon très générale, chaque fois qu'une même ligne **RXi** sera utilisée à certains moments en entrée et à d'autres moments en sortie, il ne faudra pas oublier d'initialiser correctement cette ligne via le bit **TRISXi** du registre de direction **TRISX** correspondant.

VIII.1.2. PARTICULARITES DES PORTS

Après ces généralités, faisons maintenant une analyse **un peu plus fine** des différents ports.

VIII.1.2.1. LE PORTA

Première remarque, il n'est pas complet :

seules 6 lignes sont disponibles : **RA0** à **RA5**.

① **les lignes RA0, RA1, RA2, RA3 et RA5** peuvent être aussi utilisées comme **entrées analogiques** du **CAN** (Convertisseur Analogique/Numérique) : voir chapitre sur le CAN.

D'ailleurs au reset, ces 5 broches sont par défaut affectées au CAN.

Si on veut les utiliser comme entrées/sorties logiques ou **TOR**, il faut modifier le **SFR ADCON1** qui permet justement de choisir les différentes affectations possibles pour ces lignes.

Le format de ce registre **ADCON1** est fourni sur la figure VIII.14.

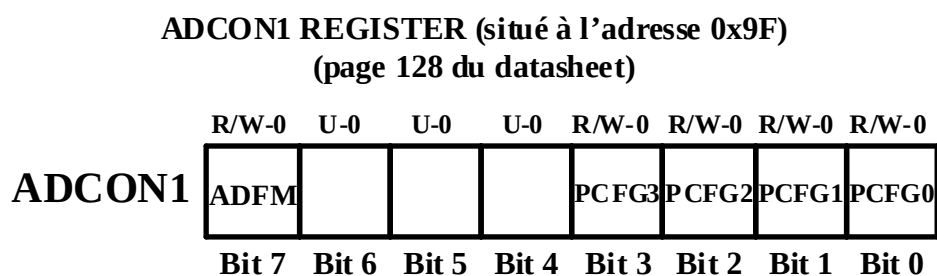


Figure VIII.14.

Remarque : le bit **ADFM** sera vu dans le chapitre sur le CAN.

Ce sont les 4 bits **PCFG3** à **PCFG0** qui permettent cette sélection conformément au tableau de la figure VIII.15. extrait du datasheet constructeur page **128** :

PCFG3: PCFG0	AN7 ⁽¹⁾ RE2	AN6 ⁽¹⁾ RE1	AN5 ⁽¹⁾ RE0	AN4 RA5	AN3 RA3	AN2 RA2	AN1 RA1	AN0 RA0	VREF+	VREF-	CHAN/ Refs ⁽²⁾
0000	A	A	A	A	A	A	A	A	VDD	VSS	8/0
0001	A	A	A	A	VREF+	A	A	A	RA3	VSS	7/1
0010	D	D	D	A	A	A	A	A	VDD	VSS	5/0
0011	D	D	D	A	VREF+	A	A	A	RA3	VSS	4/1
0100	D	D	D	D	A	D	A	A	VDD	VSS	3/0
0101	D	D	D	D	VREF+	D	A	A	RA3	VSS	2/1
011x	D	D	D	D	D	D	D	D	VDD	VSS	0/0
1000	A	A	A	A	VREF+	VREF-	A	A	RA3	RA2	6/2
1001	D	D	A	A	A	A	A	A	VDD	VSS	6/0
1010	D	D	A	A	VREF+	A	A	A	RA3	VSS	5/1
1011	D	D	A	A	VREF+	VREF-	A	A	RA3	RA2	4/2
1100	D	D	D	A	VREF+	VREF-	A	A	RA3	RA2	3/2
1101	D	D	D	D	VREF+	VREF-	A	A	RA3	RA2	2/2
1110	D	D	D	D	D	D	D	A	VDD	VSS	1/0
1111	D	D	D	D	VREF+	VREF-	D	A	RA3	RA2	1/2

A = Analog input D = Digital I/O

Figure VIII.15.

Remarque : nous reviendrons sur le tableau de la figure VIII.15. dans le chapitre sur le CAN.

Ainsi, pour utiliser ces 5 broches en entrées/sorties logiques, il faudra écrire les lignes suivantes, fournies sur la figure VIII.16.

	PORTA = 0x00 ;	// efface le PORTA
	ADCON1 = 0x06 ;	// RA0, RA1, RA2, RA3 et RA5 en E/S logiques
		// cf. tableau de la figure VIII.15.
	TRISA = 0x3F ;	// toutes ces broches + RA4 en entrée
// ou	TRISA = 0x00 ;	// toutes ces broches + RA4 en sortie

Figure VIII.16.

Attention : le fait d'utiliser les broches **RA0, RA1, RA2, RA3** ou **RA5** comme entrées analogiques **ne dispense pas** de les mettre en entrées via le registre de direction **TRISA**.

② **la broche RA4** peut quant à elle servir aussi comme l'entrée de comptage **T0CKI** du **timer 0** : voir chapitre sur les timers.

Remarque :

Si la broche **RA4** est utilisée comme sortie logique, il faut préciser que c'est une broche dite à « **drain ouvert** ». Qu'est ce que ça veut dire ?

Nous avons donné sur la figure VIII.6. le schéma type d'un étage de sortie d'une ligne de PORT que nous avons appelé « **totem pôle** ». Dans le cas d'un « **drain ouvert** », le transistor T_5 n'existe pas en interne. Le schéma de l'étage de sortie « drain ouvert » est donc celui de la figure VIII.17. (si le transistor T_0 est à technologie **bipolaire**, on parle alors plutôt de « **collecteur ouvert** »)

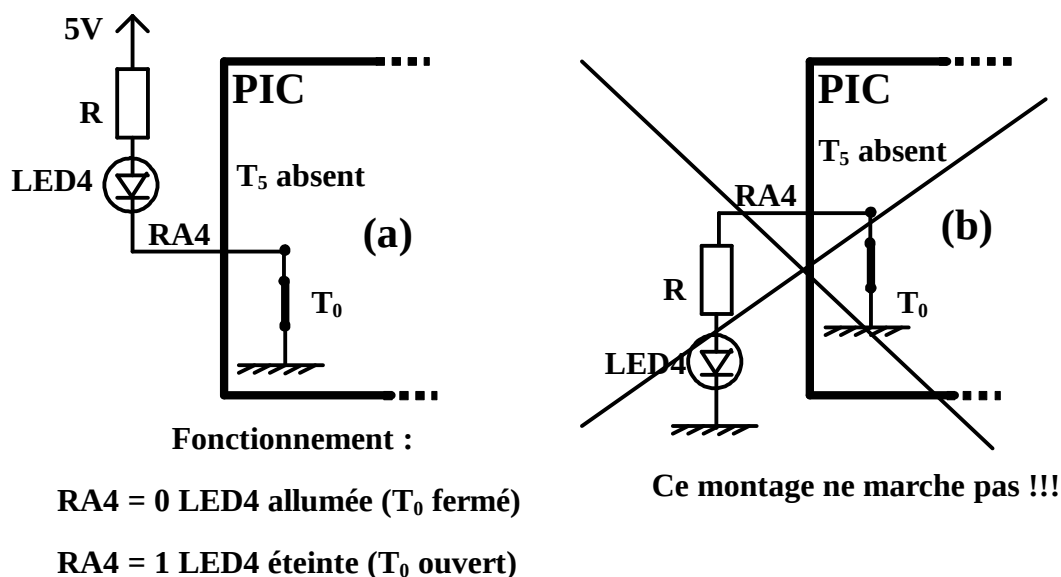


Figure VIII.17.

Remarque : si le montage de la figure VIII.17.a. est correct et permet de piloter la **led4**, celui de la figure VIII.17.b. ne fonctionne pas : on ne pourra jamais allumer la led4.

Il est intéressant, de donner à chaque fois, un résumé des registres concernés par chaque fonctionnalité interne, comme ce qui est fait dans le datasheet : cela permet, en seul coup d'œil de s'y retrouver plus facilement.

C'est ce que j'ai fait sur la figure VIII.18. pour le **PORTA**. Les parties grisées n'étant pas utilisées par la fonctionnalité concernée (ici le PORTA).

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
05h	PORTA	—	—	RA5	RA4	RA3	RA2	RA1	RA0	--0x 0000	--0u 0000
85h	TRISA	—	—	PORTA Data Direction Register						--11 1111	--11 1111
9Fh	ADCON1	ADFM	—	—	—	PCFG3	PCFG2	PCFG1	PCFG0	--0- 0000	--0- 0000

Legend: x = unknown, u = unchanged, - = unimplemented locations read as '0'.
Shaded cells are not used by PORTA.

Figure VIII.18.

VIII.1.2.2. LE PORTB

Ce port est complet (**RB0** à **RB7**).

Chaque ligne du PORTB possède une possibilité d'étage de sortie type « **totem pôle** » **débrayable** par soft.

Au reset, ou quand ces lignes sont en sortie, le transistor **T₅** de l'étage correspondant est systématiquement « enlevé ».

Par contre, on peut le « remettre » en service (pour toutes les lignes de PORTB) en plaçant le bit **RBPU** du registre **OPTION_REG** à **0**. (littéralement ligne **RB Pull Up**, actif à l'état bas).

Ce registre **OPTION_REG** est donné sur la figure VIII.19. ci-dessous :

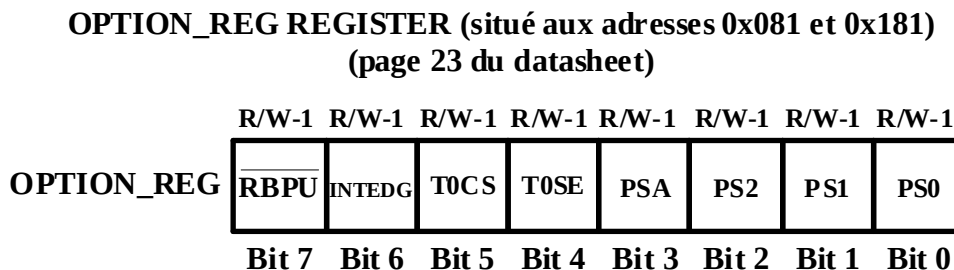


Figure VIII.19.

Remarque :

Les autres bits du registre **OPTION_REG** (**T0CS**, **T0SE**, **PSA**, **PS2**, **PS1** et **PS0**) sont utilisés pour le contrôle du **TIMER0** du PIC et seront donc vus plus tard, dans le chapitre consacré aux timers internes.

① les lignes **RB6** et **RB7** peuvent aussi être affectés au débogueur **ICD** et la ligne **RB3** au programmeur.

(cf. **chapitre III.5.** et les bits **DEBUG** et **LVP** du registre de configuration).

② les lignes **RB4** à **RB7** peuvent aussi être utilisées pour fonctionner **sous interruption en entrée** : tout **changement d'état** sur l'une quelconque de ces lignes programmée en entrée provoquera une interruption : cette possibilité sera avantageusement utilisée pour détecter l'appui sur une touche quelconque d'un clavier par exemple, pour sortir le PIC d'un mode de sommeil (**SLEEP**).

Des informations sur ce cas ② seront fournies dans le chapitre général sur les interruptions.

③ **et enfin, la ligne RB0**, programmée en entrée peut être utilisée pour générer une interruption, soit sur un front montant, soit sur un front descendant de **RB0**.

Pour choisir le type de front, c'est le bit **INTEDG (INTerrupt EDGe)** du registre **OPTION_REG** qui est utilisé (cf. figure VIII.19. précédente) :

- si **INTEDG = 1** : interruption sur front **montant** sur la broche **RB0**,
- si **INTEDG = 0** : interruption sur front **descendant** sur la broche **RB0**.

Des informations sur ce cas ③ seront fournies dans le chapitre suivant d'introduction sur les interruptions et également en TP grâce à une mise en pratique.

Au final, voilà sur la figure VIII.20. un résumé des registres concernés par le **PORTB**.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
06h, 106h	PORTB	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0	xxxx xxxx	uuuu uuuu
86h, 186h	TRISB	PORTB Data Direction Register								1111 1111	1111 1111
81h, 181h	OPTION_REG	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111

Legend: x = unknown, u = unchanged. Shaded cells are not used by PORTB.

Figure VIII.20.

VIII.1.2.3. LE PORTC

Ce port est aussi complet (**RC0 à RC7**).

C'est le port qui intervient dans le plus grand nombre de périphériques internes du PIC : timer1, modules CCP1 et CCP2, modules SPI et I²C et module USART. Plus de détails seront donc donnés dans les chapitres correspondants. Et donc, pour utiliser une ligne du PORTC comme entrée/sortie logique, il faudra s'assurer qu'elle n'est pas déjà affectée à l'un de ces différents modules.

La figure VIII.21. donne les registres concernés par le **PORTC**.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
07h	PORTC	RC7	RC6	RC5	RC4	RC3	RC2	RC1	RC0	xxxx xxxx	uuuu uuuu
87h	TRISC	PORTC Data Direction Register								1111 1111	1111 1111

Legend: x = unknown, u = unchanged

Figure VIII.21.

VIII.1.2.4. LE PORTD

Ce port est également complet (**RD0** à **RD7**).

La principale utilisation supplémentaire de **PORTD** est de fonctionner en mode port complet (sur 8 bits) assurant un protocole de **HANDSHAKING** (ou **poignée de main**).

Ce mode permet de s'interfacer en entrée ou en sortie avec des périphériques parallèles en général plus lents ou être directement relié à un bus de données 8 bits et être vu par ce bus comme une case mémoire supplémentaire (avec sélection par $\overline{CS}$, signal de lecture par $\overline{RD}$ et signal d'écriture par $\overline{WR}$) offrant ainsi un point d'accès et d'échange bidirectionnel entre le PIC et un autre micro/processeur/contrôleur (ouverture vers un système multiprocesseurs).

Pour choisir ce mode particulier (appelé aussi **Slave Port** dans le datasheet), il faut mettre à **1** le bit **PSPMODE** (comme **P**arallel **S**lave **P**ort **M**ODE) du registre **TRISE** (cf. figure VIII.22.).

Si ce bit **PSPMODE** est à **0**, le **PORTD** est alors un port **TOR** normal.

Comme ce fonctionnement en Slave Port est lié à certaines lignes du **PORTE**, nous en parlerons dans le chapitre suivant.

La figure VIII.22. résume les registres concernés par le **PORTD**.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
08h	PORTD	RD7	RD6	RD5	RD4	RD3	RD2	RD1	RD0	xxxx xxxx	uuuu uuuu
88h	TRISD	PORTD Data Direction Register								1111 1111	1111 1111
89h	TRISE	IBF	OBF	IBOV	PSPMODE	—	PORTE Data Direction Bits			0000 -111	0000 -111

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'. Shaded cells are not used by PORTD.

Figure VIII.22.

VIII.1.2.5. LE PORTE

Seules 3 lignes sont présentes (**RE0**, **RE1** et **RE2**).

Ces 3 lignes peuvent aussi être utilisées comme **entrées analogiques** du CAN. Elles le sont d'ailleurs après un reset du PIC.

Pour les utiliser comme E/S TOR, il faut également agir sur le registre **ADCON1** (comme on l'a déjà indiqué sur le tableau de la **figure VIII.15.** pour le **PORTA**) et penser également à mettre le bit **PSMODE** à **0** (cf. figure VIII.23.)

// ou	PORTE = 0x00 ;	// efface le PORTE
	ADCON1 = 0x06 ;	// RE0, RE1 et RE2 en E/S logiques
		// cf. tableau de la figure VIII.15.
	TRISE = 0x07 ;	// toutes ces broches en entrée et PSMODE=0 aussi
	TRISE = 0x00 ;	// toutes ces broches en sortie et PSMODE=0 aussi

Figure VIII.23.

Attention : le fait d'utiliser les broches **RE0**, **RE1** ou **RE2** comme entrées analogiques **ne dispense pas** de les mettre en entrées via le registre de direction **TRISE**.

La figure VIII.24. donne les registres concernés pour le **PORTE** pour ce type de fonctionnement.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
09h	PORTE	—	—	—	—	—	RE2	RE1	RE0	---- -xxx	---- -uuu
89h	TRISE	IBF	OBf	IBOV	PSPMODE	—	PORTE Data Direction Bits			0000 -111	0000 -111
9Fh	ADCON1	ADFM	—	—	—	PCFG3	PCFG2	PCFG1	PCFG0	--0- 0000	--0- 0000

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'. Shaded cells are not used by PORTE.

Figure VIII.24.

Par contre, comme on l'a déjà dit dans le chapitre précédent, si le bit **PSMODE** = **1**, alors le **PORTD** et le **PORTE** fonctionnent en mode **HANDSHAKING** (ou slave port).

- **PORTD** pour les échanges bidirectionnels de données sur 8 bits (et dans ce cas là, le registre de direction **TRISD** n'est plus pris en compte, car c'est le dispositif externe qui fixera le sens des échanges),
- **PORTE** pour les signaux de contrôle :
 - **RE0** comme entrée $\overline{RD}$,
 - **RE1** comme entrée $\overline{WR}$,
 - **RE2** comme entrée $\overline{CS}$.

Examinons le schéma fourni sur la figure VIII.25.

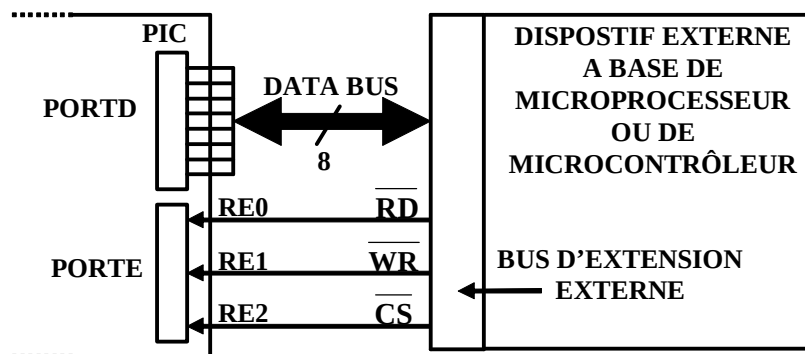


Figure VIII.25.

Envisageons le cas où le dispositif externe (relié au PIC par son bus d'extension sur la droite de la figure VIII.25.) veut envoyer (écrire) une donnée vers le **PORTD** du PIC.

C'est un **cycle d'écriture** qui sera initialisé par (cf. chronogramme de la figure VIII.26. directement issu du datasheet du PIC) :

- la donnée sur 8 bits est sortie sur le bus de données par le dispositif externe vers le **PORTD** du PIC qui fonctionne alors en entrée : instant t_0 ,
- le passage de l'entrée $\overline{CS}$ à **0** (signalant au PIC qu'on s'adresse à lui) : ce signal $\overline{CS}$ étant généré par le dispositif externe, cela signifie que le PIC aura une adresse dans le mapping de ce dispositif externe : instant t_1 , (ce signal $\overline{CS}$ restera d'ailleurs à **0** pendant toute la durée de ce cycle),
- le passage de l'entrée $\overline{WR}$ à **0** (signalant au PIC le début du cycle d'écriture) : instant t_2 , (notons que pendant tout ce cycle l'entrée $\overline{RD}$ reste à **1**)
- puis ce signal $\overline{WR}$ remonte à **1** (instant t_3) ce qui aura pour effet :
 - de faire passer à l'instant t_4 le flag **IBF** (comme **I**ntput **B**uffer **F**ull) à **1**, ce flag pouvant être utilisé (par la technique du pooling) par le PIC pour venir lire le **PORTD** : ce flag **IBF** sera automatiquement remis à **0** par une lecture du **PORTD** par le PIC,
 - et de faire passer aussi à l'instant t_4 le flag **PSPIF** (comme **P**arallel **S**lave **P**ort **I**nterrupt **F**lag) à **1**, ce flag pouvant générer quant à lui une interruption.

Notons, aussi que si un second cycle d'écriture est initié par le dispositif externe avant que le PIC n'ait eu le temps de lire la donnée précédente (donc avec **IBF** à **1**), le bit **IBOV** (comme **I**ntput **B**uffer **O**verflow) est mis à **1** pour signaler cet écrasement de données.

- puis ce cycle se termine par la remontée à **1** du signal $\overline{CS}$ à l'instant t_5 .

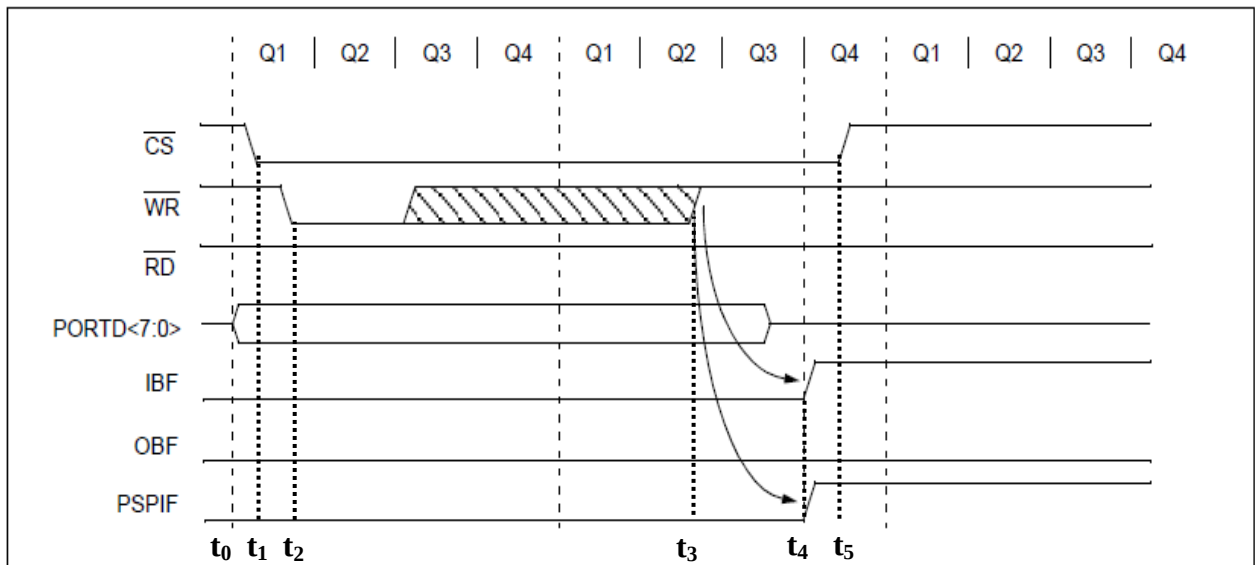


Figure VIII.26.

Je donne, sur la figure VIII.27., mais sans les commenter cette fois-ci, les chronogrammes pour un **cycle de lecture** d'une donnée par le dispositif externe, initiée cette fois-ci par le passage à **0** des signaux **CS** et **RD**.

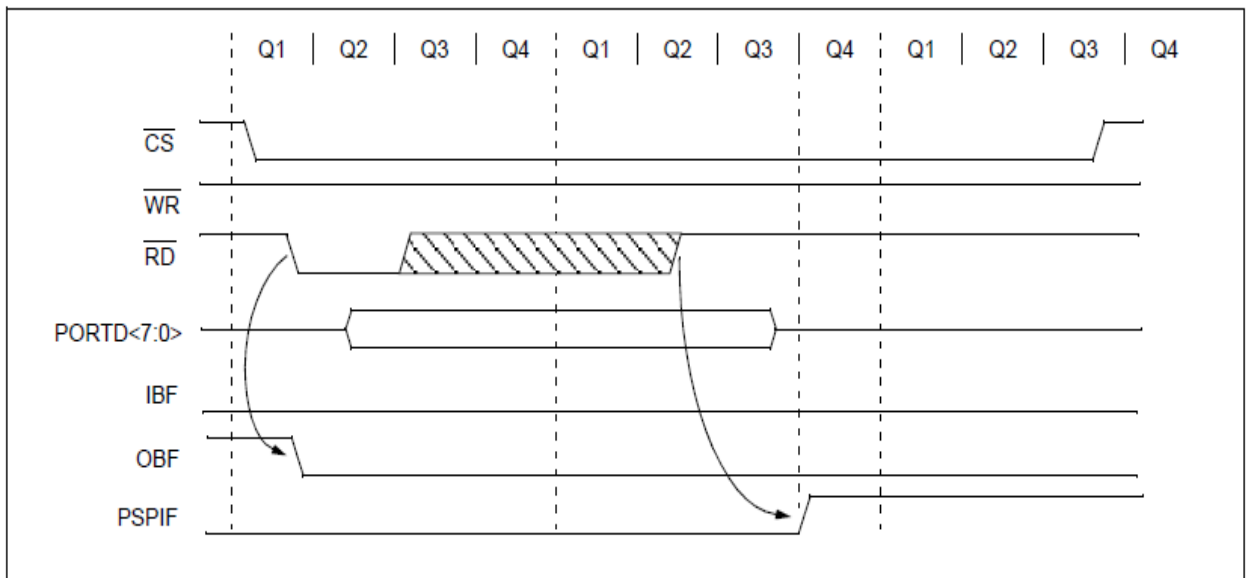


Figure VIII.27.

Le flag concerné pour cet échange est cette fois-ci **OBF** (comme **O**uput **B**uffer **F**ull).

La figure VIII.28. fournit quant à elle le résumé les différents registres concernés par les ports **PORTD** et **PORTE** en mode **Paralell Slave Port** et précise notamment la position dans le registre **TRISE** des différents flags liés aux chronogrammes ci-dessus : **IBF**, **OBF**, **IBOV** et **PSPIF**.

(le flag **PSPIE** (comme **Enable**) permet quant à lui de valider l'interruption dans le mode **Paralell Slave Port**)

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
08h	PORTD	Port Data Latch when written: Port pins when read								xxxx xxxx	uuuu uuuu
09h	PORTE	—	—	—	—	—	RE2	RE1	RE0	---- -xxx	---- -uuu
89h	TRISE	IBF	OBF	IBOV	PSPMODE	—	PORTE Data Direction Bits			0000 -111	0000 -111
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
9Fh	ADCON1	ADFM	—	—	—	PCFG3	PCFG2	PCFG1	PCFG0	--0- 0000	--0- 0000

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'. Shaded cells are not used by the Parallel Slave Port.

Note 1: Bits PSPIE and PSPIF are reserved on the PIC16F873/876; always maintain these bits clear.

Figure VIII.28.

VIII.2. PREMIERE APPROCHE DES INTERRUPTIONS MATERIELLES

(cf. ⑥ du schéma de la **figure II.4.** et cf. **pages 153 à 154** du datasheet)

VIII.2.1. INTRODUCTION GENERALE

C'est une des parties **les plus délicates à comprendre** mais peut-être aussi **à mettre en œuvre** dans le cadre d'une application.

Pourquoi ?

Tout simplement parce que travailler sous **interruption matérielle** (on dit aussi plus succinctement sous **IT**) est lié à la **notion de temps réel** et nécessite de faire un certain nombre de choses au niveau de la programmation.

Si, parmi toutes celles-ci, on en oublie une, cela ne fonctionnera pas et souvent le débogage sera assez délicat car le travail « classique » sous **ICD3** a ses limites en ce qui concerne la gestion du temps réel et des événements extérieurs.

Pourtant, les interruptions matérielles constituent un moyen **puissant, structurant et efficace** pour gérer et traiter des événements se déroulant dans l'environnement extérieur d'un microcontrôleur.

Il est donc important, pour bien les utiliser, de comprendre toute la « mécanique » sous-jacente à la gestion de ces interruptions.

C'est ce que nous ferons dans ce chapitre d'introduction en donnant des généralités valables pour la plupart des interruptions matérielles.

Les exemples concrets et les cas particuliers seront vus plus tard, au niveau de chaque périphérique interne.

Mais, si vous n'êtes pas familier avec les interruptions matérielles, je vous conseille **fortement** de commencer par la lecture du présent chapitre avant de passer aux applications concrètes.

Je sais que c'est en général souvent les cas réels et pratiques qui intéressent davantage les étudiants, mais pour bien les comprendre et les programmer correctement soi-même, il est obligatoire de savoir comment ils fonctionnent.

Remarque : sur certains micro/processeurs/contrôleurs, il existe aussi la notion d'interruption *logicielle* (utilisée en général pour des appels système) mais elle n'a pas été implantée dans les PIC.

VIII.2.2. DEFINITION ET EXEMPLES

Tout d'abord, qu'entend-on par **évènement extérieur** ?

C'est en général un évènement lié au hardware qui indique que quelque chose d'important a eu lieu dans l'environnement extérieur au PIC. On parle aussi de **la source ou de la cause** de l'IT.

Le PIC 16F877 est capable de gérer 14 sources d'IT, donc 14 types d'évènements extérieurs différents. Des exemples (liste non exhaustive) sont donnés ci-dessous. Dans chaque cas, il faut avoir en tête qu'un programme est cours d'exécution sur le PIC pendant que cet évènement extérieur se produit.

① **liés aux ports TOR** : une transition (front montant ou descendant) a lieu sur une ligne d'entrée d'un PORT : un appui sur un bouton poussoir ou une saisie sur un clavier, un objet passant devant un capteur de présence...

② **liés aux timers internes** : un compteur de temps (un registre du timer) est passé de 0xFFFF à 0x0000 (on parle alors de débordement ou d'overflow de ce registre)...

③ **liés à l'USART** : un caractère vient d'arriver sur une liaison série asynchrone...

④ **liés au convertisseur analogique/numérique** : une conversion vient de se terminer...

Ce qu'il faut bien comprendre, c'est que quelque soit cet évènement extérieur, il peut arriver n'importe quand, et donc de manière totalement **asynchrone** par rapport au programme qui s'exécute sur le PIC à cet instant.

Mais, dans tous les cas de figure, il nécessite un traitement qui devra être exécuté pour traiter cette interruption. Si on reprend les 4 exemples précédents :

① **effectuer l'action** correspondant à l'appui sur le Bouton Poussoir ou le clavier, incrémenter un compteur (d'objets) pour tenir à jour ce nombre d'objets...

② **incrémenter un compteur** (de temps) pour gérer correctement le temps qui s'écoule...

③ **lire le caractère** qui vient d'arriver sur la liaison série asynchrone (pour éviter que le caractère suivant ne vienne l'écraser avant que ce caractère ne soit lu)...

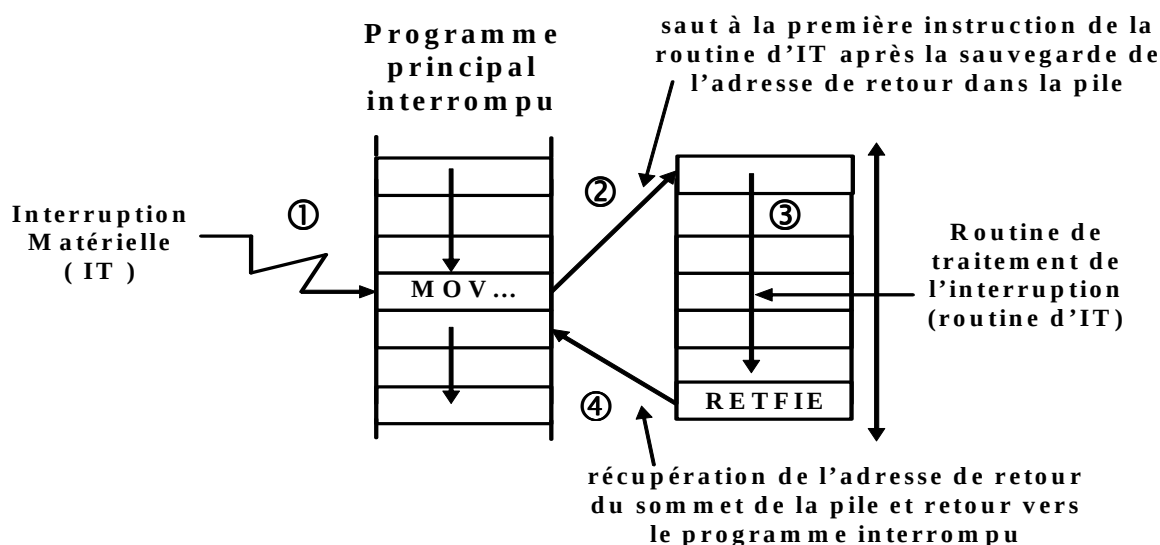
④ **lire la valeur numérique** fournie par le CAN pour pouvoir la traiter...

VIII.2.3. MECANISME D'UNE INTERRUPTION

Ce traitement qui doit être effectué à chaque interruption est appelé la **routine de traitement de l'interruption** (ou plus succinctement la routine d'IT).

Le programme en cours sur le PIC est donc « interrompu » par l'IT (①), donne la main à la routine d'IT (un peu comme pour un module ou une fonction : ②) qui s'exécute alors (③). Puis le programme interrompu reprendra son exécution à la fin de la routine d'IT (④).

Ce mécanisme est résumé sur le schéma de la figure VIII.29. obéissant bien au principe du « ticket aller-retour » qu'on a déjà présenté :



Principe du traitement d'une interruption matérielle

Figure VIII.29.

Mais attention, ce n'est pas une instruction du programme qui appelle l'interruption, comme pour une fonction en C ou un module en assembleur, mais la routine d'IT sera appelée « **automatiquement** » quand l'IT se produira.

Lorsque que j'entends des étudiants dire : « quand **j'** appelle la routine d'IT ... », c'est qu'ils n'en n'ont pas bien compris le mécanisme. Une routine d'interruption **ne s'appelle pas au moyen d'une instruction CALL existant explicitement dans le programme** (c'est-à-dire comme pour une fonction ou un module), mais l'appel est fait quand l'interruption a lieu (et c'est en ce sens que les interruptions sont **asynchrones** par rapport au programme ...).

En fait, si on y regarde bien, la seule ressemblance entre une routine d'IT et un module (ou une fonction) est que, comme eux, une routine d'IT a :

- un **nom** : qu'on supposera dans cette présentation générale être **traitement_it**,
- un **début** : première instruction de la routine d'IT,
- et une **fin** : avec un retour ensuite au programme interrompu.

Et c'est tout. Tout le reste n'est que différences. Mais allons un peu plus loin...

VIII.2.4. SPECIFICITES D'UNE ROUTINE INTERRUPTION

Notons surtout les neuf différences notables (points ① à ⑨) entre une routine d'IT et un(e) « classique » module (fonction) et mettons en évidence les spécificités du traitement sous IT :

① on le repète, les interruptions peuvent avoir lieu **n'importe quand**, de façon totalement **asynchrone** par rapport au programme et donc à l'instruction qui est exécutée par l'unité centrale au moment de cette interruption (comme l'instruction MOV... sur l'exemple de la figure VIII.29. mais ce pourrait être n'importe quelle autre instruction).

Cette instruction **se termine** avant de donner automatiquement la main à la **routine d'IT**.

En d'autre terme, **il n'y aura pas** dans votre programme une instruction comme :

~~CALL~~ traitement_it ;

② contrairement à un module ou à une fonction, une routine d'IT **ne peut pas avoir de paramètres en entrée** (impossible de les lui passer) **ni ne peut renvoyer une valeur de retour au programme interrompu** (qui n'a aucun moyen de pouvoir la traiter).

En effet, au moment de la compilation, il n'y a aucun moyen de savoir quand l'interruption aura lieu, et donc il n'est pas possible pour le compilateur d'écrire le code pour passer des paramètres ou pour récupérer une valeur de retour.

Par contre, une routine d'IT peut parfaitement modifier une variable globale (qui pourra être testée ensuite par le main).

③ une autre donnée est importante : à certains moments « stratégiques », le programme peut décider **de masquer toutes les interruptions** (un peu comme s'il devenait insensible à ce qui se passe à l'extérieur) pour ne pas être « dérangé » (car ce qu'il fait alors est pour lui « plus » primordial).

Il existe pour cela un bit particulier, le **bit 7** du registre **SFR INTCON** (comme **INT**errupt **CON**trol register) appelé le bit **GIE** (comme **G**lobal **I**nterrupt **E**nable) :

lorsque ce bit **GIE = 0**, **toutes les interruptions sont masquées**.

(c'est d'ailleurs le cas au démarrage ou après un reset du PIC).

Attention : ce bit est bien un masque **global**, valable pour masquer **toutes** les interruptions.

Conséquence : au moment où on souhaite travailler sous IT, il faudra mettre ce bit **GIE** à 1.

Il faut également noter que ce bit **GIE** est **automatiquement mis à 0** au moment du saut vers la routine d'interruption **traitement_it**, empêchant la prise en compte d'une nouvelle IT pendant le traitement de l'IT courante (la routine d'IT n'est donc, par défaut, **pas interruptible**).

Ce qui interdit, par défaut, l'imbrication des interruptions (le programmeur **peut toujours remettre à 1** le bit **GIE** dans sa routine de traitement de l'interruption, mais cela est réservé aux programmeurs les plus chevronnés !!!).

(par contre, rien n'interdit d'appeler une fonction (un module) dans la routine d'IT).

Au moment du retour vers le programme interrompu, **GIE** sera **automatiquement remis à 1**.

C'est d'ailleurs pour cette raison que la routine d'IT se termine (cf. figure VIII.29.) par l'instruction assembleur **RETFIE** (qui récupère l'adresse de retour du sommet de la pile **et** remet le bit **GIE** à 1) au lieu de l'instruction **RETURN** que l'on trouve à la fin d'un module ou d'une fonction (qui ne fait **que** récupérer l'adresse de retour du sommet de la pile).

④ mais de **façon plus « fine »** que par le bit **GIE**, chaque interruption peut aussi être **individuellement** masquée : on parle alors de masquage **local**.

Si on décide, dans cette présentation générale, d'affecter un numéro (par exemple un entier **n**) à chaque source d'IT (pour les différencier entre elles), cela signifie qu'il existe un bit particulier, (dans un des registres **SFR** du PIC), bit qu'on appellera ici **PERn_IE** (comme **PER**iphéric N°**n** **I**nterrupt **E**nable) permettant d'autoriser ou de masquer cette interruption N° **n** :

- lorsque ce bit **PERn_IE** = **1**, le fonctionnement sous IT du périphérique N°**n** est possible,
- lorsque ce bit **PERn_IE** = **0**, l'interruption du périphérique N°**n** est masquée.

Exemples pour les quatre cas d'interruption vues précédemment :

- bit **INTE** : pour valider l'interruption sur front (montant ou descendant) sur la ligne d'entrée **RB0** du **PORTB** (bit **4** du registre SFR **INTCON**),
- bit **TOIE** : pour valider l'interruption sur débordement du **timer0** (bit **5** du registre SFR **INTCON**),
- bit **RCIE** : pour valider l'interruption sur réception d'un caractère par **l'USART** (bit **5** du registre SFR **PIE1**, comme **P**eripheral **I**nterrupt **E**nable **1** register),
- bit **ADIE** : pour valider l'interruption sur conversion terminée du **CAN** (bit **6** du registre SFR **PIE1**).

⑤ la survenue de l'évènement extérieur **N°n** (associé au périphérique **N°n**) positionne un flag particulier (dans un des registres SFR du PIC). Nous appellerons ici ce flag **PERn_IF** (comme **PER**iphéric **N°n** **I**nterrupt **F**lag)

Ainsi, lorsque ce flag **PERn_IF** passe à 1 alors que le flag associé **PERn_IE** est aussi à 1, sans oublier le flag **GIE** qui doit être aussi à 1, une interruption est générée.

Exemples pour les quatre cas d'interruption vues précédemment :

- bit **INTF** : pour signaler l'apparition du front (montant ou descendant) sur la ligne d'entrée **RB0** du **PORTB** (bit 1 du registre SFR **INTCON**),
- bit **TOIF** : pour signaler le débordement du **timer0** (bit 2 du registre SFR **INTCON**),
- bit **RCIF** : pour signaler la réception d'un caractère par **l'USART** (bit 5 du registre SFR **PIR1**, comme **P**eriphérial **I**nterrupt **R**egister **1**). Il aurait mieux valu l'appeler d'ailleurs **PIF1** !!!
- bit **ADIF** : pour indiquer la fin de la conversion du **CAN** (bit 6 du registre SFR **PIR1**).

Finalement, le schéma de la figure VIII.30. représente le schéma interne qui permet de générer l'IT (sortie **InTerrupt**) :

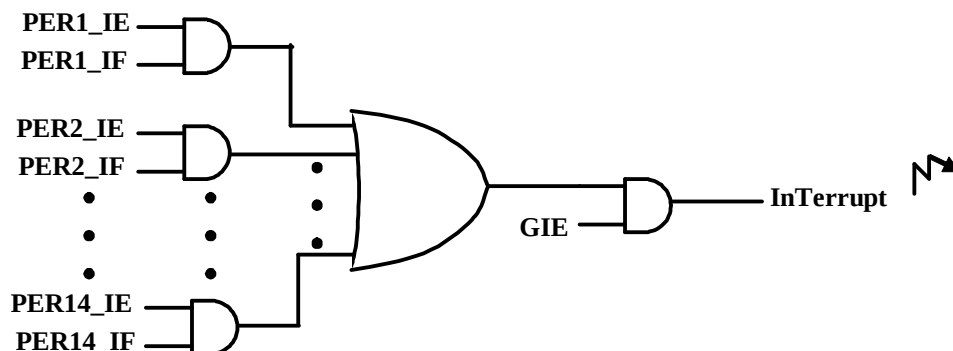


Figure VIII.30.

On y voit bien que pour que une IT soit générée (sortie **InTerrupt** passant à 1), et donc que la routine **traitement_it** soit exécutée, il faut que :

- **GIE = 1** : toutes les interruptions (**globalement**) **démasquées**,

et

- **au moins** un couple :

{	PERn_IE = 1 : interruption N°n (localement) démasquée , et PERn_IF = 1 : l'évènement N°n s'est produit .
---	---

En réalité, le schéma **réel** fourni à la page 129 du datasheet est celui de la figure VIII.31. Il diffère quelque peu de celui de la figure VIII.30. notamment avec le bit **PEIE** (comme **PE**ripheral **I**nterrupt **E**nable) qui est une sorte de masque **semi-global** pour autoriser toutes les interruptions des périphériques internes.

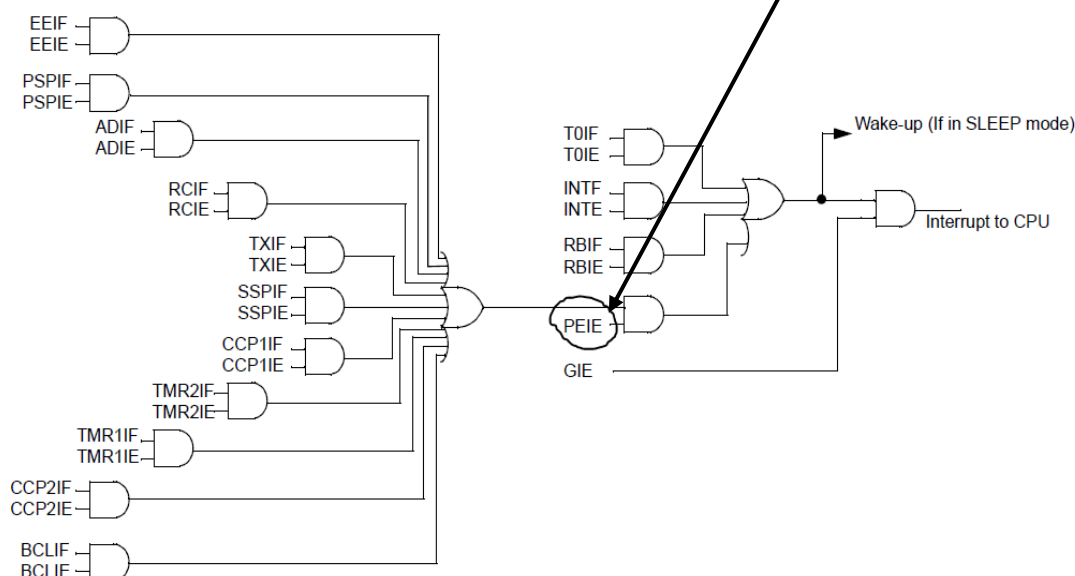


Figure VIII.31.

Mais si vous avez bien saisi le principe de la figure VIII.30. vous progressez à coup sûr dans la compréhension du mécanisme des interruptions et de tout ce qui les sous-tend.

⑥ on remarque aussi, en examinant les deux schémas précédents ; que quelque soit l'évènement extérieur, il n'y a qu'une seule routine d'IT qui sera exécutée.

Remarque : sur certains micro/processeurs/contrôleurs (famille 68xxx de chez Motorola, famille 86 et Pentium de chez Intel...), chaque cause d'IT peut provoquer **sa propre routine** de traitement d'interruption : on dit dans ce cas que les interruptions matérielles sont **vectorisées**. Mais pour le PIC, pas de vectorisation individuelle hardware.

En conséquence, pour un PIC, quelque soit l'évènement extérieur, ce sera la routine **traitement_it commune** à toutes les IT qui sera exécutée. Mais où se trouve en mémoire cette routine de traitement de l'interruption ?

Ceci est imposé par le constructeur : à l'**adresse absolue 0x0004** de l'EEPROM de programme. (contrairement aux modules et aux fonctions qui peuvent être placées n'importe où en mémoire de programme, emplacements fixés soit par la directive **ORG**, soit par le couple compilateur/linker).

Ainsi, à chaque IT, le PC sera chargé automatiquement par l'adresse **0x0004**. (ce qui permettra au PIC d'exécuter la première instruction de la routine d'IT, puis la suivante ... jusqu'à l'instruction **RETFIE** finale).

Dans le cas de la programmation en C, on indiquera le type **interrupt** devant la déclaration de la routine d'IT, pour que le compilateur/linker place cette routine à l'adresse **0x0004** et la termine comme on l'a vu par l'instruction **RETFIE** (au lieu de RETURN) (et voir aussi le point ⑨).

Le dernier point à aborder est le suivant : que faut-il mettre dans cette routine d'IT ?

La réponse à cette question nous permettra de mettre en évidence les trois dernières spécificités de la routine d'IT.

⑦ puisqu'il n'y a qu'une seule routine d'IT commune à toutes les causes d'interruption, il faudra dans cette routine trouver la cause de l'interruption pour effectuer le traitement adéquat.

Pour cela, il faut construire une espèce de « **daisy chain software** » qui va venir balayer toutes les causes possibles d'interruption et qui va détecter celle (ou celles) qui a (ont) déclenché l'interruption.

Par exemple, comme sur la figure VIII.32., toujours dans le cas des quatre exemples vus précédemment, on écrira :

```
    if (INTF && INTE)      // interruption due à un front actif sur RB0?
    {
// mettre ici le traitement à faire lors de l'appui sur le BP (ou la gestion du compteur d'objets)
    }

    if (TOIF && TOIE)      // interruption due au débordement du timer0?
    {
// mettre ici le traitement à faire pour gérer le temps qui s'écoule
    }

    if (RCIF && RCIE)      // interruption due à l'arrivée d'un caractère sur l'USART?
    {
// mettre ici le traitement à faire à chaque réception de caractère
    }

    if (ADIF && ADIE)      // interruption due au CAN?
    {
// mettre ici le traitement à faire à chaque conversion
    }
```

Figure VIII.32.

On remarque ainsi que l'ordre dans lequel ces tests seront faits donnera un niveau de priorité à chaque interruption : en effet, si par exemple on a **en même temps** (par **simultanéité** des événements extérieurs) un débordement du **TIMER0** **et** l'arrivée d'un caractère sur la liaison série, la gestion du timer sera servie (traitée) avant celle de la liaison série.

C'est donc au programmeur de déterminer quel niveau de priorité il veut affecter à chaque source d'interruption, en choisissant l'ordre avec lequel il déterminera la cause ayant provoqué l'IT.

Remarque : sur certains autres type de micro/processeurs/contrôleurs, et notamment lorsque les interruptions sont vectorisées, un niveau de priorité est affectée à chaque source d'interruption, niveau fixé une fois pour toute par le hardware ou modifiable par programmation.

⑧ voilà un point qui est souvent oublié dans la routine d'IT : **l'acquittement** de l'interruption.

Qu'est ce que c'est ?

Imaginez un évènement extérieur N°n ayant eu lieu et donc ayant positionné le flag **PERn_IF**.

Si les bits **GIE** et **PERn_IE** sont à **1**, la routine d'IT est exécutée.

Si aucun acquittement n'est fait dans cette routine d'IT, à son retour vers le programme interrompu, le flag **PERn_IF** sera toujours à 1, déclenchant tout de suite une autre IT ... et ainsi de suite. Le système est en état de permanente et répétitive interruption, il est « planté ».

Acquitter une interruption consiste donc, dans la routine d'IT, à **effacer la cause de cette interruption**.

En pratique, cela revient à **remettre à 0** le bit **PERn_IF**.

Il y a en fait deux possibilités :

- certains flags **PERi_IF** (à cause de cela, on les appelle les « **software** » flags) doivent être **explicitement** effacés par une instruction de la routine d'IT du type :

PERi_IF = 0 ;

- d'autres flags **PERj_IF** (à cause de cela, on les appelle les « **hardware** » flags) sont effacés **implicitement** par l'action liée au traitement de l'IT. Par exemple, dans le cas d'une interruption due à la réception d'un caractère par **PUSART** sur la liaison série asynchrone (**RCIF=1**), **la lecture** de ce caractère dans la routine d'IT remet **automatiquement ce flag RCIF à 0**.

Dans chaque cas, il faudra donc se référer à la documentation pour savoir comment acquitter les interruptions.

⑨ le dernier point est peut-être le plus délicat à comprendre et aussi à mettre en œuvre, surtout quand on programme en assembleur.

Il est lié à la notion de **contexte** et s'appelle la **sauvegarde et la restitution du contexte**.

Qu'est ce que c'est ?

Lorsqu'un programme est interrompu, supposons qu'au moment de l'interruption le registre **W** contienne la valeur 0x03 (valeur arbitrairement choisie pour l'exemple) utilisée pour un calcul intermédiaire. Dans la routine d'IT, qui prend la main, le registre **W** pourra être modifié (et le sera certainement). Supposons qu'au retour de la routine d'IT, **W** contienne la valeur 0x05 (valeur là aussi purement arbitraire). Le calcul dans le programme interrompu sera continué mais avec une valeur dans **W** qui aura changé et donc il sera faux. Voilà un exemple de ce qu'on appelle le **contexte**.

C'est l'ensemble des informations qui sont nécessaires pour le fonctionnement normal d'un programme et que la routine d'IT ne doit pas modifier.

Conclusion : il faut :

- les sauver au début de la routine d'IT (phase appelée **sauvegarde du contexte**),

et

- les restituer tels quels avant le retour vers le programme interrompu (phase appelée **restitution du contexte**).

Essayons de lister ces informations, pour en faire un bilan :

- le registre **W** : comme on vient de le voir.
- le registre **STATUS** : qui contient les flags positionnés par certaines instructions. Si une IT survient alors que ces flags ont été positionnés mais pas encore testés, la routine d'IT peut les modifier (et le fera certainement). Et au retour, ils ne seront plus intègres (significatifs) pour la suite du test dans le programme interrompu. Le registre **STATUS** fait donc bien partie du contexte.
- le registre **FSR** : en cas d'utilisation de l'adressage indirect, ce registre doit être préservé. Il fait donc aussi partie du contexte.
- le registre **PCLATH** : on le rappelle, il contient les 5 bits de poids forts du PC (de taille 13 bits). En effet, pour passer à la routine d'IT, le PC va être modifié avec la valeur **0x0004** : adresse de la première instruction de la routine d'IT et « ticket aller ». Pour pouvoir revenir au programme interrompu (« ticket retour »), la valeur **PCLATH** doit être récupérée intègre. Elle fait donc aussi partie du contexte.
La question que pose alors les étudiants est la suivante : pourquoi **PCL** (les 8 bits de poids faible du PC) ne fait-elle pas aussi partie du contexte ? Elle le fait bien sûr, mais on n'a pas à le gérer car elle est automatiquement mis dans la pile (par le hardware) au moment de l'interruption et tout aussi automatiquement restituée du sommet de la pile au moment de l'instruction **RETFIE**.

Remarque : sur certains micro/processeurs/contrôleurs, la sauvegarde et la restitution de tous les registres concernés par le contexte est fait **automatiquement** dans la pile au moment de l'interruption. Pour le PIC, ce n'est pas le cas.

En langage C, ces phases de sauvegarde et de restitution du contexte sont automatiquement (et de façon transparente pour le programmeur) rajoutées par le compilateur respectivement au début et à la fin de la routine d'IT (grâce au type **interrupt** déclaré devant la déclaration de la routine d'IT).

En langage assembleur, il faudra écrire, par exemple :

- pour la phase de sauvegarde du contexte (au début de la routine d'IT) :

```
ORG      0x0004      ; adresse de la routine d'IT
MOVWF    0x7F         ; sauve W en dernière adresse 0x7F zone RAM commune
SWAPF    STATUS,W     ; swappe STATUS dans W (pas de modification des flags)
```

; **attention** : une instruction comme :
; **MOVF** STATUS,W peut modifier les flags et donc **STATUS** ce qui ne le
; laisserait pas intègre !!!

```
MOVWF    0x73         ; sauve STATUS swappé à l'adresse 0x73 zone commune
MOVF     FSR,W        ; FSR dans W
MOVWF    0x74         ; sauve FSR à l'adresse 0x74 zone commune
MOVF     PCLATH,W     ; PCLATH dans W
MOVWF    0x75         ; sauve PCLATH à l'adresse 0x75 zone commune
```

- pour la phase de restitution du contexte (à la fin de la routine d'IT) :

```
MOVF     0x75,W       ; récupère PCLATH du programme interrompu dans W
MOVWF    PCLATH       ; et le remet dans PCLATH
MOVF     0x74,W       ; récupère FSR du programme interrompu dans W
MOVWF    FSR          ; et le remet dans FSR
SWAPF    0x73,W       ; « déswappe » (pas de modification des flags) et recupère
                     ; STATUS du programme interrompu dans W
MOVWF    STATUS       ; et le remet dans STATUS
SWAPF    0x7F         ; permute l'ancien W (quartet fort/quartet faible) dans 0x7F
SWAPF    0x7F,W       ; encore une fois avec résultat dans W
RETFIE                   ; retour vers le programme interrompu avec contexte restauré
```

Ces deux instructions **SWAPF** à la fin sont un peu surprenantes. Dans la lignée des précédentes, et pour récupérer **W** qui avait été sauvegardé à l'adresse 0x7F, on pouvait s'attendre plutôt à :

```
MOVF     0x7F,W       // récupère W du programme interrompu dans W
```

Mais c'était oublié que cette instruction modifie le flag **Z** de **STATUS** (en le mettant à 1 si **W**=0). Donc, cela risque de modifier le registre **STATUS** qui dans ce cas là ne sera plus intègre (car risque d'avoir une valeur différente, via **Z**, de celle qu'il avait avant l'exécution de la routine d'IT).

Par contre les deux instructions **SWAPF** ne modifient pas les flags et donc laisseront le registre **STATUS** intègre. D'où leur utilisation...

Pour en terminer avec les interruptions, il est intéressant de regrouper dans un tableau les 14 sources d'interruptions gérées par le **PIC 16F8777**.

Sont également indiqués :

- la **signification** de chacune des sources d'IT,
- les **modules/périphériques internes** concernés,
- le nom des flags **d'autorisation** de prise en compte de l'IT et ceux indiquant la **survenue** de l'évènement extérieur,
- les registres **SFR** où se trouvent ces flags,
- l'utilisation ou pas du flag semi-global **PIEI**.

C'est ce qui est fait sur la figure VIII.33.

Sources d'interruption		Flag PERn_IE	dans SFR	Flag PERn_IF	dans SFR	via PEIE
Les Ports TOR						
1	INT : front sur RB0	INTE	INTCON	INTF	INTCON	non
2	RBI : front sur RB4/RB7	RBIE	INTCON	RBIF	INTCON	non
3	PSPI : slave port PORTD et PORTE	PSPIE	PIE1	PSPIF	PIR1	oui
EEPROM						
4	EEI : fin d'écriture en EEPROM	EEIE	PIE2	EEIF	PIR2	oui
CAN						
5	ADI : fin de conversion du CAN	ADIE	PIE1	ADIF	PIR1	oui
Timer 0						
6	T0I : débordement (overflow) timer0	T0IE	INTCON	T0IF	INTCON	non
Timer 1						
7	TMR1I : débordement (overflow) timer1	TMR1IE	PIE1	TMR1IF	PIR1	oui
8	CCP1I : capture/compare CCP1	CCP1IE	PIE1	CCP1IF	PIR1	oui
9	CCP2I : capture/compare CCP2	CCP2IE	PIE2	CCP2IF	PIR2	oui
Timer 2						
10	TMR2I : valeur programmée timer2 atteinte	TMR2IE	PIE1	TMR2IF	PIR1	oui
USART						
11	RCI : réception d'un caractère par l' USART	RCIE	PIE1	RCIF	PIR1	oui
12	TXI : caractère entier émis par l' USART	TXIE	PIE1	TXIF	PIR1	oui
MSSP						
13	SSPI : caractère émis/reçu sur SPI	SSPIE	PIE1	SSPIF	PIR1	oui
14	BCLI : collision sur bus en mode I2C	BCLIE	PIE2	BCLIF	PIR2	oui

Figure VIII.33.

La figure VIII.34. fournit les adresses des registres **SFR** concernés dans le mapping de la RAM.

INTCON	0Bh	INTCON	8Bh	INTCON	10Bh	INTCON	18Bh
PIR1	0Ch	PIE1	8Ch	EEDATA	10Ch	EECON1	18Ch
PIR2	0Dh	PIE2	8Dh	EEADR	10Dh	EECON2	18Dh

Figure VIII.34.

VIII.2.5. SYNTHESE : CANEVAS EN C POUR LA ROUTINE D'IT

Je vous propose sur la figure VIII.35. un canevas type en langage C de la routine **traitement_it** permettant de gérer **deux interruptions** (la N°1 et la N°2).

Lisez bien les commentaires (des renvois ont été aussi faits aux points ① à ⑨ précédents) qui vous aideront pour une meilleure compréhension et vous permettront d'adapter avantageusement ce canevas à la plupart des applications mettant en œuvre les interruptions.

```
// routine d'interruption traitement_it de type interrupt (cf. ⑥)
// sans argument (void) et sans valeur retournée (void) (cf. ②)
// la sauvegarde et la restitution du contexte sont faits automatiquement et de façon transparente
// par le compilateur C (cf. ⑨)

void interrupt traitement_it (void) // noter le type interrupt !!!
{
    if (PER1_IF && PER1_IE) // interruption N°1 ? (cf. ⑤ et ⑦)
    {
        // oui, alors mettre ici le traitement de l'évènement extérieur N°1
        PER1_IF=0;           // acquittement de l'interruption N°1 (cf. ⑧)
    }
    if (PER2_IF && PER2_IE) // interruption N°2 ? (cf. ⑤ et ⑦)
    {
        // oui, alors mettre ici le traitement de l'évènement extérieur N°2
        PER2_IF=0;           // acquittement de l'interruption N°2 (cf. ⑧)
    }
}

void main(void)
{
    // préparation du fonctionnement sous interruption : gestion des flags et des masques
    PER1_IF = 0;           // efface une éventuelle survenue évènement N°1
    PER2_IF = 0;           // efface une éventuelle survenue évènement N°2
    PER1_IE = 1;           // démasquage local de l'interruption N°1 (cf. ④)
    PER2_IE = 1;           // démasquage local de l'interruption N°2 (cf. ④)
    GIE=1;                 // démasquage global de toutes les interruptions (cf. ③)
    while(1)               // boucle « applicative » sans fin
    {
        ...
        // ici dans cet exemple le main ne fait rien : mettez ici votre application
        // notez qu'il n'y aura jamais d'appel à traitement_it dans ce main (cf. ①)
        ...
    }
}
```

Figure VIII.35.

Remarque :

Y a-t-il une autre façon de réaliser le canevas précédent sans mettre en œuvre les interruptions ?

La réponse est oui, et vous la connaissez : c'est la technique du **polling** (de la scrutation).

On peut le formaliser comme indiqué sur la figure VIII.36.

```
void main(void)
{
    PER1_IF = 0;           // efface une éventuelle survenue événement N°1
    PER2_IF = 0;           // efface une éventuelle survenue événement N°2

    while(1)                // boucle « applicative » sans fin
    {
        if (PER1_IF)         // l'évènement N°1 a t'il eu lieu ?
        {
            // oui, alors mettre ici le traitement de l'évènement extérieur N°1
            PER1_IF = 0;      // remise à 0 du flag événement N°1
        }
        if (PER2_IF)         // l'évènement N°2 a t'il eu lieu ?
        {
            // oui, alors mettre ici le traitement de l'évènement extérieur N°2
            PER2_IF = 0;      // remise à 0 du flag événement N°2
        }
        ... // suite du main
    }
}
```

Figure VIII.36.

Remarque :

On aurait pu aussi utiliser une boucle while au lieu du test conditionnel if sur les flags PERi_IF (ce qui aurait alors rendu le traitement « bloquant » car en attente de la survenue de l'évènement extérieur).

Du coup, il est pertinent de se demander quelles sont les différences entre la méthode avec et celle sans les interruptions ? Quand utiliser plutôt l'une que l'autre ?

En réalité, comme souvent, il n'y a pas une réponse unique, donc pas de recette « miracle » qui marche tout le temps. Cela dépend en fait du type d'applications.

Essayons tout de même de donner quelques pistes de réflexion.

Dans le cas de la technique du pooling pour traiter les évènements extérieurs, le principal risque est de rater un évènement.

En effet, si l'intervalle de temps entre deux scrutations successives d'un même évènement (imposé par la durée d'exécution de l'application) est trop important, plusieurs évènements peuvent avoir eu lieu et donc n'auront pas été gérés.

Par exemple, un nouveau caractère reçu sur une liaison série asynchrone peut arriver alors que le précédent n'a pas encore été récupéré du registre de réception de l'USART : ce nouveau écrasera le précédent et on perdra (au moins) un caractère.

Conclusion : si la technique du pooling est plus simple à mettre en œuvre, il faut à tout prix s'assurer, en évaluant bien les contraintes temporelles (et notamment la durée minimum entre deux occurrences successives d'un même évènement) qu'aucun évènement ne sera raté.

Ce problème est atténué lorsqu'on utilise les interruptions car chaque évènement se signale justement en déclanchant une IT matérielle. Par contre, pour mettre en œuvre les interruptions, c'est toujours plus délicat :

- à programmer car il ne faut rien oublier : routine d'IT à écrire, détermination de la cause d'IT, sauvegarde et restitution du contexte, acquittement de l'IT, démasquage global (éventuellement semi-global) et local...

- à mettre au point sous **ICD3** à cause du caractère asynchrone des IT et des difficultés inhérentes au traitement en temps réel.

Mais, si l'intervalle de temps minimum entre deux occurrences successives d'un même évènement est de façon récurrente plus court que la durée d'exécution de la routine d'IT, il y aura également un problème pour le traitement.

Conclusion : une routine d'IT doit être la plus courte, la plus concise et donc la plus rapide possible...

Mais si vous avez bien compris ce chapitre, cela devrait vous faciliter la vie !!!

Des éléments supplémentaires sur les différentes interruptions matérielles seront donnés plus loin, lorsqu'on détaillera toutes les sources possibles d'interruptions.

Nous verrons également en TP comment concevoir un programme qui gère une interruption matérielle.

VIII.3. LES EEPROM INTERNES

(cf. ② et ⑦ du schéma de la **figure II.4.** et cf. **pages 33 à 39** du datasheet)

Il y a deux blocs EEPROM dans le PIC16F877 :

- **la mémoire EEPROM de données de taille 256 octets.**
Elle contient les données semi-permanentes (à ne pas confondre avec la RAM qui contient les SFR et les GPR et des données volatiles).
- **la mémoire EEPROM de programme (déjà vue) de taille 8K cases de 14 bits chacune.**
On la nomme aussi **l'EEPROM FLASH.**
Elle contient bien sûr le code machine du programme, mais peut aussi servir, si l'EEPROM de données précédente est pleine, pour y stocker des données semi-permanentes supplémentaires.

Dans les deux cas, ces mémoires conserveront leur contenu même après une coupure de l'alimentation du PIC.

On a déjà vu une première méthode pour écrire des valeurs dans ces mémoires au moment de la **phase de programmation** du PIC
(code en EEPROM de programme et directives **ORG 0x2100 + DE** pour les données en EEPROM de données).

Il existe également une autre méthode pour accéder **en dynamique** (pendant l'exécution du programme) en lecture ou en écriture à ces deux EEPROM internes.

Plusieurs registres SFR sont concernés et doivent être utilisés pour assurer ces accès.

Parmi ceux-ci, on trouvera :

- les registres contenant la **valeur à écrire** en EEPROM ou la **valeur lue** dans l'EEPROM :

EEDATA et EEDATH (comme **EEprom DATA** et **DATa High**)

- les registres contenant **l'adresse** de la case EEPROM concernée par l'accès en lecture ou en écriture (plus précisément l'offset de la case par rapport aux adresses de base des EEPROM soit 0x2100 pour l'EEPROM de données et 0x0000 pour celle de programme)

EEADR et EEADRH (comme **EEprom ADReSse** et **ADReSse High**)

- les registres permettant le **paramétrage** et le **contrôle** de l'accès réalisé : choix EEPROM, lecture ou écriture, autorisation d'écriture ...

EECON1 et EECON2 (comme **EEprom CONtrôle 1 et 2**)

Compte tenu :

- des capacités mémoire des EEPROM (8 bits d'adresses pour différencier les 256 ($=2^8$) cases de l'EEPROM de données et 13 bits d'adresses pour les 8192 ($=2^{13}$) cases de l'EEPROM de programme),
- des tailles de chaque case (8 bits pour l'EEPROM de données et 14 bits pour l'EEPROM de programme),

le schéma de la figure VIII.37. montre comment sont utilisés les registres **EEDATA**, **EEDATH**, **EEADR** et **EEADRH**.

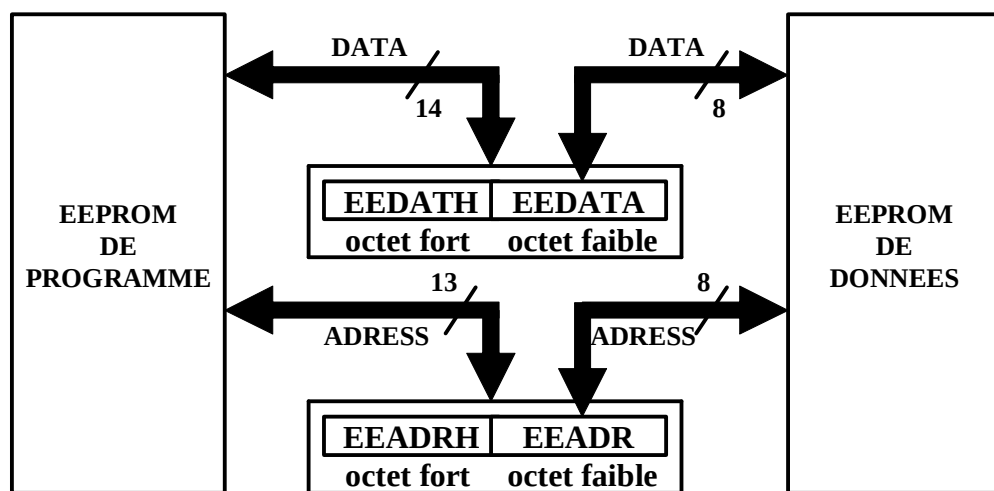


Figure VIII.37.

Le format du registre **EECON1** est fourni sur la figureVIII.38.

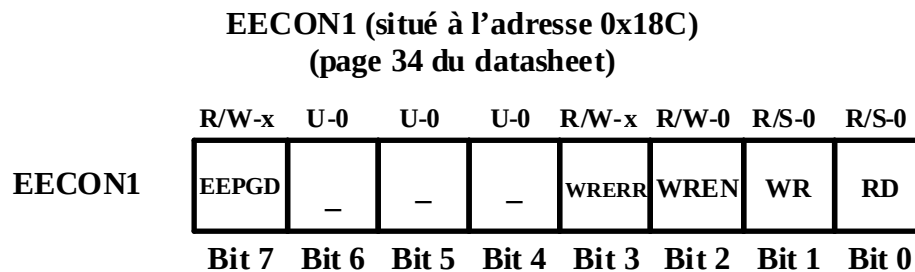


Figure VIII.38.

La signification de chacun de ses bits est fournie sur le tableau de la figure VIII.39.

Bit	Valeur	Signification et commentaires
EEPGD		Permet de sélectionner l'EEPROM concernée par l'accès
	0	Accès (lecture ou écriture) à la mémoire EEPROM de données
	1	Accès (lecture ou écriture) à la mémoire EEPROM de programme
RD		Accès EEPROM en lecture
	0	Pas de lecture
	1	Initialise un cycle de lecture EEPROM (données ou programme) Repasse automatiquement à 0 à la fin de la lecture : - à l'instruction suivante pour l'EEPROM de données, - après les 2 instructions suivantes pour l'EEPROM de programme.
WREN		Permet d'ouvrir une « fenêtre » d'écriture EEPROM
	0	Pas de « fenêtre » d'écriture
	1	Ouvre une « fenêtre » d'écriture : doit être mis à 1 avant le début du cycle d'écriture en EEPROM et remis à 0 à la fin du cycle d'écriture
WR		Accès EEPROM en écriture
	0	Pas d'écriture
	1	Initialise un cycle d'écriture EEPROM (données ou programme). Un tel cycle est plus long qu'une écriture en RAM. Il faut toujours s'assurer que ce cycle soit fini avant de refaire une autre écriture en EEPROM. Ce bit repassera automatiquement à 0 pour indiquer la fin de l'écriture EEPROM proprement dite. Le bit EEIF du registre SFR PIR2 passera alors de son côté à 1 (voir plus loin comment gérer ces bits WR ou EEIF)
WRERR		Bit d'état indiquant un problème pendant le cycle d'écriture (il y a eu par exemple un reset du PIC)
	0	Le cycle d'écriture s'est bien déroulé
	1	Le cycle d'écriture EEPROM a été interrompu et du coup la donnée n'a pas pu être écrite en EEPROM !!!

Figure VIII.39.

Le registre **EECON2** (situé à l'offset 0x8D de la bank 3 de la RAM) est utilisé pour permettre une procédure un peu spéciale permettant d'éviter des écritures « intempestives » en EEPROM, un peu comme une espèce de « **clef de protection** ». Cette procédure consiste à enchaîner (et donc **sans interruption**) une séquence de 5 instructions (imposées par MICROCHIP) avant l'écriture proprement dite. On détaillera cette protection dans les exemples de programmes vus plus loin (voir phase ⑥ de la figure VIII.41. ou de la figure VIII.43.).

Mais l'idée est que « statistiquement » cette procédure a très très très peu de chance de se produire sans qu'on le souhaite réellement (par un dysfonctionnement lié à un parasite par exemple). Ce qui évite d'écrire n'importe quoi en EEPROM de données ou de programme.

VIII.3.1. ACCES A L'EEPROM DE DONNEES

Nous donnerons dans cette partie des exemples de modules en assembleur permettant d'accéder en lecture ou en écriture à l'EEPROM de données.

En TP, nous aurons à les écrire en langage C.

VIII.3.1.1. ACCES EN LECTURE

Remarque préliminaire :

Comme on l'a vu dans le **chapitre III.5.** sur la configuration, le bit **CPD**, lorsqu'il est programmé à 0, protège le contenu de l'EEPROM de données d'une lecture initiée **par l'extérieur** du PIC (via un outil de développement comme ICD3 par exemple).

C'est une protection anti-piratage de données qui dans certaines applications doivent rester secrètes ou confidentielles.

Mais ce bit, même s'il est à 0 n'empêche en rien un accès en lecture de l'EEPROM de données par le programme qui s'exécute sur le PIC.

Pour lire un octet dans l'EEPROM de données, la procédure à respecter est la suivante :

① charger le registre SFR **EEADR** avec l'offset de la case EEPROM données concernée par la lecture.

Par exemple : **0** pour la case d'adresse absolue 0x210**0**,
 1 pour la case d'adresse absolue 0x210**1**,
 2 pour la case d'adresse absolue 0x210**2**...

② sélectionner la mémoire EEPROM de données : **EEPGD = 0**,

③ lancer un cycle de lecture : **RD = 1**,

Ce bit repassera automatiquement à 0 tout de suite après le transfert de la donnée entre l'EEPROM de données et le registre **EEDATA**.

④ récupérer le contenu du registre SFR **EEDATA** (qui contient alors le contenu de la case EEPROM concernée suite à la phase ③ précédente).

Ce qui donne le module **read_data_eeprom** de la figure VIII.40. où les 4 phases précédentes ont été effectuées (lire les commentaires!!!)

```

,*****
;
; module read_data_eeprom de prototype : char read_data_eeprom (char offset)
; lecture d'un octet à l'adresse offset de l' EEPROM de données
; en entrée : W contient l'offset dans l' EEPROM (paramètre passé par W)
; en sortie : le contenu de cette adresse est renvoyé dans W
,*****
read_data_eeprom
    banksel    EEADR        ; phase ①
    movwf     EEADR        ; initialisation de EEADR via W

    banksel    EECON1       ; phase ②
    bcf       EECON1,EEPGD  ; sélection EEPROM de données

    bsf       EECON1,RD     ; phase ③ : cycle de lecture

    banksel    EEADR        ; phase ④
    movf      EEDATA,w      ; récupération dans W

    return
,*****

```

Figure VIII.40.

VIII.3.1.2. ACCES EN ECRITURE

Pour écrire un octet dans l'EEPROM de données, la procédure à respecter est la suivante :

① charger le registre SFR **EEADR** avec l'offset de la case EEPROM données concernée par l'écriture.

Par exemple : **0** pour la case d'adresse absolue 0x210**0**,
 1 pour la case d'adresse absolue 0x210**1**,
 2 pour la case d'adresse absolue 0x210**2**...

② charger le registre SFR **EEDATA** avec la valeur à écrire dans la case EEPROM concernée,

③ sélectionner la mémoire EEPROM de données : **EEPGD = 0**,

④ ouverture d'une « fenêtre » d'écriture : **WREN = 1**,

⑤ masquer toutes les interruptions pour préparer la « clef de protection » : **GIE = 0**,

⑥ écrire les 5 instructions (qui ne doivent pas être interrompues) fournies par **MICROCHIP** pour réaliser la clef de protection contre les écritures « intempestives »,

⑦ on peut ré autoriser les interruptions : **GIE = 1**,

⑧ fermeture de la « fenêtre » d'écriture : **WREN = 0**,

Mais à partir de là, la valeur **n'est pas encore écrite dans l'EEPROM** de données.

Elle ne le sera réellement que lorsque le bit **WR** passera à 0 (et en même temps le bit **EEIF** passera de son côté à 1).

Pour éviter d'effectuer une autre écriture en EEPROM avant que la valeur courante ne soit écrite, il faut donc attendre (par pooling) que le bit **WR** passe à 0 (ou que le bit **EEIF** passe à 1). Car sinon, il y aura un conflit d'écriture en EEPROM. C'est l'objectif de la phase ⑨ suivante.

⑨ attendre par une boucle de scrutation que le bit **WR** passe à 0. Compter environ 4 à 5 ms (temps d'écriture typique dans une case EEPROM), ce qui est énorme par rapport au temps de cycle instruction qui est de 1 µs pour un quartz à 4 MHz.

Remarque 1 :

Pour la phase ⑨, on aurait pu aussi attendre que le bit **EEIF** passe à 1. Mais il aurait alors fallu ensuite ne pas oublier de le remettre à 0 !!!

Remarque 2 :

On aurait pu aussi fonctionner sous interruption en mettant le bit **EEIE** (du registre SFR **PIE2**) à 1 mais cela aurait été un peu plus compliqué.

Ce qui donne le module **write_data_eeprom** de la figure VIII.41. où les 9 phases précédentes ont été effectuées (lire les commentaires!!!)

```

;*****
; module write_data_eeprom de prototype : void write_data_eeprom (char offset, char valeur)
; écriture d'un octet valeur à l'adresse offset de l' EEPROM de données
; en entrée : W contient l'offset dans l' EEPROM (paramètre passé par W)
; write_data_eeprom_par_1 : valeur à écrire (paramètre 1)
; en sortie : aucune valeur retournée
;*****
write_data_eeprom
    banksel    EEADR                ; phase ①
    movwf     EEADR                ; initialisation de EEADR via W

    movf      write_data_eeprom_par_1,w ; phase ②
    movwf     EEDATA              ; initialisation EEDATA via W

    banksel    EECON1              ; phase ③
    bcf        EECON1,EEPGD        ; sélection EEPROM de données

    bsf        EECON1,WREN         ; phase ④
                                ; ouverture d'une « fenêtre » d'écriture

    bcf        INTCON,GIE          ; phase ⑤ : interdiction des IT pendant les 5
                                ; instructions suivantes non interruptibles

    movlw     0x55                 ; phase ⑥ : « clef de protection »
    movwf     EECON2              ; ces 5
    movlw     0xAA                 ; instructions
    movwf     EECON2              ; ne doivent pas être
    bsf        EECON1,WR          ; interrompues : lancement de l'écriture

    bsf        INTCON,GIE          ; phase ⑦ : on ré autorise les interruptions
    bcf        EECON1,WREN        ; phase ⑧
                                ; fermeture de la « fenêtre » d'écriture

write_data_eeprom_attend                ; phase ⑨
    btfsc     EECON1,WR            ; écriture EEPROM terminée ?
    goto     write_data_eeprom_attend ; non, alors on attend

    return                        ; oui, alors fin du module...
;*****

```

Figure VIII.41.

Pour améliorer la fiabilité de ce module **write_data_eeprom**, on peut y rajouter les instructions qui permettent de vérifier que la valeur qu'on souhaite écrire en EEPROM de données a bien été écrite en EEPROM de données.

Comment cela?

Tout simplement en enchaînant une lecture de la case mémoire où on vient d'écrire : on peut ainsi comparer que la valeur lue correspond bien à celle qu'on souhaite écrire.

Ce qui revient à faire du « **write and verify** ».

Dans ce cas là, il faut concevoir un module dont le prototype serait :

char **write_verify_data_eprom** (char offset, char valeur)

et donc qui renvoie en retour un code erreur comme par exemple :

- **0x01**, si **tout s'est bien passé** : valeur lue **identique** à la valeur qu'on voulait écrire,
- **0x00**, s'il y a eu **un problème** : valeur lue **différente** de la valeur qu'on voulait écrire.

VIII.3.2. ACCES A L'EEPROM DE PROGRAMME

Remarque préliminaire :

Comme on l'a vu dans le **chapitre III.5.** sur la configuration, un mécanisme de protection contre les accès extérieurs ou intérieurs de la mémoire de programme existe.

Accès internes (par programme s'exécutant sur le PIC) :

Le bit **WRT**, quand il est à 0, interdit les écritures internes dans toute la mémoire de programme. S'il est à 1, seules les zones protégées par les bits **CP1-CP0** sont inaccessibles par écriture interne, les autres zones, non protégées, peuvent donc être modifiées.

Les lectures internes sont quant à elles toujours possibles.

Dans notre cas, pour pouvoir accéder à toute la mémoire de programme en écriture et en lecture interne, il faudra donc mettre **WRT = 1** et **CP1-CP0 = 11** (soit le masque **WRTEN** ou rien par défaut).

Accès externes (par ICD3 par exemple) :

Les zones protégées par les bits **CP1-CP0** sont inaccessibles par lecture externe (anti-piratage). Les écritures externes ne sont possibles que quand aucune zone n'est pas protégée.

VIII.3.2.1. ACCES EN LECTURE

Pour lire un mot (14 bits) dans l'EEPROM de programme, la procédure à respecter est similaire à celle vue pour l'EEPROM de données.

Les principales différences sont dues au fait qu'il faut ici deux registres SFR pour l'offset de la case mémoire lue (car cet offset est sur 13 bits) et aussi deux registres SFR pour stocker le contenu de la case lue (car la valeur est sur 14 bits).

Pour comprendre cela, on peut se reporter au schéma vu précédemment sur la figure VIII.37.

Voici donc les différentes étapes de la procédure permettant de lire une valeur dans la mémoire de programme.

① charger les deux registres SFR :

- **EEADR** avec les 8 bits de poids faibles de l'offset de la case EEPROM de programme concernée par la lecture,
- **EEADRH** avec les 5 bits de poids forts de l'offset de la case EEPROM de programme concernée par la lecture.

② sélectionner la mémoire EEPROM de programme : **EEPGD = 1**,

③ lancer un cycle de lecture : **RD = 1**,

Ce bit repassera automatiquement à 0 deux cycles après le transfert de la donnée entre l'EEPROM de programme et les registres **EEDATH-EEDATA**. D'où la nécessité de la phase ④ suivante.

④ rajouter deux instructions **NOP** pour laisser le temps à la lecture,

⑤ récupérer enfin le contenu des deux registres SFR :

- **EEDATH** qui contient alors les 6 bits de poids forts de la case EEPROM concernée suite à la phase ③ précédente),
- **EEDATA** qui contient alors les 8 bits de poids faibles de la case EEPROM concernée suite à la phase ③ précédente).

Ces différentes étapes ont été réalisées dans la fonction **read_code_eeprom** dont le code est fourni sur la figure VIII.42. en langage C (lire les commentaires!!!).

Rappel : un int de type **short** est codé sur **deux octets** en mémoire.

```
// renvoie le contenu de la case EEPROM de programme d'adresse offset
short int read_code_eeprom(unsigned short int offset)
{
    EEADR = offset;                // phase ①
    EEADRH = (offset>>8) & 0x1F;  // octet faible de offset dans EEADR
                                   // 5 bits poids forts de offset dans EEADRH

    EEPGD=1;                       // phase ② : selection EEPROM programme

    RD=1;                          // phase ③ : début cycle de lecture

    #asm                          // assembleur en ligne dans code C
    NOP                          // phase ④ : temps de lecture
    NOP                          // avec deux NOP
    #endasm                      // fin code assembleur en ligne

    return(((EEDATH<<8) & 0x3FFF ) | EEDATA); // valeur retournée sur 14 bits
}
```

Figure VIII.42.

Ainsi, en écrivant :

```
short int valeur;                // sur deux octets

...
    valeur = read_code_eeprom(0x784); // valeur contient le contenu de la case 0x784
...
```

Remarquons, qu'on a gardé le terme offset pour repérer la case mémoire en EEPROM de programme pour garder la même logique que pour l'EEPROM de données. En fait, comme l'adresse de début (adresse de base) de l'EEPROM de programme est 0x0000, l'offset est aussi l'adresse absolue de la case EEPROM de code.

Pour l'EEPROM de données, pour avoir l'adresse absolue de la case, il faut rajouter son offset à l'adresse 0x2100 (adresse de base de l'EEPROM de données).

VIII.3.2.2. ACCES EN ECRITURE

Ici aussi, pour écrire un mot (14 bits) dans l'EEPROM de programme, la procédure à respecter est similaire à celle vue pour l'EEPROM de données (avec les mêmes remarques que pour un accès en lecture concernant les différences de taille pour les adresses et les valeurs à écrire).

En voici les différentes étapes :

① charger les deux registres SFR :

- **EEADR** avec les 8 bits de poids faibles de l'offset de la case EEPROM de programme concernée par la l'écriture,
- **EEADRH** avec les 5 bits de poids forts de l'offset de la case EEPROM de programme concernée par l'écriture.

② charger les deux registres SFR :

- **EEDATA** avec les 8 bits de poids faibles de la valeur à écrire dans la case de l'EEPROM de programme concernée par l'écriture,
- **EEDATH** avec les 6 bits de poids forts de la valeur à écrire dans la case de l'EEPROM de programme concernée par l'écriture.

③ sélectionner la mémoire EEPROM de programme : **EEPGD = 1**,

④ ouverture d'une « fenêtre » d'écriture : **WREN = 1**,

⑤ masquer toutes les interruptions pour préparer la « clef de protection » : **GIE = 0**,

⑥ écrire les 5 instructions (qui ne doivent pas être interrompues) fournies par **MICROCHIP** pour réaliser la clef de protection contre les écritures « intempestives ».

Le cycle d'écriture est alors lancé vers la mémoire de programme. Du coup, le PIC doit provisoirement arrêter ce programme mais s'y rebranchera deux cycles plus loin après la fin de l'écriture proprement dite. D'où la nécessité de la phase ⑦ suivante.

⑦ rajouter deux instructions **NOP** pour le retour vers cette procédure,

⑧ on peut ré autoriser les interruptions : **GIE = 1**,

⑨ fermeture de la « fenêtre » d'écriture : **WREN = 0**.

A partir de là, la valeur **est réellement écrite dans l'EEPROM** de programme, car rappelez-vous que lorsque le PIC a lancé le cycle d'écriture, il attend que celle-ci soit terminée pour revenir à cette procédure. Du coup, il n'est pas nécessaire de faire un pooling sur **WR** (car ce bit est déjà à 0) ou sur **EEIF** (car ce bit est déjà à 1) comme on l'avait fait pour une écriture en EEPROM de données.

Ces différentes étapes ont été réalisées dans la fonction **write_code_eeprom** dont le code est fourni sur la figure VIII.43. en langage C (lire les commentaires!!!).

```
// écrit valeur à l'adresse offset de l'EEPROM de programme
void write_code_eeprom(unsigned short int offset, unsigned short int valeur)
{
    // phase ①
    EEADR = offset; // octet faible de offset dans EEADR
    EEADRH = (offset>>8) & 0x1F; // 5 bits poids forts de offset dans EEADRH

    // phase ②
    EEDATA = valeur; // octet faible de valeur dans EEDATA
    EEDATH = (valeur>>8) & 0x3F; // 6 bits de poids forts de valeur dans EEDATH

    EEPGD=1; // phase ③ : selection EEPROM programme

    WREN=1; // ; phase ④ : ouverture d'une « fenêtre » d'écriture

    GIE=0; // phase ⑤ : interdiction des IT pendant les
           // instructions suivantes non interruptibles

    EECON2=0x55; // phase ⑥ : « clef de protection »
    EECON2=0xAA; // séquence imposée par MICROCHIP
    WR=1; // lancement de l'écriture

    #asm // assembleur en ligne dans code C
    NOP // phase ⑦ : temps de lecture
    NOP // avec deux NOP
    #endasm // fin code assembleur en ligne

    GIE=1; // phase ⑧ : on ré autorise les interruptions

    WREN=0; // phase ⑨ : fermeture de la « fenêtre » d'écriture
}
```

Figure VIII.43.

On pourrait là aussi là aussi augmenter la fiabilité de cette fonction par une technique du type « write and verify ».

La fonction **write_code_eeprom** que l'on vient d'écrire sur la figure VIII.43. convient pour les PIC de la série **16F87x** (sans le suffixe **A**).

Par contre, si on teste cette fonction sur un PIC **16F87xA**, comme celui qui est utilisé en TP, on remarque les points suivants :

- elle fonctionne correctement si l'adresse offset avait la forme **4n + 3** (n entier naturel),
- pour toute autre adresse, l'écriture en EEPROM de programme ne se fait pas !!!

Pour en comprendre la raison, il faut se référer au datasheet du PIC **16F87xA**.

Il y est dit que l'écriture en EEPROM de programme se fait par paquets de 4 valeurs, c'est-à-dire sur trois adresses de la forme **4n, 4n+1, 4n+2** pour peu qu'elles soient **suivies** d'une quatrième adresse de la forme **4n+3**.

Ces quatre valeurs sont « bufferisées » en interne par paquets de 4 adresses successives, **la dernière étant de la forme 4n+3 et venant valider** l'écriture effective de ces 4 buffers dans les cases mémoire EEPROM concernées.

Il faut donc trouver une solution pour concevoir une fonction d'écriture en EEPROM de programme compatible avec cette architecture des versions **A**.

On aurait pu revoir complètement la fonction **write_code_eeprom** écrite précédemment, sachant qu'elle est compatible avec les informations du datasheet du **16F87x**.

Ce n'est pas à mon avis la bonne solution, justement parce que la fonction **write_code_eeprom** respecte les recommandations du constructeur.

L'idée, toujours dans la philosophie de la programmation modulaire et de la structuration des programmes, est au contraire d'utiliser cette fonction telle quelle pour résoudre ce problème.

C'est ce que je vous propose sur le schéma de la figure VIII.44., où j'ai utilisé les fonctions **write_code_eeprom** et **read_code_eeprom** telles quelles pour réaliser une fonction **write_code_eeprom_A** (compatible avec les versions **A**).

Le principe est de tester l'adresse d'écriture (en fait ses 2 bits de poids faibles) pour savoir si elle est de la forme :

- soit **4n** (ses 2 bits de poids faibles valent alors **00**) : dans ce cas là, on vient lire (dans 3 variables « tampon ») les trois cases suivantes d'adresses **4n+1, 4n+2** et **4n+3**, puis on écrit la valeur à programmer dans la case d'adresse **4n** et on réécrit les 3 valeurs tampons dans leurs cases respectives, ce qui assure bien l'écriture dans 4 positions mémoires consécutives, la dernière ayant bien une adresse de la forme **4n+3**,
- soit **4n+1** (ses 2 bits de poids faibles valent alors **01**) : dans ce cas là, on vient lire (dans 3 variables « tampon ») les trois cases suivantes d'adresses **4n, 4n+2** et **4n+3**, puis on réécrit la première valeur tampon dans la case d'adresse **4n**, puis la valeur à programmer dans la case d'adresse **4n+1** et enfin, on réécrit les 2 dernières valeurs tampons dans leurs cases respectives, ce qui assure bien l'écriture dans 4 positions mémoires consécutives, la dernière ayant bien une adresse de la forme **4n+3** ...

(lire les commentaires pour bien en comprendre le principe).

```

// écrit valeur à l'adresse offset de l'EEPROM de programme
void write_code_eeprom_A(unsigned short int offset, unsigned short int valeur)
{
    unsigned short int base,tampon_1,tampon_2,tampon_3;

    base=offset & 0xFFFC; // adresse du début du buffer "bizarre"
                           // de la forme 4n
    if (offset==base) // adresse de la forme 4n
    {
        tampon_1=read_code_eeprom(base+1); // lecture adresse 4n+1
        tampon_2=read_code_eeprom(base+2); // lecture adresse 4n+2
        tampon_3=read_code_eeprom(base+3); // lecture adresse 4n+3
        write_code_eeprom(offset,valeur); // écriture dans adresse 4n
        write_code_eeprom(base+1,tampon_1); // écriture dans adresse 4n+1
        write_code_eeprom(base+2,tampon_2); // écriture dans adresse 4n+2
        write_code_eeprom(base+3,tampon_3); // écriture dans adresse 4n+3:validation
    }
    if (offset==base+1) // adresse de la forme 4n+1
    {
        tampon_1=read_code_eeprom(base); // lecture adresse 4n
        tampon_2=read_code_eeprom(base+2); // lecture adresse 4n+2
        tampon_3=read_code_eeprom(base+3); // lecture adresse 4n+3
        write_code_eeprom(base,tampon_1); // écriture dans adresse 4n
        write_code_eeprom(offset,valeur); // écriture dans adresse 4n+1
        write_code_eeprom(base+2,tampon_2); // écriture dans adresse 4n+2
        write_code_eeprom(base+3,tampon_3); // écriture dans adresse 4n+3:validation
    }
    if (offset==base+2) // adresse de la forme 4n+2
    {
        tampon_1=read_code_eeprom(base); // lecture adresse 4n
        tampon_2=read_code_eeprom(base+1); // lecture adresse 4n+1
        tampon_3=read_code_eeprom(base+3); // lecture adresse 4n+3
        write_code_eeprom(base,tampon_1); // écriture dans adresse 4n
        write_code_eeprom(base+1,tampon_2); // écriture dans adresse 4n+1
        write_code_eeprom(offset,valeur); // écriture dans adresse 4n+2
        write_code_eeprom(base+3,tampon_3); // écriture dans adresse 4n+3:validation
    }
    if (offset==base+3) // adresse de la forme 4n+3
    {
        tampon_1=read_code_eeprom(base); // lecture adresse 4n
        tampon_2=read_code_eeprom(base+1); // lecture adresse 4n+1
        tampon_3=read_code_eeprom(base+2); // lecture adresse 4n+2
        write_code_eeprom(base,tampon_1); // écriture dans adresse 4n
        write_code_eeprom(base+1,tampon_2); // écriture dans adresse 4n+1
        write_code_eeprom(base+2,tampon_3); // écriture dans adresse 4n+2
        write_code_eeprom(offset,valeur); // écriture dans adresse 4n+3:validation
    }
}

```

Figure VIII.44.

On donne enfin, et en résumé, sur la figure VIII.45., tous les registres concernés par les accès en EEPROM interne ainsi que les bits concernés.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
10Dh	EEADR	EEPROM Address Register, Low Byte								xxxx xxxx	uuuu uuuu
10Fh	EEADRH	—	—	—	EEPROM Address, High Byte					xxxx xxxx	uuuu uuuu
10Ch	EEDATA	EEPROM Data Register, Low Byte								xxxx xxxx	uuuu uuuu
10Eh	EEDATH	—	—	EEPROM Data Register, High Byte						xxxx xxxx	uuuu uuuu
18Ch	EECON1	EEPGD	—	—	—	WRERR	WREN	WR	RD	x--- x000	x--- u000
18Dh	EECON2	EEPROM Control Register2 (not a physical register)								—	—
8Dh	PIE2	—	(1)	—	EEIE	BCLIE	—	—	CCP2IE	-r-0 0--0	-r-0 0--0
0Dh	PIR2	—	(1)	—	EEIF	BCLIF	—	—	CCP2IF	-r-0 0--0	-r-0 0--0

Legend: x = unknown, u = unchanged, r = reserved, - = unimplemented, read as '0'.
Shaded cells are not used during FLASH/EEPROM access.

Figure VIII.45.

VIII.4. LE CONVERTISSEUR ANALOGIQUE/NUMERIQUE

(cf. ⑧ du schéma de la **figure II.4.** et cf. **pages 127 à 133** du datasheet)

VIII.4.1. GENERALITES ET PRINCIPES

VIII.4.1.1. INTERFACE ANALOGIQUE/NUMERIQUE

Le but est de passer du domaine de l'analogique (où les grandeurs physiques sont **continues** et donc valables $\forall t$) au domaine du numérique (où les variables sont **discrètes** et deviennent des nombres représentant la valeur du signal analogique à certains instants particuliers, nombres destinés à être traités par un micro/contrôleur/processeur).

Les applications sont là aussi nombreuses (liste non exhaustive) :

- phénomènes lents : mesure de paramètres comme une température, une pression, le degré d'humidité de l'air, la vitesse du vent ou d'un véhicule en déplacement, une concentration chimique, le module d'une force, un éclaircissement...
- phénomènes un peu plus rapides : numérisation du son, de la voix, de la musique...
- phénomènes encore plus rapides : numérisation d'une image, imagerie médicale...

Dans chaque cas, il s'agit de remplacer un signal analogique $V_a(t)$ par une liste de valeurs :

$V_a(t_1)$, $V_a(t_2)$, $V_a(t_3)$, ... $V_a(t_n)$, toutes prélevées sur ce signal analogique et continu $V_a(t)$ à différents instants t_1 , t_2 , t_3 , ... t_n , comme indiqué sur la figure VIII.46.

On parle alors d'**échantillonnage**. Les temps t_i sont les **instants d'échantillonnage**.

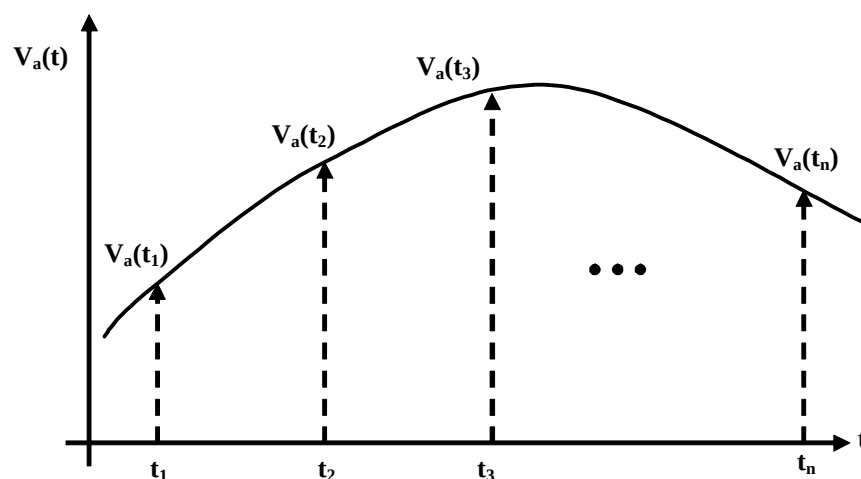


Figure VIII.46.

VIII.4.1.2. PRINCIPE ET DUREE DE CONVERSION

Il est parfois difficile de s'y retrouver dans la « jungle » des différents procédés pour réaliser le passage du monde analogique vers le monde numérique :

- convertisseurs simple ou double rampe : assez anciens, pas très précis et assez lents,
- convertisseurs « flash » : parmi les plus rapides mais aussi les plus chers,
- convertisseurs « one bit » : souvent un bon compromis entre précision, rapidité et coût...

Dans le cas du PIC, la technique utilisée par la conversion analogique/numérique s'appelle la technique par **approximations successives**.

Ce procédé n'est pas adapté lorsque les contraintes de rapidité sont cruciales comme dans le cas du traitement du son et de l'image par exemple. Mais reste bien adapté à beaucoup d'autres applications pour un coût relativement satisfaisant.

En quoi consiste cette technique ?

Comme son nom l'indique, la valeur numérique, sur 10 bits, est obtenue en plusieurs étapes, chacune étant une approximation du résultat final.

Le synoptique d'un tel convertisseur est fourni sur la figure VIII.47.

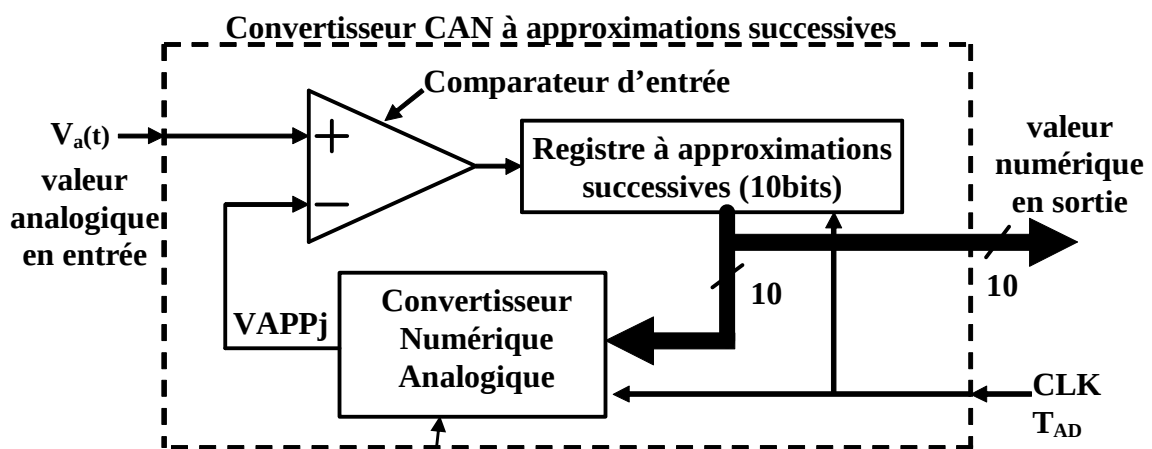


Figure VIII.47.

Notons la présence d'un **Convertisseur Numérique Analogique** (CNA) dans la boucle de retour, permettant de transformer les 10 bits du contenu courant du **registre à approximations successives** en une tension **analogique** notée **VAPPj** (pour j entier variant de 9 à 0) qui sera comparée à $V_a(t)$ grâce au **comparateur d'entrée**, comme on va le détailler maintenant.

En effet, pour en comprendre son principe de fonctionnement, il faut rentrer dans la dynamique de ce système bouclé, rythmé comme on le voit sur la figure VIII.47. par une **horloge CLK** de période T_{AD} .

La meilleure façon de le faire est de donner son organigramme de fonctionnement, en partant de son état initial (début d'une conversion) jusqu'à son état final (fin de la conversion).

C'est ce que je vous propose sur le schéma de la figure VIII.48. La tension V_{REF} qui y apparaît est une tension de référence, branchée entre les broches V_{REF}^+ et V_{REF}^- du PIC, en général 5 V pour bénéficier de l'excursion maximum du convertisseur en entrée.

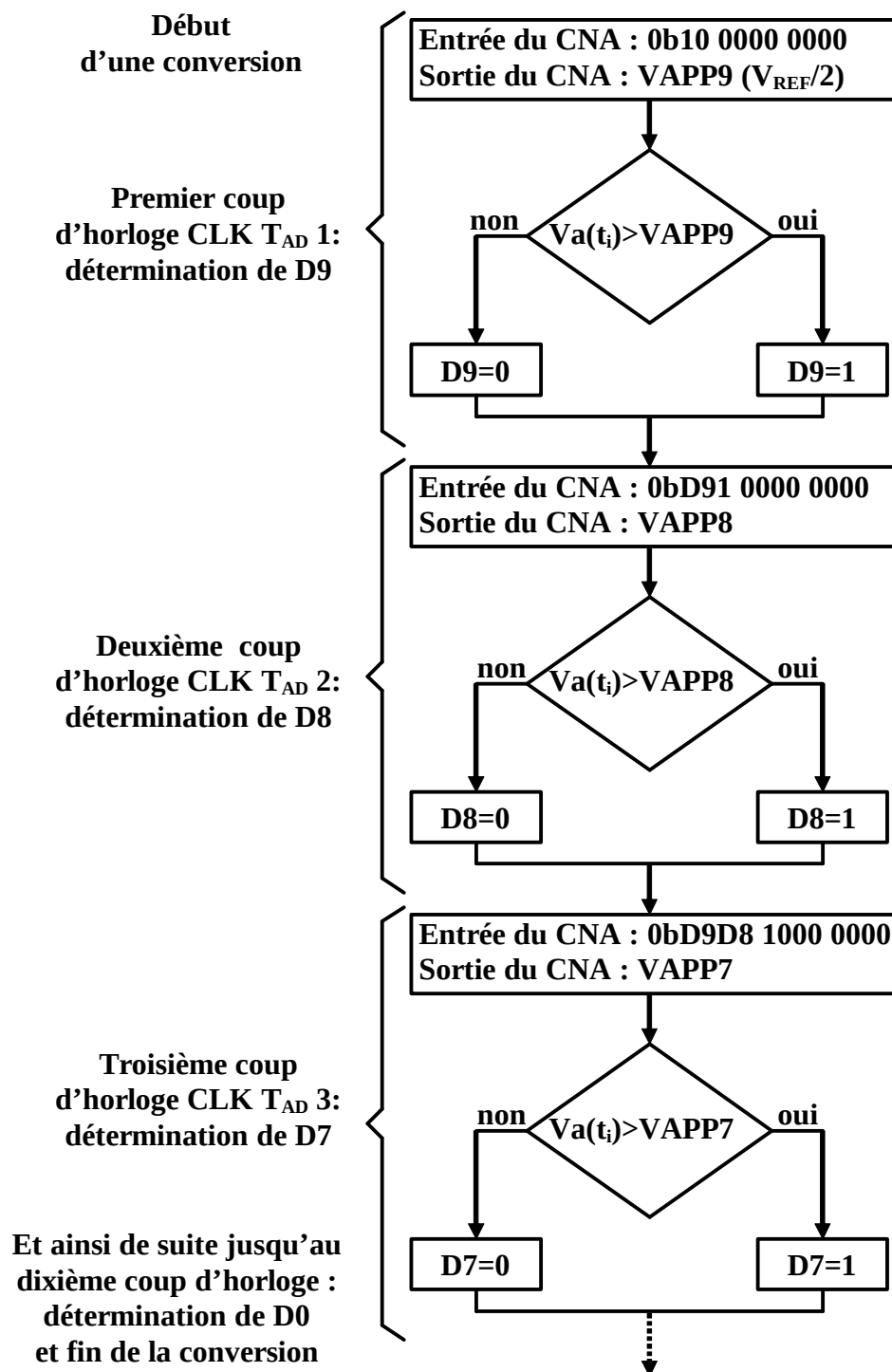


Figure VIII.48.

Face à cet organigramme, certains étudiants pensent que c'est à eux à écrire ce programme pour piloter le CAN : ils ont tort car tout ce mécanisme est **entièrement et automatiquement** géré par le CAN dès qu'un ordre de lancement de conversion est initié...

Comme on peut le voir, la conversion analogique/numérique n'est pas « instantanée » (comme pouvait le laisser croire le schéma de la figure VIII.46.) mais prendra un certain temps (puisque les 10 bits du résultat sont déterminés les uns après les autres, en partant du MSB D9 jusqu'au LSB D0). Ce temps est appelé la **durée de conversion T_C** . Si on vient lire les 10 bits de la sortie du CAN alors que ce dernier est encore en phase de conversion, le résultat sera **erronné**...

Cette durée de conversion est donc égale **au moins à $10.T_{AD}$** (T_{AD} apparaissant ainsi comme la durée de conversion pour obtenir un bit).

Par contre, il est nécessaire que la tension d'entrée du convertisseur analogique reste constante pendant toute cette durée de conversion.

Pourquoi ?

Tout simplement pour obtenir la valeur de la tension $V_a(t_i)$ à l'instant précis t_i d'échantillonnage. (après l'instant t_i , $V_a(t)$ bouge, mais l'entrée du CAN doit rester égal à $V_a(t_i)$...).

Comment y arriver ?

En interposant ce qu'on appelle un **bloqueur** entre la tension analogique à convertir $V_a(t)$ et l'entrée proprement dite du CAN, bloqueur qui n'apparaissait pas sur le schéma de la figure VIII.47. précédente.

Qu'est-ce qu'un bloqueur ?

C'est tout simplement l'association d'une capacité **C** et d'un interrupteur **K**, comme indiqué sur la figure VIII.49. :

- quand **K** est fermé, la tension $V_{CAN}(t)$ suit la tension $V_a(t)$,
- mais dès qu'on ouvre **K** à l'instant t_i et à cause de la capacité **C** qui ne peut pas se décharger, la tension $V_{CAN}(t)$ reste égale à $V_a(t_i)$ (valeur de $V_a(t)$ au moment de l'ouverture de **K**). Ce qui était bien le but recherché.

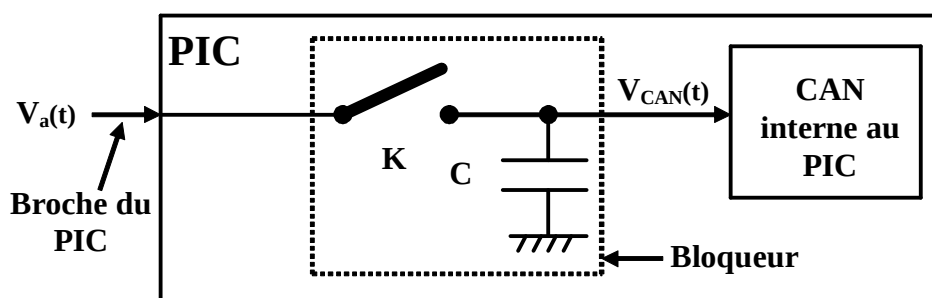


Figure VIII.49.

On peut donc reprendre le schéma de la figure VIII.46. pour indiquer la différence entre la tension analogique $V_a(t)$ (injectée sur une des broches d'entrées **RAi** ou **REi** du PIC) et la tension $V_{CAN}(t)$ (en pointillés) présente à l'entrée du CAN.

C'est ce qui apparaît sur la figure VIII.50. où on peut aussi voir l'état de l'interrupteur **K**.

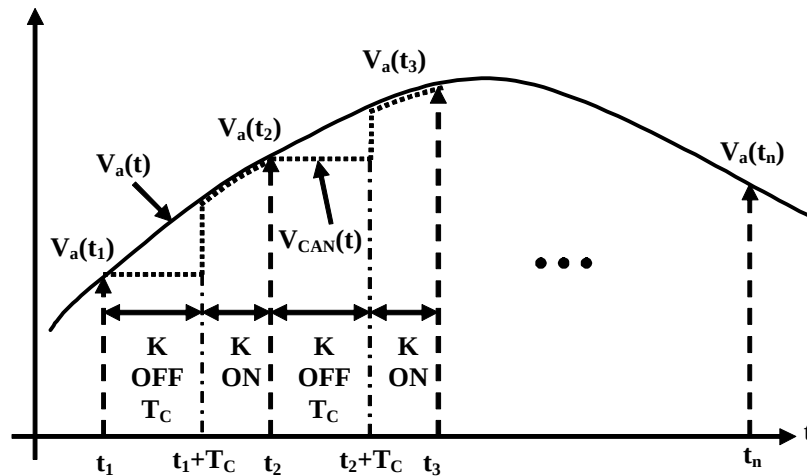


Figure VIII.50.

En réalité, pour initialiser tout ce procédé, commander l'interrupteur **K** du bloqueur ..., il faut rajouter deux périodes T_{AD} supplémentaires au $10.T_{AD}$ précédent, et au final, on retiendra que :

$$T_C = 12.T_{AD}$$

VIII.4.1.3. LINEARITE ET PRECISION

On a vu que la valeur numérique était codée sur 10 bits :
de 0b00 0000 000 = 0x000 à 0b11 1111 1111 = 0x3FF.

La **courbe de transfert** entre la valeur analogique (en ordonnées) et la valeur numérique (en abscisses) est fournie sur la figure VIII.51. :

elle est linéaire, bien qu'en fait elle soit constituée d'un très grand nombre ($2^{10} = 1024$) de marches d'escaliers comme le montre le zoom sur cette même figure VIII.51.

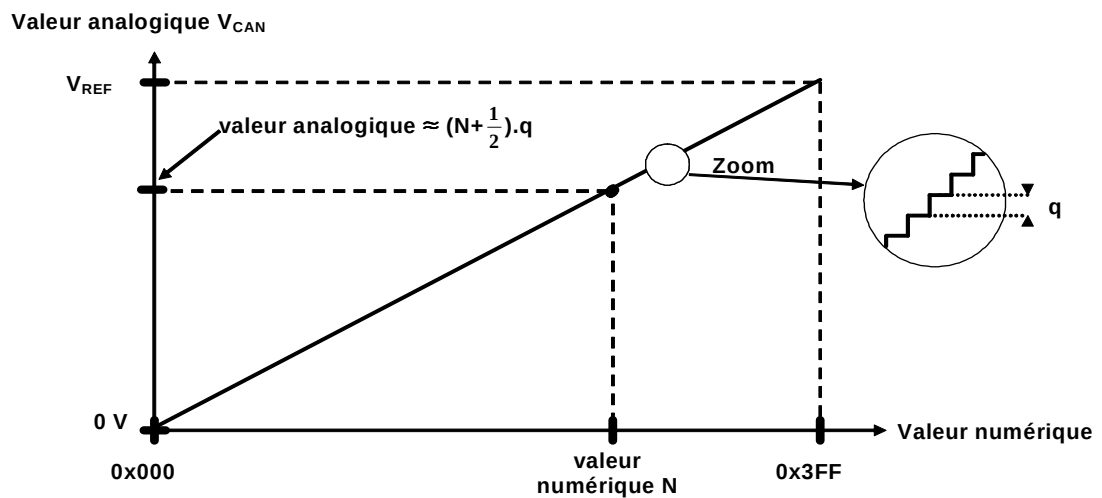


Figure VIII.51.

La valeur analogique V_{REF} détermine la **plage de conversion (Full Scale Input Voltage)**.

Cette plage de conversion est divisée en 2^{10} intervalles égaux, chacun de taille $q = \frac{V_{REF}}{2^{10}}$ appelée le **pas de quantification**.

Exemple : pour $V_{REF} = 5\text{V}$, on obtiendra : $q \approx 5\text{ mV}$.

Lorsque la valeur numérique de la tension analogique V_{CAN} vaut N, cela signifie que la tension V_{CAN} se trouve dans l'intervalle : $[N \cdot q ; (N+1) \cdot q]$, comme le montre le schéma de la figure VIII.52. ci-dessous :

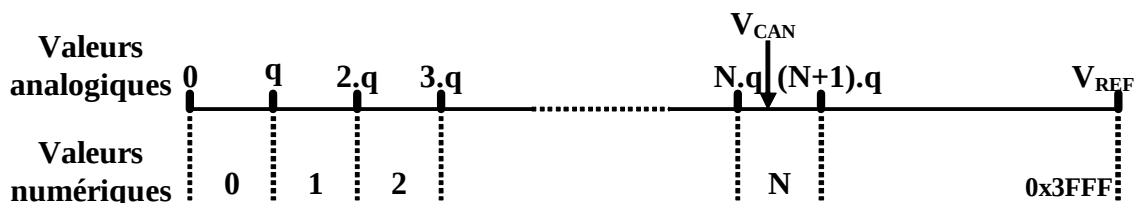


Figure VIII.52.

On a donc :
$$V_{CAN} = (N + \frac{1}{2}) \cdot q \left(\pm \frac{q}{2} \right)$$

(on parle souvent alors d'une erreur égale à $\frac{1}{2}$ LSB. Ici ce sera $\frac{q}{2} = \frac{V_{REF}}{2^{11}} = \frac{V_{REF}}{2048}$)

On peut inversement écrire :
$$N = \text{int} \left(\frac{V_{CAN}}{q} \right)$$
 (avec int = partie entière).

Remarque :

Chaque fois qu'on rajoute un bit pour le codage en numérique, on a un convertisseur deux fois plus précis.

Avec 10 bits comme ici, la précision obtenue est largement suffisante dans la plupart des applications...

VIII.4.1.4. TEMPS D'ACQUISITION

On vient de présenter un bloqueur idéal.

Pourquoi ?

Tout simplement parce qu'on a négligé, dans un premier temps, la résistance totale R_{eq} intervenant dans le schéma de la figure VIII.49. Cette résistance R_{eq} est due à plusieurs causes, et parmi celles-ci :

- la résistance de Thévenin R_{th} due à la source générant la tension $V_a(t)$,
- la résistance non nulle R_{ON} de l'interrupteur K quand il est fermé,
- les résistances R_{IC} d'InterConnexions diverses...

Ce qui fait, qu'au final, le bloqueur réel, ressemblera plutôt au schéma de la figure VIII.53.

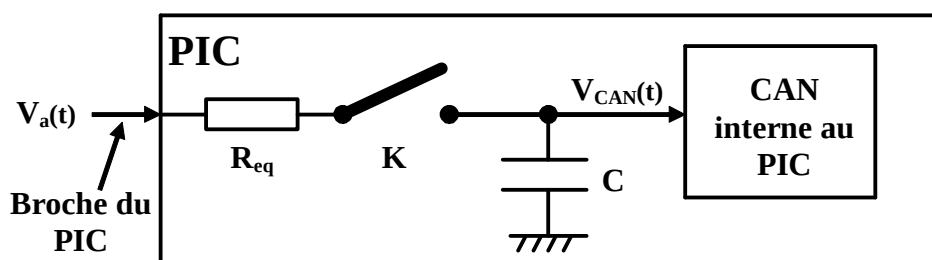


Figure VIII.53.

Dans ce cas là, quand on ferme l'interrupteur K , la tension $V_{CAN}(t)$ **ne vient pas instantanément** suivre la tension $V_a(t)$. Car, à cause de R_{eq} , il faut un certain temps pour que la capacité C se charge sous la tension $V_a(t)$.

Les courbes obtenues dans ce cas ressembleront à celles de la figure VIII.54.

On voit bien que contrairement à celui de la figure VIII.50., la tension $V_{CAN}(t)$ met un certain temps pour revenir « se coller » à la tension $V_a(t)$. Ce qui va nous permettre d'introduire la notion de temps **d'acquisition**.

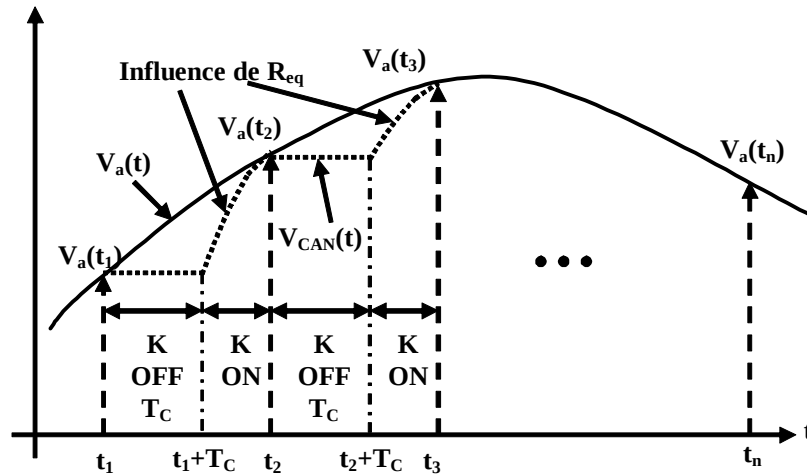


Figure VIII.54.

On va regarder par exemple ce qui se passe entre deux instants quelconques t_{i-1} et t_i .

Rigoureusement, il faudrait un temps infini pour que la tension aux bornes de **C** atteigne sa valeur asymptotique $V_a(t_i)$.

En effet, un petit rappel d'électronique analogique, montre que, quand l'interrupteur **K** est fermé, par exemple à l'instant $t_{i-1}+T_C$ (où la tension V_{CAN} vaut $V_a(t_{i-1})$), l'évolution de la tension $V_{CAN}(t)$ aux bornes de **C** est de la forme :

$$V_{CAN}(t) = A \cdot e^{-\frac{t}{\tau}} + B \text{ en posant } \tau = R_{eq} \cdot C, \text{ constante de temps du circuit } R_{eq}C.$$

(forme générale d'une charge de condensateur à travers une résistance)

Pour trouver **A** et **B**, on utilise les conditions initiales et finales :

- à $t = 0$: origine prise à l'instant $t_{i-1}+T_C$, on a $V_{CAN}(0) = A + B = V_a(t_{i-1})$,
- quand $t \rightarrow +\infty$ valeur asymptotique $= V_a(t_i) = B$.

Ce qui donne :

$$V_{CAN}(t) = (V_a(t_{i-1}) - V_a(t_i)) \cdot e^{-\frac{t}{\tau}} + V_a(t_i).$$

On voit bien qu'il faut rigoureusement un temps infini pour atteindre $V_a(t_i)$.

Mais en fait, compte tenu de la précision du CAN vue précédemment, dès que V_{CAN} atteint la valeur $V_a(t_i) - \frac{V_{REF}}{2048}$, tout se passe comme si la valeur $V_a(t_i)$ avait été atteinte (puisque l'on rentre dans les limites de la précision du CAN, qui ne pourra alors pas faire la différence entre $V_a(t_i)$ et $V_a(t_i) - \frac{V_{REF}}{2048}$).

On peut donc calculer le temps T_{AC} pour que la valeur $V_a(t_i)$ soit atteinte à $\pm \frac{V_{REF}}{2048}$ en résolvant l'équation :

$$V_a(t_i) - \frac{V_{REF}}{2048} = (V_a(t_{i-1}) - V_a(t_i)) \cdot e^{-\frac{T_{AC}}{\tau}} + V_a(t_i)$$

Ce qui donne :

$$T_{AC} = \tau \cdot \ln \left[2048 \cdot \frac{(V_a(t_i) - V_a(t_{i-1}))}{V_{REF}} \right].$$

Si on prend $V_a(t_i) - V_a(t_{i-1}) = V_{REF}$ pour être dans le cas le plus critique, on obtient :

$$T_{AC} = \tau \cdot \ln(2048)$$

On ne peut donc pas lancer une nouvelle conversion à l'instant t_i tant que la durée T_{AC} ne s'est pas écoulée après la fin de la conversion précédente, soit après l'instant $t_{i-1} + T_C$.

Sinon on ne mesurera pas la valeur $V_a(t_i)$.

Cette durée T_{AC} s'appelle la **durée d'acquisition**.

Si on prend $C = 120$ pf (cf. datasheet) et $R_{eq} \approx 10$ k Ω , on obtient : $T_{AC} \approx 10$ μ s.

En pratique, il faut en plus rajouter $2 \cdot T_{AD}$ à cette durée (temps nécessaire pour fermer l'interrupteur **K**), ce qui donne au final :

$$T_{AC} = 10 \mu s + 2 \cdot T_{AD}$$

VIII.4.2. MISE EN OEUVRE DU CAN DANS LE PIC

VIII.4.2.1. SYNOPTIQUE GENERAL D'IMPLANTATION

Son synoptique général est donné sur la figure VIII.55. ci-dessous :

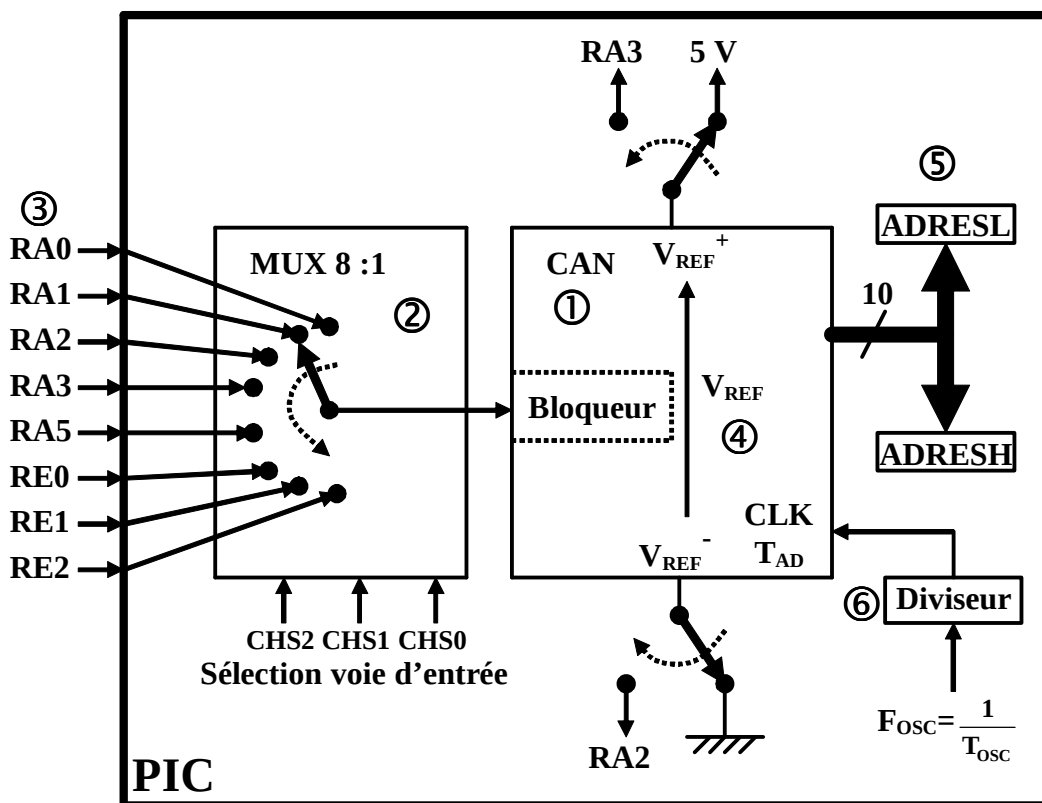


Figure VIII.55.

Si le CAN (à approximations successives cf. bloc ①) possède bien une seule entrée, reliée en interne sur son bloqueur, il est précédé d'un multiplexeur 8 entrées/1 sortie (cf. bloc ② MUX 8:1).

L'intérêt est d'avoir 8 entrées analogiques différentes et indépendantes, appelée chacune **canal de conversion** (cf. ③ broches RA0, RA1, RA2, RA3 et RA5 du PORTA et broches RE0, RE1 et RE2 du PORTE) qui **partageront** le même CAN.

(si on avait utilisé un CAN différent par entrée analogique, il aurait fallu 8 CAN à l'intérieur du PIC, ce qui aurait augmenté sensiblement le coût du microcontrôleur).

Par contre, il sera nécessaire, avant de faire une conversion, de sélectionner la voie d'entrée concernée (via les bits CHS2, CHS1 et CHS0 du registre SFR ADCON0). On dit alors que l'on fait du **multiplexage temporel** (ou « balayage » temporel).

On aperçoit aussi sur ce schéma (cf. ④) la d.d.p. $V_{REF} = V_{REF}^+ - V_{REF}^-$ qui peut valoir soit 5V, soit dépendre des tensions injectées sur les broches RA3 et RA2, suivant la configuration choisie.

Le résultat numérique sur **10 bits** de la conversion analogique/numérique sera quant à lui à récupérer dans les registres ADRESH et ADRESL (cf. ⑤).

VIII.4.2.2. REGISTRES ASSOCIES

Le pilotage du CAN passe par deux registres SFR **ADCON0** et **ADCON1**.

(comme **Analog Digital CON**trol **0** et **1**).

Le format du registre **ADCON0** est fourni sur la figure VIII.56.

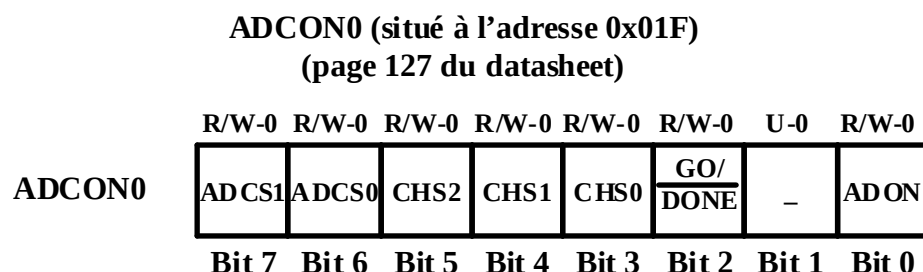


Figure VIII.56.

La signification de chacun de ses bits est fournie sur le tableau de la figure VIII.57.

Bit	Valeur	Signification et commentaires
CHS2 :CHS0		Pour sélectionner le canal de conversion (pilote le MUX 8 :1)
	000	Canal 0 : broche d'entrée RA0/AN0 reliée au CAN
	001	Canal 1 : broche d'entrée RA1/AN1 reliée au CAN
	010	Canal 2 : broche d'entrée RA2/AN2 reliée au CAN
	011	Canal 3 : broche d'entrée RA3/AN3 reliée au CAN
	100	Canal 4 : broche d'entrée RA5/AN4 reliée au CAN
	101	Canal 5 : broche d'entrée RE0/AN5 reliée au CAN
	110	Canal 6 : broche d'entrée RE1/AN6 reliée au CAN
	111	Canal 7 : broche d'entrée RE2/AN7 reliée au CAN
ADCS1 :ADCS0		Choix du diviseur pour fixer T_{AD} (cf bloc ⑥ figure VIII.55. et voir aussi remarque 2 plus loin)
	00	Diviseur = 2 donc T _{AD} = 2.T _{Osc}
	01	Diviseur = 8 donc T _{AD} = 8.T _{Osc}
	10	Diviseur = 32 donc T _{AD} = 32.T _{Osc}
	11	T _{AD} fourni par un oscillateur RC dédié au CAN (en mode SLEEP)
ADON		Permet de mettre en route le CAN
	0	Le CAN est éteint
	1	Met en route le CAN : on peut alors l'utiliser
GO/DONE		Bit de commande et d'état (significatif si bit ADON = 1) (voir aussi remarque 3 plus loin pour utiliser ce bit)
	1	Quand on le met à 1, cela lance une conversion (le faire <u>après</u> avoir mis le bit ADON à 1) et il reste à 1 pendant la conversion.
	0	Mis à 0 automatiquement à la fin de la conversion, il permet d'indiquer que la conversion est finie : il faut alors le lire. (ou pas de conversion en cours...)

Figure VIII.57.

Le format du registre **ADCON1** est quant à lui fourni sur la figure VIII.58.

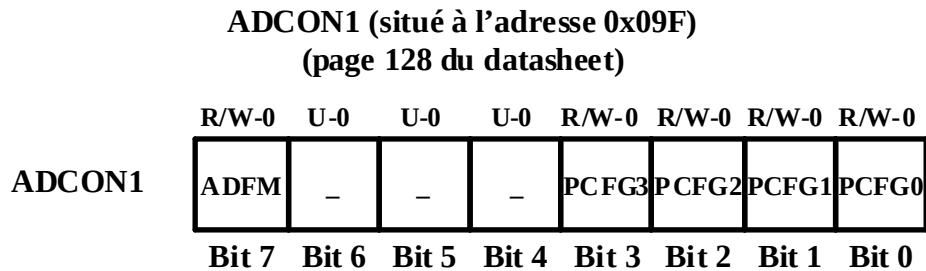


Figure VIII.58.

La signification de chacun de ses bits est fournie sur le tableau de la figure VIII.59.

Bit	Valeur	Signification et commentaires
PCFG3 :PCFG0	Voir Figure VIII.60.	Permet d'affecter les lignes concernées des PORTA et PORTE soit comme E/S TOR, soit comme entrées analogiques (cf figure VIII.15. déjà vue ou repris sur la figure VIII.60.)
ADFM		Format du résultat numérique de la conversion
	0	Justification à gauche (voir remarque 1 plus loin)
	1	Justification à droite (voir remarque 1 plus loin)

Figure VIII.59.

Précisions (cf. figure VIII.60.) sur les différentes valeurs prises par les bits **PCFG3**, **PCFG2**, **PCFG1** et **PCFG0**. Noter également la gestion de la tension **V_{REF}** :

- soit via **VDD=5V** et **VSS=0V**,
- soit fixée **extérieurement** par les tensions envoyées sur les broches **RA3** et **RA2**.
(voir **remarque 4** plus loin pour comprendre l'utilité de cette possibilité)

PCFG3: PCFG0	AN7 ⁽¹⁾ RE2	AN6 ⁽¹⁾ RE1	AN5 ⁽¹⁾ RE0	AN4 RA5	AN3 RA3	AN2 RA2	AN1 RA1	AN0 RA0	VREF+	VREF-	CHAN/ Refs ⁽²⁾
0000	A	A	A	A	A	A	A	A	VDD	VSS	8/0
0001	A	A	A	A	VREF+	A	A	A	RA3	VSS	7/1
0010	D	D	D	A	A	A	A	A	VDD	VSS	5/0
0011	D	D	D	A	VREF+	A	A	A	RA3	VSS	4/1
0100	D	D	D	D	A	D	A	A	VDD	VSS	3/0
0101	D	D	D	D	VREF+	D	A	A	RA3	VSS	2/1
011x	D	D	D	D	D	D	D	D	VDD	VSS	0/0
1000	A	A	A	A	VREF+	VREF-	A	A	RA3	RA2	6/2
1001	D	D	A	A	A	A	A	A	VDD	VSS	6/0
1010	D	D	A	A	VREF+	A	A	A	RA3	VSS	5/1
1011	D	D	A	A	VREF+	VREF-	A	A	RA3	RA2	4/2
1100	D	D	D	A	VREF+	VREF-	A	A	RA3	RA2	3/2
1101	D	D	D	D	VREF+	VREF-	A	A	RA3	RA2	2/2
1110	D	D	D	D	D	D	D	A	VDD	VSS	1/0
1111	D	D	D	D	VREF+	VREF-	D	A	RA3	RA2	1/2

A = Analog input D = Digital I/O

Figure VIII.60.

Quelques remarques et précisions sont nécessaires pour bien comprendre toutes ces informations :

Remarque 1 :

Comme on l'avait déjà vu sur la figure VIII.55. (cf. ⑤), le résultat de la conversion analogique/numérique est placé dans les registres SFR **ADRESH** (comme **H**igh) et **ADRESL** (comme **L**ow), chacun de taille 8 bits, ce qui est suffisant pour les 10 bits du résultat.

Il existe deux formats pour stocker les 10 bits D9 à D0 du résultat, formats qui sont résumés sur la figure VIII.61.

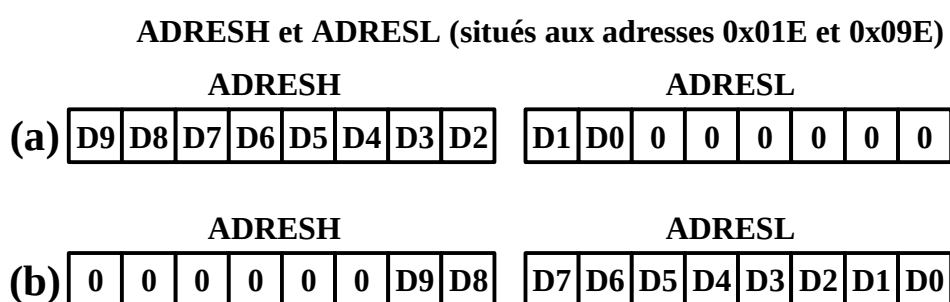


Figure VIII.61.

- la justification à **gauche** (**ADFM=0** et figure VIII.61.a.) : les 10 bits du résultat sont « rangés » dans **ADRESH/ADRESL**, en se « calant » à partir de la gauche de **ADRESH**. Du coup, les 6 bits de poids **faibles** de **ADRESL** ne sont pas significatifs (ils sont mis à 0 par le CAN).

Intérêt : en ne lisant que **ADRESH**, on ne récupère que les 8 bits de poids forts du résultat, ce qui revient à faire comme si on avait un CAN sur 8 bits (dans certaines applications, il n'est en effet pas nécessaire d'avoir la précision maximum fournie par 10 bits). Dans ce cas là, le résultat de la conversion sera déclaré comme un **unsigned char** (taille un octet).

- la justification à **droite** (**ADFM=1** et figure VIII.61.b) : les 10 bits du résultat sont « rangés » dans **ADRESH/ADRESL**, en se « calant » à partir de la droite de **ADRESL**. Les 6 bits de poids **forts** de **ADRESH** sont mis à 0 par le CAN.

Intérêt : on obtient la précision maximum sur 10 bits du CAN. Il faut alors déclarer le résultat comme un **unsigned short int** (taille deux octets).

Si le CAN n'est pas utilisé, **ADRESH** et **ADRESL** deviennent alors des GPR et peuvent être utilisés tels quels.

Remarque 2 :

Le datasheet indique que pour que le CAN fonctionne correctement, il faut que $T_{AD} \geq 1,6 \mu s$. (on rappelle que T_{AD} est le temps nécessaire pour déterminer chaque bit par approximations successives). En pratique, il faudra choisir la plus petite valeur de T_{AD} supérieure à $1,6 \mu s$, pour avoir le temps de conversion le plus petit possible.

Calculons les valeurs de T_{AD} , suivant :

- le choix du diviseur (cf. bits **ADCS1** et **ADCS0** du registre **ADCON0**),
- la fréquence F_{OSC} du quartz (ou sa période T_{OSC}) : pour **4 MHz** et **20 MHz**.

Les résultats sont résumés sur le tableau de la figure VIII.62.

Diviseur \ $F_{OSC}(T_{OSC})$	4 MHz (0,25 μs)	20 MHz (0,05 μs)
2 (ADSC1 : ADCS0 = 00)	0,5 μs	0,1 μs
8 (ADSC1 : ADCS0 = 01)	2,0 μs	0,4 μs
32 (ADSC1 : ADCS0 = 10)	8,0 μs	1,6 μs

Figure VIII.62.

Les valeurs barrées ne seront donc pas possibles, car inférieures à $1,6 \mu s$.

Avec un quartz de 4 MHz, et un diviseur par 8, on obtient : $T_{AD} = 2 \mu s$.
(on a une valeur 25% plus grande que $1,6 \mu s$, ce qui n'est pas optimum, mais avec une quartz à 4 MHz, on ne peut pas se rapprocher mieux de la valeur de $1,6 \mu s$, tout en y étant supérieur !!!)

On peut maintenant évaluer le temps complet T pris par une conversion :

- phase d'acquisition T_{AC} : on avait calculé $T_{AC} = 10 \mu s + 2.T_{AD} = 14 \mu s$.
- phase de conversion $T_C = 12.T_{AD} = 12.2 = 24 \mu s$.

On aura donc au total : $T = T_{AC} + T_C = 38 \mu s$.

Cette contrainte implique que la période d'échantillonnage T_e doit être supérieure à $38 \mu s$.

On peut par exemple prendre $T_{emin} = 40 \mu s$.

Ce qui implique que la fréquence d'échantillonnage $F_e = \frac{1}{T_e}$ doit être inférieure à environ

$$F_{emax} = \frac{1}{T_{emin}} = 25 \text{ kHz.}$$

Et donc, d'après le **théorème de Shanon**, ($F_e \geq 2.F_{max}$), on pourra traiter numériquement des signaux avec ce PIC si leur fréquence maximum F_{max} est plus petite que 12 à 12,5 kHz.

Remarque 3 :

Pour gérer la fin d'une conversion, il existe deux procédés :

① faire du polling (scrutation) sur le bit $\overline{\text{GO/DONE}}$ du registre **ADCON0** :
après avoir lancé une conversion en forçant ce bit à 1, il suffit d'attendre qu'il passe à 0, ce qui indiquera que la conversion est terminée et qu'on peut lire le résultat de la conversion dans les registres **ADRESH** et **ADRESL**.

Ce qui correspond à l'organigramme de la figure VIII.63. ci-dessous :

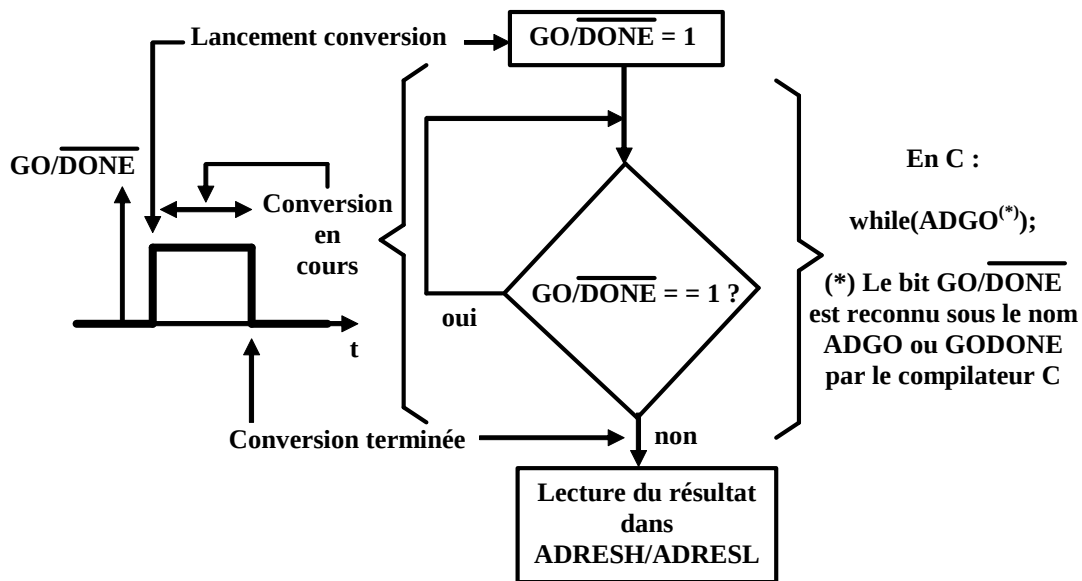


Figure VIII.63.

Remarque : si, pendant que la conversion est en cours, on fait passer le bit $\overline{\text{GO/DONE}}$ (reconnu sous le nom **ADGO** ou **GODONE** par le compilateur C) à 0, on **arrête** la conversion mais les registres **ADRESH/ADRESL** ne seront pas mis à jour avec la valeur partielle du résultat (mais contiendront la valeur de la dernière conversion qui s'est entièrement terminée).

② mais on peut aussi travailler sous interruption, car quand la conversion est finie, le flag **ADIF** du registre SFR **PIR1** passe à 1 pour indiquer que la conversion est finie, et une interruption sera générée si le bit **ADIE** du registre SFR **PIE1** a été mis à 1.

Il faudra alors penser à :

- paramétrer le CAN, le mettre en route et mettre le flag **ADIF** à 0,
- gérer les différents masques d'interruptions : local (**ADIE**), semi global (**PEIE**) et global (**GIE**),
- faire une première acquisition suivie du lancement d'une première conversion, pour initialiser la première IT,
- et dans la routine d'IT :
 - lire la valeur numérique fournie par le CAN,
 - acquitter l'interruption via le bit **ADIF**,
 - relancer l'acquisition suivante,
 - et enfin relancer la conversion suivante.

Tout ceci sera montré sur un exemple complet, détaillé plus loin sur la figure VIII.64.

Remarque 4 :

On a vu que la plage de conversion (intervalle où doit se trouver la tension analogique $V_a(t)$ pour pouvoir être correctement convertie) est fixée par :

$$V_{REF}^{+} \leq V_a(t) \leq V_{REF}^{-}$$

Sa largeur est donc : $V_{REF} = V_{REF}^{+} - V_{REF}^{-}$

La précision, on l'a vu, est donnée par : $\frac{q}{2} = \frac{V_{REF}}{2^{n+1}}$ où n est le nombre de bits en sortie du CAN.

Lorsque l'on souhaite que la plage de conversion soit la plus grande possible (pour avoir la plus grande excursion possible en entrée, et donc la dynamique la plus importante), on prendra :

$$V_{REF} = VDD = 5V \quad \text{et donc :} \quad \begin{aligned} V_{REF}^{+} &= VDD = 5 \text{ V (tension d'alimentation du PIC)} \\ V_{REF}^{-} &= VSS = 0 \text{ V (masse du PIC).} \end{aligned}$$

Mais dans certaines applications, où la tension d'entrée $V_a(t)$ varie entre V_{max} et V_{min} (ces limites étant en général imposées par le type de capteur utilisé pour générer $V_a(t)$), il n'est pas nécessaire d'avoir une excursion de 5 V, mais il est suffisant de prendre alors :

$$V_{REF} = V_{max} - V_{min} \quad \text{et donc :} \quad \begin{aligned} V_{REF}^{+} &= V_{max} : \text{à relier à la broche } \mathbf{RA3} \text{ (cf. figure VIII.55.)} \\ V_{REF}^{-} &= V_{min} : \text{à relier à la broche } \mathbf{RA2} \text{ (cf. figure VIII.55.)} \end{aligned}$$

En adaptant ainsi la plage de conversion aux caractéristiques du capteur, on augmente la précision de la mesure.

VIII.4.2.3. SEQUENCEMENT D'UNE CONVERSION

Finalement, pour réaliser une conversion, voilà les différents points à prendre en compte :

- phase d'initialisation :

- gérer les bits **PCFG3 :PCFG0** de **ADCON1** :
pour configurer les broches du PIC concernées : mettre les entrées analogiques en entrées (via **TRISA** et **TRISE**). Au reset, elles le sont d'ailleurs systématiquement.
et pour choisir la plage de conversion via les broches **RA3** et **RA2**.
- gérer le bit **ADFM** de **ADCON1** :
pour gérer le format du résultat.
- gérer les bits **ADCS1 :ADSC0** de **ADCON0** :
pour fixer une valeur de $T_{AD} \geq 1,6 \mu s$ (en fonction du quartz utilisé).
- gérer les bits **CHS2 :CHS0** de **ADCON0** :
pour sélectionner le canal concerné par la conversion,
noter qu'il faut prévoir une acquisition entre chaque changement de canal.
- gérer le bit **ADON** de **ADCON0** :
pour mettre en route le CAN.

- phase de conversion (en technique de polling) :

- attendre le temps d'acquisition avant toute nouvelle conversion
- lancer une conversion via le bit **GO/DONE** de **ADCON0**
- attendre la fin de la conversion par scrutation de ce bit **GO/DONE**
- lire le résultat de la conversion dans les registres **ADRESH/ADRESL**
(ou **ADRESH** seulement).

Il est **fortement** conseillé, pour la structuration du programme, d'écrire chacune de ces 2 phases sous la forme d'un module (si on utilise l'assembleur) ou d'une fonction (si on programme en C).

VIII.4.2.4. EXEMPLE DE MISE EN OEUVRE

En TP, nous utiliserons le CAN en utilisant la technique du pooling.

Ici, nous allons mettre en œuvre le CAN en utilisant une interruption sur fin de conversion.

Le programme est fourni sur la figure VIII.64.

On suppose que :

- la tension analogique est envoyée sur la broche **RE0** du PIC,
- la plage de conversion du CAN est de 5V,
- le résultat en numérique sera justifié à gauche (précision équivalente à un CAN 8 bits),
- quatre leds sont branchées sur les lignes **RB3** à **RB0** du PORTB.

Le but est de réaliser un petit vumètre à leds, dont le principe est rappelé ci-dessous :

**Si la tension analogique sur la broche RE0 est au minimum, aucune led n'est allumée.
Au fur et à mesure que cette tension analogique augmente, on allume les leds une à une.
Lorsque cette tension analogique est au maximum, toutes les leds sont allumées.
Et enfin, si cette tension analogique rediminue, on éteint les leds une à une.**

Remarquez bien que, à l'intérieur de la boucle sans fin while(1) du main, la variable globale k n'est pas modifiée : c'est la routine d'IT, exécutée à chaque fin de conversion qui vient mettre à jour cette variable globale en lisant le registre **ADRESH**...

Si on interdit les IT (en faisant **GIE=0** ou **PEIE=0** ou **ADIE=0**), votre vumètre ne fonctionnera plus, car la variable k restera à 0 !!!

Lisez bien les commentaires pour bien comprendre tous ces mécanismes et faire les liens avec tout ce qu'on vient de voir sur le fonctionnement du CAN dans ce chapitre.

```

unsigned char k;                // variable globale sur un octet

void delay_acq(void)            // dure 14 microsecondes: testé sur le simulateur MPLAB SIM
{
    #asm                        // début de l'assembleur en ligne
        nop
        nop
        nop
        nop
        nop
        nop
        nop
        nop
    #endasm                    // fin de l'assembleur en ligne
}

void interrupt conversion_it (void) // routine d'IT
{
    if(ADIF && ADIE)            // IT sur fin de conversion
    {
        ADIF=0;                // acquittement IT fin de conversion
        k=ADRESH;               // lecture sur 8 bits du résultat
        delay_acq();            // acquisition suivante
        ADGO=1;                // lancement conversion suivante
    }
}

void init_can(void)
{
    TRISE0 = 1;                // RE0 en entrée
    // registre ADCON0
    // ADCS1:ADCS0 = 01         // pour TAD =2 microsecondes
    // CHS2:CHS0 = 101         // pour MUX sur RE0 (canal 5)
    // GO/DONE = 0              // pas de lancement de conversion encore
    // ADON = 1                 // pour allumer le CAN
    ADCON0=0b01101001;
    // registre ADCON1
    // ADFM = 0                 // justification à gauche
    // PCFG3:PCFG0 = 1001      // RE0 entrée analogique et VREF=5V
    ADCON1=0b00001001;
}

void init_port(void)
{
    TRISB=TRISB&(0xF0);        // RB3 à RB0 en sortie pour les 4 leds
}

```

Figure VIII.64.

```

void main(void)
{
    init_port();           // 4 leds sur RB3 à RB0
    init_can();            // initialise le CAN
    ADIF=0;                // efface une éventuelle cause antérieure d'IT due au CAN
    ADIE=1;                // démasquage local IT fin de conversion du CAN
    PEIE=1;                // démasquage semi global
    GIE=1;                 // démasquage de toutes les IT
    delay_acq();           // prépare la première acquisition
    ADGO=1;                // lancement de la première conversion

    while(1)
    {
        if (k<32)          PORTB=0x00;        // vumètre avec aucune led allumée
        if((k>=32) && (k<96)) PORTB=0x01;      // vumetre avec 1 seule led allumée
        if((k>=96) && (k<160)) PORTB=0x03;     // vumetre avec 2 leds allumées
        if((k>=160) && (k<224)) PORTB=0x07;    // vumetre avec 3 leds allumées
        if((k>=224) && (k<=255)) PORTB=0x0F;   // vumetre avec les 4 leds allumées
    }
}

```

Figure VIII.64. (suite et fin)

Pour conclure, et en résumé, voici sur la figure VIII.65. tous les registres liés au Convertisseur Analogique/Numérique ainsi que les bits concernés.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR, BOR	Value on MCLR, WDT
0Bh,8Bh,10Bh,18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
1Eh	ADRESH	A/D Result Register High Byte								xxxx xxxx	uuuu uuuu
9Eh	ADRESL	A/D Result Register Low Byte								xxxx xxxx	uuuu uuuu
1Fh	ADCON0	ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/DONE	—	ADON	0000 00-0	0000 00-0
9Fh	ADCON1	ADFM	—	—	—	PCFG3	PCFG2	PCFG1	PCFG0	--0- 0000	--0- 0000
85h	TRISA	—	—	PORTA Data Direction Register						--11 1111	--11 1111
05h	PORTA	—	—	PORTA Data Latch when written: PORTA pins when read						--0x 0000	--0u 0000
89h ⁽¹⁾	TRISE	IBF	OBF	IBOV	PSPMODE	—	PORTE Data Direction bits			0000 -111	0000 -111
09h ⁽¹⁾	PORTE	—	—	—	—	—	RE2	RE1	RE0	---- -xxx	---- -uuu

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'. Shaded cells are not used for A/D conversion.

Figure VIII.65.

VIII.5. LES TIMERS INTERNES

(cf. ⑨ du schéma de la **figure II.4.** et cf. **pages 53 à 62** du datasheet)

Nous allons commencer par une présentation générale de la fonction timer qu'il est conseillé de lire, surtout si on n'est pas familier avec ces procédés.

Puis dans un deuxième temps, on passera à des éléments plus spécifiques en regardant notamment comment cette fonction est (« triplement ») implantée dans le PIC.

VIII.5.1. GENERALITES SUR LA FONCTION TIMER

Nous avons déjà effectué des temporisations logicielles sans utiliser la fonction timer proprement dite (cf. **chapitre VI.1.**). Le principal inconvénient de cette méthode, outre son manque de précision, est de monopoliser la CPU du PIC pour compter le temps qui s'écoule, et donc de ne pas pouvoir faire autre chose pendant ce temps...

La fonction **timer** permet justement d'assurer cette tâche de façon **plus précise**, et dans certain cas de libérer la CPU du PIC pour faire autre chose en parallèle...

Cette fonction timer (ou comptage du temps qui s'écoule) est donc une fonction très importante et nécessaire dans tout système à base de micro/processeur/contrôleur.

Elle fournit une référence temporelle (le plus souvent à partir du quartz du système) indispensable dans un certain nombre d'applications :

- gérer des temporisations et/ou des délais très précisément,
- connaître un instant de façon exacte par exemple pour « dater » un évènement extérieur,
- mesurer des intervalles de temps comme avec un chronomètre,
- générer un front de tension en sortie à un instant précis pour piloter un périphérique extérieur,
- générer des signaux (TTL) de caractéristiques temporelles données : signaux carrés, signaux de rapport cyclique donné ou tout autre type de signal, plus ou moins complexe,
- fabriquer des « chiens de garde » ou « watch-dog » (cf. **figure IV.43.** du **chapitre IV.3.5.**)

On le voit, les exemples sont nombreux... et j'en ai certainement oubliés !!!

Le **cœur du timer** est un registre de type **compteur** qui peut être de taille 8 bits ou 16 bits et qui **s'incrémente en permanence** (ou dans d'autres types de microcontrôleurs se décrémente) à chaque impulsion de son horloge d'entrée, appelé aussi **tick** d'horloge.

Dans cette présentation générale, nous appellerons **TIMER** ce registre compteur.

La période T_{tick} de ces ticks d'horloge à l'entrée du registre **TIMER** est aussi appelée parfois la **granularité** du timer. Du coup, **attention, à ne pas confondre T_{tick} et T_{OSC}** (période du quartz).

En effet, cette horloge d'entrée du compteur (le tick) est souvent fabriquée à partir du quartz du PIC, grâce à une chaîne en cascade de deux prédiviseurs :

- un pré diviseur de valeur **fixe f** (valeur fixée par **construction et non modifiable**),
- un pré diviseur de valeur **programmable p** (valeur choisie par **programmation**),

comme indiqué sur la figure VIII.66.

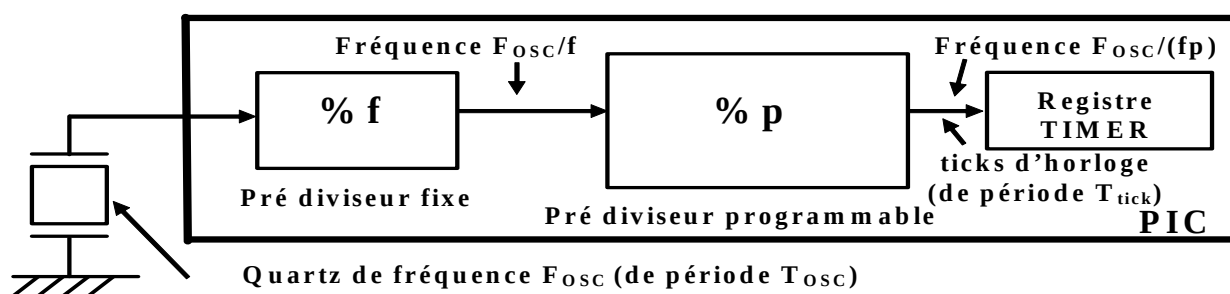


Figure VIII.66.

Trois remarques et précisions avant de continuer :

1. Cette chaîne de pré diviseurs permet de diminuer la fréquence d'entrée du quartz (en général plutôt élevée) pour obtenir des fréquences de comptage plus faibles, et donc pour fabriquer des délais de toute type de valeur, et **même des valeurs assez grandes** avec des registres compteur de taille 8 ou 16 bits.

2. Pour les PIC, le pré diviseur **fixe f** vaut **4**. En sortie du pré diviseur, on aura donc un signal dont la période correspond à la durée d'une instruction. Du coup, on a :

$$T_{\text{tick}} = f.p.T_{\text{OSC}} = 4.p.T_{\text{OSC}}$$

3. Dans certains types de fonctionnement, on peut aussi utiliser **une autre horloge** de base que le quartz du PIC. On en verra tout l'intérêt un peu plus loin.

On peut aussi utiliser la sortie d'un capteur d'objets ou d'évènements extérieurs qu'on injecte alors directement à l'entrée du registre compteur en lieu et place du signal « ticks d'horloge » de la figure VIII.66. : le timer se transforme alors **en compteur d'objets ou d'évènements extérieurs**.

Dans le cas du PIC, on trouvera deux types de registre compteur :

- les compteurs à valeur initiale **VI nulle** et à valeur finale **VF programmable** :

ils comptent donc de **VI = 0** jusqu'à **VF**, et positionnent un flag qu'on appellera dans cette présentation générale **TIMER_IF à 1 dès que la valeur VF est atteinte** (ou plus exactement quand le compteur passe de **VF** à **VF+1**) comme indiqué sur la figure VIII.67.

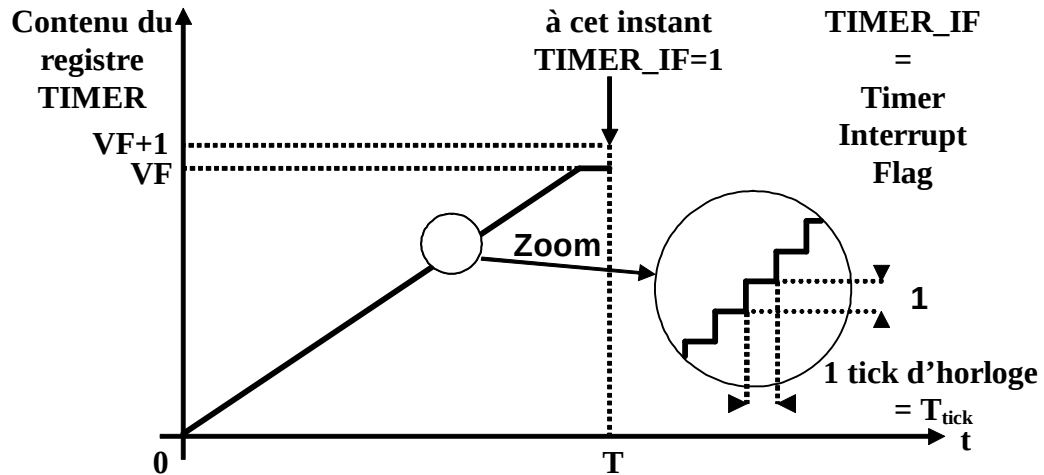


Figure VIII.67.

La durée **T** mesurée par ce type de timer, prenant en compte les deux pré diviseurs, est donc donnée par :

(1)
$$T = f.p.(VF+1).T_{OSC}$$
 en appelant $T_{OSC} = \frac{1}{F_{OSC}}$ = période du quartz du PIC.

Exemple :

On a vu que pour les PIC on avait **f = 4**. Supposons qu'on choisisse par programmation (plus exactement par initialisation du timer) : **p = 16** et **VF = 0xFF = 255** (en supposant de plus dans cet exemple que le registre compteur est sur 8 bits), on obtient, avec un quartz de **F_{OSC} = 4 MHz** soit **T_{OSC} = 250 ns** :

$$T = T_{max} = 4.16.256.250 \text{ ns} = 4,096 \text{ ms}$$

C'est la plus grande valeur de temporisation qu'on puisse avoir pour cette valeur de **p**.

Réciproquement, l'équation (1) nous permettra de trouver les valeurs de **p** et de **VF** en fonction de la valeur de **T** que l'on souhaite obtenir. Comme nous avons deux inconnues **p** et **VF**, plusieurs solutions seront possibles, et parmi celles-ci, certaines ne seront pas implantables (toutes les valeurs de **p** ne sont pas disponibles) et d'autres seront plus précises (en général celles qui correspondent à la **plus petite** valeur implantable de **p** car dans ce cas là, le tick d'horloge sera le plus petit). On dit que l'on améliore ainsi la granularité temporelle, et donc sa précision...

- les compteurs à valeur initiale **VI programmable** et à valeur finale **VF = 0xFF** (s'ils ont pour taille un octet) ou **VF = 0xFFFF** (s'ils ont pour taille deux octets) :

ils comptent donc de **VI** jusqu'à **VF = 0xFF** (ou **0xFFFF**), et positionnent un flag qu'on appellera dans cette présentation générale **TIMER_IF** à **1** dès que la valeur 0x00 (ou 0x0000) est atteinte, comme indiqué sur la figure VIII.68, c'est-à-dire 1 tick d'horloge après avoir atteint la valeur 0xFF (ou 0xFFFF). On parle alors de **débordement** (ou **overflow**) du timer.

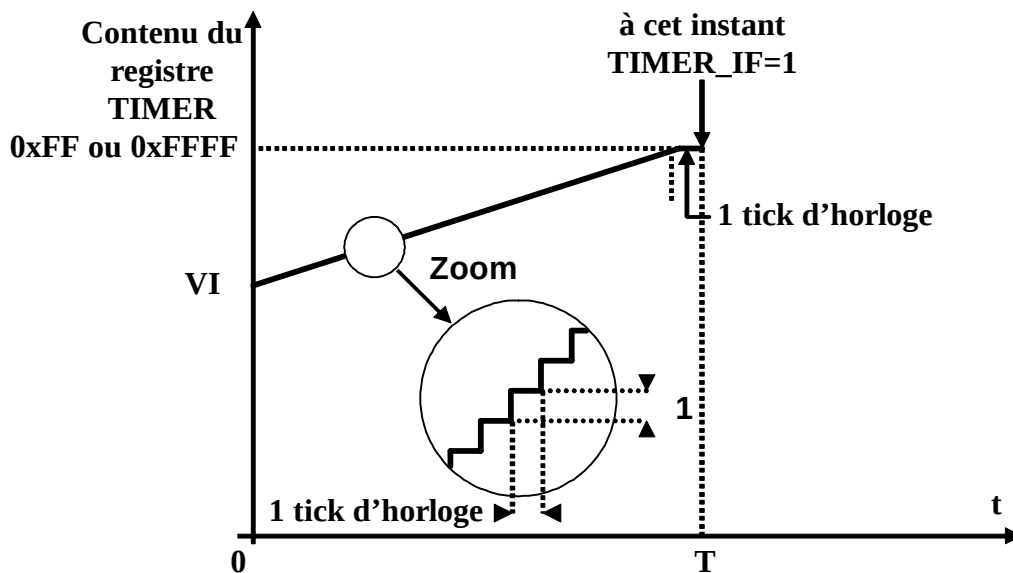


Figure VIII.68.

La durée **T** mesurée par ce type de timer, prenant en compte les deux pré diviseurs, est donc donnée par :

(2) $\boxed{T = f.p.(2^n - VI).T_{OSC}}$ en appelant : $T_{OSC} = \frac{1}{F_{OSC}}$ = période du quartz du PIC.
 n = nombre de bits du registre compteur (8 ou 16)

Exemple :

On a vu que pour les PIC on avait **f = 4**.

Supposons qu'on choisisse par programmation (plus exactement par initialisation du timer) :

p = 8 et **VI = 0x0000** (en supposant dans cet exemple que le registre compteur est sur 16 bits), on obtient, avec un quartz de **F_{OSC} = 4 MHz** soit **T_{OSC} = 250 ns** :

$$T = T_{max} = 4.8.65536.250 \text{ ns} = 524,288 \text{ ms}$$

Réciproquement, l'équation (2) nous permettra de trouver les valeurs de **p** et de **VI** en fonction de la valeur de **T** que l'on souhaite obtenir. Comme nous avons deux inconnues **p** et **VI** (n est fixé par construction), plusieurs solutions seront possibles, et parmi celles-ci certaines ne seront pas implantables (toutes les valeurs de **p** ne sont pas disponibles) et d'autres seront plus précises (en général celles qui correspondent à la plus petite valeur implantable de **p** car dans ce cas la le tick d'horloge sera le plus petit).

Comment gérer ce flag **TIMER_IF** ?

On connaît maintenant la réponse : soit par pooling, soit par interruption.

① technique du pooling :

Après avoir fait toutes les initialisations nécessaires (notamment sans oublier de mettre le flag **TIMER_IF** à 0) on lance le timer et on attend que ce flag passe à 1, ce qui est le signe, comme on l'a vu sur les figures VIII.67. et VIII.68. que la durée T programmée s'est écoulée.

Pour cette attente, il faudra écrire en C une ligne du type :

```
while ( TIMER_IF == 0 ) ;           // tant que TIMER_IF = 0, on attend
ou
while( !TIMER_IF ) ;               // idem que la ligne précédente
```

Finalement, ce n'est pas un gain important par rapport à la solution des temporisations logicielles : au lieu de « perdre » du temps en exécutant des boucles de temporisations calibrées, ici on perd du temps « à attendre » que le flag **TIMER_IF** passe à 1.

Dans les deux cas, on ne fait rien en parallèle.

Le seul avantage d'utiliser la fonction timer avec la technique du pooling est de pouvoir plus facilement, plus rapidement et de façon plus précise réaliser les temporisations.

② technique par interruption :

Il existe aussi un flag qu'on appellera dans cette présentation générale **TIMER_IE** (comme **TIMER Interrupt Enable**).

On l'a vu, quand la durée souhaitée s'est écoulée, le flag **TIMER_IF** passe à 1 et une interruption sera générée si ce flag **TIMER_IE** a été mis à 1.

Il faudra alors penser à :

- initialiser correctement le pré diviseur programmable et éventuellement écrire une valeur initiale **VI** (ou une valeur finale **VF**) dans le registre compteur du timer,
- penser à mettre en route le timer (si c'est nécessaire) et à mettre le flag **TIMER_IF** à 0,
- gérer les différents masques d'interruptions : local (**TIMER_IE**), semi global (**PEIE**) et global (**GIE**),
- et dans la routine d'IT :
 - acquitter l'interruption via le bit **TIMER_IF** (en le remettant à 0),
 - éventuellement écrire une valeur initiale **VI** (ou une valeur finale **VF**) dans le registre compteur du timer,
 - effectuer le traitement associé au fait que la durée T programmée s'est écoulée.

Tout ceci sera montré et mis en pratique plus loin sur les trois exemples de mise en œuvre des figures VIII.75., VIII.82 et VIII.91.

VIII.5.2. LES TIMERS INTERNES DU PIC

Dans le PIC 16F877, il existe 3 timers **indépendants** appelés :

TIMER0, TIMER1 et TIMER2.

Chacun possède :

- son propre pré diviseur **fixe**, de valeur $f = 4$,
- son propre pré diviseur **programmable**,
- son propre registre **compteur**.

Nous allons maintenant donner les spécificités de chacun des ces timers en précisant également les registres SFR qui y sont associés.

VIII.5.2.1. LE TIMER0

VIII.5.2.1.1. PRINCIPALES CARACTERISTIQUES

Elles sont résumées ci-dessous :

- **8** valeurs possibles du pré-diviseur **p** programmable : **2, 4, 8, 16, 32, 64, 128 et 256**,
- registre compteur **TMR0** sur **8 bits** (aux adresses **0x001** et **0x101**), accessible en lecture ou en écriture,
- fonctionnement de **VI** (valeur initiale à charger dans le registre compteur **TMR0**) à **0xFF**,
- le débordement est signalé par le flag **T0IF** (Timer **0** Interrupt Flag, situé dans le registre SFR **INTCON**) qui passe à 1 quand le registre **TMR0** passe de 0xFF à 0x00 (cf. figure VIII.68.),
- un bit **T0IE** (comme Timer **0** Interrupt Enable également situé dans le registre SFR **INTCON**) permettra, localement, **d'autoriser** (si **T0IE=1**) ou **d'interdire** (si **T0IE=0**) la génération d'une interruption lorsque le flag **T0IF** passe à 1,
- le **TIMER0** fonctionne en continu (pas de bit pour le mettre en route).

VIII.5.2.1.2. SYNOPTIQUE

Pour comprendre l'implantation du **TIMER0** dans le PIC, on pourrait se référer à son schéma issu **page 53** du datasheet constructeur et fourni sur la figure VIII.69. ci-dessous :

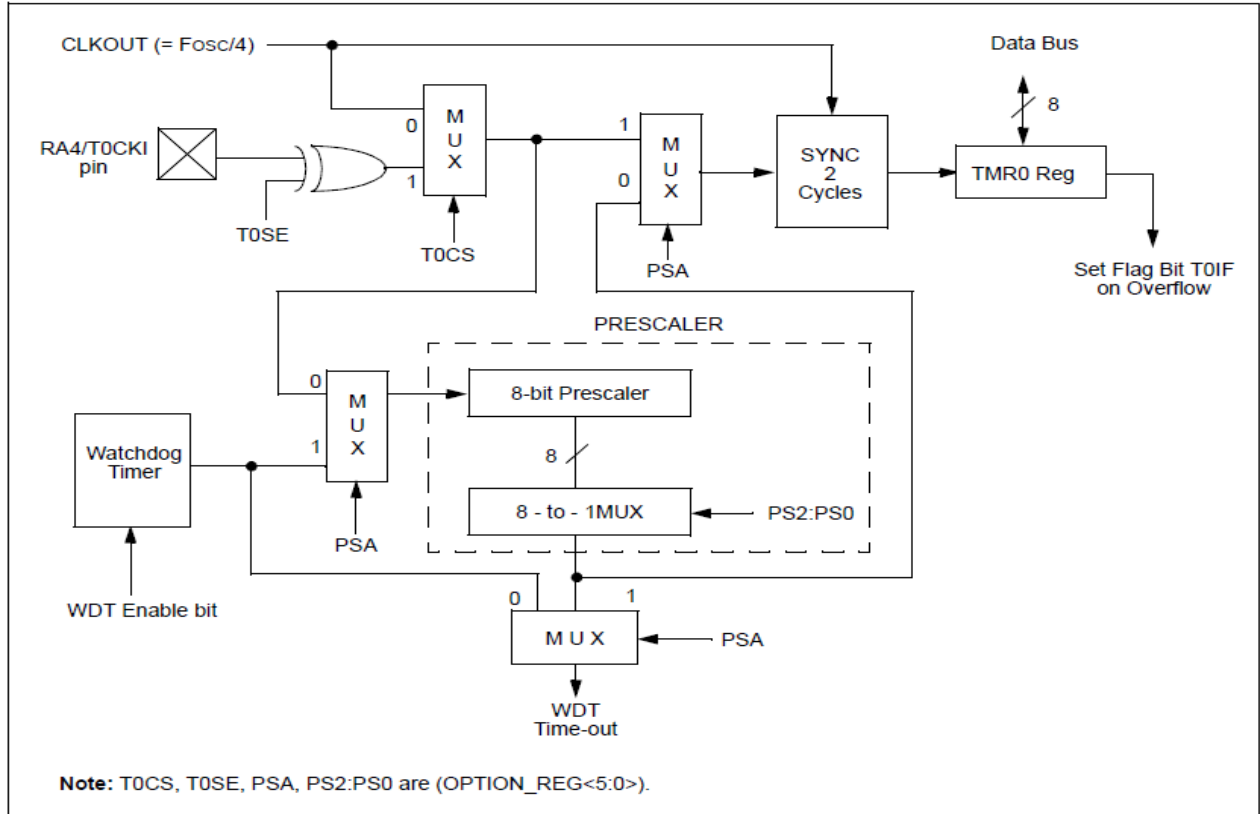


Figure VIII.69.

Ce schéma est un peu compliqué, et je préfère, celui plus synthétique, de la figure VIII.70.

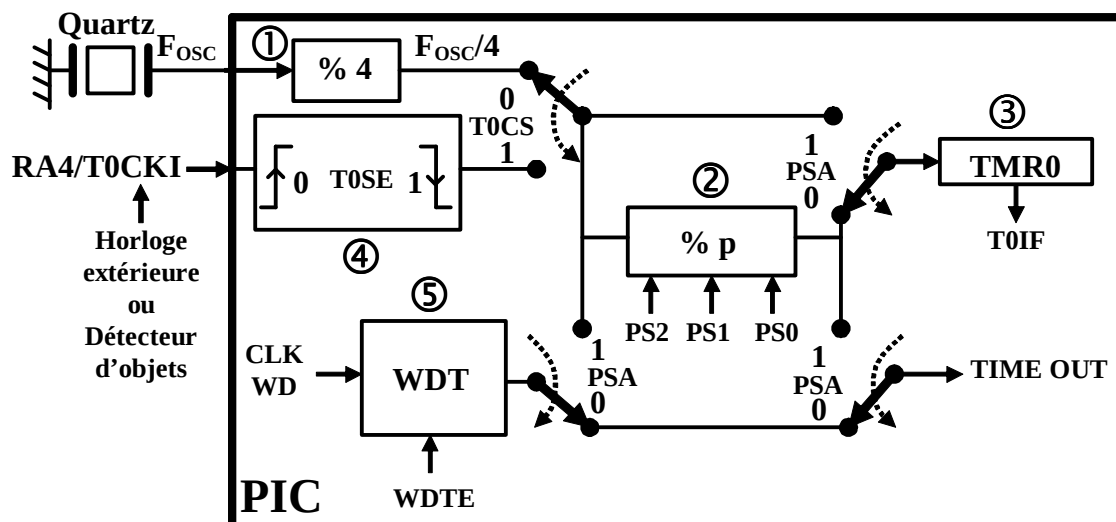


Figure VIII.70.

On y retrouve les blocs qu'on avait présentés dans le chapitre sur les généralités :

- bloc ① : le pré diviseur **fixe** par $f = 4$,
- bloc ② : le pré diviseur **programmable** par **p**, choisi par les 3 bits **PS2**, **PS1** et **PS0** (voir figure VIII.72. plus loin pour plus de détails),
- bloc ③ : le registre compteur **TMR0**, avec son bit d'overflow **T0IF** qui passera à 1 quand le contenu de **TMR0** passe de 0xFF à 0x00.

Quatre multiplexeurs (3 pilotés par le bit **PSA**, et 1 piloté par le bit **T0CS**) permettent de choisir les différents modes de fonctionnement, comme détaillés ci-dessous :

- si **T0CS = 1**, le **TIMER0** peut compter les impulsions d'une horloge extérieure ou des événements extérieurs (détecteur d'objets par exemple) signalés sur la broche **RA4/T0CKI** (comme **Timer 0 Clock Input**) du **PORTA** (**qui doit être programmée en entrée**).

Le registre **TMR0** s'incrémente alors (cf. bloc ④) :

- soit sur un front **montant** du signal sur **RA4**, si le bit **T0SE = 0**,
- soit sur un front **descendant** du signal sur **RA4**, si le bit **T0SE = 1**.

Dans le mode compteur d'événements, on prendra alors **PSA = 1**, pour que le compteur **TMR0** s'incrémente à **chaque front actif**, signalant l'arrivée d'un nouvel événement.

- si **T0CS = 0**, le **TIMER0** fonctionne alors dans sa fonction de base : compter le temps en comptant les impulsions du quartz du PIC. On voit d'ailleurs que dans ce cas là, le bit **T0SE** n'a plus d'influence, puisque la fonction comptage d'impulsions d'horloge ou d'événements extérieurs est inhibée.

Par contre, le bit **PSA** permet d'insérer (si **PSA = 0**) ou pas (si **PSA = 1**) le pré diviseur programmable, ce qui aura bien sûr des conséquences sur la durée **T** mesurée par le **TIMER0**. (cf. formule (2) précédente).

On remarque enfin le bloc ⑤, qui permet de réaliser la fonction « **watch-dog** » dont on a déjà parlé au chapitre IV.3.5.). Le but de cette fonction est de générer un signal de **TIME OUT** (qui est prévu pour provoquer un **RESET** du PIC).

Une horloge interne et spécifique au Watch Dog (**CLK WD**) est comptée par le bloc **WDT**. En principe, la durée de base du **TIME_OUT** est d'environ **18 ms**, mais on peut la multiplier par **p** en insérant le pré diviseur programmable (**PSA=1**) dans la chaîne du Watch Dog. Notons que dans ce cas là, le pré diviseur ne peut plus être affecté au **TIMER0**.

Le pré diviseur programmable sera donc affecté soit au **TIMER0**, soit à la fonction **Watch Dog**, le choix étant fait par le bit **PSA**.

Pour gérer la fonction Watch Dog, il y a deux possibilités :

- soit on la désactive totalement grâce au bit **WDTE** du registre de **configuration** (cf. figures III.17. et III.18.) en faisant **WDTE = 0**,
- soit, si **WDTE = 1**, on remet périodiquement le compteur **WDT** à **0** (par l'instruction **CLRWDT**) pour éviter que le signal de **TIME OUT** provoque le **RESET** du PIC (revoir ce principe détaillé sur la figure IV.43.).

VIII.5.2.1.3. REGISTRE DE CONTRÔLE ASSOCIE

La plupart des bits de configuration que l'on vient de présenter se trouvent dans le registre SFR **OPTION_REG**, dont le format est fourni sur la figure VIII.71.

OPTION_REG REGISTER (situé aux adresses 0x081 et 0x181) (pages 23 et 54 du datasheet)

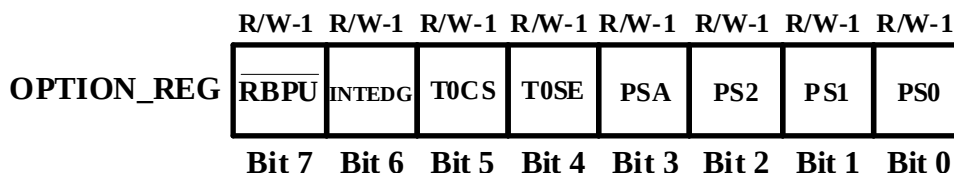


Figure VIII.71.

Les bits **RBPU** et **INTEDG** déjà été présentés sur la **figure VIII.19.**, sont liés au **PORTB** et n'ont rien à voir avec la fonction **TIMER0**.

La signification des autres bits a été expliquée sur la figure VIII.70. précédente et est résumée sur la figure VIII.72.

Bit	Valeur	Signification et commentaires	
PSA		Permet d'affecter le pré diviseur programmable soit au TIMER0 , soit au WATCH DOG	
	0	Pré diviseur affecté au TIMER0	
	1	Pré diviseur affecté au WATCH DOG	
PS2 :PS0		Pour choisir la valeur p du pré diviseur programmable	
		Si PSA=0 (pré diviseur au TIMER0)	Si PSA=1 (pré diviseur au WD)
	000	2	1
	001	4	2
	010	8	4
	011	16	8
	100	32	16
	101	64	32
	110	128	64
	111	256	128
T0CS		Timer 0 Clock Source Select : choix du mode de fonctionnement	
	0	Comptage du temps (via le quartz du PIC)	
	1	Horloge extérieure ou compteur d'évènements sur broche RA4	
T0SE		Timer 0 Select Edge : choix du front actif du comptage sur RA4	
	0	Front montant	
	1	Front descendant	

Figure VIII.72.

VIII.5.2.1.4. EXEMPLES DE MISE EN OEUVRE

L'objectif est de réaliser un signal carré périodique de fréquence $f_0 = 1 \text{ kHz}$ et disponible sur la broche **RD1** du **PORTD**, comme indiqué sur la figure VIII.73.

La période du signal à générer est donc :

$$T_0 = \frac{1}{f_0} = 1 \text{ ms.}$$

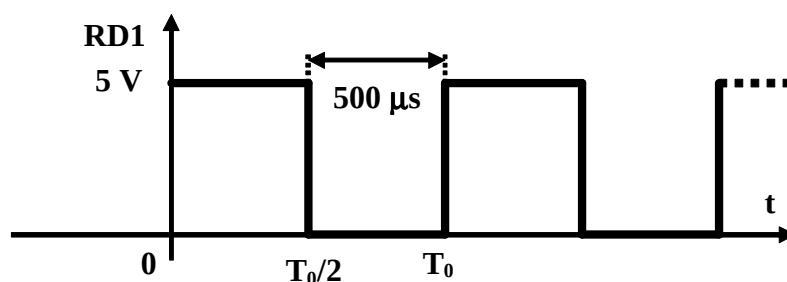


Figure VIII.73.

Il faut donc changer l'état de la sortie **RD1** toutes les $0,5 \text{ ms} = 500 \mu\text{s}$.

On va donc générer, grâce au **TIMER0**, une temporisation de $500 \mu\text{s}$.

Nous allons détailler sur cet exemple comment trouver les valeurs permettant d'initialiser le **TIMER0** pour réaliser cette temporisation de $500 \mu\text{s}$.

La formule (2) précédente nous donne :

$500 \mu\text{s} = f.p.(2^n - VI).T_{OSC}$ avec $n = 8$ et $T_{OSC} = 250 \text{ ns}$ (quartz de 4 MHz), ce qui permet d'écrire (en ns) :

$$500\,000 = 4.p.(256 - VI).250 \text{ et donc : } p.(256 - VI) = 500.$$

On prendra la plus petite valeur implantable pour p (pour avoir la plus grande précision).

Le tableau de la figure VIII.72. nous donne $p_{\min} = 2$. Et donc au final :

$256 - VI = 250$ ce qui nous fournit : **VI = 6** = valeur initiale à charger dans le registre compteur **TMR0** du **TIMER0**.

Le programme, utilisant la technique du pooling est fourni sur la figure VIII.74.

Comme d'habitude, lisez bien les commentaires !!!

```

void main(void)
{
    TRISD1 = 0;                // RD1 en sortie

    // initialisation de OPTION_REG
    // T0CS = 0                // quartz du PIC
    // T0SE = X                // indifférent
    // PSA = 0                 // pré diviseur affecté au TIMER0
    // PS2 PS1 PS0 = 000       // p = 2

    OPTION = OPTION & 0b11000000; // attention OPTION_REG est reconnu sous
                                   // le nom OPTION par le compilateur C !
                                   // on ne touche pas aux bits 7 et 6 qui ne
                                   // servent pas pour le TIMER0

    TMR0=6;                    // VI=6 (voir calcul précédent)

    T0IF=0 ;                   // mise à 0 du flag T0IF pour préparer
                                   // la première scrutation
                                   // boucle sans fin
    while (1)
    {
        while(!T0IF);          // on attend que T0IF passe à 1

        T0IF=0;                 // remise à 0 du flag pour préparer la scrutation
                                   // suivante

                                   // à partir de là T0IF est passé à 1
        TMR0=6;                 // recharge VI pour la demie période suivante

        PORTD = PORTD ^ bit(1); // changement état RD1 : ou exclusif ^
    }
}

```

Figure VIII.74.

Quelles sont les limites de ce type de fonctionnement ?

Si on veut réaliser des temporisations plus petites (ce qui revient à augmenter VI), on perd en précision.

En effet, certaines instructions du C (qui ont une durée) ne vont pas permettre de gérer le fonctionnement du timer comme on doit théoriquement le faire (test du flag **T0IF** : on perd un cycle, initialisation de **TMR0** : on perd 4 cycles ...).

Et au final, pour les faibles valeurs (VI proche de 0xFF), la temporisation obtenue pratiquement sera différente de celle souhaitée !!! En fait, ce type d'erreur existe aussi pour les temporisations plus grandes, mais on les voit moins, car en relatif, elles sont alors très faibles par rapport à la temporisation plus grande.

Les modules **CCP1** et **CCP2** qu'on va voir dans le chapitre suivant permettent de s'affranchir de ce type de problème.

Si on veut que le programme principal puisse faire autre chose et donc que la génération du signal carré sur la broche de sortie **RD1** se fasse sans aucune intervention du main, il faut alors utiliser les interruptions.

C'est ce que je propose dans le programme de la figure VIII.75. (qui réalise la même chose que celui de la figure VIII.74.) mais notez bien que le programme principal n'intervient plus dans la gestion du signal carré.

Dans cet exemple le main est une simple boucle sans fin du type **while(1)** mais ce pourrait être n'importe quoi d'autre...

```
void interrupt timer0_it (void)                // la routine d'IT qui génère le signal carré
{
    if(T0IF && T0IE)                          // IT sur overflow du TIMER0 : "daisy chain"
    {
        T0IF=0;                             // acquittement IT TIMER0
        TMR0=6;                             // recharge VI dans TMR0
        PORTD = PORTD ^ bit(1);              // changement d'état de RD1
    }
}

void main(void)
{
    TRISD1 = 0;                              // RD1 en sortie

    // initialisation de OPTION_REG
    // T0CS = 0                               // quartz du PIC
    // T0SE = X                               // indifférent
    // PSA = 0                               // pré diviseur affecté au TIMER0
    // PS2 PS1 PS0 = 000                     // p = 2

    OPTION = OPTION & 0b11000000;           // idem programme précédent
    TMR0=6;                                  // prépare première IT
    T0IF=0;                                  // efface flag « overflow » du TIMER0
    T0IE=1;                                  // démasquage local de l'IT TIMER0
    GIE=1;                                   // démasquage de toutes les IT

    while(1);                               // ici le main ne fait rien donc pourrait
                                           // toute autre chose
}
```

Figure VIII.75.

Avec le simulateur de MPLAB et VI=6 (valeur théorique), on obtient $\frac{T_0}{2} = 524 \mu s$, soit 24 μs de trop.

Avec $p = 2$, comme ici, chaque tick d'horloge incrémentant le compteur **TMR0** dure :

$$T_{\text{tick}} = 4.2.250 = 2000 \text{ ns} = 2 \mu s.$$

Il faut donc prévoir : $\frac{24}{2} = 12$ ticks d'horloge de moins, ce qui revient à prendre VI = 6 + 12 = 18

pour obtenir exactement la valeur souhaitée, soit 500 μs pour $\frac{T_0}{2}$.

Comme d'habitude et en résumé, voici sur la figure VIII.76. tous les registres associés au **TIMER0** ainsi que les bits concernés.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
01h,101h	TMR0	Timer0 Module's Register								xxxx xxxx	uuuu uuuu
0Bh,8Bh, 10Bh,18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
81h,181h	OPTION_REG	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111

Legend: x = unknown, u = unchanged, - = unimplemented locations read as '0'.

Shaded cells are not used by Timer0.

Figure VIII.76.

Sur la figure VIII.77. on fournit la même chose mais pour la fonction Watch Dog.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
2007h	Config. bits	(1)	BODEN ⁽¹⁾	CP1	CP0	PWRTE ⁽¹⁾	WDTE	FOSC1	FOSC0
81h,181h	OPTION_REG	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0

Legend: Shaded cells are not used by the Watchdog Timer.

Figure VIII.77.

VIII.5.2.2. LE TIMER1

VIII.5.2.2.1. PRINCIPALES CARACTERISTIQUES

Elles sont résumées ci-dessous :

- 4 valeurs possibles du pré diviseur **p** **programmable** : 1, 2, 4 et 8,
- registre compteur **TMR1** sur **16 bits** (son octet de poids fort **TMR1H** se trouve à l'adresse **0x00F** et son octet de poids faible **TMR1L** à l'adresse **0x00E**), accessible(s) en lecture ou en écriture,
- fonctionnement de **VI** (valeur initiale sur 16 bits à charger dans le registre compteur **TMR1**) à **0xFFFF**,
- le débordement est signalé par le flag **TMR1IF** (**TiMeR 1 Interrupt Flag** situé dans le registre SFR **PIR1**) qui passe à 1 quand le registre **TMR1** passe de 0xFFFF à 0x0000, (cf. figure VIII.68.),
- un bit **TMR1IE** (comme **TiMeR 1 Interrupt Enable** situé dans le registre SFR **PIE1**) permettra, localement, **d'autoriser** (si **TMR1IE=1**) ou **d'interdire** (si **TMR1IE=0**) la génération d'une interruption lorsque le flag **TMR1IF** passe à 1,
- le **TIMER1** possède un bit **TMR1ON** (comme **TiMeR 1 ON**) qui permet de le valider lorsque ce bit passe à 1.

VIII.5.2.2.2. SYNOPTIQUE

On va utiliser le schéma simplifié de la figure VIII.78. pour comprendre les différents modes de fonctionnement du **TIMER1** et les bits qui y sont associés.

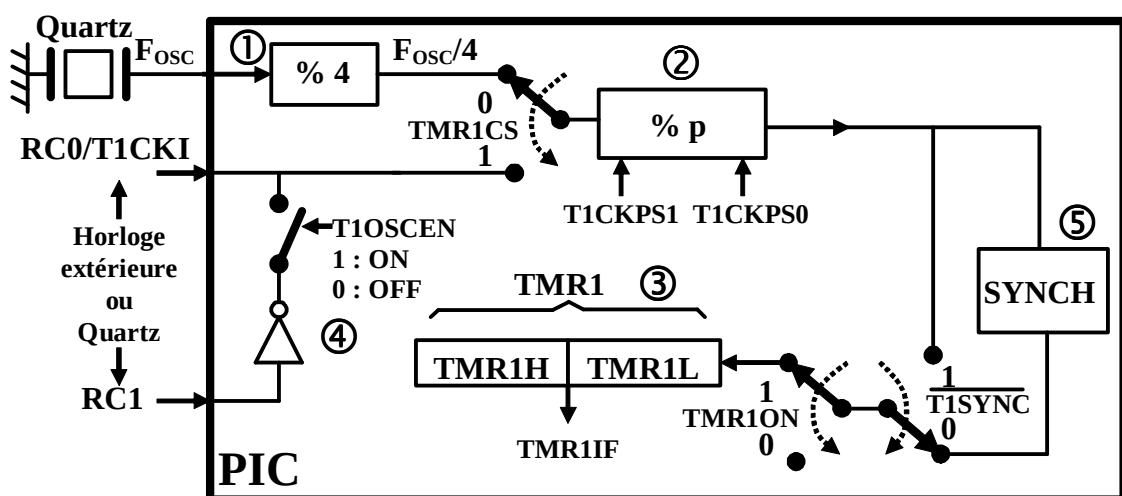


Figure VIII.78.

Là aussi, on y retrouve les blocs qu'on avait présentés dans le chapitre sur les généralités :

- bloc ① : le pré diviseur **fixe** par $f = 4$,
- bloc ② : le pré diviseur **programmable** par p , déterminé par les 2 bits **T1CKPS1** et **T1CKPS0** (voir figure VIII.80. plus loin pour plus de détails),
- bloc ③ : le registre compteur **TMR1** (**TMR1H** : **TMR1L**), avec son bit d'overflow **TMR1IF** qui passera à 1 quand le contenu de **TMR1** passe de 0xFFFF à 0x0000.

Deux multiplexeurs (pilotes respectivement par le bit **TMR1CS** et par le bit **T1SYNC**) permettent de choisir les différents modes de fonctionnement, comme détaillés ci-dessous :

- si **TMR1CS = 1**, le **TIMER1** fonctionne en compteur de front montant des impulsions (fournies par une horloge externe) ou des événements extérieurs (détecteur d'objets par exemple) signalés aussi par des fronts montants sur la broche **RC0/T1CKI** (comme **Timer 1 Clock Input**) du **PORTC**.

Dans ce dernier cas, l'interrupteur (cf. bloc ④) piloté par le bit **T1OSCEN** doit être ouvert en faisant **T1OSCEN=0**.

Mais on peut aussi brancher un quartz entre les broches **RC0** et **RC1** du **PORTC**, et alors l'interrupteur piloté par le bit **T1OSCEN** doit être fermé (justement pour faire « vibrer » ce quartz) en faisant **T1OSCEN=1**.

L'intérêt de mettre un quartz différent de celui du PIC apparaît dans certaines applications où on veut un comptage de temps précis et facile à faire (certaines valeurs de fréquence de quartz se prêtant mieux que d'autres pour réaliser une base de temps correctement étalonnée, par exemple lorsque la fréquence de ce quartz externe est une puissance de 2 comme 32768 ($= 2^{15}$) kHz).

Dans le mode compteur, grâce au bloc ⑤ de synchronisation la sortie du pré diviseur par p comptant les impulsions externes peut être synchronisée avec l'horloge système (si le bit **T1SYNC** = 0) ou pas (si le bit **T1SYNC** = 1). En mode asynchrone, le registre **TMR1** sera donc incrémenté même si l'horloge système est coupée, comme en mode **SLEEP** par exemple.

Notons également qu'en mode compteur, les broches **RC0** et **RC1** sont systématiquement programmées en entrée : il est donc inutile de configurer leurs bits de direction via **TRISC**.

- si **TMR1CS = 0**, le **TIMER1** fonctionne alors dans sa fonction de base : compter le temps en comptant les impulsions du quartz du PIC. Par contre, dans ce mode, le bit **T1SYNC** n'a aucune influence puisque par définition, les impulsions du quartz sont synchrones avec elles-mêmes.

Notons enfin l'existence du bit **TMR1ON** (si **TMR1ON=1**), qui permet d'injecter les ticks d'horloge (interne ou externe) vers l'entrée du compteur **TMR1**, validant ainsi la fonction **TIMER1**.

Lorsque **TMR1ON=0**, le **TIMER1** est inhibé (pas utilisable) ou arrêté (s'il fonctionnait).

VIII.5.2.2.3. REGISTRE DE CONTRÔLE ASSOCIE

La plupart des bits de configuration que l'on vient de présenter se trouvent dans le registre SFR **T1CON** (comme **Timer 1 CON**trol) dont le format est fourni sur la figure VIII.79.

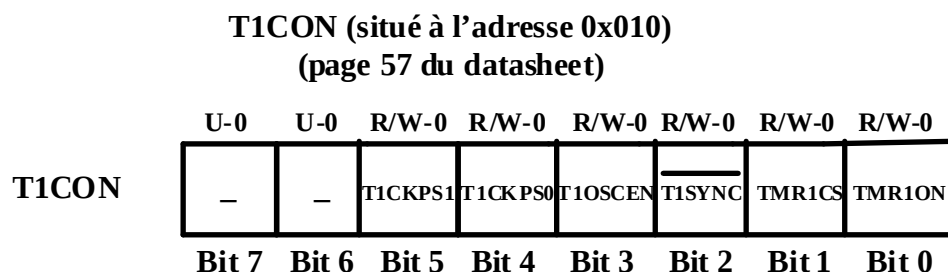


Figure VIII.79.

La signification de ces bits a été expliquée sur la figure VIII.78. précédente et est résumée sur la figure VIII.80.

Bit	Valeur	Signification et commentaires
T1CKPS1 :T1CKPS0		Pour choisir la valeur p du pré diviseur programmable
	00	1
	01	2
	10	4
	11	8
TMR1CS		TiMeR 1 Clock Source Select : choix du mode de fonctionnement
	0	Comptage du temps (via le quartz du PIC)
	1	Horloge extérieure (ou compteur d'évènements externes) sur broche RC0 ou quartz entre broches RC0 et RC1
T1OSCEN		Timer 1 OSC illator EN able
	0	Pas de quartz extérieur entre RC0 et RC1
	1	Présence d'un quartz extérieur entre RC0 et RC1
T1SYNC		Timer 1 SYN Chronised mode (valable uniquement en mode compteur externe (si TMR1CS=1))
	0	Compteur synchrone
	1	Compteur asynchrone
TMR1ON		TiMeR 1 ON
	0	Le TIMER1 est inhibé (ou arrêté)
	1	Le TIMER1 est validé : on peut l'utiliser

Figure VIII.80.

VIII.5.2.2.4. EXEMPLE DE MISE EN OEUVRE

Nous nous proposons, dans ce nouvel exemple, de générer un « Peigne de Dirac » sur une des broches de sortie du PIC.

C'est-à-dire un signal ayant la forme représentée sur la figure VIII.81.

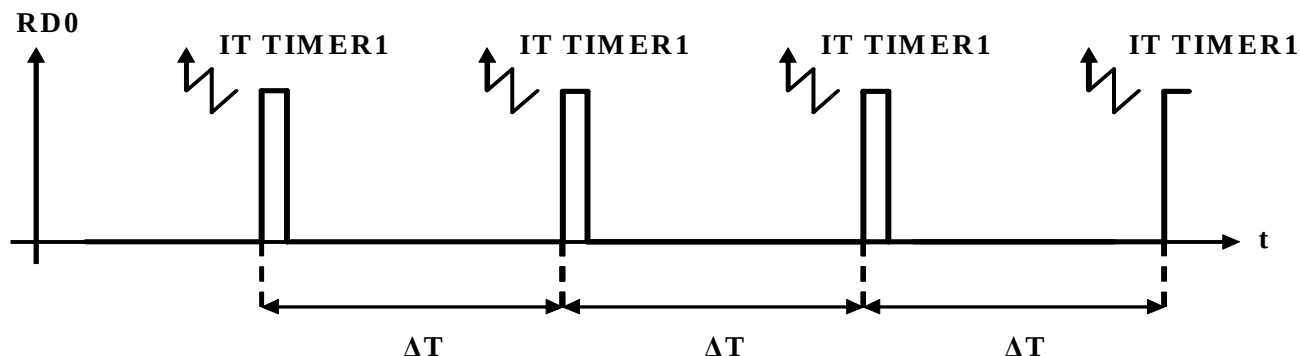


Figure VIII.81.

Ce signal, comme on le voit sur la figure VIII.81., sera obtenu en utilisant une interruption sur le **TIMER1**.

Ici, l'interruption aura pour fonction de sortir une impulsion de Dirac unitaire sur la ligne **RD0** du **PORTD** (programmée en sortie).

Mais dans un autre type d'application, cette interruption pourrait venir incrémenter un compteur ou «rafraîchir» et «mettre à jour» des données dans un périphérique ...

Dans cet exemple, on prendra $\Delta T = 100 \mu s$, qui seront mesuré par le **TIMER1**.

Le calcul, basé sur la formule précédente (2) des timers à valeur initiale **VI** nous donne :

$\Delta T = 100 \mu s = f.p.(2^n - VI).T_{OSC}$ avec $n = 16$ et $T_{OSC} = 250 ns$ (quartz de 4 MHz), ce qui permet d'écrire (en ns) :

$$100\,000 = 4.p.(65536 - VI).250 \text{ et donc : } p.(65536 - VI) = 100.$$

On prendra la plus petite valeur implantable pour p (pour avoir la plus grande précision).

Le tableau de la figure VIII.80. nous donne $p_{min} = 1$. Et donc au final :

$65536 - VI = 100$ ce qui nous fournit : **VI = 65436 = 0xFF9C** = valeur initiale à charger dans le registre compteur **TMR1** du **TIMER1**, soit 0xFF dans **TMR1H** et 0x9C dans **TMR1L**.

Le programme, utilisant la technique des interruptions est fourni sur la figure VIII.82.

Comme d'habitude, lisez bien les commentaires !!!


```

void interrupt timer1_it (void)
{
    if(TMR1IF && TMR1IE)                // IT sur overflow TIMER1
    {
        TMR1IF=0;                        // acquittement IT TIMER1
        TMR1H=0xFF;                      // recharge VIH dans TMR1H
        TMR1L=0x9C;                      // recharge VIL dans TMR1L
        PORTD = PORTD | bit(0);          // RD0 = 1
        PORTD = PORTD & (~bit(0));       // RD0 = 0
    }
}

void main(void)
{
    TRISD0 = 0;                          // peigne de Dirac sur RD0 en sortie

    // initialisation de T1CON
    // T1CKPS1 T1CKPS0=00                // p = 1
    // T1OSCEN=X                         // indifférent car pas de quartz extérieur
    // T1SYNC=X                          // indifférent car déjà synchrone
    // TMR1CS=0                          // quartz du PIC
    // TMR1ON=0                          // ne démarre pas encore TIMER1

    T1CON = T1CON & 0b00000000;
    TMR1H=0xFF;                          // VIH
    TMR1L=0x9C;                          // VIL
    TMR1IF=0;                            // efface une éventuelle cause d'IT antérieure
    TMR1IE=1;                            // démasquage local IT TIMER1
    PEIE=1;                              // démasquage semi global
    GIE=1;                               // démasquage de toutes les IT
    TMR1ON=1;                            // démarrage du TIMER1

    while(1);
}

```

Figure VIII.82.

Remarque 1:

Dans cet exemple, la valeur de **VI** est toujours la même (c'était la constante 0xFF9C).

Qu'en serait-il si c'était une variable déclarée par : **short int VI;** ?

Dans ce cas là, pour initialiser le registre **TMR1**, on pourrait écrire :

```

TMR1H=VI>>8 ;    // décalage de 8 bits à droite puis transfert de VIH dans TMR1H
TMR1L=VI ;       // transfert de VIL dans TMR1L

```

Ce qui pourrait permettre de changer ΔT en cours d'exécution du programme.

Remarque 2:

Si on teste ce programme, on s'aperçoit qu'on obtient pratiquement une valeur de $\Delta T \approx 127 \mu s$ au lieu des $100 \mu s$ souhaitées.

Quelle en est la raison ?

A cause du fait qu'on programme en C et que certaines choses comme la sauvegarde du contexte (qui n'apparaît pas dans le source) prennent du temps et augmentent le ΔT réel.

On peut alors modifier la valeur théorique de **VI** pour respecter la valeur de $100 \mu s$.

Il faut alors se rappeler qu'avec $p = 1$ comme ici, un tick d'horloge correspond à une durée de :

$$T_{\text{tick}} = 4.T_{\text{OSC}} = 4.250 \text{ ns} = 1 \mu s.$$

Puisqu'on obtient environ $27 \mu s$ de trop, cela correspond à 27 ticks d'horloge de trop.

On prendra donc : **VI = 65 436 + 27 = 65 463 = 0xFFB7.**

Et voici pour terminer, sur la figure VIII.84., la liste des registres en lien avec le **TIMER1**, ainsi que les bits concernés.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
0Eh	TMR1L	Holding Register for the Least Significant Byte of the 16-bit TMR1 Register								xxxx xxxx	uuuu uuuu
0Fh	TMR1H	Holding Register for the Most Significant Byte of the 16-bit TMR1 Register								xxxx xxxx	uuuu uuuu
10h	T1CON	—	—	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON	--00 0000	--uu uuuu

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'. Shaded cells are not used by the Timer1 module.

Figure VIII.84.

VIII.5.2.3. LE TIMER2

VIII.5.2.3.1. PRINCIPALES CARACTERISTIQUES

Elles sont reprises ci-dessous :

- **4** valeurs possibles du pré diviseur **programmable** : **1, 4, 16** et **64**,
- registre compteur **TMR2** sur **8 bits** (à l'adresse **0x011**), accessible en lecture ou en écriture,
- fonctionnement de **0x00** à **VF** (valeur finale à charger dans le registre **PR2** (comme **Period Register Time 2**)) accessible lui aussi en lecture ou en écriture, associé au **TIMER2** et situé à l'adresse **0x092**),
- existence d'un flag **TMR2IF** (**TiMeR 2 Interrupt Flag** situé dans le registre SFR **PIR1**) dont la gestion est un peu différente que pour le **TIMER0** ou le **TIMER1** à cause de l'existence d'un **post diviseur** (voir partie suivante),
- un bit **TMR2IE** (comme **TiMeR 2 Interrupt Enable** situé dans le registre SFR **PIE1**) permettra, localement, **d'autoriser** (si **TMR2IE=1**) ou **d'interdire** (si **TMR2IE=0**) la génération d'une interruption lorsque le flag **TMR2IF** passe à 1,
- le **TIMER2** possède un bit **TMR2ON** (comme **TiMeR 2 ON**) qui permet de le valider lorsque ce bit passe à 1.

VIII.5.2.3.2. SYNOPTIQUE

Par rapport au **TIMER0** et au **TIMER1**, le **TIMER2** possède trois originalités :

- il est **entièrement dédié** au comptage du temps via le quartz du PIC (pas de comptage de temps à partir d'une horloge extérieure ou de comptage d'évènements extérieurs),
- il compte non pas de VI à 0xFF (ou 0xFFFF) mais **de 0x00 à VF**,
- il possède, en plus, un **post diviseur** programmable **p'**.

Qu'est-ce c'est ?

Sans ce post-diviseur (ou lorsque ce post-diviseur vaut 1, ce qui revient au même), le flag **TMR2IF** passerait à 1 dès que le compteur **TMR2** atteint la valeur **VF** (contenu du registre **PR2**), ou plus exactement un tick d'horloge après (cf. figure VIII.67. et VIII.85.a.).

Avec ce post diviseur de valeur **p'**, il faut que la coïncidence entre **TMR2** et **PR2** ait lieu **p'** fois pour que le flag **TMR2IF** passe à 1 (cf. figure VIII.85.b.).

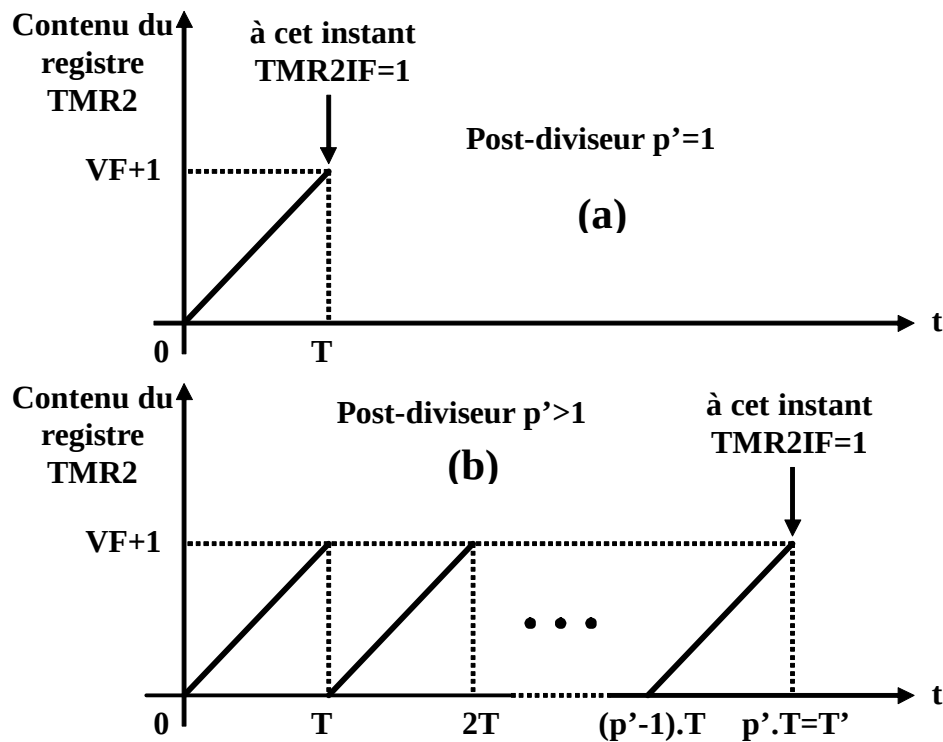


Figure VIII.85.

La durée T' nécessaire pour faire passer le flag **TMR2IF** à 1, prenant en compte les deux pré diviseurs et le post-diviseur est donc donnée par :

(1)' $\boxed{T' = 4.p.(VF+1).p'.T_{osc}}$ en appelant $T_{osc} = \frac{1}{F_{osc}}$ = période du quartz du PIC.

Comme on le voit, cette solution permet d'augmenter la durée mesurée par le **TIMER2** d'un facteur p' .

On va utiliser le schéma simplifié de la figure VIII.86. pour comprendre le fonctionnement du **TIMER2** et les bits qui y sont associés.

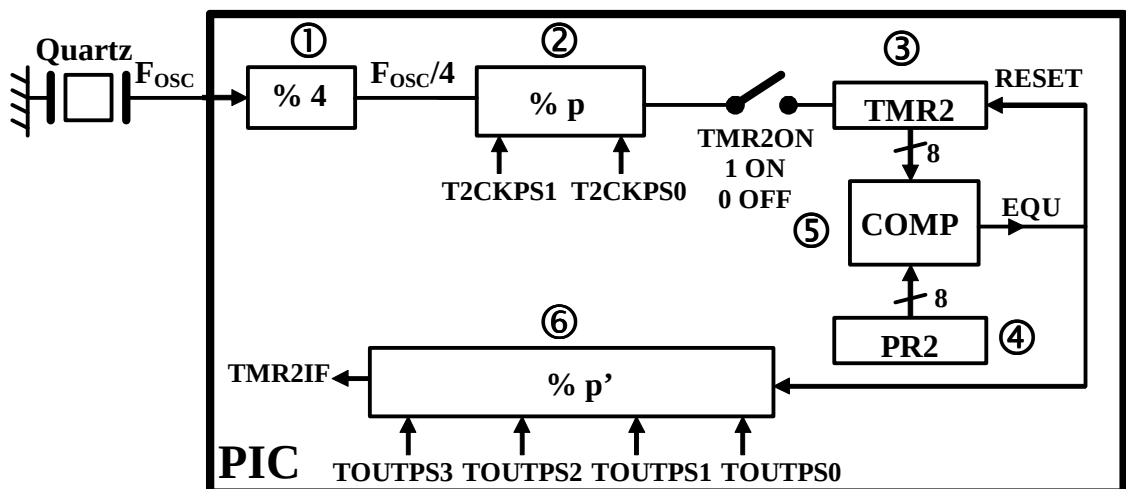


Figure VIII.86.

Là aussi, on y retrouve les blocs qu'on avait présentés dans le chapitre sur les généralités :

- bloc ① : le pré diviseur **fixe** par $f = 4$,
- bloc ② : le pré diviseur programmable par **p**, déterminé par les 2 bits **T2CKPS1** et **T2CKPS0** (voir figure VIII.88. plus loin pour plus de détails),
- bloc ③ : le registre compteur **TMR2**.

Le fonctionnement du **TIMER2** est assez simple.

Le contenu de ce registre **TMR2** (qui s'incrémente à chaque tick d'horloge) est comparé au contenu du registre de période **PR2** (cf. ④) et qui lui est insensible aux ticks d'horloge et contient la valeur **VF** qui peut être modifiée par programme) grâce à un comparateur câblé **COMP** 8 bits (cf. bloc ⑤).

La sortie **EQU** de ce comparateur passera à 1 quand **TMR2** atteindra **VF** (en fait un tick d'horloge après), ce qui aura deux conséquences importantes pour le fonctionnement du **TIMER2** :

- remise à zéro (signal **RESET** de la figure VIII.86.) du registre **TMR2** : on redémarrera donc une nouvelle période de comptage à partir de 0x00, (cela revient à dire que quand **TMR2** atteint la valeur **VF**, au lieu de continuer à **VF+1** sur le tick d'horloge suivant, il passera à la valeur 0x00...)
- envoi d'une impulsion qui sera comptée par le post compteur **p'** (cf. ⑥).

Lorsque **p'** impulsions de ce type auront été comptées par le post-diviseur, le flag **TMR2IF** sera positionné à 1 (ce qui peut générer une interruption si le flag associé **TMR2IE** du registre SFR **PIE1** est mis à 1).

La valeur de **p'** est déterminée par les 4 bits **TOUTPS3** à **TOUTPS0** (voir figure VIII.88. plus loin pour plus de détails). L'existence d'un post compteur **p'** ne change pas la granularité de **TIMER2**, mais permet d'implanter plus facilement des temporisations plus importantes.

Remarque :

Attention de ne jamais mettre une valeur nulle dans le registre **PR2**, en espérant ainsi faire une « temporisation nulle ». Car dans ce cas, la sortie **EQU** du comparateur ne changera jamais d'état et donc ne présentera jamais de front pour activer et positionner à 1 le flag **TMR2IF**. Il sera ainsi impossible de détecter une fin de temporisation : l'effet serait donc inverse à celui recherché, c'est-à-dire qu'on obtiendrait une « temporisation infinie » !!!

Notons enfin l'existence du bit **TMR2ON** (si **TMR2ON=1**), qui permet d'injecter les ticks d'horloge interne vers l'entrée du compteur **TMR2**, validant ainsi la fonction **TIMER2**.

Lorsque **TMR2ON = 0**, le **TIMER2** est inhibé (pas utilisable) ou arrêté (s'il fonctionnait).

VIII.5.2.3.3. REGISTRE DE CONTRÔLE ASSOCIE

La plupart des bits de configuration que l'on vient de présenter se trouvent dans le registre SFR **T2CON** (comme **Timer 2 CON**trol) dont le format est fourni sur la figure VIII.87.

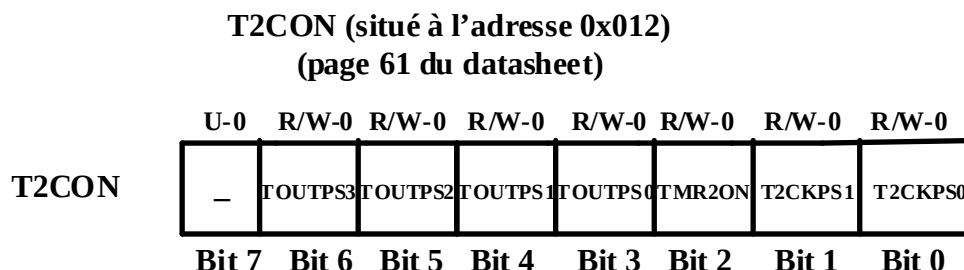


Figure VIII.87.

La signification de ces bits a été expliquée sur la figure VIII.86. précédente et est résumée sur la figure VIII.88.

Bit	Valeur	Signification et commentaires
T2CKPS1 :T2CKPS0		Pour choisir la valeur p du pré diviseur programmable
	00	1
	01	4
	10	16
	11	64 (testé bien que le datasheet ne le signale pas)
TOUTPS3 :TOUTPS0		Pour choisir la valeur p' du post diviseur programmable
	0000	1
	0001	2
	0010	3
	0011	4
	...	...
	1110	15
	1111	16
TMR2ON		TiMeR 2 ON
	0	Le TIMER2 est inhibé (ou arrêté)
	1	Le TIMER2 est validé : on peut l'utiliser

Figure VIII.88.

Nous nous proposons dans ce premier exemple de faire une temporisation **fixe** de 1 ms, qui nous servira dans un deuxième temps à réaliser une temporisation **paramétrable** de N millisecondes.

- la première de prototype : `void tempo_1_ms (void);` // dure 1 ms
- la seconde de prototype : `void tempo_N_ms (int N);` // dure N ms

(1)' $1\,000\,000 = 4.p.(VF+1).p'.T_{OSC}$ avec $T_{OSC} = 250\text{ ns}$ = période du quartz de 4MHz.

Soit : $p.p'.(VF+1) = 1\,000.$

On peut prendre : $p = 1$ et $p' = 4$ ce qui donne : $VF+1 = 250$ et donc $VF = 249$.

Ces deux fonctions sont données sur la figure VIII.89. où elles sont implantées en utilisant la technique maintenant connue du pooling sur le bit **TMR2IF** pour la fonction **tempo_1_ms**.

Figure VIII.89.

Le deuxième exemple que nous allons présenter va utiliser le **TIMER2** sous interruption.

Cet exemple va tenir à jour trois variables globales :

int **seconde, minute, heure** ;

qui indiqueront en permanence le temps qui s'écoule (sur une journée).

L'idée de base est de programmer l'interruption du **TIMER2** pour qu'elle ait lieu toutes les millisecondes en utilisant les valeurs calculées précédemment pour utiliser et initialiser le **TIMER2**, à savoir :

- **p = 1** : valeur du pré-diviseur programmable,
- **p' = 4** : valeur du pré-diviseur programmable,
- **VF = 249** : valeur à charger dans le registre de période **PR2** pour générer une interruption chaque milliseconde.

Il faut également rajouter une variable globale **it_timer2**, de type int, qui va compter le nombre d'interruptions générées.

Cette variable sera donc incrémentée à chaque interruption.

Lorsque qu'elle atteint la valeur 1000, cela signifie qu'on a eu 1000 interruptions du **TIMER2**, et donc qu'il s'est écoulé 1000 ms = 1 s.

On doit alors incrémenter, toujours dans la routine d'interruption, la variable **seconde**, sans oublier de remettre à 0 la variable **it_timer2** (cf. figure VIII.90.a.).

Il faut également penser ensuite à gérer les minutes. Ceci est fait en testant la variable **seconde** : lorsqu'elle atteint la valeur 60, on incrémente la variable **minute** et on remet **seconde** à 0. (cf. figure VIII.90.b.).

La méthode est la même pour gérer l'écoulement des heures (cf. figure VIII.90.c.).

Au bout de 24 heures, on remet la variable **heure** à 0 comme indiqué sur la figure VIII.9.d. (on ne tient pas à jour un calendrier).

Le schéma de la figure VIII.90. résume ce principe.

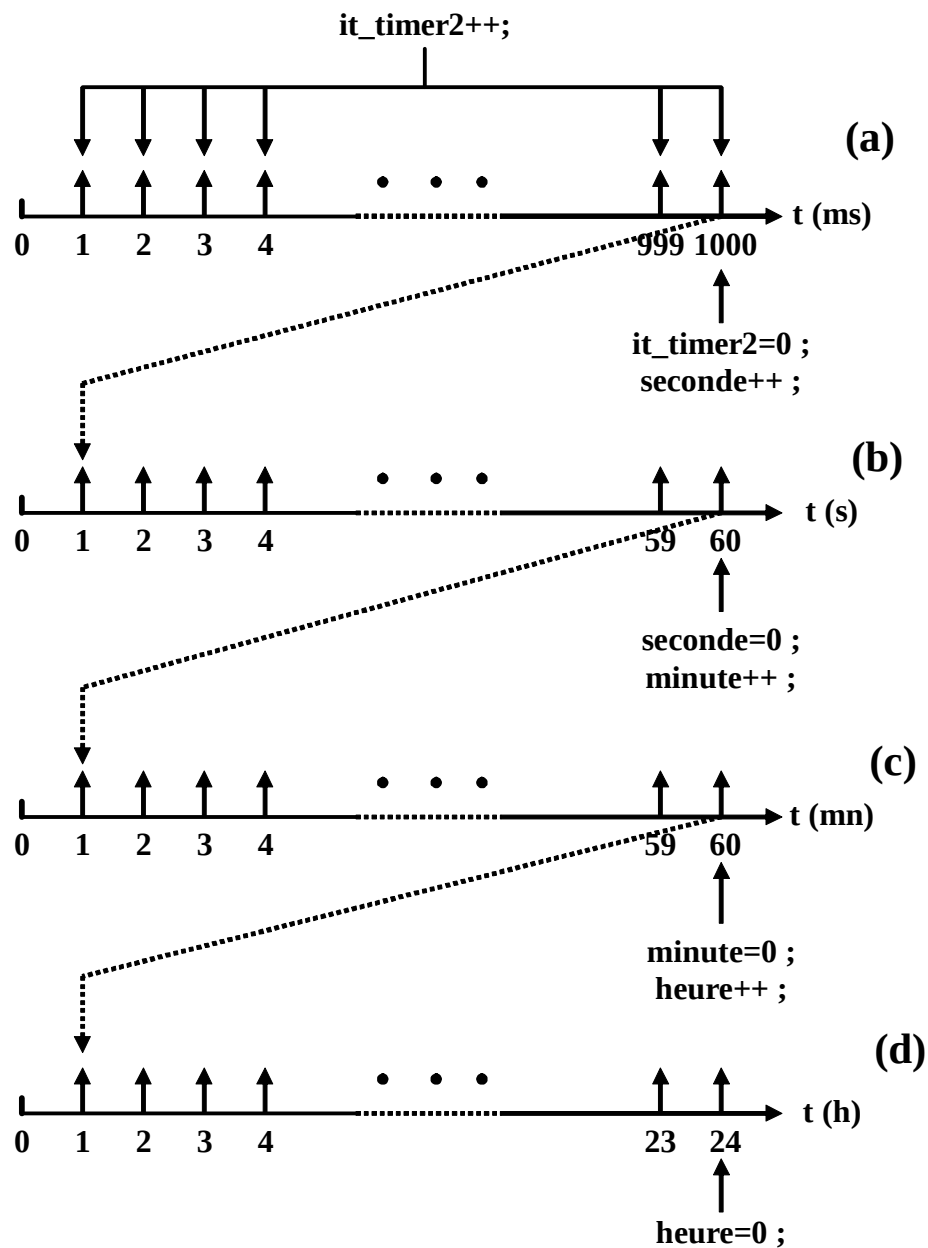


Figure VIII.90.

L'initialisation du **TIMER2** (faite dans le main) et la routine d'interruption sont données sur la figure VIII.91.

Ici, le main ne fait rien, mais on pourrait y mettre ce que l'on veut, la gestion du temps (via les variables globales **seconde**, **minute** et **heure**) faite par l'interruption est automatique et donc transparente pour le main.

```

int    it_timer2=0,           // compteur d'interruptions du TIMER2
       seconde=0,            // compteur des secondes qui s'écoulent
       minute=0,             // compteur des minutes qui s'écoulent
       heure=0;              // compteur des heures qui s'écoulent

void interrupt timer2_it (void) // IT du TIMER2 exécutée chaque ms
{
    if(TMR2IF && TMR2IE)      // daisy chain
    {
        TMR2IF=0;            // acquittement de l'IT du TIMER2
        it_timer2++;          // une IT de plus
        if (it_timer2==1000)  // 1000 IT, donc une seconde écoulée
        {
            it_timer2=0;      // on remet à 0 le compteur d'IT
            seconde++;         // une seconde de plus
            if(seconde==60)    // 60 secondes, donc une minute écoulée
            {
                seconde=0;     // on remet à 0 le compteur de secondes
                minute++;       // une minute de plus
                if(minute==60)  // 60 minutes, donc une heure écoulée
                {
                    minute=0;   // on remet à 0 le compteur de minutes
                    heure++;     // une heure de plus
                    if(heure==24) // 24 heures, donc une journée écoulée
                    {
                        heure=0; // on remet à 0 le compteur d'heures
                    }
                }
            }
        }
    }
}

void main(void)
{
    // initialisation de T2CON
    // TOUTPS3:TOUTPS0 = 0011 // p'=4
    // TMR2ON=0               // timer 2 off
    // T2CKPS1 T2CKPS0 = 00   // p=1
    T2CON = 0b0011000;
    TMR2IF=0;                // efface le flag TMR2IF
    PR2=249;                 // pour 1 ms entre chaque IT
    TMR2IE=1;                // démasquage local
    PEIE=1;                  // démasquage semi global
    GIE=1;                   // démasquage de toutes les IT
    TMR2=0;                  // initialisation du TIMER2
    TMR2ON=1;                // mise en route du TIMER2
    while(1);                // boucle sans fin !!! qui ne fait rien !!!
}

```

Figure VIII.91.

Et pour terminer, voici en résumé sur la figure VIII.92., les registres concernés par le fonctionnement du **TIMER2** et les différents bits concernés.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh,8Bh, 10Bh,18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
11h	TMR2	Timer2 Module's Register								0000 0000	0000 0000
12h	T2CON	—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0	-000 0000	-000 0000
92h	PR2	Timer2 Period Register								1111 1111	1111 1111

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'. Shaded cells are not used by the Timer2 module.

Figure VIII.92.

On peut y voir, par exemple, qu'au **RESET** du PIC, le registre de période **PR2** est initialisé avec la valeur 0xFFFF.

VIII.6. LES MODULES CCP1 ET CCP2

(cf. ⑩ du schéma de la **figure II.4.** et cf. **pages 63 à 69** du datasheet)

Les 3 timers présentés dans le chapitre précédent présentaient un défaut essentiel :

pour des faibles valeurs de la temporisation, les précisions obtenues étaient médiocres, ceci étant essentiellement dû au fait que les temps d'exécution des instructions entraînaient des délais de latence (et des durées de réaction du PIC) non négligeables devant les faibles valeurs des temporisations.

En fait, ce défaut existe aussi pour des temporisations plus importantes, mais dans ce cas là, en relatif et en absolu, il peut être négligé.

Comment résoudre cette difficulté ?

Puisque le problème, comme on vient de le dire vient des instructions et donc plutôt du software, on va le résoudre en « le sous-traitant » au hardware. En fait, on va rajouter quelques registres supplémentaires à la fonction timer, car les temps de réponse de registre à registre (et donc sans passer par la CPU) seront forcément plus courts.

C'est ce qui est fait dans les modules **CCP1** et **CCP2** que l'on va présenter maintenant.

En pratique, trois types de fonctions seront gérés par les modules **CCP1** et **CCP2** (comme **Capture/Compare/Pulse width modulation 1 et 2**) :

- la fonction **CAPTURE** : elle fonctionne en **entrée**,
- la fonction **COMPARE** : elle fonctionne en **sortie**,
- la fonction **PWM** : elle fonctionne également en **sortie**.

Comme d'habitude, nous donnerons dans un premier temps les principes généraux de chacune de ces fonctions. Et si vous n'êtes pas familier avec ces fonctions, il est fortement conseillé de commencer par là, car cela ne sert à rien de mettre en œuvre une fonction si on n'a pas bien compris comment elle fonctionne et à quoi elle sert. Le temps que vous pensez, à tort, avoir gagné en sautant cette étape, sera très vite perdu par la suite...

Il est également obligatoire que le chapitre sur les timers internes du PIC soient bien compris avant d'aborder les principes de fonctionnement et de mise en œuvre des modules **CCPi**.

VIII.6.1. LA FONCTION (INPUT) CAPTURE

VIII.6.1.1. GENERALITES SUR LA FONCTION DE CAPTURE

L'idée est de pouvoir **dater** un évènement extérieur, c'est-à-dire connaître la valeur du timer (le contenu du registre **TIMER**) quand cet évènement se produit.

Pour cela, il faut envoyer sur une des pattes d'**entrée** du PIC prévue à cet effet (appelée dans ces généralités **CCP**) un signal E(t) qui change d'état à l'instant t_c où cet évènement extérieur se produit, comme indiqué sur la figure VIII.93. suivante.

La fonction **(INPUT) CAPTURE** est prévue pour que le PIC recopie automatiquement le contenu courant du registre **TIMER** dans un registre qu'on appellera ici le registre **CAPTURE** à l'instant t_c du changement d'état de E(t).

(avec une précision inférieure à $\pm \frac{1}{2}$ tick d'horloge du timer).

Ensuite le registre **TIMER** continue toujours à augmenter mais le registre **CAPTURE** reste figé, comme si on avait « gelé » la valeur du registre **TIMER** à l'instant t_c de la capture.

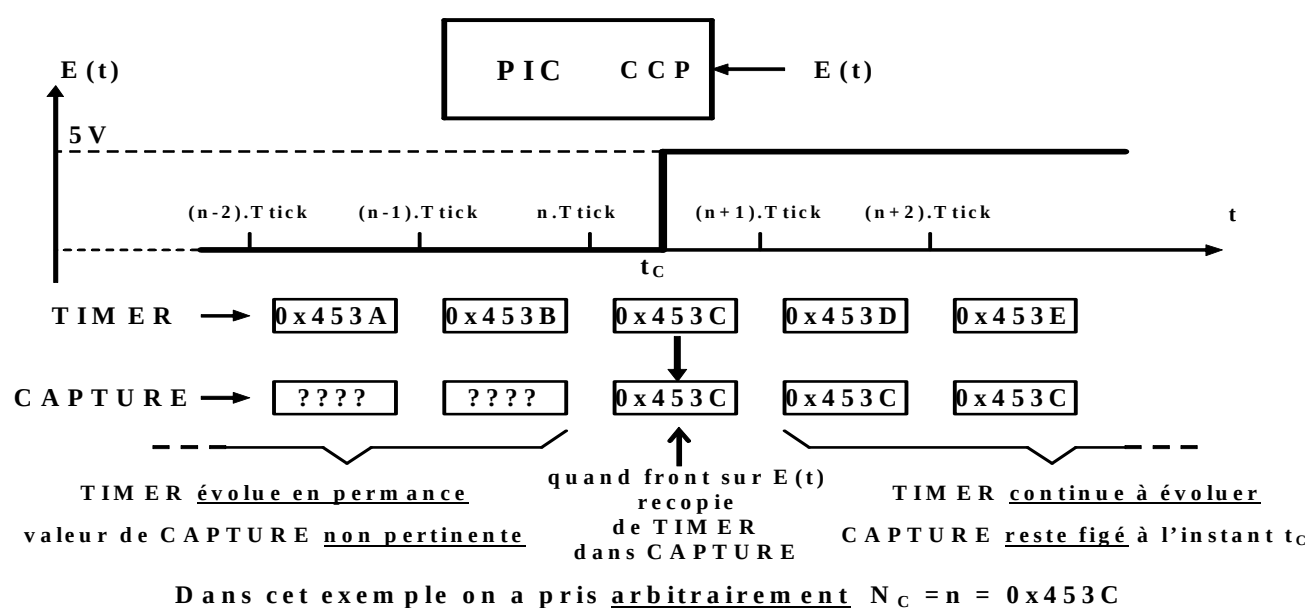


Figure VIII.93.

La taille du registre **CAPTURE** est bien évidemment la même que celle du registre **TIMER**.

Une entrée de capture **CCP** n'est pas seulement sensible que sur un front montant de E(t) comme dans l'exemple ci-dessus, mais peut aussi l'être sur un front descendant, voire sur les 2 types de fronts. Et ceci, en modifiant quelques bits dans un registre de contrôle adéquat, comme on le verra plus loin.

Ce sera au programme de venir lire le contenu du registre **CAPTURE**.

Mais attention, si un autre évènement se produit (comme un second front montant sur E(t)), une nouvelle capture dans le registre **CAPTURE** aura lieu et écrasera la valeur précédente.

Conclusion : il ne faut pas trop attendre pour venir lire le contenu du registre **CAPTURE**, sinon on risque de faire une lecture erronée. Mais grâce à la technique des interruptions, on sait que c'est possible.

Le contenu du registre **CAPTURE** (comme $N_C = 0x453C$ dans l'exemple de la figure VIII.93.) ne donne pas le temps t_C directement.

Il faut pour cela prendre en compte la période T_{tick} des ticks d'horloge incrémentant le **TIMER** et faire :

$$t_C = N_C \cdot T_{\text{tick}}$$

Dans cet exemple, on vient de dater un évènement en absolu, c'est-à-dire par rapport à un instant initial choisi par convention.

Mais, ce n'est pas la seule possibilité d'utilisation de la fonction de capture.

Une variante (très utilisée) de cette fonction permet de **mesurer un intervalle de temps**, comme par exemple la largeur d'une impulsion TTL (positive ou négative).

Le principe, donné ici pour une impulsion positive, est alors très simple :

- on initialise la fonction **CAPTURE** pour que la broche d'entrée **CCP** soit sensible aux fronts montants,
- à $t = t_{C1}$, quand le premier front montant de l'impulsion arrive, cet instant t_{C1} (en nombre de ticks d'horloge du **TIMER**) est automatiquement figé dans le registre **CAPTURE**, comme sur la figure VIII.93. On vient alors, par programme, copier ce registre dans une première variable N_{C1} . Puis on modifie la nature du front actif sur **CCP** (en choisissant la prochaine capture sur un front actif **descendant**),
- à $t = t_{C2}$, quand le prochain front descendant de l'impulsion arrive, cet instant t_{C2} (en nombre de ticks d'horloge du **TIMER**) est automatiquement figé dans le registre **CAPTURE**. On vient, par programme, copier ce registre dans une seconde variable N_{C2} .
- il ne reste plus qu'à faire la « **différence** » entre ces deux variables N_{C2} et N_{C1} (avec quelques précautions, comme on le verra dans l'exemple traité plus loin sur la figure VIII.99., exemple qui mettra notamment le doigt sur les pièges à éviter !!!)
- le résultat obtenu est la **largeur** N de l'impulsion, exprimée en nombre de ticks d'horloge.

VIII.6.1.2. SYNOPTIQUE

Sur le PIC, la fonction de capture utilise le **TIMER1** comme référence temporelle.

Cette fonction peut-être réalisée au choix par **l'un ou l'autre** des deux modules **CCPi**.

On peut même implanter deux fonctions de capture **indépendantes**, une sur **CCP1** et l'autre sur **CCP2**, mais elles partageront alors la même base de temps fournie par le **TIMER1**.

Le registre **CAPTURE** pour **CCP1** s'appelle **CCPR1** et celui de **CCP2** s'appelle **CCPR2**.
(comme **C**apture **C**ompare **P**wm **R**egister **1** ou **2**).

Ils sont tous les deux de la même taille que le registre **TMR1**, c'est-à-dire 16 bits.

L'entrée de capture est appelée **CCP1** (c'est la ligne **RC2** du **PORTC**) pour le module **CCP1**.

L'entrée de capture est appelée **CCP2** (c'est la ligne **RC1** du **PORTC**) pour le module **CCP2**.

Lorsque la fonction de capture est utilisée, ces lignes **doivent être programmées** en **entrée** via le registre **TRISC**

Le synoptique simplifié de l'une ou l'autre de la fonction capture (dans **CCP1** ou **CCP2**) est le même. Il est donné sur la figure VIII.94. pour le module **CCP1**.

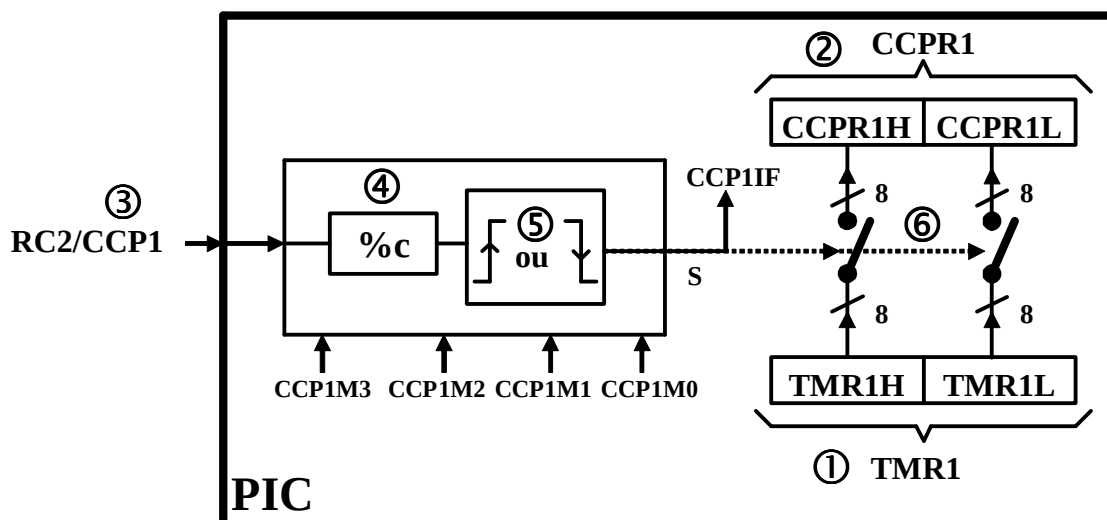


Figure VIII.94.

On y retrouve les blocs qu'on avait présentés dans le chapitre sur les généralités :

- bloc ① : le registre **TIMER**, ici réalisé par le registre **TMR1**,

Les ticks d'horloge incrémentant le registre **TMR1** n'ont pas été représentés (se référer au synoptique de la figure VIII.78.). Par contre, il faut noter que si on utilise le **TIMER1** en mode compteur asynchrone, la fonction de capture ne fonctionnera pas.

- bloc ② : le registre **CAPTURE** appelé ici **CCPR1** (ou **CCPR2**),
- ③ : l'entrée de capture de l'évènement extérieur : **CCP1/RC2** (ou **CCP2/RC1**).

Ce signal de capture passe dans un pré diviseur de valeur **paramétrable c** (cf. bloc ④ à ne pas confondre avec le pré diviseur p du **TIMER1** !!!) pouvant prendre 3 valeurs (1, 4 ou 16) déterminées par les 4 bits **CCPiM3** à **CCPiM0** (voir figure VIII.96. plus loin pour plus de détails). Cela signifie que le front actif de capture peut être généré à chaque évènement extérieur (c=1), tous les 4 évènements (c=4), ou tous les 16 évènements (c=16).

De plus, le bloc ⑤ permet de choisir le **front actif** de capture (soit un front montant, soit un front descendant) suivant les 4 bits **CCPiM3** à **CCPiM0** (voir aussi figure VIII.96. plus loin pour plus de détails).

Lorsque la sortie **S** du bloc ⑤ passe à 1 (c'est-à-dire lorsque le front actif a été détecté), il y aura deux conséquences importantes (et là on est au « cœur » de la fonction **CAPTURE**) :

- le contenu du registre **TMR1** est copié dans le registre **CCPR1** (ce qui est symbolisé par les 16 interrupteurs de la partie ⑥ du synoptique qui vont se fermer, le temps du transfert, lorsque **S** passe à 1),
- le flag **CCP1IF** (ou **CCP2IF**) passe à 1, pour signaler la capture.

Pour gérer ce flag, la technique du polling est toujours possible, mais il est préférable, surtout dans les cas où les contraintes temporelles sont « serrées » d'utiliser les interruptions en validant le bit correspondant **CCP1IE** (situé dans le registre SFR **PIE1**) ou **CCP2IE** (situé dans **PIE2**).

Remarque 1 :

En cas de changement du mode de capture (changement de front actif ou de la valeur de c), une interruption « indésirable » peut se produire. Il est donc nécessaire :

- de masquer cette interruption (**CCPiIE=0**) avant tout changement de mode de capture,
- puis de faire la modification du mode via les bits **CCPiM3** à **CCPiM0** du registre **CCPiCON**,
- ensuite de mettre à 0 le flag **CCPiIF** (pour « effacer » cette interruption « parasite »),
- et enfin de remettre à 1 le bit **CCPiIE** (pour ré autoriser à nouveau les interruptions).

Remarque 2 :

On peut simuler l'arrivée d'un évènement extérieur et donc provoquer une capture par programme en écrivant sur le bit **RC2** (ou **RC1**) du **PORTC**, à condition d'avoir préalablement mis ces lignes en sortie via le registre **TRISC**.

VIII.6.1.3. REGISTRES DE CONTRÔLE ASSOCIES

La plupart des bits de configuration que l'on vient de présenter se trouvent dans les deux registres SFR **CCP1CON** (comme **CCP1 CONtrol**) pour le module **CCP1** et **CCP2CON** (comme **CCP2 CONtrol**) pour le module **CCP2** dont le format commun est fourni sur la figure VIII.95.

**CCP1CON/CCP2CON (situés aux adresses respectives 0x017 et 0x01D)
(page 64 du datasheet)**

	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CCP1CON			CCPiX	CCPiY	CCPiM3	CCPiM2	CCPiM1	CCPiM0
CCP2CON	–	–						
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.95.

La signification de ces bits a été expliquée sur la figure VIII.94, précédente et est résumée sur la figure VIII.96.

Nous n'indiquerons ici que les combinaisons concernant la fonction de **capture**.
(pour les autres fonctions se reporter à leurs chapitres respectifs)

Bit	Valeur	Signification et commentaires
CCPiX :CCPiY		Voir fonction PWM
CCPiM3 :CCPiM0		Pour choisir le mode de fonctionnement
	0000	Modules CCPi non actif (mise à zéro des sorties CCPi)
	0100	Mode de capture à chaque front descendant sur CCPi (c=1)
	0101	Mode de capture à chaque front montant sur CCPi (c=1)
	0110	Mode de capture tous les 4 fronts montant sur CCPi (c=4)
	0111	Mode de capture tous les 16 fronts montant sur CCPi (c=16)
	1xxx	Autres modes de fonctionnement

Figure VIII.96.

VIII.6.1.4. EXEMPLE DE MISE EN OEUVRE

On se propose de mesurer la période T (et/ou la fréquence F) d'un signal TTL carré injecté sur la patte **RC2** du PIC.

On va donc utiliser le module **CCP1** en mode de capture.

La figure VIII.97. montre l'allure du signal **RC2(t)** et la période T que l'on doit mesurer.

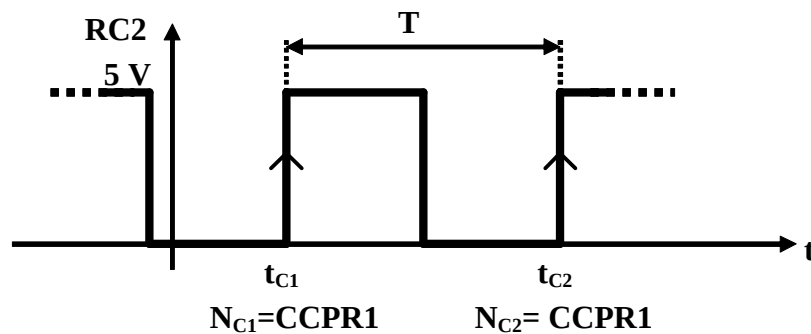


Figure VIII.97.

On vient de voir que, grâce à deux captures successives (par exemple sur deux fronts montants successifs comme sur la figure VIII.97.) on peut accéder aux valeurs de t_{c1} et de t_{c2} via les valeurs N_{C1} et N_{C2} lues dans le registre **CCPR1** à ces deux instants de capture.

La figure VIII.97. « suggère » alors que :

$$\text{si } T = t_{c2} - t_{c1} \quad \text{alors : } N = N_{C2} - N_{C1} \quad \text{et} \quad T = N \cdot T_{\text{tick}} = (N_{C2} - N_{C1}) \cdot T_{\text{tick}}$$

où T_{tick} représente comme on l'a dit précédemment la période de l'horloge incrémentant le **TIMER1**

Ce serait aller vite en besogne que d'appliquer cette formule « sans réfléchir ».

En effet, il ne faut surtout pas oublier qu'entre les instants t_{c1} et de t_{c2} , le registre **TMR1** continue à s'incrémenter et donc peut très bien avoir fait un (ou plusieurs) overflow (ou débordement : passage de 0xFFFF à 0x0000) comme indiqué sur la figure VIII.98. où chaque flèche indique un débordement de **TMR1**.

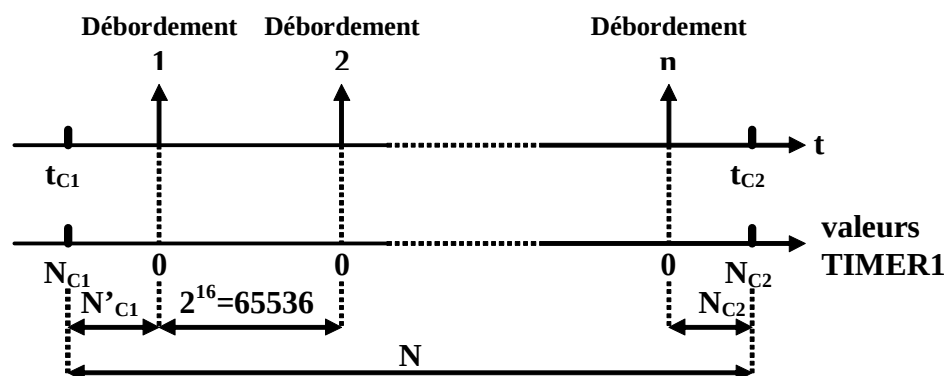


Figure VIII.98.

Comment savoir si le registre **TMR1** a débordé zéro fois, une fois, deux fois , ..., n fois ?

En validant l'interruption sur débordement du **TIMER1** et en comptant, entre deux captures successives, le nombre n d'interruptions qui correspondra au nombre n de débordements.

On voit que dans ce cas :

$N = N'_{C1} + (n-1).65536 + N_{C2}$ avec $N'_{C1} = 65536 - N_{C1}$ ce qui donne au final :

$$N = n.65536 + N_{C2} - N_{C1}$$

On retrouve d'ailleurs, pour $n = 0$ (pas de débordement entre t_{C1} et t_{C2}) la formule qu'on avait proposée au départ « sans trop réfléchir ». Et donc :

$$\boxed{T = N.T_{\text{tick}} = (n.65536 + N_{C2} - N_{C1}). T_{\text{tick}}}$$

En pratique, cette formule générale sera surtout pertinente pour de grandes valeurs de T, et donc des fréquences plutôt faibles. Car dès que la fréquence du signal injecté sur la patte **RC2** augmentera, il n'y aura pas plusieurs débordements entre t_{C1} et t_{C2} (mais au maximum 1).

Ce principe est mis en œuvre sur le programme donné sur la figure VIII.99. où on va mettre en œuvre deux interruptions :

- l'interruption de débordement du **TIMER1** qui tient à jour un compteur de débordements déclaré comme un **unsigned short** n (16 bits),
- l'interruption de capture sur front montant de **RC2** qui tient à jour les instants de capture et calcule la variable N (déclaré comme un **unsigned long** sur 32 bits) par la formule précédente où N_{C1} et N_{C2} sont des **unsigned short** (16 bits).

Remarquons également que compte tenu du choix du pré diviseur p (prise à 1) du **TIMER1**, la durée T_{tick} vaut 1 μs et donc N représente directement la durée T exprimée en μs .

```

unsigned short n=0,NC1=0,NC2=0;           // compteur de débordement et variables de capture
unsigned long   N;                         // période en microsecondes
float           F;                         // fréquence
char            buf[16];                  // buffer de formatage pour affichage LCD

// la routine d'interruption
void interrupt interruption (void)
{
    if(TMR1IF && TMR1IE)                  // daisy chain
    {                                     // IT sur débordement TIMER1
        TMR1IF=0;                        // acquittement IT débordement TIMER1
        n++;                             // compteur de débordement : une IT de plus
    }
    if (CCP1IE && CCP1IF)                  // IT de capture sur front montant de RC2
    {
        CCP1IF=0;                        // acquittement IT de capture
        NC2=256*CCPR1H+CCPR1L;           // NC2 est la nouvelle capture
//NC2=((NC2=CCPR1H)<<8)+CCPR1L;          // idem ligne précédente avec décalage 8 bits à gauche
        N=65536*n+NC2-NC1;               // calcul de N par la formule précédente
//N=((N=n)<<16)+NC2-NC1;                  // idem ligne précédente avec décalage 16 bits à gauche
        n=0;                             // remise à zéro du compteur de débordement
        NC1=NC2;                          // NC1 est l'ancienne capture pour la capture suivante
    }
}

void init_PORTC (void)
{
    TRISC2=1;                            // RC2 = CCP1 = Input
}

void init_timer1(void)
{
    // initialisation de T1CON
    // T1CKPS1 T1CKPS0=00                // p = 1 pour Ttick=1 microseconde
    // T1OSCEN=X                          // indifférent car pas de quartz extérieur
    // /T1SYNC=X                          // indifférent car déjà synchrone
    // TMR1CS=0                           // quartz du PIC
    // TMR1ON=1                           // démarre TIMER1
    T1CON = 0b00000001;                  // valeur d'initialisation

    TMR1H=0;                             // pour T1max
    TMR1L=0;                             // pour T1max
}

void init_CCP1()
{
    // initialisation de CCP1CON
    // CCP1X = X                          // non utilisé
    // CCP1Y = X                          // non utilisé
    // CCP1M3:CCP1M0 = 0101              // capture sur chaque front montant
    CCP1CON=0b00000101;                  // valeur d'initialisation
}

```

Figure VIII.99.

```

void init_interruption(void)
{
    TMR1IF=0;           // efface une éventuelle IT antérieure
    TMR1IE=1;           // valide IT TIMER1
    CCP1IF=0;           // efface une éventuelle IT antérieure
    CCP1IE=1;           // valide IT sur capture CCP1
    PEIE=1;             // démasquage semi global
}
void main(void)
{
    init_PORTD();        // fixe la direction des lignes du PORTD pour le LCD
    init_PORTC();
    init_timer1();
    init_CCP1();
    init_interruption();
    allume_LCD();
    init_LCD();
    print_string("ESSAI");
    goto_ligne_2();
    print_string("frequencemetre");
    tempo_N_ms(3000);
    efface();
    GIE=1;               // valide toutes les IT
while(1)
{
    efface();
    sprintf(buf,"T = %lu us",N); // formatage de N : entier long (l) et unsigned (u)
    go_to(1);
    print_string(buf);         // affichage période sur ligne 1 du LCD
    F=(1000./N);               // si N en µs, alors avec cette expression F est kHz
                                // attention 1000. pour que la division donne un float !!!
    sprintf(buf,"F = %5.3f kHz",F); // formatage de F : float (5 caractères affichés,3 chiffres décimaux)
    go_to(17);                 // début de la ligne 2
    print_string(buf);         // affichage fréquence sur ligne 1 du LCD
    tempo_N_ms(500);           // tempo d'affichage
}
}

```

Figure VIII.99. (suite et fin)

Ce programme utilise les fonctions de pilotage du LCD vues dans le TP N°3 pour afficher la période T en µs et la fréquence F en kHz.

Il fonctionne tel quel jusqu'à environ 3 kHz. Pour augmenter sa plage de mesure, il faut jouer sur le pré diviseur c du module CCP1 (voir figure VIII.96.) pour augmenter l'intervalle de temps entre deux fronts de capture et compenser en diminuant d'autant la valeur du pré diviseur p du **TIMER1** (voir figure VIII.80) pour maintenir la relation $T = N$. Noter qu'en augmentant p, on perd en précision (moins bonne granularité des ticks d'horloge...).

Par contre, si on augmente trop l'intervalle de temps entre deux captures, aux très basses fréquences, le temps de réponse pour obtenir le résultat de la mesure deviendra trop important.

Moralité : un « vrai » fréquencemètre s'adaptera **dynamiquement** aux signaux mesurés.

VIII.6.2. LA FONCTION (OUTPUT) COMPARE

VIII.6.2.1. GENERALITES SUR LA FONCTION COMPARAISON

C'est en quelque sorte la fonction inverse de la fonction **CAPTURE**. Le but ici est de **générer** sur une des broches du PIC programmée **en sortie** un front actif à un instant t_C voulu par le programme.

Pour cela il faut écrire cette valeur de t_C (exprimée en nombre de ticks d'horloge) dans un registre prévu à cet effet et qu'on appellera dans cette présentation générale le registre **COMPARE**.

La fonction (**OUTPUT**) **COMPARE** est prévue pour comparer en **permanence** le contenu du registre **TIMER** avec le contenu de ce registre **COMPARE**. Et quand il y a égalité entre ces 2 registres, elle envoie automatiquement un front $S(t)$ sur une patte de sortie appelée **CCP**.

Ce principe est résumé sur la figure VIII.100. ci-dessous.

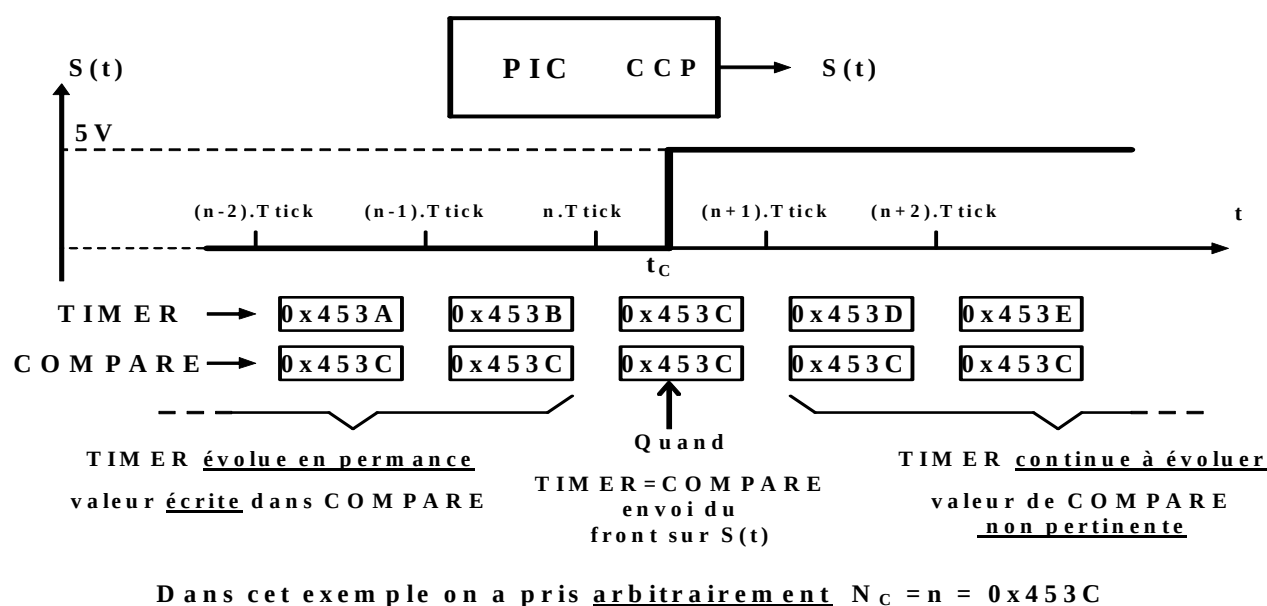


Figure VIII.100.

La taille du registre **COMPARE** est bien sûr la même que celle du registre **TIMER**.

Une sortie de capture **CCP** n'est pas seulement capable de générer un front montant sur $S(t)$ comme dans l'exemple ci-dessus, mais peut aussi générer un front descendant, voire tout simplement changer d'état (toggle) à chaque comparaison valide. Et ceci, en modifiant quelques bits dans un registre de contrôle adéquat, comme on le verra plus loin.

Remarque :

Si on y regarde de plus près, cette fonction pouvait déjà être réalisée uniquement avec les timers internes du PIC. Mais, sous contrôle logiciel, avec tout ce que cela impliquait au niveau des temps de réponse (notamment pour les très faibles temporisations) comme on l'avait déjà souligné. Avec la fonction **COMPARE**, tout est automatisé dans le hardware en intégrant un comparateur câblé et une logique de commande de la broche **CCP** également câblée. Et donc des temps de réponse réduits à leur plus petite valeur...

Ce n'est pas directement le temps t_C qu'il faut écrire dans le registre **COMPARE**, mais une valeur N_C (par exemple $N_C = 0x453C$ dans l'exemple de la figure VIII.100.), donnée par :

$$N_C = \frac{t_C}{T_{\text{tick}}}$$

où T_{tick} est la période des ticks d'horloge incrémentant le **TIMER**.

Dans cet exemple, on vient de repérer un instant en absolu, c'est-à-dire par rapport à un instant initial choisi par convention.

Mais, ce n'est pas la seule possibilité d'utilisation de la fonction de comparaison.

Une variante (très utilisée) de la fonction **COMPARE** permet de fabriquer une impulsion TTL (positive ou négative) de largeur T souhaitée.

Le principe (donné ici pour fabriquer une impulsion positive) est très simple :

- on initialise la fonction **COMPARE** pour que la broche de **sortie CCP** génère un front montant à chaque comparaison valide,
- on initialise le registre **COMPARE** avec une valeur N_{C1} (exprimée en nombre de ticks d'horloge du **TIMER**) correspondant à l'instant t_{C1} souhaité pour le début de l'impulsion,
- à $t = t_{C1}$, au moment de la première comparaison valide, la sortie **CCP** génère automatiquement le front montant de l'impulsion, comme sur la figure VIII.100. On vient alors, par programme, rajouter au registre **COMPARE**, la valeur N (en nombre de ticks d'horloge du **TIMER**) correspondant à la durée T souhaitée pour l'impulsion. Puis on modifie la nature du front actif qui sera généré sur la sortie **CCP** à la prochaine comparaison valide (en choisissant un front actif **descendant**),
- à $t = t_{C2}$, au moment de la prochaine comparaison valide, la sortie **CCP** génère automatiquement le front descendant de l'impulsion. Et c'est fini !!!

Mais attention, si la valeur N est supérieure à la durée de débordement du **TIMER** (cas des longues impulsions par exemple), il faudra prendre en compte les débordements du registre **TIMER** pour avoir la bonne largeur d'impulsions...

Fabriquer des signaux TTL de tout type de forme devient ainsi possible... et facile.

VIII.6.2.2. SYNOPTIQUE

Sur le PIC, la fonction de comparaison utilise également le **TIMER1** comme référence temporelle.

Cette fonction peut-être réalisée au choix par **l'un ou l'autre** des deux modules **CCPi**.

On peut même implanter deux fonctions de comparaison **indépendantes**, une sur **CCP1** et l'autre sur **CCP2**, mais elles partageront alors la même base de temps fournie par le **TIMER1**.

Par contre, si les fonctions de capture sont identiques sur **CCP1** et **CCP2**, quelques différences (minimes) existent entre les fonctions de comparaison de **CCP1** et de **CCP2**, pour le mode **SPECIAL EVENT TRIGGER** (terme utilisé dans la documentation **MICROCHIP**) qu'on pourrait traduire par mode de **SYNCHRONISATION (HARDWARE) SUR EVENEMENT SPECIAL** (voir plus loin la remarque expliquant ces différences et pourquoi elles ont été implantées).

Le registre **COMPARE** pour **CCP1** est le registre **CCPR1** et celui de **CCP2** est **CCPR2**. (les mêmes que pour la fonction **CAPTURE** !!!).

La sortie de comparaison valide est appelée **CCP1** (c'est la ligne **RC2** du **PORTC**) pour le module **CCP1** (la même que pour la fonction de capture !!!).

La sortie de comparaison valide est appelée **CCP2** (c'est la ligne **RC1** du **PORTC**) pour le module **CCP2** (la même que pour la fonction de capture !!!).

Lorsque la fonction de comparaison est utilisée, ces lignes **doivent être programmées** en sortie via le registre **TRISC**

Mais vous l'avez compris, à **l'intérieur d'un même module CCPi** les fonctions de capture et de comparaison utilisent des ressources hardware communes : elles sont donc exclusives l'une de l'autre.

Par exemple, si le module **CCP1** fonctionne dans le mode de capture, il ne peut pas **au même moment** fonctionner dans le mode de comparaison. Et réciproquement. Idem pour **CCP2**.

Par contre rien n'interdit :

- de faire une capture avec **CCP1** et une comparaison avec **CCP2** (et réciproquement),
- de faire à certains moments une capture avec **CCP1** et à un autre moment de faire une comparaison avec ce même module. Idem pour **CCP2**.

Le synoptique simplifié de l'une ou l'autre de la fonction compare (dans **CCP1** ou **CCP2**) est le quasiment le même (sauf pour le mode **SPECIAL EVENT TRIGGER** comme on l'a dit précédemment). Il est donné sur la figure VIII.101. pour le module **CCP1**.

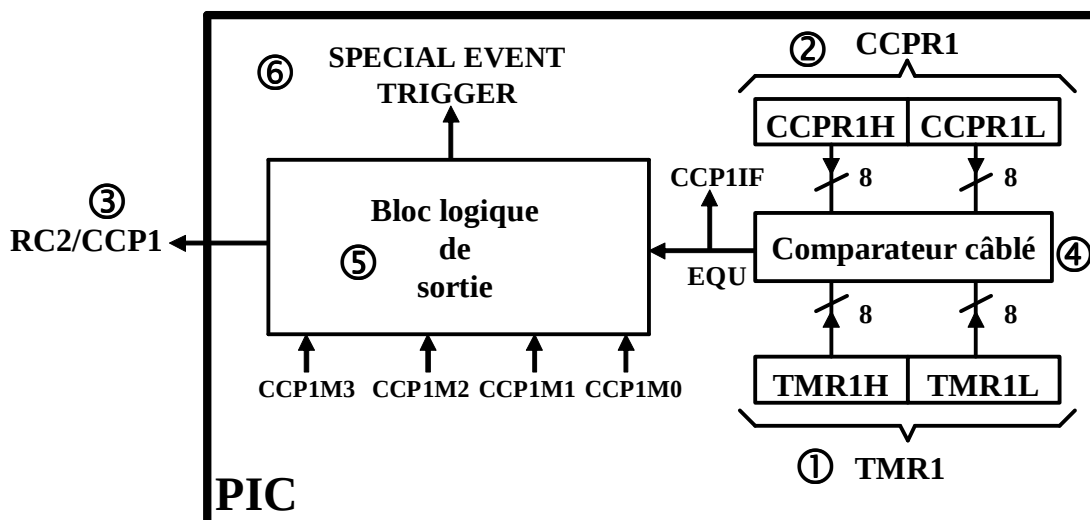


Figure VIII.101.

On y retrouve les blocs qu'on avait présentés dans le chapitre sur les généralités :

- bloc ① : le registre **TIMER**, ici réalisé par le registre **TMR1**,

Les ticks d'horloge incrémentant le registre **TMR1** n'ont pas été représentés (se référer au synoptique de la figure VIII.78.). Par contre, il faut noter que, là aussi, si on utilise le **TIMER1** en mode compteur extérieur asynchrone, la fonction de comparaison ne fonctionnera pas.

- bloc ② : le registre **COMPARE** appelé ici **CCPR1** (ou **CCPR2**),
- ③ : la sortie de comparaison valide : **CCP1/RC2** (ou **CCP2/RC1**).

Le comparateur câblé 16 bits (cf. bloc ④) compare en permanence et en **temps réel** le contenu des registres **CCPR1** (ou **CCPR2**) et **TMR1**. Sa sortie **EQU** passera à 1, dès qu'il y aura égalité. Ce qui provoquera aussi la mise à 1 du flag **CCP1IF** (ou **CCP2IF**).

Pour gérer ce flag, la technique du polling est toujours possible, mais il est préférable, surtout dans les cas où les contraintes temporelles sont « serrées » d'utiliser les interruptions en validant le bit correspondant **CCP1IE** (situé dans le registre SFR **PIE1**) ou **CCP2IE** (situé dans **PIE2**).

Enfin, le bloc logique de sortie (cf. bloc ⑤) permettra de choisir le front actif qui sera généré sur la patte de sortie **CCP1** (ou **CCP2**), en fonction des 4 bits **CCPiM3** à **CCPiM0** (voir figure VIII.103. plus loin pour plus de détails).

Remarque sur le mode SPECIAL EVENT TRIGGER :

Il est temps de dire quelques mots sur la partie ⑥ du schéma de la figure VIII.101. permettant de générer un signal **SPECIAL EVENT TRIGGER**.

Dans ce mode de fonctionnement un peu particulier de la fonction de comparaison, et pour les deux modules **CCP1** et **CCP2**, ce signal est généré à chaque comparaison valide et va venir faire un reset (remise à zéro) du registre **TMR1** (sans mettre à 1 le flag **TMR1IF**).

Pourquoi cette remise à zéro de **TMR1** ?

Tout simplement pour ne pas avoir à gérer le registre **CCPRi** à chaque comparaison valide (rappelez-vous, pour générer un signal en sortie, il fallait rajouter une valeur au registre **CCPRi** pour préparer la comparaison suivante). Le registre **CCPRi** devient alors l'équivalent d'un registre de **période** pour le **TIMER1**.

Mais attention, dans ce mode, les broches de sortie **CCPi** restent inchangées. Ce sera au programme de les gérer, par exemple en les inversant à chaque comparaison valide).

Et c'est là qu'intervient la différence entre les modules **CCP1** et **CCP2** :

si dans ces deux modules le signal **SPECIAL EVENT TRIGGER** provoque bien un reset du registre **TMR1**, en plus dans le module **CCP2**, il provoque le lancement d'une conversion analogique/numérique (si le **CAN** bien sûr est validé).

Pourquoi cette action sur le CAN ?

Tout simplement pour faciliter la numérisation d'un signal analogique : il suffit pour cela d'écrire dans le registre **CCPR2** la valeur de la période **d'échantillonnage T_e** (exprimée toujours en nombre de ticks d'horloge du **TIMER1**) et une acquisition sera automatiquement lancée à chaque interruption du module **CCP2**.

VIII.6.2.3. REGISTRES DE CONTRÔLE ASSOCIES

La plupart des bits de configuration que l'on vient de présenter se trouvent dans les deux registres SFR **CCP1CON** (comme **CCP1 CONtroll**) pour le module **CCP1** et **CCP2CON** (comme **CCP2 CONtroll**) pour le module **CCP2**, déjà présentés pour la fonction **CAPTURE** et dont le format **commun** est rappelé sur la figure VIII.102.

**CCP1CON/CCP2CON (situés aux adresses respectives 0x017 et 0x01D)
(page 64 du datasheet)**

	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CCP1CON			CCPiX	CCPiY	CCPiM3	CCPiM2	CCPiM1	CCPiM0
CCP2CON	–	–						
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.102.

La signification de ces bits a été expliquée sur la figure VIII.101. précédente et est résumée sur la figure VIII.103.

Nous n'indiquerons ici que les combinaisons concernant la fonction de comparaison.
(pour les autres fonctions se reporter à leurs chapitres respectifs)

Bit	Valeur	Signification et commentaires
CCPiX :CCPiY		Voir fonction PWM
CCPiM3 :CCPiM0		Pour choisir le mode de fonctionnement
	0000	Modules CCPi non actif (mise à zéro des sorties CCPi)
	1000	Génération d'un front montant sur la sortie CCPi à chaque comparaison valide (il faut qu'au préalable la broche CCPi soit au niveau bas !!!)
	1001	Génération d'un front descendant sur la sortie CCPi à chaque comparaison valide (il faut qu'au préalable la broche CCPi soit au niveau haut !!!)
	1010	Ne fait que mettre le flag CCPiIF à 1 à chaque comparaison valide (n'agit pas sur la sortie CCPi) Mode dit d'interruption logicielle
	1011	Mode de fonctionnement SPECIAL EVENT TRIGGER (n'agit pas sur la sortie CCPi) (voir aussi remarque précédente sur ce mode particulier)
	x1xx	Autres modes de fonctionnement

Figure VIII.103.

VIII.6.2.4. EXEMPLE DE MISE EN OEUVRE

Nous nous proposons ici de générer un signal carré TTL sur la broche de sortie **RC2/CCP1** du PIC.

La fréquence souhaitée est de $f = 10 \text{ kHz}$.

On va utiliser le module **CCP1** dans le mode **SPECIAL EVENT TRIGGER** sous interruption.

Calculs préliminaires :

La période T est de $\frac{1}{10 \cdot 10^3} = 100 \text{ } \mu\text{s}$.

Comme ce mode de fonctionnement n'agit pas sur la broche **RC2/CCP1**, il faudra inverser le signal de sortie toutes les demi périodes (voir instruction $\text{RC2} = \sim \text{RC2}$; dans la routine d'interruption).

De plus, dans ce mode, le **TIMER1** fonctionne de **0x0000** à la valeur contenue dans le registre **CCPR1** (= **CCPR1H :CCPR1L**), avant de générer une interruption. Il faut donc que l'intervalle de temps entre deux interruptions successives soit de $50 \text{ } \mu\text{s}$.

Si on choisit de prendre $p = 1$ pour le pré diviseur du **TIMER1**, alors $T_{\text{tick}} = 4 \cdot p \cdot T_{\text{OSC}} = 1 \text{ } \mu\text{s}$. (toujours en supposant un quartz à 4 MHz)

Il faudra donc initialiser **CCPR1** avec 50 (soit **CCPR1H = 0x00** et **CCPR1L = 0x32**).

On peut aussi paramétrer la période grâce à une variable du même nom et de taille 16 bits, donc déclarée comme **short int**. Pour initialiser le registre **CCPR1** il faudra par exemple écrire :

```
CCPR1H=periode>>8;           // octet fort de la variable periode dans CCPR1H
CCPR1L=periode;               // octet faible de la variable periode dans CCPR1L
```

Ce qui permettrait par exemple de faire varier la fréquence (comme dans un générateur de fonctions).

Le programme correspondant est donné sur la figure VIII.104.

```
short int periode=50;

// la routine d'interruption
void interrupt interruption (void)
{
    if (CCP1IE && CCP1IF)           // daisy chain
    {
        CCP1IF=0;                  // acquittement de l'IT CCP1
        RC2=~RC2;                  // inversion de la sortie RC2/CCP1
    }
}
```

Figure VIII.104.

```

// initialisation du PORTC
void init_PORTC (void)
{
    TRISC2=0;                // RC2 = CCP1 = output
}

void init_timer1(void)
{
    // initialisation de T1CON
    // T1CKPS1 T1CKPS0=00    // p = 1 pour Tick=1 microseconde
    // T1OSCEN=X             // indifférent car pas de quartz extérieur
    // T1SYNC=X              // indifférent car déjà synchrone
    // TMR1CS=0              // quartz du PIC
    // TMR1ON=1              // démarre TIMER1
    T1CON = 0b00000001;     // valeur d'initialisation
}

// fonction output compare en mode SPECIAL EVENT TRIGGER
void init_CCP1()
{
    // initialisation de CCP1CON
    // CCP1X = X             // non utilisé
    // CCP1Y = X             // non utilisé
    // CCP1M3:CCP1M0 = 1011 // mode Special Event Trigger
    CCP1CON=0b00001011;     // valeur d'initialisation
    CCPR1H=0x00;            // valeur de la demie période
    CCPR1L=0x32;            // ici 50 microsecondes
    //CCPR1H=periode>>8;    // pour paramétrer la période
    //CCPR1L=periode;
}

void init_interruption(void)
{
    CCP1IF=0;
    CCP1IE=1;                // autorisation des IIT du module CCP1
    PEIE=1;                  // démasquage semi global
}

void main(void)
{
    init_PORTC();
    init_timer1();
    init_CCP1();
    init_interruption();
    GIE=1;                   // démasquage global des interruptions
    while(1);                // ne fait rien !!!
}

```

Figure VIII.104. (suite et fin)

Voici en résumé sur la figure VIII.105. les registres concernés par les fonctions capture et compare ainsi que les différents bits concernés.

Remarquons que les registres associés au **TIMER1** sont également présents dans cette liste.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
0Dh	PIR2	—	—	—	—	—	—	—	CCP2IF	---- --0	---- --0
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
8Dh	PIE2	—	—	—	—	—	—	—	CCP2IE	---- --0	---- --0
87h	TRISC	PORTC Data Direction Register								1111 1111	1111 1111
0Eh	TMR1L	Holding Register for the Least Significant Byte of the 16-bit TMR1 Register								xxxx xxxx	uuuu uuuu
0Fh	TMR1H	Holding Register for the Most Significant Byte of the 16-bit TMR1 Register								xxxx xxxx	uuuu uuuu
10h	T1CON	—	—	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON	--00 0000	--uu uuuu
15h	CCPR1L	Capture/Compare/PWM Register1 (LSB)								xxxx xxxx	uuuu uuuu
16h	CCPR1H	Capture/Compare/PWM Register1 (MSB)								xxxx xxxx	uuuu uuuu
17h	CCP1CON	—	—	CCP1X	CCP1Y	CCP1M3	CCP1M2	CCP1M1	CCP1M0	--00 0000	--00 0000
1Bh	CCPR2L	Capture/Compare/PWM Register2 (LSB)								xxxx xxxx	uuuu uuuu
1Ch	CCPR2H	Capture/Compare/PWM Register2 (MSB)								xxxx xxxx	uuuu uuuu
1Dh	CCP2CON	—	—	CCP2X	CCP2Y	CCP2M3	CCP2M2	CCP2M1	CCP2M0	--00 0000	--00 0000

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'. Shaded cells are not used by Capture and Timer1.

Figure VIII.105.

VIII.6.3. LA FONCTION PULSE WIDTH MODULATION

VIII.6.3.1. GENERALITES SUR LA FONCTION PWM

Tout d'abord, qu'entend-on par signal **PWM** ?

C'est un signal **périodique** $S(t)$ dont la forme est fournie sur la figure VIII.106. (où on a de plus supposé un niveau haut compatible TTL).

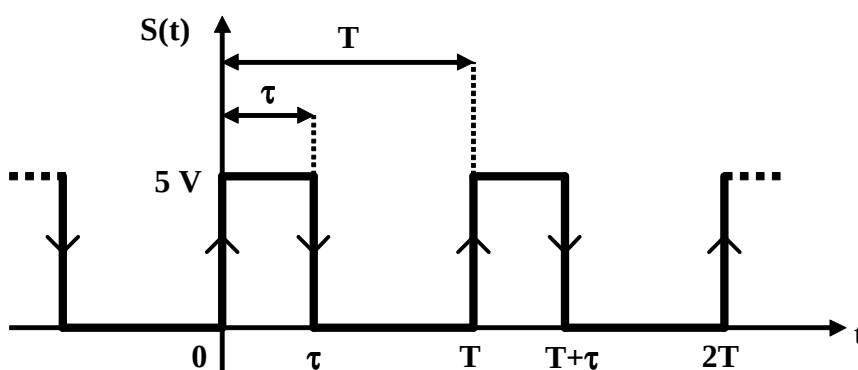


Figure VIII.106.

Ce signal possède deux paramètres importants :

- sa **période** T (ou sa fréquence $f = \frac{1}{T}$),
- sa **durée** τ à l'état haut (avec bien sûr $0 \leq \tau \leq T$).

En pratique, au lieu de donner τ , on peut aussi raisonner avec le **rapport cyclique** α défini par :

$$\alpha = \frac{\tau}{T} \quad (\text{exprimé souvent en \%})$$

Il est en effet facile de retrouver τ lorsqu'on connaît T et α .

Ce type de signal est très utile dans les systèmes de **variation de vitesse** des moteurs à courant continu. Il est souvent appelé alors signal **MLI** (comme **M**odulation de **L**argeur d'**I**mpulsions).

On le trouve aussi dans l'électronique permettant de réaliser des **alimentations à découpage**.

Certains **amplificateurs audio** (dits de classe **D**) pilotent leur étage de puissance avec un signal de cette forme. Les transistors de puissance de cet étage fonctionnent alors, non pas en régime linéaire comme dans les amplificateurs de classe **A** ou **B** mais en régime « **bloqué-saturé** ».

Mais ce signal PWM peut aussi servir dans certains procédés de modulation appelé justement **Pulse Width Modulation** (Modulation par largeur d'Impulsions).

Ce signal $S(t)$ sera sorti sur une des broches du PIC appelée **CCP**.

Comme pour les fonctions de capture et de comparaison, il s'appuie sur un **TIMER**.

Le principe de génération est assez simple :

- un premier registre **PERIODE** contient la valeur de **T** (exprimée toujours en nombre de ticks d'horloge du **TIMER**),
- un second registre **TO** contient la valeur de τ (exprimée aussi en nombre de ticks d'horloge du **TIMER**),

Quand **TIMER** = **PERIODE** (au début de chaque période de $S(t)$: instants de la forme $k.T$ avec k entier), les trois actions suivantes sont automatiquement (et simultanément) effectuées :

- remise à zéro du registre **TIMER**,
- mise à l'état **haut** de la sortie **CCP** (génération d'un front **montant**),
- chargement du registre **TO** avec la valeur de τ (exprimée aussi en nombre de ticks du **TIMER**).

Quand **TIMER** = **TO** (aux instants de la forme $k.T + \tau$ avec k entier), l'action suivante est automatiquement effectuée :

- mise à l'état **bas** de la sortie **CCP** (génération d'un front **descendant**).

Et ainsi de suite...

Remarque 1 :

Si la valeur contenue dans le registre **TO** est **supérieure** à celle contenue dans **PERIODE**, alors la sortie **CCP** restera en permanence au niveau **haut** (rapport cyclique $\alpha = 100\%$).

Remarque 2 :

Si la valeur contenue dans **TO** est **nulle**, la sortie **CCP** restera en permanence au niveau **bas**. (rapport cyclique $\alpha = 0\%$).

VIII.6.3.2. SYNOPTIQUE

Sur le PIC, la fonction **PWM** utilise le **TIMER2** comme référence temporelle.

Cette fonction peut-être réalisée au choix par l'un ou l'autre des deux modules **CCPi**.

On peut même implanter deux fonctions **PWM indépendantes**, une sur **CCP1** et l'autre sur **CCP2**, mais elles partageront alors la même base de temps fournie par le **TIMER2**.

Les deux signaux **PWM** générés auront donc la **même période**, mais pourront avoir **des rapports cycliques différents**.

De même on peut utiliser la fonction **PWM** avec le module **CCP1** et soit la fonction **CAPTURE**, soit la fonction **COMPARE** sur **CCP2**, et ceci sans aucune limitation.
Et réciproquement.

Le registre **PERIODE** pour **CCP1** (et aussi pour **CCP2**) est le registre **PR2** du **TIMER2**.
(ce qui confirme bien que si la fonction **PWM** est utilisée avec **CCP1** et **CCP2**, les deux signaux **PWM** générés auront la **même période**).

La durée τ du signal **PWM** est codée sur 10 bits. Comment cela ?

Le registre **TO** pour **CCP1** (**CCP2**) est le registre **CCPR1L** (**CCPR2L**) qui contient les 8 bits de poids forts de la durée τ (exprimée en nombre de ticks d'horloge du **TIMER2**). Ses 2 bits de poids faibles sont les bits **CCP1X** et **CCP1Y** (**CCP2X** et **CCP2Y**) du registre **CCP1CON** (**CCP2CON**). (voir plus loin les figures VIII.108. et VIII.109. pour plus de détails).

(ce qui confirme bien que si la fonction **PWM** est utilisée avec **CCP1** et **CCP2**, les deux signaux **PWM** générés auront bien **deux rapports cycliques différents**).

La sortie PWM est appelée **CCP1** (c'est la ligne **RC2** du **PORTC**) pour le module **CCP1** (toujours la même que pour les fonctions de capture et de comparaison !!!).

La sortie PWM est appelée **CCP2** (c'est la ligne **RC1** du **PORTC**) pour le module **CCP2** (toujours la même que pour les fonctions de capture et de comparaison !!!).

Lorsque la fonction **PWM** est utilisée, ces lignes **doivent être programmées** en **sortie** via le registre **TRISC**

A la découverte du PIC 16F877



On y retrouve les blocs qu'on avait présentés dans le chapitre sur les généralités :

- Les ticks d'horloge incrémentant le registre **TMR2** n'ont pas été représentés (se référer au synoptique de la figure VIII.86.). Par contre, le post diviseur **p'** n'est pas pris en compte dans le calcul de la période du signal **PWM** comme on va le voir un peu plus loin.

- bloc ② : le registre de **PERIODE** est le registre **PR2** du **TIMER2**
- ③ : la sortie **CCP** véhiculant le signal **PWM** est **CCP1/RC2** pour le module **CCP1** (ou **CCP2/RC1** pour le module **CCP2**)
- bloc ④ : le registre **TO** réalisé ici par les deux registres **CCPR1L** et **CCPR1H** : **CCPR1L** est en lecture/écriture. Par contre **CCPR1H** n'est qu'en lecture dans le mode **PWM**. En fait, le registre **CCPR1L** sert de tampon (buffer) au registre **CCPR1H**. Les 2 bits de poids faibles de τ sont les bits **CCP1X** et **CCP1Y** du registre **CCP1CON**.

Le **comparateur câblé 1** 8 bits (cf. bloc ⑤) compare en permanence et en **temps réel** le contenu des registres **TMR2** et **PR2**.

Sa sortie **EQUA** passera à 1, dès qu'il y aura égalité (à tous les instants de la forme $k.T$).

Ce qui provoquera les 3 actions suivantes :

- remise à 0 du registre **TMR2** : via le signal **RESET**,
- mise à 1 de l'entrée **S (SET)** de la bascule **RS** de sortie (bloc ⑦), ce qui provoquera la mise à 1 de sa sortie **Q**, et donc aussi de la broche de sortie **CCP1**,
- chargement des 8 bits de **CCPR1L** dans **CCPR1H** (ainsi que des 2 bits **CCP1X** et **CCP1Y** dans les 2 cases **TO1** et **TO0**) par le signal **LOAD TO**. La durée τ est bien alors codée sur 10 bits. Le fait de « double-bufferiser » la durée τ permet de la modifier par programme à n'importe quel moment, mais la valeur qui sera réellement prise en compte sera celle qui passera de **CCPR1L** à **CCPR1H** à cet instant. Cela permet d'éviter ce qu'on appelle en anglais des « **glitch** » (c'est-à-dire des légers « tremblements ») sur le front descendant du signal de sortie **PWM**.

On peut donc calculer les valeurs des paramètres caractéristiques **T** et τ du signal **PWM** :

(1) : $T = 4.T_{OSC}.p.(PR2+1)$ où : T_{OSC} est la période du quartz du PIC
 p est la valeur du pré diviseur choisi pour le **TIMER2**
(1, 4, 16 ou 64)

(2) : $\tau = T_{OSC}.p.CCPR1$ où : T_{OSC} est la période du quartz du PIC
 p est la valeur du pré diviseur choisi pour le **TIMER2**
(1, 4, 16 ou 64)
CCPR1 représente les 10 bits: **CCPR1L** : **CCP1X** : **CCP1Y**.

Le **comparateur câblé 2** 10 bits (cf. bloc ⑥) compare en permanence et en **temps réel** le contenu du registre **TMR2** (auquel ont été concaténés, en guise de poids faible, les deux bits provenant du pré diviseur p (si $p > 1$) ou de l'horloge interne **Q** si $p = 1$) et du registre **CCPR1H** (auquel ont été concaténés, en guise de poids faible, les deux bits **TO1** et **TO0** provenant des bits **CCP1X** et **CCP1Y** du registre **CCP1CON**).

Sa sortie **EQUB** passera à 1, dès qu'il y aura égalité (à tous les instants de la forme $k.T + \tau$).

Ce qui induira l'action suivante :

- mise à 1 de l'entrée **R (RESET)** de la bascule **RS** de sortie (bloc ⑦), provoquant ainsi la mise à 0 de sa sortie **Q**, et donc aussi de la broche de sortie **CCP1**.

VIII.6.3.3. REGISTRES DE CONTRÔLE ASSOCIES

La plupart des bits de configuration que l'on vient de présenter se trouvent dans les deux registres SFR **CCP1CON** (comme **CCP1 CON**trol) pour le module **CCP1** et **CCP2CON** (comme **CCP2 CON**trol) pour le module **CCP2**, déjà présentés pour les fonctions **CAPTURE** et **COMPARE** et dont le format commun est rappelé sur la figure VIII.108.

**CCP1CON/CCP2CON (situés aux adresses respectives 0x017 et 0x01D)
(page 64 du datasheet)**

	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CCP1CON			CCPiX	CCPiY	CCPiM3	CCPiM2	CCPiM1	CCPiM0
CCP2CON	–	–						
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.108.

La signification de ces bits a été expliquée sur la figure VIII.107. précédente et est résumée sur la figure VIII.109.

Nous n'indiquerons ici que les combinaisons concernant la fonction **PWM**.
(pour les autres fonctions se reporter à leurs chapitres respectifs)

Bit	Valeur	Signification et commentaires
CCPiX :CCPiY	xx	Les deux bits de poids faibles de la durée τ à l'état haut du signal PWM . (les huit bits de poids forts de τ étant contenus dans CCPRiL)
CCPiM3 :CCPiM0		Pour choisir le mode de fonctionnement
	0000	Modules CCPi non actif (mise à zéro des sorties CCPi)
	11xx	Mode PWM
	x̄xxx	Autres modes de fonctionnement

Figure VIII.109.

VIII.6.3.4. EXEMPLES DE MISE EN OEUVRE

Exemple 1 : rapport cyclique **fixe**.

Nous nous proposons ici de générer un signal **PWM** dont les caractéristiques sont les suivantes :

- fréquence : $f = 5 \text{ kHz}$,
- rapport cyclique : $\alpha = 20 \%$.

On va utiliser le module **CCP1** dans le mode **PWM**.

Le programme cette fois-ci ne fonctionne pas en utilisant les interruptions car le signal **PWM** est **entièrement** et **automatiquement** généré par le hardware sur la broche **RC2/CCP1** du PIC.

Calculs préliminaires :

On obtient : $T = \frac{1}{f} = 200 \mu\text{s}$.

De plus : $\tau = \alpha.T = 0,2.200 = 40 \mu\text{s}$.

Les formules précédentes, avec un quartz de 4 MHz ($T_{\text{OSC}} = 0,25 \mu\text{s}$) nous donnent :

(1) : $T = 4.T_{\text{OSC}}.p.(PR2+1)$ soit : $200 = 4.0,25.p.(PR2+1)$ et donc : **PR2 = 199**.
(en prenant la valeur du pré diviseur **p** du **TIMER2** la plus petite possible, soit $p = 1$)

(2) : $\tau = T_{\text{OSC}}.p.CCPR1$ soit : $40 = 0,25.CCPR1$ et donc **CCPR1 = 160**.

Il faut se rappeler que la valeur **CCPR1** de la formule **(2)** est codée sur **10 bits**.

Si on choisit de prendre ses 2 bits de poids faibles **CCP1X** et **CCP1Y** tous les deux à 0, il vient :

$CCPR1L = \frac{160}{4}$ et donc **CCPR1L = 40**.

Le programme générant ce signal **PWM** sur la broche **RC2/CCP1** est donné sur la figure VIII.110. Comme d'habitude, lisez bien les commentaires.

```
// initialisation du PORTC
void init_PORTC (void)
{
    TRISC2=0;                // RC2 = CCP1 = output
}

// le timer2 utilisé par le module CCP1
void init_timer2(void)
{
    // initialisation de T2CON
    // TOUTPS3:TOUTPS0 = 0000    // p'=1
    // TMR2ON=1                  // timer 2 on
    // T2CKPS1 T2CKPS0 = 00      // p=1 (voir calcul préalable précédent)
    T2CON = 0b00000100;         // valeur d'initialisation
    PR2=199;                    // pour avoir T = 200 microsecondes
                                // (voir calcul préalable précédent)
}

// initialisation de CCP1 en mode PWM
void init_CCP1()
{
    // initialisation de CCP1CON
    // CCP1X=0                    // TO1 de PWM (voir calcul préalable précédent)
    // CCP1Y=0                    // TO0 de PWM (voir calcul préalable précédent)
    // CCP1M3:CCP1M0 = 1100      // mode PWM
    CCP1CON=0b00001100;         // valeur d'initialisation
    CCPR1L=40;                  // pour avoir durée état haut de 40 microsecondes
                                // (voir calcul préalable précédent)
}

void main(void)
{
    init_PORTC();
    init_timer2();
    init_CCP1();
    while(1);                   // ne fait rien
}
```

Figure VIII.110.

Exemple 2 : rapport cyclique **variable** : rampe de démarrage (ou d'arrêt) pour un moteur à courant continu.

Il faut se rappeler que la vitesse de ce type de moteur est proportionnelle à la tension V_{mot} appliquée aux bornes du moteur (au moins dans le cas où le courant absorbé par le moteur n'est pas trop important).

Si on regarde la forme d'un signal **PWM** (cf. figure VIII.106.), on peut noter que sa valeur moyenne peut s'écrire $V_{\text{moy}} = E \cdot \frac{\tau}{T} = \alpha \cdot E$, en appelant E sa valeur au niveau haut. On voit que cette tension est proportionnelle au rapport cyclique α .

Si on envoie cette tension (via bien sûr un étage de puissance comme un pont en H) aux bornes d'un moteur à courant continu, la vitesse du moteur sera proportionnelle à α .

Ainsi :

- si $\alpha = 0 \%$: $V_{\text{mot}} = V_{\text{moy}} = 0$ et la vitesse du moteur est **nulle**,
- si $\alpha = 100 \%$: $V_{\text{mot}} = V_{\text{moy}} = E$ et la vitesse du moteur est **maximale**.

Pour certaines applications, il est intéressant de fabriquer une **rampe de démarrage**, c'est-à-dire un procédé permettant de passer **progressivement** de l'arrêt du moteur à sa vitesse maximale et permettre ainsi un démarrage « en douceur » du moteur.
(idem pour obtenir un arrêt progressif par une rampe d'arrêt du moteur).

L'idée est donc de faire varier le rapport cyclique du signal **PWM** entre 0 % et 100 % pendant la durée T_R de cette rampe, en choisissant souvent une variation de type linéaire, comme indiqué sur la figure VIII.111.

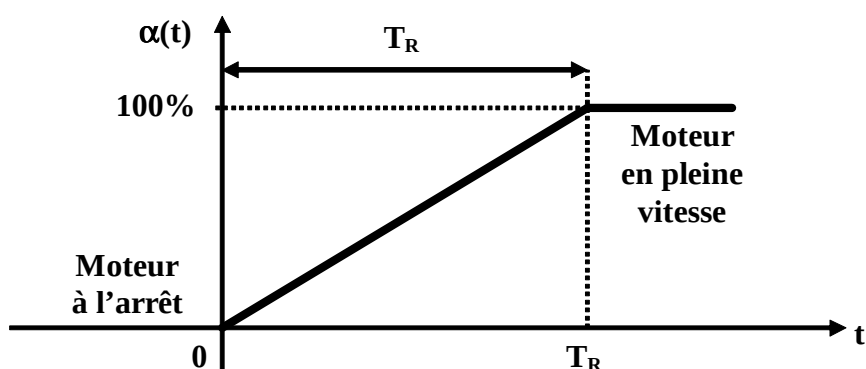


Figure VIII.111.

Nous allons essayer de paramétrer cette rampe grâce à « une série de #define » comme indiqué sur la figure VIII.112. que nous intégrerons dans un fichier **pwm.h** attaché à notre projet.

```
// paramétrage de la rampe

// valeur de la durée TR de la rampe (de démarrage ou d'arrêt) en secondes
#define      duree_rampe      10

// fréquence en kHz du PWM généré sur la sortie RC2 du PIC
// attention avec la valeur de p choisi (p=1) ne peut être inférieure à 3,9 kHz
#define      frequence        5

// période en microsecondes du PWM généré sur la sortie RC2 du PIC
#define      periode          (1000./frequence)

// nombre de pas de variation du rapport cyclique pendant les rampes
#define      nombre_pas        periode      // car incrément de CCPR1L de 1
                                           // (entre 0 et periode)
                                           // à chaque variation du rapport cyclique

// intervalle de temps (en ms) entre chaque variation du rapport cyclique pendant les rampes
#define      delta_t           (duree_rampe*1000.)/nombre_pas
```

Figure VIII.112.

Le programme permettant de générer ces rampes est fourni sur la figure VIII.113.

Au départ :

- on attend l'appui sur le bouton poussoir **S2** de la carte **PICDEM 2 PLUS**,
- on lance une rampe de démarrage d'une durée de 10 secondes, mais qu'on pourrait modifier grâce au #define `duree_rampe`,
- lorsque le rapport cyclique atteint 100 %, on attend un nouvel appui sur **S2**,
- on lance une rampe d'arrêt de 10 secondes pour stopper le moteur,
- lorsque le rapport cyclique atteint 0 %, on attend un nouvel appui sur **S2** et ainsi de suite...

Nous utilisons aussi le LCD pour afficher quelques messages pendant le déroulement de ce programme.


```

#include "pwm.h"

// initialisation du PORTC
void init_PORTC (void)
{
    TRISC2=0;                                // RC2 = CCP1 = output
}

// initialisation du PORTA
void init_PORTA (void)
{
    TRISA4=1;                                // RA4 = Bouton Poussoir S2 = input
}

// le timer2 utilisé par le module CCP1 en PWM
void init_timer2(void)
{
    // initialisation de T2CON
    // TOUTPS3:TOUTPS0 = 0000                // p'=1
    // TMR2ON=1                             // timer 2 on
    // T2CKPS1 T2CKPS0 = 00                 // p=1
    T2CON = 0b00000100;                    // valeur d'initialisation
    PR2=periode-1;                         // par periode du #define
}

// initialisation de CCP1 en mode PWM
void init_CCP1()
{
    // initialisation de CCP1CON
    // CCP1X=0                             // T01 de PWM
    // CCP1Y=0                             // T00 de PWM
    // CCP1M3:CCP0M0 = 1100                // mode PWM
    CCP1CON=0b00001100;                    // valeur d'initialisation
}

```

Figure VIII.113.

```

void main(void)
{
    init_PORTD();           // fixe la direction des lignes du PORTD
    allume_LCD();           // allume le LCD
    init_LCD();             // initialise le LCD
    goto_ligne_1();
    print_string("RAMPE DEMARRAGE");
    goto_ligne_2();
    print_string("VISUALISER RC2");
    tempo_N_ms(5000);
    efface();

    goto_ligne_1();
    print_string("appuuyer sur S2");

    init_PORTC();
    init_PORTA();

    init_timer2();

    init_CCP1();

    CCPR1L=0;               // initialisation durée à l'état haut
    pente=1;                // rampe de démarrage d'abord pente = 1
                           // pour rampe d'arrêt, on fera pente = -1

    while(1)
    {
        while(RA4);         // attend appui sur S2

        tempo_N_ms(200);    // anti_rebonds

        do
        {
            tempo_N_ms(delta_t); // entre chaque changement de durée à l'état haut
            CCPR1L=CCPR1L + pente; // changement linéaire de la durée à l'état haut
        }
        while((CCPR1L<=periode) && (CCPR1L>=0)); // limites pour rampe

        pente = -pente;      // changement type de rampe
        CCPR1L=CCPR1L+pente; // pour recommencer
    }
}

```

Figure VIII.113. (suite et fin)

Voici au final sur la figure VIII.114. les registres concernés par la fonction **PWM** ainsi que les différents bits concernés.

Remarquons que les registres associés au **TIMER2** sont également présents dans cette liste.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
0Dh	PIR2	—	—	—	—	—	—	—	CCP2IF	---- --0	---- --0
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
8Dh	PIE2	—	—	—	—	—	—	—	CCP2IE	---- --0	---- --0
87h	TRISC	PORTC Data Direction Register								1111 1111	1111 1111
11h	TMR2	Timer2 Module's Register								0000 0000	0000 0000
92h	PR2	Timer2 Module's Period Register								1111 1111	1111 1111
12h	T2CON	—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0	-000 0000	-000 0000
15h	CCPR1L	Capture/Compare/PWM Register1 (LSB)								xxxx xxxx	uuuu uuuu
16h	CCPR1H	Capture/Compare/PWM Register1 (MSB)								xxxx xxxx	uuuu uuuu
17h	CCP1CON	—	—	CCP1X	CCP1Y	CCP1M3	CCP1M2	CCP1M1	CCP1M0	--00 0000	--00 0000
1Bh	CCPR2L	Capture/Compare/PWM Register2 (LSB)								xxxx xxxx	uuuu uuuu
1Ch	CCPR2H	Capture/Compare/PWM Register2 (MSB)								xxxx xxxx	uuuu uuuu
1Dh	CCP2CON	—	—	CCP2X	CCP2Y	CCP2M3	CCP2M2	CCP2M1	CCP2M0	--00 0000	--00 0000

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'. Shaded cells are not used by PWM and Timer2.

Figure VIII.114.

VIII.7. LA LIAISON SERIE ASYNCHRONE/SYNCHRONE

(cf. ⑪ du schéma de la **figure II.4.** et cf. **pages 111 à 125** du datasheet)

Appelée aussi **U.S.A.R.T.** (comme **U**niversal **S**ynchronous **A**synchronous **R**eceiver **T**ransmitter), la liaison série **SCI** (comme **S**erial **I**nterface **C**ommunication) peut fonctionner en mode **asynchrone** (les **données série peuvent arriver n'importe quand** sur la liaison) ou en mode **synchrone** (nécessitant alors une **horloge de synchronisation** pour la liaison).

VIII.7.1. GENERALITES SUR LES LIAISONS

Si l'on désire transférer des informations entre deux systèmes informatiques :

- soit entre deux ordinateurs,
- soit entre un système superviseur et un automate programmable,
- soit entre un ordinateur et un périphérique...

on dispose de **deux grands procédés** de communication, comme indiqué sur la figure VIII.115. :

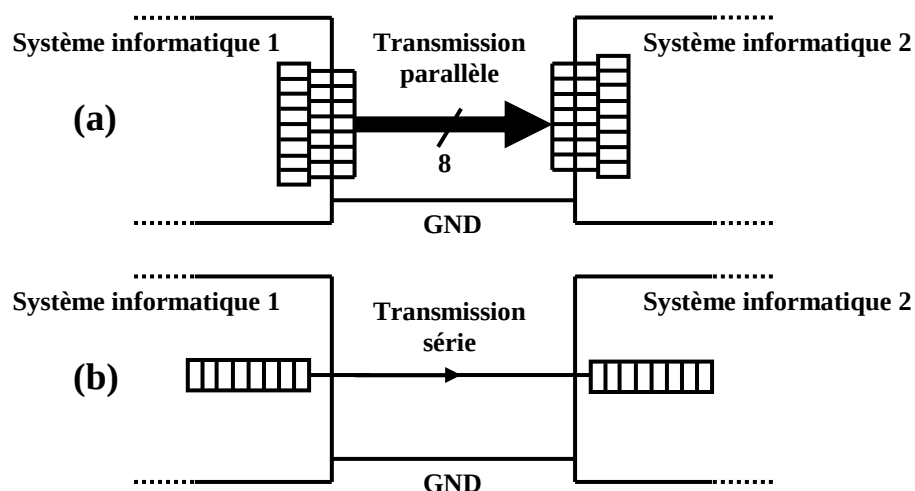


Figure VIII.115.

- la transmission parallèle (figure VIII.115.a.) :

Les octets sont échangés les uns après les autres dans le temps mais **les 8 bits constitutifs de chaque octet sont envoyés (et reçus) en même temps** (dans le même coup d'horloge).

C'est d'ailleurs souvent un port parallèle TOR qui sera utilisé pour la sortie (émission) et/ou l'entrée (réception) des octets.

Dans ce cas, **la rapidité sera importante**, mais ce type de communication nécessite **plus de fils de liaison** : au minimum neuf fils seront nécessaires (soit huit fils pour les bits des octets échangés et un pour la masse).

Exemples : ancien câble Centronix pour relier une imprimante, liaison GPIB/HPIB IEEE 448 ...

- **la transmission série** (figure VIII.115.b.) :

Les octets sont toujours échangés les uns après les autres dans le temps mais **les 8 bits constitutifs de chaque octet sont envoyés (et reçus) les uns derrière les autres**, suivant un rythme bien établi.

Comme à l'intérieur d'un système informatique les données sont traitées globalement par blocs de 8, 16, 32 ou 64 bits (voire plus), la principale fonction de l'interface série sera d'assurer une transformation parallèle $\Rightarrow$ série pour les données à l'émission et série $\Rightarrow$ parallèle pour celles à la réception (grâce à des registres à décalage).

La **rapidité d'une telle liaison est relativement plus faible** (pour une même fréquence d'horloge) que pour une liaison parallèle, mais il faut **peu de fils** pour transmettre, typiquement deux fils suffisant au minimum : le premier servant pour le signal effectif (les bits) et le second pour la référence (la masse). Mais on peut aussi avoir trois fils (toujours un pour la référence et les deux autres pour le signal, un pour chaque sens).

Exemples : liaison V24, liaison USB, réseaux informatiques sans fil avant modulation ...

Les liaisons série sont elles même divisées en deux grandes familles :

- les **liaisons série asynchrones** que nous détaillerons dans un premier temps,

- les **liaisons série synchrones**, dont nous donnerons les principales caractéristiques un peu plus loin.

VIII.7.2. LE MODE ASYNCHRONE DE L'U.S.A.R.T.

Comme d'habitude, nous donnerons dans un premier temps les principes généraux d'une **liaison série asynchrone**.

Et si vous n'êtes pas familier avec ce type de liaison, il est fortement conseillé de commencer par là, car cela ne sert à rien de mettre en œuvre la liaison **asynchrone** si on n'a pas bien compris tous les paramètres qui interviennent dans son fonctionnement. Le temps que vous pensez, à tort, avoir gagné en sautant cette étape, sera très vite perdu par la suite...

Le fonctionnement **asynchrone** correspond à un mode d'échange dont la synchronisation se fait caractère par caractère (souvent octet par octet), ceux-ci pouvant arriver ou être transmis à **n'importe quel instant**.

Pour pouvoir détecter un caractère sur une entrée asynchrone, il faut donc qu'il y ait un début de réception de caractère.

C'est pour cette raison qu'on a défini deux états :

- **5 V** pour la phase **d'attente** (ou état de **ligne au repos** appelé aussi **MARK**),
- **0 V** pour signaler l'arrivée d'un caractère (**état de travail** appelé aussi **SPACE**).

Ces deux niveaux, choisis arbitrairement correspondent à **la norme T.T.L.** mais pourraient être formellement remplacés par deux autres valeurs (par exemple et respectivement **-12V au repos** et **+12V en travail** pour la norme **V24**).

Le schéma de la figure VIII.116. résume le principe de la **synchronisation par caractère** :

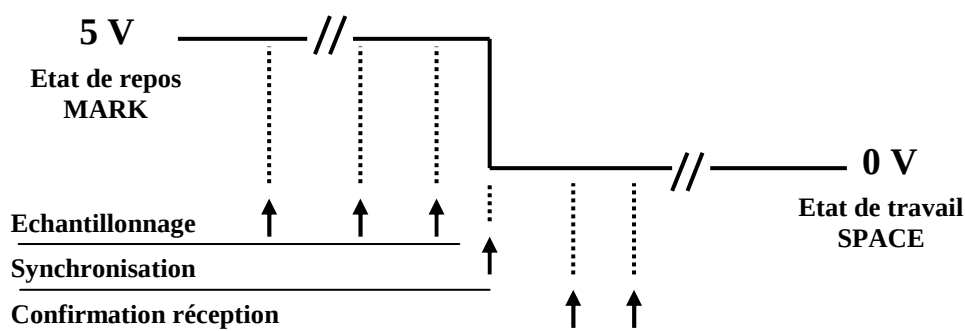


Figure VIII.116.

Cette détection de **front actif descendant** est effectuée par le gestionnaire de la liaison série, qui signalera alors dans un registre prévu à cet effet et accessible au programmeur, l'arrivée d'un nouveau caractère sur la liaison.

Il pourra même, si on lui en donne l'autorisation par logiciel, provoquer une interruption vers le micro/processeur/contrôleur.

Ce gestionnaire spécifique est appelé un **U.A.R.T.** (comme **Universal Asynchronous Receiver Transmitter**) ou un **A.C.I.A.** (comme **Asynchronous Communication Interface Adaptator**).

Ce système de détection, **indépendant du temps d'attente entre deux caractères**, est particulièrement bien adapté pour les interfaces utilisateurs comme les terminaux, où le rythme d'arrivée des caractères est, par nature, complètement aléatoire.

Le format d'une réception asynchrone est donc le suivant (cf. figure VIII.117.)

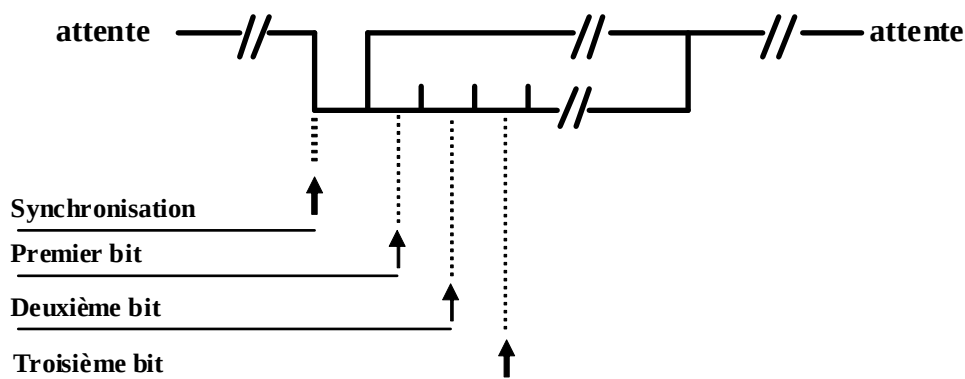


Figure VIII.117.

Chaque bit est ainsi acquis par échantillonnage de l'état situé au milieu de la durée de ce bit, grâce à une horloge interne au circuit **U.A.R.T.** et resynchronisée par le bit de départ.

A condition, bien entendu, que **les vitesses de transmission soient les mêmes** côté émetteur et côté récepteur, ce qui est **nécessaire** dans toute liaison série asynchrone.

VIII.7.2.1. PARAMETRES DE LA LIAISON SERIE ASYNCHRONE

Ce sont eux qui définissent le **format** de la liaison série asynchrone.

Ce format doit être bien sûr **le même** côté émetteur et côté récepteur.

① BIT DE DEPART (START BIT)

Lorsque la réception est **autorisée**, la ligne de réception asynchrone teste en permanence le passage de l'état 5 V à l'état 0 V pour détecter un début de transmission.

On définit une durée pendant laquelle le signal restera à un état 0 V : cette première étape de la transmission d'un caractère s'appelle **le bit de départ (start bit)**.

Ainsi, chaque caractère se trouvera précédé d'un start bit, comme sur la figure VIII.118.

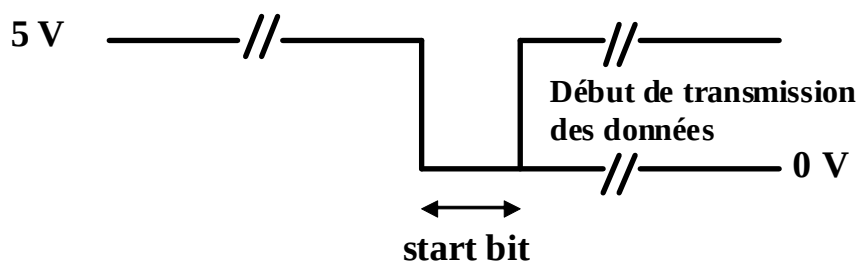


Figure VIII.118.

② VITESSES DE LA LIAISON

Nous venons de voir que l'horloge de récupération des bits était synchronisée par le front descendant du start bit.

Pendant toute la durée d'un caractère, l'horloge interne devra ensuite indiquer l'emplacement du milieu des bits pour pouvoir les échantillonner à cet instant là, où le niveau du bit est le mieux défini. (notons cependant que certains **U.A.R.T.** échantillonnent l'entrée asynchrone pendant toute la durée du bit (typiquement de 2 à 64 fois en fonction de la vitesse de transmission), ce qui améliore sensiblement l'immunité par rapport au bruit et divers parasites industriels).

Cette horloge correspond donc à une vitesse de transmission des informations en mode asynchrone.

Ces vitesses sont définies en **nombre de bits par seconde** ou **Bauds**.

Les périphériques spécialisés pour la transmission ou réception asynchrone peuvent en général travailler à des vitesses comprises entre 50 et 19200 bauds. Ces différentes valeurs sont normalisées, les vitesses successives étant calculées, au moins pour les plus élevées, en les divisant ou multipliant par 2.

Exemple : $19200 = 9600 \times 2$.

En pratique, on trouvera souvent les vitesses suivantes:

Transmission	Réception	Applications
1200/2400	1200/2400	Boucle de courant 20 mA
4800/9600	4800/9600	Terminaux ...

③ NOMBRE DE BITS DE DONNEES

Le nombre de bits d'une transmission est le nombre de bits utiles de chaque caractère proprement dit.

La plupart des U.A.R.T. sont paramétrables par logiciel et offrent souvent le choix entre 5 et 8 bits utiles par caractère. C'est le type d'information à échanger qui déterminera le nombre de bits utiles à transmettre:

- si l'on désire ne transmettre que des caractères A.S.C.I.I., ceux-ci sont alors codés sur 7 bits de **0x00** (caractère **Null** ou **ASCIIZ**) à **0x7F**,
- si l'on souhaite transmettre des informations binaires et donc codées sur un octet, il faudra alors 8 bits de données utiles.

Le premier bit à être transmis est le bit **L.S.B.** (cf figure VIII.119.)

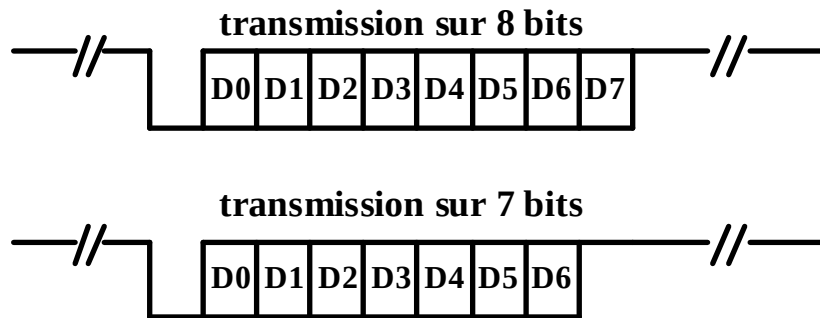


Figure VIII.119.

④ PARITE (PARITY BIT)

Lorsque l'on transmet des informations à distance, il est souhaitable que celles-ci soient reçues correctement, c'est-à-dire **sans erreur**.

Il faut donc pouvoir détecter l'intégrité de chaque caractère reçu.

Pour cela, on peut rajouter, après le dernier bit de chaque caractère, un bit spécial dit **bit de parité (Parity Bit)**.

Il existe deux types de parité :

- la **parité paire (even parity : 4 lettres à even)** :
lorsque le **nombre de bits à 1** du caractère et de ce bit de parité, est **pair**,
- la **parité impaire (odd parity : 3 lettres à odd)** :
lorsque le **nombre de bits à 1** du caractère et de ce bit de parité, est **impair**.

Attention :

Ces notions de parité et d'imparité présentées ici n'ont rien à voir avec la parité/imparité au sens arithmétique du terme (multiple de 2 ou pas) !!!

Soit par exemple à transmettre le caractère ASCII 'A', codé **0x41** = 0b 0100 0001 sur 8 bits :

- le bit de **parité P** vaudra **0** (2 bits à 1) en parité **paire**,
- en parité **impaire**, ce bit **P** vaudrait **1** (3 bits à 1).

Les chronogrammes correspondant à cet exemple sont représentés ci-dessous, sur la figure VIII.120 :

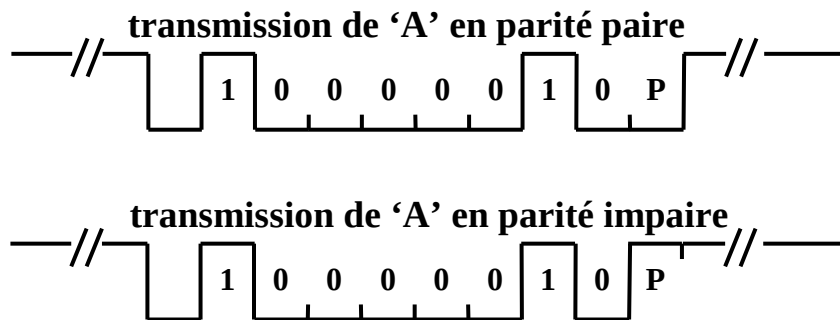
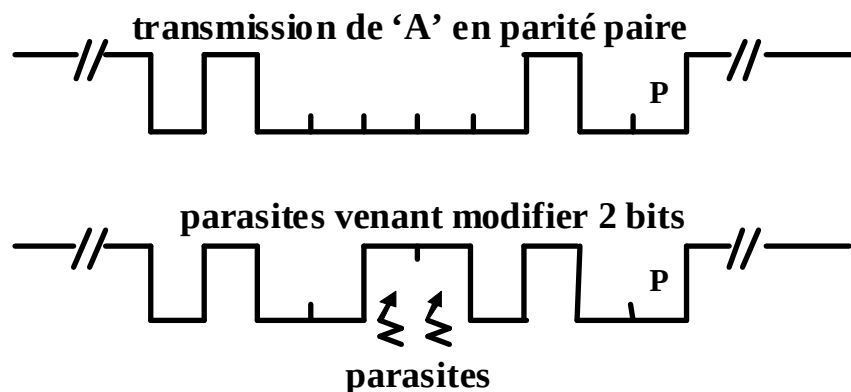


Figure VIII.120.

A la réception, on utilise bien sûr le même type de parité qu'à l'émission. Ainsi, en comptant le nombre de bits à 1, du caractère effectivement reçu, on pourra détecter une violation de parité si un bit (et plus généralement un nombre impair de bits) a changé d'état : le bit de parité reçu n'aura pas la bonne valeur, l'erreur sera détectée.

Il est clair que la parité a ses limites en termes de détection d'erreur de transmission car si un nombre pair de bits s'inversent dans un octet, le bit de parité restera correct et l'erreur ne sera pas détectée, comme on peut le voir sur la figure VIII.121. ci-dessous.

Soit à transmettre 'A' en parité paire :



2 bits s'inversent, mais la parité reste correcte : erreur non détectée

Figure VIII.121.

La plupart des **U.A.R.T.** effectue ce contrôle de parité de façon transparente pour le programmeur et signale, à chaque caractère reçu, les éventuelles **violations de parité (parity error)**.

Par contre, lorsque ce n'est pas le cas, et si on veut implanter le contrôle par parité, il faut le faire par programme.

Mais dans tout les cas (parité gérée automatiquement par l'**U.A.R.T.** ou parité gérée par logiciel), il restera à la charge du programmeur de prévoir le traitement de ces erreurs, en demandant par exemple une réémission du caractère parasité.

D'autres techniques permettent, en plus, de **corriger** les éventuelles erreurs de transmission sans faire systématiquement appel à l'émetteur.

Elles ne seront pas abordées ici. Elles font le plus souvent appel à la théorie des polynômes et sont beaucoup utilisées dans les techniques de télécommunications longues distances.

⑤ BIT(S) D'ARRET (STOP BIT(S))

Ils sont rajoutés après le bit de parité, ce qui permet un retour systématique de la ligne à l'état de repos entre chaque caractère (voir figure VIII.122.).

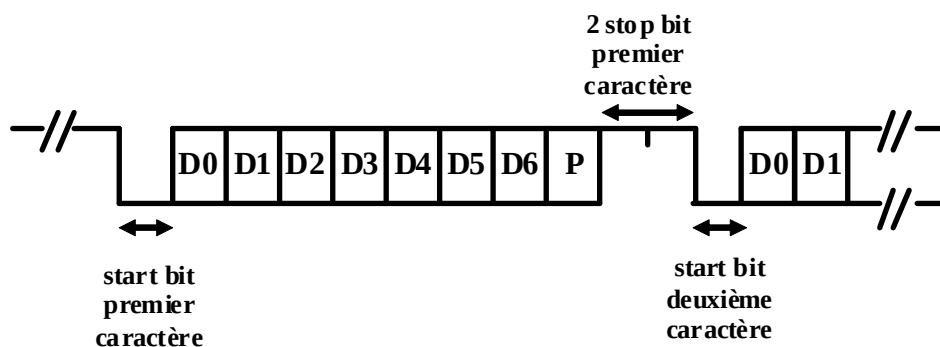


Figure VIII.122.

Vu côté émetteur, le nombre de **bits d'arrêt (stop bit)** donne le temps minimum d'attente pour la transmission d'un nouveau caractère.

Vu côté récepteur, ce sera le temps minimum d'attente **avant** une nouvelle scrutation d'un start bit.

La plupart des **U.A.R.T.** sont programmables avec 1, 1,5 ou 2 stop bits.

Ceux-ci seront rajoutés **automatiquement** par l'**U.A.R.T.** après le bit de parité.

On travaillera souvent avec un bit de stop (la possibilité d'avoir 2 bits de stop est un vestige du passé du à la lenteur de traitement des vieux téléimprimeurs entre chaque caractère).

Là encore, l'émetteur et le récepteur doivent choisir la même convention pour le nombre de bits de stop, au risque de provoquer une **erreur de format (framing error)** ou/et une erreur de parité, comme on peut le voir sur l'exemple fourni sur la figure VIII.123. où l'émetteur est initialisé avec **1 bit de stop** et le récepteur avec **2 bits de stop**.

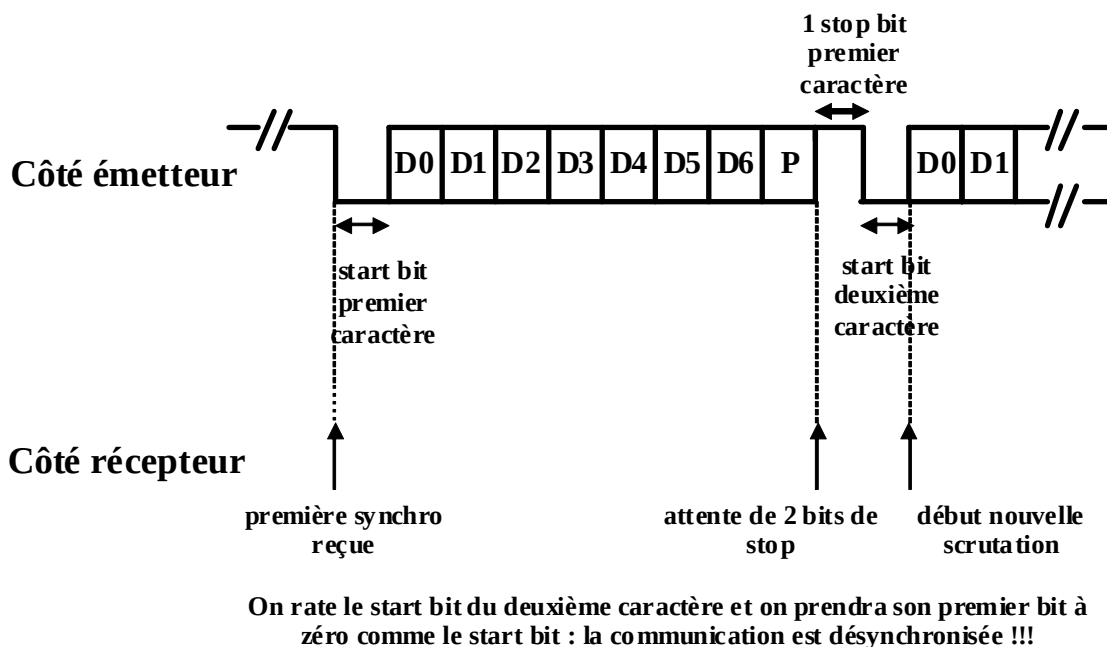


Figure VIII.123.

On voit donc finalement qu'à chaque caractère du message (l'information vraiment utile à transmettre), il faut rajouter un certain nombre de bits pour obtenir un fonctionnement satisfaisant. Ceci, au détriment de la rapidité **effective** de la liaison.

Il est un ordre de grandeur intéressant à connaître (et qui vous aidera par exemple à choisir la base de temps d'un oscilloscope en cas de tests sur une liaison série asynchrone) :

à **9600 Bauds**, la durée de transmission d'un caractère est de **1 ms environ**.

DEUX REMARQUES :

REMARQUE 1: SIGNAL BREAK

Notons également un état que l'on trouve parfois : le signal **BREAK**.

Il s'agit d'un état où la ligne se trouve à un niveau bas (space) pendant une durée très longue devant celle d'un caractère, habituellement pendant plusieurs centaines de millisecondes.

Ce signal est utilisé par exemple par un terminal qui veut se connecter sur la ligne et la plupart des **U.A.R.T.** peuvent envoyer ou détecter un break sur la ligne.

REMARQUE 2: QUELQUES PETITS CONSEILS PRATIQUES

Lorsque deux équipements, reliés entre eux, n'arrivent pas à échanger des informations, vérifier avant toute autre chose que **les formats sont les mêmes** à l'émission et à la réception.

Dans certains cas, des différences de format peuvent impliquer la réception de messages complètement erronés (caractères de la table A.S.C.I.I. étendue en cas de désaccord sur la longueur des caractères, plus de caractères reçus que de caractères transmis si débits différents...).

Si rien n'est reçu, pensez à vérifier votre cordon ou son emplacement de branchement.

Quand on met au point une liaison, il est souvent plus facile de la tester en local, c'est-à-dire en utilisant un connecteur dit "**bouchon**", où les caractères émis par l' **U.A.R.T.** sont reçus par lui-même.

Avant de développer toute application, tester la liaison en envoyant répétitivement le même caractère :

typiquement : caractère ASCII 'U' = 0x55 = 0b01010101 en parité paire, qui donnera sur votre oscilloscope un beau signal carré...

VIII.7.2.2. UN EXEMPLE : LA NORME V24 (RS232)

① INTRODUCTION GENERALE :

L'interface **V24** est une norme de communication européenne définie par le C.C.I.T.T. qui utilise le mode de transmission asynchrone, dérivée de son homologue américaine la **RS232C**.

Ce type de liaison est dite **point à point**, c'est-à-dire qu'un **émetteur** est connecté sur **un et un seul récepteur**.

On trouvera souvent dans les normes, les définitions suivantes :

- le **D.C.E. : Data Communication Equipment** (modem) appelé aussi l' **E.T.C.D. : Equipement Terminal de Circuit de Données**, chargé de transformer le signal numérique fourni par l' **E.T.T.D.** en une forme (analogique ou numérique) compatible avec les propriétés physiques du canal de transmission.

- le **D.T.E. : Data Terminal Equipment**.(ordinateur, périphérique, terminal) appelé aussi l' **E.T.T.D : Equipement Terminal de Traitement de Données**.

La norme **V24** définit un certain nombre d'éléments pour relier un **D.T.E.** avec un **D.C.E.** (ou éventuellement deux **D.T.E. entre eux**), comme on peut le voir sur la figure VIII.124.

L'intérêt est évident puisqu'elle permet de relier des équipements qui ne sont pas forcément compatibles en créant une passerelle standard entre eux.

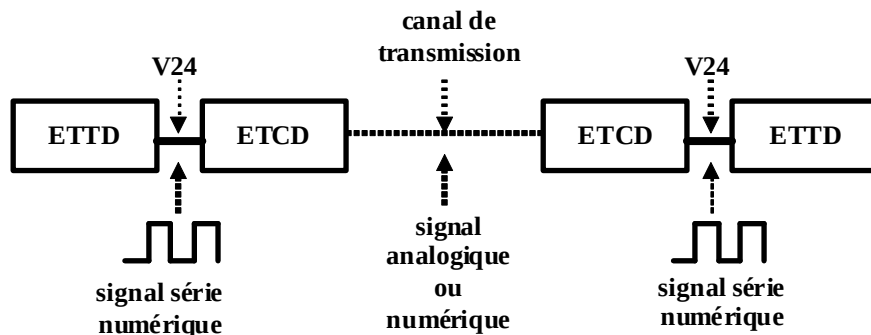


Figure VIII.124.

Note : A titre indicatif, un PC vu de ses « COM » est considéré comme terminal (DTE).

② NORMES ELECTRIQUES :

Les normes électriques permettent de définir le plus bas niveau de protocole.
(couche ISO 1 ou couche physique).

On y trouvera en particulier :

- le niveau électrique de transmission :

tensions maximum : $\pm 25V$,
tensions nominales d'utilisation : $\pm 12V$.

- l'immunité au bruit :

un signal $> +3V$ est considéré comme l'état logique 0,
un signal $< -3V$ est considéré comme l'état logique 1.

- les impédances :

d'entrée : 3 à 7 K Ω ,
de sortie : $< 300 \Omega$.

- les transitions :

caractérisées par le $\frac{dV}{dt}$, temps de montée ou de descente : $< 30 V.\mu s^{-1}$.

Une trame V24 aura donc l'allure de la figure VIII.125. suivante :

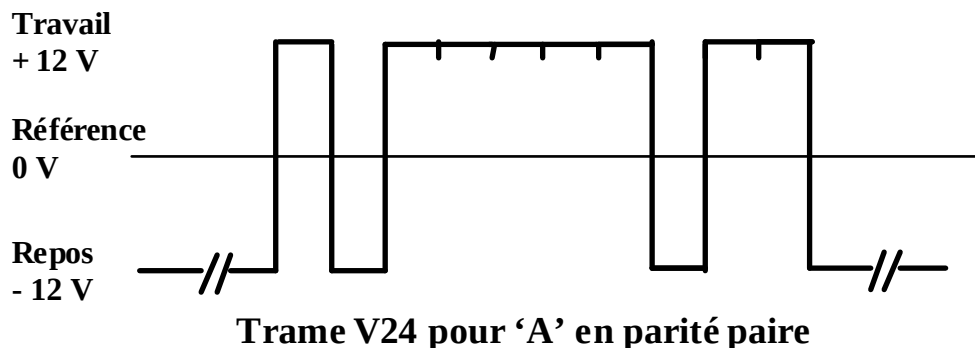


Figure VIII.125.

Le signal est donc **bipolaire**, avec inversion de polarité et donc en **logique dite négative**.

Le fait de passer en $\pm 12 V$ permet d'améliorer la protection face aux parasites industriels, et permet des distances de liaison plus grandes qu'en 0-5 V.

③ SIGNAUX D'INTERFACE ET DE PROTOCOLE HARDWARE:

Là aussi la norme est très précise, car il s'agit de pouvoir connecter sans aucune modification de câblage, sinon en adaptation de vitesse, en nombre de bits ou en parité, deux équipements de nature à priori différente.

Malheureusement, à la vue de la prolifération des boîtiers d'interconnexion, des analyseurs de signaux et autres "boîtes à straps" en tous genres et aussi à cause de l'hégémonie de certains fabricants pour qui le mot "**Standard**" est équivalent à "**Marché**", cette norme n'est pas souvent respectée.

A titre de référence, nous prendrons l'exemple de la sortie RS232 d'un terminal (écran+clavier). C'est de loin l'appareil dont la sortie RS232 est la plus standard.

Le connecteur d'un tel équipement est un connecteur 25 points Mâle (ou parfois 9 points) que l'on connaît sous diverses appellations comme "Canon 25 points", "SubD 25 points" ou "DB25" (ou "DB9" pour les 9 points).

Les principaux signaux utilisés et disponibles sur ce connecteur sont résumés dans le tableau de la figure VIII.126. ci-après.

Il est bien entendu que tout signal d'un équipement D.T.E. se retrouve symétriquement sur un appareil de type D.C.E.

Exemple:

L'entrée DCD (Data Carrier Detect) d'un D.T.E. deviendra une sortie sur un D.C.E.

N° de broche DB25	Nom usuel	Sens DTE DCE	Fonction : état affirmé = +12V ou 0V niveau TTL
1	FG (ou TP)		Connection du blindage du câble si existant, et en principe reliée à la terre.
2	TXD (ou ED)	→	Transmission (Emission) des Données : sera à relier sur la voie réception du D.C.E.(1)
3	RXD (ou RD)	←	Réception des Données : sera à connecter sur la voie de transmission du D.C.E.
4	RTS (ou DPE)	→	Request To Send (Demande Pour Emettre) : le terminal effectue une requête pour être autorisé à transmettre.(2)
5	CTS (ou PAE)	←	Clear To Send (Prêt A Emettre) : acquittement vers le terminal pour l'autoriser à émettre.
6	DSR (ou PDP)	←	Data Set Ready (Poste de Données Prêt) : le D.C.E. est bien connecté sur la ligne.
7	GND (ou MS)		Référence commune des tensions de tous les signaux.
8	DCD (ou DS)	←	Data Carrier Detect (Détection de Signal): porteuse valide reçue par le D.C.E.
20	DTR (ou TDP)	→	Data Terminal Ready (Terminal Données Prêt) : terminal sous tension et mis on-line et requête vers le D.C.E. pour qu'il se connecte sur la ligne.

Figure VIII.126.

(1) Pour un D.C.E., la transmission sera la broche 3 et la réception la broche 2.

(2) Voir aussi le protocole RTS / CTS

Au minimum seuls trois fils peuvent être suffisants pour établir une liaison bidirectionnelle simultanée :

TXD, RXD et GND.

Mais pour fiabiliser la liaison (s'assurer que le récepteur est prêt, éviter des pertes dangereuses de caractères, signaler des événements extérieurs...) d'autres signaux ont été rajoutés, ce qui permet de pratiquer un **protocole "hardware"**, par échange et interprétation des tensions présentes sur ces broches.

En théorie, la norme définit 39 signaux différents, ce qui peut donner le sentiment (à juste titre par ailleurs, car on est loin de la liaison USB où on branche et, le plus souvent ça marche !!!) d'une relative complexité, rendue cependant nécessaire par la grande variété des cas d'exploitation possibles et des différents types d'équipements existants.

Sans vouloir définir ici exhaustivement tous les circuits (désignation officielle de la norme pour un de ces 39 signaux), nous nous contenterons de préciser les signaux de contrôle les plus courants.

❶ LA SORTIE DTR, LES ENTREES DSR et DCD.

Dans le cas d'une liaison avec un modem, les signaux DTR et DSR interviennent pendant la phase d'établissement de la liaison, avant tout échange proprement dit. Le signal DCD permet de valider la réception car il indique que les données démodulées sont bien intègres (que ce ne sont pas des parasites ou du bruit non significatifs).

De façon générale, la sortie DTR indique que le terminal est prêt. La valeur présente alors sur cette sortie est +12V et pourra être utilisée pour mettre au +12V les entrées qui nécessitent un état affirmé (sur le D.T.E. lui-même ou sur l'équipement auquel il est connecté).

Exemple : connexion de deux PC

En pratique, pour une liaison entre deux PC, ces trois signaux pourront être reliés ensemble, car au boot, le PC place DTR au niveau bas, ce qui forcera les deux entrées DSR et DCD au niveau bas. Certaines fonctions du BIOS testent ces entrées et doivent les trouver actives pour valider la réception.

② LES SIGNAUX $\overline{\text{RTS}}$ / $\overline{\text{CTS}}$.

Tel qu'il est décrit dans le tableau précédent, le signal $\overline{\text{RTS}}$ est utilisé pour effectuer une demande de transmission de la part d'un ordinateur vers un modem **Half Duplex** (émission et réception non possible en même temps et donc partage du temps entre les deux sens. On dit aussi full-duplex à l'alternat).

La réponse du modem s'effectuera par l'acquittement du signal $\overline{\text{CTS}}$, signalant ainsi qu'il passe du mode réception au mode transmission.

Dans tous les autres cas de figure, ces deux signaux sont utilisés comme **contrôle de flux câblé** entre l'émetteur et le récepteur.

Ils sont alors appelés signaux de **Hand-Shake (Poignée de main !)**.

Ils permettent de faire communiquer directement deux D.T.E. entre eux (pour des liaisons courtes, ne nécessitant pas l'utilisation d'un modem).

Exemple : connexion de deux PC

On rebouclera la sortie $\overline{\text{RTS}}$ d'un PC sur l'entrée $\overline{\text{CTS}}$ de l'autre et réciproquement. Le PC qui ne peut plus recevoir des données (buffer de réception plein, risque de perte de caractères...) fait passer sa sortie $\overline{\text{RTS}}$ au niveau haut. L'émetteur, c'est à dire l'autre PC voyant son entrée $\overline{\text{CTS}}$ qui remonte, en déduit que le récepteur n'est plus prêt à recevoir ses données et stoppe la transmission. C'est en fait la **version câblée** du protocole logiciel **XON/XOFF**.

Une autre possibilité, pour placer un PC en état de recevoir des données est de reboucler sa sortie $\overline{\text{RTS}}$ sur son entrée $\overline{\text{CTS}}$: en effet, au démarrage, le PC force $\overline{\text{RTS}}$ au niveau bas, ce qui forcera en conséquence son entrée $\overline{\text{CTS}}$ au niveau bas, et ainsi les fonctions du BIOS qui permettent d'accéder au port série interpréteront cet état comme "récepteur prêt à recevoir".

VIII.7.2.3. SYNOPTIQUES ET REGISTRES ASSOCIES A L'UART

Nous allons rentrer maintenant à l'intérieur du PIC pour comprendre comment est gérée la liaison série **asynchrone**, en commençant tout d'abord par sa partie **émission**. Nous traiterons ensuite de sa partie **réception**.

Notons tout de même que la liaison série asynchrone est capable de fonctionner en **full duplex**, c'est-à-dire qu'on peut émettre et recevoir des caractères en même temps, car les parties émettrices et réceptrices de l'**U.A.R.T.** sont totalement indépendantes et peuvent donc fonctionner en parallèle et en même temps.

VIII.7.2.3.1. SYNOPTIQUE DE L'EMETTEUR

Il est fourni sur la figure VIII.127. ci-dessous

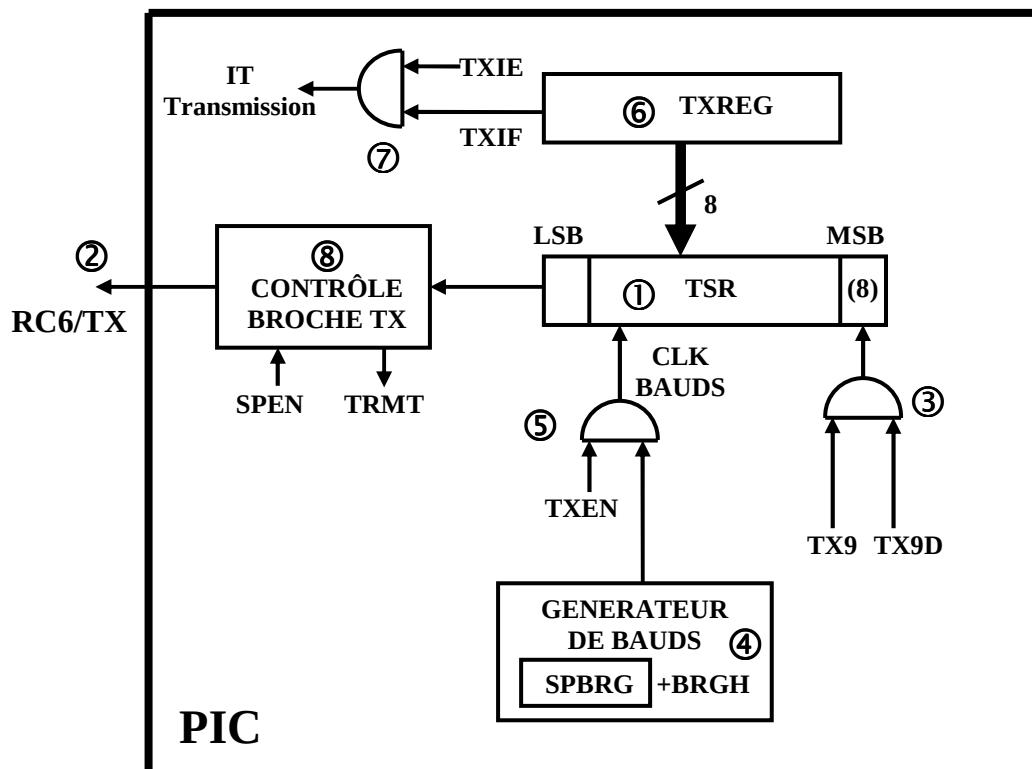


Figure VIII.127.

On retrouve sur cette figure quelques éléments déjà vus dans les généralités sur la liaison série asynchrone. Détaillons en les principaux blocs et le principe de fonctionnement.

Le cœur de l'émetteur est le registre **TSR** (cf. bloc ①) : c'est un registre à décalage qui va sérialiser (dès qu'il est rempli) son contenu bit à bit sur la broche **RC6/TX** (cf. ②) via le bloc ⑧ **CONTRÔLE BROCHE TX** qu'on verra plus loin. C'est le **LSB** qui est transmis en premier.

Les bits série sur la broche **RC6/TX** sont codés au format standard **NRZ (Non Return to Zero)** tel qu'il a été décrit plus haut dans les généralités :

- **bit à 1 : 5 V,**
- **bit à 0 : 0 V**

Le format de la liaison est aussi assez standard :

1 start bit, 8 ou 9 bits pour le caractère, et 1 stop bit.

Le plus souvent, on travaille en 8 bits par caractère, mais il est possible de rajouter un neuvième bit par la logique repérée en ③, pilotée par les bits de commande :

- **TX9** : autorisation du mode 9 bits par caractère,
- **TX9D** : le 9^{ème} bit du caractère à émettre.

Ce 9^{ème} bit pourrait être le **bit de parité**, mais qui devra alors être calculé par le programme car le bloc **U.A.R.T.** du PIC **n'a pas un bloc de gestion automatique** de la parité.

La vitesse de la liaison est paramétrable par le bloc ④ **GENERATEUR DE BAUDS.**

Les deux éléments pour déterminer cette vitesse de transmission sont :

- le registre **SPBRG** : de taille 8 bits,
- le bit de contrôle **BRGH**.

Les formules faisant le lien entre la **vitesse de transmission V** (exprimée en bauds), le contenu du registre **SPBRG** et le bit **BRGH** sont fournies sur le tableau de la figure VIII.128.

BRGH = 0 (mode low speed)	BRGH = 1 (mode high speed)
$V = \frac{F_{osc}}{64. (SPBRG + 1)}$	$V = \frac{F_{osc}}{16. (SPBRG + 1)}$

Comme d'habitude **F_{osc}** représente la fréquence du quartz du PIC exprimée en Hz.

Figure VIII.128.

Il est conseillé de travailler en mode high speed (avec **BRGH = 1**) même si le mode low speed pouvait convenir, car on obtient alors une meilleure précision sur la vitesse obtenue par rapport aux valeurs normalisées.

En pratique, on souhaite travailler à une certaine vitesse et les formules précédentes permettront de déterminer le contenu du registre **SPBRG** et la valeur du bit **BRGH**.

Sinon, dans le datasheet du PIC, **MICROCHIP** fournit des tableaux permettant de déterminer les valeurs à écrire dans **BRGH** (en fonction de la vitesse de transmission souhaitée) et la précision obtenue sur cette vitesse dans chaque cas.

Notons aussi la logique repérée en ⑤, pilotée par le bit de commande **TXEN** qui permet de valider la transmission, lorsqu'il est à 1. Sinon, quand **TXEN** = 0, l'horloge bit du registre à décalage **TSR** n'arrive plus et le caractère n'est plus sérialisé.

Le registre **TSR** n'est pas accessible à l'utilisateur directement (il est non mappé). Pour transmettre un caractère, il faut écrire (par programme) dans le registre **TXREG** (cf. bloc ⑥) : c'est un registre de taille 8 bits qui sert en fait de tampon (notons que ce registre **TXREG** est aussi accessible en lecture bien qu'il soit plus rarement utilisé dans ce mode pendant une émission).

Dès que ce registre **TXREG** est plein, son contenu est transféré **automatiquement** par l'**U.A.R.T.** dans le registre à décalage **TSR** et le transfert (la sérialisation du caractère vers la broche **RC6/TX**) peut commencer (si bien sûr le bit **TXEN** est à 1).

Notons qu'alors le flag **TXIF** passe à 1 indiquant que le registre **TXREG** est maintenant **vide** et qu'on peut écrire un autre caractère (en fait le suivant à transmettre), sans risque d'écraser le précédent.

Pour gérer cet évènement, et comme d'habitude, il y a 2 possibilités :

- soit par **pooling** du bit **TXIF** : ce qui revient, en régime établi, à attendre que le caractère précédent ait été entièrement transmis sur la ligne,
- soit par **interruption** : si le bit **TXIE** (validation de l'interruption de transmission) a été mis à 1 (cf. logique repérée en ⑦).

Par contre, on ne peut remettre ce flag **TXIF** à 0 par soft : c'est un « **flag hardware** » qui sera mis **automatiquement** à 0 par l'**U.A.R.T.** quand le registre **TXREG** sera chargé avec un nouveau caractère à transmettre, indiquant ainsi que le registre **TXREG** est maintenant **plein**.

Des exemples de ces 2 méthodes seront fournis plus loin.

Reste à parler du bloc ⑧ **CONTRÔLE BROCHE TX**, piloté par le bit **SPEN**, qui permet de valider la liaison globalement (en émission, réception, en mode asynchrone et synchrone). Ce bit devra donc être au préalable mis à 1 si on souhaite utiliser cette liaison.

Notons également l'existence d'un autre flag **TRMT** qui passera à 1 lorsque le bit de stop du caractère courant a été transmis, signalant ainsi que le registre **TSR** est vide. Mais, ce bit n'est pas prévu, comme pour le bit **TXIF**, pour générer une interruption. Si on veut le gérer, il faut le faire par la technique du polling !!!

Par contre c'est ce bit qui entraînera **automatiquement** le chargement du registre **TSR** par le registre **TXREG** (si ce dernier registre est plein).

VIII.7.2.3.2. REGISTRE ASSOCIE A L'EMETTEUR

La plupart des bits de configuration que l'on vient de présenter se trouvent dans le registre SFR **TXSTA**, dont le format est fourni sur la figure VIII.129.

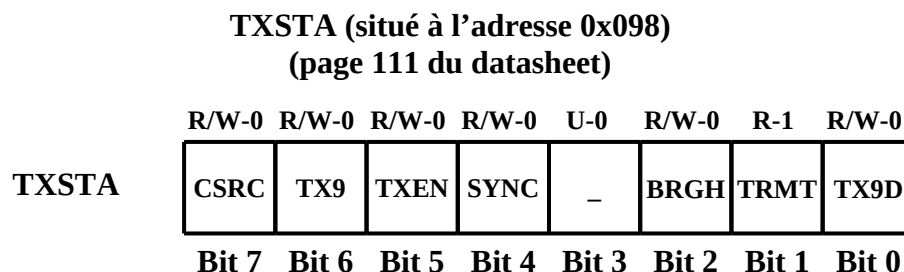


Figure VIII.129.

La signification de ces bits est résumée sur la figure VIII.130. Certains de ces bits ont d'ailleurs déjà été vus sur le synoptique de la figure VIII.127.

Bit	Valeur	Signification et commentaires
CSRC		Utilisé dans le mode synchrone de l' U.A.R.T. Et donc indifférent en mode asynchrone !!!
TX9		Pour choisir le mode 8 bits ou 9 bits par caractère transmis
	0	Mode 8 bits en transmission
	1	Mode 9 bits en transmission
TXEN		Validation de l'émetteur
	0	Emetteur non validé
	1	Emetteur validé : on peut l'utiliser !!!
SYNC		Choix du mode de fonctionnement l' U.A.R.T.
	0	Mode asynchrone
	1	Mode synchrone
BRGH		Pour la vitesse V de la liaison
	0	Mode low speed
	1	Mode high speed
TRMT		Flag indiquant l'état du registre TSR
	0	Registre TSR plein (en cours d'émission)
	1	Registre TSR vide : transmission du caractère courant finie
TX9D		9 ^{ème} bit du caractère transmis si mode 9 bits choisi (par exemple le bit de parité)
	0	Si 9 ^{ème} bit vaut 0
	1	Si 9 ^{ème} bit vaut 1

Figure VIII.130.

Notons enfin que le bit **SPEN** se trouve dans un autre registre de contrôle **RCSTA** lié à la liaison asynchrone en **réception**. On le retrouvera donc dans la partie suivante.

Le hardware ayant été présenté, quelle procédure faut-il utiliser pour transmettre un caractère sur la liaison série asynchrone ?

Cette procédure, comportant un certain nombre d'étapes, dont certaines sont facultatives (mode 9 bits, utilisation des interruptions...) est détaillée ci-dessous :

Etape 1 : gestion de la vitesse de transmission.

Mettre la bonne valeur dans le registre **SPBRG** en fonction également du bit **BRGH**.

Etape 2 : validation de la liaison série :

Ici :

- **validation de la liaison** : **SPEN = 1**,
- en mode **asynchrone** : **SYNC = 0**,

Etape 3 : gestion de la broche **RC6/TX** :

Par :

TRISC6 = 1 // ce n'est pas une erreur !!!

Même si c'est une sortie !!!

Etape 4 : fonctionnement en pooling ou sous interruption :

Et donc :

- **TXIE = 0** si polling
- **TXIE = 1** si interruption

Etape 5 : fonctionnement en mode 8 bits ou 9 bits par caractère :

Et donc :

- **TX9 = 0** si mode 8 bits
- **TX9 = 1** si mode 9 bits

Etape 6 : validation de la transmission :

Par :

TXEN = 1 ce qui placera automatiquement le bit **TXIF** à 1

Etape 7 : gestion du 9^{ème} bit, si mode 9 bits choisi (à faire impérativement avant étape 8 car sinon le bit **TX9D** ne correspondra plus au bon caractère, mais au suivant).

Par :

TX9D = 1 ou **0** : c'est en fait la valeur du 9^{ème} bit

Etape 8 : chargement du caractère à transmettre dans **TXREG**

Par :

TXREG = valeur du caractère que l'on veut envoyer sur la liaison (ce qui remet TXIF à 0).

Etape 9 : si fonctionnement sous interruption :

Il faut les démasquer par :

- **PEIE = 1** : démasquage semi-local
- **GIE = 1** : démasquage global

Certaines de ces étapes seront faites dans la fonction d'initialisation de la liaison série, d'autres dans la fonction qui gèrera la transmission (si fonctionnement en pooling sans oublier dans ce cas d'attendre que le flag **TXIF** passe à 1 avant de charger **TXREG** avec la valeur du prochain caractère à transmettre) ou dans la routine d'IT (si interruption utilisée).

VIII.7.2.3.3. SYNOPTIQUE DU RECEPTEUR

Il est fourni sur la figure VIII.131. ci-dessous.

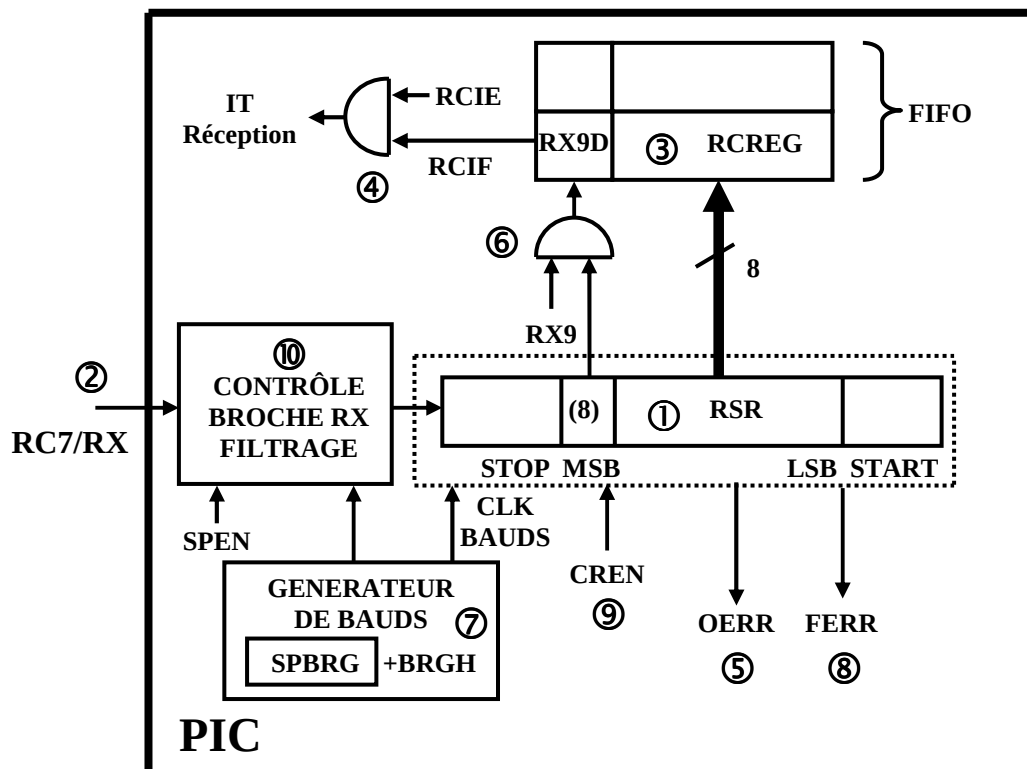


Figure VIII.131.

On retrouve là aussi sur cette figure quelques éléments déjà vus dans les généralités sur la liaison série asynchrone.

Détaillons en les principaux blocs et le principe de fonctionnement.

Notons en introduction que la logique de réception de l'**U.A.R.T.** n'est validée que si le bit **CREN** (cf. ⑨) est à 1.

Le cœur du récepteur est le registre **RSR** (cf. bloc ①) : c'est un registre à décalage qui récupère les bits série (du **start bit** au **stop bit**) arrivant sur la broche **RC7/RX** (cf. ②) via le bloc ⑩ **CONTRÔLE BROCHE RX FILTRAGE** qu'on verra plus loin.

Ce registre n'est pas directement accessible au programmeur : il n'est pas mappé.

Après réception du stop bit, le contenu du **RSR** est **automatiquement** placé dans le registre **RCREG** (cf. bloc ③) de taille 8 bits si ce dernier est vide. A ce moment, le flag **RCIF** passe à **1** indiquant qu'un caractère complet est arrivé. Pour traiter ce flag **RCIF**, et comme d'habitude, on peut le faire :

- soit par **pooling**, ce qui revient, en régime établi, à attendre que le caractère complet ait été entièrement reçu par le récepteur,
- soit sous interruption si le bit de masquage local **RCIE** a aussi été mis à **1** (cf ④).

Par contre, on ne peut remettre ce flag **TCIF** à **0** par soft : c'est un « **flag hardware** » qui sera mis **automatiquement** à **0** par l'**U.A.R.T.** quand le registre **RCREG** aura été lu par le programme.

Des exemples de ces 2 méthodes seront fournis plus loin.

Notons cependant que ce registre **RCREG** est en fait un registre « **bufferisé** » à **2 positions**, se comportant comme une **structure** de type **FIFO**, ce qui permet de pouvoir recevoir au maximum deux caractères avant d'effectivement les lire.

Par contre, si un troisième caractère arrive, alors que le **FIFO** est encore plein, le bit d'erreur **OERR** (cf ⑤) passera à **1** indiquant une erreur **d'overflow**, c'est-à-dire que ce troisième caractère sera perdu (gestion des erreurs à faire par soft : la remise à **0** du bit **OERR** sera faite en remettant **CREN** à **0** puis à **1**).

Mais notez bien :
si on ne remet pas le bit OERR à 0 lorsqu'il est passé à 1, plus aucune
réception n'est possible, car le chargement de RCREG par RSR ne se
fait plus !!!

Le format de la liaison, pour des raisons évidentes de compatibilité et de compréhension réciproques doit bien sûr être le même que du côté de l'émission :

1 start bit, 8 ou 9 bits pour le caractère, et 1 stop bit.

Le plus souvent, on travaille en 8 bits par caractère, mais il est possible de récupérer un neuvième bit par la logique repérée en ⑥, pilotée par les bits de commande :

- **RX9** : autorisation du mode 9 bits par caractère,
- **RX9D** : le 9^{ème} bit du caractère reçu.

Ce 9^{ème} bit pourrait être **le bit de parité**, mais qui devra alors être vérifié par le programme de réception car le bloc **U.A.R.T.** du PIC **n'a pas un bloc de gestion automatique** de la parité fournissant une erreur de parité.

Notons que si on travaille en mode 9 bits, il faut lire le bit **RX9D** **avant** de récupérer le caractère de **RCREG**, car sinon, si on lit le caractère avant, le bit **RX9D** sera mis à jour avec le 9^{ème} bit du caractère suivant, et ne sera donc plus valide.

La vitesse de la liaison est paramétrable par le bloc ⑦ **GENERATEUR DE BAUDS**, qui est le même que celui utilisé par l'émetteur.

Moralité : émission et réception se font à la même vitesse !!!

Notons également l'existence du flag **FERR** (cf. ⑧) indiquant une erreur de cadrage (**framing error**) qui se produit si le **bit de stop** est à **0** au lieu de **1**. Cela signifie que la communication est « désynchronisée » et donc qu'il y a un problème : vitesse ou format de liaison non compatibles entre émetteur et récepteur par exemple.

Si on veut gérer ce type d'erreur, il faut d'abord lire le bit **FERR** avant de lire le caractère reçu car si on lit le caractère avant, la valeur de **FERR** ne sera plus à jour (même raison vue plus haut que pour le bit **RXD9**)

Reste à parler du bloc ⑩ **CONTRÔLE BROCHE RX FILTRAGE**, piloté par le bit **SPEN**, dont on a déjà parlé pour l'émission et qui permet de valider la liaison globalement (en émission, réception, en mode asynchrone et synchrone). Ce bit devra donc être au préalable mis à **1** si on souhaite utiliser cette liaison.

Ce bloc effectue également un filtrage suivant la technique du « **vote majoritaire** » : pendant la durée d'un bit, plusieurs valeurs sont prises et au final ce bloc décide si ce bit est à **1** (plus de valeurs à **1** qu'à **0**) ou est à **0** (plus de valeurs à **0** qu'à **1**).

VIII.7.2.3.4. REGISTRE ASSOCIE AU RECEPTEUR

La plupart des bits de configuration que l'on vient de présenter se trouvent dans le registre SFR **RCSTA**, dont le format est fourni sur la figure VIII.132.

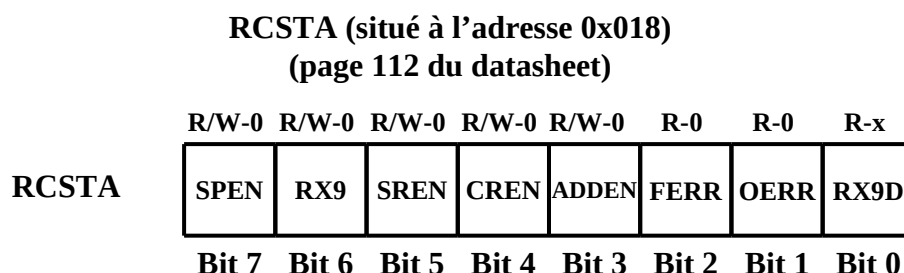


Figure VIII.132.

La signification de ces bits est résumée sur la figure VIII.133. Certains de ces bits ont d'ailleurs déjà été vus sur le synoptique de la figure VIII.131.

Bit	Valeur	Signification et commentaires
SPEN		Pour valider la liaison série complète (mode asynchrone ou synchrone)
	0	Dévalidée
	1	Validée : on peut l'utiliser
RX9		Pour choisir le mode 8 bits ou 9 bits par caractère reçu
	0	Mode 8 bits en réception
	1	Mode 9 bits en réception
SREN		Utilisé dans le mode synchrone de l' U.A.R.T. Et donc indifférent en mode asynchrone !!!
CREN		Validation du récepteur
	0	Récepteur non validé
	1	Récepteur validé : on peut l'utiliser !!!
ADDEN		En mode 9 bits, bit de détection d'adresse (voir plus loin)
	0	Mode détection d'adresses non validé
	1	Validation du mode détection d'adresses
FERR		Flag signalant une erreur de cadrage (Framing ERRor)
	0	Pas d'erreur de cadrage
	1	Erreur de cadrage
OERR		Flag signalant une erreur d'écrasement de caractère (Overrun ERRor)
	0	Pas d'erreur d'écrasement
	1	Erreur d'écrasement
RX9D		9 ^{ème} bit du caractère reçu si mode 9 bits choisi (par exemple le bit de parité)
	0	Si 9 ^{ème} bit vaut 0
	1	Si 9 ^{ème} bit vaut 1

Figure VIII.133.

Le hardware ayant été présenté, quelle procédure faut-il utiliser pour recevoir un caractère sur la liaison série asynchrone ?

Cette procédure, comportant un certain nombre d'étapes, dont certaines sont facultatives (mode 9 bits, utilisation des interruptions, gestion des erreurs de réception ...) est détaillée ci-dessous :

Etape 1 : gestion de la vitesse de réception : la même qu'à l'émission !!!

Mettre la bonne valeur dans le registre **SPBRG** en fonction également du bit **BRGH**.

Etape 2 : validation de la liaison série :

Ici :

- **validation de la liaison** : **SPEN = 1**,
- en mode **asynchrone** : **SYNC = 0**,

Etape 3 : gestion de la broche **RC7/RX** :

Par :

TRISC7 = 1 // c'est bien une entrée ici

Etape 4 : fonctionnement en pooling ou sous interruption :

Et donc :

- **RCIE = 0** si polling
- **RCIE = 1** si interruption

Etape 5 : fonctionnement en mode 8 bits ou 9 bits par caractère :

Et donc :

- **RX9 = 0** si mode 8 bits
- **RX9 = 1** si mode 9 bits

Etape 6 : validation de la réception :

Par :

CREN = 1 ce qui placera automatiquement le bit **RCIF** à 0 puis **RCIF** passera à 1 à la fin de la réception d'un caractère

Etape 7 : lecture et gestion du 9^{ème} bit **RX9D** (si mode 9 bits choisi) et détection des éventuelles erreurs de réception (lecture des flags **OERR** et **FERR**) à faire avant étape 8.

Etape 8 : lecture du caractère reçu dans **RCREG**

Par :

(char) car = RCREG ; récupération du caractère reçu (ce qui remet RCIF à 0)

Etape 9 : gestion des éventuelles erreurs de réception et mise à 0 des flags d'erreurs.

Par :

CREN = 0 puis CREN = 1

Etape 10 : si fonctionnement sous interruption :

Il faut les démasquer par :

- **PEIE = 1** : démasquage semi-local
- **GIE = 1** : démasquage global

Certaines de ces étapes seront faites dans la fonction d'initialisation de la liaison série, d'autres dans la fonction qui gèrera la réception (si fonctionnement en pooling sans oublier dans ce cas d'attendre que le flag **RCIF** passe à 1 avant de lire le prochain caractère reçu) ou dans la routine d'IT (si interruption utilisée).

VIII.7.2.4. EXEMPLES DE MISE EN ŒUVRE

Dans ce premier exemple fourni sur la figure VIII.134., on relie la carte PICDEM2PLUS à un PC **fonctionnant en mode terminal** :

tout caractère frappé au clavier du PC est envoyé sur la liaison série du PC, est reçu par la carte PICDEM2PLUS et est renvoyé en écho vers le PC qui l’affiche alors sur l’écran de son mode terminal.

La liaison série asynchrone fonctionne ici **en pooling en émission et en réception**.

```
// initialisation de la liaison série
// faire le lien avec les étapes vues plus haut dans le cours
void init_liaison_serie(void)
{
    // étape 1
    BRGH = 1;                // mode high speed
    SPBRG = DIVISEUR;        // le diviseur (en fait 25 pour 9600 Bauds)
    // étape 2
    SYNC = 0;                // mode asynchrone
    SPEN = 1;                // validation des broches RC6 et RC7 pour liaison
    // étape 3
    TRISC6 = 1;              // à 1 même si sortie!!!
    TRISC7 = 1;              // à 1 aussi
    // étape 4 : ici en pooling de TXIF ou TXIE
    TXIE = 0;                // pas de transmission sous IT : en pooling
    RCIE = 0;                // pas de réception sous IT : en pooling
    // étape 5 : ici mode 8 bits
    TX9 = 0;                 // transmission sur 8 bits
    RX9 = 0;                 // réception sur 8 bits
    // étape 6
    TXEN = 1;                // validation de l'émetteur
    CREN = 1;                // validation du récepteur
}

// envoie un caractère sur la liaison série par pooling sur TXIF
void emit_car(unsigned char car)
{
    while(!TXIF);            // attend que le TXREG soit vide
    // TXIF = 0;              // inutile car repasse à 0 par écriture dans TXREG
    TXREG = car;              // charge le caractère pour l'envoyer
}

// reçoit un caractère sur la liaison série par pooling sur RCIF
// réception bloquante : il faudrait rajouter un time out
unsigned char recoit_car(void)
{
    while(!RCIF);            // attend la réception complète d'un caractère
    // RCIF = 0;              // inutile car repasse à 0 par lecture RCREG
    return RCREG;
}
```

Figure VIII.134.

```

void emet_string(char *string)
{
    while((*string)!=0)
    {
        emet_car(*string);
// gestion des LF et RC
        if (*string==13) emet_car(10); // rajoute LF
        if (*string==10) emet_car(13); // rajoute CR
        string++;
    }
}

unsigned char car;

void main(void)
{
    init_liaison_serie();

    emet_car(10);
    emet_car(13);

    emet_string("RELIER LA CARTE PICDEM 2 PLUS AU PC\x00A"); // \x00D ou \n à la ligne
// mode terminal sur le PC à 9600 Bauds 8 bits 1 stop et no parité

    emet_string("TAPEZ AU CLAVIER\x00A");

    while((car = recoit_car()) != 13) // détection toucher enter
    {
        emet_car(car); // renvoie en écho le caractère reçu
    }

    emet_car(10); // passe à la ligne si touche enter frappée

    emet_string("TERMINE!!\x00A"); // \x00D ou \n à la ligne
}

```

Figure VIII.134. (suite et fin)

Remarque :

La constante **DIVISEUR** est définie par :

```

// la vitesse de la liaison
#define BAUD 9600

// la valeur à écrire dans le registre SPBRG
// formule table 10.1. page 97 du datasheet
// avec le bit BRGH = 1
// si on ne met que 16 (au lieu de 16UL) ne calcule pas la bonne valeur !!!
#define DIVISEUR (4000000/(16UL * BAUD) - 1)

```

Dans un deuxième exemple, on va faire fonctionner la liaison série sous interruption en réception. (la transmission (par les fonctions **emet_car** (car) ou **emet_string** (*car)) sera toujours faite par pooling sur **TXIF**, comme dans l'exemple précédent de la figure VIII.134.). Le format de la liaison est le même que dans l'exemple précédent.

Le cahier des charges est le suivant :

*un certain nombre d'octets (ici **taille_max_buffer** vaut 10= taille maximale d'une commande tapée au clavier par exemple) doivent arriver sur la liaison série et doivent être rangés dans un buffer de réception (unsigned char **buffer**[taille_max_buffer]).*

*Par contre, ces octets arrivent n'importe quand et pas forcément au début du main, et pas forcément tout de suite, d'où l'intérêt de l'interruption en réception qui permet de faire autre chose en parallèle. Si on travaillait en pooling sur **RCIF** en réception, comme dans l'exemple précédent, on serait obligé d'attendre que tous les caractères arrivent (même s'ils mettent « une heure » à arriver (au risque d'en rater) et donc on ne pourrait rien faire en parallèle !!!*

Le programme est donné sur la figure VIII.135.

```
#define taille_max_buffer      10

unsigned char i=0, car, buffer_plein=0;buffer_trop_long=0;
unsigned char buffer[taille_max_buffer];

// initialisation de la liaison série
// faire le lien avec les étapes vues plus haut dans le cours

void init_liaison_serie(void)
{
// étape 1 :
    BRGH = 1;                // mode high speed
    SPBRG = DIVISEUR;        // le diviseur (en fait 25 pour 9600 Bauds)

// étape 2 :
    SYNC = 0;                // mode asynchrone
    SPEN = 1;                // validation des broches RC6 et RC7 pour liaison

// étape 3 :
    TRISC6 = 1;              // à 1 même si sortie!!!
    TRISC7 = 1;              // à 1 aussi

// étape 4 :
    TXIE = 0;                // pas de transmission sous IT : en pooling
    RCIE = 1;                // réception sous IT

/ étape 5 : ici mode 8 bits
    TX9 = 0;                 // transmission sur 8 bits
    RX9 = 0;                 // réception sur 8 bits

// étape 6 :
    TXEN = 1;                // validation de l'émetteur
    CREN = 1;                // validation du récepteur
}
```

Figure VIII.135.

```

// la routine d'interruption en réception
void interrupt reception_it (void)
{
    if(RCIF && RCIE)                // IT sur réception de caractère
    {
        buffer[i] = RCREG;          // range le caractère reçu et remet RCIF à 0

        if (buffer[i] == 0x0D)      // est-ce le dernier ?
        {
            buffer[i+1] = 0x00;     // oui, alors rajoute 0x00
            buffer_plein = 1;        // et buffer plein
        }
        if (i == (taille_max_buffer-1)) // commande trop longue
        {
            buffer_trop_long = 1;
        }
        i++;                        // non, alors caractère suivant
    }
}

void main(void)
{
    init_liaison_serie();

    // démasquage des interruptions
    PEIE = 1;                       // démasquage semi-local des IT
    GIE = 1;                        // démasquable global des IT

    emet_car(10);
    emet_car(13);

    emet_string("RECEPTION SOUS IT : TAPEZ COMMANDE AU CLAVIER\n");

    while(1)
    {
        while ((buffer_plein == 0) && (buffer_trop_long == 0))
        {
            // mettre ici le traitement à faire en parallèle
        }

        if (buffer_plein)            emet_string(buffer);          // renvoie la commande en écho
        if (buffer_trop_long)        emet_string("TROP LONG\n"); // renvoie message d'erreur

        buffer_plein = 0;            // on recommence
        buffer_trop_long = 0;
        i = 0;
    }
}

```

Figure VIII.135. (suite et fin)

VIII.7.2.5. MODE DETECTION D'ADRESSES

Ce mode correspond à une **utilisation spécifique du bit de parité**.

En effet, dans certains modes de fonctionnements spéciaux comportant par exemple :

- un maître qui initie les communications,
- et plusieurs récepteurs identifiables chacun par une adresse qui leur est propre,

comme sur un réseau ou sur un système multiprocesseurs par exemple, on peut utiliser le bit de parité **P** comme un **flag** permettant de spécifier si le maître transmet un caractère d'**adresse (mise à 1 de P)** ou un caractère de **donnée (mise à 0 de P)**.

Le maître transmet donc une trame comprenant un premier caractère d'adresse, les suivants étant les caractères de données, comme sur le chronogramme de la figure VIII.136. :

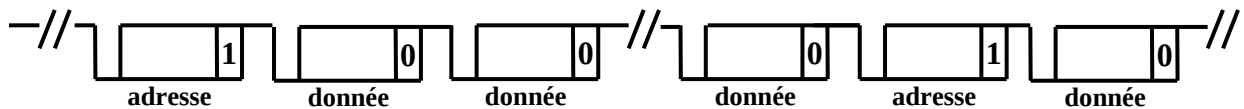


Figure VIII.136.

Dans ce cas, seul le récepteur adressé (ayant reconnu son adresse) traitera les données reçues et pourra initier aussi une communication avec le maître (suivant un protocole défini par avance).

Les autres récepteurs seront « aveugles » à ces échanges mais resteront en mode de surveillance permanente d'un nouvel octet d'adresse qui pourrait leur être destiné.

Pour utiliser ce mode, il faudra bien sûr travailler en mode 9 bits au niveau de l'émetteur et du récepteur (**TX9** et **RX9** à 1).

De plus, il faut mettre à 1, au niveau de chaque récepteur le bit **ADDEN** (bit 3 du registre **RCSTA**)

Au niveau de chaque récepteur, la partie permettant la détection d'adresse correspond à la logique repérée en ① sur le synoptique récepteur de la figure VIII.137. ci-dessous.

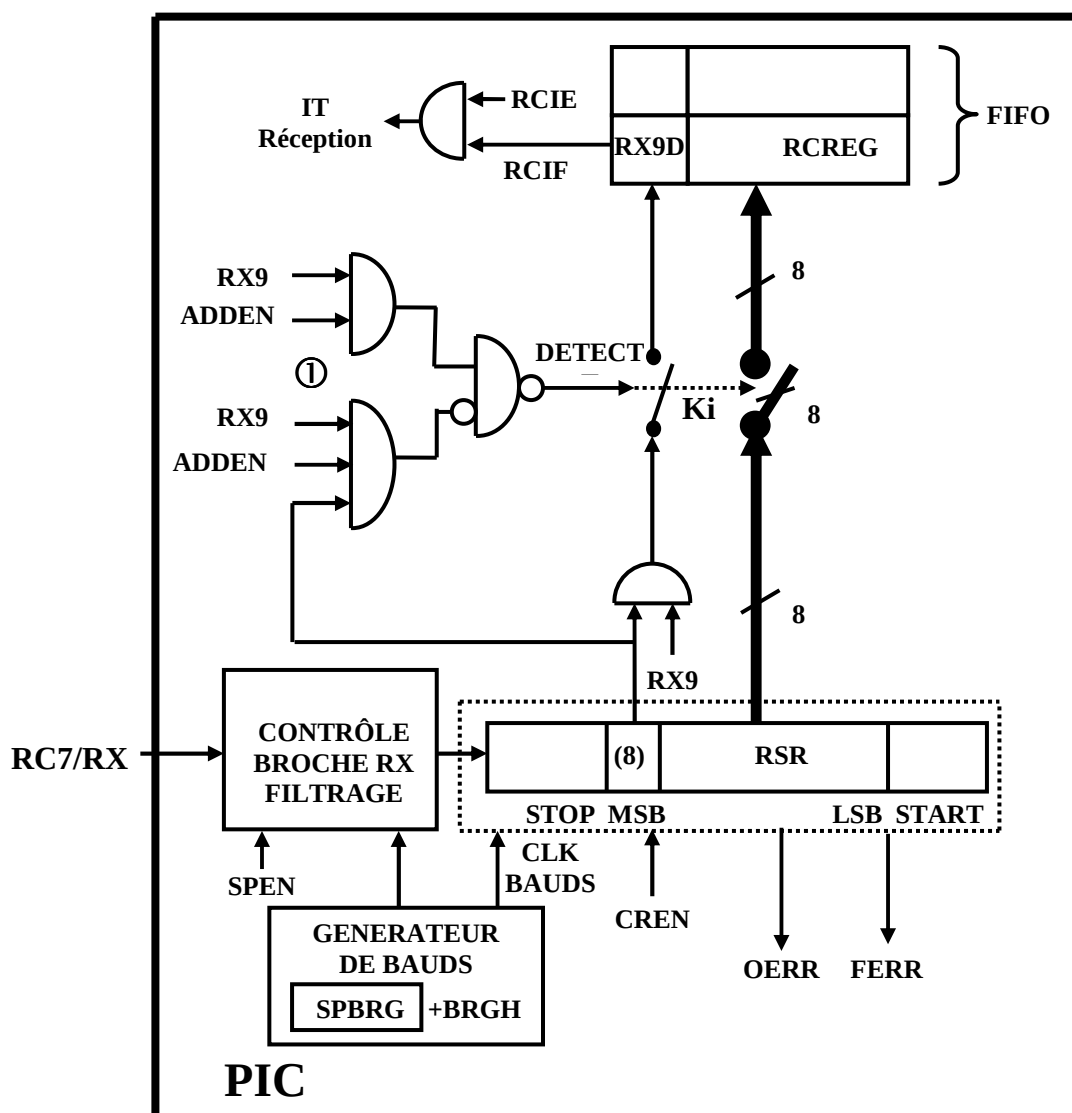


Figure VIII.137.

Lorsque la sortie **DETECT** de la logique ① passe à **1**, on vient fermer les 9 interrupteurs **Ki** pour venir charger le registre **RCREG** avec le contenu de **RSR**.

Cela se produit :

quand **RX9** = 1 et **ADDEN** = 1 et bit de parité **P** = bit 8 de **RSR** = 1
(mode détection d'adresse et octet de type adresse détecté)

ou

quand **RX9** = 0 ou **ADDEN** = 0
(adresse du récepteur concernée détectée et donc réception des octets de type caractère)

Le hardware ayant été présenté, quelle procédure faut-il utiliser pour gérer ce mode de détection d'adresse au niveau du récepteur ?

Cette procédure, comportant un certain nombre d'étapes, dont certaines sont facultatives (utilisation des interruptions, gestion des erreurs de réception ...) est détaillée ci-dessous :

Etape 1 : gestion de la vitesse de réception : la même qu'à l'émission !!!.

Mettre la bonne valeur dans le registre **SPBRG** en fonction également du bit **BRGH**.

Etape 2 : validation de la liaison série par :

- **validation de la liaison** : **SPEN = 1**,
- en mode **asynchrone** : **SYNC = 0**,

Etape 3 : gestion de la broche **RC7/RX** par :

TRISC7 = 1 // c'est bien une entrée ici

Etape 4 : fonctionnement en pooling ou sous interruption. Et donc :

- **RCIE = 0** si polling
- **RCIE = 1** si interruption

Etape 5 : fonctionnement en mode 9 bits par caractère :

Et donc :

- **RX9 = 1**

Etape 6 : fonctionnement en mode détection d'adresse:

Et donc :

- **ADDEN = 1**

Etape 7 : validation de la réception :

Par :

CREN = 1 ce qui placera automatiquement le bit **RCIF** à 0 puis **RCIF** passera à 1 à la fin de la réception d'une adresse (détection par pooling ou déclenchement d'IT)

Etape 8 : lecture et gestion du 9^{ème} bit **RX9D** et gestion des éventuelles erreurs de réception (lecture et gestion des flags **OERR** et **FERR**) à faire avant étape 9.

Etape 9 : lecture du caractère reçu dans **RCREG**

Par :

(char) car = RCREG ; récupération de l'adresse et vérification si c'est la bonne

Etape 10 : gestion des éventuelles erreurs de réception et mise à 0 des flags d'erreurs.

Par :

CREN = 0 puis CREN = 1

Etape 11 : si bonne adresse alors **mettre ADDEN à 0** et lecture et gestion des caractères suivants jusqu'à la prochaine adresse après laquelle **on remettra ADDEN à 1**. Et on continue à l'étape 8. Sinon, recommencer à l'étape 8.

Etape 12 : si fonctionnement sous interruption, ne pas oublier de les démasquer.

Il faut les démasquer par :

- **PEIE = 1** : démasquage semi-local
- **GIE = 1** : démasquage global

Voici au final sur la figure VIII.138. les registres concernés par l'U.A.R.T. ainsi que les différents bits concernés.

- pour la **transmission** (cf. figure VIII.138.a.) :

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh,18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	ROIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	—	FERR	OERR	RX9D	0000 -00x	0000 -00x
19h	TXREG	USART Transmit Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Legend: x = unknown, - = unimplemented locations read as '0'. Shaded cells are not used for asynchronous transmission.

Note 1: Bits PSPIE and PSPIF are reserved on the PIC16F873/876; always maintain these bits clear.

(a)

- pour la **réception** (cf. figure VIII.138.b.) :

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh,18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	ROIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	—	FERR	OERR	RX9D	0000 -00x	0000 -00x
1Ah	RCREG	USART Receive Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Legend: x = unknown, - = unimplemented locations read as '0'. Shaded cells are not used for asynchronous reception.

Note 1: Bits PSPIE and PSPIF are reserved on PIC16F873/876 devices; always maintain these bits clear.

(b)

Figure VIII.138.

VIII.7.3. LE MODE SYNCRONE DE L'U.S.A.R.T.

L'U.S.A.R.T. du PIC peut aussi fonctionner en mode **synchrone**. On l'appellera alors **U.S.R.T.**

Mais contrairement au mode asynchrone qui est full duplex, le mode synchrone est **half duplex** uniquement ce qui veut dire que, à tout moment :

- soit l'U.S.R.T. est **émetteur** : les bits sont émis (bits **sortants**) par le **PIC**,
- soit l'U.S.R.T. est **récepteur** : les bits sont reçus (bits **entrants**) par le **PIC**.

Mais jamais les deux en même temps, car en fait et comme on le verra plus loin, sur la liaison, il n'y a qu'un seul fil véhiculant ces bits.

Et donc, il faudra, pour établir une communication, établir un protocole pour éviter les collisions.

D'où également la notion de maître/esclave qu'on verra aussi plus loin.

VIII.7.3.1. GENERALITES SUR LES LIAISONS SYNCHRONES

Le terme **synchrone** doit être interprété de la façon suivante :

à tout instant, il existe un lien temporel entre l'émetteur et le récepteur.

Pratiquement cela signifie qu'il existe une référence de temps commune et permanente (tout au moins dans la phase de connexion) entre les deux systèmes. Cette référence peut être :

- soit fournie par une horloge véhiculée **en plus** des signaux informationnels proprement dits (solution utilisée en pratique pour les très courtes distances),
- soit en fabricant, au niveau du récepteur, une horloge extraite du message lui même grâce à un type de codage adéquat. Au moyen d'une **boucle à verrouillage de phase numérique** ou **D.P.L.L. (Digital Phase Locked Loop)** intégrée dans le composant gestionnaire de la liaison synchrone et pilotée en absence de signal reçu par une horloge (fréquence libre de la D.P.L.L.) de période 16 à 32 fois plus faible que la durée nominale d'un bit, il est possible de synthétiser en quelques cycles, une autre horloge calée sur les données reçues en recherchant les fronts montants ou descendants sur le signal reçu (cas des codages N.R.Z.I. ou F.M. par exemple),
- soit à un niveau supérieur, réalisée par des séquences de bits identifiables (flags). C'est le cas notamment des protocoles **SDLC/HDLC** (Serial or **H**igh **D**ata **L**ink **C**ontrol) utilisés dans les réseaux **X25**,
- soit présente sous la forme d'octets spécifiques (dits de synchro) dans les protocoles orientés octets. En pratique, il existe des protocoles avec un seul caractère de synchro (Monosync) ou deux caractères de synchro (Bisync),
- soit un signal externe.

De façon générale, les liaisons synchrones permettent **des vitesses de transmission plus élevées** que les liaisons série asynchrones (typiquement jusqu'à plusieurs Mégabits/seconde).

Elles seront aussi et par exemple bien adaptées pour les échanges avec des composants comme certains **CAN** ou **CNA** ou avec des mémoires à **accès série**.

VIII.7.3.2. SYNOPTIQUES ET REGISTRES ASSOCIES A L'U.S.R.T.

C'est le même hardware interne que pour le fonctionnement asynchrone vu précédemment sur les figures VIII.127. pour l'émetteur et VIII.131. pour le récepteur. Mais utilisé de façon différente.

De même, les registres de contrôle pour le fonctionnement synchrone sont aussi :

- **TXSTA** pour l'émetteur (cf. figure VIII.129. et VIII.130.),
- **RCSTA** pour le récepteur (cf. figure VIII.132. et VIII.133.).

Avec, là aussi, quelques spécificités au niveau de certains de leurs bits.

Quelles sont ces différences et ces spécificités ?

- **au niveau des broches et de la communication :**

- la broche **RC7/RX/DT** sert pour **les bits série** (signal **DT**)

Comme il n'y a qu'un seul fil **DT** pour les bits, la communication sera **half duplex**, ce qui veut dire que **transmission et réception ne peuvent se faire en même temps**, contrairement au mode asynchrone qui était full duplex.

Par contre, on peut faire du « faux » full duplex (ou appelé aussi full duplex à l'alternat), en alternant temporellement des périodes d'émission et de réception, suivant un protocole défini préalablement.

- la broche **RC6/TX/CK** sert pour **l'horloge** (signal **CK**)

Cette horloge **CK**, qui comme on l'a dit est utile pour la synchronisation des bits, sera fournie soit par l'émetteur, soit par le récepteur (mais pas par les deux en même temps).

Celui qui **fournit** cette horloge sera appelé le **maître (master)** et celui qui la **reçoit** sera l'**esclave (slave)**.

Comment est utilisée cette horloge **CK**? On peut le voir sur la figure VIII.139. ci-dessous.

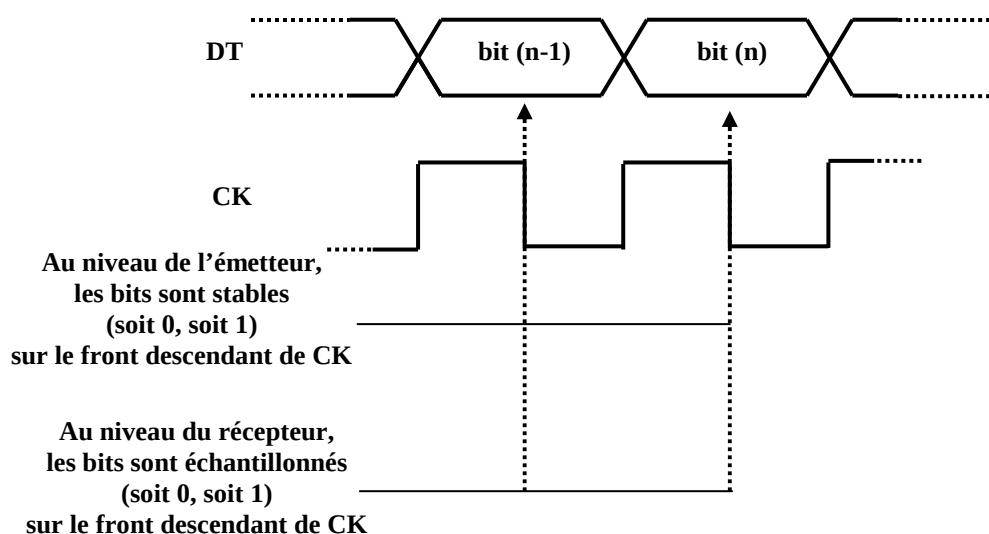


Figure VIII.139.

- au niveau des registres de contrôle et d'état :

- pour le registre de contrôle et d'état de l'émission : **TXSTA**

Les bits concernés **exclusivement** par le mode synchrone sont résumés sur la figure VIII.140.

Les autres bits du registre **TXSTA** gardent, en mode synchrone, la même signification que pour le mode asynchrone, sauf pour le bit **BRGH** qui n'est pas pris en compte en mode synchrone.

Bit	Valeur	Signification et commentaires
CSRC		Utilisation en mode maître ou en esclave de l'U.S.R.T.
	0	Mode maître (qui fabrique l'horloge CK)
	1	Mode esclave (qui reçoit l'horloge, la plupart du temps venant du maître)
SYNC		Choix du mode de fonctionnement l'U.S.A.R.T.
	0	Mode asynchrone
	1	Mode synchrone

Figure VIII.140.

- pour le registre de contrôle et d'état de la réception : **RCSTA**

Les bits concernés **exclusivement** par le mode synchrone sont résumés sur la figure VIII.141.

Les autres bits du registre **RCSTA** gardent, en mode synchrone, la même signification que pour le mode asynchrone, sauf pour le bit **ADDEN** qui n'est pas pris en compte en mode synchrone.

Bit	Valeur	Signification et commentaires
SREN		Validation de la réception pour un seul caractère (SINGLE) en mode master uniquement mais en mode slave non utilisé
	0	Single réception non validée
	1	Single réception validée Ce bit est remis à 0 à la réception de cet unique caractère
CREN		Validation de la réception continue de caractère (CONTINUOUS)
	0	Continuous réception non validée
	1	Continuous réception validée Ce bit a prédominance sur le bit SREN (si CREN = 1, même si SREN = 1 : continuous réception)

Figure VIII.141.

- pour le registre de bauds: **SPBRG**

En mode synchrone, le bit **BRGH** n'est pas pris en compte, et la vitesse **V** (et donc sa fréquence en Hz) de l'horloge **CK** générée par le maître est donnée par :

$$V = \frac{F_{osc}}{4 \cdot (SPBRG + 1)}$$

VIII.7.3.3. METHODOLOGIE ET PROGRAMMATION

VIII.7.3.3.1. LE MODE MAÎTRE

• maître en mode transmission :

Le hardware ayant été présenté, quelle procédure faut-il utiliser pour **transmettre** un caractère sur la liaison série **synchrone** en mode **maître** ?

Cette procédure, comportant un certain nombre d'étapes, dont certaines sont facultatives (mode 9 bits, utilisation des interruptions...) est détaillée ci-dessous :

Etape 1 : gestion de la vitesse de transmission.

Mettre la bonne valeur dans le registre **SPBRG** en fonction de la fréquence souhaitée pour **CK**.

Etape 2 : validation de la liaison série en mode maître synchrone. Ici :

- en mode **synchrone** : **SYNC = 1**,
- **validation de la liaison** : **SPEN = 1**,
- et en mode **maître** : **CSRC = 1**.

Etape 3 : fonctionnement en pooling ou sous interruption :

Et donc :

- **TXIE = 0** si polling
- **TXIE = 1** si interruption

Etape 4 : fonctionnement en mode 8 bits ou 9 bits par caractère :

Et donc :

- **TX9 = 0** si mode 8 bits
- **TX9 = 1** si mode 9 bits

Etape 5 : validation de la transmission :

Par :

TXEN = 1 ce qui placera automatiquement le bit **TXIF** à 1

Etape 6 : gestion du 9^{ème} bit, si mode 9 bits choisi (à faire impérativement avant étape 7 car sinon le bit **TX9D** ne correspondra plus au bon caractère, mais au suivant).

Par :

TX9D = 1 ou **0** : c'est en fait la valeur du 9^{ème} bit

Etape 7 : chargement du caractère à transmettre dans **TXREG**

Par :

TXREG = valeur du caractère que l'on veut envoyer sur la liaison (ce qui remet TXIF à 0).

Etape 8 : si fonctionnement sous interruption :

Il faut les démasquer par :

- **PEIE = 1** : démasquage semi-local
- **GIE = 1** : démasquage global

• **maître en mode réception :**

Le hardware ayant été présenté, quelle procédure faut-il utiliser pour **recevoir** un caractère sur la liaison série **synchrone** en mode **maître** ?

Cette procédure, comportant un certain nombre d'étapes, dont certaines sont facultatives (mode 9 bits, utilisation des interruptions, gestion des erreurs de réception ...) est détaillée ci-dessous :

Etape 1 : gestion de la vitesse de réception : la même qu'à l'émission !!!.

Mettre la bonne valeur dans le registre **SPBRG** en fonction de la fréquence souhaitée pour **CK**.

Etape 2 : validation de la liaison série en mode maître synchrone. Ici :

- en mode **synchrone** : **SYNC = 1**,
- **validation de la liaison** : **SPEN = 1**,
- et en mode **maître** : **CSRC = 1**.

Etape 3 : initialisation du choix du mode de réception. Ici, en faisant :

- **SREN = 0**,
- **CREN = 0**.

Etape 4 : fonctionnement en pooling ou sous interruption :

Et donc :

- **RCIE = 0** si polling
- **RCIE = 1** si interruption

Etape 5 : fonctionnement en mode 8 bits ou 9 bits par caractère. Et donc :

- **RX9 = 0** si mode 8 bits
- **RX9 = 1** si mode 9 bits

Etape 6 : gestion effective du mode de réception (single ou continuous), en faisant :

- **SREN = 1** si mode réception **d'un seul caractère** choisi.

ou

- **CREN = 1** si mode de réception **continue** choisi.

RCIF passera à 1 à la fin de la réception du premier caractère.

Etape 7 : lecture et gestion du 9^{ème} bit **RX9D** (si mode 9 bits choisi) et détection des éventuelles erreurs de réception (lecture du flag **OERR**) à faire avant étape 8.

Etape 8 : lecture du caractère reçu dans **RCREG**

Par :

(char) car = RCREG ; récupération du caractère reçu (ce qui remet RCIF à 0)

Etape 9 : gestion des éventuelles erreurs de réception et mise à 0 des flags d'erreurs.

Par :

CREN = 0 puis CREN = 1

Etape 10 : si fonctionnement sous interruption :

Il faut les démasquer par :

- **PEIE = 1** : démasquage semi-local
- **GIE = 1** : démasquage global

VIII.7.3.3.2. LE MODE ESCLAVE

● esclave en mode transmission :

Notez bien qu'il n'y a pas ici de gestion de la fréquence de **CK** car l'horloge, en mode esclave est fournie par l'extérieur (en général celle du maître) sur la patte **RC6/TX/CK**

Etape 1 : validation de la liaison série en mode esclave synchrone. Ici :

- en mode **synchrone** : **SYNC = 1**,
- **validation de la liaison** : **SPEN = 1**,
- et en mode **esclave** : **CSRC = 0**.

Etape 2 : mise à 0 du choix du mode de réception. Ici, en faisant :

- **SREN = 0**,
- **CREN = 0**.

Etape 3 : fonctionnement en pooling ou sous interruption :

Et donc :

- **TXIE = 0** si polling
- **TXIE = 1** si interruption

Etape 4 : fonctionnement en mode 8 bits ou 9 bits par caractère :

Et donc :

- **TX9 = 0** si mode 8 bits
- **TX9 = 1** si mode 9 bits

Etape 5 : validation de la transmission :

Par :

TXEN = 1 ce qui placera automatiquement le bit **TXIF** à 1

Etape 6 : gestion du 9^{ème} bit, si mode 9 bits choisi (à faire impérativement avant étape 7 car sinon le bit **TX9D** ne correspondra plus au bon caractère, mais au suivant).

Par :

TX9D = 1 ou **0** : c'est en fait la valeur du 9^{ème} bit

Etape 7 : chargement du caractère à transmettre dans **TXREG**

Par :

TXREG = valeur du caractère que l'on veut envoyer sur la liaison (ce qui remet TXIF à 0).

Etape 8 : si fonctionnement sous interruption :

Il faut les démasquer par :

- **PEIE = 1** : démasquage semi-local
- **GIE = 1** : démasquage global

- **esclave en mode réception :**

Notez bien qu'il n'y a pas ici de gestion de la fréquence de **CK** car l'horloge, en mode esclave est fournie par l'extérieur (en général celle du maître) sur la patte **RC6/TX/CK**

Etape 1 : validation de la liaison série en mode esclave synchrone.

Ici :

- en mode **synchrone** : **SYNC = 1**,
- **validation de la liaison** : **SPEN = 1**,
- et en mode **esclave** : **CSRC = 0**.

Etape 2 : fonctionnement en pooling ou sous interruption :

Et donc :

- **RCIE = 0** si polling
- **RCIE = 1** si interruption

Etape 3 : fonctionnement en mode 8 bits ou 9 bits par caractère.

Et donc :

- **RX9 = 0** si mode 8 bits
- **RX9 = 1** si mode 9 bits

Etape 4 : validation de la réception.

En faisant :

- **CREN = 1**

RCIF passera à 1 à la fin de la réception du premier caractère.

Etape 5 : lecture et gestion du 9^{ème} bit **RX9D** (si mode 9 bits choisi) et détection des éventuelles erreurs de réception (lecture du flag **OERR**) à faire avant étape 6.

Etape 6 : lecture du caractère reçu dans **RCREG**

Par :

(char) car = RCREG = récupération du caractère reçu (ce qui remet RCIF à 0)

Etape 7 : gestion des éventuelles erreurs de réception et mise à 0 des flags d'erreurs.

Par :

CREN = 0 puis CREN = 1

Etape 8 : si fonctionnement sous interruption :

Il faut les démasquer par :

- **PEIE = 1** : démasquage semi-local
- **GIE = 1** : démasquage global

Voici au final sur la figure VIII.142. les registres concernés par l'U.S.R.T. ainsi que les différents bits concernés.

- pour la **transmission** en mode **maître** (cf. figure VIII.142.a.) :

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	ROIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	—	FERR	OERR	RX9D	0000 -00x	0000 -00x
19h	TXREG	USART Transmit Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Legend: x = unknown, - = unimplemented, read as '0'. Shaded cells are not used for synchronous master transmission.

Note 1: Bits PSPIE and PSPIF are reserved on PIC16F873/876 devices; always maintain these bits clear.

(a)

- pour la **réception** en mode **maître** (cf. figure VIII.142.b.) :

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	ROIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	—	FERR	OERR	RX9D	0000 -00x	0000 -00x
1Ah	RCREG	USART Receive Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Legend: x = unknown, - = unimplemented, read as '0'. Shaded cells are not used for synchronous master reception.

Note 1: Bits PSPIE and PSPIF are reserved on PIC16F873/876 devices; always maintain these bits clear.

(b)

Figure VIII.142.

- pour la **transmission** en mode **esclave** (cf. figure VIII.143.a.) :

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	ROIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000x	0000 000x
19h	TXREG	USART Transmit Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Legend: x = unknown, - = unimplemented, read as '0'. Shaded cells are not used for synchronous slave transmission.

Note 1: Bits PSPIE and PSPIF are reserved on PIC16F873/876 devices; always maintain these bits clear.

(a)

- pour la **réception** en mode **esclave** (cf. figure VIII.143.b.) :

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on all other RESETS
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	ROIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
18h	RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	0000 000x	0000 000x
1Ah	RCREG	USART Receive Register								0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
98h	TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D	0000 -010	0000 -010
99h	SPBRG	Baud Rate Generator Register								0000 0000	0000 0000

Legend: x = unknown, - = unimplemented, read as '0'. Shaded cells are not used for synchronous slave reception.

Note 1: Bits PSPIE and PSPIF are reserved on PIC16F873/876 devices; always maintain these bits clear.

(b)

Figure VIII.143.

VIII.8. LE MODULE MSSP

(cf. **12** du schéma de la **figure II.4.** et cf. **pages 71 à 109** du datasheet)

Le module MSSP (Master Synchronous Serial Port) est, comme son nom l'indique, une autre interface série très utilisée (et presque plus souvent que la liaison série synchrone pilotée par le module U.S.R.T. précédemment vue) pour interfacer d'autres périphériques comme :

- des mémoires série,
- des afficheurs,
- des CAN ou des CNA,
- différents types de capteurs,
- des circuits audio et ou vidéo,
- des récepteurs divers,
- des circuits « expanseurs » de bus...
- mais aussi bien d'autres systèmes à base de micro/processeurs/contrôleurs, disposant tous de ce type d'interface.

Ce module **MSSP** peut fonctionner selon 2 modes différents :

- le **mode SPI** que nous présenterons dans un premier temps,
- le **mode I²C** que nous détaillerons ensuite.

Autres exemples de bus série synchrones :

- le bus **CAN** (Controler Area Network), surtout utilisé dans le secteur de l'automobile dans le but de réduire la quantité de câbles dans les véhicules (il peut y avoir jusqu'à 1 voire 2 km de câble par voiture !!!) et qui permet de faire communiquer entre eux les différents organes de commande.
- le bus **USB** (Universal Serial Bus) servant à connecter des périphériques informatiques à un ordinateur. Il permet en outre de connecter des périphériques à chaud, en bénéficiant du **Plug and Play** grâce à un protocole se rajoutant à la couche matérielle.

VIII.8.1. LE MODE SPI

(cf. pages 71 à 79 du datasheet)

VIII.8.1.1. GENERALITES SUR LE BUS SPI

Comme d'habitude, nous donnerons dans un premier temps les principes généraux du mode fonctionnement d'une liaison **SPI**.

Et si vous n'êtes pas familier avec ces notions, il est fortement conseillé de commencer par là, car cela ne sert à rien de mettre en œuvre la liaison **SPI** si on n'a pas bien compris comment elle fonctionne et à quoi elle sert. Le temps que vous pensez, à tort, avoir gagné en sautant cette étape, sera très vite perdu par la suite...

La **SPI** (Serial Peripheral Interface) est une interface de type **série synchrone** qui permet la connexion en **Full Duplex** de plusieurs circuits en mode **Maître/Esclave (Master/Slave)**.

Dans le mode **SPI**, **c'est le maître qui s'occupe totalement de la communication**, l'(ou les) esclave(s) répondant aux requêtes du maître.

Les données et l'horloge sont sur des fils différents, l'horloge n'est pas « incluse » (puis « extraite ») dans le flot des bits série, mais doit bien être véhiculée sur un fil supplémentaire.

Il y aura donc au total **4 fils** sur une liaison **SPI** :

- **2 fils pour les bits échangés** : un fil pour chaque sens de la communication, notés, dans cette introduction générale :
 - **M2S** comme **Master To Slave** : sens **maître vers esclave**.
 - **S2M** comme **Slave To Master** : sens **esclave vers maître**.
- **1 fil pour l'horloge** noté **SCK** (comme **Serial Clock**).
- **1 fil pour la masse GND**.

Ce qui est résumé sur la figure VIII.144. ci-dessous, pour le cas simple d'un maître et d'un seul esclave.

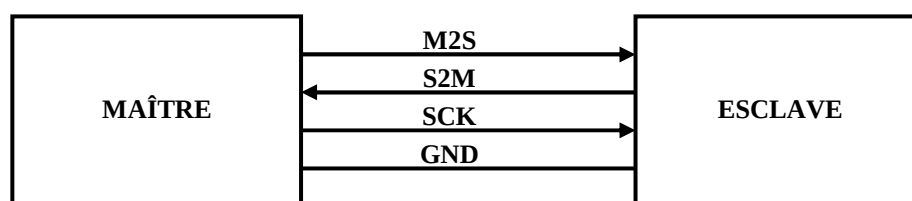


Figure VIII.144.

Il faut maintenant rentrer à l'intérieur du maître et de l'esclave pour comprendre le principe général d'une liaison **SPI** comme indiqué sur la figure VIII.145. montrant un exemple de connexion entre un maître (à gauche) et son esclave (à droite).

Au cœur de la **SPI**, se trouvent **deux registres à décalage**, à **entrée série (SDI** comme **Serial Data In**) et à **sortie série (SDO** comme **Serial Data Out**), de taille **8 bits** chacun :

- le registre **MSR** : comme **Master Sérial Register** situé dans le **maître**,
- le registre **SSR** : comme **Slave Sérial Register** situé dans l'**esclave**.

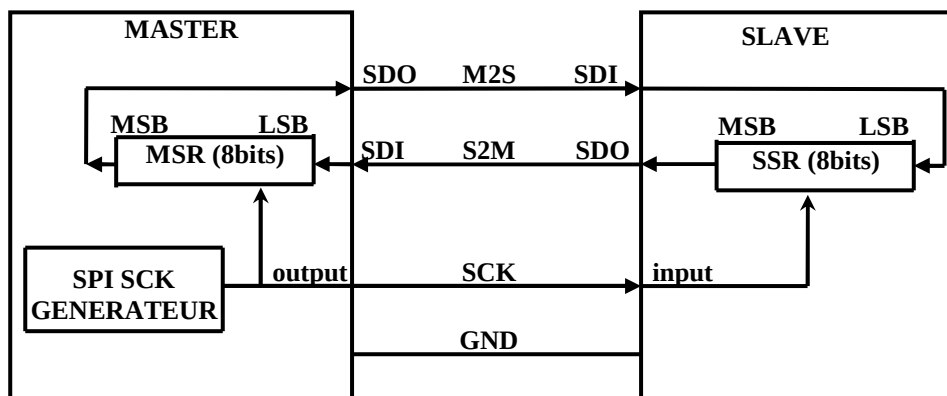


Figure VIII.145.

On peut faire notamment les remarques suivantes :

- ligne **SCK** :

c'est le signal d'horloge qui est une sortie pour le maître et une entrée pour l'esclave car c'est bien **le maître qui génère l'horloge** par son bloc **SPI SCK GENERATEUR**.

Elle permet la synchronisation (à l'émission) et l'échantillonnage (à la réception) des bits série.

- ligne **M2S** :

entre la broche de sortie **SDO** du maître et la broche d'entrée **SDI** de l'esclave, elle assure bien le passage des bits série **du maître vers l'esclave**,

- ligne **S2M** :

entre la broche de sortie **SDO** de l'esclave et la broche d'entrée **SDI** du maître, elle assure bien le passage des bits série de **l'esclave vers le maître**.

Ainsi, à chaque coup d'horloge, le maître et l'esclave s'échangent un bit (le **MSB** en premier).

Après huit coups d'horloge le maître a transmis un octet complet à l'esclave et vice-versa.

On voit d'ailleurs sur le schéma de la figure VIII.145. précédente que l'ensemble des 2 registres à décalage 8 bits (**MSR** pour le maître et **SSR** pour l'esclave) forment **l'équivalent d'un registre à décalage 16 bits unique** partagé entre le maître et l'esclave :

l'émission par le maître d'un octet contenu dans le **MSR** vers l'esclave provoque en même temps la réception par le maître de l'octet contenu le **SSR** de l'esclave (le registre à décalage du maître « se vide » dans celui de l'esclave pendant que celui de l'esclave « remplit » celui du maître).

**Mais c'est toujours le maître qui décide des transferts :
il doit émettre un octet, pour en recevoir un autre.**

VIII.8.1.2. SYNOPTIQUE

Le synoptique de l'électronique gérant la liaison **SPI** est fourni sur la figure VIII.146. ci-dessous.

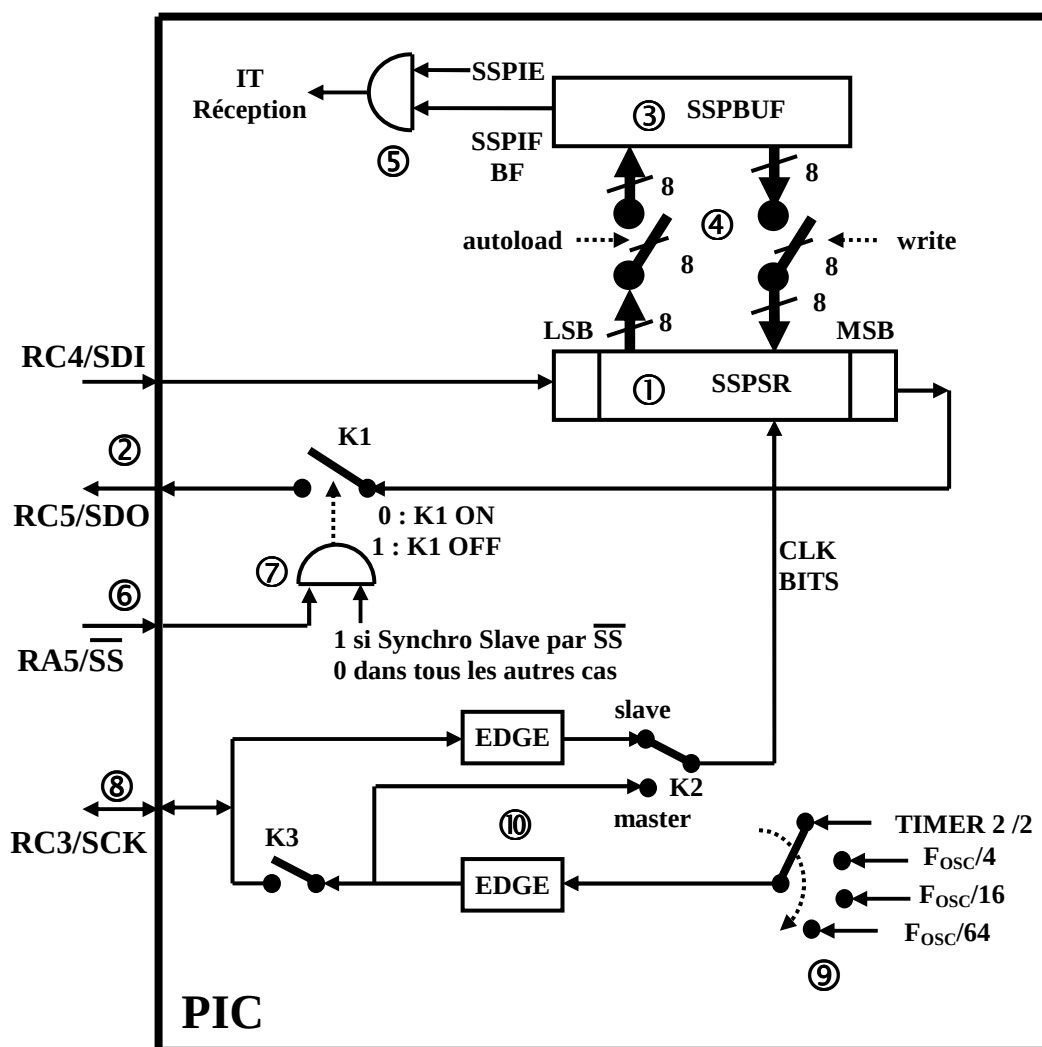


Figure VIII.146.

Le cœur du module **SPI** est le registre à décalage **SSPSR** (cf ①) qui contient l'octet à envoyer au début de l'échange et qui contiendra l'octet reçu à la fin de l'échange (après 8 coups d'horloge).

Les bits sortent un à un (avec le **MSB** en premier) par la sortie **SDO** (broche **RC5/SDO**) et rentrent également un par un, sur le même coup d'horloge par l'entrée **SDI** (broche **RC4/SDI**) comme indiqué en ②.

Le registre **SSPSR** n'étant pas mappé, il n'est pas directement accessible au programmeur.

Il faut utiliser le registre tampon **SSPBUF** ③ pour échanger les données avec la **SPI**.

Quand le maître écrit un octet dans **SSPBUF** (commande **write** du bloc ④), cet octet est **automatiquement** écrit dans **SSPSR**.

Et le transfert peut commencer... : une salve de 8 coups d'horloge **SCK** est générée par le maître et les bits émis sortent par la sortie **SDO** et ceux reçus en même temps arrivent en provenance de l'esclave sur l'entrée **SDI** à chaque coup d'horloge.

Lorsque les 8 bits de l'octet reçu sont arrivés dans **SSPR**, cet octet est **automatiquement** transféré (commande **autoload** du bloc ④) dans le registre **SSPBF** et le bits **BF** et **SSPIF** passent à 1 (ce qui peut provoquer une interruption de réception si le bit **SSPIE** est à 1 : cf. ⑤). L'acquittement de cette interruption se fera en remettant **SSPIF** à 0 par soft : **SSPIF = 0**)

Si on travaille en pooling sur le bit **BF**, il faudra attendre que ce bit passe à 1 (attendre que l'octet complet soit arrivé), puis lire le registre **SSPBUF** (octet reçu, ce qui remettra **BF** à 0) puis écrire dans ce même registre **SSPBUF** le prochain caractère à envoyer.

Remarquons également que si on écrit dans le registre **SSPBF** alors que le transfert est en cours (octet précédent pas encore reçu), cette erreur est signalée par la mise à 1 du bit **WCOL** (signalant une **collision**).

De même, en mode esclave, si un nouvel octet arrive alors que le précédent n'a pas encore été lu (toujours présent dans **SSPBUF**), le bit **SSPOV** (signalant un **overflow**) passera à 1.

Expliquons maintenant le rôle de la broche **RA5/SS** (cf. ⑥).

- **en mode maître**, l'interrupteur **K1** est toujours fermé (il est recommandé dans ce cas de mettre une résistance de pull up vers le 5V sur la broche **RA5/SS** du maître, pour forcer cette entrée à 1) ce qui assure la sortie des bits série, sauf si la broche **RC5/SDO** est mise en entrée par : **TRISC5 = 1** et dans ce cas, le maître ne fait que recevoir et il n'émet rien.

- **en mode esclave**, la logique repérée ⑦ sur le schéma, peut permettre, si le mode de synchronisation par **SS** a été choisi, de n'avoir **K1** fermé que si la broche **RA5/SS** est au niveau bas.

Lorsque cette broche est au niveau haut, **K1** est ouvert, et les bits ne sortent plus sur la broche de sortie **RC5/SDO** pour cet esclave.

Ce procédé est utile par exemple, lorsque l'on a plusieurs esclaves sur le bus **SPI**.

Seul l'esclave ayant sa broche d'entrée **RA5/SS** au niveau bas répondra au maître, les autres seront muets.

Il faudra bien sûr dans ce cas rajouter une petite logique de décodage extérieure, pilotée par le maître (ou un port parallèle du maître) pour sélectionner l'esclave qui doit répondre à la requête du maître.

Il ne reste plus qu'à parler de l'horloge **SCK**, qui est, rappelons le :

- **générée** par le maître (interrupteur **K2** en position **master**),
- et **reçue** par l'esclave (interrupteur **K2** en position **slave**)

sur la broche **RC3/SCK** (cf. ⑧).

- pour le maître, l'interrupteur **K3** est fermé pendant les phases de transfert, ce qui assure bien que l'horloge **SCK** sera présente sur la broche **RC3/SCK**.

Grâce au multiplexeur ⑨ à 4 entrées et 1 sortie, le signal de **SCK** peut provenir soit de la sortie du **TIMER 2** (après avoir divisé sa fréquence par 2), soit directement du quartz (**F_{OSC}**) dont la fréquence a été divisée par 4, 16 ou 64.

Ainsi, avec un quartz à 4 MHz, on peut faire fonctionner la liaison SPI à une vitesse de 1 Méga bits par seconde.

- pour l'esclave, l'interrupteur **K3** est ouvert en permanence, l'horloge **SCK** arrive de l'extérieur sur la broche **RC3/SCK**.

Les deux blocs **EDGE** référencés ⑩ sur le synoptique précédent servent pour fixer les instants d'horloge **SCK** où les bits seront sortis sur la broche **SDO** (pour l'émission) et les instants où les bits seront échantillonnés sur la broche **SDI** (pour la réception), sachant qu'en absence d'émission, le signal **SCK** est dans un état dit de **repos**. L'état opposé du repos est alors appelé **actif**.

En pratique, ce sont deux bits qui permettent de définir ces instants :

- bit **CKP** (ClocK Polarity) pour la **polarité** de l'horloge **SCK**, avec :

- **CKP = 1** : niveau de repos **haut** pour **SCK**,
- **CKP = 0** : niveau de repos **bas** pour **SCK**.

Ce bit doit être **le même** pour le maître et le (ou les) esclave(s).

- bit **CKE** (ClocK Edge) pour le front **actif** de l'horloge **SCK**, avec :

- **CKE = 1** : front entre état **actif** et état de **repos**,
- **CKE = 0** : front entre état de **repos** et état **actif**.

Ce qui donne au final 4 possibilités pour la sortie des bits série sur **SDO** :

CKP	CKE	Front actif
0	0	SCK à 0 au repos, premier front montant de SCK
0	1	SCK à 0 au repos, premier front descendant de SCK
1	0	SCK à 1 au repos, premier front descendant de SCK
1	1	SCK à 1 au repos, premier front montant de SCK

Ce qui est résumé sur la figure VIII.147. pour l'émission d'un octet (écrit dans **SSPBUF**) par le maître.

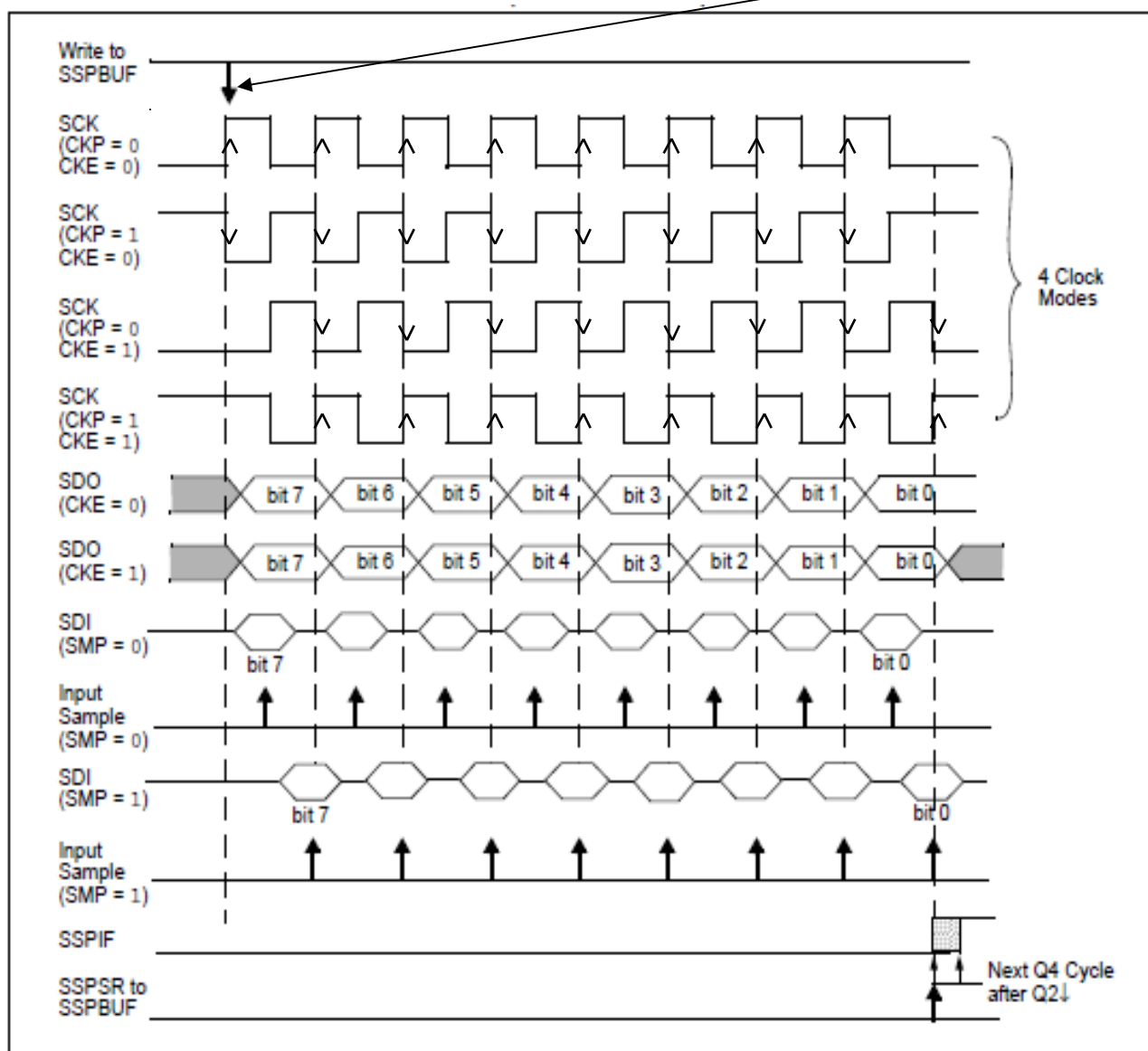


Figure VIII.147.

On voit que si **CKE = 1**, le bit sur **SDO** est stable avant le front actif.

La réception de chaque bit par l'esclave sur son entrée **SDI** se fera sur le front de **SCK** qui suit le front pour lequel il a été programmé pour sa propre sortie **SDO**, et donc au milieu de chaque bit reçu.

La configuration de la **SPI** est souvent nommée dans les documentations techniques sous forme du nombre (appelé alors le **mode**) variant entre 0 et 3.

La correspondance entre le mode et les bits **CKP** et **CKE** est donnée dans le tableau suivant :

Mode	CKP	CKE
0	0	1
1	0	0
2	1	1
3	1	0

Les modes 0 et 3 sont identiques en transmission, seul change l'état de la ligne d'horloge au repos. (idem pour les modes 1 et 2)

C'est pour cette raison que de nombreux périphériques peuvent être capables de dialoguer soit en mode 0 et 3, soit en mode 1 et 2.

Un bit supplémentaire, le bit **SMP** comme **SaMP**le bit, valable uniquement en mode maître, permet de savoir l'instant où le maître échantillonne son entrée **SDI** :

- si **SMP = 1** : mode d'échantillonnage dit **fin de cycle** :
lecture du bit sur **SDI** au moment où il sort le bit suivant sur **SDO** (fin de cycle)
(cas où l'esclave place son bit sur sa sortie **SDO** en milieu de cycle : **CKE** différent pour maître et esclave)
- si **SMP = 0** : mode d'échantillonnage dit **milieu de cycle** :
lecture du bit sur **SDI** sur le front de **SCK** qui suit l'émission du bit courant.
(cas où l'esclave place son bit sur sa sortie **SDO** en début de cycle : même **CKE** pour maître et esclave)

Pour bien comprendre toute cette mécanique d'émission et de réception de bit, on résume sur la figure VIII.148. le cas d'un échange complet entre le maître et l'esclave avec :

- **CKP = 1** : horloge **SCK** au repos au niveau **haut**,
- **CKE = 0** : sortie des bits par le maître et l'esclave sur premier front **descendant** de **SCK**,
- **SMP = 0** : lecture par le maître et l'esclave **au milieu** du bit.

ce qui correspond à une configuration assez fréquemment utilisée.

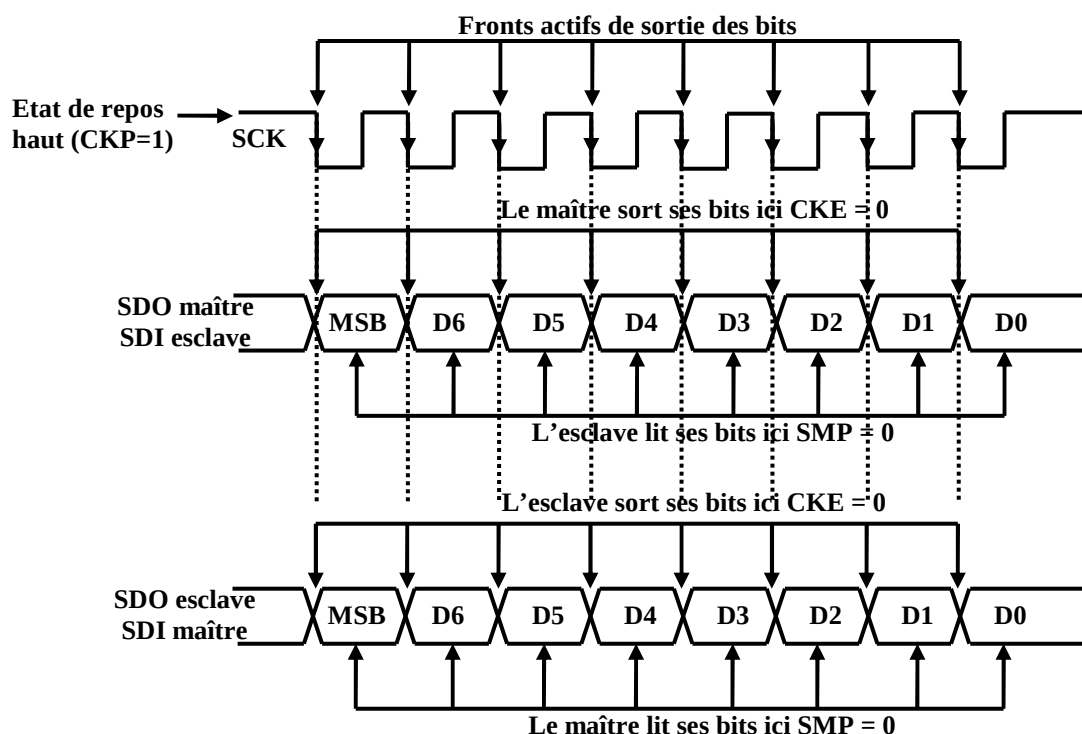


Figure VIII.148.

Voici enfin comment configurer les broches concernées par la liaison SPI :

- **SDI** est **automatiquement contrôlé** par le module **SPI** (rien à faire),
- **SDO** est une **sortie** : **TRISC5 = 0**,
- **SCK du maître** est une **sortie** : **TRISC3 = 0**,
- **SCK de l'esclave** est une **entrée** : **TRISC3 = 1**,
- **SS** est une **entrée** : **TRISA5 = 1**, (penser aussi à initialiser les bits **PCFG3 :PCFG0** du registre **ADCON1** pour déclarer la broche **RA5** comme E/S logique)

L'inconvénient de la liaison **SPI**, c'est qu'il n'y a pas d'acquittement de la part de l'esclave dans le cas d'un échange d'octet : le maître peut parler « dans le vide » sans le savoir. D'où le mode **I²C** qu'on présentera dans le chapitre suivant.

L'avantage de la liaison **SPI** par rapport à l'**U.S.R.T.** est d'avoir des vitesses de liaison plus importantes.

VIII.8.1.3. REGISTRES DE CONTRÔLE ASSOCIES

D'abord bien sûr le registre **SSPBUF** qui contient l'octet à émettre et celui reçu :

- **en mode maître** : une écriture dans **SSPBUF** lance le transfert. Lorsque le transfert est terminé, ce registre contient la donnée renvoyée par l'esclave, que le maître pourra lire.
- **en mode esclave** : la donnée placée dans **SSPBUF** sera renvoyée vers le maître quand ce dernier exécutera un transfert. Lorsque le transfert sera terminé, ce registre contiendra la donnée envoyée par le maître, que l'esclave pourra alors lire.

De plus, la plupart des bits de configuration que l'on vient de présenter se trouvent dans deux registres qui permettent de gérer la liaison **SPI**.

① le registre SFR **SSPSTAT**, dont le format est fourni sur la figure VIII.149.

SSPSTAT (situé à l'adresse 0x094)
(page 72 du datasheet)

	R/W-0	R/W-0	R-0	R-0	R-0	R-0	R-0	R-0
SSPSTAT	SMP	CKE	« I ² C »	« I ² C »	« I ² C »	« I ² C »	« I ² C »	BF
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.149.

Les bits repérés « I²C » ne servent pas dans le mode **SPI** du **MSSP** (mais en mode I²C).

La signification de ces bits est résumée sur la figure VIII.150. Certains de ces bits ont d'ailleurs déjà été vus sur le synoptique de la figure VIII.146.

Bit	Valeur	Signification et commentaires
SMP		SaMPl e Bit : pour échantillonnage bits sur entrée SDI
		Mode maître
	0	Echantillonnage au milieu du bit sorti sur SDO
	1	Echantillonnage à la fin du bit sorti sur SDO
		Mode esclave : toujours 0
CKE		ClocK Edge : front actif de sortie des bits sur SDO
	0	Front repos vers actif de SCK
	1	Front actif vers repos de SCK
Remarque : si on choisit CKE = 1 , il faut obligatoirement choisir le mode synchronisation par SS au niveau de l'esclave pour assurer un premier front permettant de positionner le MSB de l'esclave sur sa sortie SDO .		
BF		Buffer Full
	0	Octet reçu non complètement arrivé (en cours de réception) : SSPBUFF est vide
	1	Octet reçu complètement arrivé : SSPBUFF est plein (remis à 0 par lecture de SSPBUF)

Figure VIII.150.

② le registre SFR **SSPCON**, dont le format est fourni sur la figure VIII.151.

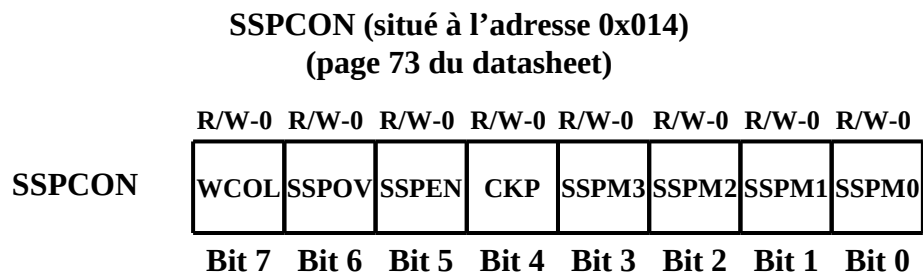


Figure VIII.151.

La signification de ces bits est résumée sur la figure VIII.152. Certains de ces bits ont d'ailleurs déjà été vus sur le synoptique de la figure VIII.146.

Bit	Valeur	Signification et commentaires
WCOL		Write COLL ision detect bit
	0	Pas de collision
	1	Ecriture dans SSPBUF pendant un transfert en cours (remis à 0 par soft par WCOL = 0)
SSPOV		O Verflow detect bit : écrasement d'octet à la réception par l'esclave
	0	Pas d'overflow
	1	Un octet est reçu par l'esclave alors que le précédent n'a pas été lu dans SSPBUF (remis à 0 par soft par SSPOV = 0)
Remarque : en principe SSPOV ne passe jamais à 1 en mode maître car la transmission commence dès qu'on écrit (après l'avoir lu) dans SSPBUF		
SSPEN		Validation de la liaison SPI
	0	SPI non utilisée : les lignes concernées sont des I/O classiques
	1	SPI validée : lignes concernées (SDI , SDO , SCK et SS) affectées à la SPI
Remarque : la direction des lignes (in ou out) concernées doit être faite avant de mettre SSPEN à 1		
CKP		ClocK Polarité : fixe l'état de repos de SCK
	0	Repos à l'état bas
	1	Repos à l'état haut
SSPM3:SSPM0		Différents modes de fonctionnement de la liaison SPI
	0000	Mode maître : SCK générée sur RC3 à la fréquence de F_{osc}/4
	0001	Mode maître : SCK générée sur RC3 à la fréquence de F_{osc}/16
	0010	Mode maître : SCK générée sur RC3 à la fréquence de F_{osc}/64
	0011	Mode maître : SCK générée sur RC3 à la fréquence de TIMER 2 /2
	0100	Mode esclave , SCK extérieure sur RC3 , synchronisation par SS validé
	0101	Mode esclave , SCK extérieure sur RC3 , synchronisation par SS non validé

Figure VIII.152.

Le hardware ayant été présenté, quelle procédure faut-il utiliser pour gérer la liaison **SPI** ?

- **au niveau du maître :**

Voici les initialisations à effectuer :

- gérer la directionnalité des broches concernées par la **SPI** : **RC3/SCK**, **RC4/SDI** et **RC5/SDO**,
- initialiser le registre **SSPSTAT** :
 - instant de lecture des bits entrants sur **SDI** : bit **SMP** (souvent SMP = 0)
 - instant d'écriture des bits sortants sur **SDO** : bit **CKE** (souvent CKE = 0)
- initialiser le registre **SSPCON** :
 - polarité de l'horloge générée sur **SCK** : bit **CKP** (souvent CKP = 1)
 - choix de la source d'horloge et de sa fréquence : bits **SSPM3 :SSPM0**
 - validation de la **SPI** : bits **SSPEN**

On va détailler maintenant les étapes à effectuer en mode pooling.

Pour démarrer un transfert, il faut procéder ainsi :

- si on a plusieurs esclaves, sélectionner l'esclave avec lequel on veut communiquer (ce qui revient à faire passer l'entrée **SS** de cet esclave, et elle seule, à 0).
- écrire un octet à envoyer à cet esclave dans **SSPBUF** : cela démarre une salve de 8 coups d'horloge
- attendre (par pooling) que le bit **BF** passe à **1** : cela indique que l'octet reçu de l'esclave est présent dans **SSPBUF**
- lire cet octet dans **SSPBUF** : cela remet **BF** à **0**

Et recommencer ces étapes autant de fois qu'il y a d'octets à émettre vers cet esclave.

Remarques :

Dans le cas où le maître ne fait que transmettre, il lira un octet « bidon » (il n'en tiendra pas compte) dans **SSPBUF** (pour remettre **BF** à **0**, car sinon il y aura une erreur d'overflow).

Dans le cas où le maître ne veut que recevoir, il devra tout de même écrire un octet « bidon » dans **SSPBUF** pour démarrer les 8 coups d'horloge.

- **au niveau de l'esclave :**

Voici les initialisations à effectuer :

- gérer la directionnalité des broches concernées par la **SPI** :
RC3/SCK, RC4/SDI, RC5/SDO et **RA5/SS** (si utilisée)
- initialiser le registre **SSPSTAT** :
 - instant de lecture des bits entrants sur **SDI** : bit **SMP** = **0** (toujours pour l'esclave)
 - instant d'écriture des bits sortants sur **SDO** : bit **CKE** (souvent CKE = 0)
- initialiser le registre **SSPCON** :
 - polarité de l'horloge générée sur **SCK** : bit **CKP** (la même que pour le maître)
 - choix du mode de fonctionnement esclave : bits **SSPM3 :SSPM0**

Mais, si l'esclave veut émettre un premier octet au premier échange, il faut écrire cet octet dans **SSPBUF** avant de valider la liaison **SPI**.

- validation de la **SPI** : bits **SSPEN**

A partir de là, l'esclave est dans un état d'attente de l'horloge **SCK** du maître. L'idéal est donc de travailler sous interruption pour que l'esclave puisse faire autre chose en parallèle et qu'il ne soit prévenu que quand le premier octet du maître soit entièrement arrivé (signalé par le passage à **1** du bit **SSPIF** avec déclenchement de l'interruption si **SSPIE** est à **1**).

Si l'esclave travaille en pooling :

- attendre (par pooling) que le bit **BF** passe à **1** : cela indique que l'octet reçu du maître est présent dans **SSPBUF**
- lire cet octet dans **SSPBUF** : cela remet **BF** à **0**
- écrire l'octet suivant à émettre dans **SSPBUF**

Et recommencer l'attente par pooling sur **BF**.

Remarques :

Dans le cas où l'esclave ne fait que transmettre, il lira un octet « bidon » (il n'en tiendra pas compte) dans **SSPBUF** (pour remettre **BF** à **0**, car sinon il y aura une erreur d'overflow).

Dans le cas où l'esclave ne veut que recevoir, il n'a pas besoin d'écrire dans **SSPBUF** car ce n'est pas lui qui génère l'horloge.

Voici au final sur la figure VIII.153. un résumé des registres concernés par la liaison **SPI** ainsi que les différents bits concernés.

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR, BOR	Value on all other Resets
INTCON	GIE/ GIEH	PEIE/ GIEL	TMR0IE	INT0IE	RBIE	TMR0IF	INT0IF	RBIF	0000 000x	0000 000u
PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
TRISC	PORTC Data Direction Register								1111 1111	1111 1111
SSPBUF	Synchronous Serial Port Receive Buffer/Transmit Register								xxxx xxxx	uuuu uuuu
SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0	0000 0000	0000 0000
TRISA	—	PORTA Data Direction Register								--11 1111
SSPSTAT	SMP	CKE	D/ $\bar{A}$	P	S	R/ $\bar{W}$	UA	BF	0000 0000	0000 0000

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'.

Shaded cells are not used by the MSSP in SPI mode.

Note 1: The PSPIF, PSPIE and PSPIP bits are reserved on 28-pin devices; always maintain these bits clear.

Figure VIII.153.

VIII.8.1.4. EXEMPLE DE MISE EN ŒUVRE

Le cahier des charges est le suivant :

*L'objectif est de réaliser un port de **sortie parallèle** TTL de taille 8 bits à partir d'une liaison **série SPI**, en utilisant un registre à décalage de type **74HC164** (utilisé en convertisseur série/parallèle) dont la documentation est fournie sur la figure VIII.154. de la page suivante.*

*Cette solution est intéressante car elle permet de rajouter un port de sortie 8 bits supplémentaire sur le **PIC**. On enverra les 8 bits de l'octet à sortir sur ce port par l'intermédiaire de la liaison **SPI**.*

Remarque : on pourrait, de manière équivalente, rajouter un **port parallèle d'entrée** de taille 8 bits sur le **PIC** en utilisant un autre type de registre à décalage permettant de faire cette fois-ci la conversion parallèle/série.

En examinant la documentation du **74HC164**, on voit que :

- l'entrée asynchrone **RESET** (active low) du **74HC164**, doit être reliée au niveau **haut** (5V),

- l'entrée **CLOCK** du **74HC164** (à relier à la ligne **SCK** de la **SPI**) est active sur un **front montant** : ce front décale les bits dans le registre à décalage :

QH perdu, QG → QH, QF → QG, ... , QA → QB, entrée série DATA → QA.

Ainsi, après 8 coups d'horloge, le nouvel octet est écrit dans le **74HC164**.

- la première entrée série **A1** doit être reliée à la ligne **SDO** (broche **RC5**) du **PIC** qui fonctionne en mode maître. Par contre la ligne **SDI** (broche **RC4**) du **PIC** n'est pas utilisée, car aucune information n'est renvoyée de l'esclave (le **74HC164**) vers le maître (le **PIC**).

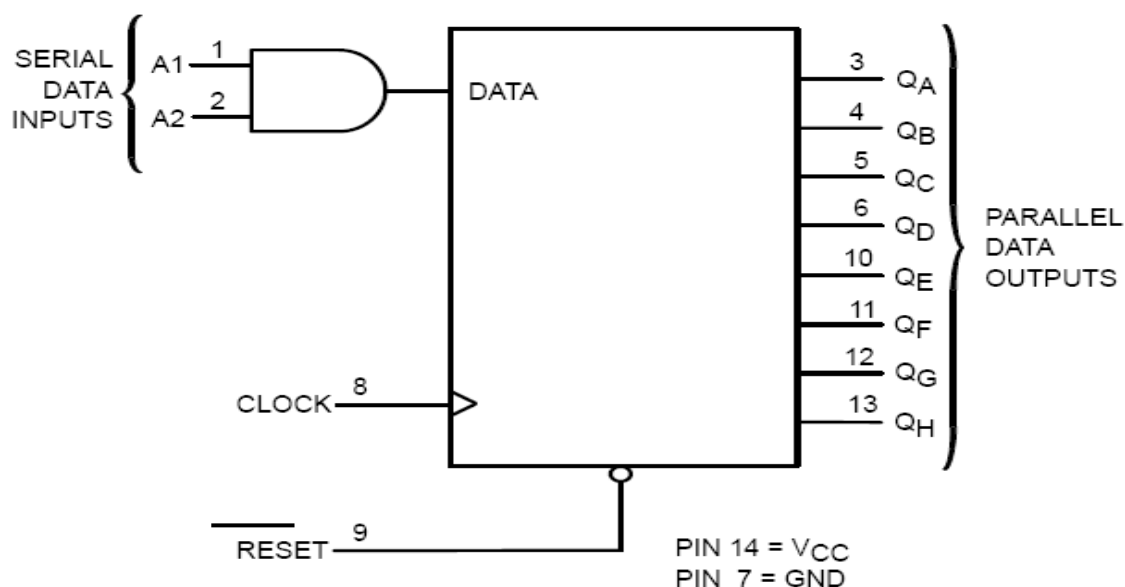
- la seconde entrée série **A2** doit être reliée au niveau haut (5 V) pour laisser passer la valeur de **A1** en sortie de la porte **ET** vers l'entrée série **DATA** du **74HC164**.

8-Bit Serial-Input/ Parallel-Output Shift Register High-Performance Silicon-Gate CMOS

The MC54/74HC164 is identical in pinout to the LS164. The device inputs are compatible with standard CMOS outputs; with pullup resistors, they are compatible with LSTTL outputs.

The MC54/74HC164 is an 8-bit, serial-input to parallel-output shift register. Two serial data inputs, A1 and A2, are provided so that one input may be used as a data enable. Data is entered on each rising edge of the clock. The active-low asynchronous Reset overrides the Clock and Serial Data inputs.

LOGIC DIAGRAM



FUNCTION TABLE

Inputs				Outputs			
Reset	Clock	A1	A2	QA	QB	...	QH
L	X	X	X	L	L	...	L
H		X	X	No Change			
H		H	D	D	QAn	...	QGn
H		D	H	D	QAn	...	QGn

D = data input

QAn – QGn = data shifted from the preceding stage on a rising edge at the clock input.

Figure VII.154.

The diagram illustrates the connection between a PIC microcontroller and a 74HC164 shift register. The PIC microcontroller has pins SDO, SDI, SCK, and SS. The 74HC164 has pins A1, A2, CLOCK, RESET, and eight output pins (QA through QH). The connections are as follows:

- SDO** is connected to **A1**.
- SDI** is connected to **NC** (Not Connected).
- SCK** is connected to **CLOCK**.
- SS** is connected to **NC** through a pull-up resistor to **5V**.
- A2** is connected to **5V**.
- RESET** is connected to **5V**.
- The eight output pins (**QA** through **QH**) are grouped together and labeled as **8 lignes du nouveau PORT de sortie**.

PIC

74HC164

NC = Non Connecté

Configuration de la SPI :

SMP = X : non utilisé ici car pas de réception de la part du maître
CKE = 0 : SCK actif sur front montant (état de repos à 0)

CKP = 0 : état de **repos** de **SCK** au niveau **bas**
SSPM3 :SSPM0 = 0010 : mode **maître** à fréquence **SCK = FOSC/64 = 62,5 kHz** si le quartz du **PIC** est supposé être à **4 MHz**
SSPEN = 1 : **validation** de la **SPI**

TRISC5 = 0 : SDO est une **sortie**
 TRISC3 = 0 : SCK est une **sortie**
 TRISA5 = 1 : SS est une **entrée**

PCFG3 =1
PCFG2 =1
PCFG1 =1
PCFG0 =1

On va écrire deux fonctions :

- une pour l'**initialisation** des broches **SDO**, **SDI**, **SCK** et **SS** et pour l'initialisation de la **SPI**,
- une pour **écrire un octet** sur la liaison **SPI** et donc dans le nouveau **PORT** de sortie.

D'où le programme fourni sur la figure VIII.156.

```
//*****
// initialisation des broches SPI et de la liaison SPI
//*****
void init_spi(void)
{
// voir texte précédent pour les explications de ces différentes valeurs
// directionnalité des broches utilisées par le PIC par la SPI
    TRISC5 = 0 ;
    TRISC3 = 0 ;
    PCFG3 = 1 ;
    PCFG2 = 1 ;
    PCFG1 = 1 ;
    PCFG0 = 1 ;
    TRISA5 = 1 ;
// configuration de la liaison SPI
    CKE = 0 ;
    CKP = 0 ;
    SSPM3 = 0 ;
    SSPM2 = 0 ;
    SSPM1 = 1 ;
    SSPM0 = 0 ;
// validation de la liaison SPI
    SSPEN = 1 ;
}

//*****
// écrit l'octet octet_a_sortir sur la SPI et donc dans le nouveau PORT
//*****
void out_new_port(unsigned char octet_a_sortir)
{
    unsigned char bidon ;           // pour lecture octet reçu mais non géré
    SSPBUF = octet_a_sortir ;       // débute la transmission des bits de octet_a_sortir sur SDO
    while( !BF ) ;                  // attente fin de transmission
    bidon = SSPBUF ;                 // remet BF à 0
}

void main(void)
{
    init_spi();
// test en envoyant 0x55 en sortie sur le nouveau PORT : 0b 0101 0101
    out_new_port(0x55);
}
```

Figure VIII.156.

VIII.8.2. LE MODE I²C

(cf. **pages 80 à 109** du datasheet)

Comme d'habitude, nous donnerons dans un premier temps les principes généraux du mode fonctionnement du bus I²C.

Et si vous n'êtes pas familier avec ces notions, il est fortement conseillé de commencer par là, car cela ne sert à rien de mettre en œuvre un bus I²C si on n'a pas bien compris comment il fonctionne et à quoi il sert. Le temps que vous pensez, à tort, avoir gagné en sautant cette étape, sera très vite perdu par la suite...

VIII.8.2.1. GENERALITES SUR LE BUS I²C



Le bus I²C (Inter Integrated Circuit bus) a été mis au point par la société **Philips** dans les années 1980 pour des applications de domotique et d'électronique grand public. Il permet à un système à base de micro/contrôleur/processeur de se connecter à énormément de composants annexes (très utilisé notamment au début dans les téléviseurs, magnétoscopes ou lecteurs CD). Le but était de diminuer la connectique et le nombre de fils entre les différents circuits numériques d'une application.

Le bus I²C est une interface de type **série synchrone** qui permet la connexion en **Full Duplex à l'alternat** (tantôt dans un sens, tantôt dans l'autre, mais jamais dans les deux sens en même temps) de plusieurs circuits en mode **Maître/Esclave (Master/Slave)**.

Les données et l'horloge sont sur des fils différents, l'horloge n'est pas « incluse » (puis « extraite ») dans le flot des bits série, mais doit bien être véhiculée sur un fil supplémentaire.

Il y aura donc au total **3 fils** sur un bus I²C :

- **1 fil pour les bits échangés** appelé **SDA** (comme **S**ignal **D**Ata) entre les broches **SDA**,
- **1 fil pour l'horloge** noté **SCL** (comme **S**erial **C**Lock) entre les broches **SCL**,
- **1 fil pour la masse GND**.

Ce qui est résumé sur la figure VIII.157. ci-dessous, pour le cas simple d'un maître et d'un seul esclave.

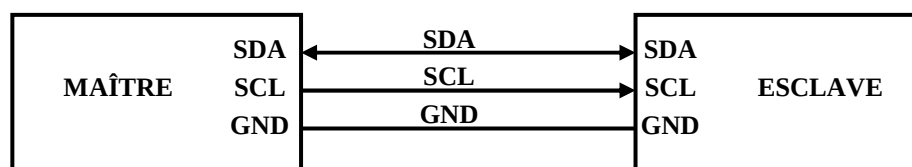


Figure VIII.157.

VIII.8.2.1.1. HARDWARE

Contrairement au bus **SPI** où on reliait une **sortie (SDO)** sur une **entrée (SDI)**, ce qui ne posait absolument aucun problème d'adaptation, ici on relie ensemble toutes les broches **SDA** du maître et du (ou des) esclave(s).

Et comme ces broches peuvent chacune être potentiellement des sorties (quand le circuit émet ses bits), cela pose un problème, car la plupart des circuits numériques ont une sortie de type « totem pôle » et on ne peut pas relier entre elles deux sorties de ce type. Pourquoi ?

Il faut se rappeler le schéma de l'étage de sortie type « totem_pôle » fourni sur la figure VIII.158.

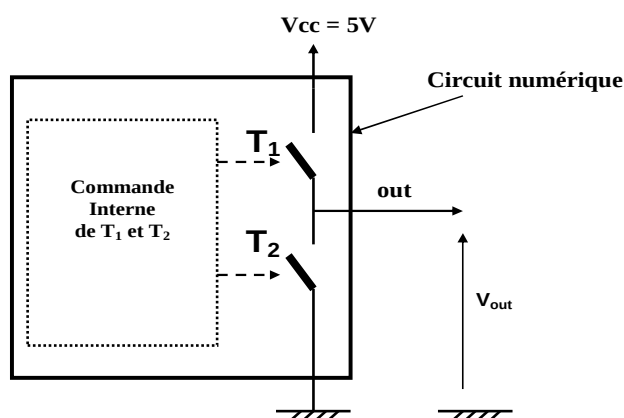


Figure VIII.158.

T_1 et T_2 sont 2 transistors qui fonctionnent comme des **interrupteurs idéaux**. La tension sur la broche de sortie V_{out} dépend de l'état de ces deux transistors, suivant le tableau ci-dessous :

Etat de T_1	Etat de T_2	V_{out}
Passant = fermé = ON	Bloqué = ouvert = OFF	Niveau haut = $V_{cc} = 5\text{ V}$
Bloqué = ouvert = OFF	Passant = fermé = ON	Niveau bas = GND = 0 V
Bloqué = ouvert = OFF	Bloqué = ouvert = OFF	High Z = haute impédance
Passant = fermé = ON	Passant = fermé = ON	Interdit car V_{cc} en court-circuit

Si on relie deux sorties de ce type entre elles, on obtient la configuration représentée sur le schéma de la figure VIII.159.

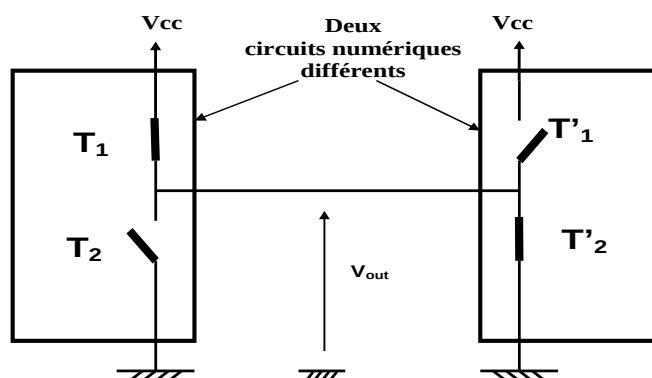


Figure VIII.159.

Un problème se posera par exemple lorsque le circuit de gauche sera en sortie au niveau **haut** (T_1 **ON** et T_2 **OFF**) et celui de droite sera au niveau **bas** (T'_1 **OFF** et T'_2 **ON**) comme sur la figure VIII.159. précédente :

l'alimentation V_{cc} sera en court-circuit via T_1 et T'_2 ce qui risque de faire circuler un courant important dans ces 2 transistors et d'endommager les composants.

Une autre façon de voir les choses mais à un niveau logique :

si une sortie qui vaut 5 V est reliée à une sortie qui vaut 0 V, quelle sera la valeur logique résultante ?

Réponse : impossible à savoir !!! mais à coup sûr un problème de conflit entre ces deux sorties !!!

La solution qui a été utilisée sur le bus I^2C pour relier les broches **SDA** ensemble consiste à utiliser un étage de sortie type « collecteur ouvert », comme indiqué sur la figure VIII.160. qui montre à quoi est reliée cette broche **SDA** à l'intérieur du circuit I^2C .

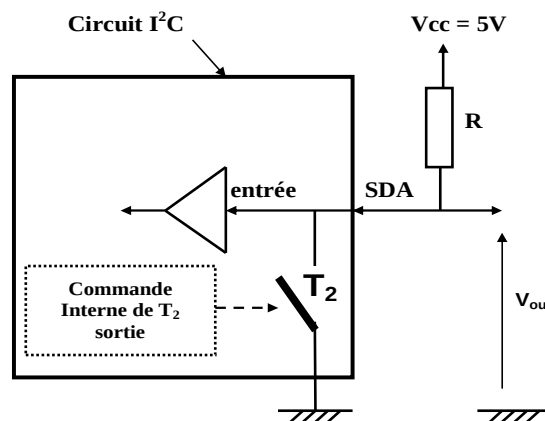


Figure VIII.160.

R est une résistance de « pull up » à rajouter extérieurement pour fixer le potentiel sur la broche **SDA** quand le transistor T_2 (ici, il n'y en a plus qu'un seul) est ouvert.

Fonctionnement en sortie :

- si T_2 est **fermé** on a : $V_{out} = 0$ V et donc **SDA = 0**,
- si T_2 est **ouvert** on a : $V_{out} = 5$ V (grâce à R) et donc **SDA = 1**.

Fonctionnement en entrée : T_2 est alors **ouvert**.

La valeur sur la broche **SDA** est fixée par la tension qu'on lui applique, soit 0 V, soit 5 V.

Le risque de court-circuit de V_{cc} a disparu !!!

Ainsi, en reliant plusieurs circuits, comme indiqué sur la figure VIII.161., il n'y a plus de soucis.

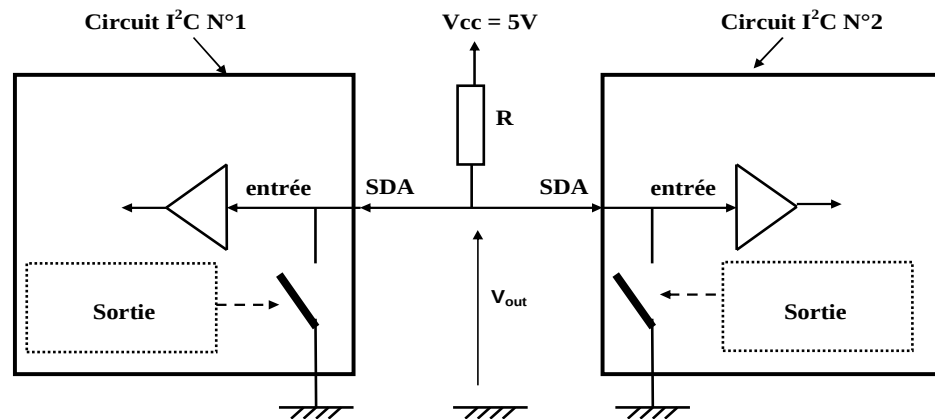


Figure VIII.161.

En pratique, ce problème est le même pour la ligne **SCL** (qui peut être à certains moments une entrée pour un circuit et à d'autres moments une sortie pour ce même circuit).

Au final, comme un bus **I²C** peut comporter un nombre plus important de circuits, on aura donc un hardware qui sera conforme au schéma de la figure VIII.162.

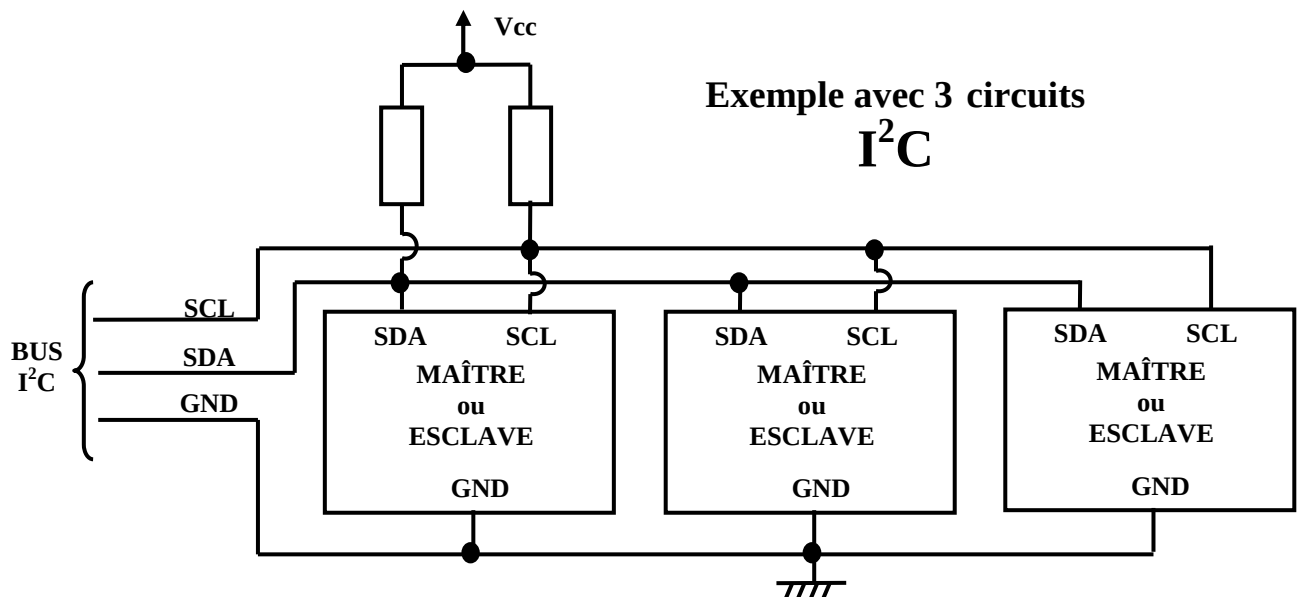


Figure VIII.162.

Remarquons, sur cette figure, que chaque composant peut être soit **maître**, soit **esclave**.

Sachant que dans le cas général, on aura des composants qui ne pourront être qu'esclave, d'autres qui ne seront que maître, et enfin, comme sur la figure, certains qui pourront être tantôt l'un tantôt l'autre.

Mais à chaque instant, il n'y aura qu'un **seul maître** (ce sera lui qui générera alors l'horloge **SCL** et qui initiera les séquences de communication). Les autres seront **esclaves** et recevront l'horloge du maître. De plus, chaque esclave aura sa propre adresse, et seul l'esclave ayant reconnu son adresse dans la requête du maître pourra recevoir des données du maître ou envoyer des données vers le maître, les autres (non adressés) resteront sourds ou muets.

Tous ces éléments seront bien sûr précisés par la suite.

VIII.8.2.1.2. PRISE DE CONTRÔLE DU BUS

Au repos, lorsqu'il n'y a aucun échange de bits sur **SDA**, les lignes **SDA** et **SCL** sont au niveau **haut**.

On dit alors que le bus est **disponible**.

Le circuit qui désire prendre le contrôle du bus I²C, **et qui en deviendra de fait le maître**, fait passer sa sortie **SDA** à **0** (alors que **SCL** est au niveau haut) puis le confirme en faisant passer sa sortie **SCL** aussi à 0.

Cette séquence de prise de contrôle du bus par un maître, résumée sur la figure VIII. 163. est appelée une **START CONDITION**.

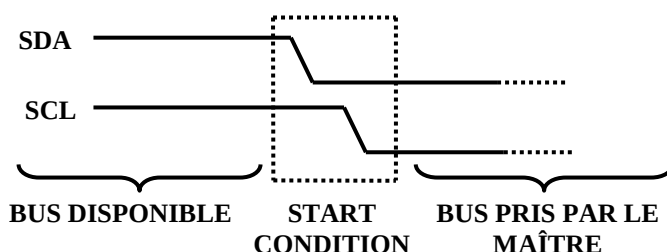


Figure VIII.163.

A partir de cet instant, ce sera le maître qui générera le signal **SCL** : la broche **SCL** est une **sortie pour le maître**, quelque soit le sens de transfert sur la ligne **SDA**.

Les autres circuits, devenus esclaves, la recevront sur leur broches **SCL** qui devient **une entrée pour eux**.

De même, à cet instant, la broche **SDA** est une sortie pour le maître et une entrée pour les esclaves.

Le maître peut commencer alors à initier une émission en envoyant des bits sur la ligne **SDA** au rythme de son horloge **SCL**.

VIII.8.2.1.3. TRANSMISSION DES BITS

C'est assez simple.

Il suffit de placer la valeur de chacun des bits sur la ligne **SDA** (quand **SCL** est au niveau bas) et d'envoyer une impulsion d'horloge de validation sur la ligne **SCL**, comme indiqué sur la figure VIII.164.a. donnant le principe de la transmission d'un bit (la valeur du bit sur **SDA** (0 ou 1) doit être stable avant le front montant de **SCL** et il ne peut pas y avoir un changement d'état sur **SDA** quand **SCL** est au niveau haut comme pour une **START CONDITION**).

Cette impulsion, reçue par le (ou les) esclaves, lui (leur) signalera qu'un bit valide est disponible sur leur entrée **SDA** et qu'il(s) peut(vent) le lire.

Cette séquence sera répétée 8 fois (par 8 impulsions d'horloge) pour transmettre un octet, comme sur la figure VIII.164.b., le **MSB D7** étant le premier bit de l'octet.

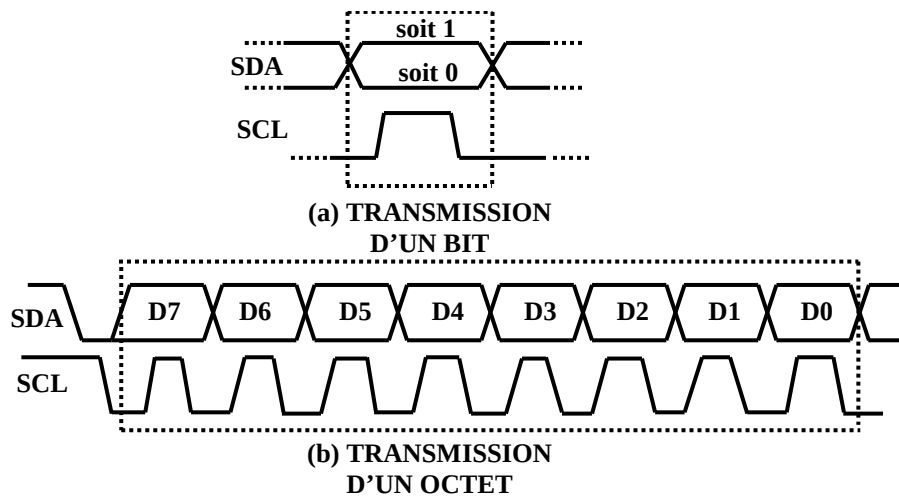


Figure VIII.164.

VIII.8.2.1.4. ACQUITTEMENT

Une fois les 8 bits transmis par le maître, rien n'assure que ces bits ont bien été reçus par l'esclave et que le maître n'a pas émis « dans le vide » !!!

C'est pour cela qu'un **système d'acquittement** ou **accusé de réception** a été rajouté pour le bus I²C (et qui n'existait pas sur le bus SPI) après la transmission du 8^{ème} bit de l'octet émis :

- le maître fait passer sa broche **SDA** à **1** :
cette broche devient alors une **entrée** pour le maître,
- si tout c'est bien passé, l'esclave qui a reçu l'octet fait passer sa broche **SDA** à **0** :
cette broche devient une **sortie** pour l'esclave.
Le bit ainsi renvoyé par l'esclave, au **niveau bas**, s'appelle alors **ACK** comme **ACKnowledge**.
- sinon (par exemple si l'esclave sélectionné a détecté un problème sur l'octet reçu ou si aucun esclave n'a reçu cet octet, bref il y a eu une erreur), la ligne **SDA** restera au **niveau haut**.
Le bit (ainsi renvoyé par l'esclave ou par aucun esclave), au **niveau haut** s'appelle alors **NOACK** comme **NOACKnowledge**.

Ce qui est résumé sur la figure VIII.165.

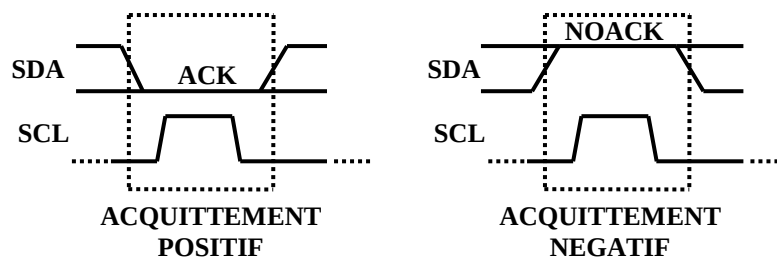


Figure VIII.165.

Grâce à l'impulsion **SCL** suivante (et donc une 9^{ème} générée par le maître), le maître peut lire ce bit et savoir ainsi si l'octet qu'il vient juste d'envoyer avant est acquitté ou pas.

Si le maître reçoit un **NOACK**, et donc s'il y a eu une erreur, il peut générer une condition d'arrêt (voir plus loin **STOP CONDITION**) pour libérer le bus ou renvoyer l'octet une ou plusieurs fois de suite suivant une stratégie qui sera déterminée par le programme pilotant le maître.

Finalement, et dans le cas général, lors de tout transfert d'octet entre l'émetteur (qui n'est pas forcément le maître mais peut être l'esclave) et le récepteur (qui n'est pas forcément l'esclave mais peut être le maître), l'acquiescement n'est rien d'autre qu'un 9^{ème} bit, rajouté à la suite de cet octet, et qui transite dans le sens récepteur de l'octet vers émetteur de cet octet.

Il faudra donc 9 coups d'horloge **SCL** pour transmettre un octet (cf. figure VIII.166.)

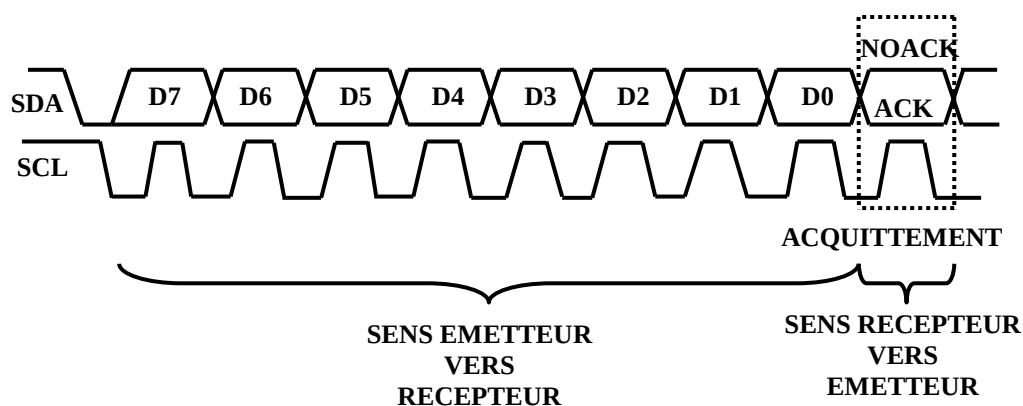


Figure VIII.166.

VIII.8.2.1.5. TRANSMISSION D'UNE ADRESSE

On a vu qu'il pouvait y avoir plusieurs composants sur un bus **I²C**.

Pour pouvoir les différencier, chacun doit avoir une adresse **unique**.

Historiquement, cette adresse a été codée sur **7 bits**, notés **A0** à **A6**, ce qui donne 2⁷ combinaisons différentes.

On verra dans la partie consacrée aux évolutions de la norme **I²C**, qu'on peut maintenant coder cette adresse sur **10 bits**.

Ce sera au concepteur du système à veiller à l'unicité de l'adresse pour chaque composant, sachant aussi que certaines plages d'adresses sont réservées et servent pour des utilisations plus spécifiques.

D'autres plages sont normalisées et correspondent à des composants spécifiques.

Souvent, il existe en plus sur les composants compatibles **I²C** des broches d'entrée d'adresses permettant de choisir par hardware une plage d'adresses pour ce composant.

A titre indicatif, jetez un coup d'œil sur la figure VIII.167. donnant le synoptique de « l'expenseur » de PORT **PCF8574** et notamment sur ses pattes **A0**, **A1** et **A2** (qu'on peut mettre à **0** ou à **1**) permettant de choisir **8 combinaisons d'adresses possibles** dans la plage normalisée pour ce composant à : **[0x20 ; 0x27]**.

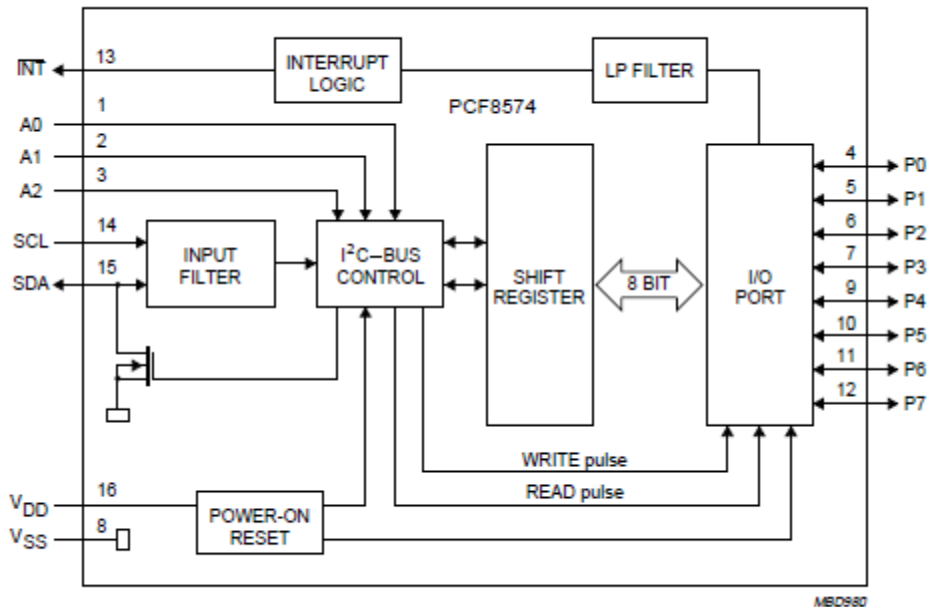


Figure VIII.167.

Question subsidiaire : ce composant peut-il à priori être en position maître sur le bus **I²C** ?

Ainsi, lorsqu'un circuit prend le bus (après émission d'une **START CONDITION**) et en devient donc de fait le maître, le premier octet qui est envoyé par ce maître est un **octet dit d'adresses** : le maître envoie sur sa sortie **SDA** l'adresse de l'esclave avec lequel il veut initier une séquence d'échange.

L'adresse étant codée sur 7 bits, le maître utilise le 8^{ème} bit comme bit particulier, appelé **R/W**.

- si ce bit **R/W** = **0** (**Write**) cela signifie que l'esclave sélectionné est en **mode écriture** et donc qu'il doit s'attendre à **recevoir** du maître, juste après, un ou plusieurs octets,
- si ce bit **R/W** = **1** (**Read**) cela signifie que l'esclave sélectionné est en **mode lecture** et donc qu'il doit **renvoyer** vers le maître, juste après, un ou plusieurs octets.

Bien sûr, comme pour toute transmission d'octet, l'octet d'adresse se termine par un acquittement positif (**ACK**) ou négatif (**NOACK**) émanant de l'esclave sélectionné.

Si le maître reçoit **ACK**, l'échange continue soit dans un sens, soit dans l'autre en fonction du bit **R/W**, comme on l'a vu précédemment.

Si le maître reçoit **NOACK**, et donc s'il y a eu une erreur, le maître peut générer une condition d'arrêt (voir plus loin **STOP CONDITION**) pour libérer le bus ou renvoyer l'adresse de l'esclave une ou plusieurs fois de suite suivant une stratégie qui sera déterminée par le programme pilotant le maître.

Tous ces éléments sur le format de l'octet d'adresse sont résumés sur la figure VIII. 168.

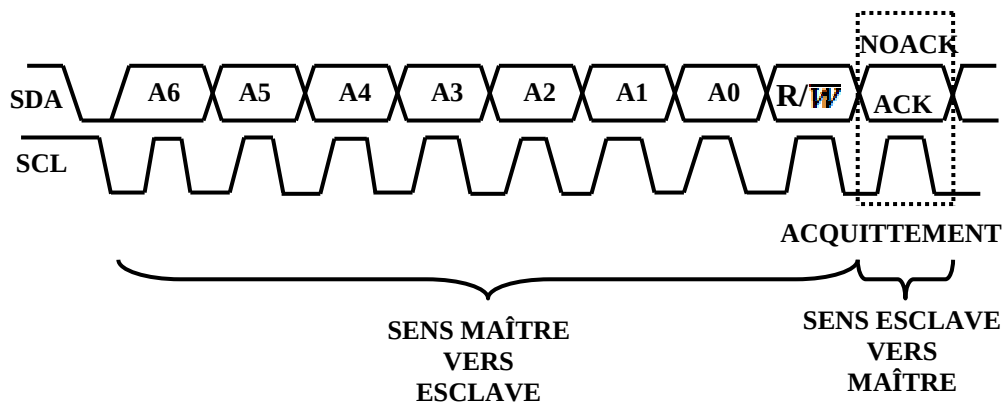


Figure VIII.168.

VIII.8.2.1.6. ECRITURE D'UNE DONNEE

Dans le cas où le bit **R/W** de l'octet d'adresse vaut **0** et si l'acquittement renvoyé par l'esclave est **ACK**, le maître fait suivre, juste après, le premier octet de donnée à transmettre à l'esclave sélectionné.

S'il y a plus qu'un octet de données à transmettre, ils le seront les uns à la suite des autres.

Chaque octet reçu par l'esclave doit être validé par l'esclave par un **ACK**.

Par contre, le maître peut mettre, à tout moment, fin à l'émission :

- soit à la réception d'un **NOACK (erreur)**,
- soit à la fin de l'émission de tous les octets à envoyer à l'esclave (**fin normale**),
- ou même après la réception d'un **ACK** quelconque (**émission interrompue par décision unilatérale du maître**),

en générant une condition d'arrêt (voir plus loin **STOP CONDITION**).

Ce procédé d'écriture est résumé sur la figure VIII.169. utilisant un autre type de représentation des trames **I²C** sur la ligne **SDA** rencontré assez souvent dans la littérature technique et ne montrant plus les impulsions d'horloge sur la ligne **SCK**.

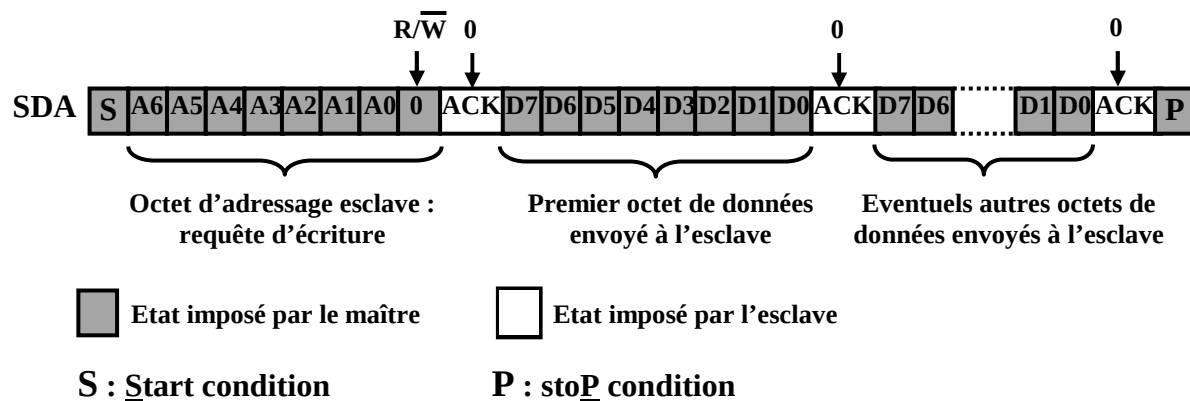


Figure VIII.169.

VIII.8.2.1.7. LECTURE D'UNE DONNEE

Dans le cas où le bit **R/W** de l'octet d'adresse vaut **1** et si l'acquittement renvoyé par l'esclave est **ACK**, il y a **inversion du sens de l'échange** puisque c'est l'esclave qui va renvoyer un (ou plusieurs octets) en réponse à la demande de lecture du maître.

Pour cela, le maître lâche le contrôle de la ligne **SDA** en faisant passer sa broche **SDA** au niveau haut (la broche **SDA** devient alors une entrée pour le maître), et c'est l'esclave qui va piloter la ligne **SDA** (la broche **SDA** devient alors une sortie pour l'esclave).

Mais l'horloge SCL est toujours générée par le maître !!!

S'il y a plus qu'un octet de données à renvoyer vers le maître, ils le seront les uns à la suite des autres, car si le premier octet reçu par le maître est validé par le maître par un **ACK**, cela signifie que le maître attend un autre octet (l'esclave sait interpréter ce **ACK** et donc qu'il doit émettre un octet supplémentaire) et ainsi de suite....

Par contre, si à la réception d'un octet par le maître (y compris pour le premier octet) le maître renvoie un **NOACK**, cela signifie que la lecture est terminée. L'esclave saura interpréter ce **NOACK** comme une fin d'émission de données de sa part.

Notons ici cependant que ce **NOACK** ne signifie pas une condition d'erreur, mais tout simplement une fin de lecture normale. On voit encore que tout cet échange repose sur le contrôle unilatéral du maître.

Ce **NOACK** sera ensuite suivi d'une condition d'arrêt (voir plus loin **STOP CONDITION**) générée par le maître.

Tout ce principe de lecture est résumé sur la figure VIII.170. suivante.

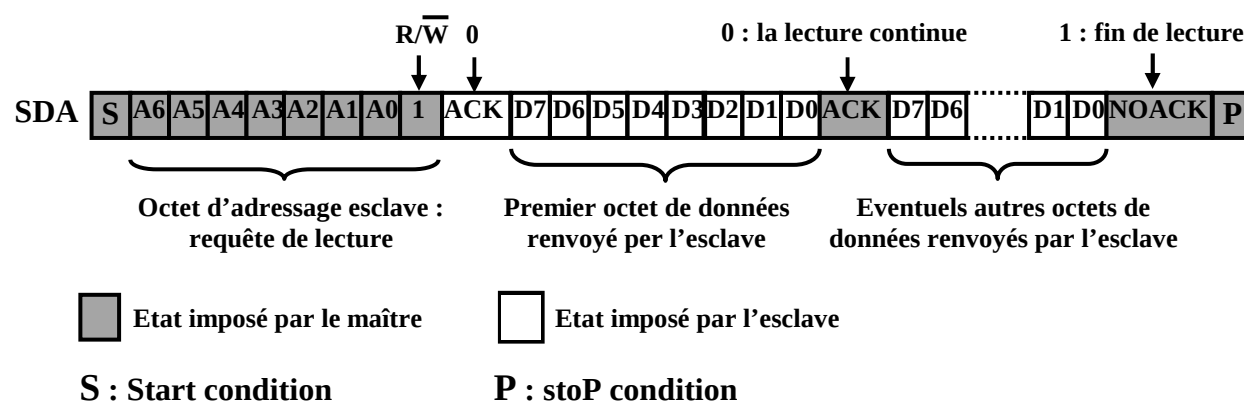


Figure VIII.170.

VIII.8.2.1.8. LIBERATION DU BUS

On a parlé plusieurs fois dans le texte précédent de la condition d'arrêt (**STOP CONDITION**) permettant de libérer le bus.

De façon symétrique à la **START CONDITION**, le maître, à la fin d'une séquence d'échange, génère une condition d'arrêt permettant **de libérer** le bus.

Pour cela, à partir de l'état où les lignes **SDA** et **SCL** sont au niveau bas, le maître commence par faire passer **SCL** au niveau haut puis place ensuite **SDA** au niveau haut.

Cette séquence de libération du bus par un maître, résumée sur la figure VIII. 171. est appelée **STOP CONDITION**.

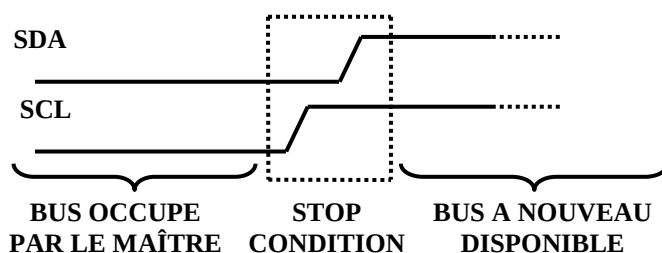


Figure VIII.171.

A partir de cet instant, le bus **I²C** est à nouveau disponible pour une nouvelle prise de contrôle par un autre maître.

Deux remarques avant d'aller plus loin :

❶ En pratique, et pour certains composants comme les **RAM** compatibles **I²C** par exemple, il est possible d'enchaîner écriture et lecture de l'esclave sans passer par une condition d'arrêt.

Le maître envoie, avec le bit **R/W** = 0 (ce qui annonce un cycle d'écriture), l'adresse de base de la **RAM** esclave concernée qui lui renvoie **ACK**. Puis le maître transmet l'adresse de la case mémoire visée dans cette **RAM**, qui lui renvoie **ACK**. Et enfin, il devrait envoyer la valeur de l'octet à écrire puis terminer cet échange d'écriture par une **STOP CONDITION**.

Mais le maître peut enchaîner une nouvelle **START CONDITION**, sans passer par l'envoi de cet octet et de cette **STOP CONDITION** (on parle de **RESTART** ou de **REPEATED START CONDITION**) et envoyer à nouveau, mais cette fois-ci avec le bit **R/W** = 1 (ce qui annonce un cycle de lecture) l'adresse de base de cette **RAM**, qui lui retourne **ACK**.

Et là, le sens de l'échange s'inverse, et la **RAM** renvoie le contenu de la case mémoire demandée.

Si le maître renvoie **ACK**, la **RAM** est prévue pour envoyer le contenu de la case suivante et ainsi de suite.

Mais quand le maître décide que cet échange est terminé, il renvoie **NOACK** puis génère une **STOP CONDITION**.

❷ Finalement, les **START**, **REPEATED START** et **STOP** sont les seuls cas où la ligne **SDA** peut changer de niveau alors que la ligne **SCL** est au niveau haut.

Dans tous les autres cas, **SDA** doit changer de niveau alors que lorsque **SCL** est au niveau bas !!!

VIII.8.2.1.9. CONFLITS SUR LE BUS I²C

Sur un bus I²C, chaque composant peut potentiellement en devenir le maître (et si bien sûr son hardware propre est prévu pour qu'il puisse fonctionner en mode maître), sachant qu'à un instant donné, il ne peut y avoir **qu'un seul maître**.

Se pose alors le problème, inhérent à tout type de réseaux ne possédant qu'une seule ligne de communication, d'assurer **qu'un et qu'un seul** composant en soit le maître, pour éviter que les données présentes sur le bus ne soient corrompues par la présence de plusieurs maîtres qui émettraient en même temps des données contradictoires sur la même ligne.

A cause de la structure « collecteur ouvert » qui est utilisée pour les broches **SDA** et **SCL** de tous les composants reliés sur le bus I²C, il n'y aura pas de problèmes purement électriques (pas de court-circuits potentiels à craindre), mais on va voir qu'en cas de conflits sur le bus, il se produit quelque chose d'un peu étonnant.

Mais tout d'abord qu'entend-on par conflit ?

C'est une situation qui se produit à la prise de bus.

En effet, lorsqu'un circuit prend le bus à partir de l'état « bus disponible » (**SDA** et **SCL** au niveau haut pendant plus de 5 µs), il peut très bien arriver qu'un autre (ou plusieurs autres) composant(s) prenne(nt) également le bus en même temps (ou dans un délai très faible après le premier).

On aura donc plusieurs maîtres en même temps sur le bus : c'est ce qu'on appelle un **conflit**.

Comment s'en apercevoir ?

Chaque maître, ayant imposé **SDA** à l'état bas, peut relire l'état de la ligne **SDA**. Mais il ne pourra pas détecter de conflit, car dès qu'un composant impose **SDA** à 0, la ligne **SDA** passe à 0. (**ET logique** entre toutes les sorties **SDA**). Donc rien d'anormal.

Et chaque maître va alors générer le signal **SCL** comme s'il était seul.

Supposons, pour simplifier, qu'on ait deux maîtres à cet instant et appelons **SDA1** et **SDA2** leurs sorties respectives de données, toutes les deux reliées à la ligne **SDA** commune du bus I²C.

On a, à cause de la structure « collecteur ouvert » de ces sorties :

$$\text{SDA} = \text{SDA1} \bullet \text{SDA2} \text{ (ET logique entre SDA1 et SDA2)}$$

De même, appelons **SCL1** et **SCL2** leurs sorties d'horloges respectives, reliées elles aussi toutes les deux à la ligne **SCL** commune du bus I²C.

Pour les mêmes raisons, on a :

$$\text{SCL} = \text{SCL1} \bullet \text{SCL2} \text{ (ET logique entre SCL1 et SCL2)}$$

Dessignons, sur la figure VIII.172. les chronogrammes possibles pour ces signaux :

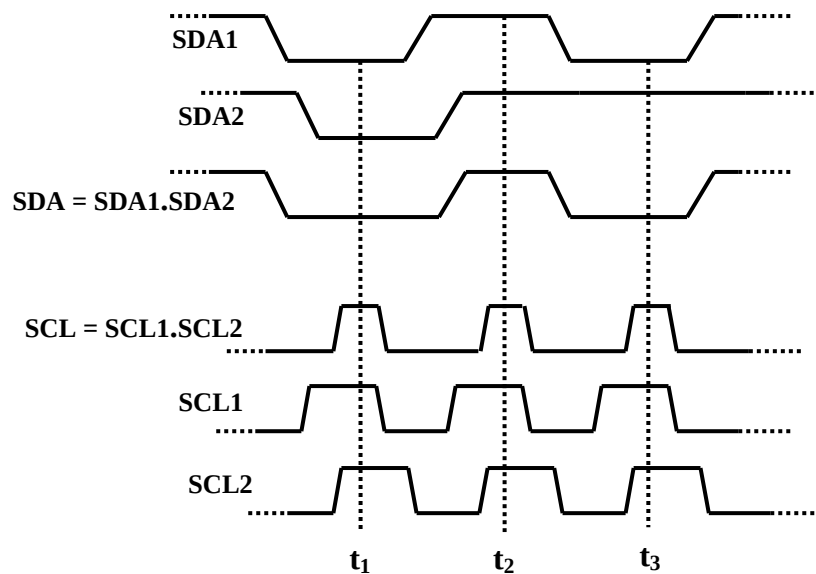


Figure VIII.172.

A l'instant t_1 (milieu du signal haut sur la ligne **SCL**) :

- le **maître 1** impose **SDA1 = 0**,
- le **maître 2** impose **SDA2 = 0 ...**
- et il y a bien **0** sur la ligne **SDA**.

Donc impossible pour les deux maîtres de détecter le conflit (en fait les 2 maîtres sortant le même bit, il n'y a pas d'erreur malgré le conflit).

A l'instant t_2 (milieu suivant du signal haut sur la ligne **SCL**) :

- le **maître 1** impose **SDA1 = 1**,
- le **maître 2** impose **SDA2 = 1 ...**
- et il y a bien **1** sur la ligne **SDA**.

Donc, là aussi, impossible pour les deux maîtres de détecter le conflit (en fait les 2 maîtres sortant le même bit, il n'y a pas d'erreur malgré le conflit).

A l'instant **t₃** (milieu suivant du signal haut sur la ligne **SCL**) :

- le **maître 1** impose **SDA1 = 0**,
- le **maître 2** impose **SDA2 = 1 ...**
- et il y a **0** sur la ligne **SDA**.

Là, par contre, le **maître 2** peut détecter le conflit (il met **1** sur sa ligne **SDA2** et il lit **0** sur la ligne **SDA**, donc conflit, donc problème).

On voit finalement que le conflit est détectable dès qu'au moins un des maîtres impose un niveau bas sur sa broche **SDAi** et qu'au moins un des autres maîtres impose un niveau haut sur sa broche **SDAj**.

En effet, celui (ou ceux) qui impose(nt), au même moment, un niveau haut sur sa (leurs) broche(s) **SDAj** détectera(ront) ce conflit et instantanément se mettra(ont) en position esclave(s) (comme il(s) l'aurai(en)t fait s'il(s) étai(en)t resté(s) esclave(s), au cas où le message courant leur(s) soit destiné, en tant qu'esclave(s) !!!) pour continuer à lire les bits imposés par celui (ou ceux) qui reste(nt) le(s) maître(s).

On pourrait envisager, en poussant à l'extrême ce raisonnement, le cas (très peu probable) où le conflit ne serait pas détectable par aucun des maîtres ayant pris le contrôle du bus (ni sur l'octet d'adresse, ni sur le (ou les) octet(s) de données).

Ce serait le cas si tous les bits étaient identiques mais dans ce cas, malgré le fait qu'il y ait plusieurs maîtres, le message est bon (non corrompu) et donc il n'y a pas de conflit car chaque maître a pu réaliser ce qu'il voulait, exactement en même temps !!!

VIII.8.2.1.10. QUELQUES PARTICULARITES

Les éléments fournis dans cette partie ne sont pas exhaustifs. Il est conseillé, pour les compléter en cas de besoin, de se renseigner sur le site de Philips ou tout autre site donnant des informations plus précises sur la normalisation du bus **I²C**.

Les adresses comprises entre **0x00** et **0x07** incluses ne sont pas utilisées par les composants reliés au bus **I²C**.

Par exemple l'adresse **0x00** correspond à **un appel général** sur le bus : les circuits qui ont la capacité de le traiter émettent en retour un **ACK**.

Après cet appel général, l'octet suivant permet de préciser la raison de cet appel :

- si c'est **0x06** : **reset** des esclaves,
- si c'est **0x04** : relecture par les esclaves de leurs adresses respectives ...
- ...

Un autre point intéressant à noter est lié au problème de la lenteur de certains esclaves qui ne peuvent pas répondre au rythme imposé par une horloge **SCL** du maître trop rapide.

Dans ce cas, l'esclave « lent » peut maintenir la ligne **SCL** au niveau bas : on appelle cela dans la documentation faire du « **clock stretching** ».

Le maître, en scrutant en permanence le niveau présent sur la ligne **SCL** pourra détecter qu'elle se trouve au niveau bas alors qu'il lui a imposé un niveau haut.

Et il rajoutera ce qu'on appelle des « **WAIT STATE** » ou état d'attente, jusqu'à ce que la ligne **SCL** soit libérée par l'esclave et passe au niveau haut.

Ce ralentissement de la vitesse du maître est résumé sur les chronogrammes de la figure VIII. 173.

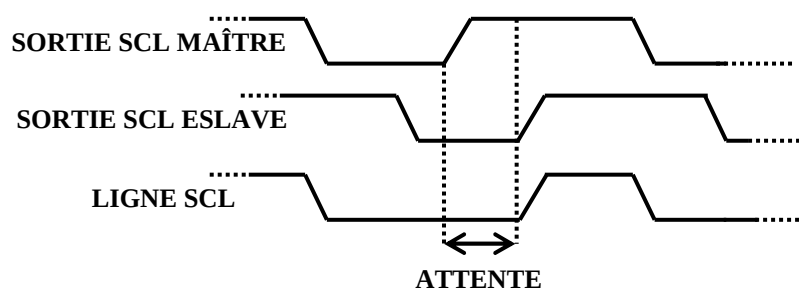


Figure VIII.173.

VIII.8.2.1.11. QUELQUES EVOLUTIONS DU BUS I²C

Dans les dernières spécifications du bus **I²C**, l'adresse des esclaves est codée sur **10 bits**, ce qui multiplie potentiellement **par 8** le nombre de circuits implantables sur le bus **I²C**.

Il faudra donc deux octets pour transmettre l'adresse complètement de l'esclave concerné par l'échange, comme indiqué sur la figure VIII.174.

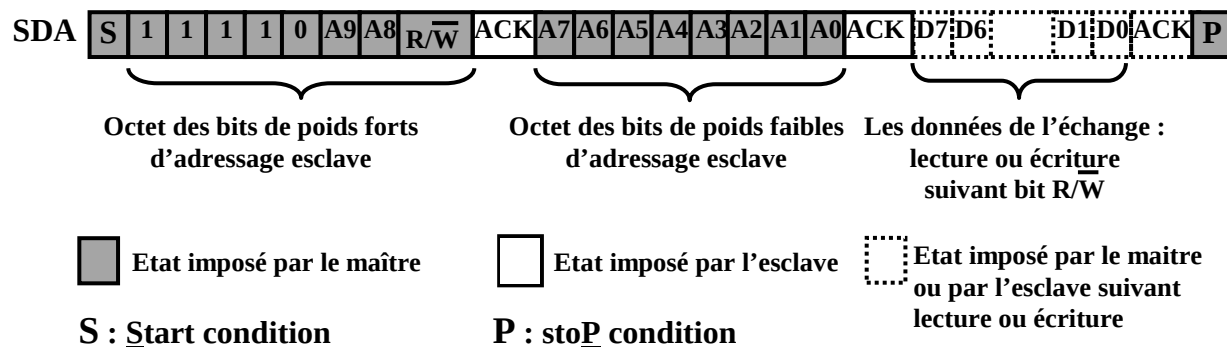


Figure VIII.174.

La vitesse (fréquence de l'horloge **SCL**) a été augmentée et peut maintenant atteindre des débits pouvant aller jusqu'à **4 Mégabits/s**.

La sensibilité aux parasites et autres bruits divers sur les signaux a été diminuée grâce à l'utilisation de **trigger de Schmitt** sur les broches **SDA** et **SCL** quand elles fonctionnent en entrée.

Les circuits présents sur le bus mais **non alimentés** sont mis systématiquement en haute impédance afin d'éviter qu'ils n'interfèrent sur le bus, ou même qu'ils le bloquent tout simplement.

...

Et la liaison **I²C** dans le **PIC** ?

Souvent, lorsqu'on utilise un **PIC** sur un bus **I²C**, ce sera lui qui en sera le maître « principal » (car il possède, grâce à sa **CPU** et au programme qui le fait fonctionner, une capacité de traitement de l'information) les autres circuits étant alors des esclaves (capteurs et actionneurs divers, circuits périphériques ou mémoires auxiliaires...).

L'interface **I²C** du module **MSSP** fonctionnera alors en mode **maître**.

Mais dans le cas de systèmes **multiprocesseurs**, on peut très bien envisager de relier plusieurs **PIC** (ou même des micro/processeurs/contrôleurs différents) entre eux par un bus **I²C**.

Dans ce cas, à certains moments le **PIC** peut être **maître** et à d'autres, il peut être **esclave**.

L'interface **I²C** du module **MSSP** devra donc **aussi** pouvoir fonctionner en mode **esclave**.

Le hardware pilotant le module **MSSP** dans le mode **I²C** présente :

- des parties commune avec celui, vu dans la partie précédente et pilotant le module **MSSP** en mode **SPI**, notamment au niveau des registres de fonctionnement et de contrôle,
- mais aussi beaucoup de parties **spécifiques** et **propres** à l'interface **I²C**.

Ainsi, dans un premier temps, on va présenter ici quelques éléments valables dans les deux modes de fonctionnement (mode **maître** et mode **esclave**) et qui permettent de faire le lien avec ce qui a été dit dans les généralités introductives au bus **I²C**.

Puis, nous nous occuperons dans un deuxième temps du fonctionnement en mode maître, puis ensuite du fonctionnement en mode esclave.

VIII.8.2.2. SYNOPTIQUE SIMPLIFIÉ, BROCHES ET REGISTRES

Un synoptique général du module **MSSP**, mais très **simplifié** car très **incomplet**, est fourni sur la figure VIII.175.

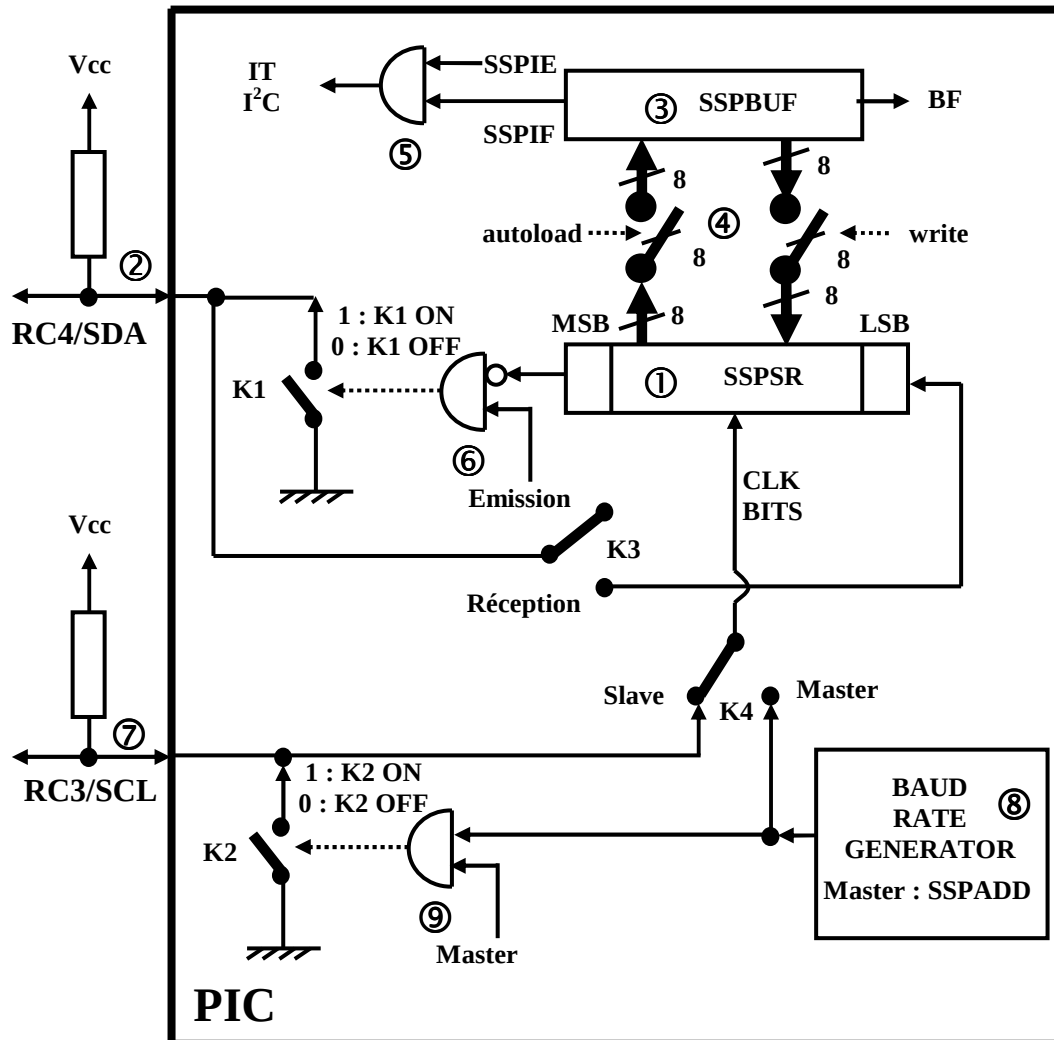


Figure VIII.175.

Le cœur du module **I²C** est le registre à décalage **SSPSR** (cf ①) qui contient l'octet à transmettre ou celui qui a été reçu sur la liaison.

A l'émission (interrupteurs **K1** piloté par la sortie **MSB** de **SSPSR** (cf ⑥) et **K3** en position **Emission**) les bits sortent un à un (avec le **MSB** en premier) par la broche **RC4/SDA** (cf ②) et à la réception (interrupteurs **K1** ouvert (cf ⑥) et **K3** en position **Réception**) ils arrivent aussi par cette même broche **RC4/SDA** et rentrent par l'entrée **LSB** de **SSPSR**.

Noter également la présence de la résistance (à rajouter extérieurement) de « pull-up » entre cette broche **RC4/RDA** et l'alimentation **Vcc**.

Le registre **SSPSR** n'étant pas mappé, il n'est pas directement accessible au programmeur.

Il faut utiliser le registre tampon **SSPBUF** ③ pour échanger les données avec la liaison **I²C**.

① procédure de transmission d'un octet :

Quand l'émetteur (maître ou esclave) écrit un octet dans **SSPBUF** (commande **write** du bloc ④) :

- cet octet est automatiquement écrit dans **SSPSR**,
- et le flag **BF** passe automatiquement à **1** : indiquant que le registre **SSPBUF** est plein et qu'une phase d'émission est en cours.

La transmission peut alors commencer...

A la fin de la transmission de l'octet :

- le flag **BF** repasse automatiquement à **0**,
- et le flag **SSPIF** passe à **1**, ce qui peut déclencher une interruption **I²C** si elle a été localement démasquée par **SSPIE = 1** (cf. ⑤).

Le flag **BF** servira donc, par pooling, à savoir si le registre **SSPBUF** est libre ou pas.

Et si on écrit dans **SSPBUF** alors que **BF** vaut **1**, un autre flag **WCOL** passera à **1**, indiquant une erreur de **COL**lision et l'écriture dans **SSPBUF** ne se fera pas !!!

② procédure de réception d'un octet :

De même, à la réception d'un octet complet dans **SSPSR** (par le maître ou par l'esclave) :

- cet octet est automatiquement transféré dans le registre **SSPBUF** (commande **autoload** du bloc ④)
- le flag **BF** passe automatiquement à **1** : indiquant qu'un octet est arrivé et qu'on peut le lire dans **SSPBUF**, ce qui remettra automatiquement **BF** à **0**,
- et le flag **SSPIF** passe aussi à **1** ce qui peut également déclencher une interruption **I²C** si elle a été localement démasquée par **SSPIE = 1** (cf. ⑤).

Là aussi, le flag **BF** servira, par pooling, à savoir si un octet est complètement arrivé pour pouvoir le lire dans **SSPBUF**.

Et si un nouvel octet arrive alors que **BF** est à **1** (ce qui signifie que le précédent n'a pas été lu) le flag **SSPOV** passera à **1**, indiquant une erreur d'**é**crasement (**O**Verflow) et cet octet sera donc perdu.

Il ne reste plus qu'à parler de l'horloge **SCL**, qui est, rappelons le :

- **générée** par le maître,
- et **reçue** par l'esclave.

sur la broche **RC3/SCL** (cf. ⑦).

Notons également la présence d'une deuxième résistance (à rajouter extérieurement) de « pull-up » entre cette broche **RC3/SCL** et l'alimentation **Vcc**.

① en mode maître :

L'interrupteur **K4** est en position **master** et fournit l'horloge de décalage des bits (signal **CLK BITS**) au registre **SSPSR**.

Ce signal d'horloge provient du bloc **BAUD RATE GENERATEUR** référencé ⑧ et utilisé uniquement en mode maître.

La fréquence et donc la vitesse de transfert sur la liaison **I²C** sera obtenue, en mode **maître** grâce à un registre **SSPADD** (en mode esclave, ce registre contient l'adresse de l'esclave et donc sert à tout autre chose...).

Notons que ce signal d'horloge, en mode **maître**, se retrouve aussi, grâce à l'interrupteur **K2** piloté par la logique ⑨ sur la broche de sortie **RC3/SCL** pour tous les esclaves reliés au bus.

② en mode esclave :

L'interrupteur **K4** est en position **slave** et le bloc **BAUD RATE GENERATEUR** n'est plus utilisé. L'interrupteur **K2** est ouvert.

L'horloge de décalage des bits **CLK BITS** provient alors de la broche d'entrée **RC3/SCL** et donc elle est bien fournie par le maître du bus.

Les cinq principaux registres de configuration et de contrôle intervenant dans le mode **I²C** sont :

- ① **SSPBUF** (déjà vu pour le mode **SPI**),
- ② **SSPCON** (déjà vu pour le mode **SPI**),
- ③ **SSPSTAT** (déjà vu pour le mode **SPI**),
- ④ **SSPCON2** (spécifique au mode **I²C**),
- ⑤ **SSPADD** (spécifique au mode **I²C**).

REMARQUE PRELIMINAIRE IMPORTANTE :

Pour utiliser le module **MSSP** en mode **I²C**, il faut programmer au départ les lignes **RC3/SCL** et **RC4/SDA en entrée**.
(via bien sûr le registre **TRISC**).

Le module **MSSP** les mettra automatiquement en sortie quand ce sera nécessaire (en fonction du mode maître ou esclave programmé, suivant le sens de l'échange...).

Une des principales difficultés quand on travaille en mode **I²C** est de bien savoir et de bien comprendre ce qui est généré automatiquement par le module **MSSP** du PIC dans ce mode de fonctionnement et ce qui relève du programmeur via le programme pilotant le bus **I²C**.

VIII.8.2.3. LE MODE MAÎTRE

VIII.8.2.3.1. REGISTRES DE CONTRÔLE ASSOCIES

D'abord bien sûr le registre :

① **SSPBUF** qui contient comme on l'a dit précédemment :

- **l'octet à émettre** : si le maître est en **transmission**,

ou

- **l'octet reçu** : si le maître est en **réception**,

② le registre SFR **SSPCON** (déjà vu dans le mode **SPI**) dont le format est rappelé sur la figure VIII.176. suivante :

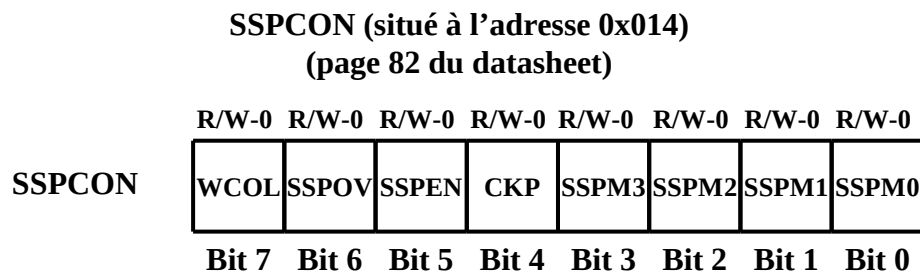


Figure VIII.176.

La signification de ces bits est donnée pour le mode **I²C maître** sur la figure VIII.177.

Bit	Valeur	Signification et commentaires
WCOL		Write COLLision detect bit
	0	Pas de collision
	1	Ecriture dans SSPBUF pendant un transfert en cours (remis à 0 par soft par WCOL = 0)
SSPOV		OVerflow detect bit : écrasement d'octet à la réception
	0	Pas d'overflow
	1	Un octet est reçu alors que le précédent n'a pas été lu dans SSPBUF (remis à 0 par soft par SSPOV = 0)
Remarque : SSPOV n'est pas utilisé en transmission		
SSPEN		Validation de la liaison I²C
	0	I ² C non utilisée : les lignes concernées sont des I/O classiques
	1	I ² C validée : lignes concernées (SDA et SCL) affectées à l'I ² C
Remarque : la direction des lignes (input) concernées doit être faite avant de mettre SSPEN à 1		
CKP		Non utilisé en mode maître
SSPM3:SSPM0		Différents modes de fonctionnement de la liaison I ² C
	1000	Mode maître : $F = F_{OSC}/(4*(SSPAD+1))$ Pour fixer la fréquence (la vitesse) de la liaison I ² C

Figure VIII.177.

③ le registre SFR **SSPSTAT** (déjà vu dans le mode **SPI**) dont le format est rappelé sur la figure VIII.178.

**SSPSTAT (situé à l'adresse 0x094)
(page 81 du datasheet)**

	R/W-0	R/W-0	R-0	R-0	R-0	R-0	R-0	R-0
SSPSTAT	SMP	CKE	D/ $\overline{A}$	P	S	R/ $\overline{W}$	UA	BF
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.178.

La signification de ces bits est donnée pour le mode **I²C maître** sur la figure VIII.179.

Bit	Valeur	Signification et commentaires
SMP		Contrôle du slew rate de l'horloge
	0	pour F = 400 kHz
	1	pour F = 100 kHz et 1 MHz
CKE		Contrôle des niveaux de tension en entrée
	0	compatibilité I²C
	1	compatibilité SMBUS
D/$\overline{A}$		Non utilisé en mode maître
P		Indique la détection d'une STOP CONDITION (libération bus)
	0	pas de détection de STOP CONDITION
	1	détection d'une STOP CONDITION
Remis automatiquement à 0 quand une autre condition est détectée		
S		Indique la détection d'une START CONDITION (prise de contrôle bus)
	0	pas de détection de START CONDITION
	1	détection d'une START CONDITION
Remis automatiquement à 0 quand une autre condition est détectée		
R/$\overline{W}$		Indique qu'une transmission du maître est en cours
	0	pas de transmission en cours
	1	transmission en cours
UA		Non utilisé en mode maître
BF		Buffer Full
En transmission :		
	0	pas de transmission en cours ou terminée : SSPBUF est vide
	1	transmission en cours : SSPBUF est plein
En réception :		
	0	réception en cours ou pas de réception : SSPBUF est vide
	1	réception terminée : SSPBUF est plein

Figure VIII.179.

- ④ le registre SFR **SSPCON2** (spécifique au mode **I²C** et cf. figure VIII.180. suivante)
SSPCON2 (situé à l'adresse 0x014)
(page 83 du datasheet)

	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SSPCON2	GCEN	ACKSTAT	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.180.

La signification de ces bits est donnée pour le mode **I²C maître** sur la figure VIII.181.

Bit	Valeur	Signification et commentaires
GCEN		Non utilisé en mode maître
ACKSTAT		Maître en transmission : image de l'acquittement renvoyé par l'esclave concerné
	0	ACK reçu de l'esclave concerné
	1	NOACK reçu de l'esclave concerné
ACKDT		Maître en réception : valeur du bit d'acquittement renvoyé par le maître vers l'esclave quand la séquence acquittement sera initiée par ACKEN=1
	0	le maître renverra ACK
	1	le maître renverra NOACK
ACKEN		Maître en réception : permet de lancer une séquence d'acquittement du maître vers l'esclave et de renvoyer le bit ACKDT
	0	pas de lancement d'une séquence d'acquittement du maître vers l'esclave
	1	lancement d'une séquence d'acquittement du maître vers l'esclave remis automatiquement à 0 par le hardware après l'acquittement
RCEN		pour placer le maître en réception
	0	réception inhibée
	1	passage en mode réception
PEN		pour démarrer une séquence de STOP CONDITION et libérer le bus
	0	pas de lancement d'une séquence de STOP CONDITION
	1	lancement d'une séquence de STOP CONDITION remis automatiquement à 0 par le hardware après STOP CONDITION
RSEN		pour démarrer une séquence de RESTART CONDITION
	0	pas de lancement d'une séquence de RESTART CONDITION
	1	lancement d'une séquence de RESTART CONDITION remis automatiquement à 0 par le hard après RESTART CONDITION
SEN		pour démarrer une séquence de START CONDITION et prendre le bus
	0	pas de lancement d'une séquence de START CONDITION
	1	lancement d'une séquence de START CONDITION remis automatiquement à 0 par le hard après START CONDITION

Figure VIII.181.

⑤ le registre SFR **SSPADD** (spécifique au mode **I²C**) n'a pas le même rôle en mode maître qu'en mode esclave. On verra plus loin sa signification en mode esclave.

En mode maître, il permet de définir la fréquence (vitesse) **F** de l'horloge **SCL** générée par le maître du bus **I²C** suivant la formule :

$$F = \frac{F_{osc}}{4 \cdot (SSPADD + 1)}$$

où **F_{osc}** représente la valeur du quartz du PIC et **SSPADD** le contenu du registre **SSPADD**.

En pratique, cette formule permet de trouver la valeur à écrire dans le registre **SSPADD** en fonction de la vitesse de communication **F** souhaitée.

Exemple :

De façon générale, les vitesses sont normalisées : **100 kHz**, **400 kHz** et **1 MHz**.

Avec un quartz à **F_{osc} = 4 MHz** et si l'on souhaite travailler à **F = 100 kHz** (ce qui veut dire que la durée **T** d'un bit sur le bus **I²C** sera de $T = \frac{1}{F} = 10 \mu s$ et donc une vitesse de 100 kbits/s), on aura :

$$SSPADD = \frac{F_{osc}}{4 \cdot F} - 1 = 9$$

VIII.8.2.3.2. FONCTIONNEMENT EN MAÎTRE

Lorsque le PIC fonctionne en mode maître, il y a six actions possibles à programmer :

- ① générer une **START CONDITION** pour la prise de contrôle du bus (après avoir vérifié éventuellement que le bus est libre),
- ② générer une **RESTART (ou REPEATED START) CONDITION** pour une inversion du sens de l'échange avec l'esclave concerné sans libérer le bus,
- ③ générer une **STOP CONDITION** pour signaler la fin de l'échange en cours et libérer le bus,
- ④ émettre une adresse ou une donnée sur le bus et gérer l'acquittement (**ACK** ou **NOACK**) renvoyé par l'esclave concerné,
- ⑤ passer en mode réception et recevoir une donnée venant de l'esclave concerné,
- ⑥ générer un acquittement (**ACK** ou **NOACK**) pour accuser réception de la donnée émanant de l'esclave concerné.

Pour améliorer la structuration, la lisibilité et l'évolutivité des programmes, on aura tout intérêt à concevoir ces différentes actions sous la forme de modules en assembleur ou de fonctions en C.

VIII.8.2.3.2.1. MAÎTRE EN TRANSMISSION

Voici, sur la figure VIII.182, un chronogramme constructeur des différents signaux intervenant pendant la phase de **transmission** du maître.

FIGURE 9-21: I²C MASTER MODE WAVEFORM (TRANSMISSION, 7 OR 10-BIT ADDRESS)

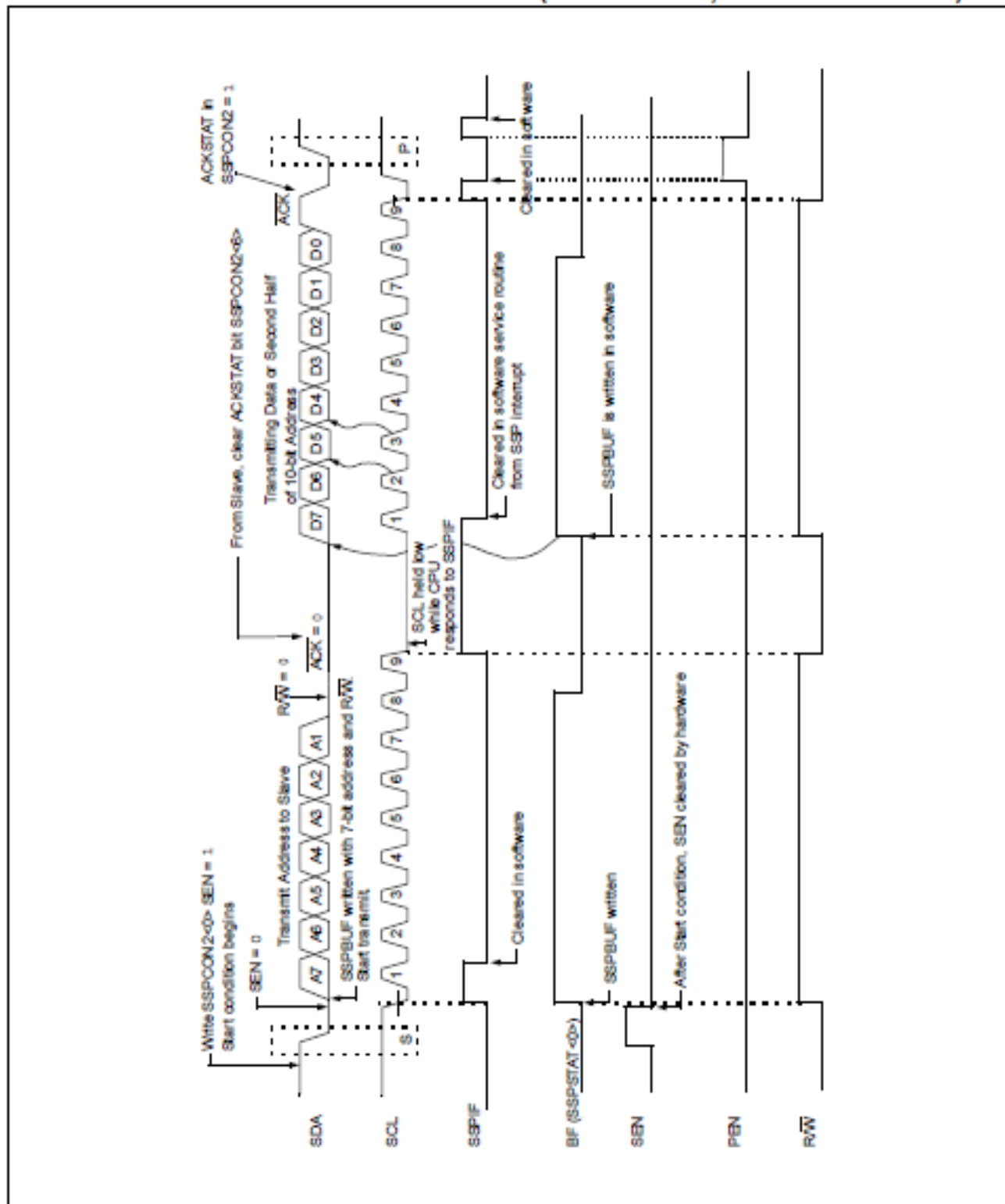


Figure VIII.182.

Ce qui peut se traduire par l'organigramme de la figure VIII.183. ci-dessous résumant les différentes étapes de la procédure à respecter pour un master en **transmission**.

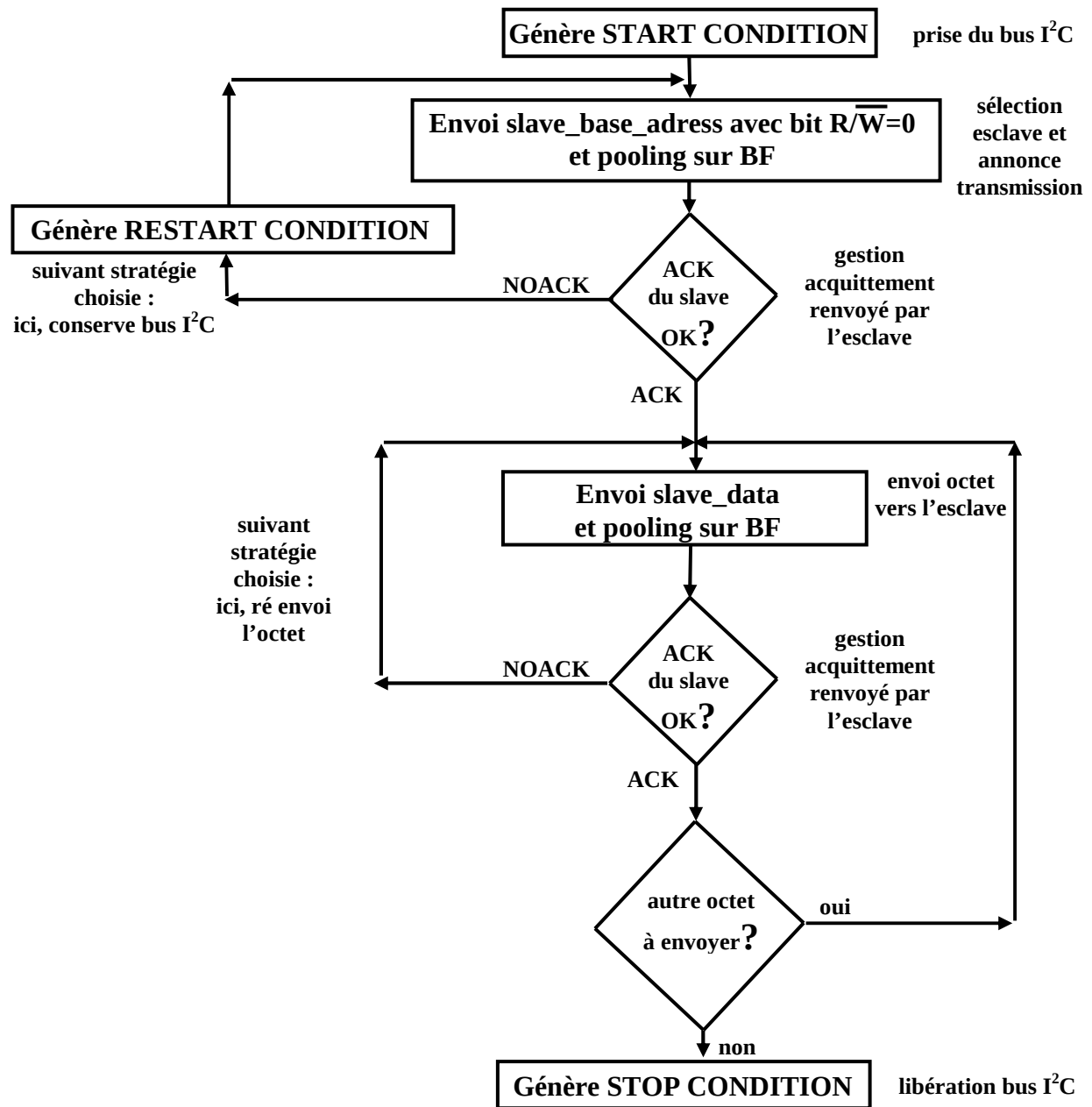


Figure VIII.183.

Mais, de façon générale, quand on veut mettre en œuvre un protocole **I²C**, il est toujours conseillé de se reporter aux séquences d'échanges qui sont données dans la documentation des circuits dialoguant sur le bus et de les respecter.

VIII.8.2.3.2.2. MAÎTRE EN RECEPTION

Voici, sur la figure VIII.184. un chronogramme constructeur des différents signaux intervenant pendant la phase de **réception** du maître.

FIGURE 9-22: I²C MASTER MODE WAVEFORM (RECEPTION, 7-BIT ADDRESS)

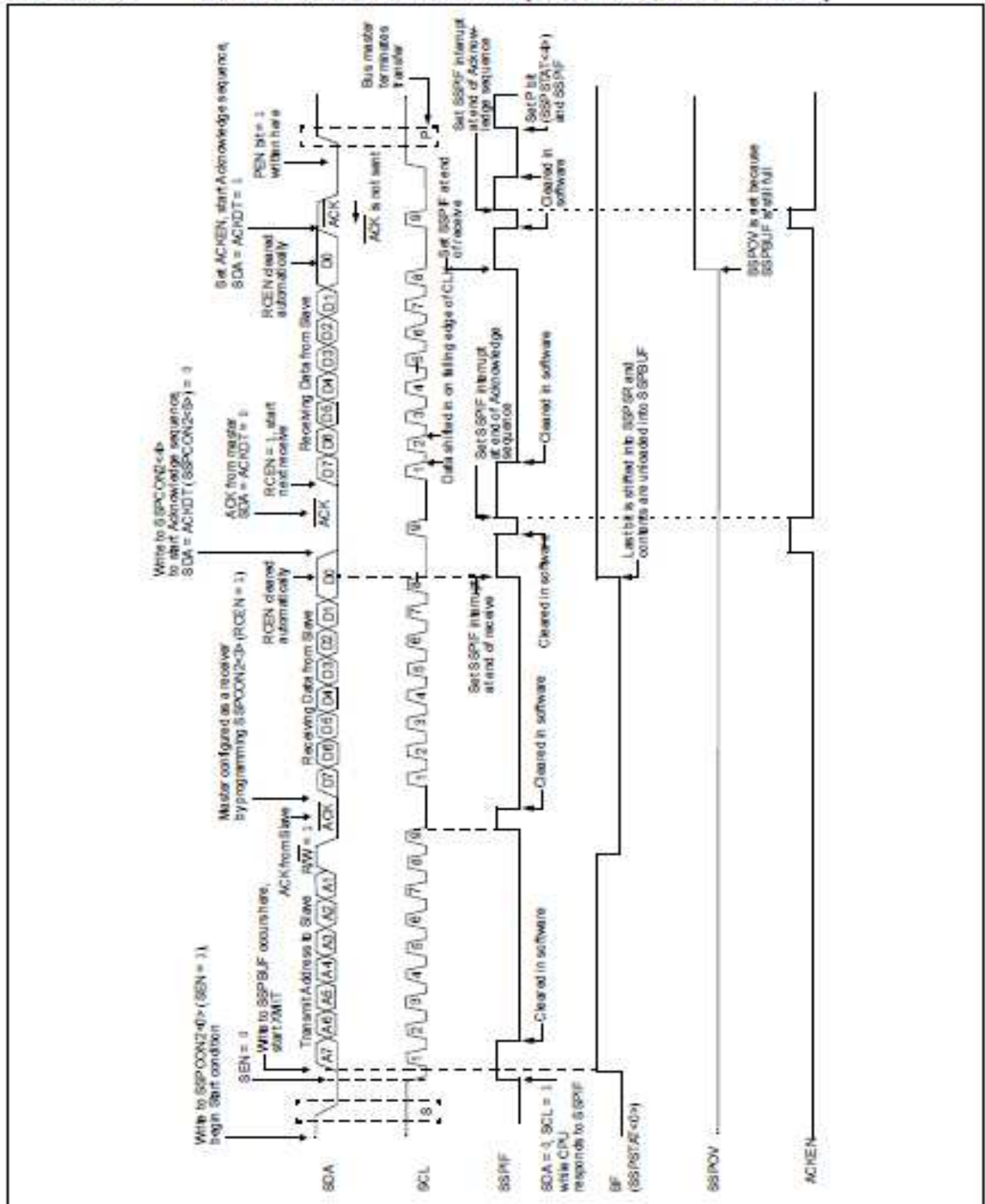


Figure VIII.184.

Ce qui peut se traduire par l'organigramme de la figure VIII.185. ci-dessous résumant les différentes étapes de la procédure à respecter pour un master en **réception**.

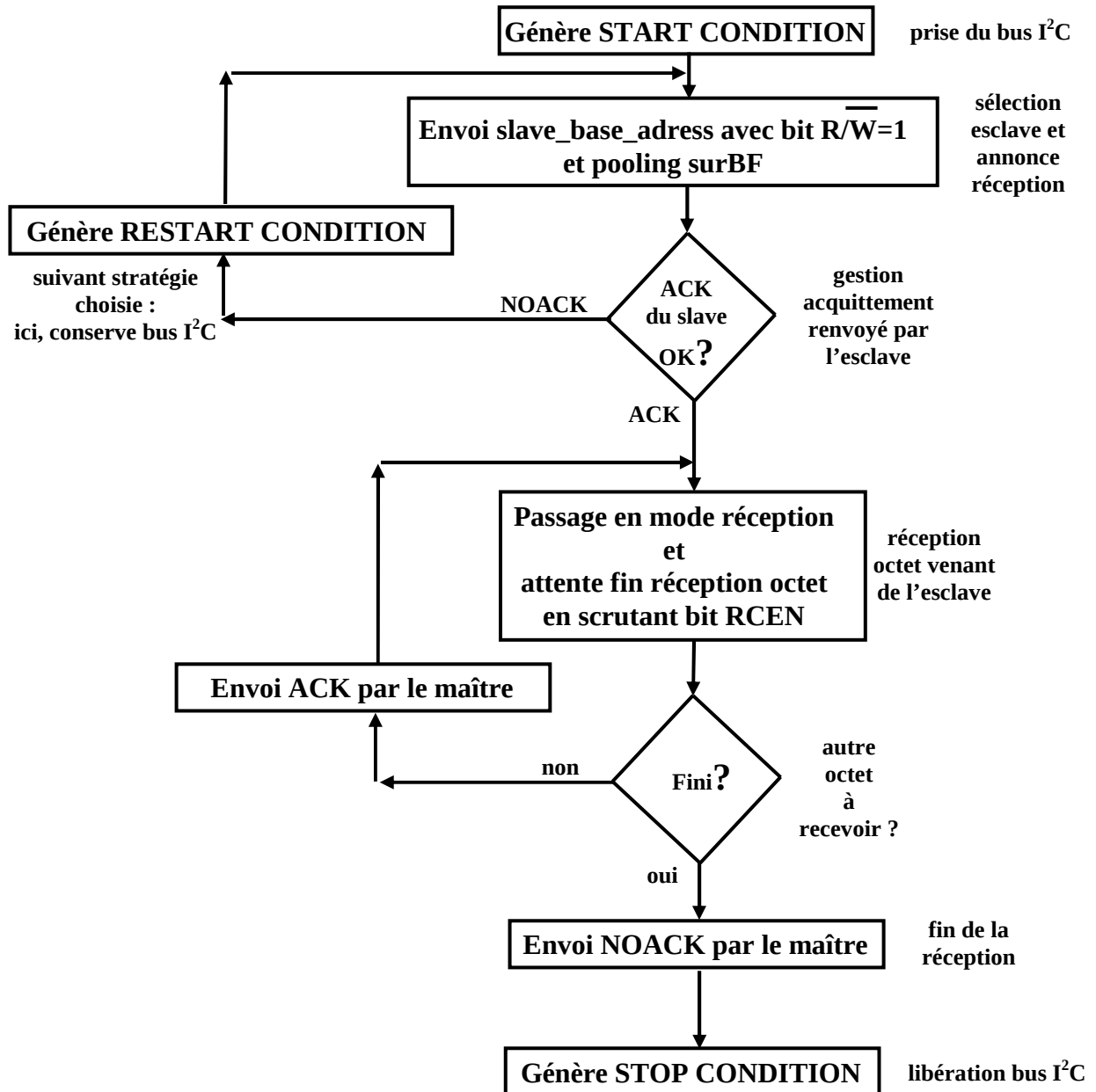


Figure VIII.185.

Là aussi, de façon générale, quand on veut mettre en œuvre un protocole I²C, il est toujours conseillé de se reporter aux séquences d'échanges qui sont données dans la documentation des circuits dialoguant sur le bus et de les respecter.

VIII.8.2.4. LE MODE ESCLAVE

VIII.8.2.4.1. REGISTRES DE CONTRÔLE ASSOCIES

Ce sont les mêmes que pour le mode maître, mais la signification de certains bits ou champs de bits est différente dans le mode esclave.

Toujours bien sûr le registre :

① **SSPBUF** qui contient comme on l'a dit précédemment :

- **l'octet à émettre** : si l'esclave est en **transmission**,

ou

- **l'octet reçu** : si l'esclave est en **réception**,

② le registre SFR **SSPCON**, aussi utilisé en mode maître et dont le format est rappelé sur la figure VIII.186. suivante :

SSPCON (situé à l'adresse 0x014) (page 82 du datasheet)							
R/W-0 R/W-0 R/W-0 R/W-0 R/W-0 R/W-0 R/W-0 R/W-0							
SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1
	SSPM0						
Bit 7 Bit 6 Bit 5 Bit 4 Bit 3 Bit 2 Bit 1 Bit 0							

Figure VIII.186.

La signification de ces bits est donnée pour le mode **I²C esclave** sur la figure VIII.187.

Bit	Valeur	Signification et commentaires
WCOL		Write COLL ision detect bit
	0	Pas de collision
	1	Ecriture dans SSPBUF pendant un transfert en cours (remis à 0 par soft par WCOL = 0)
SSPOV		O verflow detect bit : écrasement d'octet à la réception
	0	Pas d'overflow
	1	Un octet est reçu alors que le précédent n'a pas été lu dans SSPBUF (remis à 0 par soft par SSPOV = 0)
Remarque : SSPOV n'est pas utilisé en transmission		
SSPEN		Validation de la liaison I²C
	0	I²C non utilisée : les lignes concernées sont des I/O classiques
	1	I²C validée : lignes concernées (SDA et SCL) affectées à l' I²C
Remarque : la direction des lignes (input) concernées doit être faite avant de mettre SSPEN à 1		
CKP		Permet à l'esclave de générer des pauses sur SCL pour s'adapter à des vitesses de transfert trop importantes imposées par le maître (voir aussi le bit SEN du registre SSPCON2)
	0	SCL forcée à 0 : clock stretching
	1	SCL validée (« non stretchée ») : c'est une entrée pour l'esclave
SSPM3:SSPM0		Différents modes de fonctionnement de la liaison I²C
	0110	Mode esclave : adresse sur 7 bits
	0111	Mode esclave : adresse sur 10 bits
	1110	Mode esclave : adresse sur 7 bits et génération d'interruption sur détection d'une START ou STOP CONDITION
	1111	Mode esclave : adresse sur 10 bits et génération d'interruption sur détection d'une START ou STOP CONDITION
	1011	Mode esclave : esclave forcé au repos (idle mode)

Figure VIII.187.

③ le registre SFR **SSPSTAT**, aussi utilisé en mode maître et dont le format est rappelé sur la figure VIII.188.

**SSPSTAT (situé à l'adresse 0x094)
(page 81 du datasheet)**

	R/W-0	R/W-0	R-0	R-0	R-0	R-0	R-0	R-0
SSPSTAT	SMP	CKE	D/A	P	S	R/W	UA	BF
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.188.

La signification de ces bits est donnée pour le mode **I²C esclave** sur la figure VIII.189.

Bit	Valeur	Signification et commentaires
SMP		Contrôle du slew rate de l'horloge
	0	pour F = 400 kHz
	1	pour F = 100 kHz et 1 MHz
CKE		Contrôle des niveaux de tension en entrée
	0	compatibilité I²C
	1	compatibilité SMBUS
D/A		Bit d'état donnée/adresse
	0	le dernier octet envoyé ou reçu est une adresse
	1	le dernier octet envoyé ou reçu est une donnée
P		Indique la détection d'une STOP CONDITION (libération bus)
	0	pas de détection de STOP CONDITION
	1	détection d'une STOP CONDITION
Remis automatiquement à 0 quand une autre condition est détectée		
S		Indique la détection d'une START CONDITION (prise de contrôle bus)
	0	pas de détection de START CONDITION
	1	détection d'une START CONDITION
Remis automatiquement à 0 quand une autre condition est détectée		
R/W		Indique le type d'échange avec le maître : image du bit R/W reçu du maître dans le premier octet après la START CONDITION
	0	c'est une écriture
	1	c'est une lecture
UA		Update Address : indique en mode adresse sur 10 bits (deux octets d'adresse pour l'esclave) que le registre SSPADD doit être remis à jour
	0	pas de mise à jour sur SSPADD à faire
	1	mise à jour sur SSPADD à faire
Remis automatiquement à 0 après l'écriture dans le registre SSPADD		
BF		Buffer Full
En transmission :		
	0	pas de transmission en cours ou terminée : SSPBUF est vide
	1	transmission en cours : SSPBUF est plein
En réception :		
	0	réception en cours ou pas de réception : SSPBUF est vide
	1	réception terminée : SSPBUF est plein

Figure VIII.189.

- ④ le registre SFR **SSPCON2** (spécifique au mode **I²C** et cf. figure VIII.190. suivante)
SSPCON2 (situé à l'adresse 0x014)
(page 83 du datasheet)

	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SSPCON2	GCEN	ACKSTAT	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Figure VIII.190.

La signification de ces bits est donnée pour le mode **I²C maître** sur la figure VIII.191.

Bit	Valeur	Signification et commentaires
GCEN		General Call ENable : autorisation pour l'esclave de répondre (en plus qu'à son adresse ou plage d'adresses propre) à un appel général 0x0000
	0	pas de réponse à un appel général
	1	réponse à un appel général
ACKSTAT		Non utilisé en mode esclave
ACKDT		Non utilisé en mode esclave
ACKEN		Non utilisé en mode esclave
RCEN		Non utilisé en mode esclave
PEN		Non utilisé en mode esclave
RSEN		Non utilisé en mode esclave
SEN		pour autoriser l'esclave à ralentir l'horloge SCL : clock stretching (voir aussi le bit CKP du registre SSPCON)
	0	autorisation du ralentissement de SCL par l'esclave uniquement à la transmission
	1	autorisation du ralentissement de SCL par l'esclave à la transmission et à la réception

Figure VIII.191.

⑤ le registre SFR **SSPADD** (spécifique au mode **I²C**) n'a pas la même signification en mode esclave.

Si on se rappelle qu'en mode esclave l'horloge est imposée par le maître, lorsque le module **MSSP** fonctionne en esclave, il n'a pas besoin de générer une horloge.

Dans ce cas, le registre **SSPADD** sert à stocker l'adresse (ou la plage d'adresses) à laquelle l'esclave devra répondre.

Dans le cas d'un bus **I²C** fonctionnant avec **7 bits d'adresses**, le registre **SSPADD** est de taille suffisante pour contenir l'adresse de l'esclave.

Mais lorsqu'il utilise un format avec **10 bits d'adresses** (il faut alors transmettre pour le maître et reconnaître pour les esclaves deux octets d'adresses), il sera nécessaire, au moment de la réception de ces deux octets par l'esclave de mettre à jour le registre **SSPADD** (octet fort puis octet faible : voir la remarque après la page suivante...).

VIII.8.2.4.2. FONCTIONNEMENT EN ESCLAVE

On va supposer ici qu'on travaille avec **une adresse sur 7 bits**.

Un fois programmé en esclave, le module **MSSP** du PIC se met à l'écoute du bus **I²C** pour détecter l'apparition d'une **START CONDITION**.

Dès que c'est le cas, il positionne son bit **S** et récupère dans l'octet qui suit :

- l'adresse,
- et le bit **R/W** qu'il recopie dans le bit **R/W** du registre **SSPSTAT**.

Il met également à jour le bit **D/A** de ce même registre **SSPSTAT** (ici à **0** car il vient de recevoir une adresse).

Suivant les bits de mode **SSPM3 :SSPM0** du registre **SSPCON**, il peut aussi générer une interruption à la détection d'une **START CONDITION**.

Si cette adresse n'est pas la sienne (ce qu'il sait en comparant l'adresse reçue et le contenu de son registre **SSPADD**), il reste muet et se met à attendre une **STOP CONDITION** (et dès qu'il la détectera, il remettra son bit **S** à 0 et positionnera son bit **P**).

Par contre, si cette adresse est la sienne **et si les 2 conditions suivantes sont vérifiées** :

- bit **BF** = **0** : le registre **SSPBUF** est vide (son contenu a été exploité avant),
- bit **SSPOV** = **0** : il n'y a pas eu un overflow (écrasement) avant,

alors :

- l'adresse reçue dans le registre **SSPR** est **automatiquement** transférée dans **SSPBUF** et **BF** passe à **1** si le bit **R/W** est à **0** (esclave en **réception**) mais sinon **BF** reste à **0** si le bit **R/W** est à **1** (esclave en **transmission**),

- un bit d'acquittement vers le maître (**ACK**) est **automatiquement** généré par le module **MSSP** du PIC esclave,

- et le bit **SSPIF** passe à **1** (ce qui peut déclencher une interruption du module **MSSP** si le masque **SSPIE** a été mis à 1).

Le programme pilotant le PIC esclave devra alors lire l'adresse contenue dans le registre **SSBUF** :

- soit après pooling sur **BF** (ce qui fera automatiquement, au moment de la lecture de **SSPBUF**, repasser **BF** à **0**),
- soit dans la routine d'interruption en pensant à remettre par soft **SSPIF** à **0** (acquiescement de l'IT).

La suite dépend si c'est une réception ou une transmission (suivant le bit **R/W** reçu avec l'adresse) : voir les 2 sous-chapitres suivants...

Dans le cas où **BF** ou **SSPOV** sont à **1**, c'est un bit de non acquiescement **NOACK** qui sera automatiquement renvoyé vers le maître qui pourra :

- soit générer une **RESTART CONDITION** et ré émettre l'adresse de l'esclave une autre (ou plusieurs) fois,
- soit mettre fin à cette tentative d'échange avortée en générant une **STOP CONDITION**,
suivant la stratégie choisie.

Remarque :

Dans le cas où l'adresse de l'esclave est sur **10 bits**, elle est émise par le maître sur 2 octets (octet fort puis octet faible) et le programme pilotant le PIC esclave doit toujours s'assurer que le registre **SSPADD** contient au moment de la reconnaissance d'une **START CONDITION** la partie haute de l'adresse.

Pour cela, après réception de l'octet fort d'adresse (par pooling sur le bit **UA** du registre **SSPSTAT**), il faut remettre à jour **SSPADD** avec l'octet faible d'adresses (ce qui remet automatiquement le bit **UA** à **0**).

Puis après réception de l'octet faible d'adresse (par pooling sur le bit **UA** du registre **SSPSTAT**), il faut remettre à jour **SSPADD** avec l'octet fort d'adresse (ce qui remet automatiquement le bit **UA** à **0** et permet de s'assurer que c'est bien l'octet fort d'adresse qui est dans **SSPADD** lors de la prochaine **START CONDITION**).

VIII.8.2.4.2.1. ESCLAVE EN RECEPTION

Voici, sur la figure VIII.192. un chronogramme constructeur des différents signaux intervenant pendant la phase de **réception** de l'esclave **en mode 7 bits par adresse**, dans le cas bien sûr où cet esclave a reconnu son adresse !!!

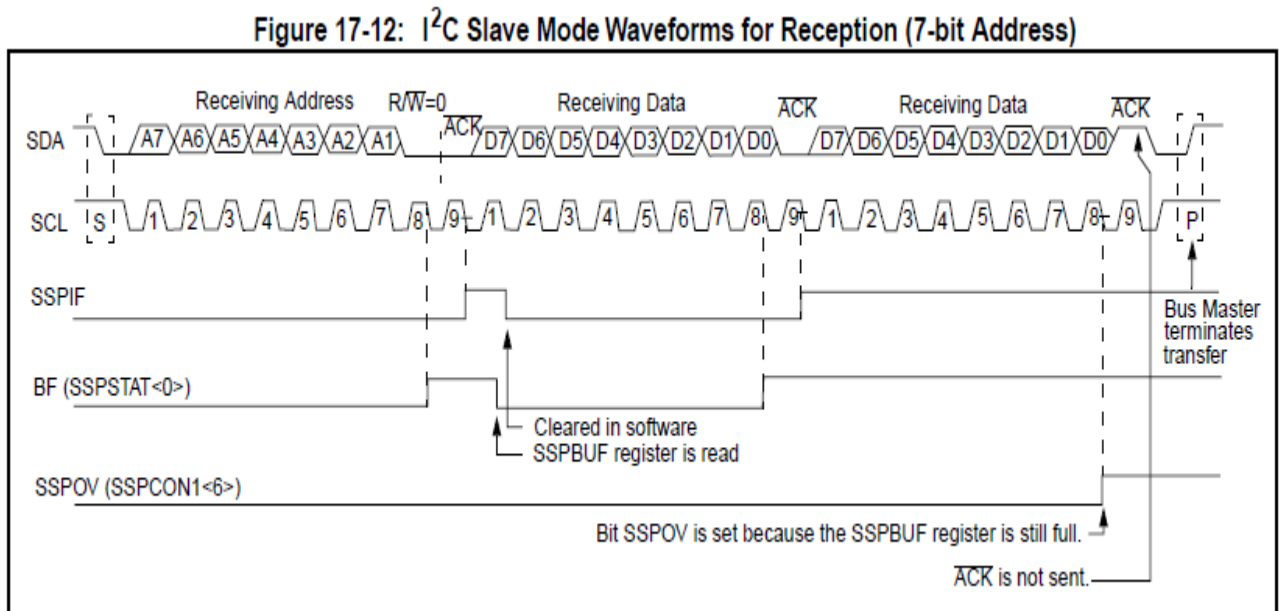


Figure VIII.192.

Après avoir vidé le registre **SSPBUF** (qui contient l'adresse valide de l'esclave) pour remettre le bit **BF** à **0**, le programme de l'esclave se met à scruter le bit **BF** (attendre qu'il passe à **1**) pour attendre que l'octet qui suit l'adresse soit complètement arrivé dans le registre **SSPBUF**.

Il suffit ensuite de lire cette donnée dans ce registre **SSPBUF** puis de la traiter et de continuer ainsi de suite...pour recevoir une autre donnée.

Par contre, il faut aussi examiner si le maître n'a pas mis fin à la communication en émettant une **STOP CONDITION**. Pour cela, il faut aussi examiner le bit **P**, ce qui, s'il passe à **1**, mettra fin à cet échange avec le maître.

Et ensuite, le programme de l'esclave devra se mettre à l'écoute de la **START CONDITION** suivante en examinant le bit **S**. Et ainsi de suite....

Un organigramme possible en mode esclave sera fourni plus loin sur la figure VIII.195. pour réaliser l'ensemble de ces opérations.

VIII.8.2.4.2.2. ESCLAVE EN TRANSMISSION

Voici, sur la figure VIII.193, un chronogramme constructeur des différents signaux intervenant pendant la phase de **transmission** de l'esclave en **mode 7 bits par adresse**, dans le cas bien sûr où cet esclave a reconnu son adresse !!!

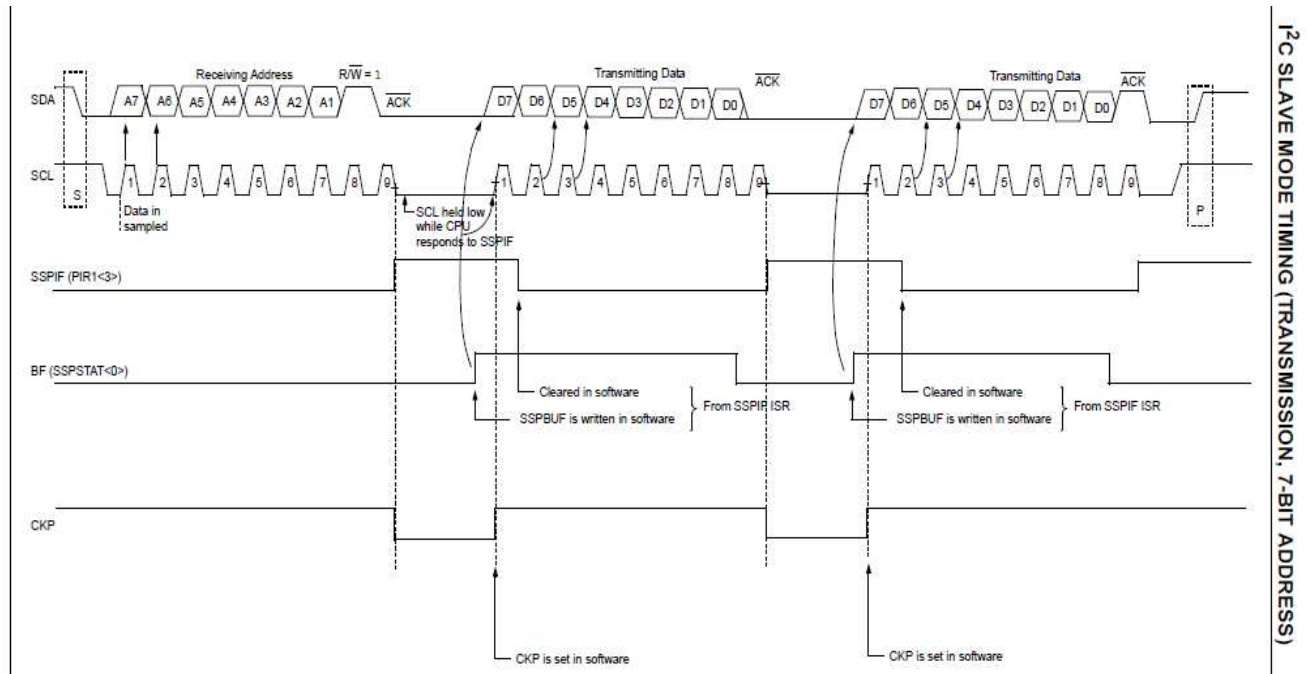


Figure VIII.193.

On voit, comme on l'a dit précédemment, que lorsque le module **MSSP** de l'esclave reconnaît son adresse et que le bit **R/W** est à **1**, le bit **BF** ne passe pas à **1** (comme dans le cas de l'esclave en réception) bien que l'adresse soit chargée dans le registre **SSPBUF**.

Ce qui est logique, car comme c'est l'esclave (ici rappelons le en transmission) qui doit émettre des données, ce bit **BF** passera à **1** quand le programme de l'esclave écrira sa donnée à émettre dans le registre **SSPBUF**. Mais le bit **SSPIF** lui passera à **1** quand même. Cette différence permettra, dans le programme pilotant l'esclave, de faire la part entre un fonctionnement esclave en réception et esclave en transmission.

De plus, notons que le module **MSSP** place le bit **CKP** à **0** (« clock stretching » pour faire une pause sur l'horloge imposée par le maître et laisser ainsi le temps à l'esclave de répondre à sa demande).

Le programme de l'esclave devra donc ensuite :

- écrire la donnée à émettre dans **SSPBUF** (ce qui fera passer le bit **BF** à **1**),
- faire passer le bit **CKP** à **1** pour « libérer » l'horloge du maître,
- remettre le bit **SSPIF** à **0**.

L'octet émis sortira alors de l'esclave au rythme de l'horloge **SCL** imposée par le maître.

Ensuite :

- soit le maître renvoie un acquittement de type **ACK** ce qui fera passer le bit **CKP** de l'esclave à **0**. Ainsi, en faisant du pooling au niveau du programme de l'esclave sur le bit **SSPIF**, dès que ce bit passe à **1**, il faudra examiner le bit **CKP** et s'il est à **0**, on saura que tout c'est bien passé pour cette émission et on pourra continuer à émettre un autre octet suivant le même procédé.

- soit le maître renvoie un non acquittement de type **NOACK** et dans ce cas le bit **CKP** de l'esclave sera à **1** au moment où le bit **SSPIF** passera à **1**. Cela signifie que le maître a mis fin à l'échange. L'esclave peut alors se remettre à l'écoute de la **START CONDITION** suivante en examinant le bit **S**. Et ainsi de suite....

Un organigramme possible en mode esclave sera fourni plus loin sur la figure VIII.195. pour réaliser l'ensemble de ces opérations.

Remarque :

Dans le cas où on souhaite travailler sous interruption, voici, sur la figure VIII.194., un tableau résumant, l'état de quelques bits du registre **SSPSTAT** et du bit **CKP** au moment de l'interruption et du traitement à effectuer en conséquence.

R/W	D/A	BF	CKP	Cause	Traitement à effectuer
0	0	1	X	Réception d'une adresse En mode réception	Lire l'adresse dans SSPBUF (vidage de SSPBUF) pour refaire passer BF à 0 : c'est une donnée qui suit ensuite.
0	1	1	X	Réception d'une donnée En mode réception	Lire la donnée dans SSPBUF (vidage de SSPBUF) pour refaire passer BF à 0 : c'est une donnée qui suit ensuite ou une STOP CONDITION .
1	0	0	0	Réception d'une adresse En mode transmission	Ecrire la donnée à émettre dans SSPBUF et libérer l'horloge par CKP = 1 .
1	1	0	0	On vient d'émettre une donnée avec ACK du maître.	Ecrire la donnée suivante à émettre dans SSPBUF et libérer l'horloge en faisant CKP = 1 .
1	1	0	1	On vient d'émettre une donnée avec NOACK du maître.	Fin de l'échange avec le maître.

Figure VIII.194.

VIII.8.2.4.2.3. ORGANIGRAMME ESCLAVE

En synthèse, et pour gérer à la fois le fonctionnement en mode **réception** et en mode **transmission** de l'esclave, voici, sous la forme de l'organigramme de la figure VIII.195., une trame possible de traitement à implanter sur un PIC **esclave** en mode pooling.

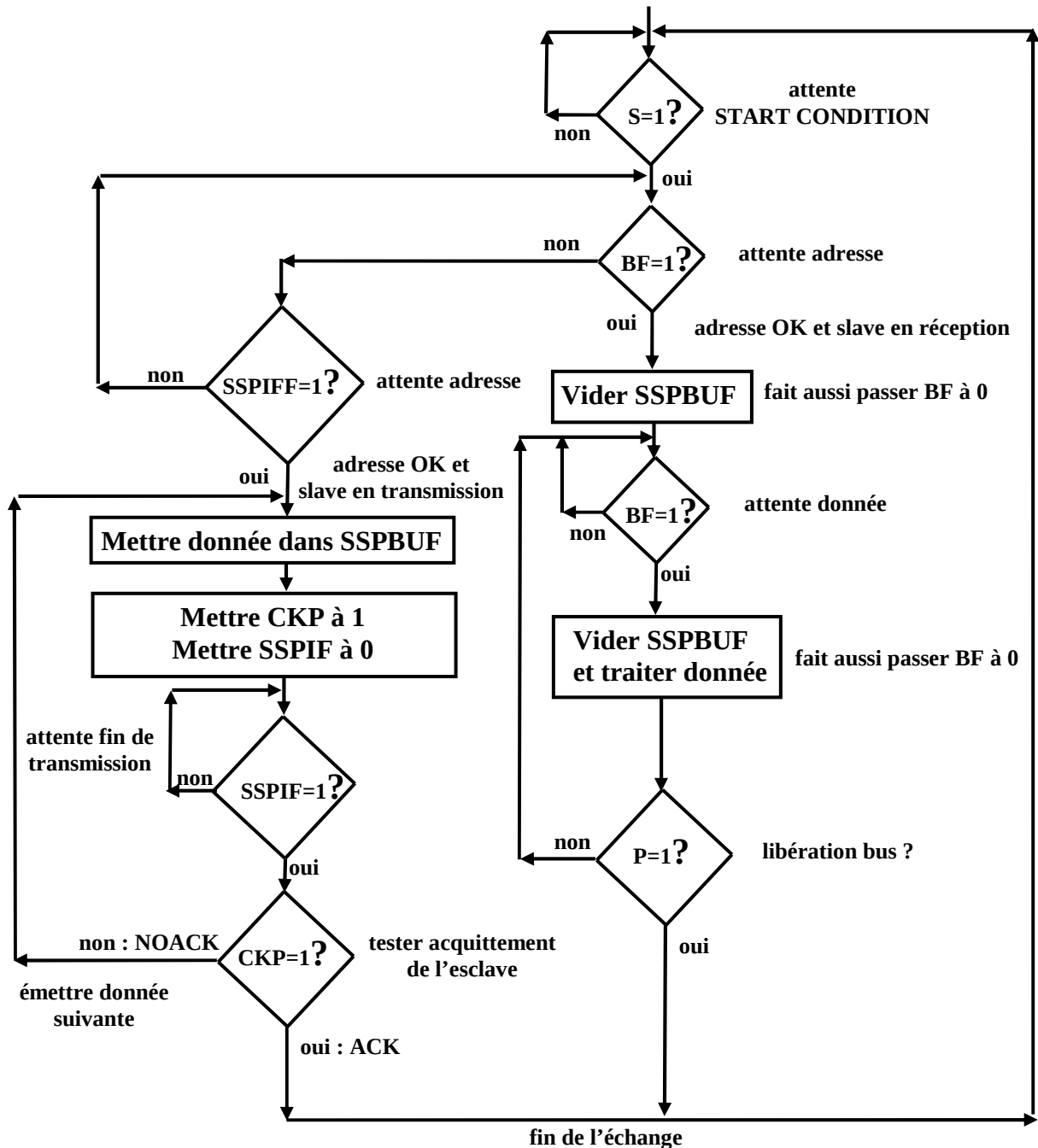


Figure VIII.195.

Voici au final sur la figure VIII.196. un résumé des registres concernés par la liaison I²C ainsi que les différents bits concernés.

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on: POR, BOR	Value on: MCLR, WDT
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	0000 000x	0000 000u
0Ch	PIR1	PSPIF ⁽¹⁾	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	0000 0000	0000 0000
8Ch	PIE1	PSPIE ⁽¹⁾	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	0000 0000	0000 0000
0Dh	PIR2	—	(2)	—	EEIF	BCLIF	—	—	CCP2IF	-r-0 0--0	-r-0 0--0
8Dh	PIE2	—	(2)	—	EEIE	BCLIE	—	—	CCP2IE	-r-0 0--0	-r-0 0--0
13h	SSPBUF	Synchronous Serial Port Receive Buffer/Transmit Register								xxxx xxxx	uuuu uuuu
14h	SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0	0000 0000	0000 0000
91h	SSPCON2	GCEN	ACKSTAT	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN	0000 0000	0000 0000
93h	SSPAD	I ² C Slave Address/Master Baud Rate Register								0000 0000	0000 0000
94h	SSPSTAT	SMP	CKE	D/A	P	S	R/W	UA	BF	0000 0000	0000 0000

Legend: x = unknown, u = unchanged, - = unimplemented, read as '0'. Shaded cells are not used by the SSP in I²C mode.

Note 1: These bits are reserved on PIC16F873/876 devices; always maintain these bits clear.

2: These bits are reserved on these devices; always maintain these bits clear.

Figure VIII.196.

VIII.8.2.5. EXEMPLE DE MISE EN ŒUVRE

Nous allons mettre en pratique les connaissances théoriques précédentes concernant le bus I^2C en pilotant une mémoire série compatible I^2C et de type **EEPROM**.

Il s'agit d'une mémoire de référence **24LC256** (taille **256 Kbits**, soit **32 Koctets**).

Pour plus de détails, il est conseillé de se reporter à sa documentation complète fournie sur les PC des labos.

Le schéma d'implantation de cette mémoire sur la carte **PICDEM 2 PLUS** est fourni sur la figure VIII.197. suivante.

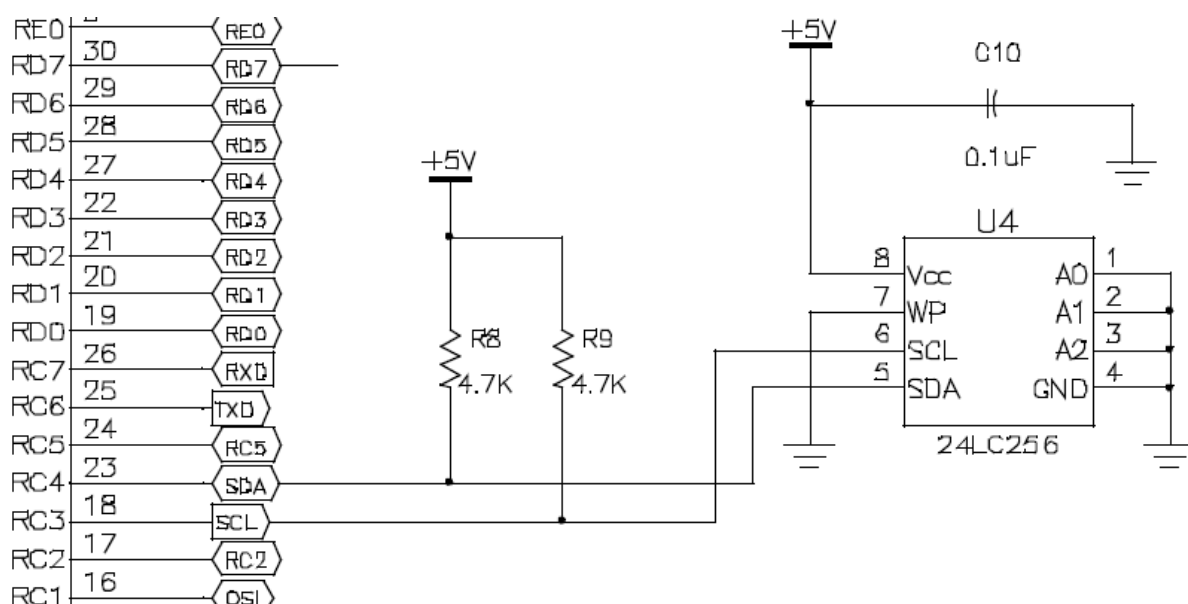


Figure VIII.197.

On y voit les 2 résistances de « pull-up » **R8** et **R9** sur les lignes **SCL** et **SDA** du bus I^2C .

L'entrée **WP** (comme **Write Protect**) est mise à **0** par la masse, ce qui implique que cette mémoire n'aura ici **aucune protection en écriture**.

Cette mémoire fonctionne tout le temps en tant qu'**esclave** : le module **MSSP** du PIC sera donc initialisé en **mode maître** et on choisit un débit de **100 kbit/s** et donc la fréquence **F** de **SCL** vaudra 100 kHz.

D'après la documentation, l'adresse de base pour l'EEPROM (adresse de sélection sur le bus I²C à laquelle cette EEPROM est sensible) est constituée du **CONTROL BYTE** apparaissant sur la figure VIII.198.

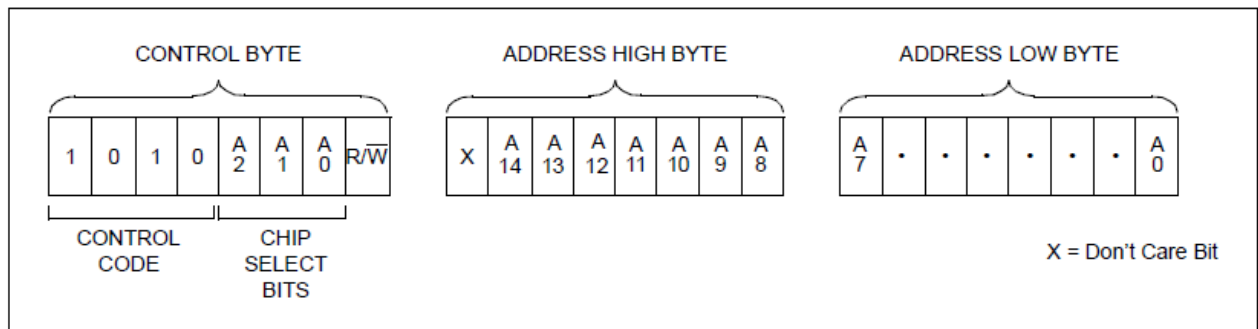


Figure VIII.198.

Les valeurs des bits **A₂**, **A₁** et **A₀** y apparaissant correspondent aux valeurs des tensions qui ont été appliquées sur les pattes d'entrées **A₂**, **A₁** et **A₀** de l'EEPROM. Comme sur la carte **PICDEM 2 PLUS**, elles sont reliées à la masse, il faudra prendre : **A₂ = 0**, **A₁ = 0** et **A₀ = 0**.

Les deux octets qui suivent **ADDRESS BYTE HIGH** et **ADDRESS BYTE LOW** permettent de sélectionner une adresse dans l'EEPROM.

❶ Ecriture dans l'EEPROM :

Il existe **2 modes d'écriture** (ou de programmation) de l'EEPROM :

- le mode « **octet par octet** »,
- le mode par « **page** ».

Nous nous intéresserons au mode « **octet par octet** », dont la séquence d'écriture est donnée sur la figure VIII.199. suivante, issue du datasheet de la mémoire **24LC256**.

FIGURE 6-1: **BYTE WRITE**

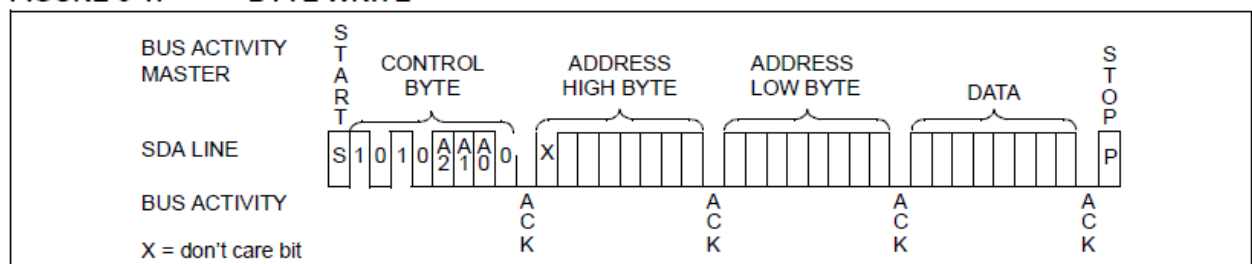


Figure VIII.199.

La valeur **DATA** y apparaissant sera écrite à l'adresse sélectionnée par les deux octets **ADDRESS BYTE HIGH** et **ADDRESS BYTE LOW**.

② Lecture de l'EEPROM :

Il existe **3 modes de lecture** de l'EEPROM :

- le mode « **adresse courante** »,
- le mode « **aléatoire** »,
- le mode « **séquentiel** ».

Nous nous intéresserons au mode « **aléatoire** », dont la séquence de lecture est donnée sur la figure VIII.200. suivante, issue également du datasheet de la mémoire **24LC256**.

FIGURE 8-2: RANDOM READ

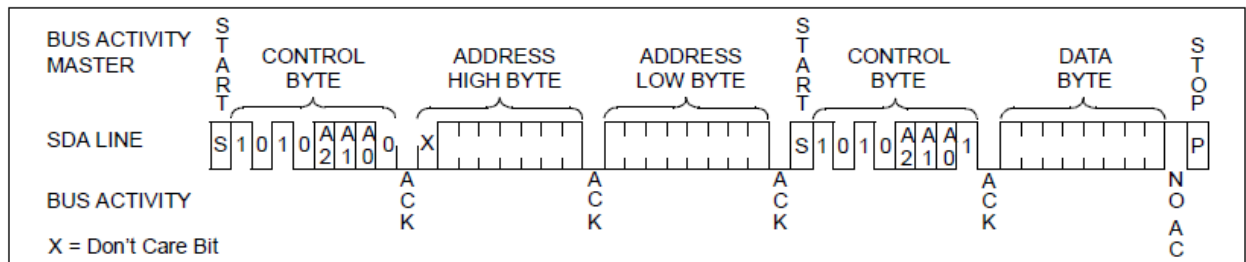


Figure VIII.200.

La valeur **DATA BYTE** y apparaissant correspond au contenu (renvoyé par l'EEPROM) de l'adresse sélectionnée par les deux octets **ADDRESS BYTE HIGH** et **ADDRESS BYTE LOW**.

Nous donnons sur les figures VIII.201. suivantes quelques fonctions permettant d'écrire ou de lire l'EEPROM.

Ces fonctions (et vous pouvez le vérifier !!!) sont conformes aux organigrammes donnés sur les figures VIII.183. et VIII.185. précédentes et également sur les séquences constructeur fournies sur les figures VIII.199. et VIII.200. On aura tout intérêt d'ailleurs à les déclarer dans un fichier header et à mettre leur code dans un fichier C, tous deux attaché à l'application qui gère la mémoire I²C...

Car on obéît toujours au même principe :

programmation structurée!!!

Certaines sont :

- de bas niveau (c'est-à-dire proche du hardware),
- et d'autres de niveau plus élevé (c'est-à-dire proche de l'utilisation) et se servent celles de bas niveau,

Car on obéît toujours au même principe :

programmation modulaire !!!

```

// quelques constantes software
#define      ACK                0
#define      NOACK              1
#define      TRANSMISSION      0      // mode transmission du PIC
#define      RECEPTION          1      // mode réception du PIC.
#define      EEPROM             0xA0   // control code 24LC256

//*****
// fonction void init_i2c(void) pour initialiser le module MSSP du PIC :
// - SCL et SDA en entrée,
// - contrôle du slew rate,
// - conformité i2c,
// - F = 100 kHz,
// - validation du module MSSP,
// - fonctionnement en mode i2c maître.
//*****
void init_i2c(void)
{
// avant toute utilisation du bus i2c SDA et SCL en input
    TRISC3 = 1;      //SCL en entrée
    TRISC4 = 1;      //SDA en entrée

// contrôle du slew rate
    SMP = 1;         // pour 100 kHz ou 1 MHz (standard)
//    SMP = 0;         // si 400 kHz

// conformité norme i2c
    CKE = 0;
//    CKE = 1;         // pour conformité SMBUS (System Manadgement Bus)
// le SMBUS est un dérivé de l'i2c mais plus lent, avec des niveaux de tension différents...
// utilisé dans les PC (alim, batterie, divers capteurs...)

// définition de la vitesse du bus i2c : fréquence F de SLK
// F = Fosc/(4*(SSPADD+1)) avec ici Fosc = 4 MHz
// si on veut travailler à F = 100 kHz
    SSPADD = 9;

// validation du module MSSP
    SSPEN = 1;

// CKP non utilisé en mode maître

// mode i2c en master et F de SCL fixé par SSPADD
    SSPM3 = 1;
    SSPM2 = 0;
    SSPM1 = 0;
    SSPM0 = 0;
}

```

Figure VIII.201.a.

```

//*****
// fonction void genere_start(void) : prise de contrôle du bus
// génère une START CONDITION en vérifiant avant que le bus i2c est libre
//*****
void genere_start(void)
{
// on vérifie avant que le bus i2c est libre
// en testant les 6 bits suivants qui doivent être à 0 pour vérifier qu'aucune activité n'est en cours
// SEN : si à 1 : indique qu'une séquence de type START CONDITION est en cours
// RSEN : si à 1 : indique qu'une séquence de type RESTART CONDITION est en cours
// PEN : si à 1 : indique qu'une séquence de type STOP CONDITION est en cours
// RCEN : si à 1 : indique qu'une réception par le master est en cours
// RW : si à 1 : indique qu'une transmission par le master est en cours
// ACKEN : si à 1 : indique qu'une séquence d'acquittement par le master est en cours

// attente que ces 6 bits soient à 0 indiquant que le bus i2c est libre
while( SEN | RSEN | PEN | RCEN | RW | ACKEN );

    SEN = 1;           // démarre une séquence de START CONDITION
                       // et remis à 0 automatiquement à 0 à la fin de cette séquence

    while(SEN);        // attente fin de séquence START
}

//*****
// fonction void genere_restart(void): inversion de l'échange en gardant le contrôle du bus i2c
// génère une RESTART CONDITION en vérifiant avant que le bus i2c est libre
//*****
void genere_restart(void)
{
// attente que ces 6 bits soient à 0 indiquant que le bus i2c est libre
while( SEN | RSEN | PEN | RCEN | RW | ACKEN );

    RSEN = 1;          // démarre une séquence de RESTART CONDITION
                       // et remis à 0 automatiquement à 0 à la fin de cette séquence

    while(RSEN);       // attente fin de séquence RESTART
}

```

Figure VIII.201.b.

```

//*****
// fonction void genere_stop(void) : libération du bus
// génère une STOP CONDITION en vérifiant avant que le bus i2c est libre
//*****
void genere_stop(void)
{
// attente que ces 6 bits soient à 0 indiquant que le bus i2c est libre
while( SEN | RSEN | PEN | RCEN | RW | ACKEN );

    PEN = 1;           // démarre une séquence de STOP CONDITION
                        // et remis à 0 automatiquement à 0 à la fin de cette séquence

    while(PEN);        // attente fin de séquence STOP
}

//*****
// fonction envoi_base_adresse(char slave_base_adresse,char mode)
// sélection esclave par son adresse de base sur 7 bits (avec LSB = 0)
// et mode d'échange
// - mode = 0 : écriture dans esclave
// - mode = 1 : lecture de l'esclave
//*****
void envoi_base_adresse(char slave_base_adresse,char mode)
{
// précaution :
// attente que ces 6 bits soient à 0 indiquant que le bus i2c est libre
while( SEN | RSEN | PEN | RCEN | RW | ACKEN );

// écriture dans SSPBUF
SSPBUF = slave_base_adresse | mode;
}

//*****
// fonction void envoi_adresse_in_slave(char adresse_in_slave)
// sélection l'adresse adresse_in_slave dans esclave (sur 8 bits)
//*****
void envoi_adresse_in_slave(char adresse_in_slave)
{
// précaution :
// attente que ces 6 bits soient à 0 indiquant que le bus i2c est libre
while( SEN | RSEN | PEN | RCEN | RW | ACKEN );

// écriture dans SSPBUF
SSPBUF = adresse_in_slave;
}

```

Figure VIII.201.c.

```

//*****
// fonction void envoi_donnee(char data)
// envoi d'un octet data dans l'esclave sélectionné : maître en transmission
//*****
void envoi_donnee(char data)
{
// précaution :
// attente que ces 6 bits soient à 0 indiquant que le bus i2c est libre
    while( SEN | RSEN | PEN | RCEN | RW | ACKEN );

// écriture dans SSPBUF
    SSPBUF = data;
}

//*****
// fonction char recoit_donnee(void)
// reçoit un octet de l'esclave sélectionné : maître en réception
// avec passage en mode de réception
//*****
char recoit_donnee(void)
{
// précaution :
// attente que ces 6 bits soient à 0 indiquant que le bus i2c est libre
    while( SEN | RSEN | PEN | RCEN | RW | ACKEN );

// passage en mode réception d'une donnée par le maître
    RCEN = 1;

// pooling sur RCEN qui passe à 0 à la réception du huitième bit
    while(RCEN); // pooling : attend la réception du huitième bit

// lecture et retour de SSPBUF qui contient l'octet reçu de l'esclave
    return SSPBUF;
}

```

Figure VIII.201.d.

```

//*****
// fonction void ack_slave_to_master(void)
// gestion de l'acquittement venant de l'esclave
// maître en transmission
// ici on attend que l'acquittement passe à 0
//*****
void ack_slave_to_master(void)
{
// précaution :
// attente que ces 6 bits soient à 0 indiquant que le bus i2c est libre
    while( SEN | RSEN | PEN | RCEN | RW | ACKEN );

// attente de la fin de transmission des 8 bits
    while(BF);

// attente du ACK de l'esclave
    while(ACKSTAT);
}

//*****
// fonction void ack_master_to_slave(char acquittement)
// gestion de l'acquittement venant du maître
// maître en réception
// - soit ACK
// - soit NOACK
//*****
void ack_master_to_slave(char acquittement)
{
// précaution :
// attente que ces 6 bits soient à 0 indiquant que le bus i2c est libre
    while( SEN | RSEN | PEN | RCEN | RW | ACKEN );

    ACKDT = acquittement;

// démarre une séquence d'acquittement vers esclave
    ACKEN = 1;
// ACKEN remis à 0 automatiquement à la fin de cette séquence
    while(ACKEN);
}

```

Figure VIII.201.e.


```

//*****
// fonction void byte_write_eepromi2c(short int adresse,char data)
// ecrit l'octet data à l'adresse adresse dans l'eprom i2c
// maître en transmission
// fonction conforme à la figure VIII.199. précédente
//*****

void byte_write_eepromi2c(short int adresse,char data)
{
    char adresse_high, adresse_low;
    int t;                                // pour la temporisation : durée d'écriture

    adresse_low = adresse;
    adresse_high = (adresse >> 8);

    genere_start();

    envoi_base_adresse(EEPROM,TRANSMISSION);
    ack_slave_to_master();

    envoi_adresse_in_slave(adresse_high);
    ack_slave_to_master();

    envoi_adresse_in_slave(adresse_low);
    ack_slave_to_master();

    envoi_donnee(data);
    ack_slave_to_master();

    genere_stop();

    // attente supérieure à 5 ms que le cycle d'écriture soit fini

    for(t=1;t<400;t++);
}

```

Figure VIII.201.f.

```

//*****
// fonction char random_read_eepromi2c(short int adresse)
// retourne le contenu de l'adresse adresse de l'eeprom i2c
// maître en réception
// fonction conforme à la figure VIII.200. précédente
//*****
char random_read_eepromi2c(short int adresse)
{
    char adresse_high, adresse_low, data;

    adresse_low = adresse;
    adresse_high = (adresse >> 8);

    genere_start();
    envoi_base_adresse(EEPROM,TRANSMISSION);
    ack_slave_to_master();

    envoi_adresse_in_slave(adresse_high);
    ack_slave_to_master();

    envoi_adresse_in_slave(adresse_low);
    ack_slave_to_master();

// attention : il faut une restart condition avec genere_start() ne marche pas!!!
    genere_restart();

    envoi_base_adresse(EEPROM,RECEPTION);
    ack_slave_to_master();

    data = recoit_donnee();

    ack_master_to_slave(NOACK);

    genere_stop();

    return data;
}

char i;

void main(void)
{
    init_i2c();                // configuration i2c

    byte_write_eepromi2c(0x1210,0xAA);    // écrit 0xAA dans la case 0x1210

    i = random_read_eepromi2c(0x1210);    // lecture de la case 0x1210
}

```

Figure VIII.201.g

SOMMAIRE

I. GENERALITES ET NOTIONS DE BASE	005
I.1. MICROPROCESSEUR ET MICROCONTROLEUR	005
I.2. RAPPELS ET INTRODUCTION DE QUELQUES NOTIONS DE BASE	009
I.2.1. REGISRE ET CODAGE HEXADECIMAL	009
I.2.2. MEMOIRE ET DIFFERENTS TYPES DE MEMOIRE VIVE	016
I.2.3. SYNOPTIQUE SIMPLIFIE	029
I.2.4. DECODAGE D'ADRESSES ET MAPPING MEMOIRE	031
I.2.5. EXERCICE : ANALYSE D'UN CAS CONCRET	037
I.2.6. VERS UNE ARCHITECTURE TYPE HARVARD	045
I.2.8. SEQUENCEMENT D'UNE INSTRUCTION ET NOTION DE PIPE	048
I.2.9. STRUCTURE CISC OU RISC	052
I.2.10. CODAGE ASCII	054
II. ARCHITECTURE INTERNE ET « EXTERNE » DU PIC 16F877	057
II.1. LA FAMILLE PIC CHEZ MICROCHIP	057
II.2. LE BROCHAGE EXTERNE DU PIC 16F877	058
II.3. ARCHITECTURE INTERNE DU PIC 16F877	060
III. CPU, MEMOIRES, PILE ET CONFIGURATION	065
III.1. ACCUMULATEUR W ET UNITE ARITHMETIQUE ET LOGIQUE	065
III.2. LA MEMOIRE INTERNE RAM : SFR ET GPR	068
III.2.1. LE REGISTRE STATUS	073
III.2.1.1. BITS RP1-RP0 : ADRESSAGE DIRECT	073
III.2.1.2. LES INDICATEURS D'ETAT	074
III.2.1.3. BIT IRP : ADRESSAGE INDIRECT	080
III.2.2. LE REGISTRE PC	084
III.3. LA MEMOIRE INTERNE ROM	085
III.4. LA PILE	093
III.5. LA CONFIGURATION	097
IV. LE JEU D'INSTRUCTIONS	101
IV.1. INTRODUCTION	101
IV.2. LISTE COMPLETE DES INSTRUCTIONS	103
IV.3. DETAILS ET EXEMPLES D'INSTRUCTIONS	108
IV.3.1. INSTRUCTIONS DE TRANSFERT	110
FAMILLE MOV.....	110
IV.3.2. INSTRUCTIONS DE TRAITEMENT ARITHMETIQUE	113
FAMILLES ADD..., SUB..., INC..., DEC... et CLR.....	113
IV.3.3. INSTRUCTIONS DE TRAITEMENT LOGIQUE	121
FAMILLES AND..., IOR..., XOR..., COMF, RxF..., SWAPF et BxF.....	121
IV.3.4. INSTRUCTIONS DE RUPTURE DE SEQUENCE	134
IV.3.4.1. INSTRUCTIONS DE RUPTURE INCONDITIONNELLE	134
FAMILLES GOTO, CALL et RET.....	134
IV.3.4.2. INSTRUCTIONS DE RUPTURE CONDITIONNELLE	140
FAMILLES DECFSZ, INCFSZ et BITFSx	140
IV.3.5. AUTRES INSTRUCTIONS	147
NOP, SLEEP et CLRWDt	147
IV.4. DIRECTIVES D'ASSEMBLAGE	151
IV.4.1. DIRECTIVE LIST	152
IV.4.2. DIRECTIVE #INCLUDE	152
IV.4.3. DIRECTIVE _CONFIG	153
IV.4.4. DIRECTIVE EQU	153
IV.4.5. DIRECTIVE #DEFINE	155
IV.4.6. DIRECTIVE ORG	156
IV.4.7. DIRECTIVE END	157
IV.4.8. DIRECTIVE CBLOCK/ENDC	158
IV.4.9. DIRECTIVES DB et DW	159
IV.4.10. DIRECTIVE DE	160
IV.4.11. AUTRES DIRECTIVES INTERESSANTES	161
IV.4.12. LE FICHIER VIERGE_ASM.ASM	162

V. PETITS EXERCICES D'INITIATION CORRIGES.....	165
V.1. « SIGNATURE » DES BANKS DE LA RAM	165
V.2. « ZONE COMMUNE » DES QUATRE BANKS DE LA RAM	166
V.3. « PETITE ADDITION » ENTRE DEUX VARIABLES	167
V.4. « PETITE SOUSTRACTION » ENTRE DEUX VARIABLES	168
V.5. « COMPARAISON » ENTRE DEUX VARIABLES	169
V.6. « SORTIE DE BOUCLE » PAR DEPASSEMENT DE SEUIL	172
V.7. « TEST D'EGALITE » PAR OPERATEUR XOR	174
V.8. « CONCEPTION » D'UN MODULE	175
V.9. LES « DIFFERENTS » MOV	181
V.10. « POINTEUR DU C » ET « ADRESSAGE INDIRECT »	184
VI. EXEMPLES EN ASSEMBLEUR ET EN C	189
VI.1. LES TEMPORISATIONS LOGICIELLES	190
VI.1.1. CAS D'UNE SIMPLE BOUCLE	191
VI.1.2. PAR IMBRICATION DE BOUCLES	193
VI.1.3. PAR PROGRAMMATION MODULAIRE	194
VI.1.4. PAR LE LANGAGE C	197
VI.2. ACCES A UNE ZONE DE MEMOIRE PAR ADRESSAGE INDIRECT	200
VI.2.1. REMPLISSAGE D'UNE ZONE DE MEMOIRE EN ASSEMBLEUR	200
VI.2.2. REMPLISSAGE D'UNE ZONE DE MEMOIRE EN C	206
VI.3. POSITIONNEMENT ET TEST DE BITS INDIVIDUELS	207
VI.3.1. POSITIONNEMENT DE BITS INDIVIDUELS EN C	208
VI.3.2. TEST DE BITS INDIVIDUELS EN C	210
VII. METHODOLOGIE DE TRAVAIL EN TP	213
VII.1. SIMULATION	213
VII.2. DEBUGAGE SUR LE COMPOSANT	214
VII.3. PROGRAMMATION	215
VIII. LES PERIPHERIQUES ET INTERFACES DU PIC 16F877	217
VIII.1. LES ENTREES/SORTIES PARALLELES	217
VIII.1.1. GENERALITES ET PRINCIPES	217
VIII.1.1.1. FONCTIONNEMENT EN SORTIE	222
VIII.1.1.2. FONCTIONNEMENT EN ENTREE	225
VIII.1.2. PARTICULARITES DES PORTS	228
VIII.1.2.1. LE PORTA	228
VIII.1.2.2. LE PORTB	231
VIII.1.2.3. LE PORTC	232
VIII.1.2.4. LE PORTD	233
VIII.1.2.5. LE PORTE	234
VIII.2. PREMIERE APPROCHE DES INTERRUPTIONS MATERIELLES.....	238
VIII.2.1. INTRODUCTION GENERALE	238
VIII.2.2. DEFINITION ET EXEMPLES	239
VIII.2.3. MECANISME D'UNE INTERRUPTION	240
VIII.2.4. SPECIFICITES D'UNE ROUTINE INTERRUPTION	241
VIII.2.5. SYNTHESE : CANEVAS POSSIBLE EN C POUR LA ROUTNE D'IT ...	250
VIII.3. LES EEPROM INTERNES	253
VIII.3.1. ACCES A L'EEPROM DE DONNEES	256
VIII.3.1.1. ACCES EN LECTURE	256
VIII.3.1.2. ACCES EN ECRITURE	258
VIII.3.2. ACCES A L'EEPROM DE PROGRAMME	260
VIII.3.2.1. ACCES EN LECTURE	261
VIII.3.2.2. ACCES EN ECRITURE	263
VIII.4. LE CONVERTISSEUR ANALOGIQUE/NUMERIQUE	268
VIII.4.1. GENERALITES ET PRINCIPES	268
VIII.4.1.1. INTERFACE ANALOGIQUE/NUMERIQUE	268
VIII.4.1.2. PRINCIPE ET DUREE DE CONVERSION	269
VIII.4.1.3. LINEARITE ET PRECISION	273
VIII.4.1.4. TEMPS D'ACQUISITION	274

VIII.4.2. MISE EN OEUVRE DU CAN DANS LE PIC	277
VIII.4.2.1. SYNOPTIQUE GENERAL D'IMPLANTATION	277
VIII.4.2.2. REGISTRES ASSOCIES	278
VIII.4.2.3. SEQUENCEMENT D'UNE CONVERSION	284
VIII.4.2.4. EXEMPLE DE MISE EN ŒUVRE	285
VIII.5. LES TIMERS INTERNES	288
VIII.5.1. GENERALITES SUR LA FONCTION TIMER	288
VIII.5.2. LES TIMERS INTERNES DU PIC	293
VIII.5.2.1. LE TIMER0	293
VIII.5.2.1.1. PRINCIPALES CARACTERISTIQUES	293
VIII.5.2.1.2. SYNOPTIQUE	294
VIII.5.2.1.3. REGISTRE DE CONTRÔLE ASSOCIE	296
VIII.5.2.1.4. EXEMPLES DE MISE EN ŒUVRE	297
VIII.5.2.2. LE TIMER1	301
VIII.5.2.2.1. PRINCIPALES CARACTERISTIQUES	301
VIII.5.2.2.2. SYNOPTIQUE	301
VIII.5.2.2.3. REGISTRE DE CONTRÔLE ASSOCIE	303
VIII.5.2.2.4. EXEMPLE DE MISE EN ŒUVRE	304
VIII.5.2.3. LE TIMER2	307
VIII.5.2.3.1. PRINCIPALES CARACTERISTIQUES	307
VIII.5.2.3.2. SYNOPTIQUE	307
VIII.5.2.3.3. REGISTRE DE CONTRÔLE ASSOCIE	310
VIII.5.2.3.4. EXEMPLES DE MISE EN ŒUVRE	311
VIII.6. LES MODULES CCP1 ET CCP2	316
VIII.6.1. LA FONCTION (INPUT) CAPTURE	317
VIII.6.1.1. GENERALITES SUR LA FONCTION DE CAPTURE	317
VIII.6.1.2. SYNOPTIQUE	319
VIII.6.1.3. REGISTRES DE CONTRÔLE ASSOCIES	321
VIII.6.1.4. EXEMPLE DE MISE EN ŒUVRE	322
VIII.6.2. LA FONCTION (OUTPUT) COMPARE	326
VIII.6.2.1. GENERALITES SUR LA FONCTION COMPARAISON	326
VIII.6.2.2. SYNOPTIQUE	328
VIII.6.2.3. REGISTRES DE CONTRÔLE ASSOCIES	331
VIII.6.2.4. EXEMPLE DE MISE EN ŒUVRE	332
VIII.6.3. LA FONCTION PULSE WIDH MODULATION	335
VIII.6.3.1. GENERALITES SUR LA FONCTION PWM	335
VIII.6.3.2. SYNOPTIQUE	337
VIII.6.3.3. REGISTRES DE CONTRÔLE ASSOCIES	340
VIII.6.3.4. EXEMPLES DE MISE EN ŒUVRE	341
VIII.7. LA LIAISON SERIE ASYNCHRONE/SYNCHRONE	348
VIII.7.1. GENERALITES SUR LES LIAISONS	348
VIII.7.2. LE MODE ASYNCHRONE DE L'U.S.A.R.T.	350
VIII.7.2.1. PARAMETRES DE LA LIAISON SERIE ASYNCHRONE	351
VIII.7.2.2. UN EXEMPLE : LA NORME V24 (RS232C)	358
VIII.7.2.3. SYNOPTIQUES ET REGISTRES ASSOCIES A L'UART.....	363
VIII.7.2.3.1. SYNOPTIQUE DE L'EMETTEUR	363
VIII.7.2.3.2. REGISTRE ASSOCIE A L'EMETTEUR	366
VIII.7.2.3.3. SYNOPTIQUE DU RECEPTEUR	368
VIII.7.2.3.4. REGISTRE ASSOCIE AU RECEPTEUR	371
VIII.7.2.4. EXEMPLES DE MISE EN ŒUVRE	373
VIII.7.2.5. MODE DETECTION D'ADRESSES	377
VIII.7.3. LE MODE SYNCRONE DE L'U.S.A.R.T.	381
VIII.7.3.1. GENERALITES SUR LES LIAISONS SYNCHRONES	381
VIII.7.3.2. SYNOPTIQUES ET REGISTRES ASSOCIES A L'USRT	382
VIII.7.3.3. METHODOLOGIE ET PROGRAMMATION	384
VIII.7.3.3.1. LE MODE MAÎTRE	384
VIII.7.3.3.2. LE MODE ESCLAVE	386

VIII.8. LE MODULE MSSP	390
VIII.8.1. LE MODE SPI	391
VIII.8.1.1. GENERALITES SUR LE BUS SPI	391
VIII.8.1.2. SYNOPTIQUE	393
VIII.8.1.3. REGISTRES DE CONTRÔLE ASSOCIES	399
VIII.8.1.4. EXEMPLE DE MISE EN ŒUVRE	404
VIII.8.2. LE MODE I ² C	408
VIII.8.2.1. GENERALITES SUR LE BUS I ² C	408
VIII.8.2.1.1. HARDWARE	409
VIII.8.2.1.2. PRISE DE CONTRÔLE DU BUS	412
VIII.8.2.1.3. TRANSMISSION DES BITS	412
VIII.8.2.1.4. ACQUITTEMENT	413
VIII.8.2.1.5. TRANSMISSION D'UNE ADRESSE	414
VIII.8.2.1.6. ECRITURE D'UNE DONNEE	416
VIII.8.2.1.7. LECTURE D'UNE DONNEE	417
VIII.8.2.1.8. LIBERATION DU BUS	418
VIII.8.2.1.9. CONFLITS SUR LE BUS I ² C	419
VIII.8.2.1.10. QUELQUES PARTICULARITES	421
VIII.8.2.1.11. QUELQUES EVOLUTIONS DU BUS I ² C	422
VIII.8.2.2. SYNOPTIQUE SIMPLIFIE, BROCHES ET REGISTRES	424
VIII.8.2.3. LE MODE MAÎTRE	428
VIII.8.2.3.1. REGISTRES DE CONTRÔLE ASSOCIES	428
VIII.8.2.3.2. FONCTIONNEMENT EN MAÎTRE	432
VIII.8.2.3.2.1. MAÎTRE EN TRANSMISSION ...	433
VIII.8.2.3.2.2. MAÎTRE EN RECEPTION	435
VIII.8.2.4. LE MODE ESCLAVE	437
VIII.8.2.4.1. REGISTRES DE CONTRÔLE ASSOCIES	437
VIII.8.2.4.2. FONCTIONNEMENT EN ESCLAVE	442
VIII.8.2.4.2.1. ESCLAVE EN RECEPTION	444
VIII.8.2.4.2.2. ESCLAVE EN TRANSMISSION .	445
VIII.8.2.4.2.3. ORGANIGRAMME ESCLAVE ...	447
VIII.8.2.5. EXEMPLE DE MISE EN ŒUVRE	449